Hashtag Treinamentos

CURSO COMPLETO

SQL

IMPRESSIONADOR

ORACLE®

SUMÁRIO

MÓDULO 1	Introdução	004
MÓDULO 2	Instalação	031
MÓDULO 3	Administração de Usuários	054
MÓDULO 4	Introdução ao SQL	088
MÓDULO 5	Ordenando e Filtrando Dados	151
MÓDULO 6	Funções Single Row (Pt. 1)	233
MÓDULO 7	Funções Single Row (Pt. 2)	288
MÓDULO 8	Criando Agrupamentos	322
MÓDULO 9	Joins e Operadores SET	372
MÓDULO 10	CRUD	434

Próxima Página

SUMÁRIO

MÓDULO 11	Constraints, Sequences e Índices	487
MÓDULO 12	Views	564
MÓDULO 13	Subqueries	600
MÓDULO 14	Blocos Anônimos	659
MÓDULO 15	Estruturas de Controle	709
MÓDULO 16	Functions	770
MÓDULO 17	Procedures	812
MÓDULO 18	Triggers	895
MÓDULO 19	Tratamento de Erros (Exceções)	940
MÓDULO 20	Cursores	997
MÓDULO 21	Segurança	1058

MÓDULO 1

INTRODUÇÃO

INTRODUÇÃO
INTRODUÇÃO

O QUE VEREMOS NESTE MÓDULO

MÓDULO 1 INTRODUÇÃO

01	A HISTÓRIA DOS BANCOS DE DADOS	006
02	BANCO DE DADOS RELACIONAL	008
03	CONCEITOS BÁSICOS DE BANCOS DE DADOS RELACIONAIS	011
04	A HISTÓRIA DA LINGUAGEM SQL	012
05	SQL ORACLE VS. SQL ANSI	013
06	MER, SGBD, ACID, CRUD ... QUE TAL UM DICIONÁRIO?	014
07	RESUMO DO MÓDULO	030

A história dos Bancos de Dados

Todos sabemos que os dados/informações foram armazenados durante décadas dentro de bibliotecas, salas de registros, pastas físicas, fichas em papel, enfim, armazenados de maneira impressa para **consultas** posteriores.

Com décadas de informações geradas, é claro que esse volume todo trazia uma série de desafios em sua manipulação e organização.

A partir dos anos 60, os computadores passam a ser um recurso importante dentro das empresas, e junto com eles, veio a possibilidade de armazenar informações em **formato digital** (deixando de ser exclusivamente impresso).

A história dos Bancos de Dados

Após alguma evolução nos modelos de bancos de dados, no início dos anos 70, Edgar Frank propõe o **modelo de dados relacional**, um marco na forma de pensar em bancos de dados.

Ainda na década de 70, foram desenvolvidos dois protótipos de sistema relacional.

1. **Ingres**, desenvolvido pela UCB.
2. **System R**, desenvolvido pela IBM.

Foi neste período que também foi criado o termo **Sistema de Gerenciamento de Banco de Dados Relacional** (SGBDR, ou RDBMS em inglês).

Mas o que seria um sistema relacional?

Banco de Dados Relacional

Um BANCO DE DADOS RELACIONAL é um conjunto de tabelas que armazenam informações sobre algum negócio. Essas tabelas se relacionam por meio de colunas em comum, conhecidas como chaves.

Observe o Banco de Dados abaixo, composto por duas tabelas: **EMPLOYEES** (funcionários) e **DEPARTMENTS** (departamentos). Essas duas tabelas possuem uma coluna em comum, a coluna **DEPARTMENT_ID**.

FUNCIONÁRIOS

EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_ID
100	Steven	90
101	Neena	90
102	Lex	90
103	Alexander	60
104	Bruce	60
105	David	60
106	Valli	60
107	Diana	60
108	Nancy	100
109	Daniel	100
110	John	100
111	Ismael	100
112	Jose Manuel	100
113	Luis	100
114	Den	30
115	Alexander	30

DEPARTAMENTOS

DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
30	Purchasing
40	Human Resources
50	Shipping
60	IT
70	Public Relations
80	Sales
90	Executive

Índice Geral

Índice deste Módulo

Banco de Dados Relacional

Por meio da coluna **DEPARTMENT_ID** é possível estabelecer uma relação entre as tabelas. Repare que o Steven, de EMPLOYEE_ID igual a 100 da tabela **FUNCIONÁRIOS** é do **departamento 90**. Mas qual é o departamento 90?

O departamento 90 é o Executive, que podemos verificar na tabela **DEPARTAMENTOS**.

FUNCIONÁRIOS

EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_ID
100	Steven	90

DEPARTAMENTOS

DEPARTMENT_ID	DEPARTMENT_NAME
90	Executive

Banco de Dados Relacional

Relacionar as tabelas de um banco de dados relacional é a sua principal característica. **Por meio de uma coluna em comum, conhecida como chave, é possível conectar as informações de duas ou mais tabelas.**

Você deve lembrar que essas chaves podem ser identificadas como **CHAVE PRIMÁRIA** (ou **PK**, do inglês **primary key**) e **CHAVE ESTRANGEIRA** (ou **FK**, do inglês **foreign key**). Se você não lembra, pode ficar tranquilo, pois mais a frente vamos voltar neste assunto.

FUNCIONÁRIOS

EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_ID
100	Steven	90
101	Neena	90
102	Lex	90
103	Alexander	60
104	Bruce	60
105	David	60
106	Valli	60
107	Diana	60
108	Nancy	100
109	Daniel	100
110	John	100
111	Ismael	100
112	Jose Manuel	100
113	Luis	100
114	Den	30
115	Alexander	30

Chave Estrangeira

DEPARTAMENTOS

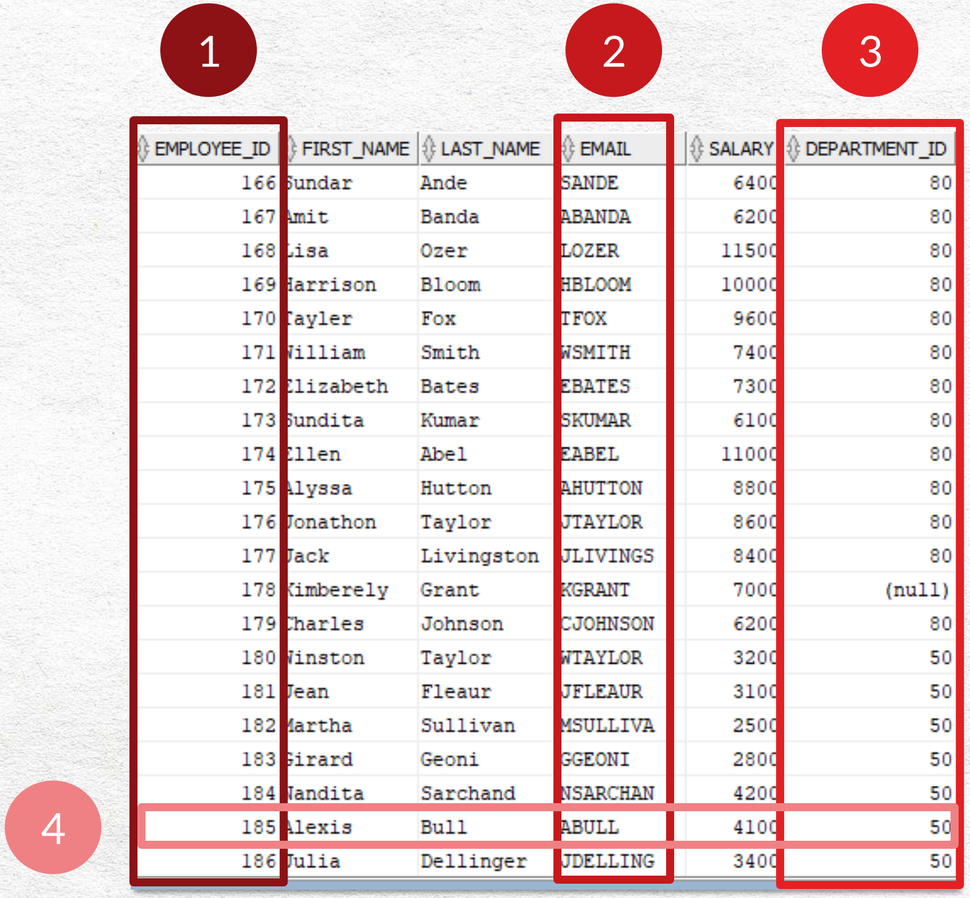
DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
30	Purchasing
40	Human Resources
50	Shipping
60	IT
70	Public Relations
80	Sales
90	Executive

Chave Primária

Conceitos básicos de bancos de dados relacionais

Uma tabela dentro de um banco de dados é dividida em colunas e linhas, mas temos muito mais definições do que se imagina.

Observe a imagem ao lado.



EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	SALARY	DEPARTMENT_ID
166	Sundar	Ande	SANDE	6400	80
167	Amit	Banda	ABANDA	6200	80
168	Lisa	Ozer	LOZER	11500	80
169	Harrison	Bloom	HBLOOM	10000	80
170	Tayler	Fox	TFOX	9600	80
171	William	Smith	WSMITH	7400	80
172	Elizabeth	Bates	EBATES	7300	80
173	Sundita	Kumar	SKUMAR	6100	80
174	Ellen	Abel	EABEL	11000	80
175	Alyssa	Hutton	AHUTTON	8800	80
176	Jonathon	Taylor	JTAYLOR	8600	80
177	Jack	Livingston	JLIVINGS	8400	80
178	Kimberely	Grant	KGRANT	7000	(null)
179	Charles	Johnson	CJOHNSON	6200	80
180	Winston	Taylor	WTAYLOR	3200	50
181	Jean	Fleaur	JFLEAUR	3100	50
182	Martha	Sullivan	MSULLIVA	2500	50
183	Girard	Geoni	GGEONI	2800	50
184	Nandita	Sarchand	NSARCHAN	4200	50
185	Alexis	Bull	ABULL	4100	50
186	Julia	Dellinger	JDELLING	3400	50

1 **Chave Primária**
Uma coluna que identifica de forma única cada linha da tabela. É composta por valores que nunca se repetem. Ela será fundamental para se criar relações com outras tabelas.

2 **Campo/Atributo**
É o nome dado para uma **coluna** de uma tabela. A coluna sempre será de um tipo específico: número, texto, data.

3 **Chave Estrangeira**
Nome dado à coluna que irá se relacionar com a chave primária de uma outra tabela. No exemplo ao lado, a coluna DEPARTMENT_ID possibilitará a relação com uma tabela de departamentos.

4 **Registro/Tupla**
É o nome dado para uma **linha** de uma tabela.

Índice Geral
Índice deste Módulo

A história da Linguagem SQL

Agora, voltando aos dois protótipos de sistema relacional mencionados anteriormente:

1. Ingres, desenvolvido pela UCB.
2. System R, desenvolvido pela IBM.

Cada um desses protótipos usava linguagens para consultas aos bancos de dados. Respectivamente, as linguagens QUEL e SEQUEL. Mas, com a popularização dos modelos de bancos de dados relacionais e o surgimento de SGBDRs, viu-se a necessidade de se criar um padrão para a linguagem de bancos de dados relacionais.

Foi aí que, na década de 1980, a linguagem foi padronizada pela American National Standards Institute (ANSI), sendo criado o **SQL** (*Structured Query Language* ou, Linguagem de Consulta Estruturada).

SQL Oracle vs. SQL ANSI

Apesar da linguagem SQL ter sido padronizada para uso em bancos de dados relacionais, temos várias empresas desenvolvedoras de SGBDRs, tais como Microsoft (SQL Server), IBM (DB2) e a Oracle (Oracle Database e MySQL), assim como SGBDRs de código aberto (como o PostgreSQL, MariaDB e SQLite).

Portanto, é natural que cada SGBD tenha alguma variação do SQL, e é aí que surgem o T-SQL (Transact-SQL, variação do SQL da Microsoft) e o PL-SQL (Procedural Language SQL – variação do SQL para o Oracle).

De qualquer forma, todos os SQLs têm uma mesma base de comandos padrão, proveniente da padronização feita pela ANSI. Por isso, ao começar a aprender SQL, foque em aprender SQL, não se preocupe com as possíveis variações. Conforme você for avançando, você vai entendendo as particularidades existentes entre os diferentes SGBDs.

MER, SGBD, ACID, CRUD ... que tal um dicionário?

Ao começar a aprender SQL, nos deparamos com algumas siglas e termos, a começar pela própria sigla SQL. E a cada passo que damos, nos deparamos com outras, como SGBD, ANSI, SQL Server, MySQL, PL-SQL e assim vai.

Vamos agora falar de mais alguns. A ideia é dar uma noção geral, até porque vamos nos aprofundar mais a frente, mas neste momento vale falar brevemente sobre isso.

SQL

Não poderia começar de outro jeito. A primeira sigla é SQL, que significa **Structured Query Language** (Linguagem de Consulta Estruturada).

É uma linguagem padrão utilizada em praticamente todos os sistemas de bancos de dados relacionais, tais como: SQL Server, MySQL, PostgreSQL, Oracle, SQLite, Db2, Access, MariaDB, etc.

É conhecida também como **SEQUEL** (*Structured English Query Language*).

Query (Consulta)

Uma ***QUERY*** é uma consulta de informações dentro de um banco de dados. É quando a gente “pede” para um banco de dados mostrar as informações que estiverem presentes dentro dele. Esse pedido é feito baseado nos comandos SQL.

Lembre-se: SQL significa *Structured **Query** Language* ou, traduzindo, Linguagem de **Consulta** Estruturada.

ANSI e ISO

ANSI significa *American National Standards Institute* (Instituto Nacional Americano de Padrões).
Já a **ISO** significa *International Organization for Standardization* (Organização Internacional de Normalização).

Ambos são responsáveis pela padronização de produtos, sistemas, processos para que possam ser utilizados no mundo todo.

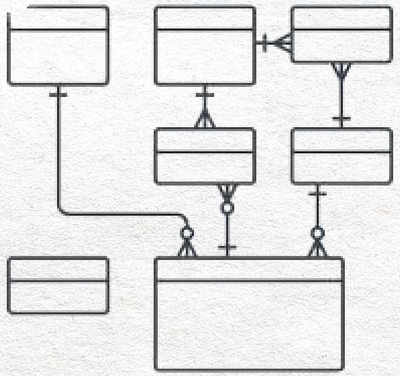
BD (ou DB)

Significa **Banco de Dados** (*Database*). Um banco de dados é um lugar onde armazenamos dados/informações dentro de tabelas.

MER e DER

O MER (**Modelo Entidade Relacionamento**) é um modelo que descreve como vai se dar o relacionamento entre cada entidade (tabela) de um banco de dados. Por exemplo, como uma entidade *Produto* se relaciona com uma entidade *Venda*.

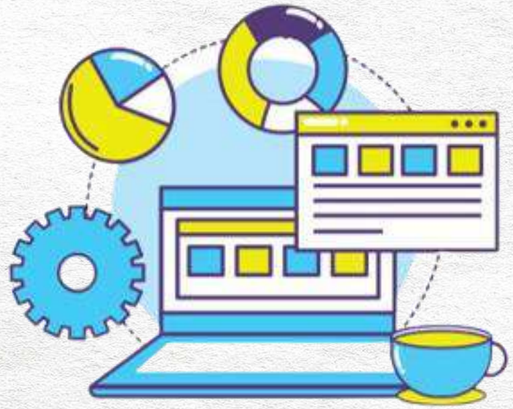
Já o DER (**Diagrama Entidade Relacionamento**) é uma representação gráfica do MER.



SGBDR

SGBDR significa **Sistema de Gerenciamento de Banco de Dados Relacional** (no inglês, RDBMS, *Relational Database Management System*).

É um programa que vamos usar para acessar, guardar, criar e gerenciar os Bancos de Dados Relacionais por meio da linguagem SQL.



MySQL, SQL Server, PostgreSQL, SQLite, Oracle, Db2, MariaDB, Access

São alguns nomes dos diferentes sistemas de gerenciamento de bancos de dados relacionais (SGBDR/RDBMS) onde podemos usar o SQL.

T-SQL

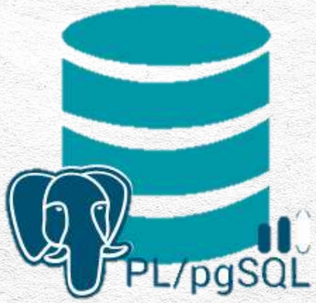
T-SQL significa Transact-SQL. É uma variação do SQL utilizada no SGBDR da Microsoft: o SQL Server. Possui base na própria linguagem SQL.

PL-SQL

PL-SQL significa Procedural Language-SQL. É uma variação do SQL utilizada no SGBDR da Oracle: o Oracle Database. Possui base na própria linguagem SQL.

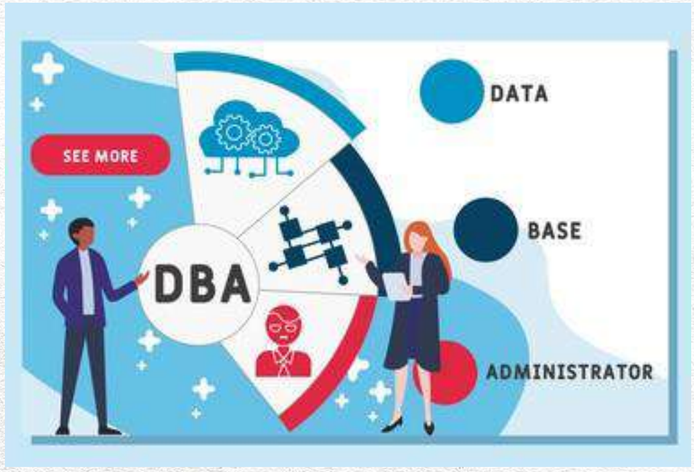
PL/pgSQL

PL/pgSQL também significa Procedural Language-SQL. É uma variação do SQL utilizada no SGBDR PostgreSQL. Possui base na própria linguagem SQL.



DBA

DBA significa **Administrador de Bancos de Dados** (*Database Administrator*). É uma das profissões mais conhecidas na área de bancos de dados.



CRUD

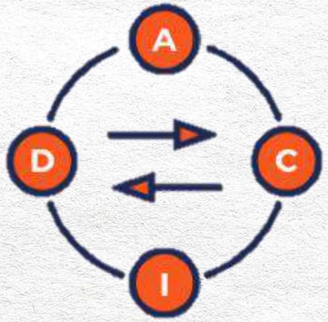
É um conjunto de comandos realizados dentro de bancos de dados para **CRIAR** (*Create*), **LER** (*Read*), **ATUALIZAR** (*Update*) e **DELETAR** (*Delete*) dados.

Esses nomes não são necessariamente os nomes dos comandos no SQL, apenas resumem a ideia do que este grupo de comandos faz.

ACID

Significa *ATOMICIDADE*, *CONSISTÊNCIA*, *ISOLAMENTO* e *DURABILIDADE*.

São propriedades das transações em bancos de dados (como *INSERT*, *UPDATE* e *DELETE*).



DML, DDL, DQL, DCL e TCL

São grupos de comandos dentro do SQL.

• DML (***Data Manipulation Language***): Linguagem de Manipulação de Dados é um conjunto de comandos para manipulação dos dados armazenados dentro das tabelas em um banco de dados:
Exemplos: INSERT, UPDATE e DELETE.

• DDL (***Data Definition Language***): Linguagem de definição de dados é usada para criar e modificar a estrutura dos objetos armazenados em um banco de dados.
Exemplos: ALTER, CREATE, DROP.

• DQL (***Data Query Language***): Linguagem de consulta de dados são os comandos de consulta aos dados armazenados em um banco de dados.
Exemplo: SELECT (Obs: em alguns lugares esse comando está agrupado no DML).

DML, DDL, DQL, DCL e TCL

São grupos grupos de comandos dentro do SQL.

• DCL (***Data Control Language***): Linguagem de Controle de Dados é usada para controle de acesso e permissões dos usuários em um banco de dados.
Exemplos: GRANT, REVOKE e DENY.

• TCL (***Transaction Control Language***): Linguagem de controle de transação são comandos usados para gerenciar as mudanças feitas pelos comandos DML.
Exemplos: COMMIT, ROLLBACK e SAVEPOINT.

Resumo do Módulo

Neste módulo, aprendemos que:

1. Computadores trouxeram a possibilidade de armazenar grandes volumes de dados, dentro dos chamados BANCOS DE DADOS RELACIONAIS.

2. BANCOS DE DADOS RELACIONAIS são conjuntos de tabelas que podem se relacionar, a partir de colunas em comum, chamadas de chaves. Podemos acessar, gerenciar e criar esses bancos de dados dentro dos chamados SGBDRs (Sistemas de Gerenciamento de Bancos de Dados Relacionais).

3. O SQL é a LINGUAGEM PADRÃO para se trabalhar com BANCOS DE DADOS RELACIONAIS.

4. A diversidade de SGBDRs (IBM, Microsoft, Oracle) resultou em algumas variações do SQL, como o T-SQL, PL-SQL e PL/pgSQL. Porém, todos têm a mesma base padronizada do SQL.

5. Conforme começamos a aprender sobre SQL e Bancos de Dados, nos deparamos com uma série de termos e siglas com muitos significados.

MÓDULO 2

INSTALAÇÃO

INSTALAÇÃO
INSTALAÇÃO

O QUE VEREMOS NESTE MÓDULO

MÓDULO 2 INSTALAÇÃO

01	CRIANDO CONTA GRATUITA NO SITE DA ORACLE	033
02	FAZENDO LOGIN NO SITE DA ORACLE	034
03	DOWNLOAD E INSTALAÇÃO DO ORACLE DATABASE 21C XE	035
04	DOWNLOAD E INSTALAÇÃO DO ORACLE SQL DEVELOPER	047

Criando conta gratuita no site da Oracle

O primeiro passo será criar uma conta Oracle para que seja possível realizar o download dos recursos que serão necessários para utilização do Oracle Database.

É possível criar uma conta gratuita acessando este site aqui:

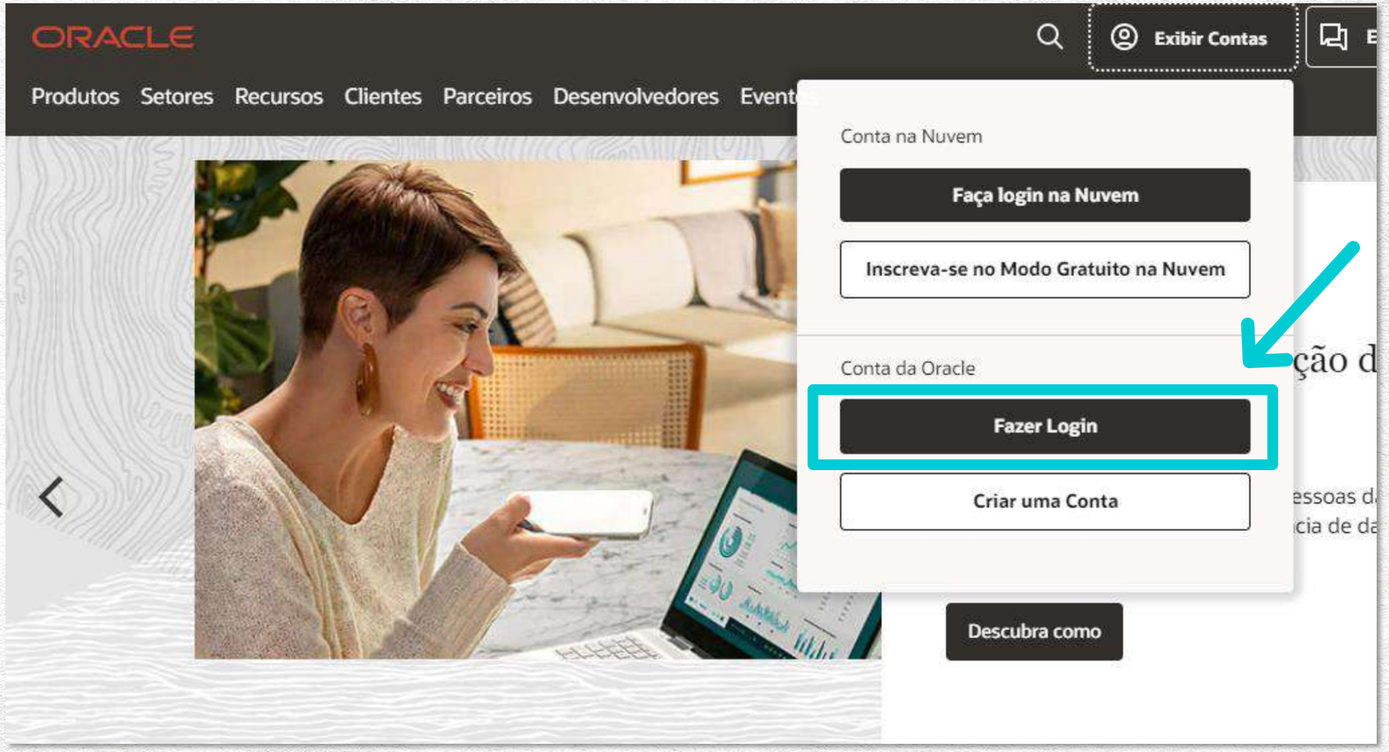
https://profile.oracle.com/myprofile/account/create-account.jspx

Preencha os campos solicitados para criar sua conta e depois clique em Criar Conta.

Você receberá um e-mail pedindo confirmação.

Fazendo login no site da Oracle

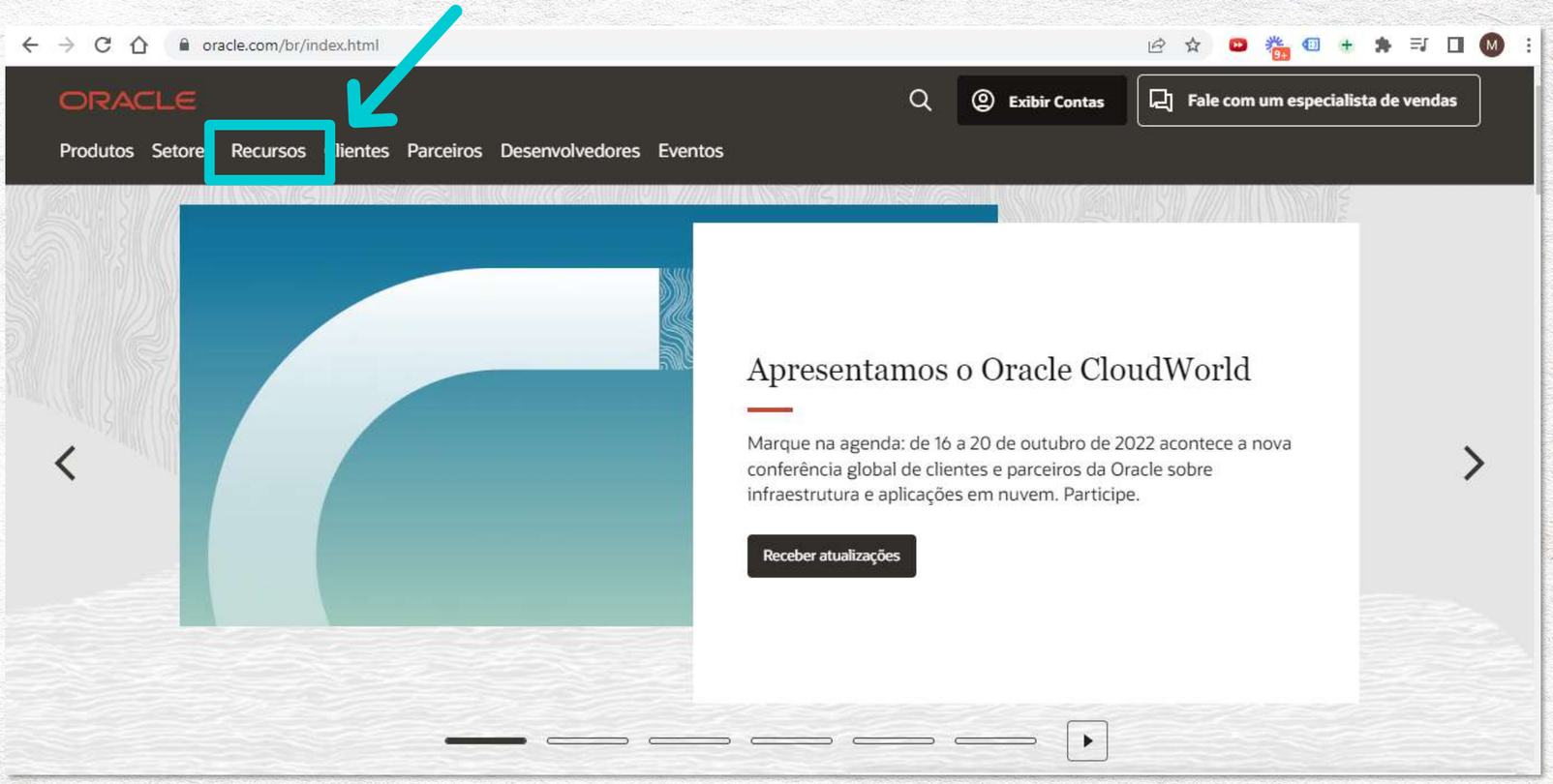
Após confirmar seu e-mail através do e-mail enviado pela Oracle, você deverá fazer o login da sua conta no site oracle.com/br. Para isto, basta clicar em Fazer Login, inserir seu e-mail e senha.



Download e instalação do Oracle Database 21c XE

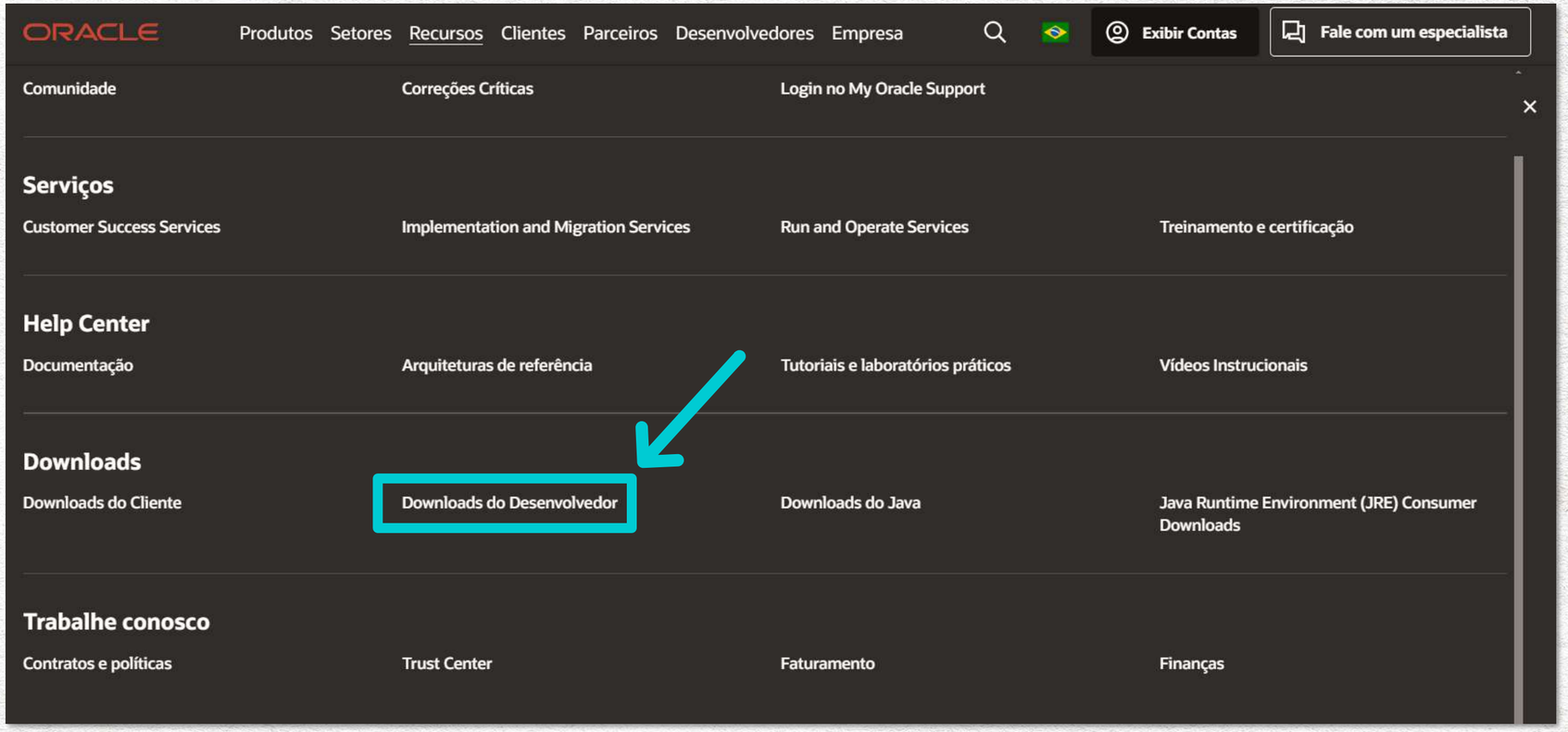
Agora que fizemos o login, estamos prontos para fazer o download do **Oracle Database 21c Express Edition**.

Na página ao lado, vamos clicar em **Recursos**.



Download e instalação do Oracle Database 21c XE

Em seguida, na seção **Downloads**, clique em **Downloads do Desenvolvedor**.

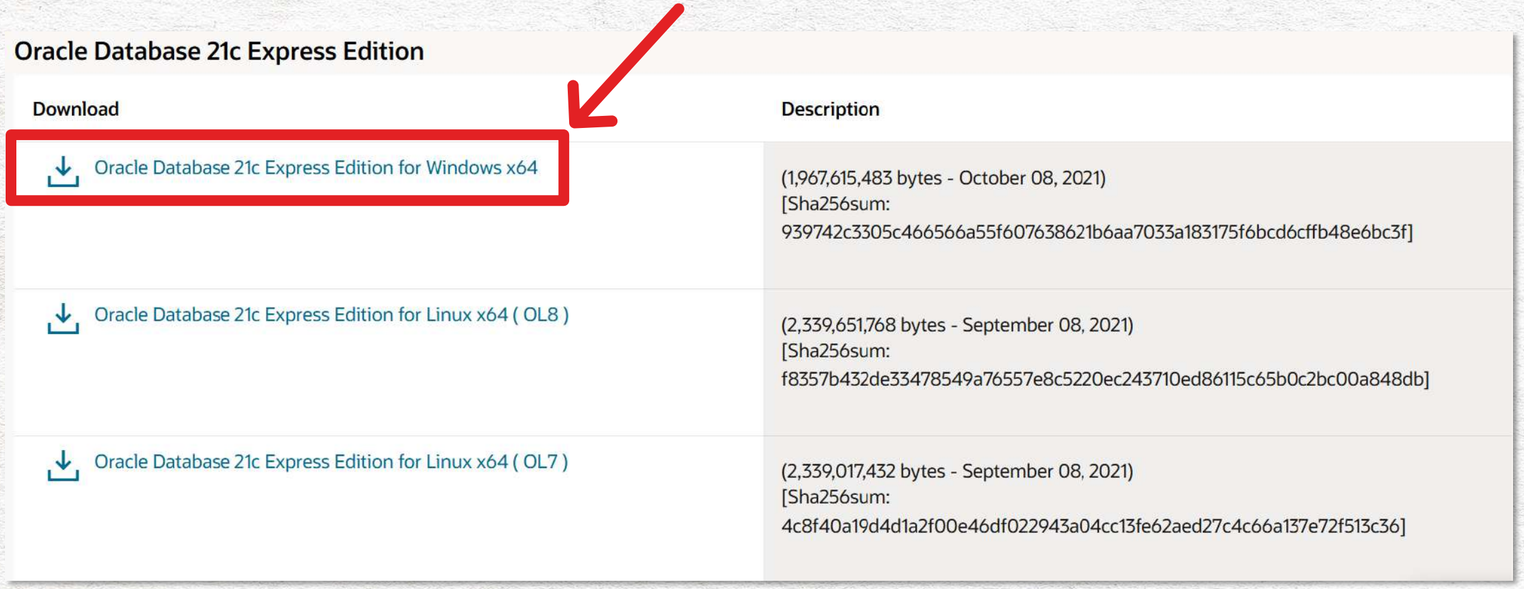


Download e instalação do Oracle Database 21c XE

Na página seguinte, na seção **Banco de Dados**, clique em **Database e Express Edition**:

Download e instalação do Oracle Database 21c XE

E na página seguinte, selecione a opção compatível com o seu sistema operacional (Windows ou Linux). Por exemplo, para **Windows**, selecione a opção indicada abaixo:



Oracle Database 21c Express Edition

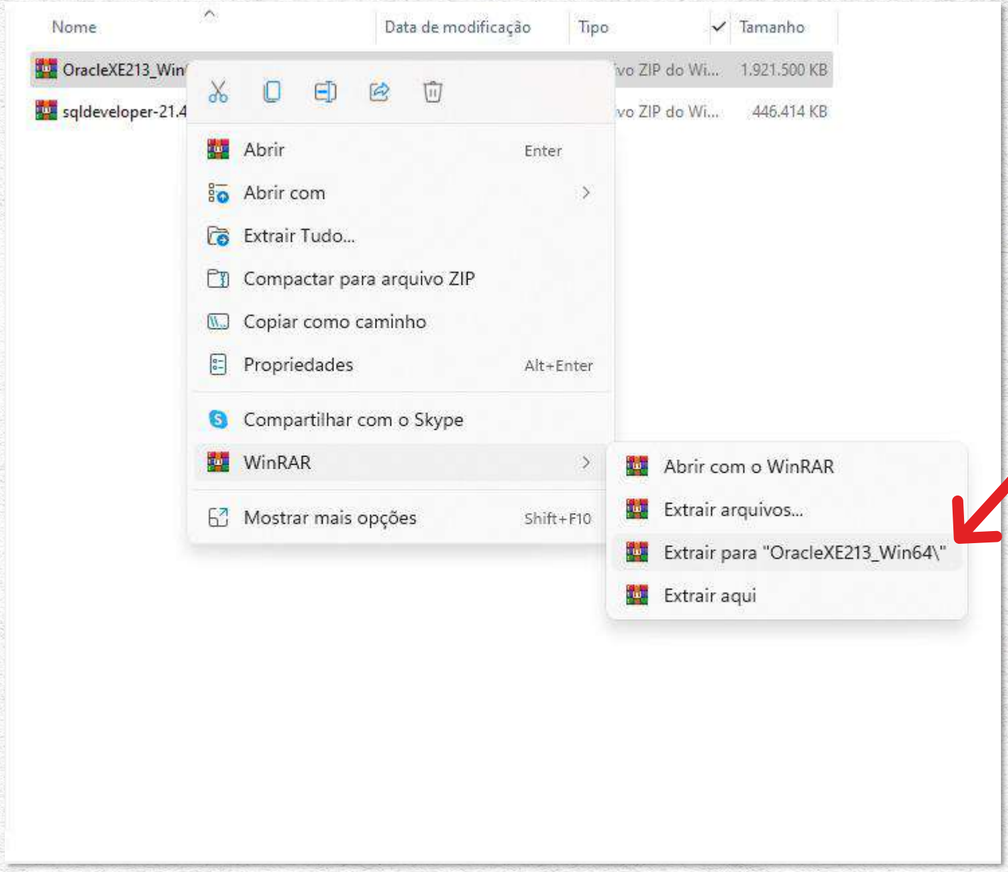
Download	Description
Oracle Database 21c Express Edition for Windows x64	(1,967,615,483 bytes - October 08, 2021) [Sha256sum: 939742c3305c466566a55f607638621b6aa7033a183175f6bcd6cffb48e6bc3f]
Oracle Database 21c Express Edition for Linux x64 (OL8)	(2,339,651,768 bytes - September 08, 2021) [Sha256sum: f8357b432de33478549a76557e8c5220ec243710ed86115c65b0c2bc00a848db]
Oracle Database 21c Express Edition for Linux x64 (OL7)	(2,339,017,432 bytes - September 08, 2021) [Sha256sum: 4c8f40a19d4d1a2f00e46df022943a04cc13fe62aed27c4c66a137e72f513c36]

Verifique se o seu sistema operacional e o hardware do seu computador preenchem os pré-requisitos necessários nestes links aqui e aqui da documentação do Oracle.

Download e instalação do Oracle Database 21c XE

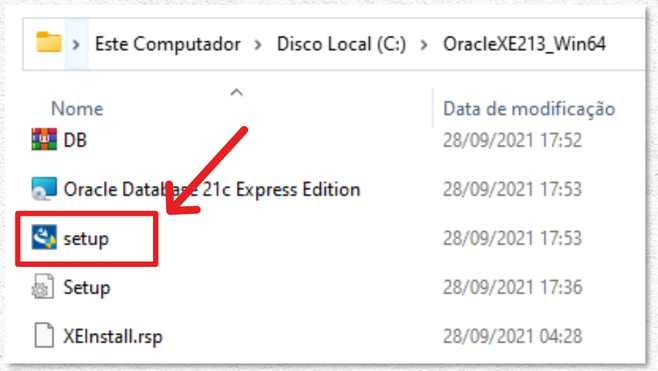
Após o download, precisaremos descompactar o arquivo zip chamado **OracleXE213_Win64**.

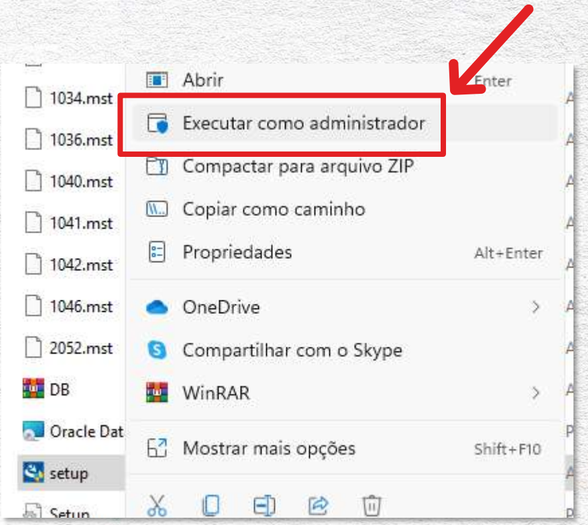
Você pode escolher uma pasta do seu computador para descompactar.



Download e instalação do Oracle Database 21c XE

Dentro da pasta descompactada, procure pelo arquivo setup, clique nele com o botão direito e depois em **Executar como administrador**.

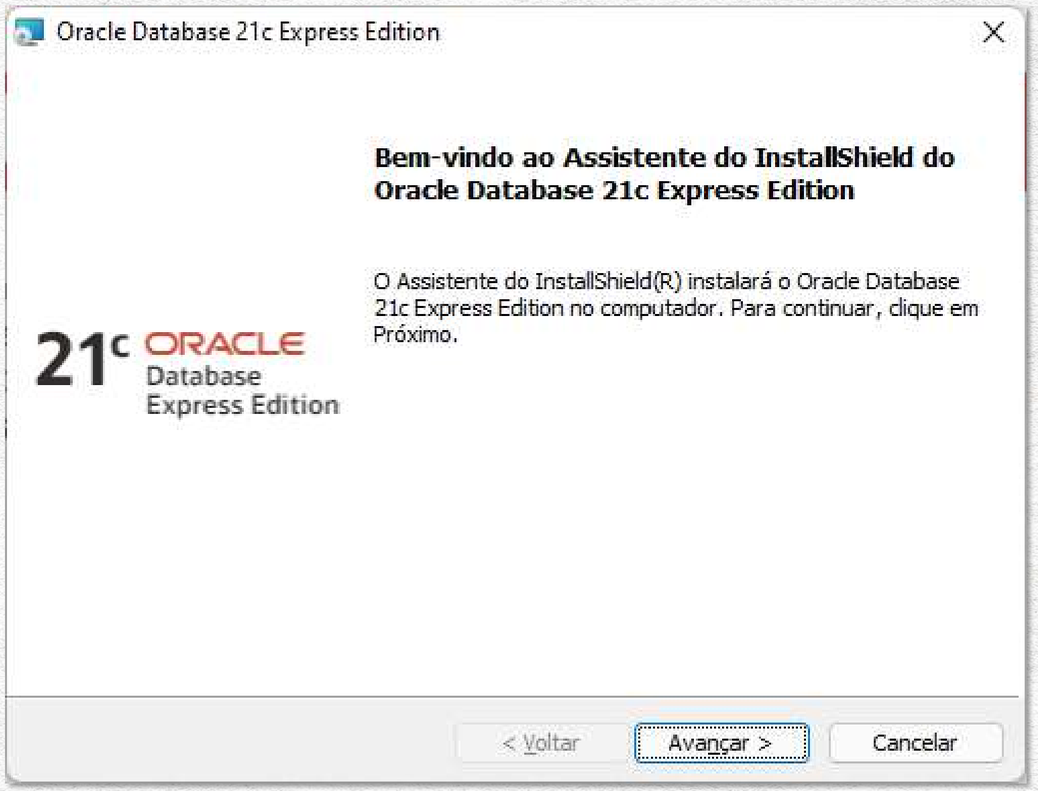




Download e instalação do Oracle Database 21c XE

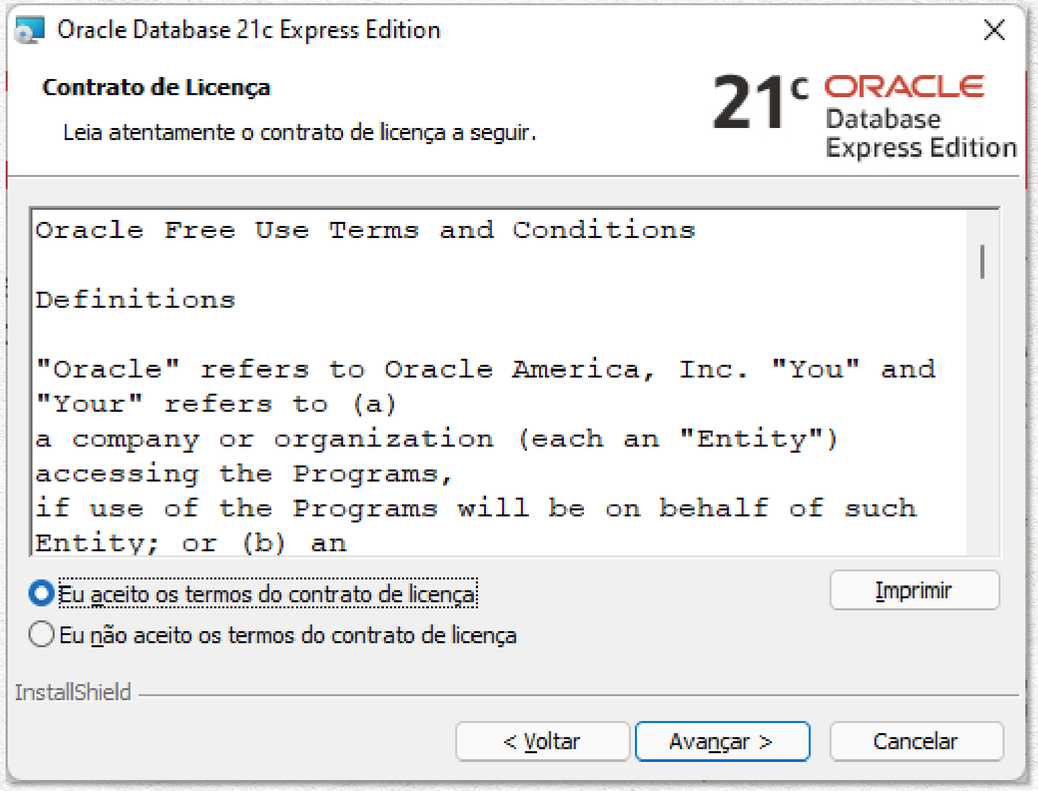
O assistente de instalação será aberto.

Clique em **Avançar**.



Download e instalação do Oracle Database 21c XE

Aceite os termos de instalação e clique em **Avançar** novamente.

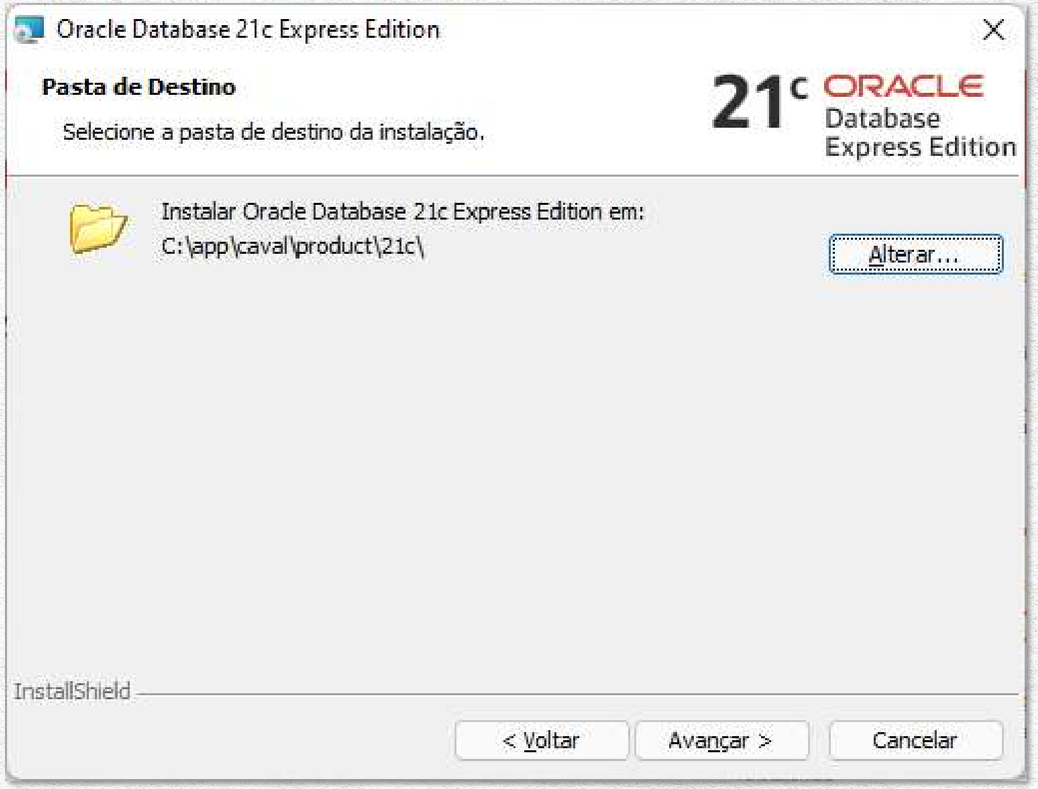


Download e instalação do Oracle Database 21c XE

Se desejar, altere a pasta de destino.

Aqui vamos deixar o padrão mesmo.

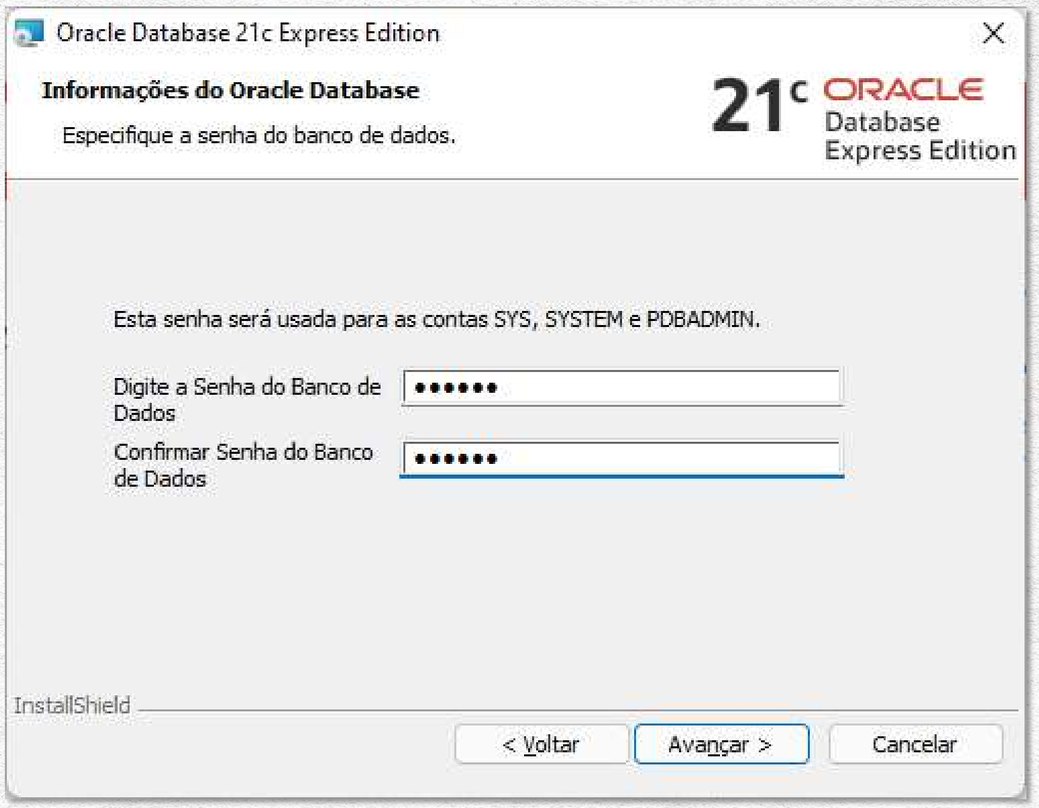
Em seguida, clique em **Avançar** novamente.



Download e instalação do Oracle Database 21c XE

Configure uma senha para o banco de dados. Esta senha será usada para as contas SYS, SYSTEM e PDBADMIN.

Como aqui estamos em um ambiente para fins didáticos, vamos configurar a senha ‘oracle’. Você pode usar a mesma senha ou escolher uma que desejar. Em seguida, clique em **Avançar**.

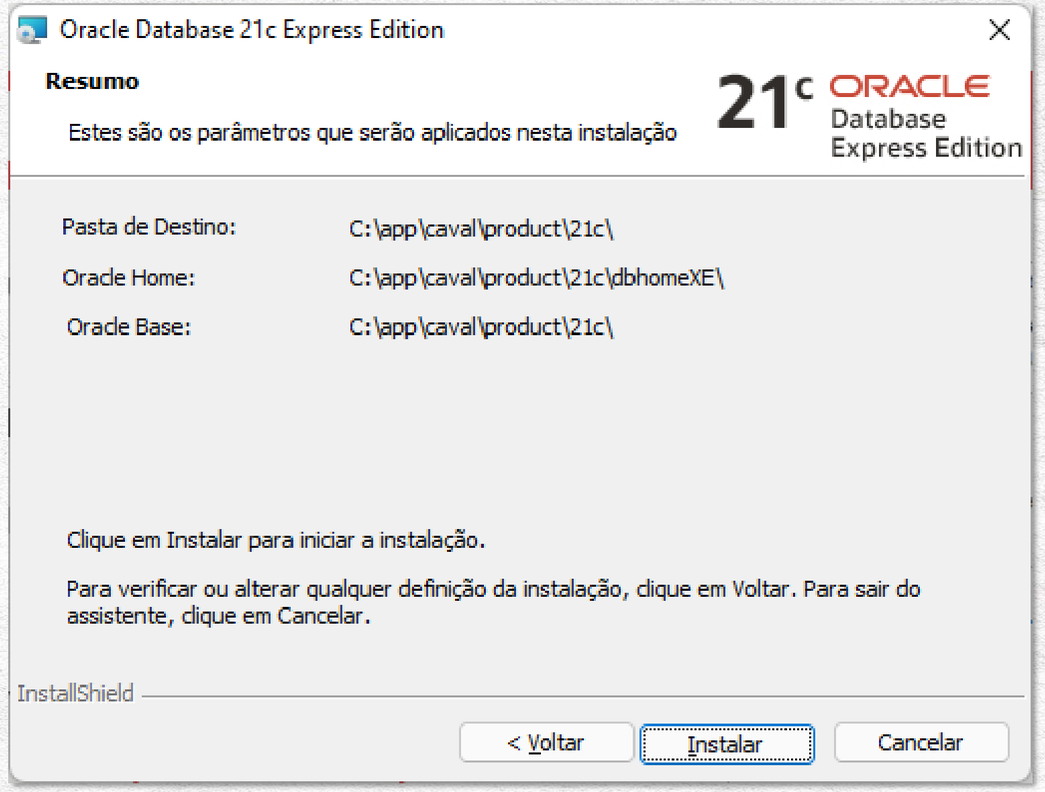


IMPORTANTE: anote a sua senha para não correr o risco de esquecê-la e evitar possíveis dores de cabeça no futuro.

Download e instalação do Oracle Database 21c XE

Na janela seguinte, basta clicar em Instalar.

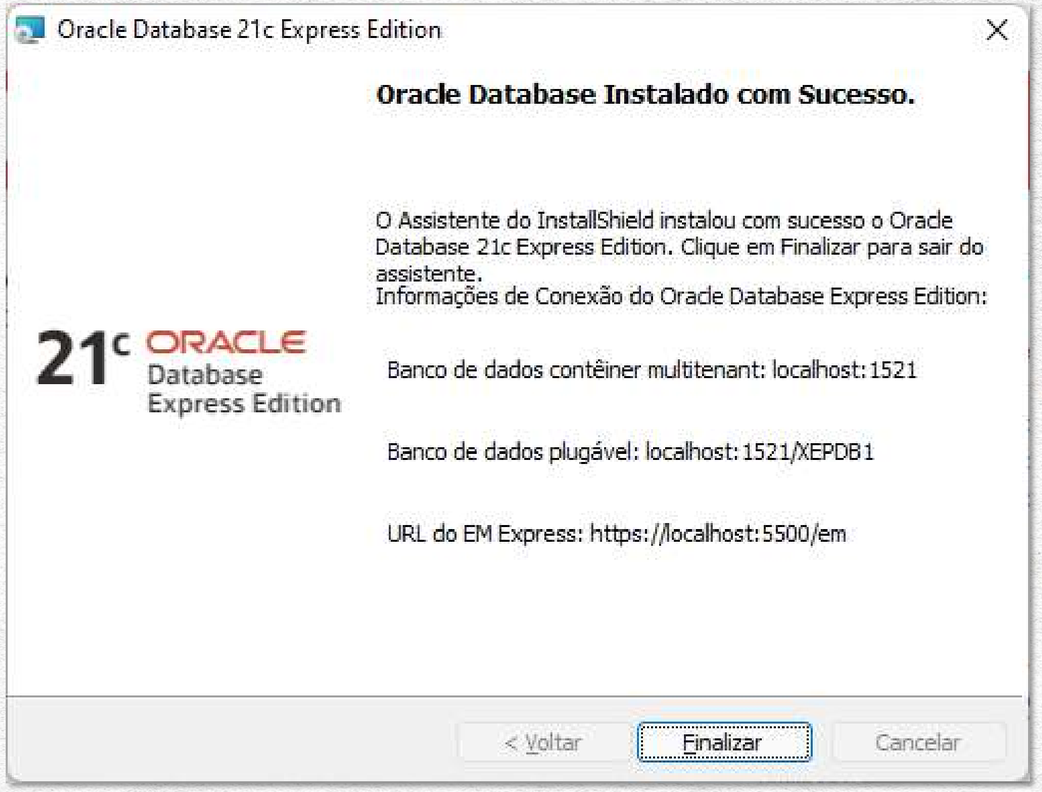
A instalação pode levar alguns minutos.



Download e instalação do Oracle Database 21c XE

Por fim, a instalação do Oracle Database 21c XE está concluída.

Clique em **Finalizar**.

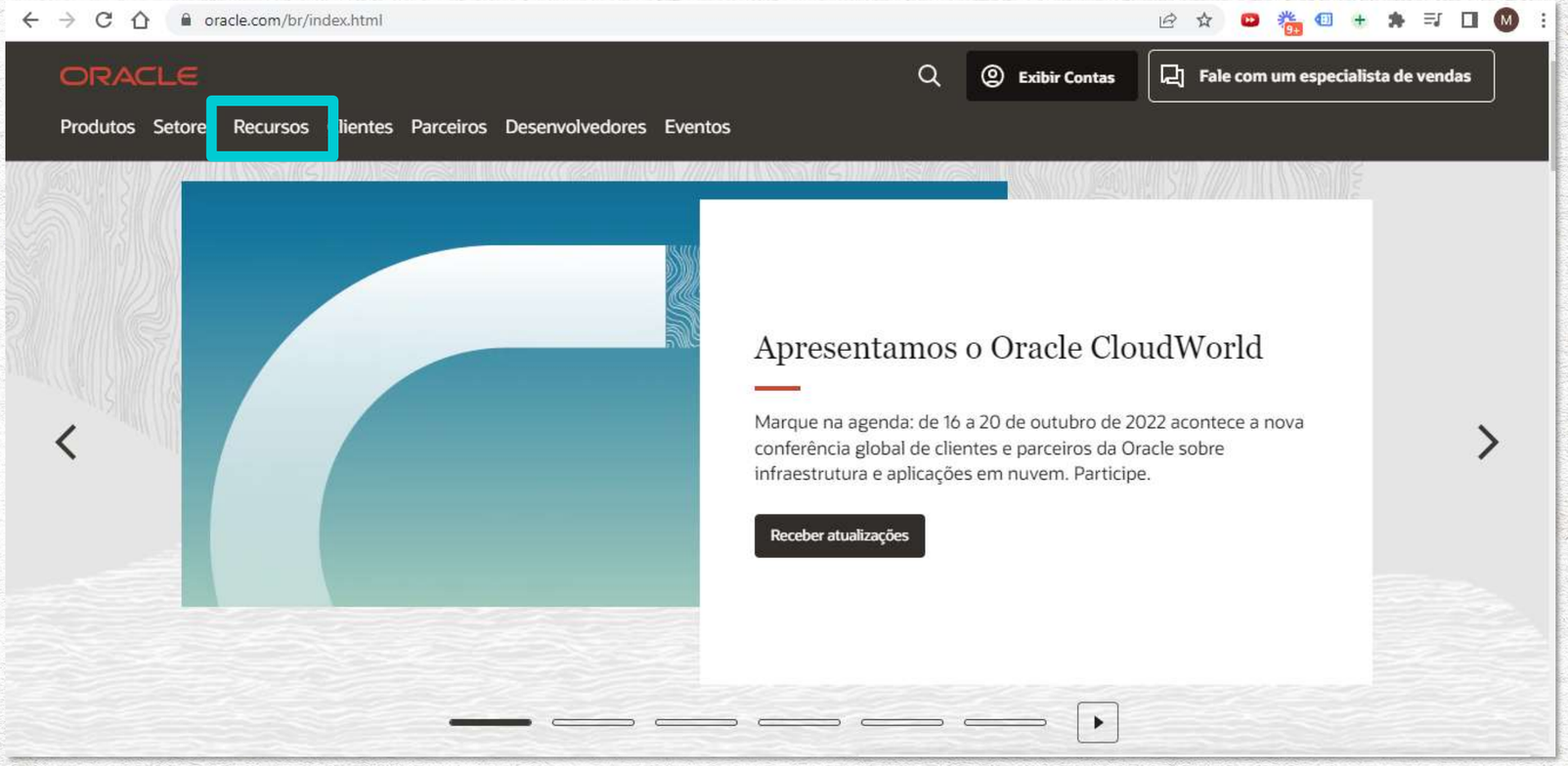


Download e instalação do Oracle SQL Developer

Agora que instalamos o Oracle Database, será necessário instalar a interface onde conseguiremos gerenciar os bancos de dados e utilizar os comandos SQL.

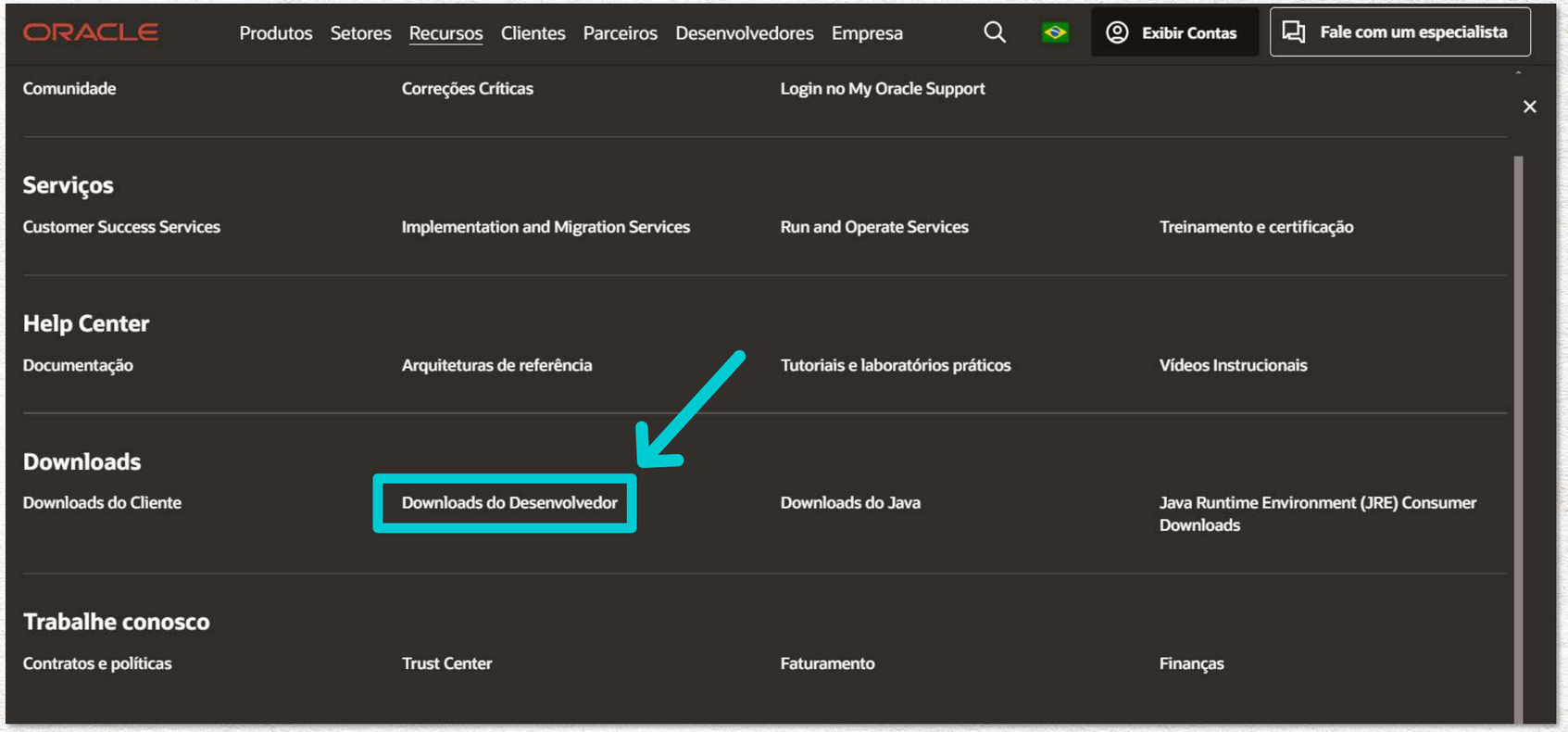
Para isso, vamos acessar o site oracle.com/br para fazer o download desta interface, que no caso será o **SQL Developer**.

Na página ao lado, vamos clicar em Recursos.



Download e instalação do Oracle SQL Developer

Em seguida, na seção **Downloads**, clique em **Downloads do Desenvolvedor**.

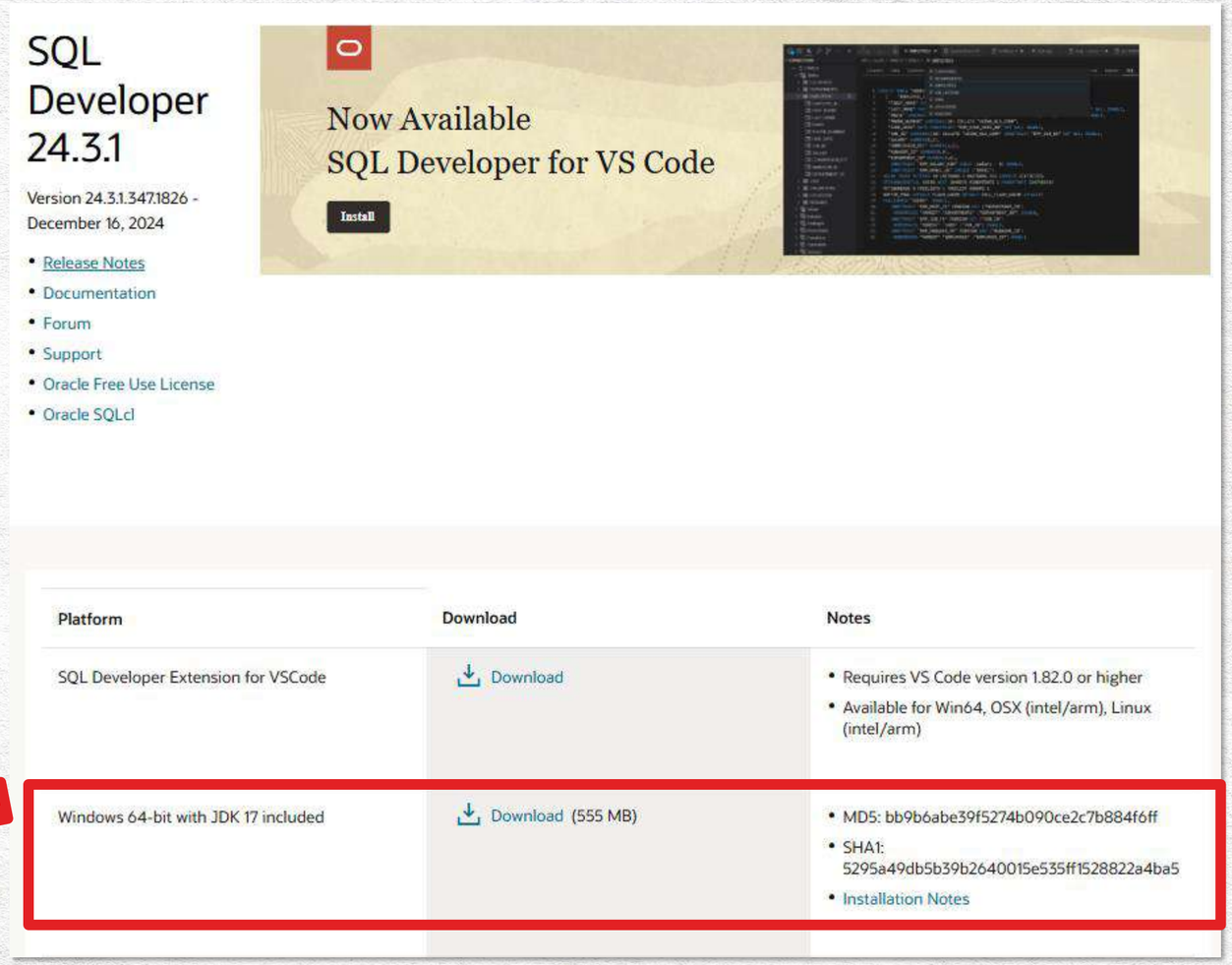


Download e instalação do Oracle SQL Developer

Na página seguinte, na seção **Ferramentas de desenvolvedor**, clique em **SQL Developer**:

Download e instalação do Oracle SQL Developer

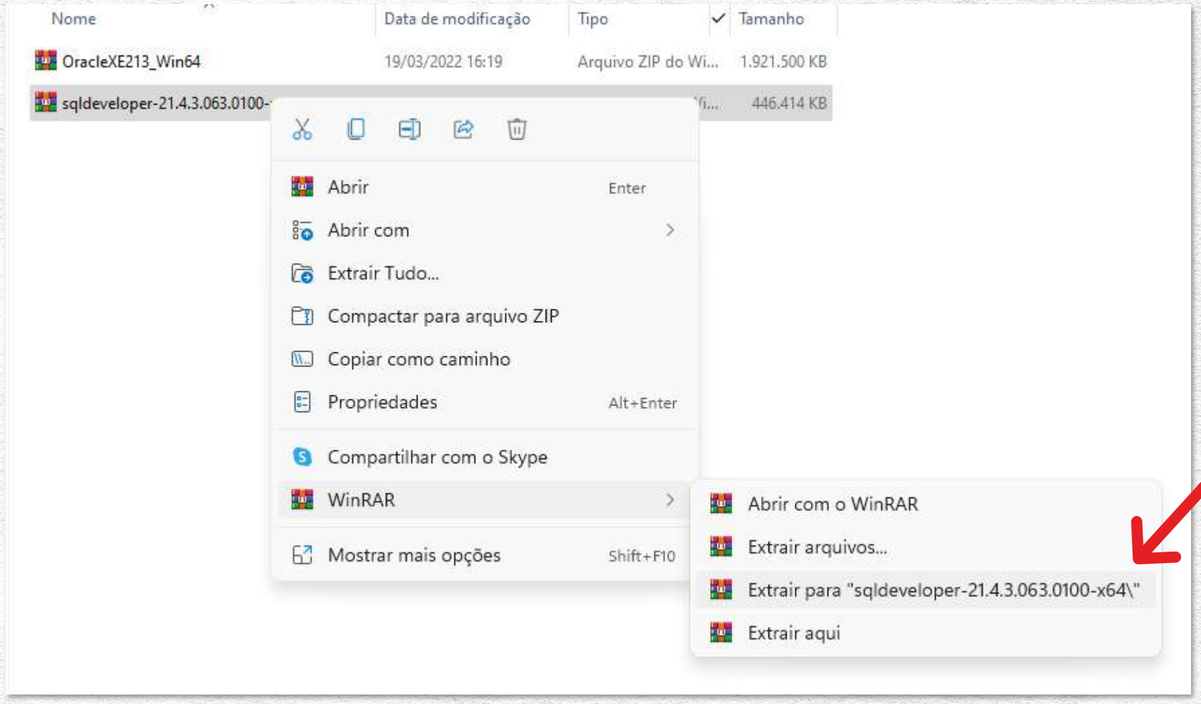
Se seu sistema operacional for Windows, escolha a opção destacada no print ao lado:



Download e instalação do Oracle SQL Developer

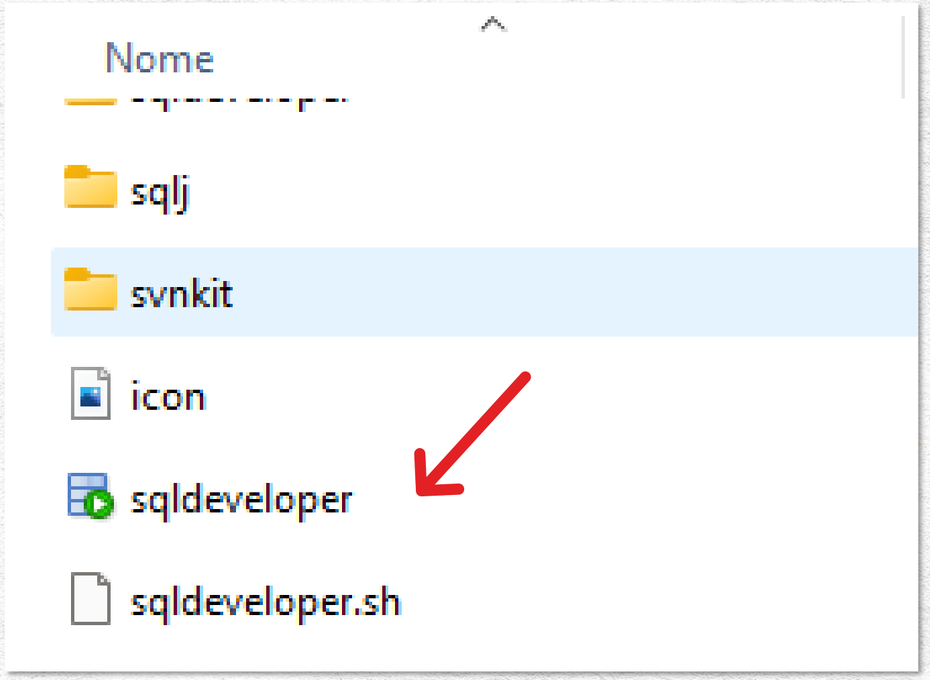
Após o download, precisaremos descompactar o arquivo zip chamado **sqldeveloper**.

Você pode escolher uma pasta do seu computador para descompactar.



Download e instalação do Oracle SQL Developer

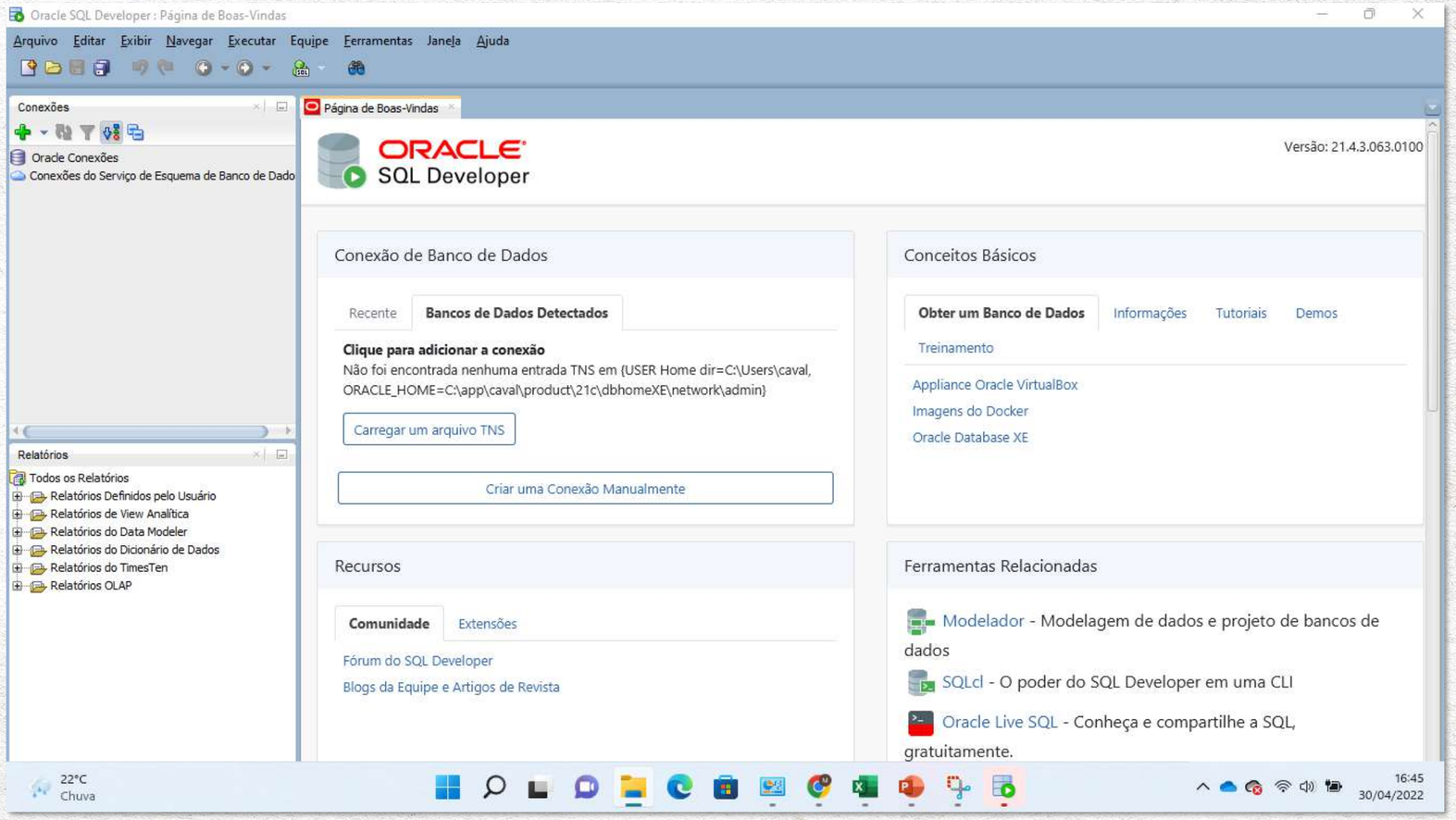
Na pasta que você descompactou os arquivos, procure pelo executável do sqldeveloper.



Download e instalação do Oracle SQL Developer

Pronto!

Agora temos tudo que é necessário para começar os trabalhos.



MÓDULO 3

ADMINISTRAÇÃO DE USUÁRIOS

ADMINISTRAÇÃO DE USUÁRIOS

ADMINISTRAÇÃO DE USUÁRIOS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 3 ADMINISTRAÇÃO DE USUÁRIOS

01	SOBRE CONTAS DE USUÁRIOS	056
02	PRIVILÉGIOS E PAPÉIS DE USUÁRIOS	069
03	USUÁRIOS PRÉ-DEFINIDOS	070
04	SOBRE AS CONTAS ADMINISTRATIVAS E PRIVILÉGIOS	071
05	CRIANDO A CONEXÃO SYS NO BANCO DE DADOS	072
06	CRIANDO O USUÁRIO HR	074
07	CRIANDO UMA CONEXÃO PARA O SCHEMA HR	084
08	CRIANDO AS TABELAS DO USUÁRIO HR	086

Sobre contas de usuários

Neste módulo, vamos falar brevemente sobre administração e privilégios de usuários.

Para que as pessoas possam acessar seu banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles. Imagine que você tem um sistema na sua empresa, e precisa que pessoas possam usá-lo. Dentro desse sistema, será possível fazer uma série de tarefas, porém cada funcionário poderá usar apenas parte desses recursos, ou a sua totalidade, dependendo da sua função.

Por exemplo, se você tem um sistema de pagamentos, você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema. Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.

Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.

A Oracle recomenda que você forneça a cada usuário apenas os privilégios necessários para que ele execute o trabalho.

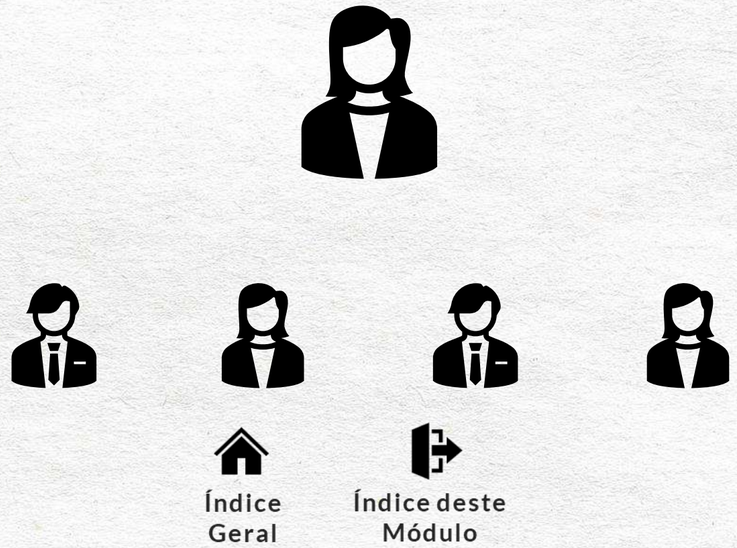
Sobre contas de usuários

Para acessar um banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles.

Imagine que você tem um sistema na sua empresa, por exemplo, um sistema de pagamentos. Você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema.

Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.



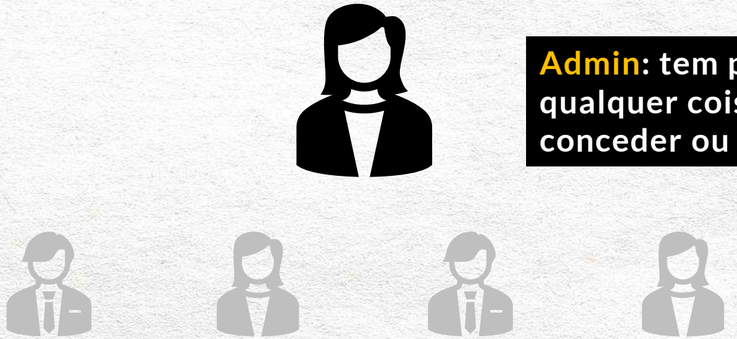
Sobre contas de usuários

Para acessar um banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles.

Imagine que você tem um sistema na sua empresa, por exemplo, um sistema de pagamentos. Você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema.

Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.



Admin: tem permissão para fazer qualquer coisa, como por exemplo, conceder ou não privilégios aos demais

Sobre contas de usuários

Para acessar um banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles.

Imagine que você tem um sistema na sua empresa, por exemplo, um sistema de pagamentos. Você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema.

Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.

Usuário 1: tem permissão para tudo, exceto excluir usuários

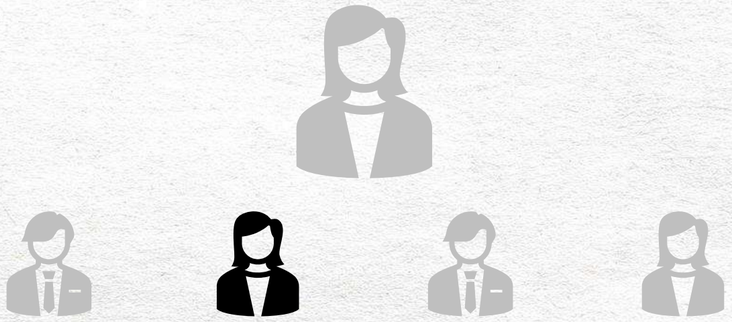
Sobre contas de usuários

Para acessar um banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles.

Imagine que você tem um sistema na sua empresa, por exemplo, um sistema de pagamentos. Você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema.

Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.



Usuário 2: tem permissão apenas para criar e excluir novos produtos

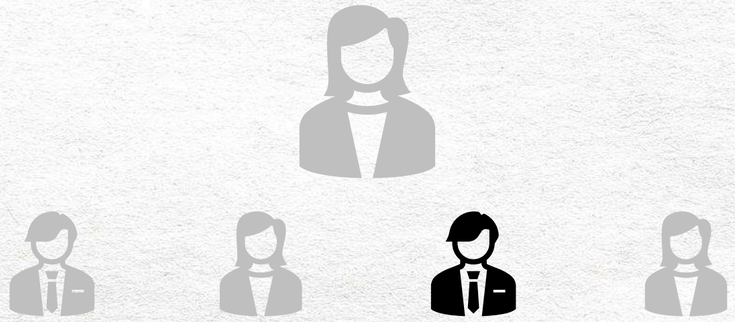
Sobre contas de usuários

Para acessar um banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles.

Imagine que você tem um sistema na sua empresa, por exemplo, um sistema de pagamentos. Você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema.

Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.



Usuário 3: tem permissão apenas para consultar vendas

Sobre contas de usuários

Para acessar um banco de dados, você precisa criar usuários e fornecer os devidos privilégios a eles.

Imagine que você tem um sistema na sua empresa, por exemplo, um sistema de pagamentos. Você poderá ter uma série de níveis de privilégios dos usuários que acessam esse sistema.

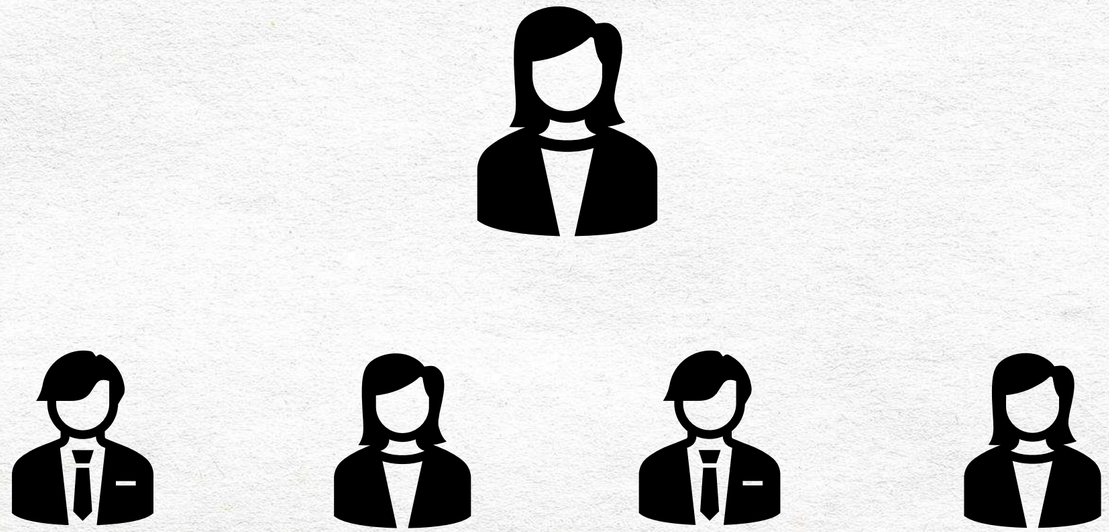
Alguns poderão fazer tudo (normalmente os admins), como criar/editar/excluir usuários, configurar formas de pagamentos, criar novos produtos, visualizar as vendas, reembolsar, e assim vai.

Outros poderão apenas visualizar as vendas e reembolsar, e alguns poderão apenas visualizar as vendas.

Usuário 4: tem permissão apenas para visualizar vendas e reembolsar

Sobre contas de usuários

Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.



Sobre contas de usuários

Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.

USUÁRIO SYS: admin, aquele que pode fazer qualquer coisa em um banco de dados

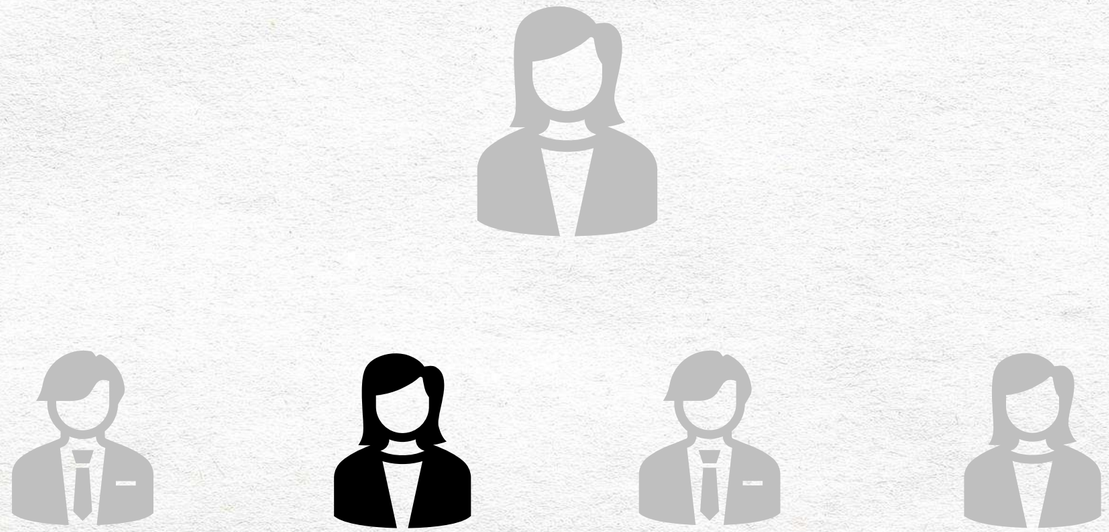
Sobre contas de usuários

Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.

USUÁRIO 1: usuário que pode fazer tudo exceto criar outros usuários

Sobre contas de usuários

Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.



USUÁRIO 2: usuário que pode apenas criar tabelas no banco de dados

Sobre contas de usuários

Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.

USUÁRIO 3: usuário que pode criar Triggers, Functions, Procedures, atualizar tabelas, mas não pode excluir nem criar tabelas

Sobre contas de usuários

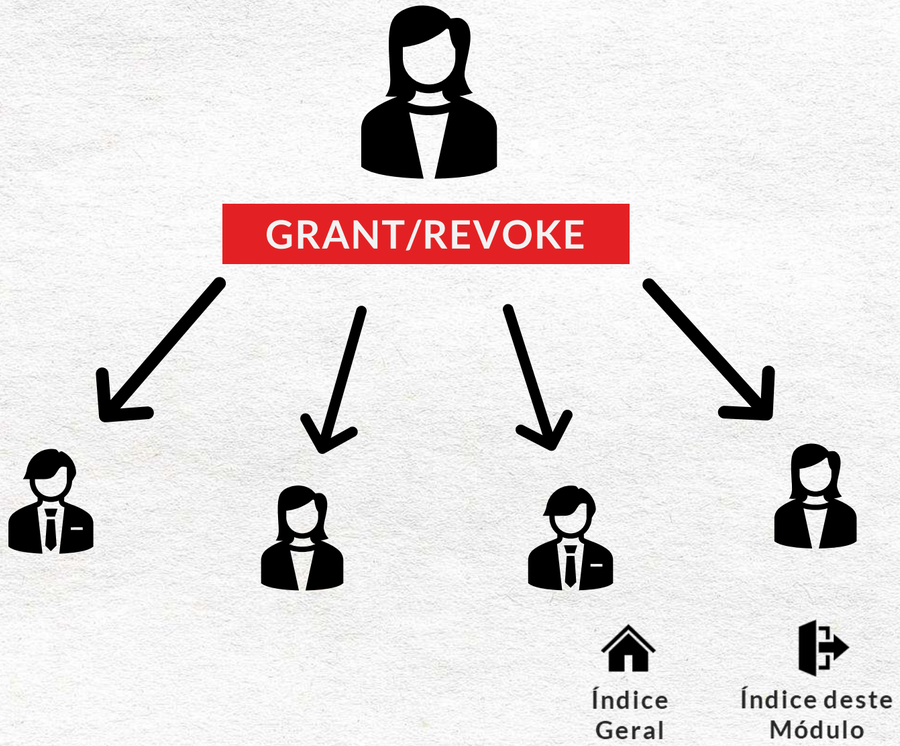
Do ponto de vista do Oracle Database, teremos algo semelhante. Na hora de criar um usuário para usar os serviços do banco de dados, é necessário não só dar a ele um login e senha, como também fornecer a ele os privilégios que a sua conta terá.

USUÁRIO 4: usuário que pode apenas selecionar/visualizar tabelas

Privilégios e papéis de usuários

Os privilégios de usuários oferecem níveis de segurança ao que pode ser feito em um banco de dados. Os privilégios são usados para controlar o acesso aos dados e também limitar quais tipos de comandos (DDL, DML, DCL, TCL, etc.) podem ser executados por aquele usuário.

Quando você cria um usuário, você concede (GRANT) privilégios para permitir ao usuário se conectar a um banco de dados, executar queries, criar objetos no banco de dados ou atualizar informações, dentre outras coisas.



A seguir, vamos criar o usuário HR no Oracle Database.

Utilizaremos alguns comandos que nos aprofundaremos apenas mais a frente.

Neste primeiro momento, precisaremos apenas criar o usuário para dar continuidade ao curso.

Usuários pré-definidos

Alguns usuários são criados automaticamente durante a instalação. São eles: **SYS**, **SYSTEM**, **SYSMAN** e **DBSNMP**.

Estas são contas administrativas, ou seja, contas que possuem vários privilégios, como o ADMIN de um sistema. Ao fazer o login no Oracle Database, usamos uma destas contas.

Também veremos que é possível criar novos usuários, configurando diferentes níveis de privilégios a cada um deles.

Sobre as contas administrativas e privilégios

CONTAS ADMINISTRATIVAS permitem a você executar funções administrativas como: gerenciar usuários, gerenciar a memória de bancos de dados, iniciar ou desconectar um banco de dados, dentre outras coisas.

1. Usuários SYS e SYSTEM

Estes usuários são criados automaticamente quando você instala o Oracle Database. Ambos são criados com a senha informada durante a instalação e os dois automaticamente recebem os privilégios de um DBA (Administrador de Banco de Dados).

Dentre os dois, o SYS possui mais privilégios sob o ponto de vista do banco de dados.

2. Privilégios SYSDBA e SYSOPER

Estes são privilégios administrativos, necessários para executar operações de alto nível em um banco de dados, como criar, iniciar, desconectar, criar backup ou recuperar bancos de dados. Dentre os dois privilégios, o SYSDBA é mais completo.

Criando a conexão sys no Banco de Dados

Vamos então criar a conexão sys dentro do Oracle Database.

1 Clique na opção ✚ para criar uma nova conexão para o banco de dados.

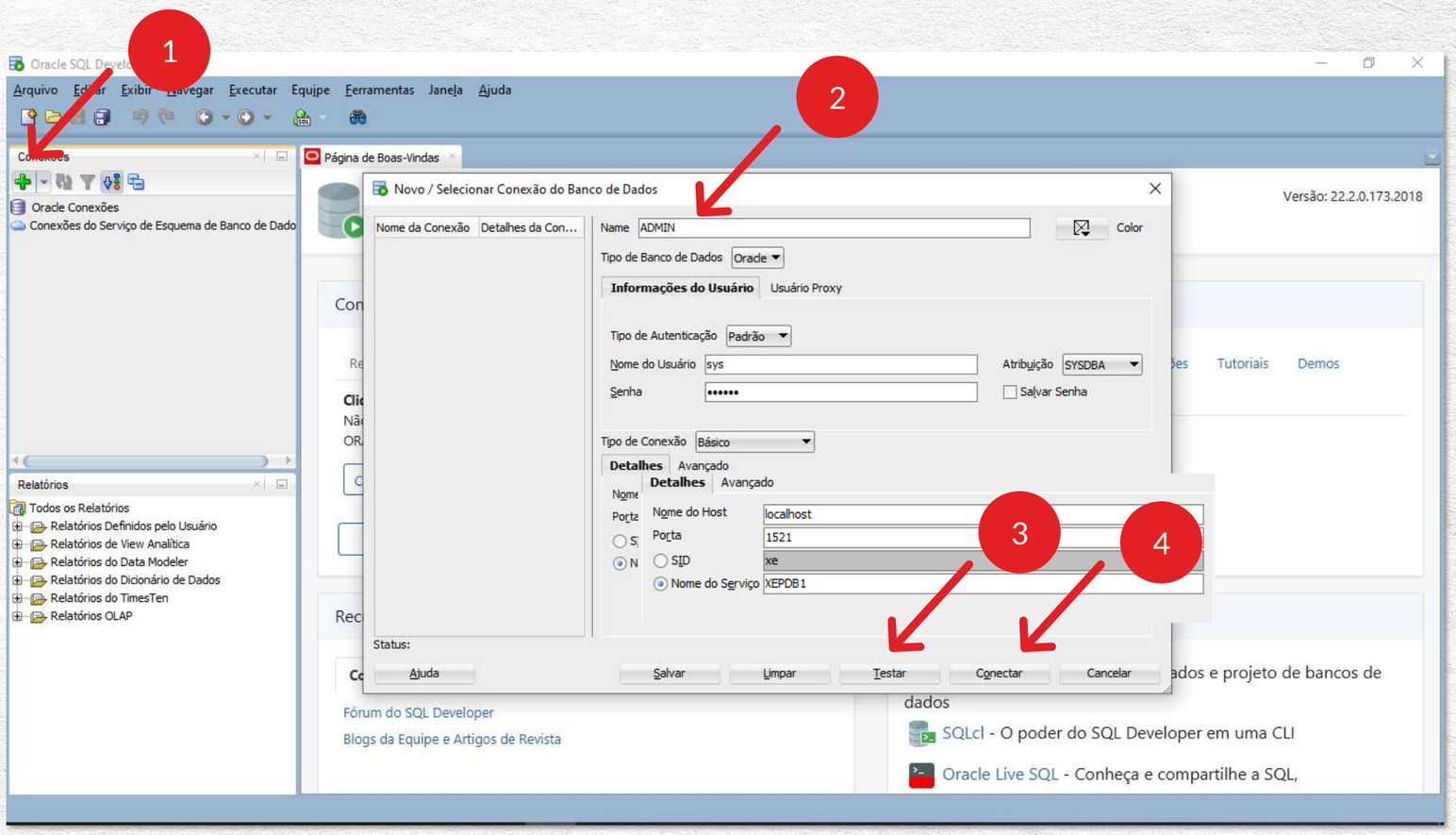
2 Na janela que se abrir, preencha com os seguintes dados:

- Nome: ADMIN
- Nome do Usuário: sys
- Senha: oracle (ou a senha que você configurou)
- Atribuição: SYSDBA
- Nome do Serviço: XEPDB1

O resto pode deixar o padrão.

3 Clique em Testar para testar a conexão. O status deverá aparecer como **Com sucesso**.

4 Clique em Conectar.



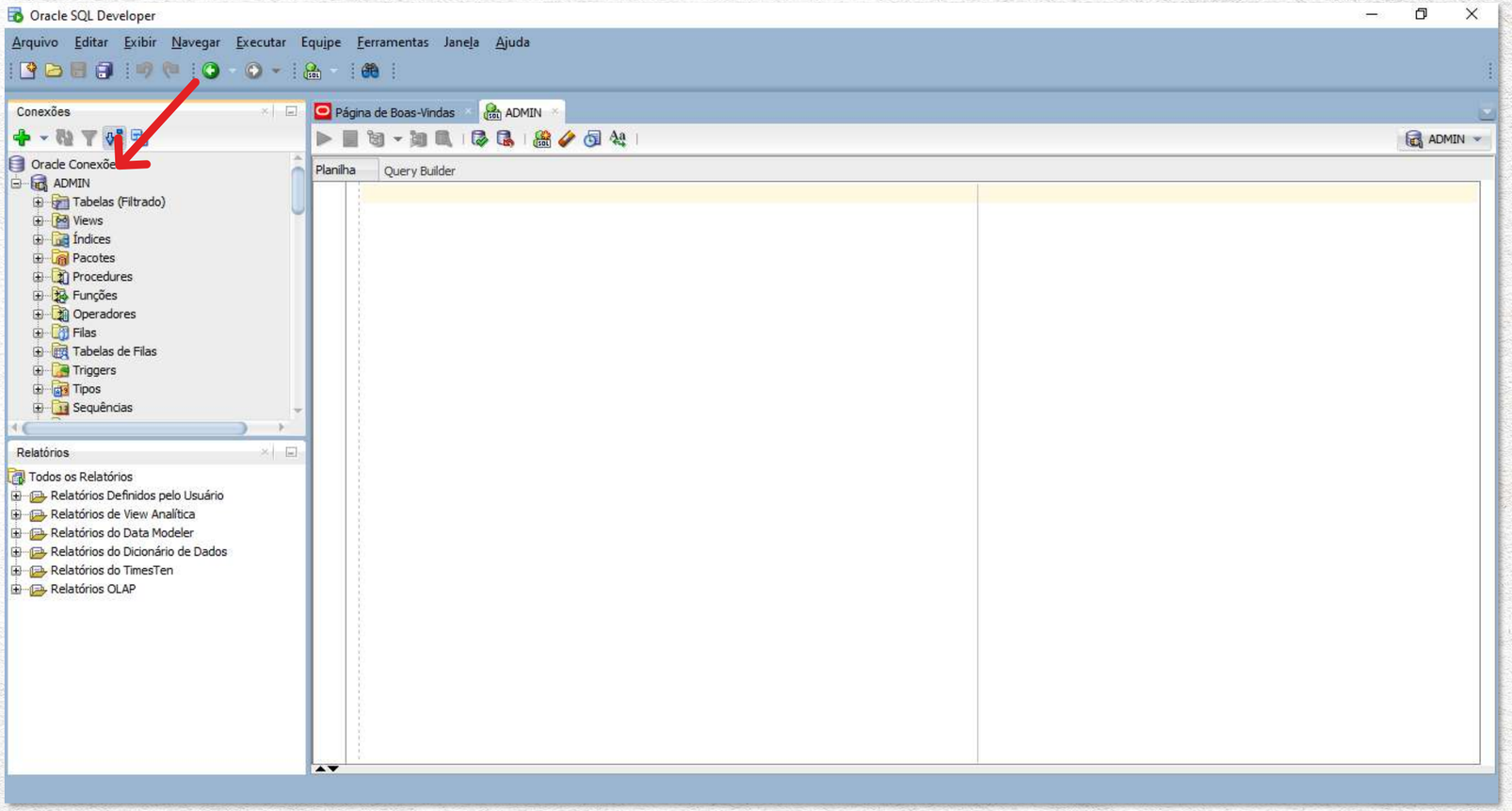
Criando a conexão sys no Banco de Dados

Pronto!

Já criamos a nossa conexão ADMIN que terá os privilégios administrativos.

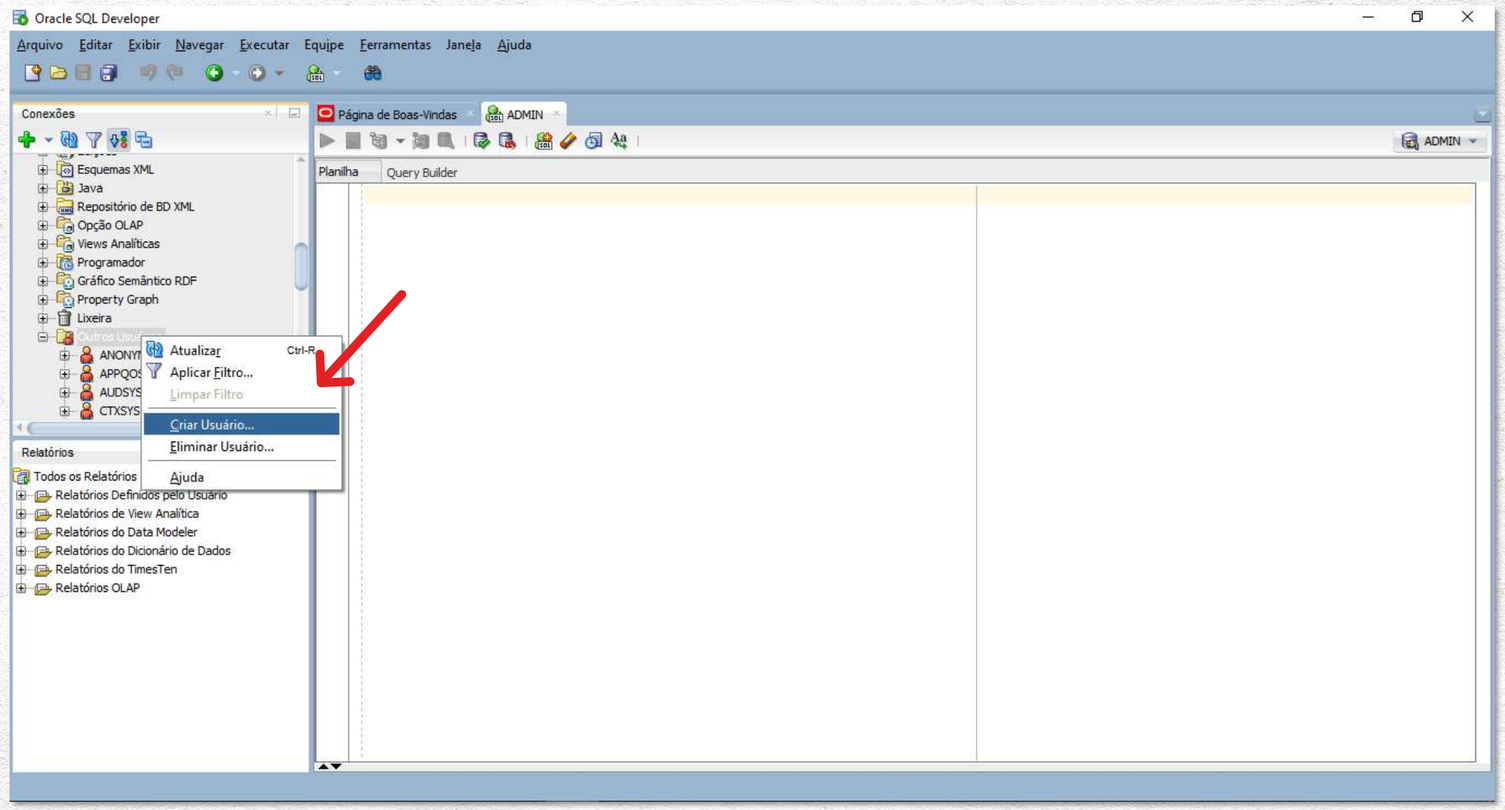
O próximo passo será criar uma conexão para o banco de dados que vamos usar no curso: o HR.

Mas antes, precisaremos criar um usuário HR, e conceder a ele permissões específicas.



Criando o usuário HR

Para criar o usuário HR, na conexão ADMIN, vá até Outros Usuários, clique com o botão direito, em seguida em Criar Usuário...

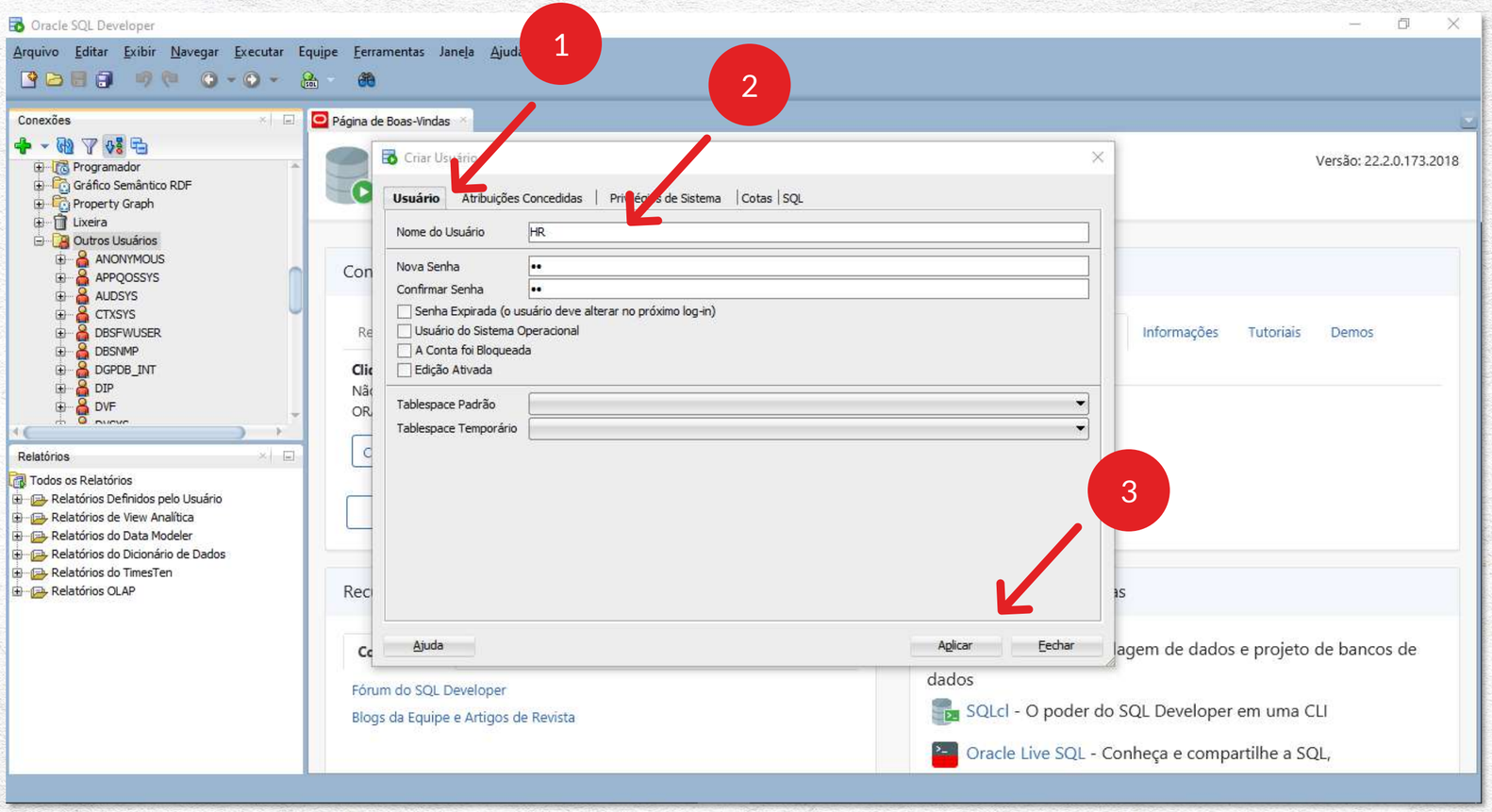


Criando o usuário HR

1 Na aba Usuário da janela que se abrir, preencha com os seguintes dados:

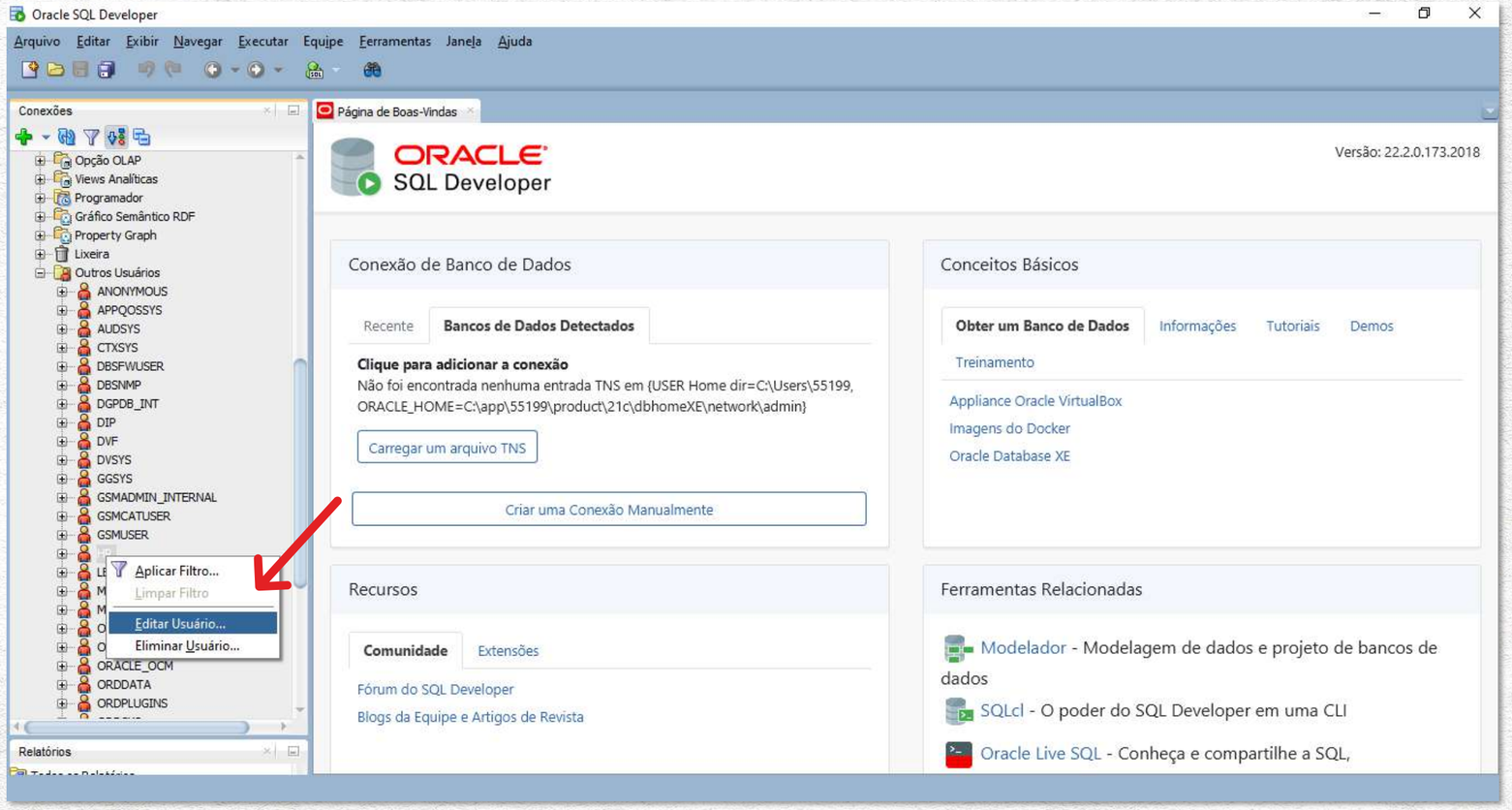
2
- Nome do Usuário: HR
- Nova Senha: hr
- Confirmar Senha: hr

3 Clique em Aplicar.



Criando o usuário HR

Voltando em Outros Usuários, procure o usuário HR que você acabou de criar, clique com o botão direito em cima dele, em seguida em Editar Usuário...

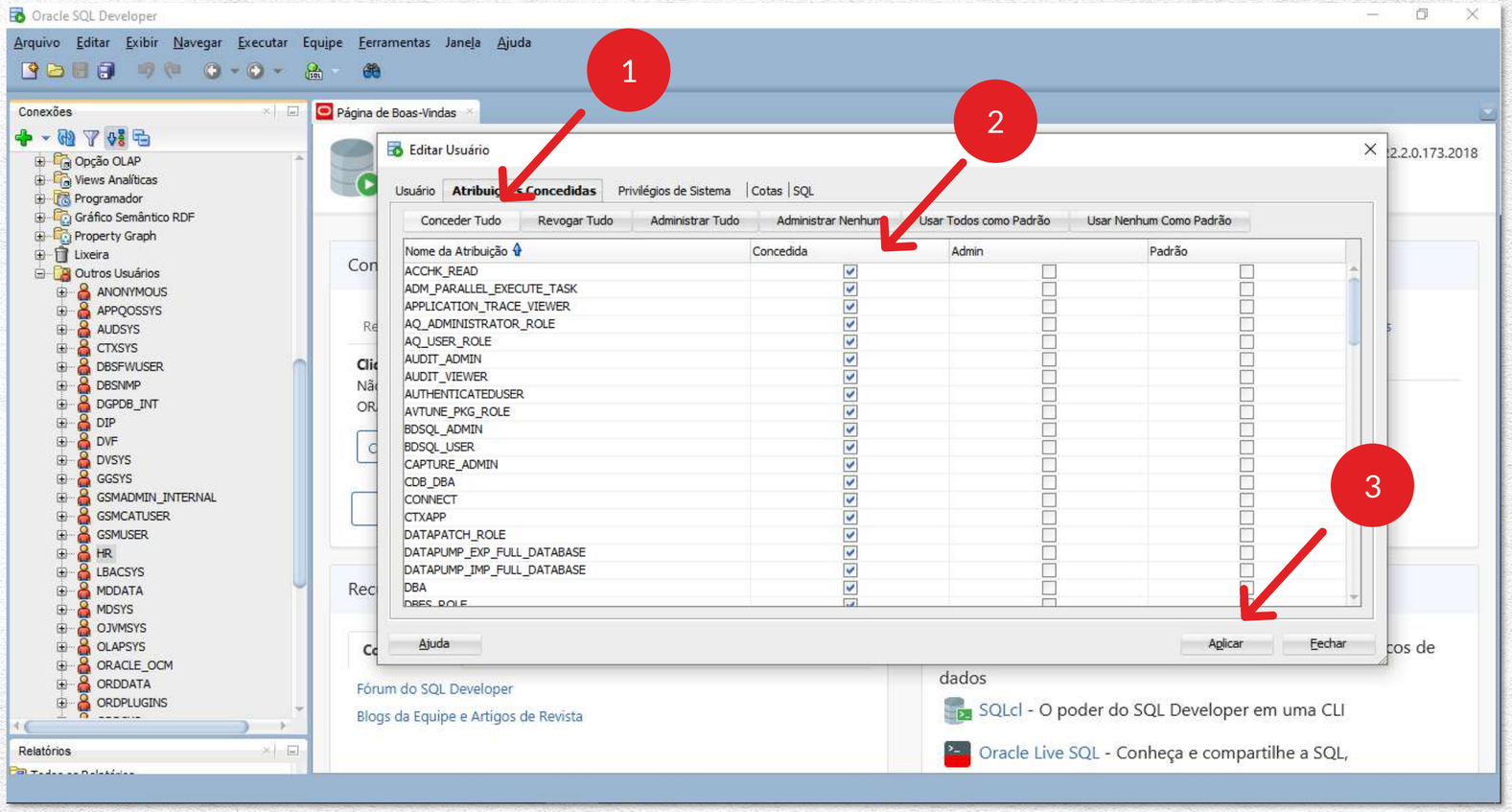


Criando o usuário HR

1. Na nova janela que se abrir, com a aba Atribuições Concedidas selecionada, clique em Conceder Tudo:

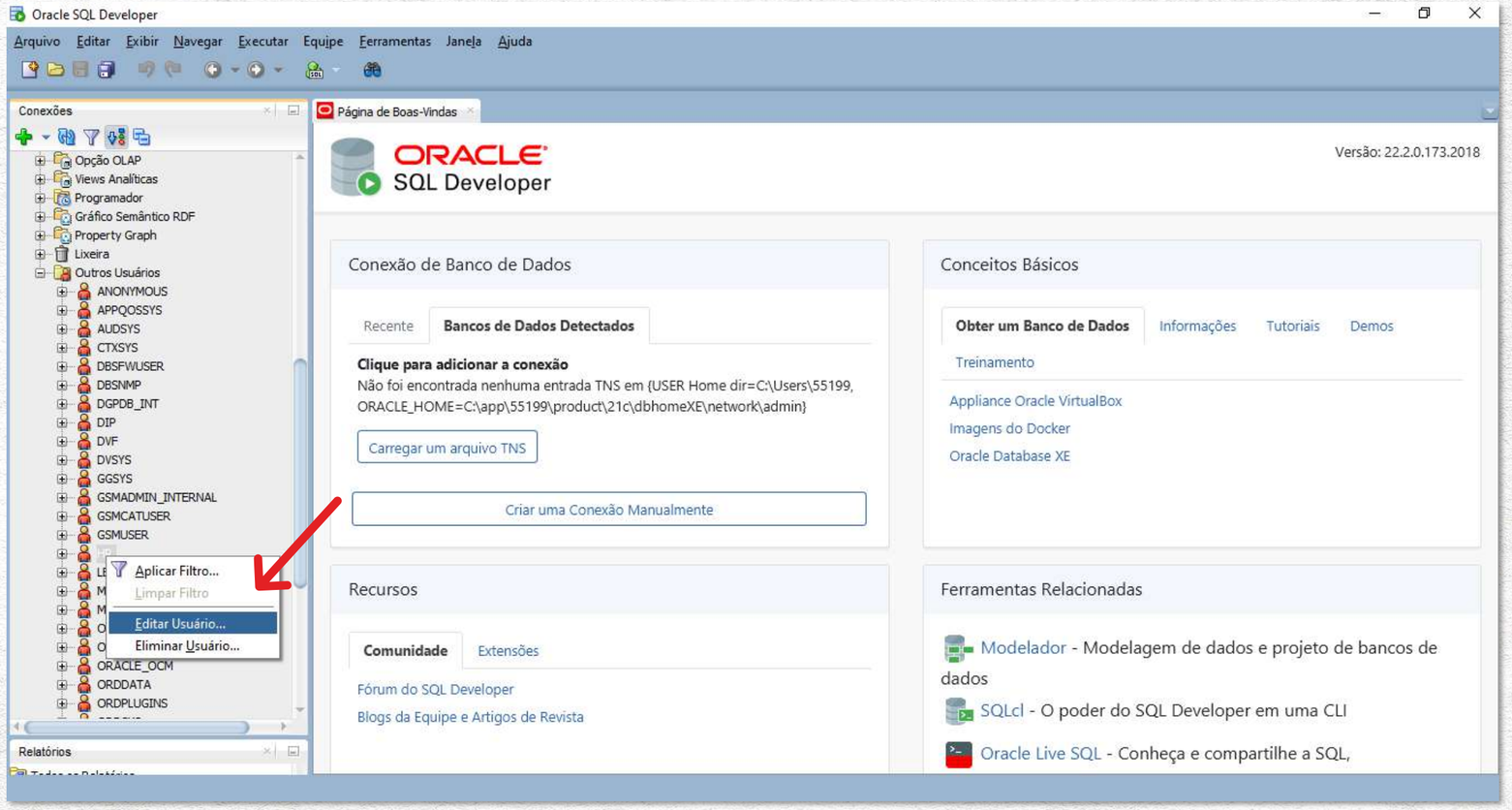
2. Repare que todas as opções da coluna Concedida serão selecionadas.

3. Clique em Aplicar.



Criando o usuário HR

Vá novamente em Outros Usuários, procure o usuário HR, clique com o botão direito em cima dele, em seguida em Editar Usuário...

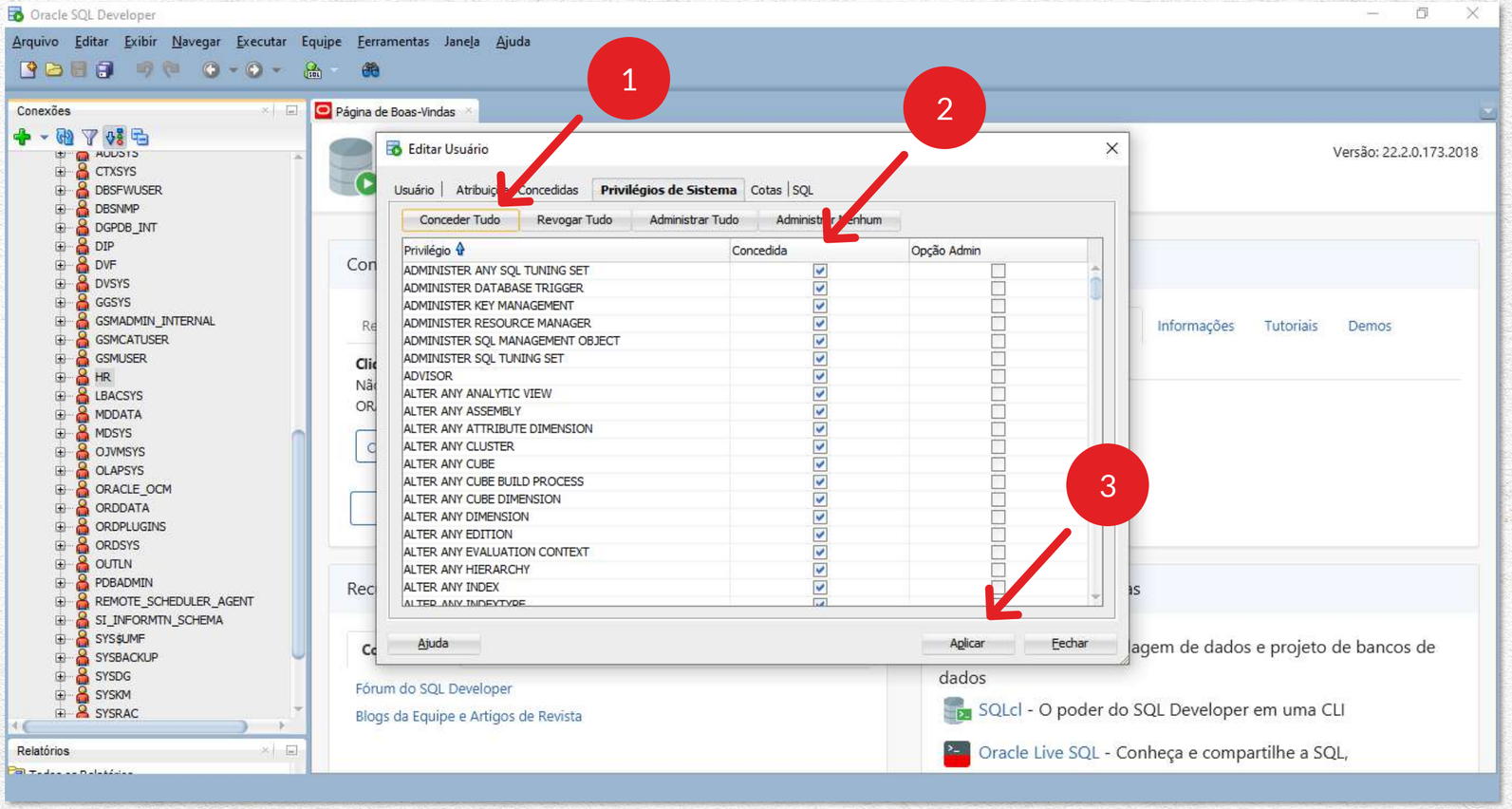


Criando o usuário HR

1. Na nova janela que se abrir, com a aba Atribuições Concedidas selecionada, clique em Conceder Tudo:

2. Repare que todas as opções da coluna Concedida serão selecionadas.

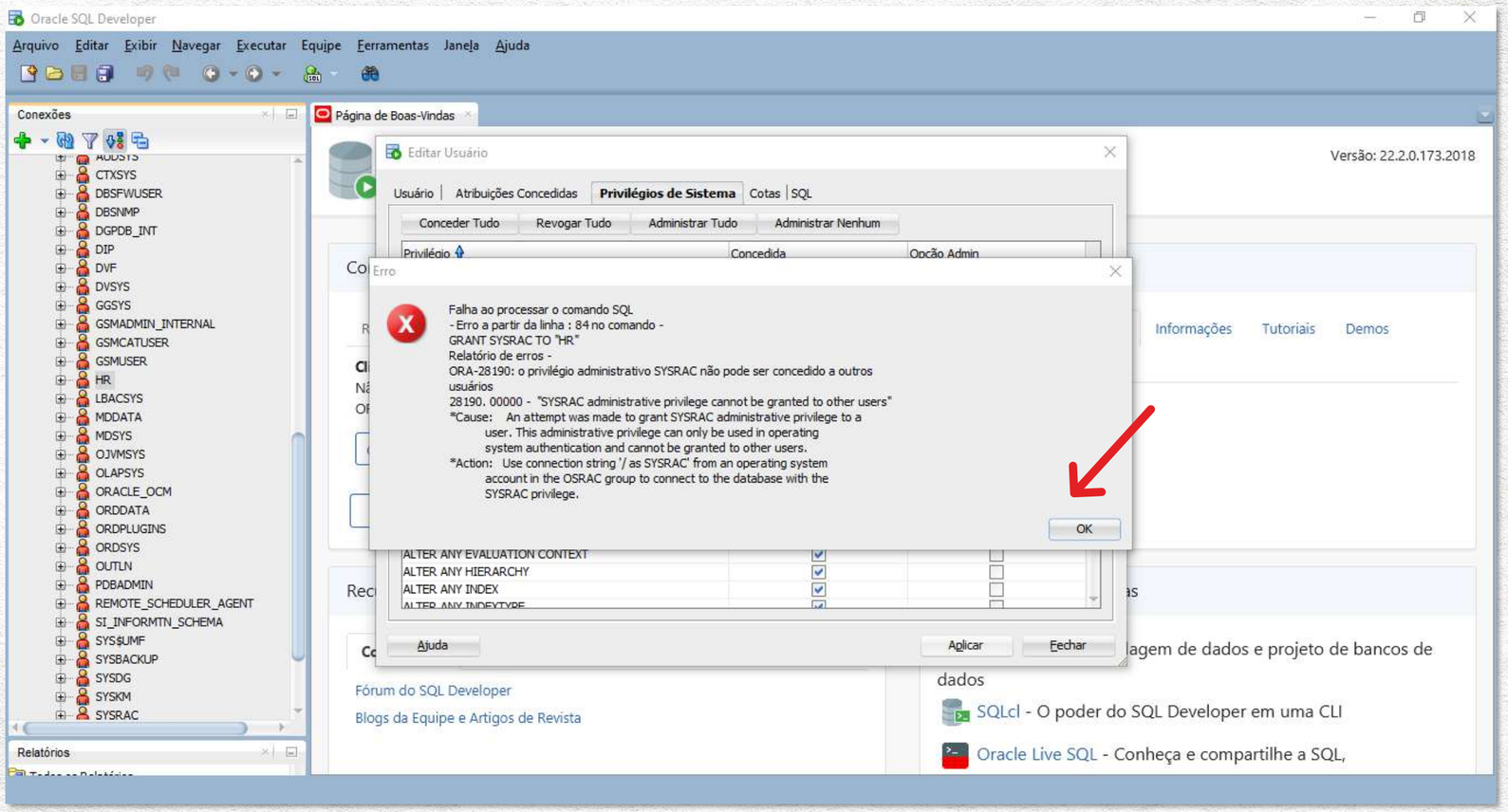
3. Clique em Aplicar.



Criando o usuário HR

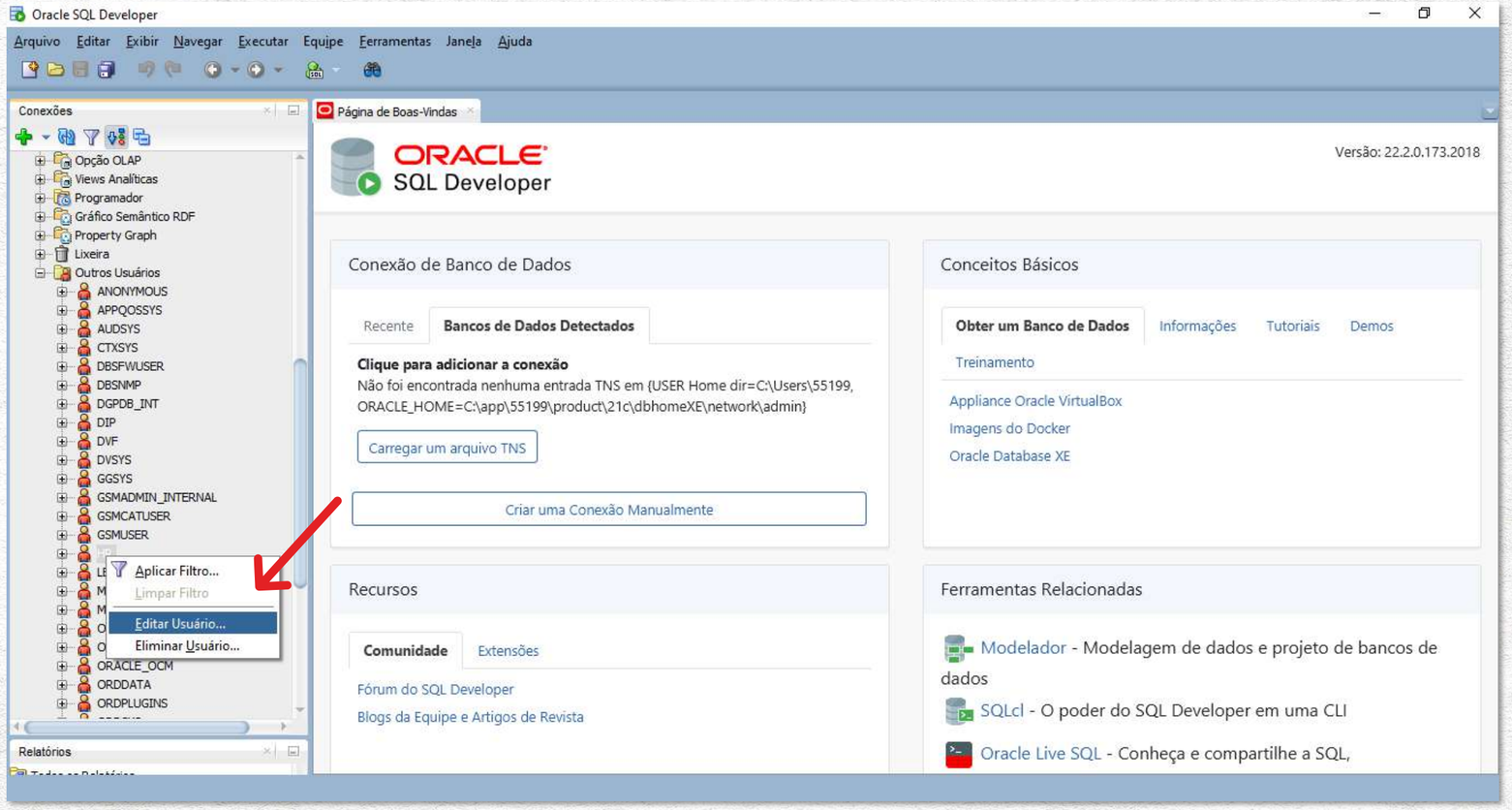
Neste momento, será retornado um erro, mas não se preocupe: este erro acontece porque o Oracle Database entende que não se pode conceder todos os privilégios de sistema a mais um usuário.

Para resolver, vamos clicar em OK...



Criando o usuário HR

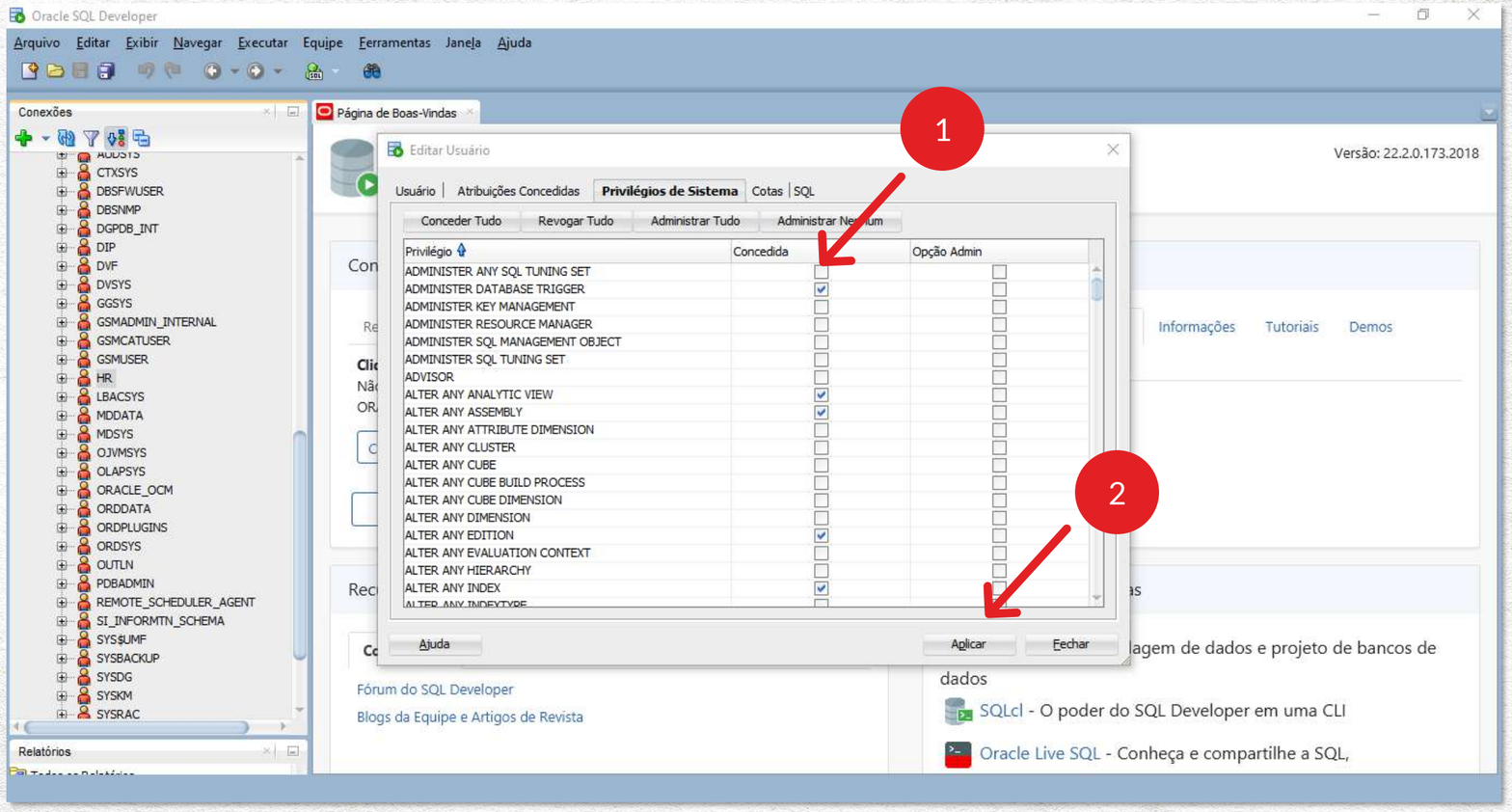
Vá novamente em Outros Usuários, procure o usuário HR, clique com o botão direito em cima dele, em seguida em Editar Usuário...



Criando o usuário HR

1 Na Janela que se abrir, com a aba Privilégios de Sistema selecionada, repare que, agora, apenas algumas opções da coluna Concedida encontram-se selecionadas:

2 Clique em Aplicar.

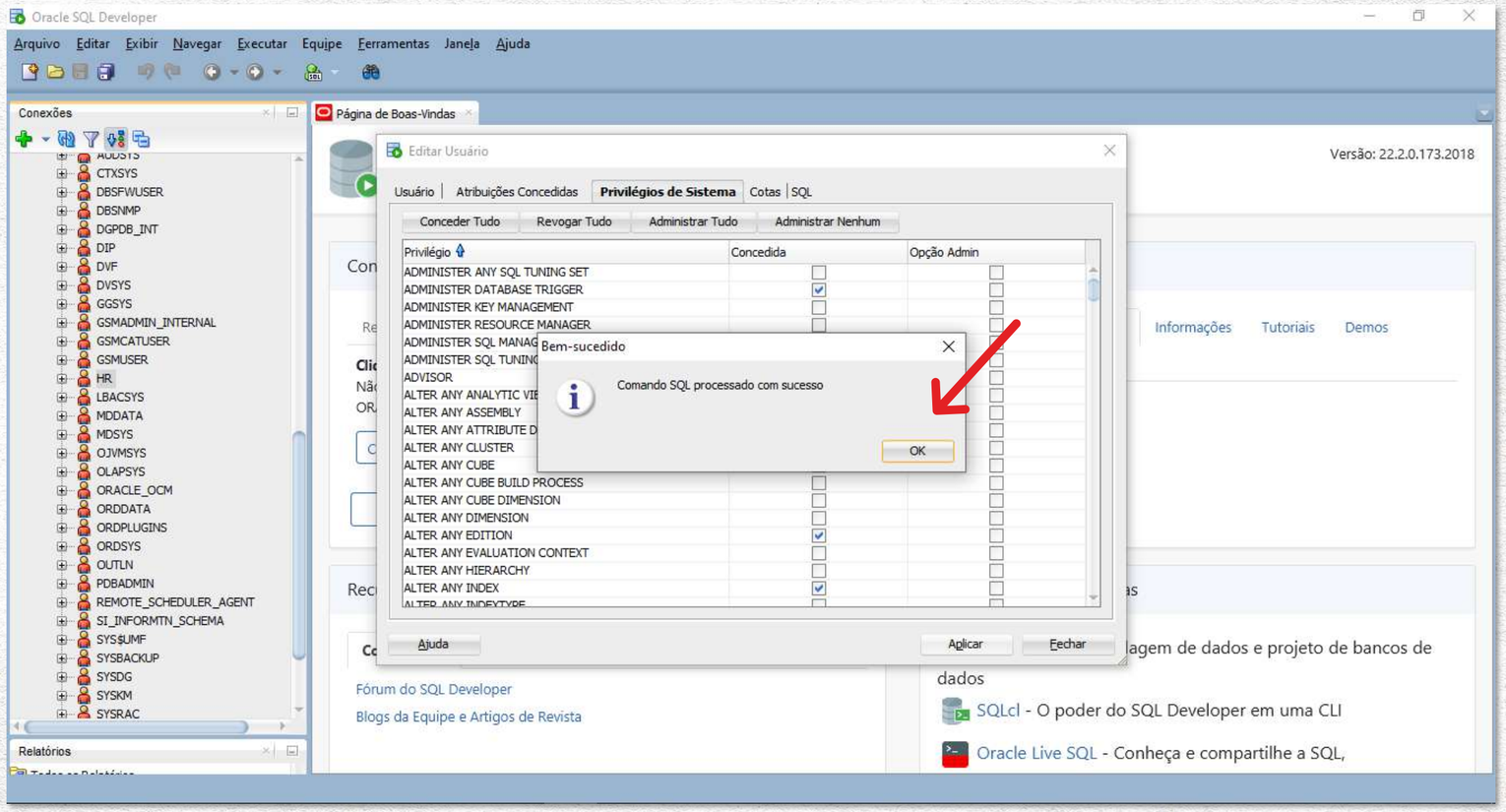


Criando o usuário HR

Pronto!

Já criamos o nosso usuário HR.

O próximo passo será criar uma conexão para o banco de dados que vamos usar no curso: o HR.



Criando uma conexão para o schema HR

Vamos então criar o nosso banco de dados RH dentro do Oracle Database.

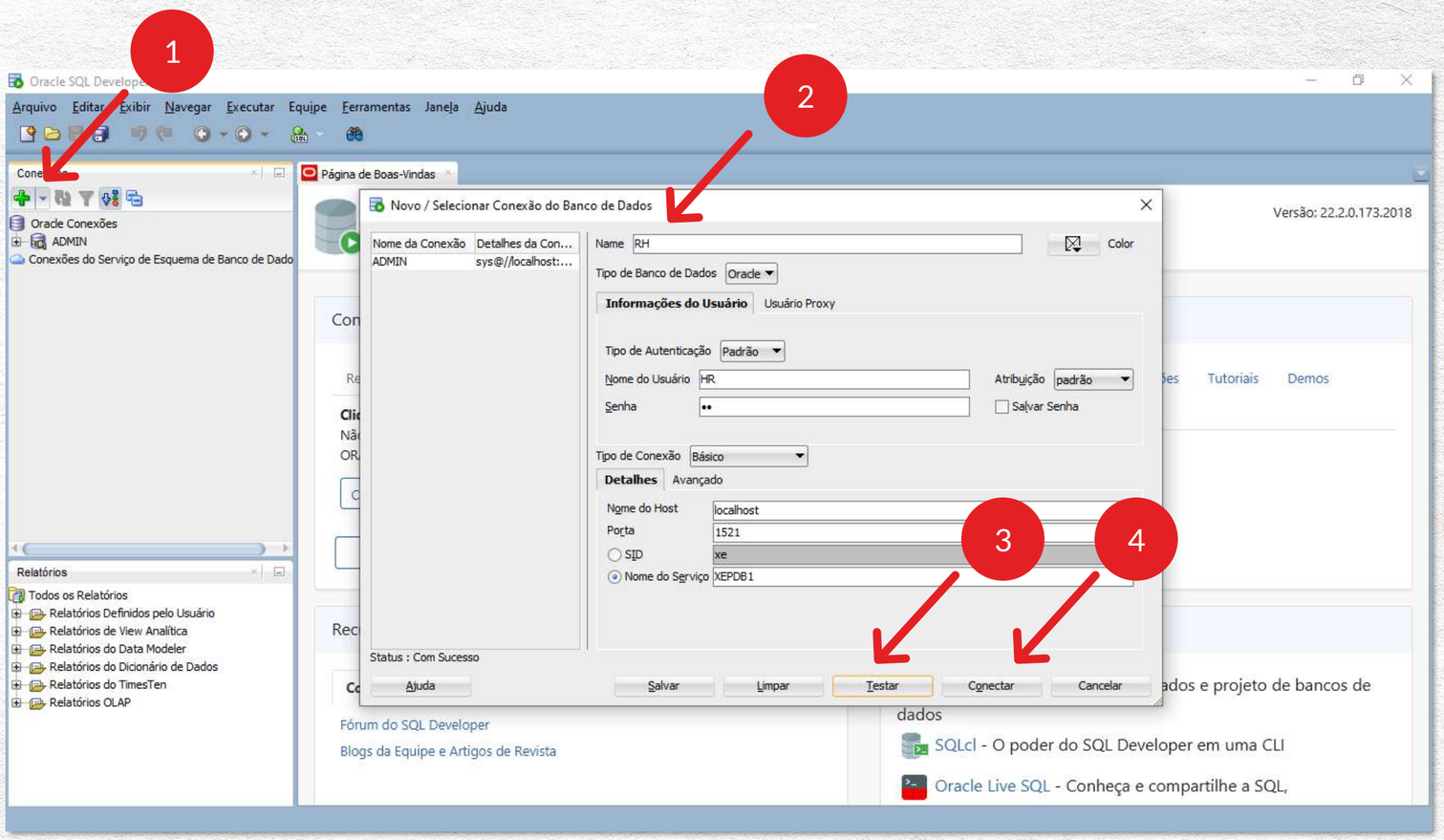
1 Clique na opção ➕ para criar uma nova conexão para o banco de dados.

2 Na janela que se abrir, preencha com os seguintes dados:
- Nome: RH
- Nome do Usuário: HR
- Senha: hr
- Atribuição: Padrão
- Nome do Serviço: XEPDB1

O resto pode deixar o padrão.

3 Clique em Testar para testar a conexão. O status deverá aparecer como **Com sucesso**.

4 Clique em Conectar.

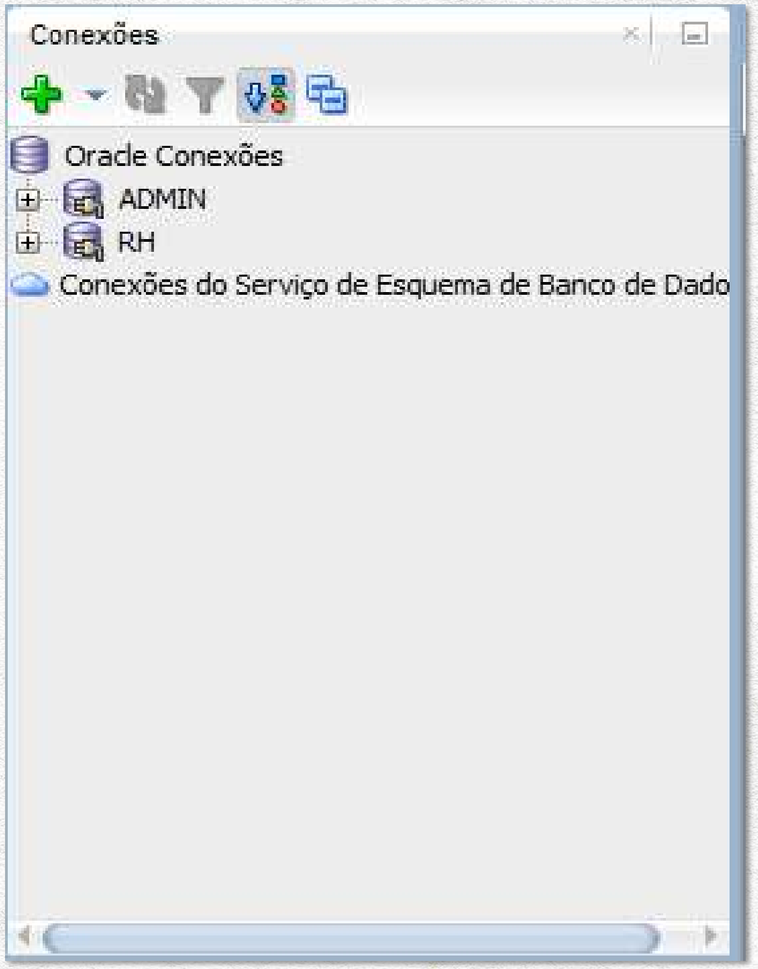


Criando uma conexão para o schema HR

Pronto!

Agora temos a nossa conexão HR.

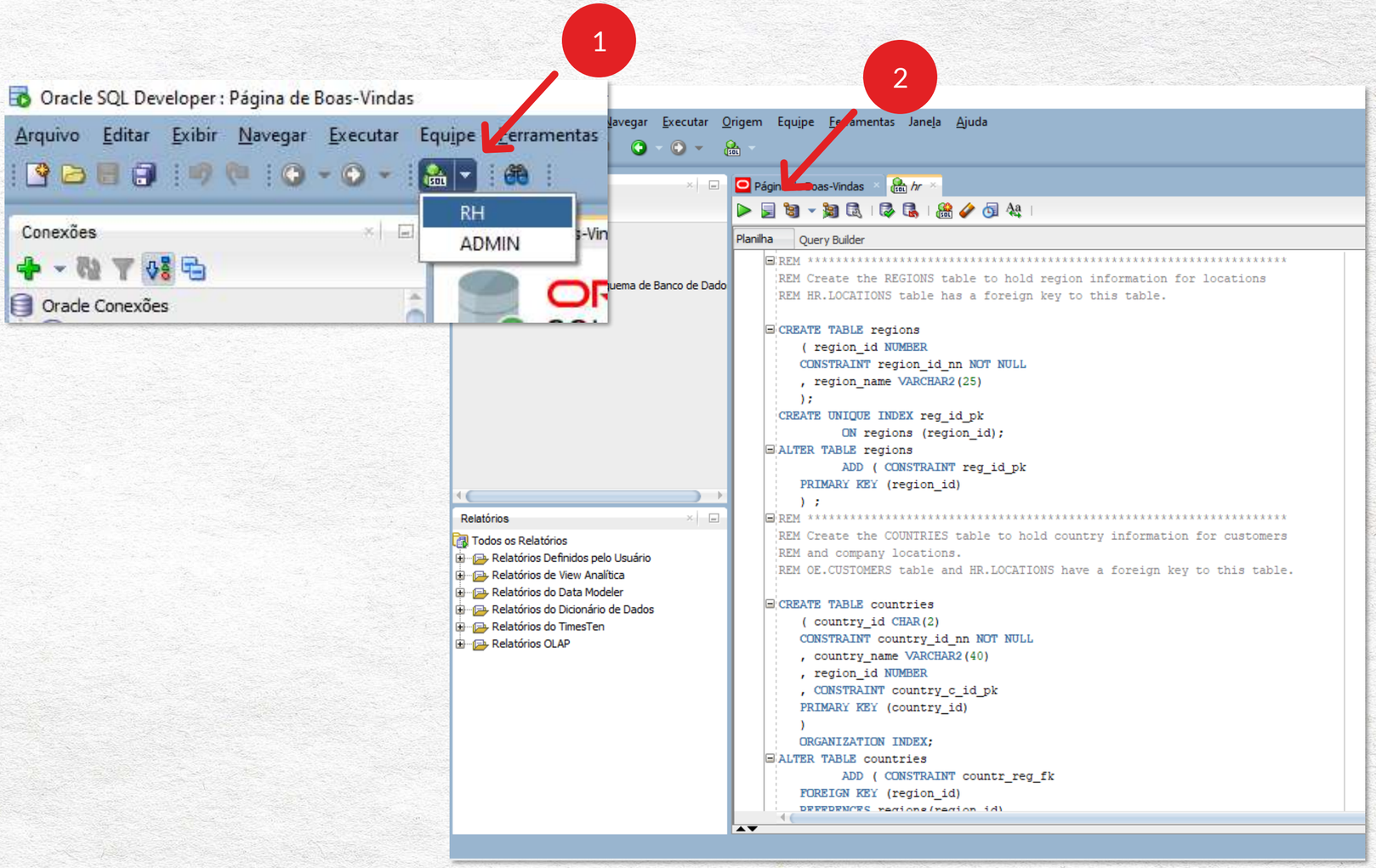
Podemos partir para a criação das tabelas.



Criando as tabelas do usuário HR

Para criar as tabelas do banco de dados HR, o primeiro passo é criar uma janela de query e escolher a conexão que vamos usar para criar as tabelas. No caso, será a HR.

Feito isso, vamos colar o código do arquivo hr_schema disponibilizado na aula na janela de query e depois clicar em executar.

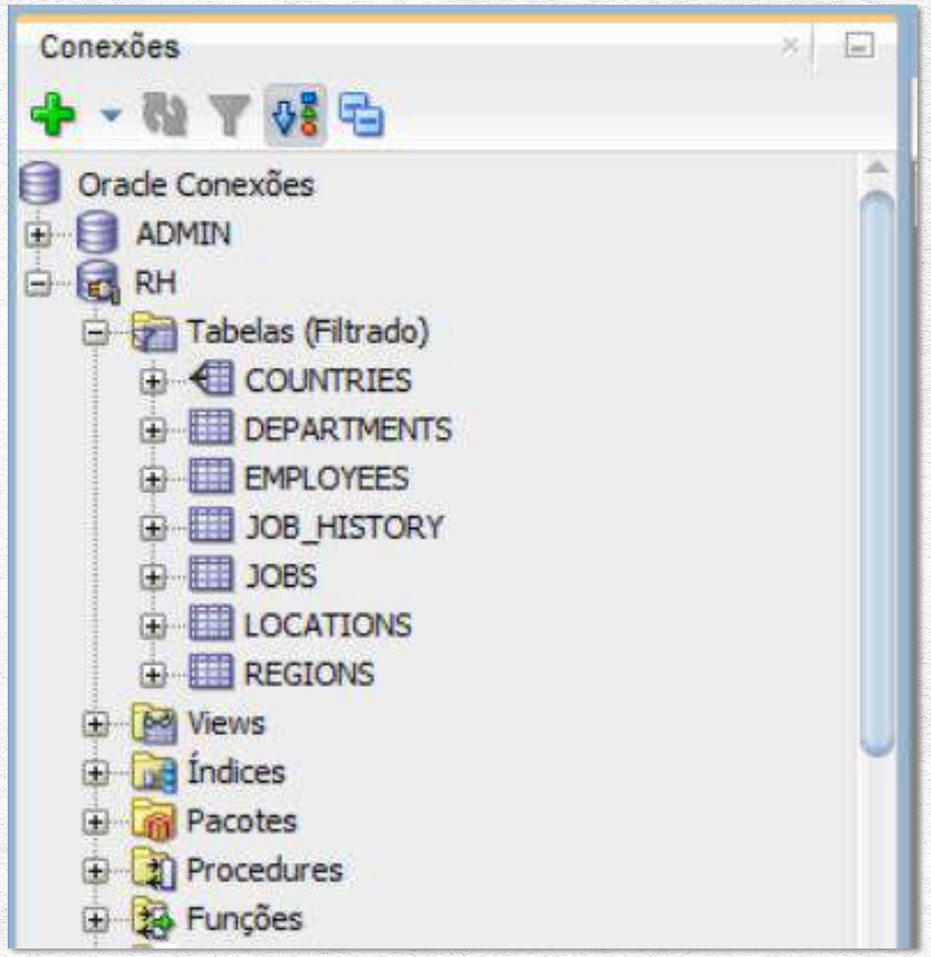


Criando as tabelas do usuário HR

Feito isso, podemos ir até o banco de dados HR e expandir as tabelas.

Veremos que as nossas 7 tabelas foram criadas com sucesso.

Agora, estamos prontos para começar o curso!



MÓDULO 4

INTRODUÇÃO AO SQL

INTRODUÇÃO AO SQL
INTRODUÇÃO AO SQL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 4 INTRODUÇÃO AO SQL

01	INTRODUÇÃO AO SQL	090
02	DOCUMENTAÇÃO ORACLE SQL	096
03	O SCHEMA HR	099
04	EXPLORANDO AS TABELAS DO BANCO HR	100
05	TABELA COUNTRIES	101
06	TABELA DEPARTMENTS	102
07	DESCRIBE/DESC - EXIBINDO A ESTRUTURA DE TABELAS	103
08	SELECT - SELECIONANDO OS DADOS DA TABELA	105
09	FORMAS DE EXECUTAR UM CÓDIGO	112
10	COMENTÁRIOS	113
11	ALIAS DE COLUNAS	114
12	ALIAS DE TABELAS	117
13	OPERADOR DE CONCATENAÇÃO ||	118
14	RESOLVENDO PROBLEMA DO ' DENTRO DE UMA STRING	119
15	CÁLCULOS SIMPLES NO SQL	120
16	TABELA DUAL	122
17	SELECT DISTINCT – SELECIONANDO VALORES DISTINTOS	123
18	SELECIONANDO VALORES DISTINTOS (MAIS DE UMA COLUNA)	125
19	RESUMO DO MÓDULO	126
20	EXERCÍCIOS	138
21	GABARITO EXPLICATIVO	142

Introdução ao SQL

O SQL é a linguagem padrão para se trabalhar com bancos de dados relacionais.

Ou seja, conseguimos criar, ler, atualizar, excluir, enfim, manipular dados de forma geral dentro de bancos de dados e tabelas através de comandos em SQL.

Utilizamos o SQL dentro de programas específicos, chamados de SGBDRs.

Exemplos de SGBDRs são:

- SQL Server
- MySQL
- PostgreSQL
- Oracle (SGBD que vamos trabalhar neste momento)

Introdução ao SQL

A linguagem SQL é composta por uma série de grupos de comandos, que inclusive já até introduzimos na seção de dicionário aqui da apostila, no módulo 1. Esses grupos de comando são os seguintes:

DDL

Data Definition Language

CREATE
Cria uma nova tabela, view ou outro objeto dentro do banco de dados.

ALTER
Modifica um objeto dentro do banco de dados (tabela, view, etc).

DROP
Exclui um objeto dentro do banco de dados (tabela, view, etc).

DML

Data Manipulation Language

INSERT
Adiciona uma nova linha em uma tabela.

UPDATE
Atualiza os valores das linhas de uma tabela.

DELETE
Exclui linhas de uma tabela.

TCL

Transaction Control Language

COMMIT
Salva as alterações no banco.

ROLLBACK
Desfaz alterações no banco.

DCL

Data Control Language

GRANT
Dá privilégios a um usuário.

REVOKE
Retira privilégios de um usuário.

Introdução ao SQL

A linguagem SQL é composta por uma série de grupos de comandos, que inclusive já até introduzimos na seção de dicionário aqui da apostila, no módulo 1. Esses grupos de comando são os seguintes:

DDL

Data Definition Language

CREATE
Cria uma nova tabela, view ou outro objeto dentro do banco de dados.

ALTER
Modifica um objeto dentro do banco de dados (tabela, view, etc).

DROP
Exclui um objeto dentro do banco de dados (tabela, view, etc).

DDL – Data Definition Language

Linguagem de Definição de Dados

São os comandos que interagem com **objetos** (Tabelas, Views, Functions, Procedures, etc.) dentro de um banco de dados.

Introdução ao SQL

A linguagem SQL é composta por uma série de grupos de comandos, que inclusive já até introduzimos na seção de dicionário aqui da apostila, no módulo 1. Esses grupos de comando são os seguintes:

DML

Data Manipulation Language

INSERT
Adiciona uma nova linha em uma tabela.

UPDATE
Atualiza os valores das linhas de uma tabela.

DELETE
Exclui linhas de uma tabela.

DML – Data Manipulation Language

Linguagem de Manipulação de Dados

São os comandos que interagem com **dados** dentro de tabelas.

Obs.: O comando **SELECT** pode se encaixar neste grupo de comandos.

Introdução ao SQL

A linguagem SQL é composta por uma série de grupos de comandos, que inclusive já até introduzimos na seção de dicionário aqui da apostila, no módulo 1. Esses grupos de comando são os seguintes:

TCL

Transaction
Control Language

COMMIT
Salva as alterações no banco.

ROLLBACK
Desfaz alterações no banco.

TCL – Transaction Control Language

Linguagem de Controle de Transações

São os comandos para controle de transações no SQL.

Os comandos DML (INSERT, UPDATE e DELETE) são exemplos de transações.

Introdução ao SQL

A linguagem SQL é composta por uma série de grupos de comandos, que inclusive já até introduzimos na seção de dicionário aqui da apostila, no módulo 1. Esses grupos de comando são os seguintes:

DCL

Data Control Language

GRANT
Dá privilégios a um usuário.

REVOKE
Retira privilégios de um usuário.

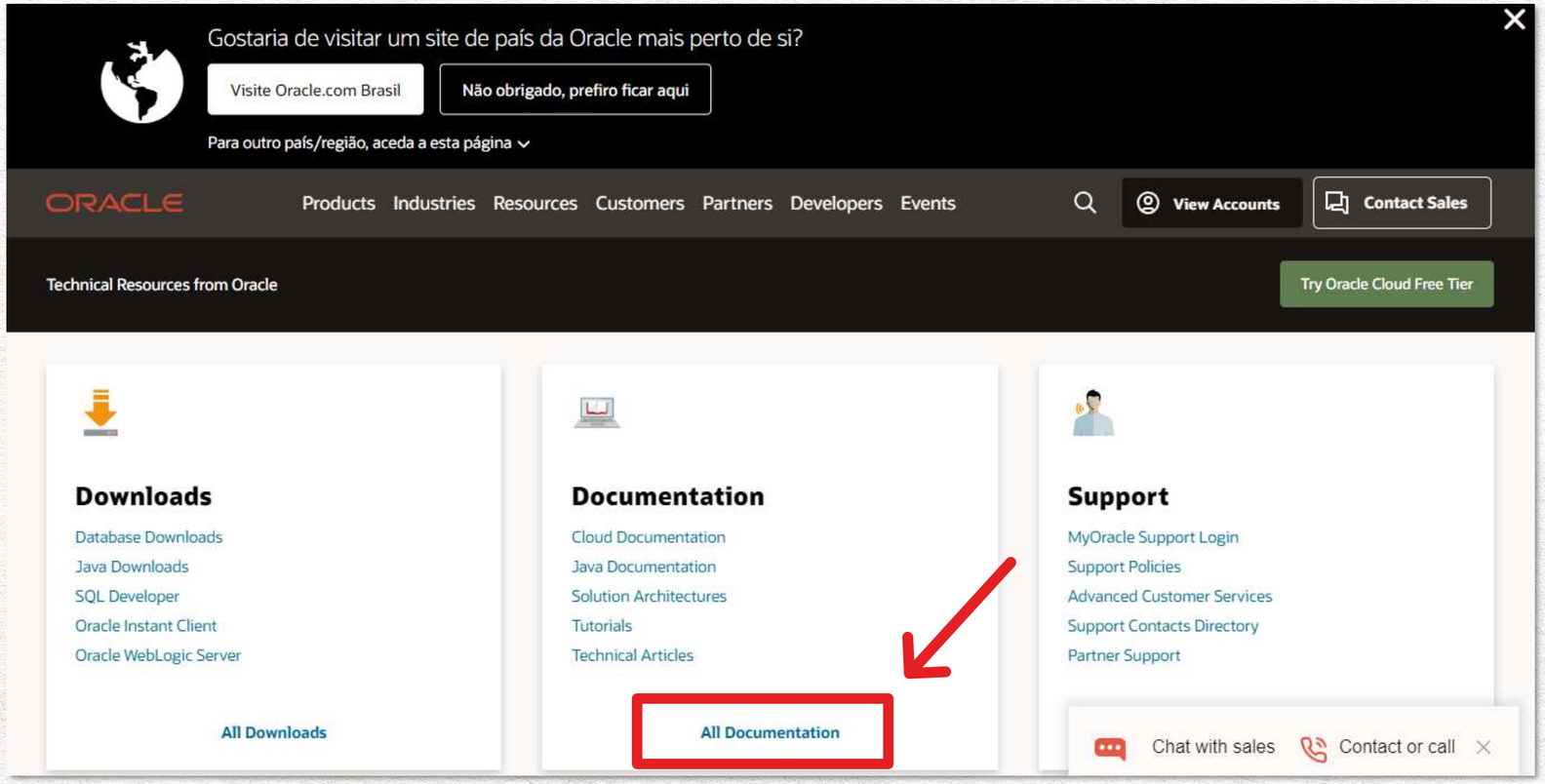
DCL – Data Control Language

Linguagem de Controle de Dados

São os comandos para controlar a parte de segurança do banco de dados. Basicamente, diz quem terá permissão para quê, e quem não terá permissão a quê.

Documentação Oracle SQL

É possível encontrar uma documentação completa do SQL Oracle neste site: https://otn.oracle.com

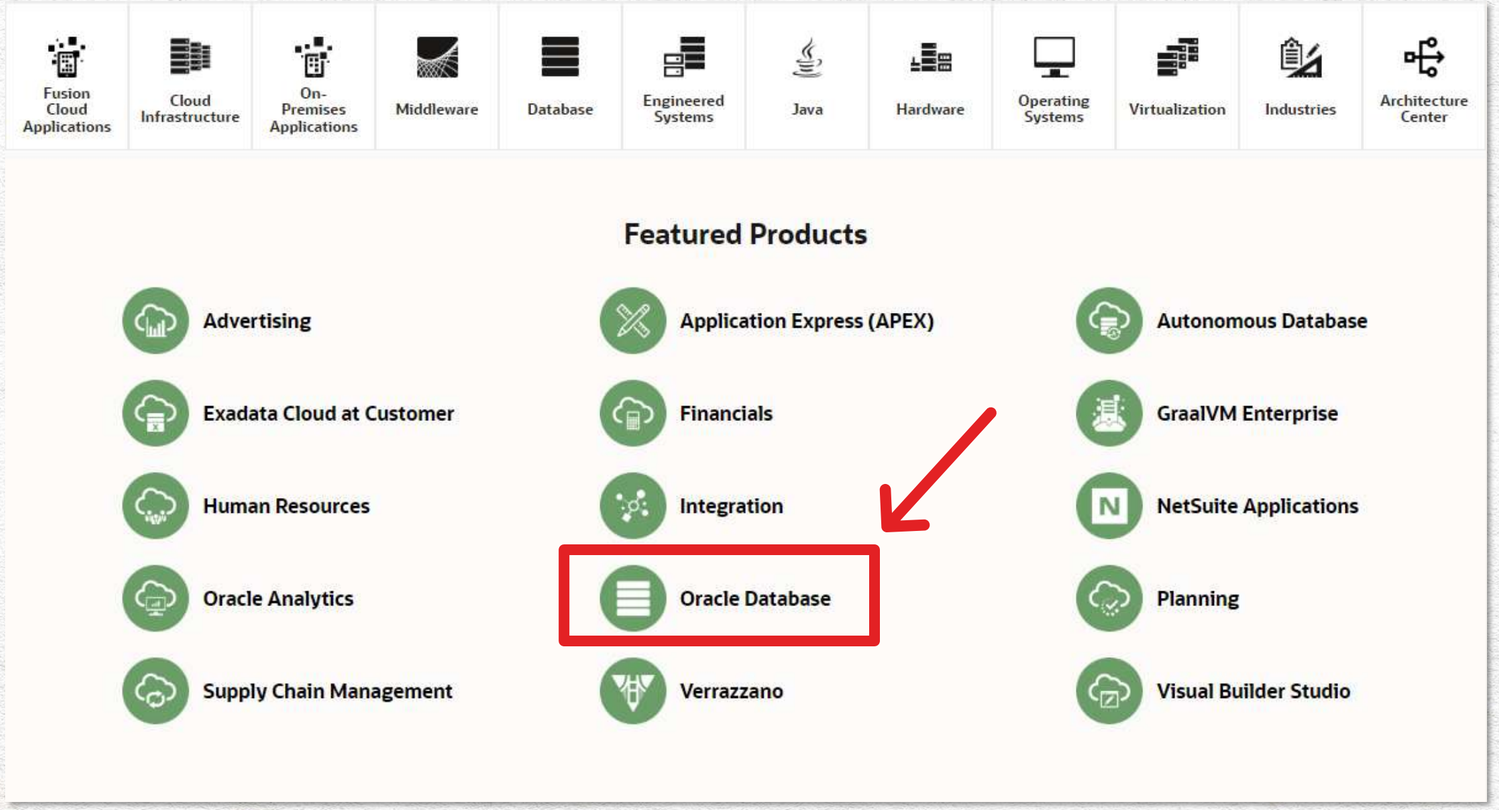


Índice Geral

Índice deste Módulo

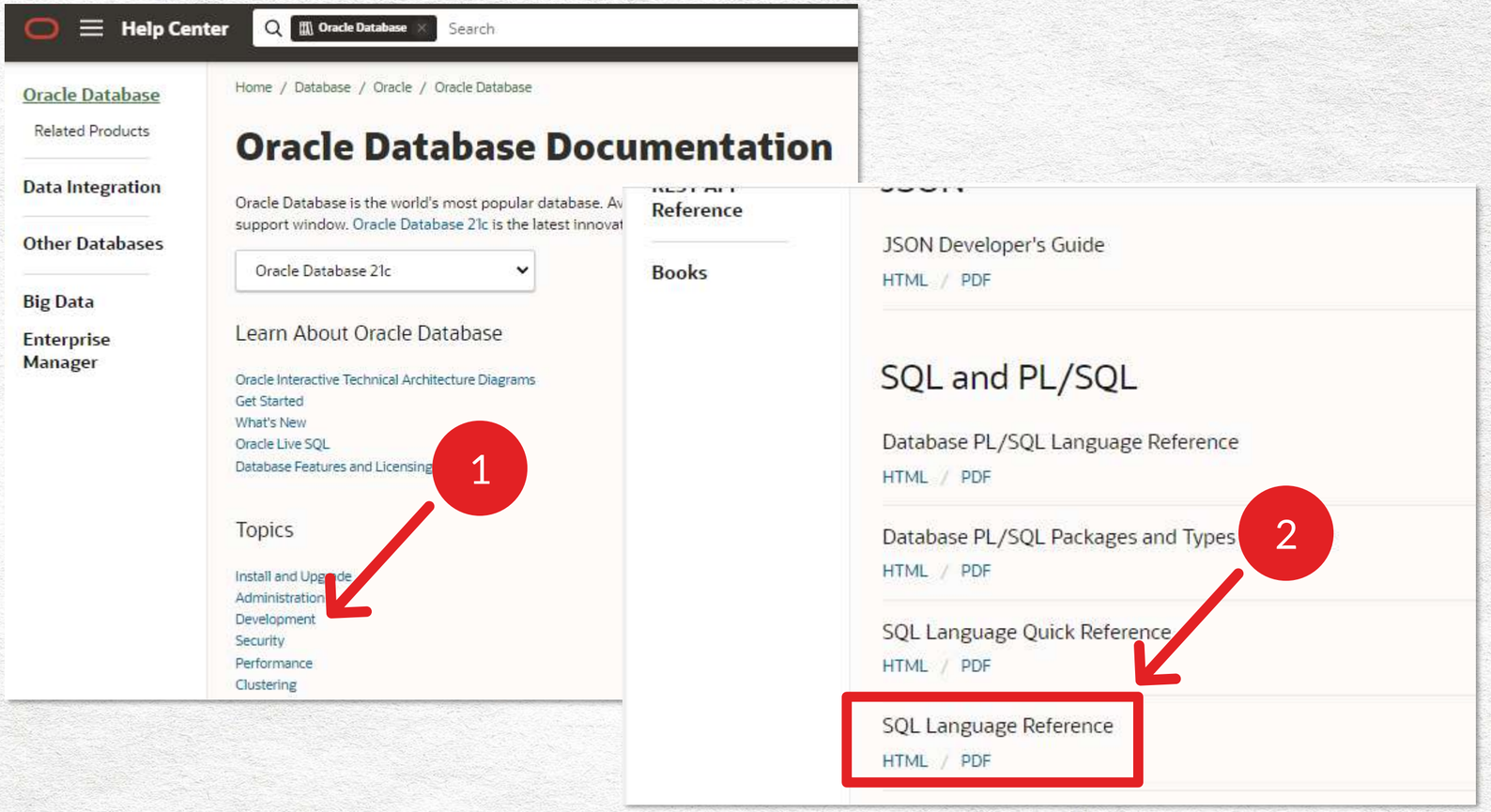
Documentação Oracle SQL

Clique em Oracle Database para acessar toda a documentação disponível.



Documentação Oracle SQL

Vá até a opção Development (como mostrado em 1) e depois em SQL Language Reference (em 2) para acessar um arquivo com um manual de referência da linguagem SQL no Oracle.



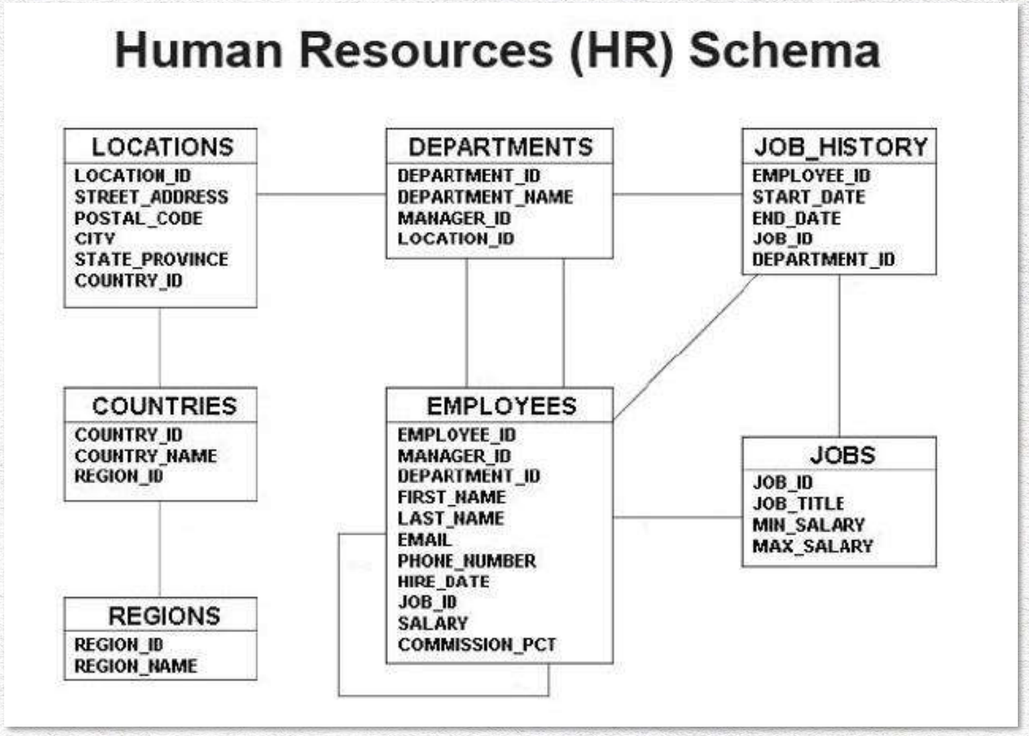
O schema HR

A Oracle disponibiliza um banco de dados (schema) para fins educacionais. O HR é um banco com informações de uma empresa de Recursos Humanos (Human Resources – HR).

Este é um banco de dados relacional, onde as diferentes tabelas possuem alguma relação entre si.

O banco de dados HR possui 7 tabelas, chamadas:

- COUNTRIES
- DEPARTMENTS
- EMPLOYEES
- JOB_HISTORY
- JOBS
- LOCATIONS
- REGIONS



As linhas que ligam as tabelas indicam a relação que elas têm entre si. Por exemplo, a tabela **EMPLOYEES** (funcionários) possui uma relação entre as tabelas **DEPARTMENTS**, **JOB_HISTORY**, **JOBS** e **com ela mesma**.

Explorando as tabelas do banco HR

O banco de dados HR possui informações sobre os Recursos Humanos de uma empresa, e é composto por 7 tabelas:

1. COUNTRIES
2. DEPARTMENTS
3. EMPLOYEES
4. JOB_HISTORY
5. JOBS
6. LOCATIONS
7. REGIONS

Se a gente clicar em cima do nome da tabela, será aberta uma nova janela onde conseguimos ver mais informações sobre essa tabela, além dos dados armazenados. Vamos fazer este exercício.

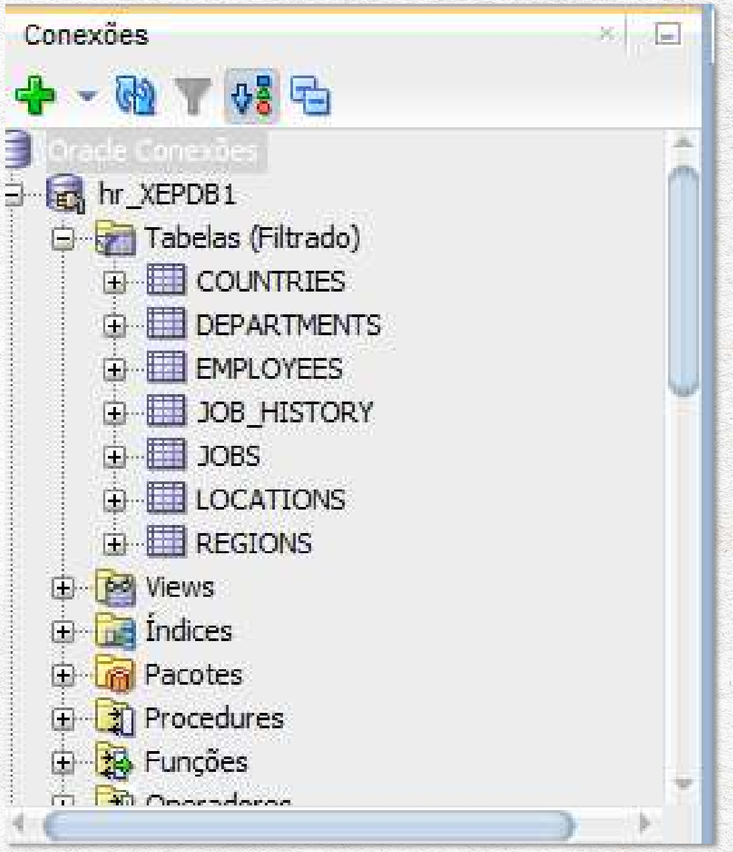
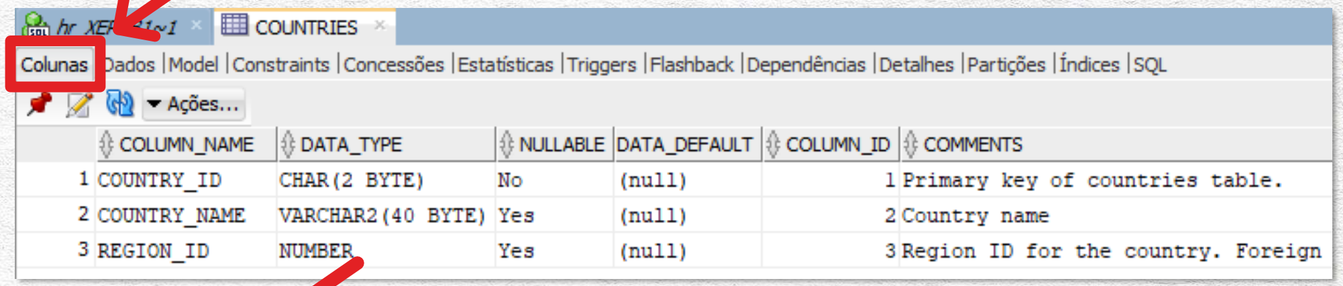
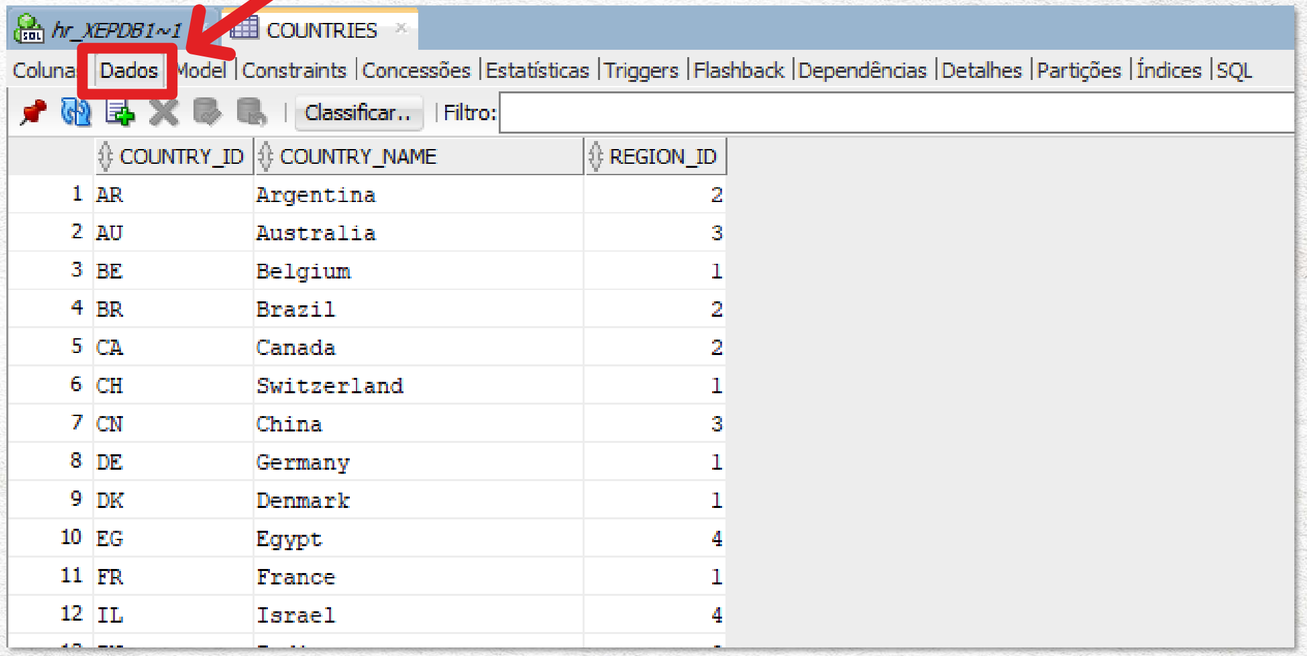


Tabela COUNTRIES

Ao clicar na tabela COUNTRIES, é aberta uma janela com a informação dos nomes das **Colunas**, além de uma outra janela chamada **Dados**, na qual é possível visualizar os dados da tabela.





Esta tabela armazena informações de países e possui 3 colunas:

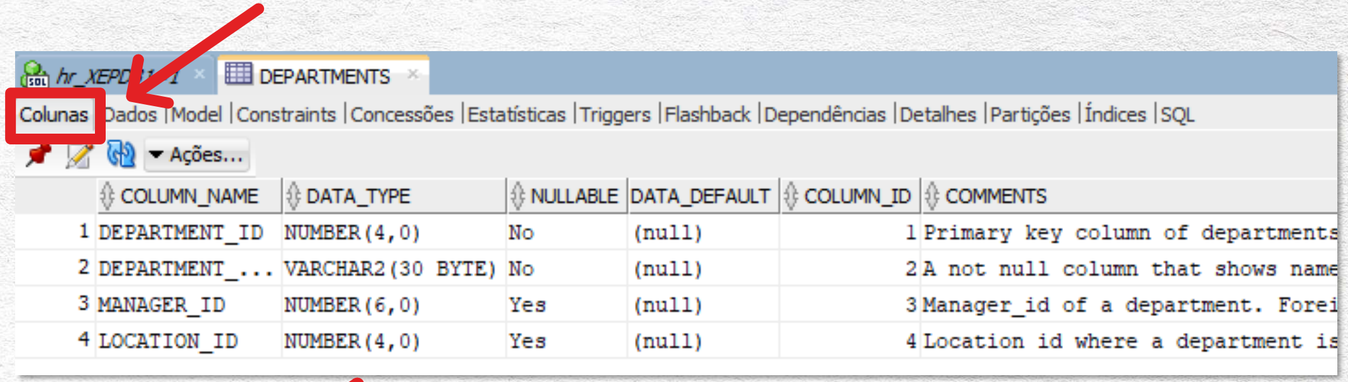
- COUNTRY_ID
- COUNTRY_NAME
- REGION_ID

Tabela DEPARTMENTS

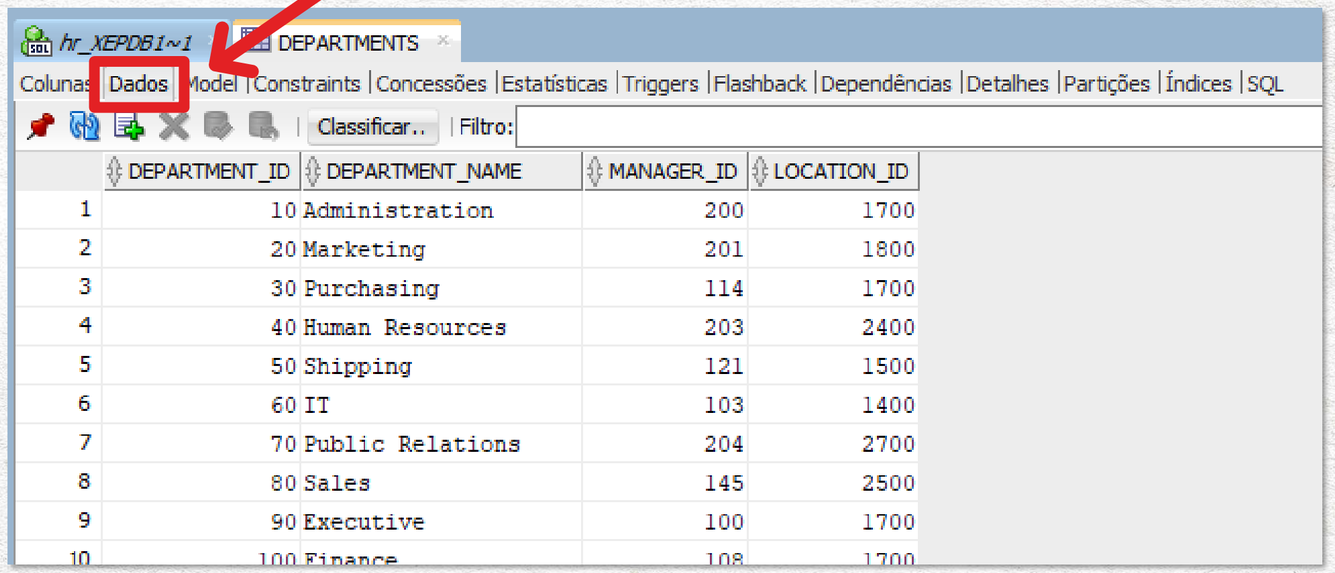
A tabela DEPARTMENTS armazena as informações dos departamentos da empresa, e possui 4 colunas:

- DEPARTMENT_ID
- DEPARTMENT_NAME
- MANAGER_ID
- LOCATION_ID

Fica de exercício para você explorar as demais tabelas do banco de dados.



	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	DEPARTMENT_ID	NUMBER(4,0)	No	(null)	1	Primary key column of departments
2	DEPARTMENT_...	VARCHAR2(30 BYTE)	No	(null)	2	A not null column that shows name
3	MANAGER_ID	NUMBER(6,0)	Yes	(null)	3	Manager_id of a department. Forei
4	LOCATION_ID	NUMBER(4,0)	Yes	(null)	4	Location id where a department is



	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700
4	40	Human Resources	203	2400
5	50	Shipping	121	1500
6	60	IT	103	1400
7	70	Public Relations	204	2700
8	80	Sales	145	2500
9	90	Executive	100	1700
10	100	Finance	108	1700

DESCRIBE/DESC – Exibindo a estrutura de tabelas

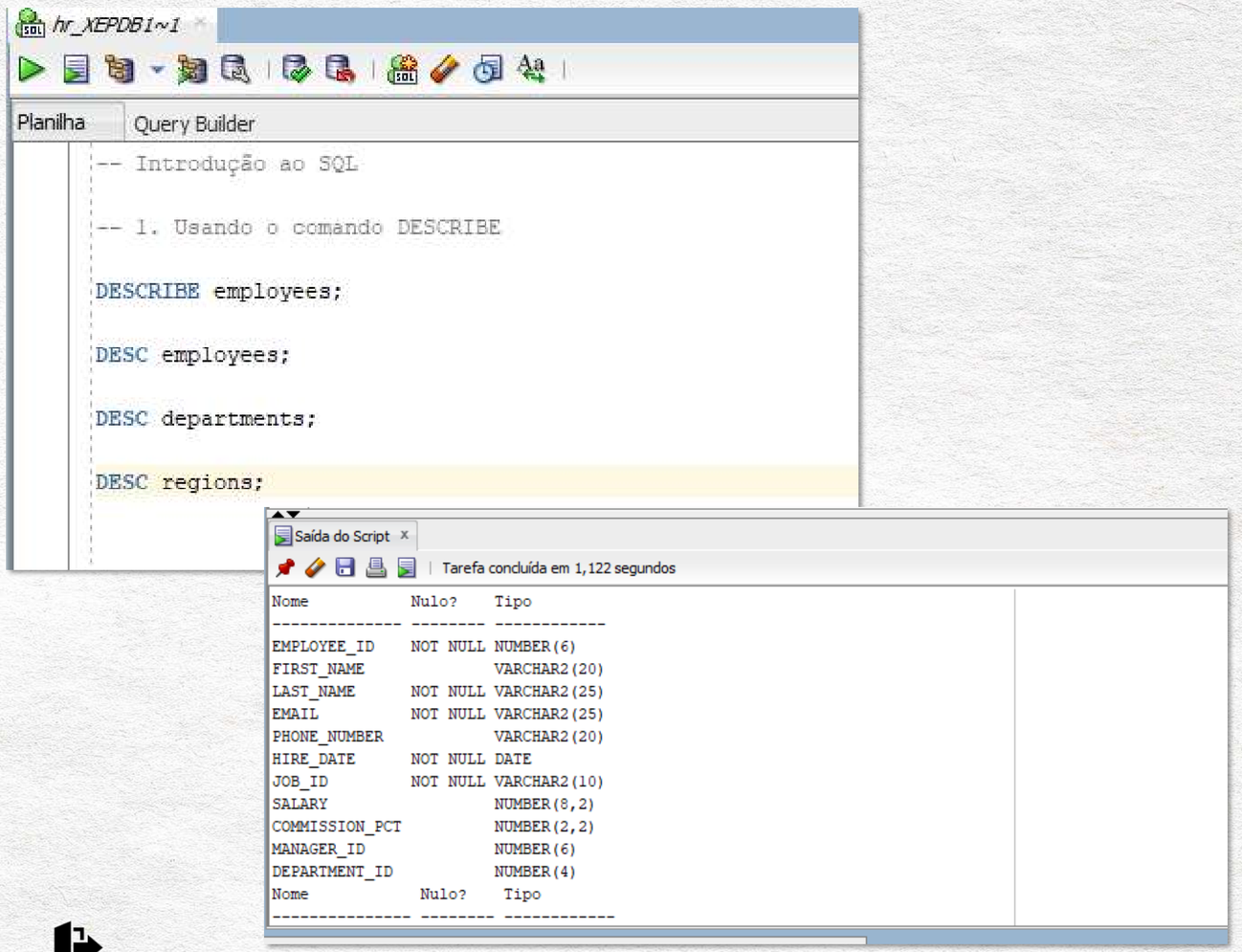
Vamos agora começar com os primeiros comandos em SQL dentro do Oracle.

Podemos usar o comando DESCRIBE (ou simplesmente DESC) para visualizar a estrutura das tabelas do nosso banco de dados:

- Nomes das colunas
- Constraints (Restrições)
- Tipos dos dados das colunas

Na imagem ao lado, temos a aplicação do comando.

Para executá-lo, basta selecionar a linha do código e clicar em , no canto superior direito da janela de consulta.

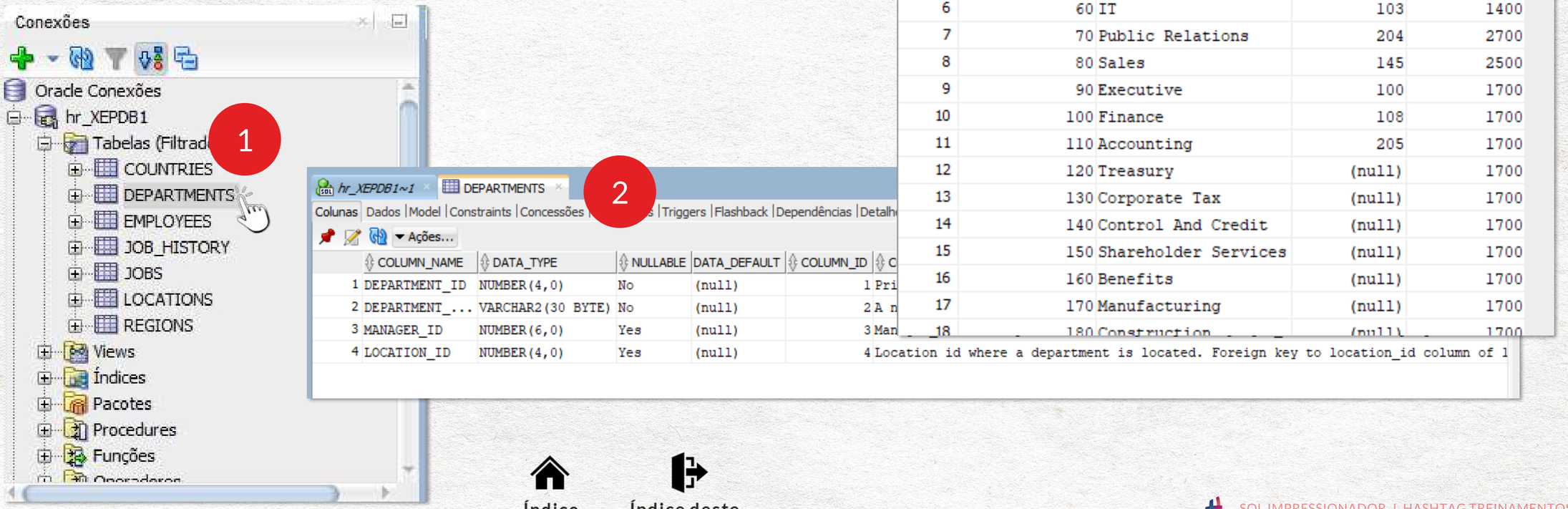


DESCRIBE/DESC – Exibindo a estrutura de tabelas

Uma outra forma de visualizar a estrutura das tabelas é clicando no nome da tabela que queremos (figura 1).

Automaticamente será aberta uma guia com as informações sobre aquela tabela, como pode ser visto na figura 2.

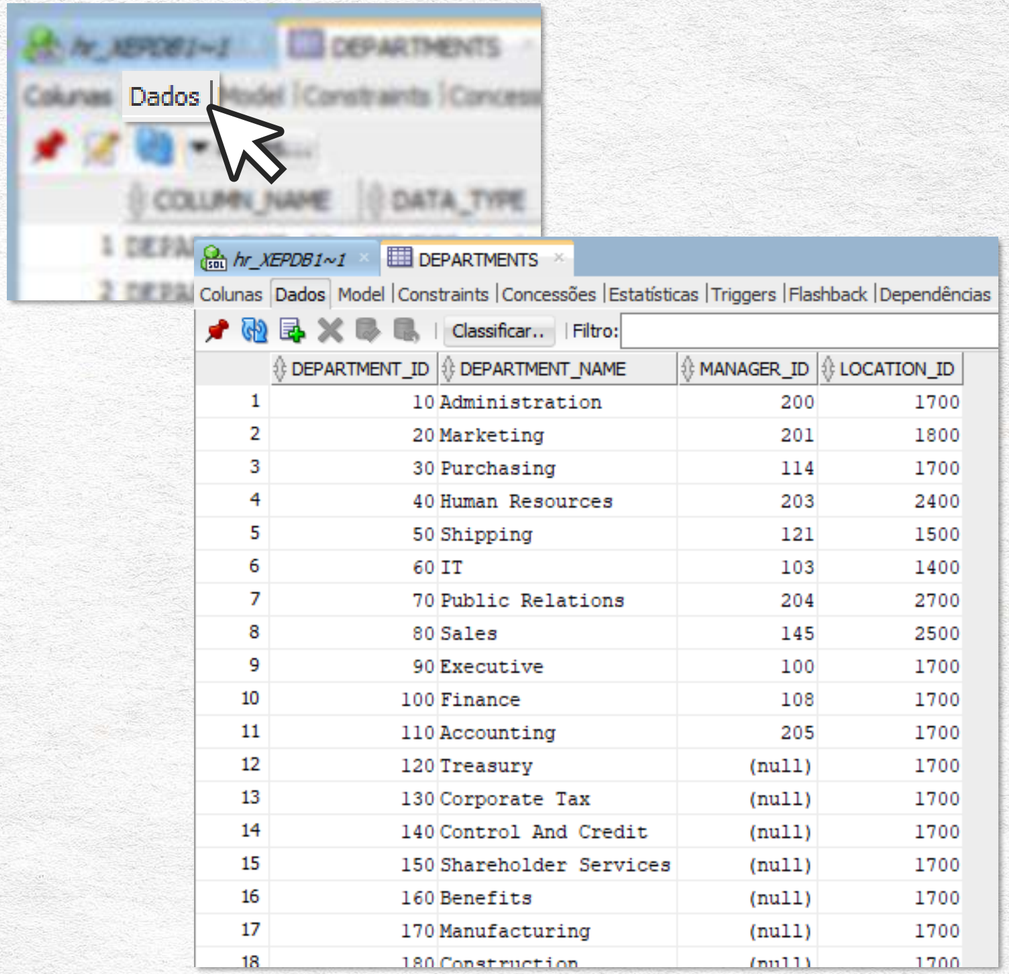
Para visualizar seus dados, podemos clicar na opção Dados (figura 3).



SELECT – Selecionando os dados da tabela

Vimos anteriormente que é possível visualizar os dados de uma tabela simplesmente clicando no nome dela e, automaticamente, na nova janela que se abrir, ao clicar em **DADOS**, são mostradas todas as linhas e colunas daquela tabela.

Podemos também usar um comando em SQL para fazer essa seleção. O comando é o **SELECT**. O comando **SELECT** pode ser usado de duas maneiras. A seguir, vamos ver cada uma delas.



	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700
4	40	Human Resources	203	2400
5	50	Shipping	121	1500
6	60	IT	103	1400
7	70	Public Relations	204	2700
8	80	Sales	145	2500
9	90	Executive	100	1700
10	100	Finance	108	1700
11	110	Accounting	205	1700
12	120	Treasury	(null)	1700
13	130	Corporate Tax	(null)	1700
14	140	Control And Credit	(null)	1700
15	150	Shareholder Services	(null)	1700
16	160	Benefits	(null)	1700
17	170	Manufacturing	(null)	1700
18	180	Construction	(null)	1700

SELECT – Selecionando os dados da tabela

1 Selecionando **todas as colunas** da tabela

```
SELECT *
FROM tabela;
```

A **primeira maneira** é selecionar todas as colunas de uma determinada tabela.

Para isso, usamos o comando **SELECT**, seguido do caractere * (asterisco). Este asterisco significa que queremos exibir todas as colunas.

A seguir, após o comando **FROM**, informamos o nome da tabela de onde queremos visualizar aquelas colunas.

SELECT – Selecionando os dados da tabela

2 Selecionando **colunas específicas** da tabela

```
SELECT
      coluna1,
      coluna2,
      coluna3
FROM tabela;
```

A **segunda maneira** é selecionar apenas algumas colunas da tabela, dado que nem sempre queremos visualizar todas.

Neste caso, usamos o comando **SELECT**, seguido de cada uma das colunas da tabela que queremos visualizar.

A seguir, após o comando **FROM**, informamos o nome da tabela de onde queremos visualizar aquelas colunas.

SELECT – Selecionando os dados da tabela

Em resumo, temos o seguinte:

SELECT identifica as colunas ou expressões a serem exibidas. O * (asterisco) significa que queremos visualizar todas as colunas da tabela.

FROM identifica as tabelas que contêm as colunas selecionadas no SELECT.

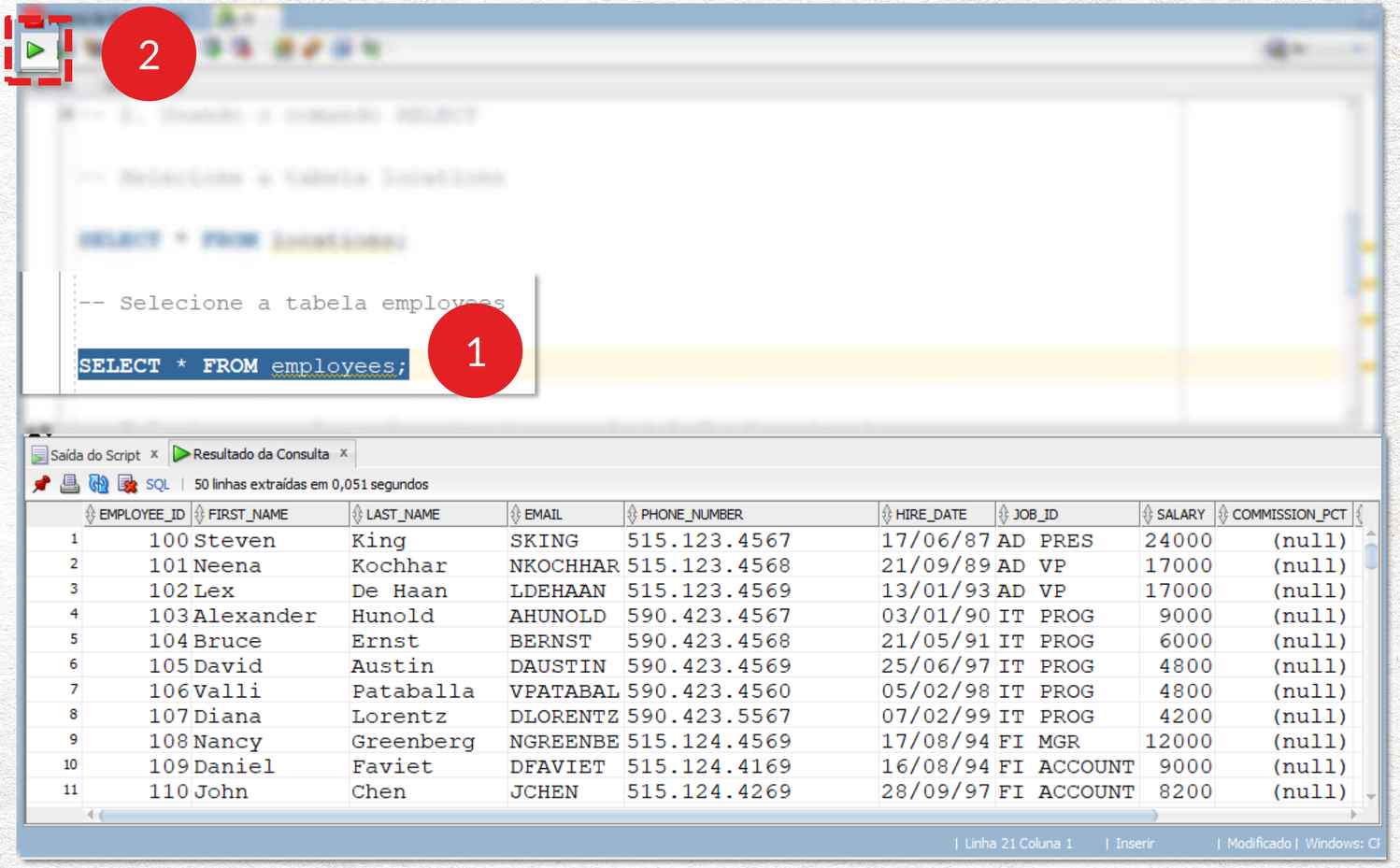
SELECT – Selecionando os dados da tabela

Ao lado, temos uma aplicação do comando SELECT para selecionar a tabela de EMPLOYEES.

Para executar a linha, posicione o cursor no final dela (após o ponto e vírgula, conforme mostrado em 1) e depois clique em ▶ (conforme mostrado em 2).

Logo abaixo será retornada a tabela resultante.

Observe que todas as colunas da tabela foram retornadas, pois usamos o comando SELECT *. Lembrando que o * significa que queremos retornar **todas as colunas**.

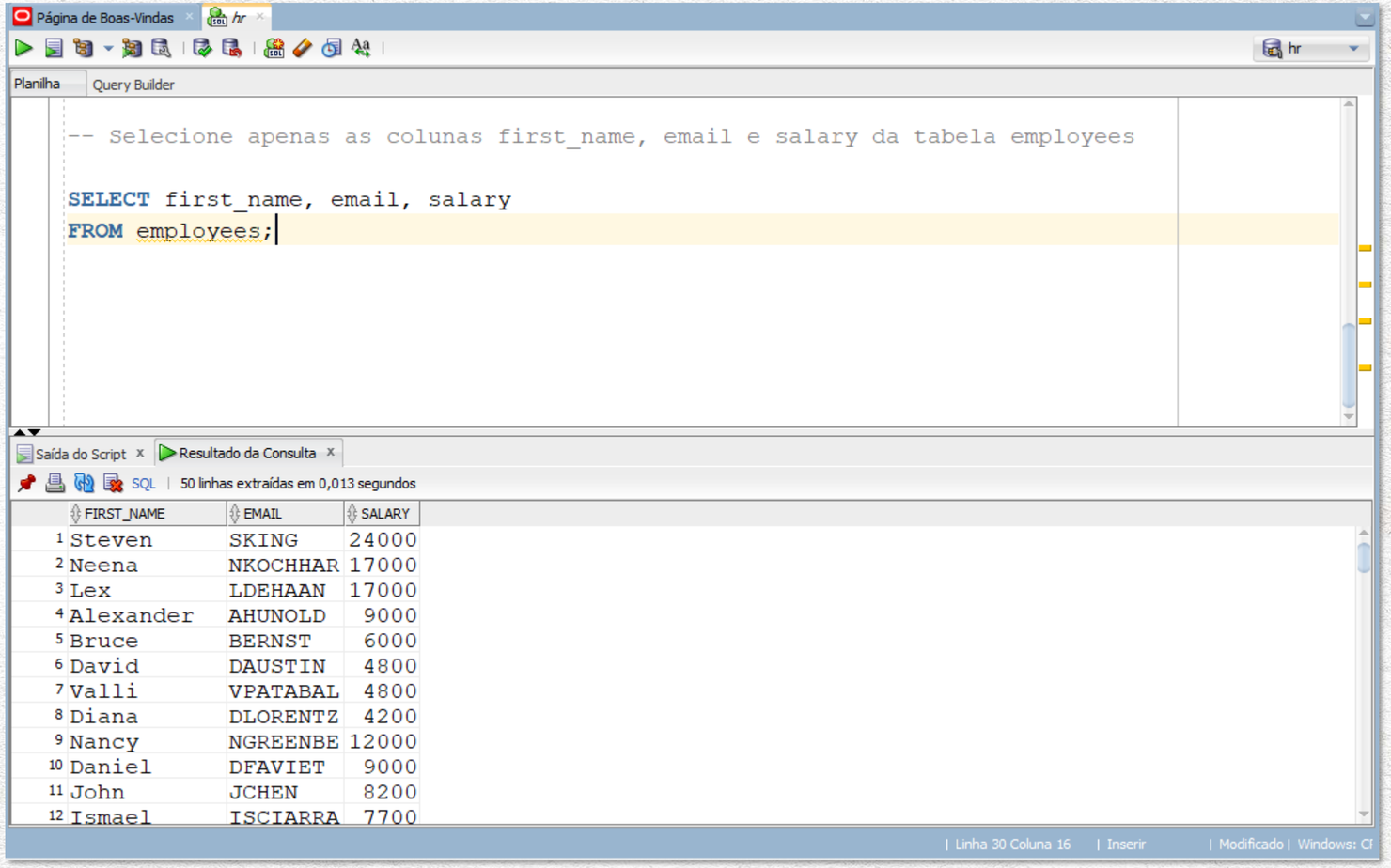


	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)
6	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)
7	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)
9	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)
10	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)
11	110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)

SELECT – Selecionando os dados da tabela

Pode ser que a gente não queira visualizar todas as colunas da tabela.

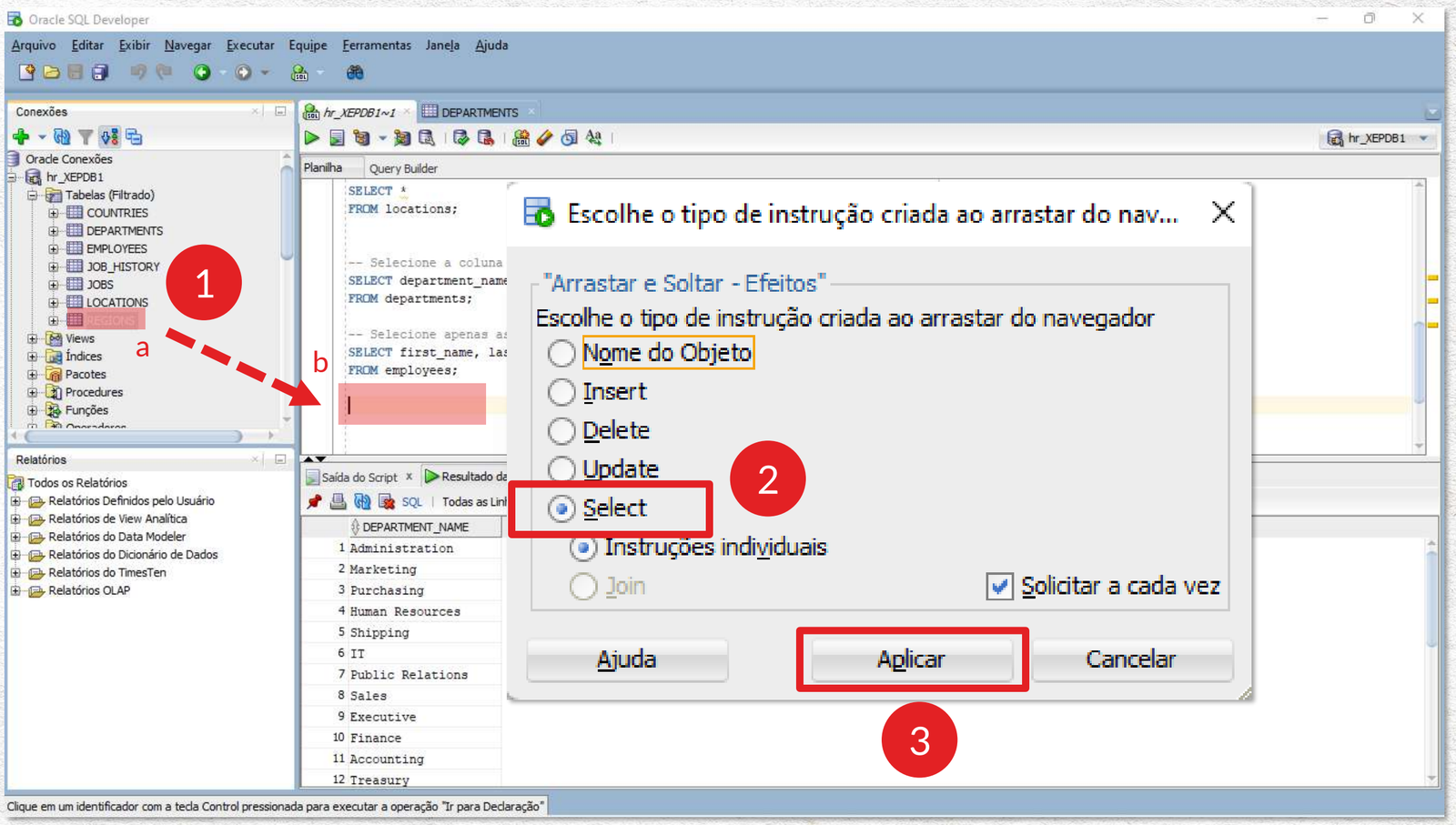
Dessa forma, especificamos quais colunas desejamos visualizar, em vez de simplesmente usar o asterisco.



SELECT – Selecionando os dados da tabela

Além de selecionar tabelas clicando no nome delas, ou então usando um código SQL para isso, temos também uma outra possibilidade no Oracle.

Se você clicar com o mouse na tabela, depois arrastar até a janela de consulta e soltar 1 , vai aparecer uma janela onde você pode informar que deseja fazer um SELECT 2 . Depois é só clicar em Aplicar 3 que a janela estará feita.



A seta tracejada significa que estamos clicando no ponto a, arrastando e soltando no ponto b.

Formas de executar um código

Temos duas formas de executar códigos no Oracle. Os botões de execução são identificados pelos ícones ao lado.

O ícone da **esquerda** executa apenas a linha onde o cursor está selecionado. Podemos usar o atalho CTRL + ENTER.

Já o da **direita** executa toda a janela de código. Ou seja, todos os comandos que estiverem ali serão executados. Diferente do anterior, que executará apenas a linha onde o cursor do mouse estiver posicionado. Para esta opção de execução, temos o bom e velho atalho F5.

Comentários

Quando começamos a programar, uma das coisas mais comuns é a gente se deparar com **comentários** dentro dos códigos.

Comentários são muito usados por alguns motivos:

1) Documentar e explicar o que está sendo feito no código.
2) Documentar quem foi o desenvolvedor do script e quando foi desenvolvido, para o caso de consultas futuras.

Comentários geralmente trazem alguma clareza e pode ser uma boa prática em diversas situações. Apenas tome cuidado para não fazer redações do ENEM nos seus códigos! Utilize os comentários de forma concisa e resumida.

No Oracle, podemos criar comentários de duas formas:

1) Comentários de uma linha, com o **--** .

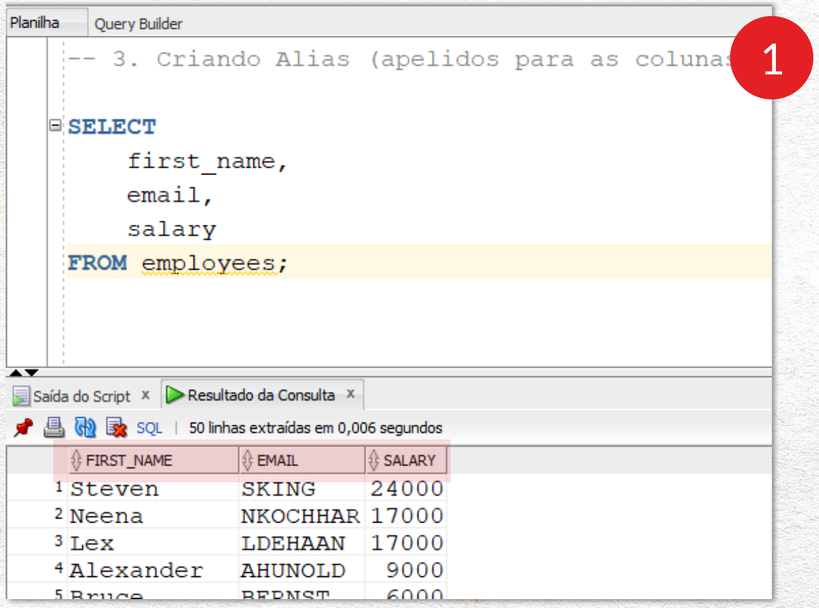
2) Comentários em blocos, começando com /* e terminando com */, o que permite comentar mais de 1 linha de uma vez.

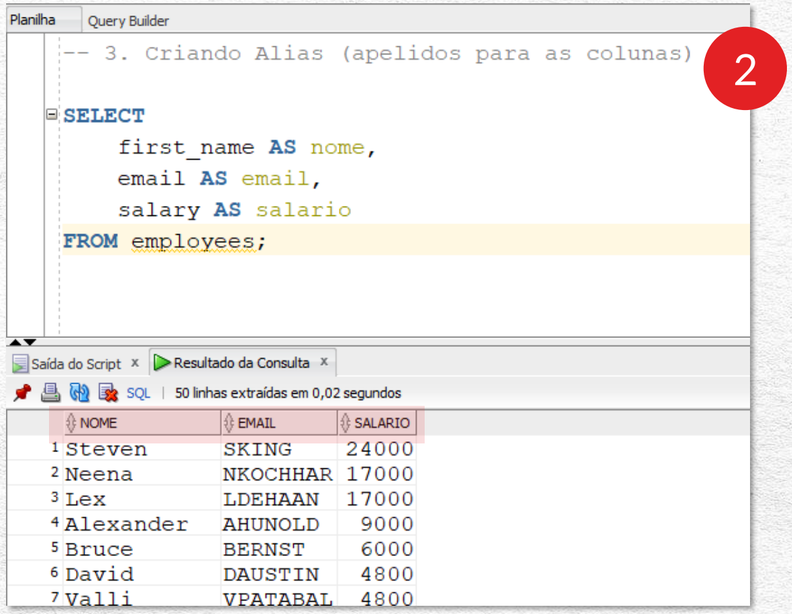
```
-- Comentando o código

-- É um comentário de 1 linha.
SELECT * FROM employees;

/*

SELECT *
FROM employees;

permite
comentar
múltiplas
linhas

*/
```

Alias de Colunas

Por padrão, quando criamos uma consulta a uma tabela do banco de dados, os cabeçalhos das colunas resultantes têm os mesmos nomes das colunas do banco de dados. Observe em 1 que a tabela foi retornada com os nomes originais (em inglês) das colunas nos bancos de dados.

Já em 2, utilizando o alias (comando AS) conseguimos renomear estes cabeçalhos.
Importante dizer que o AS não altera os nomes dos cabeçalhos no banco de dados. No banco de dados eles permanecem iguais.





Alias de Colunas

Também poderíamos utilizar o alias sem necessariamente colocar o AS, da forma como está mostrado ao lado.

Um detalhe importante sobre a nomeação das colunas: podemos também utilizar nomes compostos, mas uma boa prática é evitar espaços nos nomes, acentos e caracteres especiais. Busque usar um nome com o texto escrito todo junto, pra evitar qualquer problema.

Então, por exemplo, se você quisesse traduzir literalmente o **first_name**, poderia escrever **primeiro_nome**. Ou seja, com um underline (_) separando a primeira da segunda palavra.

```
SELECT
    first_name nome,
    email email,
    salary salario
FROM employees;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,006 segundos

	NOME	EMAIL	SALARIO
1	Steven	SKING	24000
2	Neena	NKOCHHAR	17000
3	Lex	LDEHAAN	17000
4	Alexander	AHUNOLD	9000
5	Bruce	BERNST	6000
6	David	DAUSTIN	4800
7	Valli	VPATABAL	4800
8	Diana	DLORENTZ	4200
9	Nancy	NGREENBE	12000
10	Daniel	DFAVIET	9000
11	John	JCHEN	8200
12	Ismael	ISCIARRA	7700
13	Jose Manuel	JMURMAN	7800

Alias de Colunas

Se ainda assim você quiser usar espaços, caracteres especiais e acentuação, você pode colocar os nomes entre aspas duplas, assim como mostrado ao lado.

```
-- Selecione as colunas first_name, email e salary da tabela employees.
-- Dê um nome para as colunas da tabela.

SELECT
    first_name "primeiro nome",
    email "e-mail",
    salary "salário (R$)"
FROM employees;
```

	primeiro nome	e-mail	salário (R$)
1	Steven	SKING	24000
2	Neena	NKOCHHAR	17000
3	Lex	LDEHAAN	17000
4	Alexander	AHUNOLD	9000
5	Bruce	BERNST	6000
6	David	DAUSTIN	4800
7	Valli	VPATABAL	4800
8	Diana	DLORENTZ	4200
9	Nancy	NGREENBE	12000

Alias de Tabelas

Uma aplicação do Alias bem importante será a de renomear tabelas dentro dos nossos códigos, para torná-los mais reduzidos.

No exemplo ao lado, fizemos uma relação entre as tabelas **employees** e **departments**. Observe que logo depois do nome employees utilizamos um **e**, e logo depois do departments usamos um **d**. Em seguida, em todos os lugares onde precisamos referenciar essas tabelas, utilizamos os seus apelidos, em vez de seus nomes completos.

Voltaremos nesse tipo de alias mais a frente, quando entrarmos na parte de relacionamentos entre as tabelas por meio dos JOINs. Por isso, não precisa se preocupar em entender isso agora.

```
-- Relaciona as tabelas employees e departments por meio de um LEFT JOIN.
-- Além disso, as tabelas são renomeadas com o aliasing.
SELECT
    e.employee_id,
    e.first_name,
    e.email,
    e.department_id,
    d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	EMAIL	DEPARTMENT_ID	DEPARTMENT_NAME
1	100	Steven	SKING	90	Executive
2	101	Neena	NKOCHHAR	90	Executive
3	102	Lex	LDEHAAN	90	Executive
4	103	Alexander	AHUNOLD	60	IT
5	104	Bruce	BERNST	60	IT
6	105	David	DAUSTIN	60	IT
7	106	Valli	VPATABAL	60	IT
8	107	Diana	DLORENTZ	60	IT
9	108	Nancy	NGREENBE	100	Finance

Operador de concatenação ||

Existe um operador especial no Oracle que permite concatenar textos, que seria a barra vertical dupla ||.

Com ela, podemos concatenar diversos textos, como mostrado ao lado.

Duas observações:

1- Textos no código são escritos com aspas simples (exceto no alias, que usamos aspas duplas)

2- Para concatenar os nomes, colocamos um espaço entre first_name e last_name para que os textos não ficassem colados.

```
-- 4. Operador de concatenação

-- Faça uma consulta que retorne o nome completo dos funcionários

SELECT
    first_name || ' ' || last_name "Nome Completo"
FROM employees;
```

	Nome Completo
1	Ellen Abel
2	Sundar Ande
3	Mozhe Atkinson
4	David Austin
5	Hermann Baer
6	Shelli Baida
7	Amit Banda
8	Elizabeth Bates
9	Sarah Bell

Resolvendo problema do ‘ dentro de uma string

Em algumas situações, precisamos ter uma aspa simples dentro do texto. Porém, a aspa simples serve para delimitar os textos dentro do SQL Oracle.

O que fazer?

Neste caso, vamos colocar o texto que possui uma aspa simples dentro de colchetes, iniciados por um q (q de *quote*, do inglês ‘aspas’).

O resultado é mostrado ao lado.

```
-- Faça uma consulta que retorne a lista de funcionários e seus respectivos gerentes

SELECT
    first_name || q'[. Employee's manager id: ]' || manager_id "Funcionário e Gerente"
FROM employees;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,009 segundos

	Funcionário e Gerente
1	Steven. Employee's manager id:
2	Neena. Employee's manager id: 100
3	Lex. Employee's manager id: 100
4	Alexander. Employee's manager id: 102
5	Bruce. Employee's manager id: 103
6	David. Employee's manager id: 103
7	Valli. Employee's manager id: 103
8	Diana. Employee's manager id: 103
9	Nancy. Employee's manager id: 101

Cálculos simples no SQL

É possível realizar cálculos básicos de soma, subtração, multiplicação e divisão no SQL.

Utilizamos os seguintes operadores:

+ para realizar a soma
- para realizar subtração
* para realizar multiplicação
/ para realizar divisão

Ao lado, temos um exemplo bem simples. Aplicamos um bônus de R$100 ao salário de todos os funcionários.

```
-- 5. Cálculos simples no SQL

-- Adicione um bônus de R$100 aos salários de todos os funcionários

SELECT
    first_name,
    last_name,
    salary,
    salary + 100 "salário + bônus"
FROM employees;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,005 segundos

	FIRST_NAME	LAST_NAME	SALARY	salário + bônus
1	Steven	King	24000	24100
2	Neena	Kochhar	17000	17100
3	Lex	De Haan	17000	17100
4	Alexander	Hunold	9000	9100
5	Bruce	Ernst	6000	6100
6	David	Austin	4800	4900
7	Valli	Pataballa	4800	4900
8	Diana	Lorentz	4200	4300
9	Nancy	Greenberg	12000	12100
10	Daniel	Faviet	9000	9100

Cálculos simples no SQL

Podemos incluir parênteses para fazer cálculos mais avançados, como no exemplo ao lado.

Aplicamos um bônus de 25% sobre o salário de todos os funcionários.

```
-- Adicione um bônus de 25% do salário para todos os funcionários

SELECT
    first_name,
    last_name,
    salary,
    salary * (1 + 0.25) "salário + bônus (25%)"
FROM employees;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,005 segundos

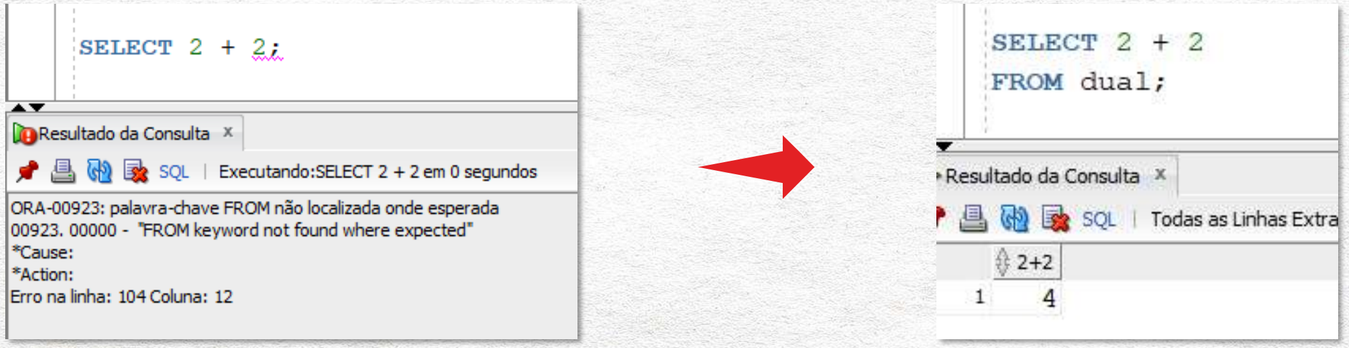
	FIRST_NAME	LAST_NAME	SALARY	salário + bônus (25%)
1	Steven	King	24000	30000
2	Neena	Kochhar	17000	21250
3	Lex	De Haan	17000	21250
4	Alexander	Hunold	9000	11250
5	Bruce	Ernst	6000	7500
6	David	Austin	4800	6000
7	Valli	Pataballa	4800	6000
8	Diana	Lorentz	4200	5250
9	Nancy	Greenberg	12000	15000
10	Daniel	Faviet	9000	11250
11	John	Chen	8200	10250
12	Ismael	Sciarra	7700	9625
13	Jose Manuel	Urman	7800	9750

Tabela Dual

A tabela DUAL do Oracle é usada para fazer operações com o SELECT quando não é necessário fazer uma seleção de dados de uma tabela.

Ela ajuda a manter a sintaxe de um SELECT em que não teremos uma tabela na consulta, mantendo a cláusula FROM.

Por exemplo, imagine que a gente queira fazer um cálculo simples de 2 + 2. Em outro banco de dados, usaríamos um SELECT 2 + 2 e já seria suficiente. Porém, no Oracle não é possível omitir a informação do FROM com a tabela. Ele retorna um erro. Para contornar isso, usamos a tabela dual.



Usaremos o FROM dual sempre que precisarmos fazer qualquer operação que não seja necessário selecionar uma tabela de um banco de dados.

SELECT DISTINCT – Selecionando valores distintos

Quando selecionamos uma coluna de uma tabela, o SQL retorna todas as linhas dessa tabela.

Observe na imagem ao lado: criamos um código para trazer a coluna department_id da tabela EMPLOYEES.

Porém, como todas as linhas foram retornadas, tivemos vários IDs se repetindo. Isso porque podemos ter vários funcionários de um mesmo departamento.

Como fazer então para retornar apenas os IDs **distintos**?

```
-- Qual é o total de departamentos? Faça esta análise em cima da tabela

SELECT
    department_id
FROM employees;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,01 segundos

	DEPARTMENT_ID
1	90
2	90
3	90
4	60
5	60
6	60
7	60
8	60
9	100
10	100
11	100
12	100
13	100
14	100

SELECT DISTINCT – Selecionando valores distintos

É aí que entra o **SELECT DISTINCT**. Este comando nos permite retornar apenas os valores distintos de uma coluna ou de uma combinação de colunas, e sua aplicação é mostrada na imagem ao lado.

Observe que no resultado temos apenas os department_ids distintos, nenhum está repetido.

```
-- Qual é o total de departamentos? Faça esta análise em cima da tabela

SELECT
    DISTINCT department_id
FROM employees;
```

Saída do Script × | Resultado da Consulta ×

SQL | Todas as Linhas Extraídas: 12 em 0,011 segundos

	DEPARTMENT_ID
1	90
2	60
3	100
4	30
5	50
6	80
7	(null)
8	10
9	20
10	40
11	70
12	110

Selecionando valores distintos (mais de uma coluna)

Podemos querer saber também os valores distintos considerando mais de uma coluna por vez.

No exemplo ao lado, queremos os distintos considerando tanto o first_name quanto o last_name. Dessa maneira, será feita uma espécie de concatenação dos valores para então retornar com os valores distintos.

Por exemplo, neste caso, os nomes David Austin e David Bernstein seriam nomes distintos e portanto estariam presentes em linhas diferentes da tabela.

```
-- Retorne os valores distintos de nomes completos dos funcionários

SELECT
    DISTINCT first_name, last_name
FROM employees;
```

Saída do Script | Resultado da Consulta

SQL | 50 linhas extraídas em 0,011 segundos

	FIRST_NAME	LAST_NAME
1	Ellen	Abel
2	Sundar	Ande
3	Mozhe	Atkinson
4	David	Austin
5	Hermann	Baer
6	Shelli	Baida
7	Amit	Banda
8	Elizabeth	Bates
9	Sarah	Bell
10	David	Bernstein
11	Laura	Bissot
12	Harrison	Bloom
13	Alexis	Bull

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. Os comandos em SQL não diferenciam maiúsculas de minúsculas. Usar SELECT ou select funcionará do mesmo jeito. Mas, uma prática comum é colocar os nomes de comandos em MAIÚSCULA, já os nomes de colunas, tabelas, escrever em minúscula.
2. Podemos escrever todo o código em uma linha só, ou em várias linhas. Geralmente, colocamos os comandos (SELECT, FROM, WHERE, GROUP BY, etc.) em linhas separadas.
3. Se for realizar consultas de uma linha, utilize a TABELA DUAL.
4. INDENTAR um código não é obrigatório, mas facilita o entendimento.
5. Terminamos os comandos em SQL usando o ponto e vírgula.

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

6 Podemos executar os códigos de duas formas: com o para rodar apenas a linha onde o cursor está posicionado ou com o para executar o código por completo.

7 Comentários nos códigos são uma boa prática. Podemos comentar uma única linha com o -- ou comentar várias linhas de uma vez com o /* */. Apenas tome cuidado para não abusar muito dos comentários.

8 É possível personalizar os nomes das colunas e tabelas na hora de criar consultas por meio do ALIASING.

9 Utilize o operador || para concatenar colunas do banco de dados.

10 O comando DISTINCT retorna os valores distintos de uma coluna. Se utilizarmos mais de uma coluna na hora de usar o DISTINCT, ele irá retornar os valores distintos considerando uma concatenação das colunas que você tiver informado.

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1 Os comandos em SQL não diferenciam maiúsculas de minúsculas. Usar SELECT ou select funcionará do mesmo jeito. Mas, uma prática comum é colocar os nomes de comandos em MAIÚSCULA, já os nomes de colunas, tabelas, escrever em minúscula.

```sql
-- Selecione a tabela de funcionários (EMPLOYEES)
SELECT *
FROM employees;
```

```sql
-- Selecione a tabela de funcionários (EMPLOYEES)
select *
from employees;
```

Não faz diferença usar os comandos em maiúsculas ou minúsculas.

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

2

Podemos escrever todo o código em uma linha só, ou em várias linhas.
Geralmente, colocamos os comandos (SELECT, FROM, WHERE, GROUP BY, etc.) em linhas separadas.

```
-- Selecione apenas as colunas first_name, last_name e salary da tabela EMPLOYEES
SELECT first_name, last_name, salary FROM employees;
```

```
SELECT
    first_name,
    last_name,
    salary
FROM employees;
```

Os dois códigos acima fazem a mesma coisa.
Porém, o código da imagem inferior é bem mais intuitivo e organizado do que o código da imagem superior.

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

3 Se for realizar consultas de uma linha, utilize a TABELA DUAL.

```
SELECT sysdate
FROM dual;
```

```
SELECT 2 * 2
FROM dual;
```

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

4 INDENTAR um código não é obrigatório, mas facilita o entendimento.

```
SELECT
first_name,
last_name,
salary
FROM employees;
```

```
SELECT
    first_name,
    last_name,
    salary
FROM employees;
```

Os dois códigos acima fazem a mesma coisa.
Porém, no código da imagem à direita fica mais fácil de entender que as colunas fazem parte do comando SELECT.
Esses espaçamentos são chamados de indentação e facilitam muito o entendimento do código.

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

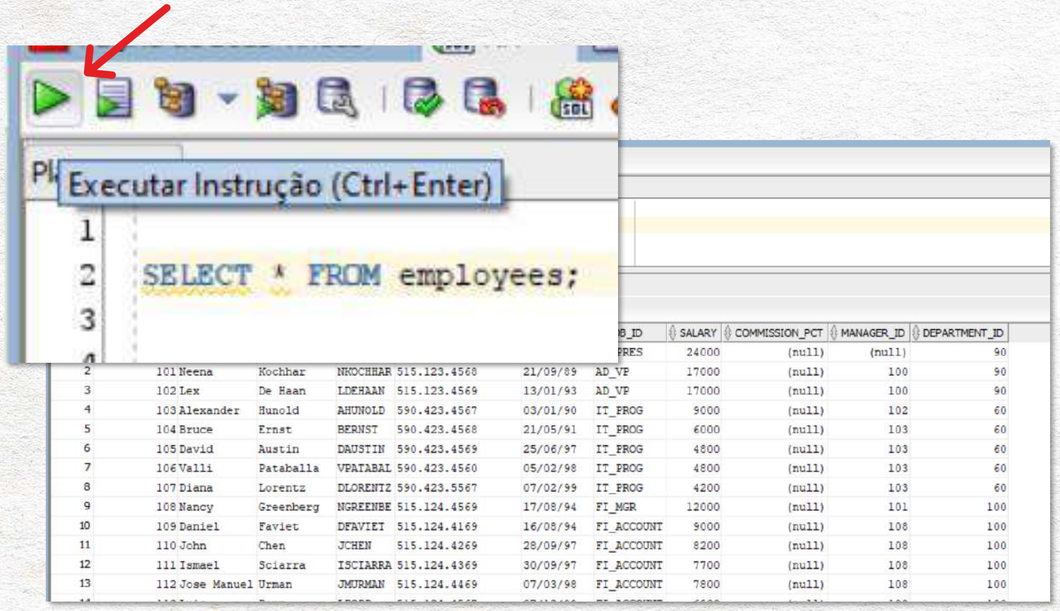
5 Terminamos os comandos em SQL usando o ponto e vírgula.

```
-- Selecione apenas as colunas first_name, last_name e salary da tabela EMPLOYEES
SELECT first_name, last_name, salary FROM employees;
```

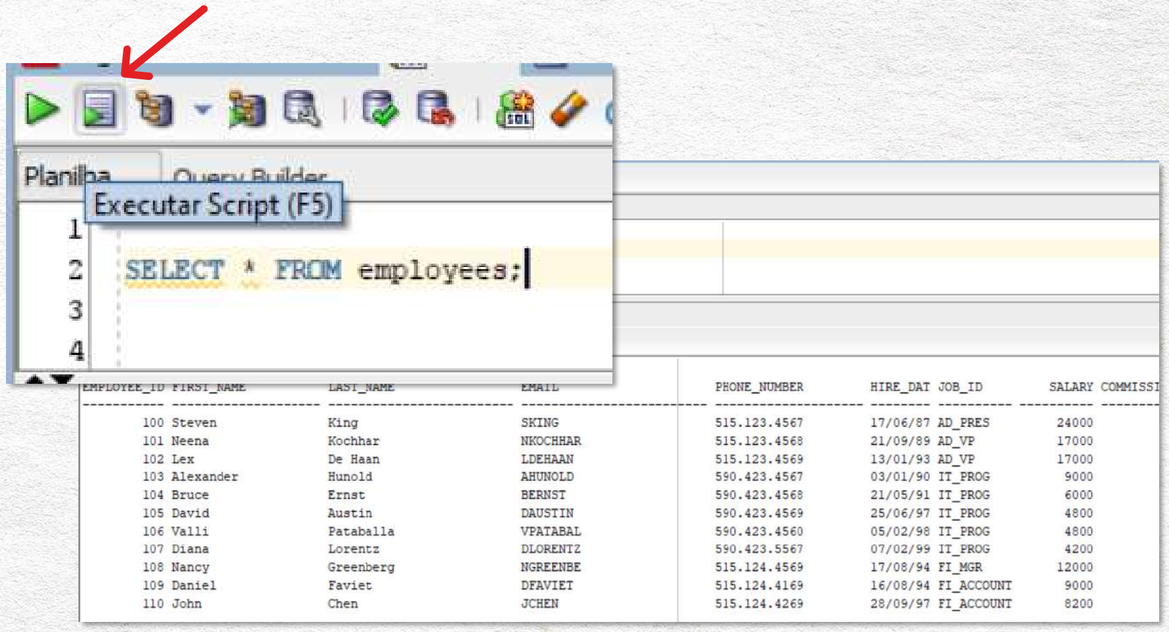
Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

6 Podemos executar os códigos de duas formas: com o ▶ para rodar apenas a linha onde o cursor está posicionado ou com o 📄 para executar o código por completo.



	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60
6	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
7	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60
9	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
10	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
11	110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)	108	100
12	111	Ismael	Sciarra	ISCIARRA	515.124.4369	30/09/97	FI_ACCOUNT	7700	(null)	108	100
13	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	(null)	108	100



EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DAT	JOB_ID	SALARY	COMMISSI
100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	
102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	
103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	
104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	
105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	
106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	
107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	
108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	
109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	
110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

7 Comentários nos códigos são uma boa prática. Podemos comentar uma única linha com o -- ou comentar várias linhas de uma vez com o /* */. Apenas tome cuidado para não abusar muito dos comentários.

```
-- É um comentário de 1 linha.
SELECT * FROM employees;
```

```
/*

SELECT *
FROM employees;

permite
comentar
múltiplas
linhas

*/
```

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

8 É possível personalizar os nomes das colunas e tabelas na hora de criar consultas por meio do ALIASING.

```sql
-- Selecione as colunas FIRST_NAME, EMAIL e SALARY da tabela EMPLOYEES.
-- Dê um nome para as colunas da tabela.

SELECT
    first_name "primeiro nome",
    email "e-mail",
    salary "salário (R$)"
FROM employees;

-- Selecione todas as colunas da tabela JOBS.
-- Dê um nome para a tabela JOBS.

SELECT *
FROM JOBS J;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,026 segundos

	primeiro nome	e-mail	salário (R$)
1	Steven	SKING	24000
2	Neena	NKOCHHAR	17000
3	Lex	LDEHAAN	17000

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,03 segundos

	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
1	AD_PRES	President	20000	40000
2	AD_VP	Administration Vice President	15000	30000
3	AD_ASST	Administration Assistant	3000	6000

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

9 Utilize o operador || para concatenar colunas do banco de dados.

```
SELECT
    first_name || ' ' || last_name "Nome Completo"
FROM employees;
```

Resultado da Consulta

SQL | 50 linhas e

	Nome Completo
1	Ellen Abel
2	Sundar Ande
3	Mozhe Atkinson
4	David Austin
5	Hermann Baer
6	Shelli Baida
7	Amit Banda
8	Elizabeth Bates

Resumo do Módulo e Boas Práticas

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

10 O comando DISTINCT retorna os valores distintos de uma coluna. Se utilizarmos mais de uma coluna na hora de usar o DISTINCT, ele irá retornar os valores distintos considerando uma concatenação das colunas que você tiver informado.

```
SELECT
    DISTINCT department_id
FROM employees;
```

Resultado da Consulta

	DEPARTMENT_ID
1	90
2	60
3	100
4	30
5	50
6	80
7	(null)
8	10
9	20
10	40
11	70
12	110

EXERCÍCIOS

1

Questão 1

A equipe de Recursos Humanos precisa gerar relatórios diários com a data atual para acompanhar processos internos, como admissões, desligamentos e aniversários de colaboradores. Para garantir que a data do relatório seja sempre exibida corretamente, é necessário testar a obtenção da data do sistema.

Retorne um texto que exiba a seguinte mensagem: "Hoje é dia: ", concatenada com a data atual. Chame essa coluna de "Data Atual".

2

Questão 2

A equipe de planejamento estratégico da empresa está revisando a estrutura organizacional para avaliar o desempenho de cada departamento. Para facilitar o planejamento de novos projetos, é necessário obter informações completas sobre todos os departamentos da empresa.

Crie uma consulta que retorne todos os dados referentes aos departamentos da empresa.

(Utilize a tabela departments)

EXERCÍCIOS

3

Questão 3

O departamento de Recursos Humanos está realizando uma análise para entender a diversidade de cargos dentro da organização. Eles precisam identificar todos os cargos disponíveis para revisar as funções e responsabilidades de cada um, além de verificar se há necessidade de criar novos cargos ou reestruturar algum existente.

Crie uma consulta que retorne a lista distinta de títulos de cargos (job_title).

(Utilize a tabela jobs)

4

Questão 4

A equipe de TI precisa verificar a estrutura da tabela locations para entender melhor os dados relacionados às localizações das filiais da empresa. Eles precisam garantir que todas as colunas necessárias para a análise de endereços estão presentes, e também verificar os tipos de dados armazenados.

Utilize o comando SQL que exibe a estrutura da tabela locations, incluindo as colunas e seus tipos de dados.

(Utilize a tabela locations)

EXERCÍCIOS

5

Questão 5

O time de marketing está planejando uma campanha regionalizada e precisa identificar todas as regiões em que a empresa atua. Para isso, é necessário obter os nomes das regiões, para que possam segmentar a campanha adequadamente e adaptar as ações de marketing a cada local.

Escreva uma consulta SQL para retornar os nomes das regiões onde a empresa possui filiais.

(Utilize a tabela regions)

6

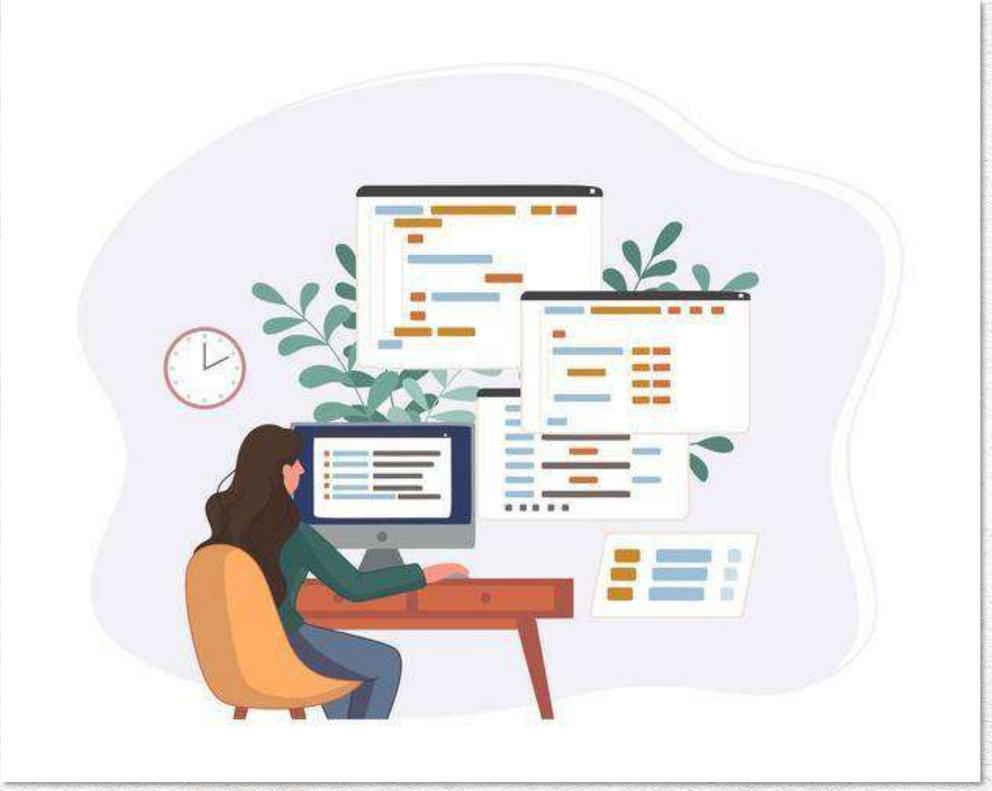
Questão 6

O time de planejamento de carreira da empresa está revisando as diferentes funções que os colaboradores ocupam para ajustar os planos de desenvolvimento e as políticas de remuneração. Para facilitar essa análise, eles precisam identificar todos os tipos de cargos distintos registrados nos dados dos colaboradores.

Escreva uma consulta SQL para obter todos os tipos de cargos distintos (job_id) ocupados pelos colaboradores.

(Utilize a tabela employees)

EXERCÍCIOS



7

Questão 7

O time de logística está revisando as informações de países onde a empresa tem operações para planejar a distribuição de produtos e serviços. Eles precisam consultar os códigos e nomes dos países para entender melhor a cobertura geográfica da empresa e garantir que as operações estejam alinhadas a cada local.

Crie uma consulta que retorne os códigos e nomes dos países, renomeando as colunas para "Código do País" e "País".

(Utilize a tabela countries)

8

Questão 8

O departamento de Recursos Humanos está revisando a estrutura salarial da empresa e precisa calcular o impacto de um aumento de 15% no piso e no teto salarial de cada cargo. Essa análise ajudará a ajustar os valores de salário para se alinhar às novas diretrizes de remuneração.

Escreva uma consulta SQL para retornar o "ID do Cargo", o "Cargo", o "Piso Salarial" e o "Teto Salarial" com o aumento de 15% aplicado. As colunas devem ser renomeadas conforme descrito acima.

(Utilize a tabela jobs)

GABARITO EXPLICATIVO

1

Questão 1

A equipe de Recursos Humanos precisa gerar relatórios diários com a data atual para acompanhar processos internos, como admissões, desligamentos e aniversários de colaboradores. Para garantir que a data do relatório seja sempre exibida corretamente, é necessário testar a obtenção da data do sistema.

Retorne um texto que exiba a seguinte mensagem: "Hoje é dia: ", concatenada com a data atual. Chame essa coluna de "Data Atual".

Explicação e Resultado

Para retornar um texto que exiba a mensagem "Hoje é dia: ", concatenada com a data atual, fazemos um **SELECT** que retorna o texto **"Hoje é dia: "** e a data atual obtida pela função **sysdate**.

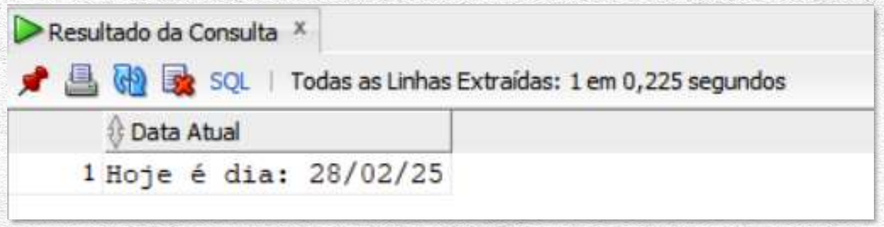
Para concatenar as duas partes, usamos o operador **||**.

E para exibir essa coluna com o nome **"Data Atual"**, aplicamos um **alias** a ela.

Por fim, como não queremos acessar nenhum dado de uma tabela específica de um banco de dados, acrescentamos **FROM dual** ao final para manter a sintaxe padrão do PL/SQL (versão do SQL aplicada ao Oracle Database):

```
SELECT
'Hoje é dia: ' || sysdate "Data Atual"
FROM dual;
```

Dessa forma, exibimos o seguinte resultado:



GABARITO EXPLICATIVO

2

Questão 2

A equipe de planejamento estratégico da empresa está revisando a estrutura organizacional para avaliar o desempenho de cada departamento. Para facilitar o planejamento de novos projetos, é necessário obter informações completas sobre todos os departamentos da empresa.

Crie uma consulta que retorne todos os dados referentes aos departamentos da empresa.

(Utilize a tabela departments)

Explicação e Resultado

Para retornar todos os dados referentes aos departamentos da empresa, devemos retornar **todas as linhas e colunas da tabela departments**. Para isso, utilizamos o comando **SELECT**, conforme mostrado abaixo:

```
SELECT *
FROM departments;
```

Dessa forma, retornamos o resultado mostrado ao lado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 27 em 0,055 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700
4	40	Human Resources	203	2400
5	50	Shipping	121	1500
6	60	IT	103	1400
7	70	Public Relations	204	2700
8	80	Sales	145	2500
9	90	Executive	100	1700
10	100	Finance	108	1700
11	110	Accounting	205	1700
12	120	Treasury	(null)	1700
13	130	Corporate Tax	(null)	1700
14	140	Control And Credit	(null)	1700
15	150	Shareholder Services	(null)	1700
16	160	Benefits	(null)	1700
17	170	Manufacturing	(null)	1700
18	180	Construction	(null)	1700
19	190	Contracting	(null)	1700
20	200	Operations	(null)	1700
21	210	IT Support	(null)	1700
22	220	NOC	(null)	1700
23	230	IT Helpdesk	(null)	1700
24	240	Government Sales	(null)	1700
25	250	Retail Sales	(null)	1700
26	260	Recruiting	(null)	1700
27	270	Payroll	(null)	1700

GABARITO EXPLICATIVO

3

Questão 3

O departamento de Recursos Humanos está realizando uma análise para entender a diversidade de cargos dentro da organização. Eles precisam identificar todos os cargos disponíveis para revisar as funções e responsabilidades de cada um, além de verificar se há necessidade de criar novos cargos ou reestruturar algum existente.

Crie uma consulta que retorne a lista distinta de títulos de cargos (job_title).

(Utilize a tabela jobs)

Explicação e Resultado

Para retornar a lista distinta de títulos de cargos, devemos aplicar um **SELECT DISTINCT** à coluna **job_title** da tabela **jobs**. Dessa forma, eliminamos o risco de retornar job_titles em duplicidade, caso existam:

```
SELECT
    DISTINCT job_title
FROM jobs;
```

Dessa forma, retornamos o resultado mostrado ao lado:

Resultado da Consulta | SQL | Todas as Linhas Extraídas: 19 em 0,042 segundos

	JOB_TITLE
1	President
2	Administration Vice President
3	Administration Assistant
4	Finance Manager
5	Accountant
6	Accounting Manager
7	Public Accountant
8	Sales Manager
9	Sales Representative
10	Purchasing Manager
11	Purchasing Clerk
12	Stock Manager
13	Stock Clerk
14	Shipping Clerk
15	Programmer
16	Marketing Manager
17	Marketing Representative
18	Human Resources Representative
19	Public Relations Representative

GABARITO EXPLICATIVO

4

Questão 4

A equipe de TI precisa verificar a estrutura da tabela locations para entender melhor os dados relacionados às localizações das filiais da empresa. Eles precisam garantir que todas as colunas necessárias para a análise de endereços estão presentes, e também verificar os tipos de dados armazenados.

Utilize o comando SQL que exibe a estrutura da tabela locations, incluindo as colunas e seus tipos de dados.

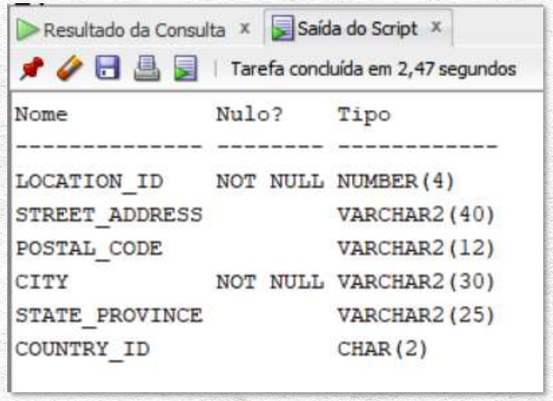
(Utilize a tabela locations)

Explicação e Resultado

O comando SQL capaz de retornar os nomes das colunas e seus tipos de dados em um único resultado é o **DESCRIBE** (ou **DESC**). Sendo assim, para exibir essas informações da tabela **locations**, executamos o comando abaixo:

```
DESCRIBE locations;
```

Dessa forma, retornamos o seguinte resultado:



GABARITO EXPLICATIVO

5

Questão 5

O time de marketing está planejando uma campanha regionalizada e precisa identificar todas as regiões em que a empresa atua. Para isso, é necessário obter os nomes das regiões, para que possam segmentar a campanha adequadamente e adaptar as ações de marketing a cada local.

Escreva uma consulta SQL para retornar os nomes das regiões onde a empresa possui filiais.

(Utilize a tabela regions)

Explicação e Resultado

Para retornar os nomes das regiões onde a empresa possui filiais, devemos fazer um **SELECT** à tabela **regions**, retornando sua coluna chamada **region_name**:

```
SELECT
    region_name
FROM regions;
```

Dessa forma, exibimos o resultado abaixo:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 4 em 0,018 segundos

	REGION_NAME
1	Europe
2	Americas
3	Asia
4	Middle East and Africa

GABARITO EXPLICATIVO

6

Questão 6

O time de planejamento de carreira da empresa está revisando as diferentes funções que os colaboradores ocupam para ajustar os planos de desenvolvimento e as políticas de remuneração. Para facilitar essa análise, eles precisam identificar todos os tipos de cargos distintos registrados nos dados dos colaboradores.

Escreva uma consulta SQL para obter todos os tipos de cargos distintos (job_id) ocupados pelos colaboradores.

(Utilize a tabela employees)

Explicação e Resultado

Para retornar todos os tipos de cargos distintos ocupados pelos colaboradores, devemos aplicar um **SELECT DISTINCT** à coluna **job_id** da tabela **employees**. Dessa forma, eliminamos o risco de retornar job_ids em duplicidade, caso existam:

```
SELECT
    DISTINCT job_id
FROM employees;
```

Dessa forma, retornamos o resultado mostrado ao lado:

Resultado da Consulta

	JOB_ID
1	AC_ACCOUNT
2	AC_MGR
3	AD_ASST
4	AD_PRES
5	AD_VP
6	FI_ACCOUNT
7	FI_MGR
8	HR_REP
9	IT_PROG
10	MK_MAN
11	MK_REP
12	PR_REP
13	PU_CLERK
14	PU_MAN
15	SA_MAN
16	SA_REP
17	SH_CLERK
18	ST_CLERK
19	ST_MAN

GABARITO EXPLICATIVO

7

Questão 7

O time de logística está revisando as informações de países onde a empresa tem operações para planejar a distribuição de produtos e serviços. Eles precisam consultar os códigos e nomes dos países para entender melhor a cobertura geográfica da empresa e garantir que as operações estejam alinhadas a cada local.

Crie uma consulta que retorne os códigos e nomes dos países, renomeando as colunas para "Código do País" e "País".

(Utilize a tabela countries)

Explicação e Resultado

Para retornar os códigos e os nomes dos países, devemos fazer um **SELECT** à tabela **countries** para retornar suas colunas **country_id** e **country_name**.

Para exibir essas colunas com os nomes **"Código do País"** e **"País"**, respectivamente, aplicamos **aliases** a elas:

```
SELECT
    country_id "Código do País",
    country_name "País"
FROM countries;
```

Dessa forma, retornamos o resultado ao lado, com as colunas renomeadas:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 25 em 0,026 segundos

	Código do País	País
1	AR	Argentina
2	AU	Australia
3	BE	Belgium
4	BR	Brazil
5	CA	Canada
6	CH	Switzerland
7	CN	China
8	DE	Germany
9	DK	Denmark
10	EG	Egypt
11	FR	France
12	HK	HongKong
13	IL	Israel
14	IN	India
15	IT	Italy
16	JP	Japan
17	KW	Kuwait
18	MX	Mexico
19	NG	Nigeria
20	NL	Netherlands
21	SG	Singapore
22	UK	United Kingdom
23	US	United States of America
24	ZM	Zambia
25	ZW	Zimbabwe

GABARITO EXPLICATIVO

8

Questão 8

O departamento de Recursos Humanos está revisando a estrutura salarial da empresa e precisa calcular o impacto de um aumento de 15% no piso e no teto salarial de cada cargo. Essa análise ajudará a ajustar os valores de salário para se alinhar às novas diretrizes de remuneração.

Escreva uma consulta SQL para retornar o "ID do Cargo", o "Cargo", o "Piso Salarial" e o "Teto Salarial" com o aumento de 15% aplicado. As colunas devem ser renomeadas conforme descrito acima.

(Utilize a tabela jobs)

Explicação e Resultado

Para retornar o **job_id**, o **job_title**, o **min_salary** e o **max_salary**, devemos fazer um **SELECT** à tabela **jobs** para exibir tais colunas.

Como queremos aplicar um **aumento de 15%** no piso salarial e no teto salarial, **multiplicamos** as colunas **min_salary** e **max_salary** por **(1 + 0.15)**.

Para exibir essas colunas com os nomes **"ID do Cargo"**, **"Cargo"**, **"Piso Salarial"** e **"Teto Salarial"**, respectivamente, aplicamos **aliases** a elas:

```
SELECT
    job_id "ID do Cargo",
    job_title "Cargo",
    min_salary * (1 + 0.15) "Piso Salarial",
    max_salary * (1 + 0.15) "Teto Salarial"
FROM jobs;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

O departamento de Recursos Humanos está revisando a estrutura salarial da empresa e precisa calcular o impacto de um aumento de 15% no piso e no teto salarial de cada cargo. Essa análise ajudará a ajustar os valores de salário para se alinhar às novas diretrizes de remuneração.

Escreva uma consulta SQL para retornar o "ID do Cargo", o "Cargo", o "Piso Salarial" e o "Teto Salarial" com o aumento de 15% aplicado. As colunas devem ser renomeadas conforme descrito acima.

(Utilize a tabela jobs)

Explicação e Resultado

Dessa forma, retornamos o resultado ao lado, com as colunas renomeadas:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,019 segundos

	ID do Cargo	Cargo	Piso Salarial	Teto Salarial
1	AD_PRES	President	23000	46000
2	AD_VP	Administration Vice President	17250	34500
3	AD_ASST	Administration Assistant	3450	6900
4	FI_MGR	Finance Manager	9430	18400
5	FI_ACCOUNT	Accountant	4830	10350
6	AC_MGR	Accounting Manager	9430	18400
7	AC_ACCOUNT	Public Accountant	4830	10350
8	SA_MAN	Sales Manager	11500	23000
9	SA_REP	Sales Representative	6900	13800
10	PU_MAN	Purchasing Manager	9200	17250
11	PU_CLERK	Purchasing Clerk	2875	6325
12	ST_MAN	Stock Manager	6325	9775
13	ST_CLERK	Stock Clerk	2300	5750
14	SH_CLERK	Shipping Clerk	2875	6325
15	IT_PROG	Programmer	4600	11500
16	MK_MAN	Marketing Manager	10350	17250
17	MK_REP	Marketing Representative	4600	10350
18	HR_REP	Human Resources Representative	4600	10350
19	PR_REP	Public Relations Representative	5175	12075

MÓDULO 5

ORDENANDO E FILTRANDO DADOS

ORDENANDO E FILTRANDO DADOS

ORDENANDO E FILTRANDO DADOS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 5 ORDENANDO E FILTRANDO DADOS

01	ORDER BY: ORDENANDO OS DADOS DE UMA TABELA	139
02	ORDER BY: ORDENANDO MAIS DE UMA COLUNA	140
03	ORDER BY: SINTAXE	141
04	ORDER BY ASC: APLICAÇÃO	142
05	ORDER BY DESC: APLICAÇÃO	145
06	ORDER BY: NULLS FIRST e NULLS LAST	148
07	NULLS FIRST E NULLS LAST: APLICAÇÃO	149
08	FETCH NEXT: LIMITANDO LINHAS DA TABELA	152
09	WHERE: INTRODUÇÃO	156
10	WHERE: SINTAXE	158
11	OPERADORES BÁSICOS DE FILTROS	159
12	WHERE	161
13	WHERE: FILTROS DE NÚMEROS	162
14	WHERE: FILTROS DE TEXTO	164
15	WHERE: FILTROS DE DATA	165
16	OPERADORES AND, OR e NOT	166
17	OPERADOR AND	167
18	OPERADOR OR	168
19	OPERADOR NOT	169
20	OPERADOR LIKE e WILDCARDS	170
21	OPERADOR BETWEEN	176
22	OPERADOR BETWEEN: NÚMEROS	178
23	OPERADOR BETWEEN: TEXTOS	179
24	OPERADOR BETWEEN: DATAS	180
25	OPERADOR IN	181
26	OPERADOR IS NULL	184
27	REGRAS DE PRECEDÊNCIA	187
28	VARIÁVEIS DE SUBSTITUIÇÃO	191
29	RESUMO DO MÓDULO	193
30	EXERCÍCIOS	208
31	GABARITO EXPLICATIVO	213

Índice Geral

ORDER BY: Ordenando os dados de uma tabela

Vamos agora falar sobre o ORDER BY. O objetivo do ORDER BY é ordenar uma ou mais colunas de uma tabela de forma ascendente ou descendente. Mas o que significa ascendente ou descendente?

Bom, vai depender do tipo de dado que a coluna a ser ordenada possui.

Se a coluna a ser ordenada for do tipo **NÚMERO**, a ordenação ASCENDENTE significará uma ordenação CRESCENTE, enquanto uma ordenação DESCENDENTE significará uma ordenação DECRESCENTE.

Caso a coluna a ser ordenada seja do tipo **TEXTO**, a ordenação ASCENDENTE significará uma ordenação ALFABÉTICA (A-Z), enquanto uma ordenação DESCENDENTE significará uma ordenação "ANTI"-ALFABÉTICA (Z-A).

Já para uma coluna do tipo **DATA**, a ordenação ASCENDENTE significará uma ordenação CRESCENTE de data (data mais antiga para a data mais recente), enquanto uma ordenação DESCENDENTE significará uma ordenação DECRESCENTE de data (data mais recente para a data mais antiga).

- Usamos a palavra-chave **ASC** para ordenar de forma ASCENDENTE.
- Usamos a palavra-chave **DESC** para ordenar de forma DESCENDENTE.
- Caso o ORDER BY seja usado sem especificar uma das duas formas anteriores, a ordenação padrão é ASCENDENTE (ASC).

ORDER BY: Ordenando mais de uma coluna

Podemos ordenar uma tabela a partir de mais de uma coluna. No exemplo ao lado, ordenamos a tabela EMPLOYEES de acordo com as colunas first_name e last_name, ambas em ordem ASC (de A até Z).

Isso significa que, caso tenhamos dois funcionários com o mesmo first_name, o que vai decidir a ordenação final dos nomes será o last_name.

Observe por exemplo o nome ALEXANDER. Temos dois funcionários com esse first_name, mas como ordenamos também pela coluna last_name em ordem ASC, por isso o ALEXANDER HUNOLD vem antes do ALEXANDER KHOO (H vem antes de K).

```
-- a) Alguns funcionários possuem o mesmo first_name (por exemplo, o JAMES).
-- Ordene a tabela EMPLOYEES de acordo com first_name e last_name para fazer o desempate.

SELECT * FROM employees
ORDER BY first_name ASC, last_name ASC;

-- 1.5. Comando ORDER BY com NULLS FIRST e NULLS LAST.
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,127 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	10
2	196	Alana	Walsh	AWALSH	650.507.9811	24/04/98	SH_CLERK	3100	(null)	12
3	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	10
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	10
5	115	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	(null)	11
6	185	Alexis	Bull	ABULL	650.509.2876	20/02/97	SH_CLERK	4100	(null)	12
7	158	Allan	McEwen	AMCEWEN	011.44.1345.829268	01/08/96	SA_REP	9000	0,35	14
8	175	Alyssa	Hutton	AHUTTON	011.44.1644.429266	19/03/97	SA_REP	8800	0,25	14
9	167	Amit	Banda	ABANDA	011.44.1346.729268	21/04/00	SA_REP	6200	0,1	14
10	187	Anthony	Cabrio	ACABRIO	650.509.4876	07/02/99	SH_CLERK	3000	(null)	12
11	193	Britney	Everett	BEVERETT	650.501.2876	03/03/97	SH_CLERK	3900	(null)	12
12	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	10
13	179	Charles	Johnson	CJOHNSON	011.44.1644.429262	04/01/00	SA_REP	6200	0,1	14
14	153	Christopher	Olsen	COLSEN	011.44.1344.498718	30/03/98	SA_REP	8000	0,2	14
15	162	Clara	Vishney	CVISHNEY	011.44.1346.129268	11/11/97	SA_REP	10500	0,25	14

ORDER BY: Sintaxe

A sintaxe usada para o comando ORDER BY é a seguinte:

- **SELECT** * **FROM** **Tabela** **ORDER BY** **coluna1**;

Seleciona a tabela ordenando pela coluna1 em ordem crescente (ASC). Como não especificamos se é ASC ou DESC, a ordenação padrão será ASC.

- **SELECT** * **FROM** **Tabela** **ORDER BY** **coluna1** **ASC**;

Seleciona a tabela ordenando pela coluna1 em ordem crescente (ASC). Aqui especificamos que a ordenação será ASC.

- **SELECT** * **FROM** **Tabela** **ORDER BY** **coluna1** **DESC**;

Seleciona a tabela ordenando pela coluna1 em ordem decrescente (DESC). Aqui especificamos que a ordenação será DESC.

- **SELECT** * **FROM** **Tabela** **ORDER BY** **coluna1** **ASC**, **coluna2** **DESC**;

Seleciona a tabela ordenando por duas colunas: coluna1 em ordem crescente (ASC) e coluna2 em ordem decrescente (DESC).

ORDER BY ASC: Aplicação

Agora que já sabemos a lógica por trás do ORDER BY, podemos praticar algumas aplicações.

Podemos selecionar a tabela EMPLOYEES e ordenar de acordo com a coluna salary, que é uma coluna de **números**, em ordem crescente (do menor para o maior).

No exemplo ao lado, observe que a coluna SALARY está ordenada do menor salário para o maior salário, ou seja, em ordem crescente (ASC).

```
-- 1. Ordenando os dados de uma tabela
-- 1.1. Comando ORDER BY com coluna de NÚMEROS

-- a) Faça uma ordenação na tabela EMPLOYEES para visualizar os funcionários do menor
-- para o maior salário.

SELECT * FROM employees
ORDER BY salary;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,011 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	132	TJ	Olson	TJOLSON	650.124.8234	10/04/99	ST_CLERK	2100	(null)	121	5
2	128	Steven	Markle	SMARKLE	650.124.1434	08/03/00	ST_CLERK	2200	(null)	120	5
3	136	Hazel	Philtanker	HPHILTAN	650.127.1634	06/02/00	ST_CLERK	2200	(null)	122	5
4	127	James	Landry	JLANDRY	650.124.1334	14/01/99	ST_CLERK	2400	(null)	120	5
5	135	Ki	Gee	KGEE	650.127.1734	12/12/99	ST_CLERK	2400	(null)	122	5
6	119	Karen	Colmenares	KCOLMENA	515.127.4566	10/08/99	PU_CLERK	2500	(null)	114	3
7	131	James	Marlow	JAMRLOW	650.124.7234	16/02/97	ST_CLERK	2500	(null)	121	5
8	140	Joshua	Patel	JPATEL	650.121.1834	06/04/98	ST_CLERK	2500	(null)	123	5
9	144	Peter	Vargas	PVARGAS	650.121.2004	09/07/98	ST_CLERK	2500	(null)	124	5
10	182	Martha	Sullivan	MSULLIVA	650.507.9878	21/06/99	SH_CLERK	2500	(null)	120	5
11	191	Randall	Perkins	RPERKINS	650.505.4876	19/12/99	SH_CLERK	2500	(null)	122	5

ORDER BY ASC: Aplicação

Seguindo a mesma lógica, podemos ordenar uma coluna de **textos** em ordem ascendente (alfabética).

No exemplo ao lado, ordenamos a tabela EMPLOYEES de acordo com a coluna de nome.

Observe que a coluna FIRST_NAME está ordenando os textos de A até Z.

```
-- a) Ordene a tabela de EMPLOYEES para mostrar os funcionários em ordem alfabética.

SELECT * FROM employees
ORDER BY first_name;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,007 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	12	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	10
2	19	Alana	Walsh	AWALSH	650.507.9811	24/04/98	SH_CLERK	3100	(null)	12
3	14	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	10
4	10	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	10
5	11	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	(null)	11
6	18	Alexis	Bull	ABULL	650.509.2876	20/02/97	SH_CLERK	4100	(null)	12
7	15	Allan	McEwen	AMCEWEN	011.44.1345.829268	01/08/96	SA_REP	9000	0,35	14
8	17	Alyssa	Hutton	AHUTTON	011.44.1644.429266	19/03/97	SA_REP	8800	0,25	14
9	16	Amit	Banda	ABANDA	011.44.1346.729268	21/04/00	SA_REP	6200	0,1	14
10	18	Anthony	Cabrio	ACABRIO	650.509.4876	07/02/99	SH_CLERK	3000	(null)	12
11	19	Britney	Everett	BEVERETT	650.501.2876	03/03/97	SH_CLERK	3900	(null)	12
12	10	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	10
13	17	Charles	Johnson	CJOHNSON	011.44.1644.429262	04/01/00	SA_REP	6200	0,1	14
14	15	Christopher	Olsen	COLSEN	011.44.1344.498718	30/03/98	SA_REP	8000	0,2	14
15	16	Clara	Vishney	CVISHNEY	011.44.1346.129268	11/11/97	SA_REP	10500	0,25	14
16	14	Curtis	Davies	CDAVIES	650.121.2994	29/01/97	ST_CLERK	3100	(null)	12
17	10	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	10

ORDER BY ASC: Aplicação

Por fim, podemos ordenar uma coluna de **datas** em ordem ascendente (data mais antiga para a mais recente).

No exemplo ao lado, ordenamos a tabela EMPLOYEES de acordo com a coluna hire_date (data de contratação).

Observe que a coluna HIRE_DATE mostra as datas da mais antiga para a mais recente.

```
-- a) Ordene a tabela de EMPLOYEES para mostrar os funcionários de acordo com a data de
-- contratação: da mais antiga pra mais recente.

SELECT * FROM employees
ORDER BY hire_date;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,006 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEP
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	
2	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	
3	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	
6	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	
7	203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	(null)	101	
8	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	
9	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	
10	206	William	Gietz	WGIETZ	515.123.8181	07/06/94	AC_ACCOUNT	8300	(null)	205	
11	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	
12	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	
13	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	
14	122	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	(null)	100	
15	115	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	(null)	114	
16	137	Renske	Ladwig	RLADWIG	650.121.1234	14/07/95	ST_CLERK	3600	(null)	123	

ORDER BY DESC: Aplicação

Agora vamos ver como fica a ordenação contrária: uma ordenação DESC (descendente).

Podemos selecionar a tabela EMPLOYEES e ordenar de acordo com a coluna salary, que é uma coluna de **números**, em ordem decrescente (do maior para o menor).

No exemplo ao lado, observe que a coluna SALARY está ordenada do maior salário para o menor salário, ou seja, em ordem decrescente (DESC).

```
-- b) Faça uma ordenação na tabela EMPLOYEES para visualizar os funcionários do maior
-- para o menor salário.

SELECT * FROM employees
ORDER BY salary DESC;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,011 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100
4	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100
5	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100
6	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100
7	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101
8	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100
9	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101
10	168	Lisa	Ozer	LOZER	011.44.1343.929268	11/03/97	SA_REP	11500	0,25	148
11	174	Ellen	Abel	EABEL	011.44.1644.429267	11/05/96	SA_REP	11000	0,3	149
12	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100
13	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100
14	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29/01/00	SA_MAN	10500	0,2	100
15	162	Clara	Vishney	CVISHNEY	011.44.1346.129268	11/11/97	SA_REP	10500	0,25	147

ORDER BY DESC: Aplicação

Seguindo a mesma lógica, podemos ordenar uma coluna de **textos** em ordem decrescente (“anti”-alfabética).

No exemplo ao lado, ordenamos a tabela EMPLOYEES de acordo com a coluna de nome.

Observe que a coluna FIRST_NAME está ordenando os textos de Z até A.

```
-- b) Ordene a tabela de EMPLOYEES para mostrar os funcionários em ordem "anti"-alfabética.

SELECT * FROM employees
ORDER BY first_name DESC;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,007 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEP
1	180	Winston	Taylor	WTAYLOR	650.507.9876	24/01/98	SH_CLERK	3200	(null)	120	
2	171	William	Smith	WSMITH	011.44.1343.629268	23/02/99	SA_REP	7400	0,15	148	
3	206	William	Gietz	WGIETZ	515.123.8181	07/06/94	AC_ACCOUNT	8300	(null)	205	
4	195	Vance	Jones	VJONES	650.501.4876	17/03/99	SH_CLERK	2800	(null)	123	
5	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	
6	141	Trenna	Rajs	TRAJS	650.121.8009	17/10/95	ST_CLERK	3500	(null)	124	
7	132	TJ	Olson	TJOLSON	650.124.8234	10/04/99	ST_CLERK	2100	(null)	121	
8	190	Timothy	Gates	TGATES	650.505.3876	11/07/98	SH_CLERK	2900	(null)	122	
9	170	Tayler	Fox	TFOX	011.44.1343.729268	24/01/98	SA_REP	9600	0,2	148	
10	203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	(null)	101	
11	173	Sundita	Kumar	SKUMAR	011.44.1343.329268	21/04/00	SA_REP	6100	0,1	148	
12	166	Sundar	Ande	SANDE	011.44.1346.629268	24/03/00	SA_REP	6400	0,1	147	
13	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	
14	128	Steven	Markle	SMARKLE	650.124.1434	08/03/00	ST_CLERK	2200	(null)	120	
15	138	Stephen	Stiles	SSTILES	650.121.2034	26/10/97	ST_CLERK	3200	(null)	123	

ORDER BY DESC: Aplicação

Por fim, podemos ordenar uma coluna de **datas** em ordem decrescente (data mais recente para a mais antiga).

No exemplo ao lado, ordenamos a tabela EMPLOYEES de acordo com a coluna hire_date (data de contratação).

Observe que a coluna HIRE_DATE mostra as datas da mais recente para a mais antiga, ou seja, em ordem descendente.

```
-- b) Ordene a tabela de EMPLOYEES para mostrar os funcionários de acordo com a data de
-- contratação: da mais recente pra mais antiga.

SELECT * FROM employees
ORDER BY hire_date DESC;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,01 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	173	Sundita	Kumar	SKUMAR	011.44.1343.329268	21/04/00	SA_REP	6100	0,1	14
2	167	Amit	Banda	ABANDA	011.44.1346.729268	21/04/00	SA_REP	6200	0,1	14
3	166	Sundar	Ande	SANDE	011.44.1346.629268	24/03/00	SA_REP	6400	0,1	14
4	128	Steven	Markle	SMARKLE	650.124.1434	08/03/00	ST_CLERK	2200	(null)	12
5	165	David	Lee	DLEE	011.44.1346.529268	23/02/00	SA_REP	6800	0,1	14
6	136	Hazel	Philtanker	HPHILTAN	650.127.1634	06/02/00	ST_CLERK	2200	(null)	12
7	183	Girard	Geoni	GGEONI	650.507.9879	03/02/00	SH_CLERK	2800	(null)	12
8	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29/01/00	SA_MAN	10500	0,2	10
9	164	Mattea	Marvins	MMARVINS	011.44.1346.329268	24/01/00	SA_REP	7200	0,1	14
10	199	Douglas	Grant	DGRANT	650.507.9844	13/01/00	SH_CLERK	2600	(null)	12
11	179	Charles	Johnson	CJOHNSON	011.44.1644.429262	04/01/00	SA_REP	6200	0,1	14
12	191	Randall	Perkins	RPERKINS	650.505.4876	19/12/99	SH_CLERK	2500	(null)	12
13	135	Ki	Gee	KGEE	650.127.1734	12/12/99	ST_CLERK	2400	(null)	12
14	113	Luis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	(null)	10
15	155	Oliver	Tuvault	OTUVAULT	011.44.1344.486508	23/11/99	SA_REP	7000	0,15	14

ORDER BY: NULLS FIRST e NULLS LAST

Também é possível ordenar uma coluna de forma que os valores nulos (NULL) sejam mostrados no começo ou no final da coluna.

- **SELECT * FROM Tabela ORDER BY coluna1 NULLS FIRST;**

Seleciona a tabela ordenando pela coluna1 em ordem crescente, de forma que os valores nulos apareçam nas primeiras linhas.

- **SELECT * FROM Tabela ORDER BY coluna1 NULLS LAST;**

Seleciona a tabela ordenando pela coluna1 em ordem crescente, de forma que os valores nulos apareçam nas últimas linhas.

- **SELECT * FROM Tabela ORDER BY coluna1 DESC NULLS FIRST;**

Seleciona a tabela ordenando pela coluna1 em ordem decrescente, de forma que os valores nulos apareçam nas primeiras linhas.

- **SELECT * FROM Tabela ORDER BY coluna1 DESC NULLS LAST;**

Seleciona a tabela ordenando pela coluna1 em ordem decrescente, de forma que os valores nulos apareçam nas últimas linhas.

NULLS FIRST e NULLS LAST: Aplicação

A aplicação do NULLS FIRST é bem simples.

No exemplo ao lado, usamos o NULLS FIRST para ordenar a coluna manager_id da tabela DEPARTMENTS para que os valores (null) apareçam primeiro na coluna.

```
-- a) A coluna MANAGER_ID da tabela DEPARTMENTS possui valores nulos. Faça uma ordenação dessa
-- coluna utilizando NULLS FIRST e NULLS LAST.

SELECT * FROM departments
ORDER BY manager_id NULLS FIRST;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 27 em 0,022 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	140	Control And Credit	(null)	1700
2	260	Recruiting	(null)	1700
3	250	Retail Sales	(null)	1700
4	240	Government Sales	(null)	1700
5	230	IT Helpdesk	(null)	1700
6	220	NOC	(null)	1700
7	210	IT Support	(null)	1700
8	200	Operations	(null)	1700
9	190	Contracting	(null)	1700
10	180	Construction	(null)	1700
11	170	Manufacturing	(null)	1700
12	120	Treasury	(null)	1700
13	130	Corporate Tax	(null)	1700
14	270	Payroll	(null)	1700
15	150	Shareholder Services	(null)	1700

NULLS FIRST e NULLS LAST: Aplicação

Já o NULLS LAST será usado para ordenar uma coluna de forma que os valores (null) apareçam por último.

No exemplo ao lado, usamos o NULLS LAST para ordenar a coluna manager_id da tabela DEPARTMENTS para que os valores (null) apareçam por último na coluna.

```
-- a) A coluna MANAGER_ID da tabela DEPARTMENTS possui valores nulos. Faça uma ordenação dessa
-- coluna utilizando NULLS FIRST e NULLS LAST.

SELECT * FROM departments
ORDER BY manager_id NULLS LAST;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 27 em 0,006 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	90	Executive	100	1700
2	60	IT	103	1400
3	100	Finance	108	1700
4	30	Purchasing	114	1700
5	50	Shipping	121	1500
6	80	Sales	145	2500
7	10	Administration	200	1700
8	20	Marketing	201	1800
9	40	Human Resources	203	2400
10	70	Public Relations	204	2700
11	110	Accounting	205	1700
12	230	IT Helpdesk	(null)	1700
13	240	Government Sales	(null)	1700
14	250	Retail Sales	(null)	1700
15	260	Recruiting	(null)	1700
16	220	NOC	(null)	1700

NULLS FIRST e NULLS LAST: Aplicação

Podemos também combinar o NULLS FIRST/NULLS LAST com o DESC, para escolher a forma de ordenação para descendente.

Por padrão, NULLS FIRST/NULLS LAST sozinhos ordenarão os valores em ordem ascendente.

Para ordenar os valores em ordem descendente, incluímos o DESC logo antes do NULLS FIRST/NULLS LAST, como mostrado na imagem ao lado.

O exemplo ao lado é bem semelhante ao do slide anterior (volte ao slide anterior para comparar), com a diferença que a coluna manager_id da tabela DEPARTMENTS agora é organizada em ordem decrescente.

```
-- a) A coluna MANAGER_ID da tabela DEPARTMENTS possui valores nulos. Faça uma ordenação dessa
-- coluna utilizando NULLS FIRST e NULLS LAST.

SELECT * FROM departments
ORDER BY manager_id DESC NULLS LAST;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 27 em 0,048 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	110	Accounting	205	1700
2	70	Public Relations	204	2700
3	40	Human Resources	203	2400
4	20	Marketing	201	1800
5	10	Administration	200	1700
6	80	Sales	145	2500
7	50	Shipping	121	1500
8	30	Purchasing	114	1700
9	100	Finance	108	1700
10	60	IT	103	1400
11	90	Executive	100	1700
12	230	IT Helpdesk	(null)	1700
13	240	Government Sales	(null)	1700
14	250	Retail Sales	(null)	1700
15	260	Recruiting	(null)	1700
16	220	NOC	(null)	1700

FETCH NEXT: Limitando linhas da tabela

O FETCH NEXT vai permitir que a gente escolha a quantidade de linhas a serem retornadas de uma tabela.

Aplicação:

a) Retornando as TOP N linhas:

FETCH NEXT N ROWS ONLY

```
SELECT *
FROM employees
ORDER BY salary DESC
FETCH NEXT 10 ROWS ONLY;
```

Exemplo:

Retorne as 10 primeiras linhas da tabela EMPLOYEES, ordenadas pela coluna salary em ordem decrescente (do maior para o menor).

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 10 em 0,122 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
5	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80
6	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
7	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
8	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
9	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	80
10	168	Lisa	Ozer	LOZER	011.44.1343.929268	11/03/97	SA_REP	11500	0,25	148	80

FETCH NEXT: Limitando linhas da tabela

O FETCH NEXT vai permitir que a gente escolha a quantidade de linhas a serem retornadas de uma tabela.

Aplicação:

b) Retornando TOP N linhas, incluindo os empatados:

FETCH NEXT N ROWS WITH TIES

Exemplo:

Retorne as 12 primeiras linhas da tabela EMPLOYEES, ordenadas pela coluna salary em ordem decrescente (do maior para o menor). Caso haja empate, retorne todos os empates.

```
SELECT *
FROM employees
ORDER BY salary DESC
FETCH NEXT 12 ROWS WITH TIES;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 13 em 0,002 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
5	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80
6	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
7	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
8	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
9	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	80
10	168	Lisa	Ozer	LOZER	011.44.1343.929268	11/03/97	SA_REP	11500	0,25	148	80
11	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30
12	174	Ellen	Abel	EABEL	011.44.1644.429267	11/05/96	SA_REP	11000	0,3	149	80
13	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100	80

Retornou 13 (não 12), pois o 13° empatou com o 12°.

FETCH NEXT: Limitando linhas da tabela

O FETCH NEXT vai permitir que a gente escolha a quantidade de linhas a serem retornadas de uma tabela.

Aplicação:

c) Limitando por um percentual de linhas:

FETCH NEXT N PERCENT ROWS ONLY

```
SELECT *
FROM employees
ORDER BY salary DESC
FETCH NEXT 10 PERCENT ROWS ONLY;
```

Exemplo:

Retorne os 10% dos funcionários com salário mais alto.

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
5	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80
6	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
7	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
8	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	80
9	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
10	168	Lisa	Ozer	LOZER	011.44.1343.929268	11/03/97	SA_REP	11500	0,25	148	80
11	174	Ellen	Abel	EABEL	011.44.1644.429267	11/05/96	SA_REP	11000	0,3	149	80

Retornou 11 dos 107 funcionários (10 %) com o salário mais alto.

FETCH NEXT: Limitando linhas da tabela

O FETCH NEXT vai permitir que a gente escolha a quantidade de linhas a serem retornadas de uma tabela.

Aplicação:

d) Criando um OFFSET (deslocamento) de linhas

OFFSET N ROWS
FETCH NEXT N ROWS ONLY

```
SELECT *
FROM employees
ORDER BY employee_id
OFFSET 5 ROWS
FETCH NEXT 10 ROWS ONLY;
```

Exemplo:

Retorne os 10 primeiros funcionários logo após os 5 primeiros (ordenando por employee_id). Ou seja, retorne do 6º ao 15º funcionário cadastrado.

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 10 em 0,002 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
2	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
3	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60
4	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
5	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
6	110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)	108	100
7	111	Ismael	Sciarra	ISCIARRA	515.124.4369	30/09/97	FI_ACCOUNT	7700	(null)	108	100
8	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	(null)	108	100
9	113	Luis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	(null)	108	100
10	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30

Pulou os funcionários com employee_id 100 a 104, retornando do 105 ao 114.

WHERE: Introdução

O comando WHERE nos permite criar filtros nas tabelas dos bancos de dados.

Imagine que tenhamos a tabela ao lado. Ela possui 4 colunas, dentre elas a coluna JOB_ID, que identifica qual é o ID do cargo de cada funcionário.

Nessa tabela temos todos os funcionários, incluindo todos os cargos. Mas e se a gente quisesse analisar apenas os funcionários com JOB_ID igual a IT_PROG?

Neste caso, não faria sentido a gente ter todos os dados na tabela, precisaríamos apenas dos funcionários com JOB_ID='IT_PROG'.

Para isso, teríamos que criar um filtro na nossa coluna JOB_ID.

EMPLOYEE_ID	FIRST_NAME	JOB_ID	SALARY
100	Steven	AD_PRES	24000
101	Neena	AD_VP	17000
102	Lex	AD_VP	17000
103	Alexander	IT_PROG	9000
104	Bruce	IT_PROG	6000
105	David	IT_PROG	4800
106	Valli	IT_PROG	4800
107	Diana	IT_PROG	4200
108	Nancy	FI_MGR	12000
109	Daniel	FI_ACCOUNT	9000
110	John	FI_ACCOUNT	8200
111	Ismael	FI_ACCOUNT	7700
112	Jose Manuel	FI_ACCOUNT	7800
113	Luis	FI_ACCOUNT	6900
114	Den	PU_MAN	11000
115	Alexander	PU_CLERK	3100
116	Shelli	PU_CLERK	2900
117	Sigal	PU_CLERK	2800
118	Guy	PU_CLERK	2600
119	Karen	PU_CLERK	2500
120	Matthew	ST_MAN	8000
121	Adam	ST_MAN	8200
122	Payam	ST_MAN	7900

WHERE: Introdução

Observe que, de todos os funcionários da tabela EMPLOYEES, temos apenas 5 que são IT_PROG.

Conseguimos agora visualizar apenas os dados que nos interessam, deixando a tabela muito mais simplificada e de fácil entendimento.

Para conseguir chegar a esse resultado, temos que fazer um filtro na nossa tabela. Este filtro conseguiremos fazer utilizando o comando **WHERE**.

O WHERE nos permitirá fazer uma série de filtros nas nossas tabelas, desde os mais básicos até os mais avançados.

Vamos então aprender a usar estes filtros na prática.

EMPLOYEE_ID	FIRST_NAME	JOB_ID	SALARY
103	Alexander	IT_PROG	9000
104	Bruce	IT_PROG	6000
105	David	IT_PROG	4800
106	Valli	IT_PROG	4800
107	Diana	IT_PROG	4200

WHERE: Sintaxe

A sintaxe do WHERE é bem simples e é mostrada na figura 1 a lado.

Fazemos um SELECT FROM normal e ao final adicionamos o WHERE seguido de uma **condição**.

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE condicao;
```

1

Por exemplo, para chegar à tabela do slide anterior, precisamos filtrar a nossa tabela EMPLOYEES para que mostrasse apenas os funcionários cujo JOB_ID é igual a 'IT_PROG'.

Para isso, usamos o código ao lado, onde a nossa **condição** é **JOB_ID = 'IT_PROG'**. (figura 2)

```
SELECT employee_id, first_name, job_id, salary
FROM employees
WHERE job_id = 'IT_PROG';
```

2

Operadores básicos de filtros

Podemos utilizar o WHERE em conjunto com uma série de operadores, listados a seguir.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

Operadores básicos de filtros

Vamos começar com os operadores básicos listados a seguir.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

WHERE

A forma mais básica de usar o WHERE é para filtrar colunas dos 3 tipos a seguir.

NÚMEROS

STRINGS

DATAS

WHERE: Filtros de Números

Nossa primeira aplicação do WHERE será com filtros de **números**. De acordo com a tabela anterior, temos diversos comparadores que podemos usar: =, <, >, <=, >=, <>.

No exemplo ao lado, filtramos a tabela EMPLOYEES para mostrar apenas os funcionários que são do departamento igual a 100.

```
-- a) Filtre a tabela EMPLOYEES e retorne apenas
-- os funcionários do departamento ID = 100

SELECT employee_id, first_name, salary, department_id
FROM employees
WHERE department_id = 100;

-- b) Filtre a tabela JOBS e retorne aqueles que tem um sal
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 6 em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_ID
1	108	Nancy	12000	100
2	109	Daniel	9000	100
3	110	John	8200	100
4	111	Ismael	7700	100
5	112	Jose Manuel	7800	100
6	113	Luis	6900	100

WHERE: Filtros de Números

Já no exemplo ao lado, filtramos a tabela JOBS para mostrar apenas os cargos com salário mínimo maior que 6.000.

```
-- b) Filtre a tabela JOBS e retorne aqueles que
-- tem um salário mínimo maior que 6000

SELECT *
FROM jobs
WHERE min_salary > 6000;


-- c) Filtre a tabela EMPLOYEES para retornar apenas o job_id = ST_MAN

SELECT
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 7 em 0,022 segundos

	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
1	AD_PRES	President	20000	40000
2	AD_VP	Administration Vice President	15000	30000
3	FI_MGR	Finance Manager	8200	16000
4	AC_MGR	Accounting Manager	8200	16000
5	SA_MAN	Sales Manager	10000	20000
6	PU_MAN	Purchasing Manager	8000	15000
7	MK_MAN	Marketing Manager	9000	15000

WHERE: Filtros de Texto

Para filtrar colunas de **texto**, basta seguir o exemplo ao lado.

Queremos todos os job_id iguais a 'ST_MAN'.

Observe que o texto ST_MAN é escrito entre aspas simples. Sempre que quisermos filtrar uma tabela a partir de uma coluna de textos, a palavra deve estar entre aspas simples.

OBS: o Oracle é case sensitive, ou seja:

- **WHERE job_id = 'ST_MAN'**

e

- **WHERE job_id = 'st_man'**

são coisas diferentes.

```
-- c) Filtre a tabela EMPLOYEES para retornar apenas o job_id = ST_MAN

SELECT
    first_name,
    last_name,
    salary,
    job_id
FROM employees
WHERE job_id = 'ST_MAN';
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 5 em 0,009 segundos

	FIRST_NAME	LAST_NAME	SALARY	JOB_ID
1	Matthew	Weiss	8000	ST_MAN
2	Adam	Fripp	8200	ST_MAN
3	Payam	Kaufling	7900	ST_MAN
4	Shanta	Vollman	6500	ST_MAN
5	Kevin	Mourgos	5800	ST_MAN

WHERE: Filtros de Data

Para filtrar colunas de **data**, também colocamos essa data entre aspas simples, como se fosse um texto.

No exemplo ao lado, estamos retornando todos os funcionários da tabela EMPLOYEES que têm uma data de contratação maior ou igual a '01/01/2000'.

```
-- d) Filtre a tabela EMPLOYEES para mostrar apenas os funcionários que foram contratados
-- após o dia 01/01/00

SELECT
    *
FROM employees
WHERE hire_date >= '01/01/00';
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,011 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID
1	128	Steven	Markle	SMARKLE	650.124.1434	08/03/00	ST_CLERK	2200	(null)	120
2	136	Hazel	Philtanker	HPHILTAN	650.127.1634	06/02/00	ST_CLERK	2200	(null)	122
3	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29/01/00	SA_MAN	10500	0,2	100
4	164	Mattea	Marvins	MMARVINS	011.44.1346.329268	24/01/00	SA_REP	7200	0,1	147
5	165	David	Lee	DLEE	011.44.1346.529268	23/02/00	SA_REP	6800	0,1	147
6	166	Sundar	Ande	SANDE	011.44.1346.629268	24/03/00	SA_REP	6400	0,1	147
7	167	Amit	Banda	ABANDA	011.44.1346.729268	21/04/00	SA_REP	6200	0,1	147
8	173	Sundita	Kumar	SKUMAR	011.44.1343.329268	21/04/00	SA_REP	6100	0,1	148
9	179	Charles	Johnson	CJOHNSON	011.44.1644.429262	04/01/00	SA_REP	6200	0,1	149
10	183	Girard	Geoni	GGEONI	650.507.9879	03/02/00	SH_CLERK	2800	(null)	120

Operadores AND, OR e NOT

Vamos agora usar os operadores AND, OR e NOT.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

Operador AND

Até agora, vimos como fazer filtros usando um critério em apenas uma coluna (department_id=100, salary>6000, job_id='ST_MAN').

O que fazemos quando precisamos filtrar a partir de mais de uma coluna?
Imagine que você queira uma lista com todos os funcionários com job_id='IT_PROG', mas especificamente aqueles que tem o salary>=5000. Neste caso, precisamos filtrar a partir de duas colunas: **job_id** e **salary**. A solução é muito simples, basta usarmos o operador AND para fazer estes múltiplos testes, como mostrado na imagem abaixo.

```
-- 1.2. Filtro WHERE combinado com AND e OR.

-- a) Quais funcionários têm o JOB_ID = 'IT_PROG' e SALARY >= 5000?

SELECT * FROM employees
WHERE job_id = 'IT_PROG' AND salary >= 5000;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 2 em 0,14 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
2	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60

Operador OR

Agora observe a seguinte situação: queremos todos os funcionários que são do **department_id** igual a 90 ou igual a 100. Como resolver nesse caso?

O AND não vai nos ajudar porque ele pressupõe que todos os testes sejam verdade para que as linhas da tabela sejam retornadas. Só que dessa vez, basta que uma das duas condições sejam atendidas (department_id=90 OU department_id=100) e pra gente já é o suficiente. Se as duas forem verdadeiras então, melhor ainda. O código e o resultado são mostrados abaixo.

```
-- b) Quais funcionários são do departamento 90 ou 100?

SELECT * FROM employees
WHERE department_id = 90 OR department_id = 100;


-- 1.3. Filtros LIKE, BETWEEN, IN, IS NULL e NOT.

-- a) Quais funcionários possuem um JOB_ID que começam com o texto: 'ST'?
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 9 em 0,013 segundos

	LOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
5	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
6	110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)	108	100
7	111	Ismael	Sciarra	ISCIARRA	515.124.4369	30/09/97	FI_ACCOUNT	7700	(null)	108	100
8	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	(null)	108	100
9	113	Luis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	(null)	108	100

Operador NOT

O operador NOT é um operador de negação, que sempre vai trazer o oposto do que estiver no filtro. Por exemplo, se quisermos todos os funcionários que são do **department_id** que não seja o 90, podemos usar a solução da imagem abaixo (também poderíamos usar o operador <> para fazer o filtro de 'diferente de', mas com o NOT conseguiremos negar qualquer tipo de filtro, como veremos em aplicações mais a frente).

```
-- b) Quais funcionários são do departamento 90 ou 100?

SELECT * FROM employees
WHERE NOT department_id = 90
ORDER BY department_id DESC;


-- 1.3. Filtros LIKE, BETWEEN, IN, IS NULL e NOT.
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,03 segundos

	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	nelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
2	illiam	Gietz	WGIETZ	515.123.8181	07/06/94	AC_ACCOUNT	8300	(null)	205	110
3	ohn	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)	108	100
4	ose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	(null)	108	100
5	uis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	(null)	108	100
6	aniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
7	ancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
8	smael	Sciarra	ISCIARRA	515.124.4369	30/09/97	FI_ACCOUNT	7700	(null)	108	100
9	ohn	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
10	aren	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80

Operador LIKE e Wildcards

Vamos agora usar o operador LIKE e os Wildcards.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

Operador LIKE e Wildcards

O operador LIKE é usado junto com o WHERE para procurar por algum padrão específico de texto em uma coluna.

Associado ao LIKE, usamos também os chamados *wildcards* (caracteres curinga) que vão dar essa flexibilidade de busca por padrões de texto.

Existem dois *wildcards* frequentemente usados junto com o LIKE:

- O sinal de porcentagem (%) representa zero, um ou múltiplos caracteres.
- O underline (_) um único caractere

Podemos usar tanto o % quando o _ em conjunto para criar filtros ainda mais personalizados.

A sintaxe do LIKE é mostrada na imagem ao lado.

```
SELECT coluna1, coluna2 ...
FROM tabela
WHERE colunaN LIKE padrao;
```

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN NOT LIKE padrao;
```

Operador LIKE e Wildcards

Vamos a alguns exemplos mostrando os diferentes casos do operador LIKE em conjunto com os *wildcards* '%' e '_'.

Aplicação do LIKE	Descrição
WHERE first_name LIKE 'a%'	Encontra valores que começam com "a"
WHERE first_name LIKE '%a'	Encontra valores que terminam com "a"
WHERE first_name LIKE '%ao%'	Encontra valores que têm o "ao" em qualquer posição do texto
WHERE first_name LIKE '_a%'	Encontra valores que começam com uma letra qualquer e que têm a letra "a" na segunda posição
WHERE first_name LIKE 'a_%'	Encontra valores que começam com a letra "a" e têm pelo menos 2 caracteres
WHERE first_name LIKE 'a__%'	Encontra valores que começam com a letra "a" e têm pelo menos 3 caracteres
WHERE first_name LIKE 'a%o'	Encontra valores que começam com a letra "a" e terminam com a letra "o"

Veremos na prática.

Operador LIKE e Wildcards

Comecemos com o exemplo abaixo.

Vamos retornar a lista de funcionários que possuem um JOB_ID **que começa** com o texto 'ST'.

De acordo com o que vimos na tabela anterior, a solução é mostrada abaixo:

```
-- a) Quais funcionários possuem um JOB_ID que começam com o texto: 'ST'?

SELECT * FROM employees
WHERE job_id LIKE 'ST%';
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 25 em 0,052 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	125	Julia	Nayer	JNAYER	650.124.1214	16/07/97	ST_CLERK	3200	(null)	120	50
2	126	Irene	Mikkilineni	IMIKKILI	650.124.1224	28/09/98	ST_CLERK	2700	(null)	120	50
3	127	James	Landry	JLANDRY	650.124.1334	14/01/99	ST_CLERK	2400	(null)	120	50
4	128	Steven	Markle	SMARKLE	650.124.1434	08/03/00	ST_CLERK	2200	(null)	120	50
5	129	Laura	Bissot	LBISSOT	650.124.5234	20/08/97	ST_CLERK	3300	(null)	121	50
6	130	Mozhe	Atkinson	MATKINSO	650.124.6234	30/10/97	ST_CLERK	2800	(null)	121	50
7	131	James	Marlow	JAMRLOW	650.124.7234	16/02/97	ST_CLERK	2500	(null)	121	50
8	132	TJ	Olson	TJOLSON	650.124.8234	10/04/99	ST_CLERK	2100	(null)	121	50
9	133	Jason	Mallin	JMALLIN	650.127.1934	14/06/96	ST_CLERK	3300	(null)	122	50
10	134	Michael	Rogers	MROGERS	650.127.1834	26/08/98	ST_CLERK	2900	(null)	122	50
11	135	Ki	Gee	KGEE	650.127.1734	12/12/99	ST_CLERK	2400	(null)	122	50

Operador LIKE e Wildcards

Neste outro exemplo, queremos todos os funcionários que têm o número '123' na segunda parte do telefone. Como o padrão dos números é sempre ter 3 caracteres no começo + ponto + a segunda parte + ponto + 4 caracteres, podemos usar o *wildcard* _ , como mostrado na imagem abaixo. Ou seja, começamos repetindo o _ 4 vezes no começo, e depois do 123, repetimos mais 5 vezes (lembrando que o ponto também conta como um caractere).

```
-- c) O telefone é formado por 3 partes: AAA.BBB.CCCC. Quais telefones têm a 2ª parte igual a 123?

SELECT * FROM employees
WHERE phone_number LIKE '____123_____';
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 15 em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	120	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	(null)	100	50
5	121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	50
6	122	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	(null)	100	50
7	123	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	(null)	100	50
8	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16/11/99	ST_MAN	5800	(null)	100	50
9	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
10	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20

Índice Geral

Índice deste Módulo

Operador LIKE e Wildcards

No exemplo anterior você pode ter ficado com a seguinte dúvida: por que não usar o '%123%' pra encontrar a sequência de números no meio?

Lembre-se que o *wildcard* % significa zero, um ou múltiplos caracteres. Então, quando colocamos % no começo e % no fim, estamos dizendo que o 123 pode estar em qualquer lugar dentro do texto.

Se usássemos o '%123%', observe na linha 9 o funcionário Renske. A segunda parte do telefone dele não é 123 e sim 121. E porque ele retornou essa linha? Porque na terceira parte, a sequência 123 está presente (...1234). Por isso essa linha foi retornada.

Portanto, avalie a situação e veja qual dos *wildcards* fazem mais sentido de usar: o % ou o _ .

```
-- c) O telefone é formado por 3 partes: AAA.BBB.CCCC. Quais telefones têm a 2ª parte igual a 123?

SELECT * FROM employees
WHERE phone_number LIKE '%123%';
```

Resultado da Consulta

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER
1	100	Steven	King	SKING	515.123.4567
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568
3	102	Lex	De Haan	LDEHAAN	515.123.4569
4	120	Matthew	Weiss	MWEISS	650.123.1234
5	121	Adam	Fripp	AFRIPP	650.123.2234
6	122	Payam	Kaufling	PKAUFLIN	650.123.3234
7	123	Shanta	Vollman	SVOLLMAN	650.123.4234
8	124	Kevin	Mourgos	KMOURGOS	650.123.5234
9	137	Renske	Ladwig	RLADWIG	650.121.1234
10	200	Jennifer	Whalen	JWHALEN	515.123.4444

Operador BETWEEN

Vamos agora usar o operador BETWEEN.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

Operador BETWEEN

O operador BETWEEN é usado junto com o WHERE para selecionar valores dentro de um intervalo. Esses valores podem ser números, textos ou datas.

O operador BETWEEN é inclusivo, ou seja, os valores do começo e do fim do intervalo também estão incluídos.

A sintaxe desse operador é mostrada ao lado.

Vamos agora ver alguns exemplos.

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN BETWEEN valor1 AND valor2;
```

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN NOT BETWEEN valor1 AND valor2;
```

Operador BETWEEN: números

Neste exemplo queremos selecionar todos os funcionários que têm um salário entre 10 mil e 30 mil. A aplicação do BETWEEN é bem intuitiva.

```
-- a) Selecione os funcionários que recebam um salário ENTRE 10.000 e 30.000.

SELECT * FROM employees
WHERE salary BETWEEN 10000 AND 30000;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,008 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMEN
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	
4	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	
5	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	
6	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	
7	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	
8	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	
9	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100	
10	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29/01/00	SA_MAN	10500	0,2	100	

Operador BETWEEN: textos

Agora, em um caso de textos: podemos usar o BETWEEN para mostrar a lista de funcionários que têm o primeiro nome começando em 'A' até os funcionários que têm o primeiro nome começando em 'D'.

Lembre-se que o Oracle é case sensitive, então como os nomes começam com letra maiúscula, as letras A e D do filtro também devem estar em maiúscula.

```
-- b) Selecione os funcionários que têm nomes começando com a letra 'A' até a letra 'D'.

SELECT * FROM employees
WHERE first_name BETWEEN 'A' AND 'D';
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 16 em 0,012 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEP
1	16	Amit	Banda	ABANDA	011.44.1346.729268	21/04/00	SA_REP	6200	0,1	147	
2	18	Alexis	Bull	ABULL	650.509.2876	20/02/97	SH_CLERK	4100	(null)	121	
3	18	Anthony	Cabrio	ACABRIO	650.509.4876	07/02/99	SH_CLERK	3000	(null)	121	
4	14	Curtis	Davies	CDAVIES	650.121.2994	29/01/97	ST_CLERK	3100	(null)	124	
5	10	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	
6	14	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	
7	19	Britney	Everett	BEVERETT	650.501.2876	03/03/97	SH_CLERK	3900	(null)	123	
8	12	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	
9	10	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	
10	17	Alyssa	Hutton	AHUTTON	011.44.1644.429266	19/03/97	SA_REP	8800	0,25	149	

Operador BETWEEN: datas

Também podemos usar o BETWEEN com datas. Aqui, queremos retornar todos os funcionários que foram contratados entre os dias '01/01/1999' e '31/12/2000'.

```
-- c) Selecione os funcionários que foram contratados entre as datas '01/01/99' e 31/12/00'

SELECT * FROM employees
WHERE hire_date BETWEEN '01/01/99' AND '31/12/00';
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 29 em 0,004 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	D
1	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	
2	113	Luis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	(null)	108	
3	119	Karen	Colmenares	KCOLMENA	515.127.4566	10/08/99	PU_CLERK	2500	(null)	114	
4	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16/11/99	ST_MAN	5800	(null)	100	
5	127	James	Landry	JLANDRY	650.124.1334	14/01/99	ST_CLERK	2400	(null)	120	
6	128	Steven	Markle	SMARKLE	650.124.1434	08/03/00	ST_CLERK	2200	(null)	120	
7	132	TJ	Olson	TJOLSON	650.124.8234	10/04/99	ST_CLERK	2100	(null)	121	
8	135	Ki	Gee	KGEE	650.127.1734	12/12/99	ST_CLERK	2400	(null)	122	
9	136	Hazel	Philtanker	HPHILTAN	650.127.1634	06/02/00	ST_CLERK	2200	(null)	122	
10	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100	

Operador IN

Vamos agora usar o operador IN.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

Operador IN

O operador IN é usado junto com o WHERE e permite especificar múltiplos valores no filtro.

A sintaxe desse operador é mostrada ao lado.

Vamos agora ver um exemplo.

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN IN (valor1, valor2, ...);
```

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN NOT IN (valor1, valor2, ...);
```

Operador IN

Queremos retornar todos os funcionários dos departamentos 30, 50 ou 80.

```
-- a) Selecione os funcionários que são de um dos seguintes departamentos: 30, 50, 80.

SELECT * FROM employees
WHERE department_id IN (30, 50, 80);
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,016 segundos

	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30
2	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	(null)	114	30
3	Shelli	Baida	SBAIDA	515.127.4563	24/12/97	PU_CLERK	2900	(null)	114	30
4	Sigal	Tobias	STOBIAS	515.127.4564	24/07/97	PU_CLERK	2800	(null)	114	30
5	Guy	Himuro	GHIMURO	515.127.4565	15/11/98	PU_CLERK	2600	(null)	114	30
6	Karen	Colmenares	KCOLMENA	515.127.4566	10/08/99	PU_CLERK	2500	(null)	114	30
7	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	(null)	100	50
8	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	50
9	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	(null)	100	50
10	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	(null)	100	50

Operador IS NULL

Vamos agora usar o operador IS NULL.

Operador	Tradução	Finalidade
=	Igual a	Filtra todos os valores que são iguais a um determinado valor.
>	Maior que	Filtra todos os valores que são maiores que um determinado valor.
<	Menor que	Filtra todos os valores que são menores que um determinado valor.
>=	Maior ou igual a	Filtra todos os valores que são maiores ou iguais a um determinado valor.
<=	Menor ou igual a	Filtra todos os valores que são menores ou iguais a um determinado valor.
<>	Diferente de	Filtra todos os valores que são diferentes de determinado valor.
AND	E	Usado para filtrar múltiplas condições usando a lógica E. O AND mostra as linhas se todas as condições forem verdadeiras.
OR	OU	Usado para filtrar múltiplas condições usando a lógica OU. O OR mostra as linhas se uma ou mais condições forem verdadeiras.
LIKE	COMO, SEMELHANTE	Usado para pesquisar um padrão especificado em uma coluna.
IN	EM	Permite especificar vários valores dentro do WHERE. Pode ser uma alternativa ao OR.
BETWEEN	ENTRE	Permite selecionar valores dentro de um determinado intervalo de forma inclusiva: os valores inicial e final são incluídos.
IS NULL	É NULO	Permite filtrar valores nulos.
NOT	NÃO	Permite fazer a negação dos operadores anteriores.

Operador IS NULL

O operador IS NULL é usado junto com o WHERE e permite filtrar valores nulos (que são diferentes de zero e diferentes de vazio).

A sintaxe desse operador é mostrada ao lado.

Vamos agora ver um exemplo.

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN IS NULL;
```

```
SELECT coluna1, coluna2, ...
FROM tabela
WHERE colunaN IS NOT NULL;
```

Operador IS NULL

Neste exemplo, filtramos a tabela EMPLOYEES para mostrar todos os funcionários que possuem um percentual de comissão nulo. Utilizamos para isso o operador IS NULL.

```
-- a) Quais funcionários têm o percentual de comissão igual a (null)?

SELECT * FROM employees
WHERE commission_pct IS NULL;

-- b) Quais funcionários têm o percentual de comissão diferente de null.
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,012 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPART
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	
6	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	
7	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	
9	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	
10	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	

Regras de precedência

Até agora vimos diversos OPERADORES aplicados juntos ao WHERE, e a dúvida que pode surgir é: quando temos mais de um sendo usado ao mesmo tempo, quem “ganha” de quem? Quem será executado primeiro?

A seguir, listamos a ordem de prioridade de cada um dos operadores:

1. Condições de comparação
2. IS NULL, LIKE, IN
3. BETWEEN
4. NOT
5. AND
6. OR

O que mais gera dúvida é quando usamos múltiplos AND e OR no mesmo filtro, que é o tipo de combinação mais comum.

Vejamos a seguir um exemplo.

Regras de precedência

No exemplo abaixo, temos TRÊS TESTES LÓGICOS. Qual você acha que será o resultado?

(1) Será que primeiro ele vai ver quais são os funcionários de job_id = 'IT_PROG' ou 'ST_MAN' e só depois vai trazer quem tem salário acima de 5000 dentre esses dois?

(2) Ou primeiro ele vai trazer os funcionários 'ST_MAN' com salário acima de 5000, e depois todos os funcionários 'IT_PROG' (neste último, independentemente do salário)?

```
SELECT first_name, job_id, salary
FROM employees
WHERE job_id = 'IT_PROG' OR job_id = 'ST_MAN' AND salary > 5000;
```

Regras de precedência

Se você respondeu a opção (2), então acertou. De acordo com as regras de precedência, o operador AND vem primeiro que o operador OR. Portanto, no exemplo anterior, teremos a seguinte ordem:

```
SELECT first_name, job_id, salary
FROM employees
WHERE job_id = 'IT_PROG' OR job_id = 'ST_MAN' AND salary > 5000;
```

Primeiro, é feito o filtro de funcionários com **JOB_ID = 'ST_MAN' AND salary > 5000**

Em seguida, é feito o filtro de funcionários com **JOB_ID = 'IT_PROG' OR o resultado anterior.**

Observe, portanto, que no resultado temos: funcionários IT_PROG OU funcionários ST_MAN com salário acima de 5000.

Os funcionários que têm o salário abaixo de 5000 são necessariamente IT_PROG.

	FIRST_NA...	JOB_ID	SALARY
1	Alexander	IT_PROG	9000
2	Bruce	IT_PROG	6000
3	David	IT_PROG	4800
4	Valli	IT_PROG	4800
5	Diana	IT_PROG	4200
6	Matthew	ST_MAN	8000
7	Adam	ST_MAN	8200
8	Payam	ST_MAN	7900
9	Shanta	ST_MAN	6500
10	Kevin	ST_MAN	5800

Regras de precedência

E se quiséssemos os funcionários IT_PROG ou ST_MAN, mas apenas os que têm salário acima de 5000 (agora, considerando ambos). Como ficaria?

Então, teríamos que usar parênteses para especificar o que queremos que seja feito primeiro.

Isso porque nesse novo caso precisamos que o OR seja executado primeiro, e só depois o AND. O código da solução é mostrado ao lado.

Dessa vez, o filtro de salário só será aplicado depois que a gente tiver apenas os funcionários IT_PROG ou ST_MAN.

Observe que a consulta traz apenas os funcionários IT_PROG ou ST_MAN, mas que necessariamente têm um salário acima de 5000.

```
SELECT first_name, job_id, salary
FROM employees
WHERE (job_id = 'IT_PROG' OR job_id = 'ST_MAN') AND salary > 5000;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 7 em 0,021 segundos

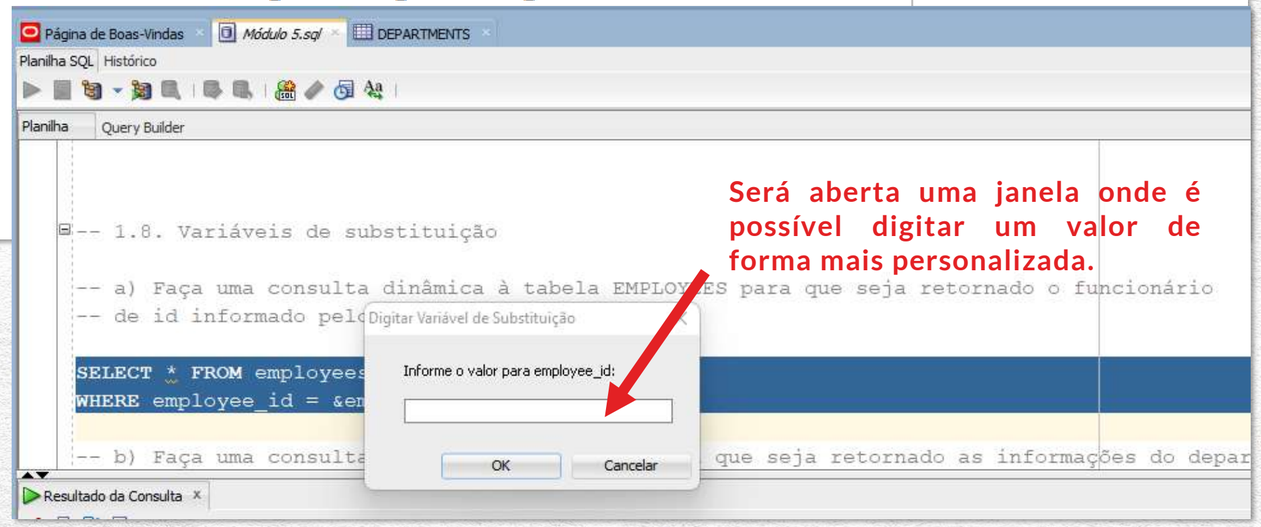
	FIRST_NAME	JOB_ID	SALARY
1	Alexander	IT_PROG	9000
2	Bruce	IT_PROG	6000
3	Matthew	ST_MAN	8000
4	Adam	ST_MAN	8200
5	Payam	ST_MAN	7900
6	Shanta	ST_MAN	6500
7	Kevin	ST_MAN	5800

Variáveis de substituição

VARIÁVEIS DE SUBSTITUIÇÃO permitem que a gente interaja com o Oracle com algum input do usuário, abrindo uma caixinha onde podemos digitar um valor.

Por exemplo, imagine que você deseje retornar o funcionário de um ID que você quer informar de maneira dinâmica.
Para isso, você vai usar o & conforme mostrado no print abaixo:

```
-- a) Faça uma consulta dinâmica à tabela EMPLOYEES para que seja retornado o funcionário
-- de id informado pelo usuário.

SELECT * FROM employees
WHERE employee_id = &employee_id;
```



Variáveis de substituição

Caso o seu filtro seja sobre uma coluna de texto ou de data, é necessário colocar entre aspas simples.

```
SELECT * FROM departments
WHERE department_name = '&department_name';
```

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. Utilizamos o comando ORDER BY para ordenar os dados de uma tabela a partir de uma ou mais colunas.
Já o FETCH NEXT é usado para limitar as linhas da tabela.

2. O comando WHERE será usado para filtrar tabelas. Podemos usá-lo da forma mais básica (filtrando colunas de número, texto e data usando os operadores >, <, >=, <=, <>) ou combiná-lo com uma série de operadores mais avançados: AND, OR, BETWEEN, LIKE, IN, IS NULL.

3. Devemos tomar cuidado com as REGRAS DE PRECEDÊNCIA e usar os parênteses sempre que necessário.

4. Utilize VARIÁVEIS DE SUBSTITUIÇÃO para criar interação entre o usuário e a consulta SQL. Lembre-se apenas que este é um recurso para teste dos programas, na prática NÃO utilizamos as variáveis de substituição para interagir com bancos de dados.

5. O Oracle é CASE SENSITIVE. Ou seja, em um filtro, 'ST_MAN' é diferente de 'st_man'.

EXERCÍCIOS

1

Questão 1

A equipe de Expansão Internacional precisa entender a distribuição dos países onde a empresa já atua, organizados por região. Como a empresa está focada em expandir primeiro nas regiões de maior prioridade estratégica, a listagem dos países deve ser exibida em ordem decrescente de região e, dentro de cada uma, em ordem alfabética.

Liste todos os países da tabela countries em ordem numérica decrescente de região e, dentro de cada região, em ordem alfabética crescente pelo nome do país.

(Utilize a tabela countries)

2

Questão 2

O setor de Recursos Humanos está organizando um evento de comemoração aos primeiros funcionários contratados pela empresa. Para homenageá-los, é necessário listar os três primeiros colaboradores que ingressaram na companhia, ordenados pela data de contratação.

Liste os 3 primeiros funcionários contratados, com base na data de contratação.

(Utilize a tabela employees)

EXERCÍCIOS

3

Questão 3

O Departamento de Gestão de Pessoas está realizando um levantamento sobre os gestores da empresa para verificar quantos líderes existem.

Liste todos os IDs dos gerentes distintos (manager_id) da tabela employees, exibindo primeiro os valores preenchidos em ordem decrescente e por último os valores nulos.

(Utilize a tabela employees)

4

Questão 4

A equipe de Finanças recebeu um pedido da Diretoria para verificar quantos funcionários recebem exatamente $9000, pois há um plano de ajuste salarial para esse grupo.

Liste os funcionários que ganham exatamente $9000, ordenando por employee_id para facilitar a identificação.

(Utilize a tabela employees)

EXERCÍCIOS

5

Questão 5

O Departamento de Recursos Humanos está revisando a política salarial da empresa e precisa identificar quais cargos possuem um salário mínimo superior a $5000 e um salário máximo inferior a $20000. Esses cargos podem ser ajustados para se adequarem ao novo plano de remuneração.

Liste os cargos cujo piso salarial seja maior que $5000 e o teto salarial seja menor que $20000.

(Utilize a tabela jobs)

6

Questão 6

A empresa oferece benefícios especiais para funcionários que completaram 30 anos de casa. O setor de Recursos Humanos precisa listar todos os colaboradores que foram contratados no ano de 1994 para preparar essa bonificação.

Liste os funcionários contratados entre 1º de janeiro de 1994 e 31 de dezembro de 1994, ordenados pelo hire_date.

(Utilize a tabela employees)

EXERCÍCIOS

7

Questão 7

O Setor de Comunicação Interna quer personalizar mensagens para os funcionários com nomes que começam com "H" ou "J". O objetivo é testar a eficácia de campanhas direcionadas e personalizadas para diferentes perfis dentro da empresa.

Liste todos os funcionários cujo first_name começa com a letra "H" ou "J".

(Utilize a tabela employees)

8

Questão 8

A equipe de Recursos Humanos quer identificar quais funcionários ocupam os cargos de IT_PROG (Programador de TI), ST_MAN (Gerente de Estoque) e MK_REP (Representante de Marketing) e possuem um gerente atribuído. Isso faz parte de uma auditoria interna para verificar a estrutura de liderança dessas funções.

Liste os funcionários que possuem gerente e cujo job_id está na lista de cargos "IT_PROG", "ST_MAN" ou "MK_REP".

(Utilize a tabela employees)

EXERCÍCIOS

9

Questão 9

O setor Financeiro quer analisar os funcionários que estão entre os 6º e 20º lugares no ranking de maiores salários da empresa. A análise faz parte de um estudo para garantir que os colaboradores melhor remunerados estejam alinhados com o desempenho esperado.

Liste os funcionários que estão entre as posições 6 e 20 em termos de maior salário.

(Utilize a tabela employees)

10

Questão 10

O Departamento Financeiro precisa analisar a quantidade de funcionários que recebem acima de um determinado valor, que será informado pelo usuário no momento da consulta. Essa análise ajudará na revisão de orçamento e planejamento salarial.

Escreva uma consulta que pergunte ao usuário um valor de salário e retorne todos os funcionários que ganham acima desse valor.

(Utilize a tabela employees)

GABARITO EXPLICATIVO

1

Questão 1

A equipe de Expansão Internacional precisa entender a distribuição dos países onde a empresa já atua, organizados por região. Como a empresa está focada em expandir primeiro nas regiões de maior prioridade estratégica, a listagem dos países deve ser exibida em ordem decrescente de região e, dentro de cada uma, em ordem alfabética.

Liste todos os países da tabela countries em ordem numérica decrescente de região e, dentro de cada região, em ordem alfabética crescente pelo nome do país.

(Utilize a tabela countries)

Explicação e Resultado

Para listar todos os países da tabela countries em ordem numérica decrescente de região e, dentro de cada região, em ordem alfabética crescente pelo nome do país, fazemos o seguinte:

1. **SELECT * FROM countries**: selecionamos todas as colunas da tabela countries.

2. Utilizamos o **ORDER BY** para:

- ✓ Primeiramente ordenar de forma decrescente (**DESC**) pela coluna **region_id**;
- ✓ Em seguida, ordenamos também pela coluna **country_name** em ordem crescente (**ASC**):

```
SELECT *
FROM countries
ORDER BY region_id DESC, country_name ASC;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

A equipe de Expansão Internacional precisa entender a distribuição dos países onde a empresa já atua, organizados por região. Como a empresa está focada em expandir primeiro nas regiões de maior prioridade estratégica, a listagem dos países deve ser exibida em ordem decrescente de região e, dentro de cada uma, em ordem alfabética.

Liste todos os países da tabela countries em ordem numérica decrescente de região e, dentro de cada região, em ordem alfabética crescente pelo nome do país.

(Utilize a tabela countries)

Explicação e Resultado

Dessa forma, retornamos o resultado ao lado, com todas as linhas ordenadas de forma decrescente por region_id e, em cada region_id, por country_name de forma crescente:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 25 em 0,041 segundos

	COUNTRY_ID	COUNTRY_NAME	REGION_ID
1	EG	Egypt	4
2	IL	Israel	4
3	KW	Kuwait	4
4	NG	Nigeria	4
5	ZM	Zambia	4
6	ZW	Zimbabwe	4
7	AU	Australia	3
8	CN	China	3
9	HK	HongKong	3
10	IN	India	3
11	JP	Japan	3
12	SG	Singapore	3
13	AR	Argentina	2
14	BR	Brazil	2
15	CA	Canada	2
16	MX	Mexico	2
17	US	United States of America	2
18	BE	Belgium	1
19	DK	Denmark	1
20	FR	France	1
21	DE	Germany	1
22	IT	Italy	1
23	NL	Netherlands	1
24	CH	Switzerland	1
25	UK	United Kingdom	1

GABARITO EXPLICATIVO

2

Questão 2

O setor de Recursos Humanos está organizando um evento de comemoração aos primeiros funcionários contratados pela empresa. Para homenageá-los, é necessário listar os três primeiros colaboradores que ingressaram na companhia, ordenados pela data de contratação.

Liste os 3 primeiros funcionários contratados, com base na data de contratação.

(Utilize a tabela employees)

Explicação e Resultado

Para listar os 3 primeiros funcionários contratados, com base na data de contratação, fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees.

2. Utilizamos o **ORDER BY** para ordenar os funcionários pela data de contratação (**hire_date**) em ordem crescente (do mais antigo para o mais recente).

3. Utilizamos **FETCH NEXT 3 ROWS ONLY** para limitar o resultado aos três primeiros registros:

```
SELECT *
FROM employees
ORDER BY hire_date
FETCH NEXT 3 ROWS ONLY;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

O setor de Recursos Humanos está organizando um evento de comemoração aos primeiros funcionários contratados pela empresa. Para homenageá-los, é necessário listar os três primeiros colaboradores que ingressaram na companhia, ordenados pela data de contratação.

Liste os 3 primeiros funcionários contratados, com base na data de contratação.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, retornamos os três primeiros funcionários contratados, com base na data de contratação:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 3 em 0,006 segundos

	EMPLO...	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMI...	MANAGE...	DEPA...
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
3	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90

GABARITO EXPLICATIVO

3

Questão 3

O Departamento de Gestão de Pessoas está realizando um levantamento sobre os gestores da empresa para verificar quantos líderes existem.

Liste todos os IDs dos gerentes distintos (manager_id) da tabela employees, exibindo primeiro os valores preenchidos em ordem decrescente e por último os valores nulos.

(Utilize a tabela employees)

Explicação e Resultado

Para listar todos os IDs dos gerentes distintos (manager_id) da tabela employees, exibindo primeiro os valores preenchidos em ordem decrescente e, por último, os valores nulos, fazemos o seguinte:

1. **SELECT DISTINCT manager_id FROM employees**: selecionamos apenas os valores distintos da coluna manager_id, garantindo que cada gerente apareça apenas uma vez.

2. Utilizamos o **ORDER BY** para:

- ✓ Ordenar os valores preenchidos de **manager_id** em ordem decrescente (**DESC**), exibindo primeiro os IDs mais altos.
- ✓ Utilizar **NULLS LAST** para garantir que os valores nulos apareçam por último na listagem:

```
SELECT DISTINCT manager_id
FROM employees
ORDER BY manager_id DESC NULLS LAST;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O Departamento de Gestão de Pessoas está realizando um levantamento sobre os gestores da empresa para verificar quantos líderes existem.

Liste todos os IDs dos gerentes distintos (manager_id) da tabela employees, exibindo primeiro os valores preenchidos em ordem decrescente e por último os valores nulos.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, retornamos a lista de IDs dos gerentes, com 18 IDs distintos, organizados de forma decrescente, e o campo nulo ao final:

Resultado da Consulta

	MANAGER_ID
1	205
2	201
3	149
4	148
5	147
6	146
7	145
8	124
9	123
10	122
11	121
12	120
13	114
14	108
15	103
16	102
17	101
18	100
19	(null)

GABARITO EXPLICATIVO

4

Questão 4

A equipe de Finanças recebeu um pedido da Diretoria para verificar quantos funcionários recebem exatamente $9000, pois há um plano de ajuste salarial para esse grupo.

Liste os funcionários que ganham exatamente $9000, ordenando por employee_id para facilitar a identificação.

(Utilize a tabela employees)

Explicação e Resultado

Para listar os funcionários que ganham exatamente $9000, ordenando por employee_id, fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees, pois precisamos das informações completas dos funcionários.

2. Aplicamos a cláusula **WHERE salary = 9000** para filtrar apenas os funcionários cujo salário seja exatamente $9000.

3. Utilizamos **ORDER BY employee_id** para ordenar os resultados pelo ID do funcionário, facilitando a identificação:

```
SELECT *
FROM employees
WHERE salary = 9000
ORDER BY employee_id;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A equipe de Finanças recebeu um pedido da Diretoria para verificar quantos funcionários recebem exatamente $9000, pois há um plano de ajuste salarial para esse grupo.

Liste os funcionários que ganham exatamente $9000, ordenando por employee_id para facilitar a identificação.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, retornamos a lista dos funcionários que recebem exatamente $9000, organizados em ordem crescente de employee_id:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 4 em 0,013 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSI...	MANA...	DEPAR...
1	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
2	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
3	152	Peter	Hall	PHALL	011.44.1344.478968	20/08/97	SA_REP	9000	0,25	145	80
4	158	Allan	McEwen	AMCEWEN	011.44.1345.829268	01/08/96	SA_REP	9000	0,35	146	80

GABARITO EXPLICATIVO

5

Questão 5

O Departamento de Recursos Humanos está revisando a política salarial da empresa e precisa identificar quais cargos possuem um salário mínimo superior a $5000 e um salário máximo inferior a $20000. Esses cargos podem ser ajustados para se adequarem ao novo plano de remuneração.

Liste os cargos cujo piso salarial seja maior que $5000 e o teto salarial seja menor que $20000.

(Utilize a tabela jobs)

Explicação e Resultado

Para listar os cargos cujo piso salarial seja maior que $5000 e o teto salarial menor que $20000, fazemos o seguinte:

1. **SELECT * FROM jobs**: selecionamos todas as colunas da tabela jobs, pois queremos informações completas dos cargos.

2. Aplicamos a cláusula **WHERE** com as seguintes condições:

- ✓ **min_salary > 5000** para filtrar apenas os cargos com salário mínimo acima de $5000.
- ✓ **max_salary < 20000** para garantir que o salário máximo desses cargos seja inferior a $20000:

```
SELECT *
FROM jobs
WHERE min_salary > 5000 AND max_salary < 20000;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

O Departamento de Recursos Humanos está revisando a política salarial da empresa e precisa identificar quais cargos possuem um salário mínimo superior a $5000 e um salário máximo inferior a $20000. Esses cargos podem ser ajustados para se adequarem ao novo plano de remuneração.

Liste os cargos cujo piso salarial seja maior que $5000 e o teto salarial seja menor que $20000.

(Utilize a tabela jobs)

Explicação e Resultado

Dessa forma, retornamos a lista dos cargos que se encaixam nos critérios salariais estabelecidos pelo Departamento de Recursos Humanos:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 6 em 0,003 segundos

	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
1	FI_MGR	Finance Manager	8200	16000
2	AC_MGR	Accounting Manager	8200	16000
3	SA_REP	Sales Representative	6000	12000
4	PU_MAN	Purchasing Manager	8000	15000
5	ST_MAN	Stock Manager	5500	8500
6	MK_MAN	Marketing Manager	9000	15000

GABARITO EXPLICATIVO

6

Questão 6

A empresa oferece benefícios especiais para funcionários que completaram 30 anos de casa. O setor de Recursos Humanos precisa listar todos os colaboradores que foram contratados no ano de 1994 para preparar essa bonificação.

Liste os funcionários contratados entre 1º de janeiro de 1994 e 31 de dezembro de 1994, ordenados pelo hire_date.

(Utilize a tabela employees)

Explicação e Resultado

Para listar os funcionários contratados entre 1º de janeiro de 1994 e 31 de dezembro de 1994, ordenados por data de contratação, fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees, pois queremos informações completas dos funcionários.

2. Utilizamos a cláusula **WHERE** com **BETWEEN** para filtrar os funcionários contratados dentro do ano de 1994.
   - ✓ **hire_date BETWEEN '01/01/94' AND '31/12/94'** seleciona apenas aqueles que começaram nesse período.

Ordenamos o resultado pela coluna **hire_date** em **ordem crescente** para exibir os funcionários na sequência de contratação:

```
SELECT *
FROM employees
WHERE hire_date BETWEEN '01/01/94' AND '31/12/94'
ORDER BY hire_date;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

A empresa oferece benefícios especiais para funcionários que completaram 30 anos de casa. O setor de Recursos Humanos precisa listar todos os colaboradores que foram contratados no ano de 1994 para preparar essa bonificação.

Liste os funcionários contratados entre 1º de janeiro de 1994 e 31 de dezembro de 1994, ordenados pelo hire_date.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, obtemos a lista dos funcionários que completaram 30 anos de empresa (considerando-se o ano de 2025), permitindo ao RH organizar a bonificação:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 7 em 0,002 segundos

	EMPL...	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISS...	MANA...	DEPA...
1	203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	(null)	101	40
2	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
3	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	70
4	206	William	Gietz	WGIETZ	515.123.8181	07/06/94	AC_ACCOUNT	8300	(null)	205	110
5	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
6	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
7	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30

GABARITO EXPLICATIVO

7

Questão 7

O Setor de Comunicação Interna quer personalizar mensagens para os funcionários com nomes que começam com "H" ou "J". O objetivo é testar a eficácia de campanhas direcionadas e personalizadas para diferentes perfis dentro da empresa.

Liste todos os funcionários cujo first_name começa com a letra "H" ou "J".

(Utilize a tabela employees)

Explicação e Resultado

Para listar todos os funcionários cujo first_name começa com a letra "H" ou "J", fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees, pois queremos informações completas dos funcionários.

2. Utilizamos a cláusula **WHERE** com **LIKE** para filtrar os nomes que começam com "H" **OU** "J":

- ✓ **first_name LIKE 'H%'** retorna os funcionários cujos nomes começam com "H".
- ✓ **OR first_name LIKE 'J%'** adiciona ao resultado aqueles que começam com "J":

```
SELECT *
FROM employees
WHERE first_name LIKE 'H%' OR first_name LIKE 'J%';
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

O Setor de Comunicação Interna quer personalizar mensagens para os funcionários com nomes que começam com "H" ou "J". O objetivo é testar a eficácia de campanhas direcionadas e personalizadas para diferentes perfis dentro da empresa.

Liste todos os funcionários cujo first_name começa com a letra "H" ou "J".

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, obtemos a lista de funcionários que receberão mensagens personalizadas no teste de comunicação interna:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,004 segundos

	EMP...	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMI...	MAN...	DEPA...
1	110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)	108	100
2	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	(null)	108	100
3	125	Julia	Nayer	JNAYER	650.124.1214	16/07/97	ST_CLERK	3200	(null)	120	50
4	127	James	Landry	JLANDRY	650.124.1334	14/01/99	ST_CLERK	2400	(null)	120	50
5	131	James	Marlow	JAMRLOW	650.124.7234	16/02/97	ST_CLERK	2500	(null)	121	50
6	133	Jason	Mallin	JMALLIN	650.127.1934	14/06/96	ST_CLERK	3300	(null)	122	50
7	136	Hazel	Philtanker	HPHILTAN	650.127.1634	06/02/00	ST_CLERK	2200	(null)	122	50
8	139	John	Seo	JSEO	650.121.2019	12/02/98	ST_CLERK	2700	(null)	123	50
9	140	Joshua	Patel	JPATEL	650.121.1834	06/04/98	ST_CLERK	2500	(null)	123	50
10	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
11	156	Janette	King	JKING	011.44.1345.429268	30/01/96	SA_REP	10000	0,35	146	80
12	169	Harrison	Bloom	HBLOOM	011.44.1343.829268	23/03/98	SA_REP	10000	0,2	148	80
13	176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	24/03/98	SA_REP	8600	0,2	149	80
14	177	Jack	Livingston	JLIVINGS	011.44.1644.429264	23/04/98	SA_REP	8400	0,2	149	80
15	181	Jean	Fleaur	JFLEAUR	650.507.9877	23/02/98	SH_CLERK	3100	(null)	120	50
16	186	Julia	Dellinger	JDELLING	650.509.3876	24/06/98	SH_CLERK	3400	(null)	121	50
17	189	Jennifer	Dilly	JDILLY	650.505.2876	13/08/97	SH_CLERK	3600	(null)	122	50
18	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
19	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	70

GABARITO EXPLICATIVO

8

Questão 8

A equipe de Recursos Humanos quer identificar quais funcionários ocupam os cargos de IT_PROG (Programador de TI), ST_MAN (Gerente de Estoque) e MK_REP (Representante de Marketing) e possuem um gerente atribuído. Isso faz parte de uma auditoria interna para verificar a estrutura de liderança dentro desses departamentos.

Liste os funcionários que possuem gerente e cujo job_id está na lista de cargos "IT_PROG", "ST_MAN" ou "MK_REP".

(Utilize a tabela employees)

Explicação e Resultado

Para listar os funcionários que possuem gerente e ocupam os cargos "IT_PROG", "ST_MAN" ou "MK_REP", fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees, pois queremos informações completas dos funcionários.

2. Utilizamos a cláusula **WHERE** com **AND** para aplicar dois filtros:

- ✓ **manager_id IS NOT NULL**: garante que apenas funcionários com um gerente atribuído sejam retornados.
- ✓ **AND job_id IN ('IT_PROG', 'ST_MAN', 'MK_REP')**: e filtra, dentre as linhas retornadas pela primeira condição, somente os funcionários que possuem um dos três cargos específicos:

```
SELECT *
FROM employees
WHERE manager_id IS NOT NULL AND job_id IN ('IT_PROG', 'ST_MAN', 'MK_REP');
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A equipe de Recursos Humanos quer identificar quais funcionários ocupam os cargos de IT_PROG (Programador de TI), ST_MAN (Gerente de Estoque) e MK_REP (Representante de Marketing) e possuem um gerente atribuído. Isso faz parte de uma auditoria interna para verificar a estrutura de liderança dessas funções.

Liste os funcionários que possuem gerente e cujo job_id está na lista de cargos "IT_PROG", "ST_MAN" ou "MK_REP".

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, obtemos a lista de funcionários que pertencem a esses cargos e possuem supervisão, ajudando na auditoria interna sobre a estrutura de liderança:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,012 segundos

	EMPL...	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMI...	MANAGER_ID	DEP...
1	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
2	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60
3	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
4	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
5	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60
6	202	Pat	Fay	PFAY	603.123.6666	17/08/97	MK_REP	6000	(null)	201	20
7	120	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	(null)	100	50
8	121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	50
9	122	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	(null)	100	50
10	123	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	(null)	100	50
11	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16/11/99	ST_MAN	5800	(null)	100	50

GABARITO EXPLICATIVO

9

Questão 9

O setor Financeiro quer analisar os funcionários que estão entre os 6º e 20º lugares no ranking de maiores salários da empresa. A análise faz parte de um estudo para garantir que os colaboradores melhor remunerados estejam alinhados com o desempenho esperado.

Liste os funcionários que estão entre as posições 6 e 20 em termos de maior salário.

(Utilize a tabela employees)

Explicação e Resultado

Para listar os funcionários que estão entre as posições 6 e 20 no ranking de maiores salários, fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees, pois queremos informações completas dos funcionários.

2. Utilizamos **ORDER BY salary DESC** para ordenar os funcionários do maior para o menor salário.

3. Aplicamos **OFFSET 5 ROWS** para ignorar os 5 primeiros registros, pois queremos começar a partir do 6º maior salário.

4. Usamos **FETCH NEXT 15 ROWS ONLY** para limitar a consulta aos próximos 15 funcionários (da posição 6 até a 20):

```
SELECT *
FROM employees
ORDER BY salary DESC
OFFSET 5 ROWS FETCH NEXT 15 ROWS ONLY;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O setor Financeiro quer analisar os funcionários que estão entre os 6º e 20º lugares no ranking de maiores salários da empresa. A análise faz parte de um estudo para garantir que os colaboradores melhor remunerados estejam alinhados com o desempenho esperado.

Liste os funcionários que estão entre as posições 6 e 20 em termos de maior salário.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, retornamos exatamente os funcionários que ocupam essas posições no ranking salarial, permitindo a análise financeira desejada:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 15 em 0,007 segundos

	EMPLO...	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMIS...	MAN...	DEPA...
1	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
2	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
3	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
4	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	80
5	168	Lisa	Ozer	LOZER	011.44.1343.929268	11/03/97	SA_REP	11500	0,25	148	80
6	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30
7	174	Ellen	Abel	EABEL	011.44.1644.429267	11/05/96	SA_REP	11000	0,3	149	80
8	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100	80
9	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29/01/00	SA_MAN	10500	0,2	100	80
10	162	Clara	Vishney	CVISHNEY	011.44.1346.129268	11/11/97	SA_REP	10500	0,25	147	80
11	150	Peter	Tucker	PTUCKER	011.44.1344.129268	30/01/97	SA_REP	10000	0,3	145	80
12	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	70
13	169	Harrison	Bloom	HBLOOM	011.44.1343.829268	23/03/98	SA_REP	10000	0,2	148	80
14	156	Janette	King	JKING	011.44.1345.429268	30/01/96	SA_REP	10000	0,35	146	80
15	170	Tayler	Fox	TFOX	011.44.1343.729268	24/01/98	SA_REP	9600	0,2	148	80

GABARITO EXPLICATIVO

10

Questão 10

O Departamento Financeiro precisa analisar a quantidade de funcionários que recebem acima de um determinado valor, que será informado pelo usuário no momento da consulta. Essa análise ajudará na revisão de orçamento e planejamento salarial.

Escreva uma consulta que pergunte ao usuário um valor de salário e retorne todos os funcionários que ganham acima desse valor.

(Utilize a tabela employees)

Explicação e Resultado

Para listar todos os funcionários que ganham acima de um valor informado pelo usuário no momento da consulta, fazemos o seguinte:

1. **SELECT * FROM employees**: selecionamos todas as colunas da tabela employees, pois queremos informações completas dos funcionários.

2. Utilizamos **WHERE salary > &salario_informado** para filtrar os funcionários cujo salário é maior do que o valor digitado pelo usuário e armazenado temporariamente na variável de substituição &salario_informado.

3. Aplicamos **ORDER BY salary** para exibir os resultados em ordem crescente de salário, facilitando a análise:

```
SELECT *
FROM employees
WHERE salary > &salario_informado
ORDER BY salary;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

10

Questão 10

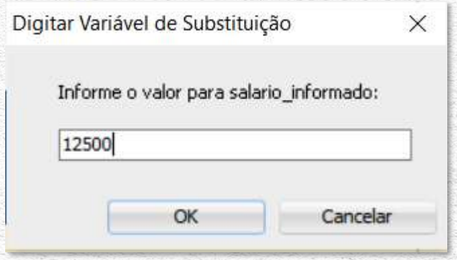
O Departamento Financeiro precisa analisar a quantidade de funcionários que recebem acima de um determinado valor, que será informado pelo usuário no momento da consulta. Essa análise ajudará na revisão de orçamento e planejamento salarial.

Escreva uma consulta que pergunte ao usuário um valor de salário e retorne todos os funcionários que ganham acima desse valor.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, a consulta solicitará um valor ao usuário...



... E retornará todos os funcionários que recebem acima desse montante:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 6 em 0,006 segundos

	EMPL...	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISS...	MANAG...	DEPA...
1	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
2	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80
3	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
4	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
5	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
6	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90

MÓDULO 6

FUNÇÕES SINGLE ROW (PT. 1)

FUNÇÕES SINGLE ROW (PT. 1)

FUNÇÕES SINGLE ROW (PT. 1)

O QUE VEREMOS NESTE MÓDULO

MÓDULO 6 FUNÇÕES SINGLE ROW: NÚMEROS, DATAS e TEXTOS

01	FUNÇÕES SINGLE ROW	235
02	FUNÇÕES DE TEXTO	237
03	LOWER	238
04	UPPER	239
05	INITCAP	240
06	EXEMPLO DE APLICAÇÃO	241
07	CONCAT	243
08	SUBSTR	244
09	INSTR	245
10	LENGTH	246
11	LPAD	247
12	RPAD	248
13	REPLACE	249
14	FUNÇÕES DE NÚMERO	250
15	ROUND E TRUNC	251
16	MOD	252
17	FUNÇÕES DE DATA	253
18	MONTHS_BETWEEN	254
19	ADD_MONTHS	255
20	NEXT_DAY	256
21	LAST_DAY	257
22	EXTRACT	258
23	EXERCÍCIOS	259
24	GABARITO EXPLICATIVO	265

Funções Single Row

As FUNÇÕES DE LINHA ÚNICA (SINGLE ROW) retornam uma única linha de resultado para cada linha de uma tabela consultada. Essas funções podem aparecer dentro do SELECT, cláusulas WHERE e cláusulas HAVING.

Existem diversas CATEGORIAS de funções Single Row.
Neste módulo e no próximo, vamos focar nas mais comuns, listadas a seguir:

NÚMEROS

Funções para trabalhar com **NÚMEROS**.

- ROUND
- TRUNC
- MOD

STRINGS

Funções para trabalhar com **TEXTOS**.

- LOWER
- UPPER
- INITCAP
- CONCAT
- SUBSTR
- LENGTH
- INSTR
- LPAD
- RPAD
- REPLACE

DATAS

Funções para trabalhar com **DATAS**.

- MONTHS_BETWEEN
- ADD_MONTHS
- NEXT_DAY
- LAST_DAY
- EXTRACT

CONVERSÃO

Funções para **converter tipos de dados** (texto para número, texto para data, etc.)

- TO_CHAR
- TO_NUMBER
- TO_DATE

NULL

Funções para se trabalhar com **valores nulos.**

- NVL
- NVL2
- COALESCE
- NULLIF

Índice Geral

Índice deste Módulo

Funções Single Row

Neste módulo, focaremos nas funções de Números, Strings e Datas.
No próximo, falaremos sobre as funções de conversão e funções para trabalhar com valores nulos.

NÚMEROS

Funções para trabalhar com **NÚMEROS**.

- ROUND
- TRUNC
- MOD

STRINGS

Funções para trabalhar com **TEXTOS**.

- LOWER
- UPPER
- INITCAP
- CONCAT
- SUBSTR
- LENGTH
- INSTR
- LPAD
- RPAD
- REPLACE

DATAS

Funções para trabalhar com **DATAS**.

- MONTHS_BETWEEN
- ADD_MONTHS
- NEXT_DAY
- LAST_DAY
- EXTRACT

CONVERSÃO

Funções para **converter tipos de dados** (texto para número, texto para data, etc.)

- TO_CHAR
- TO_NUMBER
- TO_DATE

NULL

Funções para se trabalhar com **valores nulos.**

- NVL
- NVL2
- COALESCE
- NULLIF

Índice Geral

Índice deste Módulo

Funções de Texto

O primeiro conjunto de funções que vamos trabalhar são as FUNÇÕES DE TEXTO.

Essas funções nos permitem manipular textos das mais variadas formas, e a seguir vamos entender como cada uma vai funcionar.

STRINGS

Funções para trabalhar com **TEXTOS**.

- LOWER
- UPPER
- INITCAP
- CONCAT
- SUBSTR
- LENGTH
- INSTR
- LPAD
- RPAD
- REPLACE

LOWER

A função de texto **LOWER** nos permite converter as letras de uma palavra para o formato **minúsculo**. Ela nos pede apenas um argumento: o texto que queremos converter.

Ao lado, temos um exemplo de aplicação dessa função.

```
8  -- 1. LOWER: Coloca todos os caracteres de uma palavra em minúsculas.
9
10 SELECT
11     first_name,
12     LOWER(first_name)
13 FROM employees;
14
15
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,243 segundos

	FIRST_NAME	LOWER(FIRST_NAME)
1	Ellen	ellen
2	Sundar	sundar
3	Mozhe	mozhe
4	David	david
5	Hermann	hermann

UPPER

A próxima função de texto é a **UPPER**, que nos permite converter as letras de uma palavra para o formato **maiúsculo**. Ela nos pede apenas um argumento: o texto que queremos converter.

Ao lado, temos um exemplo de aplicação dessa função.

```
16  -- 2. UPPER: Coloca todos os caracteres de uma palavra em MAIÚSCULAS.
17
18  SELECT
19      first_name,
20      UPPER(first_name)
21  FROM employees;
22
23
24
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,012 segundos

	FIRST_NAME	UPPER(FIRST_NAME)
1	Ellen	ELLEN
2	Sundar	SUNDAR
3	Mozhe	MOZHE
4	David	DAVID
5	Hermann	HERMANN
6	Shelli	SHELLI
7	Amit	AMIT
8	Elizabeth	ELIZABETH

INITCAP

Temos agora a função **INITCAP**, que converte a **primeira letra** de cada palavra em **maiúscula**, e o resto fica em minúscula.

Ao lado, temos um exemplo de aplicação dessa função.

```
26  -- 3. INITCAP: Deixa o primeiro caractere de cada palavra em MAIÚSCULA e o resto em minúscula.
27
28  SELECT
29      job_id,
30      INITCAP(job_id)
31  FROM employees;
32
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,015 segundos

	JOB_ID	INITCAP(JOB_ID)
1	AC_ACCOUNT	Ac_Account
2	AC_MGR	Ac_Mgr
3	AD_ASST	Ad_Asst
4	AD_PRES	Ad_Pres
5	AD_VP	Ad_Vp
6	AD_VP	Ad_Vp
7	FI_ACCOUNT	Fi_Account
8	FI_ACCOUNT	Fi_Account
9	FI_ACCOUNT	Fi_Account
10	FI_ACCOUNT	Fi_Account

Exemplo de aplicação

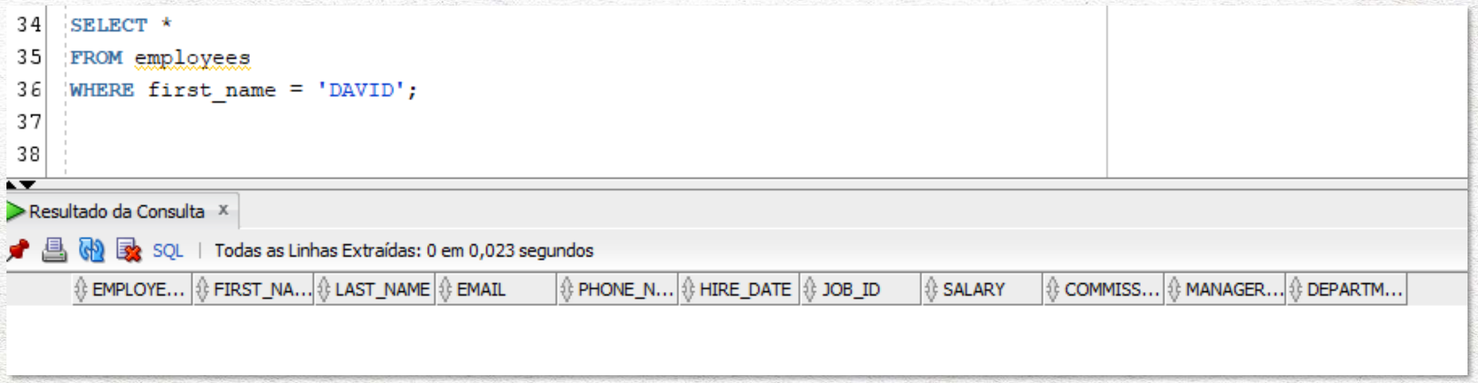
É natural que neste ponto você se pergunte qual é a utilidade dessas funções, uma vez que aparentemente elas não tem muita aplicação.

Pra que eu vou querer fazer esses tipos de tratamento nos meus dados? Parece um tanto inútil, certo?

Pois pense no seguinte cenário: imagine que no Oracle você deseje filtrar a tabela EMPLOYEES para mostrar todos os funcionários com o primeiro nome igual a “David”.

Você deve lembrar que o Oracle é *case sensitive* e portanto, se não soubermos exatamente como está escrito o nome na nossa base, teremos um problema.

Observe ao lado, que ao filtrar a coluna first_name = ‘DAVID’, ele não retorna valor algum.



Exemplo de aplicação

O que podemos fazer, então, é usar a função UPPER na coluna first_name, e agora sim escrever o texto ‘DAVID’. Dessa vez, não importa como o nome está escrito na tabela, a função UPPER vai converter todos os textos para maiúscula e consequentemente vai retornar as linhas que tenham este nome.

Observe que o UPPER não altera a forma como os nomes aparecem na tabela, uma vez que usamos essa função no filtro WHERE.

```
34  SELECT *
35  FROM employees
36  WHERE UPPER(first_name) = 'DAVID';
37
38
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 3 em 0,005 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DE
1	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	
2	151	David	Bernstein	DBERNSTE	011.44.1344.345268	24/03/97	SA_REP	9500	0,25	145	
3	165	David	Lee	DLEE	011.44.1346.529268	23/02/00	SA_REP	6800	0,1	147	

CONCAT

Outra função muito útil é a **CONCAT**. Ela permite **juntar textos**, conforme exemplo ao lado.

Para usar as colunas first_name e last_name para que, juntas, formem uma nova coluna chamada NOME_COMPLETO, usamos a função CONCAT, como mostrado.

Um detalhe sobre essa função é que ela aceita apenas dois argumentos por vez, então precisamos usar um CONCAT dentro do outro para conseguir concatenar os nomes (uma vez que precisamos colocar um espaço para separar as duas palavras).

```
55  -- 4. CONCAT: Junta dois textos em 1 só. Tem a mesma aplicação do ||.
56
57  SELECT
58      first_name,
59      last_name,
60      CONCAT(first_name, CONCAT(' ', last_name)) nome_completo
61  FROM employees;
62
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,023 segundos

	FIRST_NAME	LAST_NAME	NOME_COMPLETO
1	Ellen	Abel	Ellen Abel
2	Sundar	Ande	Sundar Ande
3	Mozhe	Atkinson	Mozhe Atkinson
4	David	Austin	David Austin
5	Hermann	Baer	Hermann Baer
6	Shelli	Baida	Shelli Baida
7	Amit	Banda	Amit Banda
8	Elizabeth	Bates	Elizabeth Bates
9	Sarah	Bell	Sarah Bell
10	David	Bernstein	David Bernstein

SUBSTR

A função **SUBSTR extrai** uma série de **caracteres** de dentro de um texto.

Essa função nos pede 3 argumentos:

1) texto,
2) caractere inicial que queremos começar a extrair o texto,
3) quantos caracteres queremos.

Por exemplo, imagine que a gente queira extrair a segunda parte de cada JOB_ID.

Essa segunda parte começa sempre a partir do caractere 4. Então, para pegar a segunda parte de cada JOB_ID, usamos a função da forma ao lado (o número 100 é apenas um recurso para retornar tudo o que estiver à direita, já que a partir do caractere 4, queremos todo o texto até o final).

```
78  -- 5. SUBSTR: Extrai uma parte de dentro de um texto.
79
80  SELECT
81      job_id,
82      SUBSTR(job_id, 4, 100)
83  FROM employees;
84
85
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,019 segundos

	JOB_ID	SUBSTR(JOB_ID,4,100)
1	AC_ACCOUNT	ACCOUNT
2	AC_MGR	MGR
3	AD_ASST	ASST
4	AD_PRES	PRES
5	AD_VP	VP
6	AD_VP	VP
7	FI_ACCOUNT	ACCOUNT
8	FI_ACCOUNT	ACCOUNT
9	FI_ACCOUNT	ACCOUNT

INSTR

A função **INSTR** nos **retorna a posição** de um caractere dentro de um texto.

Essa função nos pede 2 argumentos:

1) texto,
2) de qual caractere queremos que seja retornada a posição.

Por exemplo, imagine que a gente queira saber em que posição se encontra o conjunto de caracteres “MGR” em cada texto.

Se você observar a coluna JOB_ID, verá que nem todos eles têm o texto “MGR”. Mas para os que têm, esse texto começa a partir do caractere 4, como mostrado no resultado ao lado.

```
91  -- 6. INSTR: Retorna a posição de um caractere dentro de um texto.
92
93  SELECT
94      job_id,
95      INSTR(job_id, 'MGR')
96  FROM employees;
97
98
99
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,011 segundos

	JOB_ID	INSTR(JOB_ID,'MGR')
1	AC_ACCOUNT	0
2	AC_MGR	4
3	AD_ASST	0
4	AD_PRES	0
5	AD_VP	0
6	AD_VP	0
7	FI_ACCOUNT	0
8	FI_ACCOUNT	0

LENGTH

A função **LENGTH** nos **retorna a quantidade** de caracteres que cada texto tem.

Sua aplicação é bem intuitiva e é mostrada no exemplo ao lado, no qual é retornado o total de caracteres de cada first_name.

```
104 -- 7. LENGTH: Retorna o tamanho de uma palavra em quantidade de caracteres.
105
106 SELECT
107     first_name,
108     LENGTH(first_name)
109 FROM employees;
110
111
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,014 segundos

	FIRST_NAME	LENGTH(FIRST_NAME)
1	Ellen	5
2	Sundar	6
3	Mozhe	5
4	David	5
5	Hermann	7
6	Shelli	6
7	Amit	4
8	Elizabeth	9
9	Sarah	5
10	David	5

LPAD

A função **LPAD** recebe um texto e retorna o mesmo texto, com a quantidade de caracteres especificada e, caso a quantidade de caracteres inicial não seja suficiente, completa com algum caractere desejado, repetindo-o diversas vezes à esquerda.

Confuso?

Observe o exemplo ao lado: a função LPAD vai receber 3 argumentos: o primeiro é o texto, o segundo é a quantidade final de caracteres que o texto deve ter e o terceiro é o caractere que será preenchido à esquerda para que o texto final fique com a quantidade de caracteres desejada.

```
117 -- 8. LPAD: Cria um novo texto com tamanho N, alinha o texto antigo à direita e completa
118 -- os caracteres restantes com o caractere especificado.
119
120 SELECT
121     job_id,
122     LENGTH(job_id),
123     LPAD(job_id, 20, '*')
124 FROM employees;
125
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,02 segundos

	JOB_ID	LENGTH(JOB_ID)	LPAD(JOB_ID,20,'*')
1	AC_ACCOUNT	10	**********AC_ACCOUNT
2	AC_MGR	6	**************AC_MGR
3	AD_ASST	7	*************AD_ASST
4	AD_PRES	7	*************AD_PRES
5	AD_VP	5	***************AD_VP
6	AD_VP	5	***************AD_VP
7	FI_ACCOUNT	10	**********FI_ACCOUNT
8	FI_ACCOUNT	10	**********FI_ACCOUNT
9	FI_ACCOUNT	10	**********FI_ACCOUNT
10	FI_ACCOUNT	10	**********FI_ACCOUNT

RPAD

A função **RPAD** fará algo muito parecido com a função LPAD, com a diferença de que o caractere adicional será preenchido à direita do texto.

```
140  -- 9. RPAD: Cria um novo texto com tamanho N, alinha o texto antigo à esquerda e completa
141  -- os caracteres restantes com o caractere especificado.
142
143  SELECT
144      job_id,
145      LENGTH(job_id),
146      RPAD(job_id, 20, '*')
147  FROM employees;
148
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,112 segundos

	JOB_ID	LENGTH(JOB_ID)	RPAD(JOB_ID,20,'*')
1	AC_ACCOUNT	10	AC_ACCOUNT**********
2	AC_MGR	6	AC_MGR**************
3	AD_ASST	7	AD_ASST*************
4	AD_PRES	7	AD_PRES*************
5	AD_VP	5	AD_VP***************
6	AD_VP	5	AD_VP***************
7	FI_ACCOUNT	10	FI_ACCOUNT**********
8	FI_ACCOUNT	10	FI_ACCOUNT**********

REPLACE

A função **REPLACE** já é muito conhecida em qualquer ferramenta. Ela permite que a gente **substitua** um texto por outro texto.

Ao lado, temos um exemplo de aplicação.

Substituímos o texto “PROG” pelo texto “PR” na coluna job_id.

```
90  -- 10. REPLACE: Substitui um texto por outro texto.
91
92  SELECT
93      job_id,
94      REPLACE(job_id, 'PROG', 'PR')
95  FROM employees;
96
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,027 segundos

	JOB_ID	REPLACE(JOB_ID,'PROG','PR')
10	FI_ACCOUNT	FI_ACCOUNT
11	FI_ACCOUNT	FI_ACCOUNT
12	FI_MGR	FI_MGR
13	HR_REP	HR_REP
14	IT_PROG	IT_PR
15	IT_PROG	IT_PR
16	IT_PROG	IT_PR
17	IT_PROG	IT_PR

Funções de Número

O próximo conjunto de funções que vamos trabalhar são as FUNÇÕES DE NÚMERO.

Essas funções nos permitem manipular números, e a seguir vamos entender como cada uma vai funcionar.

NÚMEROS

Funções para trabalhar com **NÚMEROS**.

- ROUND
- TRUNC
- MOD

ROUND E TRUNC

A função **ROUND arredonda** um valor para uma quantidade de casas desejada. Este arredondamento segue a mesma regra matemática. Importante dizer que a função ROUND altera o valor que foi arredondado.

No exemplo ao lado, o valor 37.548 se transformou no valor 37.55 após ser arredondado com duas casas decimais, e 38 ao ser arredondado sem casas decimais:

```
172  -- 1. ROUND: Arredonda um valor para a quantidade de casas decimais especificada.
173
174  SELECT
175      ROUND(37.548, 2) duas_casas,
176      ROUND(37.548, 0) zero_casas
177  FROM dual;
178
179
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,011 segundos

	DUAS_CASAS	ZERO_CASAS
1	37,55	38

```
180  -- 2. TRUNC: Trunca o valor para a casa decimal especificada.
181
182  SELECT
183      TRUNC(37.548, 2) duas_casas,
184      TRUNC(37.548, 0) zero_casas
185  FROM dual;
186
187
188
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,003 segundos

	DUAS_CASAS	ZERO_CASAS
1	37,54	37

Já a função **TRUNC não arredonda**, apenas trunca (corta) o número na casa decimal desejada.

No exemplo ao lado, o valor 37.548, ao ser cortado na segunda casa decimal se torna 37.54 (sem arredondar para 37.55), e 37, ao serem cortadas todas as casas decimais (novamente, sem arredondar para 38):

MOD

A função **MOD** retorna o **resto** da divisão de um número por outro número.

No exemplo ao lado, o resto da divisão de 10 por 4 é 2.

```
192  -- 3. MOD: Retorna o resto de uma divisão.
193
194  SELECT
195      MOD(10, 4) resto
196  FROM dual;
197
198
199
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,007 segundos

	RESTO
1	2

Funções de Data

Vamos finalizar essa etapa com o conjunto de FUNÇÕES DE DATA.

Essas funções nos permitem manipular datas das mais variadas formas, e a seguir vamos entender como cada uma vai funcionar.

DATAS

Funções para trabalhar com **DATAS**.

- MONTHS_BETWEEN
- ADD_MONTHS
- NEXT_DAY
- LAST_DAY
- EXTRACT

MONTHS_BETWEEN

A função **MONTHS_BETWEEN** retorna a quantidade de meses **entre duas datas**.

Podemos usá-la junto com a função TRUNC para truncar o resultado e retornar uma quantidade inteira de meses.

```
228  -- 2. MONTHS_BETWEEN: Retorna o número de meses entre duas datas.
229
230  SELECT
231      first_name,
232      hire_date,
233      TRUNC(MONTHS_BETWEEN(sysdate, hire_date), 0) meses_ativo
234  FROM employees;
235
236
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,026 segundos

	FIRST_NAME	HIRE_DATE	MESES_ATIVO
1	Donald	21/06/99	276
2	Douglas	13/01/00	269
3	Jennifer	17/09/87	417
4	Michael	17/02/96	316
5	Pat	17/08/97	298
6	Susan	07/06/94	337
7	Hermann	07/06/94	337
8	Shelley	07/06/94	337

ADD_MONTHS

A função **ADD_MONTHS** retorna a data final depois de **adicionarmos** uma quantidade de meses a uma determinada data.

```
242  -- 3. ADD_MONTHS: Adiciona meses a uma data.
243
244  SELECT
245      sysdate,
246      ADD_MONTHS(sysdate, 3)
247  FROM dual;
248
249
250
251
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,008 segundos

	SYSDATE	ADD_MONTHS(SYSDATE,3)
1	11/07/22	11/10/22

NEXT_DAY

A função **NEXT_DAY** retorna o **próximo dia** relativo à data especificada.

```
148  -- 4. NEXT_DAY: Retorna o próximo dia relativo à data especificada.
149
150  SELECT
151      sysdate,
152      NEXT_DAY(sysdate, 'DOMINGO')
153  FROM dual;
154
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,003 segundos

	SYSDATE	NEXT_DAY(SYSDATE,'DOMINGO')
1	31/07/22	07/08/22

LAST_DAY

A função **LAST_DAY** retorna o **último dia do mês** referente ao mês de uma determinada data.

Como sysdate se passa em julho, o último dia de julho é o dia 31.

```
271  -- 5. LAST_DAY: Retorna o último dia do mês.
272
273  SELECT
274      sysdate,
275      LAST_DAY(sysdate)
276  FROM dual;
277
278
279
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,008 segundos

	SYSDATE	LAST_DAY(SYSDATE)
1	11/07/22	31/07/22

EXTRACT

A função **EXTRACT** retorna determinadas **informações** de uma data, como por exemplo: dia, mês e ano.

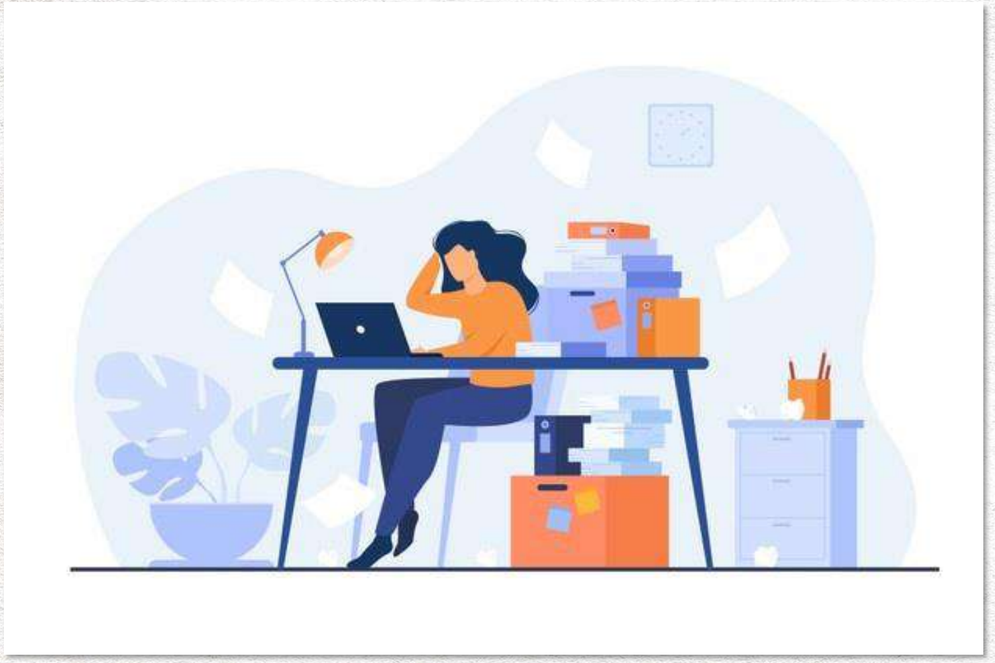
```
280  -- 6. EXTRACT: Extrai informações de uma data
281
282  SELECT
283      sysdate data_atual,
284      EXTRACT(YEAR FROM sysdate) ano,
285      EXTRACT(MONTH FROM sysdate) mes,
286      EXTRACT(DAY FROM sysdate) dia
287  FROM dual;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,007 segundos

	DATA_ATUAL	ANO	MES	DIA
1	11/07/22	2022	7	11

EXERCÍCIOS



1

Questão 1

A empresa deseja padronizar os e-mails de seus funcionários, garantindo que todos os endereços sejam armazenados em letras minúsculas.

Escreva uma consulta para exibir o nome do funcionário e seu e-mail convertido para minúsculas.

(Utilize a tabela employees)

2

Questão 2

A empresa está analisando os salários de seus funcionários para entender os valores remanescentes após a divisão dos salários por 1000. Esse tipo de análise ajuda a identificar padrões nos salários e pode ser útil para ajustes financeiros ou na definição de faixas salariais.

Crie uma consulta que mostre o nome de cada funcionário e se seu salário, após dividido por mil, deixa resto ou não.

(Utilize a tabela employees)

EXERCÍCIOS

3

Questão 3

A equipe de recrutamento quer exibir os títulos dos cargos com a primeira letra de cada palavra em maiúscula para maior legibilidade.

Crie uma consulta que retorne o nome completo do funcionário em uma única coluna, bem como seu job_id nos moldes acima especificados.

(Utilize a tabela employees)

4

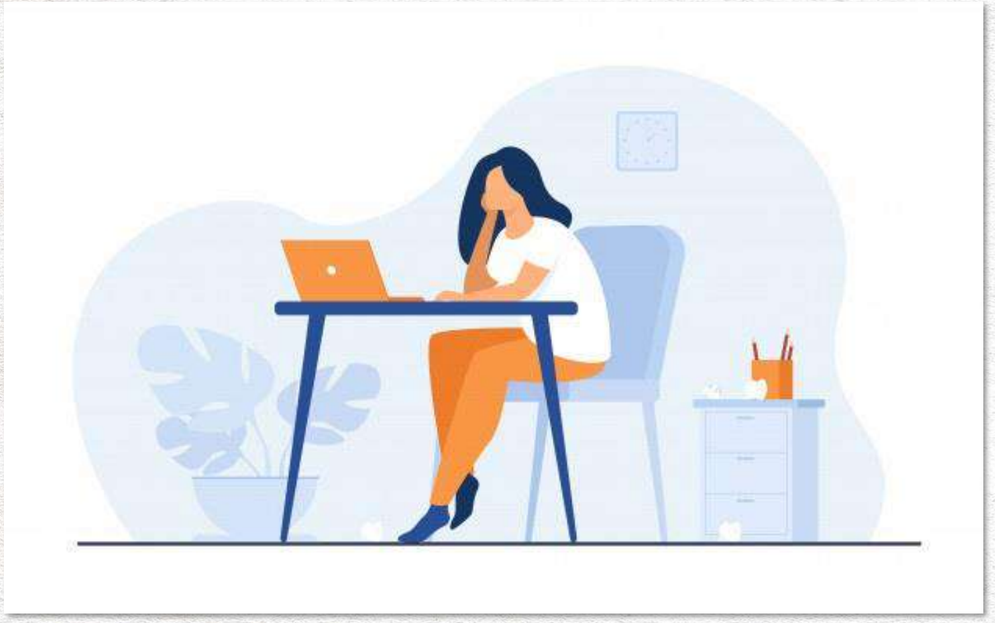
Questão 4

O setor de RH da empresa precisa organizar os contatos dos funcionários por código de área telefônico.

Para isso, devemos criar uma consulta que retorne o nome e sobrenome do funcionário, bem como extraia os três primeiros dígitos dos números de telefone cadastrados no banco de dados, pois esse código representa a região do telefone e facilitará a segmentação dos funcionários por localidade.

(Utilize a tabela employees)

EXERCÍCIOS



5

Questão 5

O departamento de Recursos Humanos está analisando os cargos da empresa para identificar quais funções estão relacionadas a vendas. Para isso, é preciso localizar a posição da palavra "Sales" dentro dos cargos cadastrados no banco de dados. Se o cargo estiver relacionado a vendas, será retornada a posição da palavra 'Sales'; caso contrário, será retornado o valor 0, uma vez que não existe a palavra 'Sales' nos cargos não relacionados a vendas. Essa análise ajudará na organização de treinamentos específicos para a equipe de vendas.

Crie a consulta que retorne o job_title e a posição da palavra 'Sales' dentro de cada job_title.

(Utilize a tabela jobs)

6

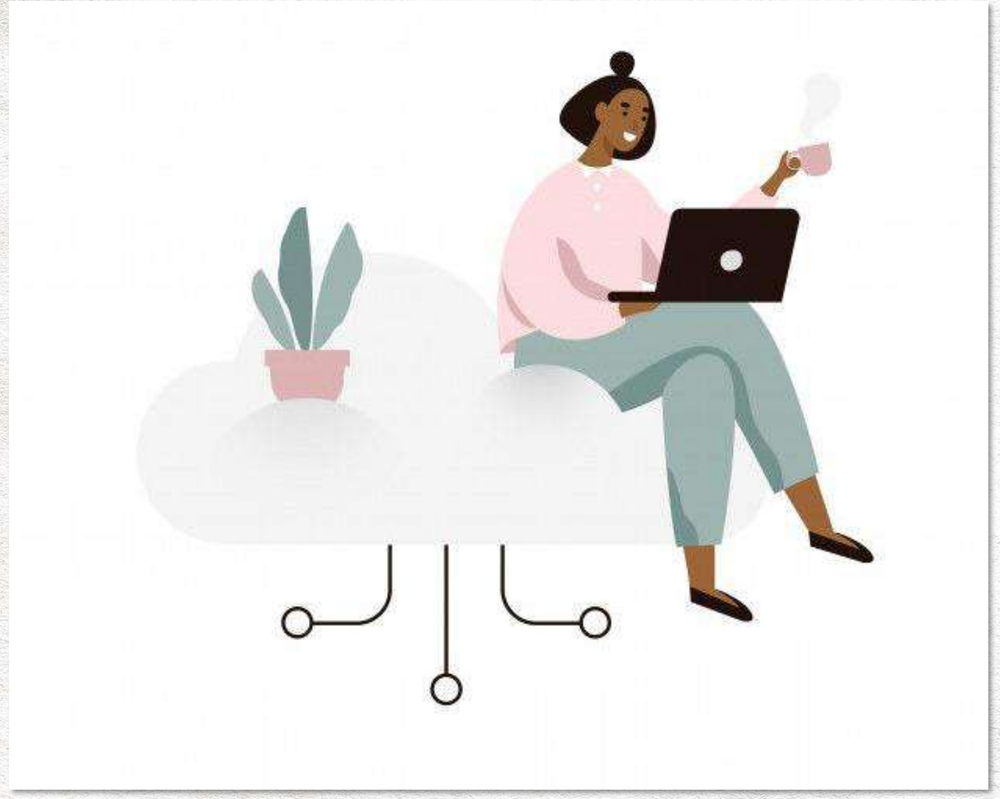
Questão 6

A equipe de controle está preparando um relatório para impressão e deseja que todos os nomes dos departamentos apareçam em letras maiúsculas.

Crie uma consulta que exiba os nomes dos departamentos em caixa alta.

(Utilize a tabela departments)

EXERCÍCIOS



7

Questão 7

A área de Recursos Humanos da empresa está realizando um levantamento sobre o tempo de permanência dos funcionários em seus respectivos cargos. O objetivo é analisar o tempo de experiência acumulado de cada colaborador em suas funções anteriores e utilizá-lo para decisões sobre promoções e treinamentos.

Crie uma consulta que retorne o ID do funcionário, o ID do cargo, a data de início e a data final, além da contagem de quantos meses o funcionário se manteve no respectivo cargo.

(Utilize a tabela job_history)

8

Questão 8

A empresa decidiu substituir a sigla "MAN" (de manager) por "DIR" (de director) nos códigos dos cargos para padronização.

Crie a consulta que faça essa substituição.

(Utilize a tabela jobs)

EXERCÍCIOS

9

Questão 9

A empresa está planejando uma análise para identificar quando os funcionários completam seis meses de trabalho, a fim de avaliar o momento ideal para promoções ou revisões salariais. Essa análise ajudará a garantir que as promoções sejam feitas de forma estratégica e no momento adequado para cada colaborador.

Crie uma consulta que retorne o nome do funcionário, sua data de contratação e a data prevista para sua promoção, baseada no marco de seis meses de trabalho.

(Utilize a tabela employees)

10

Questão 10

A empresa deseja analisar o percentual de comissão de seus funcionários. Para isso, é necessário criar um relatório que mostre o nome de cada funcionário, a comissão recebida, a comissão arredondada para uma casa decimal e a comissão truncada para uma casa decimal. O objetivo é entender como essas duas operações (arredondamento e truncamento) impactam no valor final da comissão.

Crie a consulta que retorne as informações desejadas.

(Utilize a tabela employees)

EXERCÍCIOS

11

Questão 11

A equipe de análise precisa extrair informações específicas da data de contratação para estatísticas.

Crie uma consulta que retorne o dia, o mês e o ano da data de contratação em colunas separadas.

(Utilize a tabela employees)

12

Questão 12

O time de TI está padronizando os números de telefone dos funcionários para garantir que todos tenham o mesmo comprimento no sistema.

Para isso, você precisa criar uma consulta que:

- ✓ Exiba o nome e o sobrenome de cada funcionário.
- ✓ Exiba o número de telefone de cada funcionário.
- ✓ Mostre quantos caracteres cada número possui.
- ✓ Garanta que todos os números tenham 20 caracteres, adicionando o caractere "#" à esquerda caso faltem caracteres.

Essa padronização ajudará na geração de relatórios e integração com outros sistemas.

(Utilize a tabela employees)

GABARITO EXPLICATIVO

1

Questão 1

A empresa deseja padronizar os e-mails de seus funcionários, garantindo que todos os endereços sejam armazenados em letras minúsculas.

Escreva uma consulta para exibir o nome do funcionário e seu e-mail convertido para minúsculas.

(Utilize a tabela employees)

Explicação e Resultado

Para exibir o nome do funcionário e seu e-mail convertido para letras minúsculas, fazemos o seguinte:

Selecionamos as colunas desejadas da tabela **employees**:

- ✓ **first_name**: para exibir o nome do funcionário.
- ✓ **LOWER(email) AS email_padronizado**: aplicamos a função **LOWER()** para converter o e-mail para minúsculas e renomeamos a coluna para **email_padronizado**.

```
SELECT
    first_name,
    LOWER(email) AS email_padronizado
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

A empresa deseja padronizar os e-mails de seus funcionários, garantindo que todos os endereços sejam retornados em letras minúsculas.

Escreva uma consulta para exibir o nome do funcionário e seu e-mail convertido para minúsculas.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, garantimos que os e-mails dos 107 funcionários sejam retornados no mesmo padrão:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,017 segundos

	FIRST_NAME	EMAIL_PADRONIZADO
1	Steven	sking
2	Neena	nkochhar
3	Lex	ldehaan
4	Alexander	ahunold
5	Bruce	bernst
6	David	daustin
7	Valli	vpatabal
8	Diana	dlorentz
9	Nancy	ngreenbe
10	Daniel	dfaviet
11	John	jchen
12	Ismael	isciarra
13	Jose Manuel	jmurman
14	Luis	lpopp
15	Den	drapheal
16	Alexander	akhoo
17	Shelli	sbaida
18	Sigal	stobias
19	Guy	ghimuro

GABARITO EXPLICATIVO

2

Questão 2

A empresa está analisando os salários de seus funcionários para entender os valores remanescentes após a divisão dos salários por 1000. Esse tipo de análise ajuda a identificar padrões nos salários e pode ser útil para ajustes financeiros ou na definição de faixas salariais.

Crie uma consulta que mostre o nome de cada funcionário e se seu salário, após dividido por mil, deixa resto ou não.

(Utilize a tabela employees)

Explicação e Resultado

Para exibir o nome do funcionário e verificar o restante da divisão do salário por 1000, seguimos estes passos:

Selecionamos as colunas necessárias da tabela **employees**:

- ✓ **first_name**: para exibir o nome do funcionário.
- ✓ **salary**: para mostrar o valor do salário.
- ✓ **MOD(salary, 1000) AS resto_divisao**: utilizamos a função **MOD()** para calcular o resto da divisão do salário por 1000 e renomeamos a coluna como **resto_divisao**:

```
SELECT
    first_name,
    salary,
    MOD(salary, 1000) AS resto_divisao
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A empresa está analisando os salários de seus funcionários para entender os valores remanescentes após a divisão dos salários por 1000. Esse tipo de análise ajuda a identificar padrões nos salários e pode ser útil para ajustes financeiros ou na definição de faixas salariais.

Crie uma consulta que mostre o nome de cada funcionário e se seu salário, após dividido por mil, deixa resto ou não.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, ao executamos a consulta, conseguimos identificar se há valores residuais na divisão dos salários dos 107 funcionários:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em

	FIRST_NAME	SALARY	RESTO_DIVISAO
1	Steven	24000	0
2	Neena	17000	0
3	Lex	17000	0
4	Alexander	9000	0
5	Bruce	6000	0
6	David	4800	800
7	Valli	4800	800
8	Diana	4200	200
9	Nancy	12000	0
10	Daniel	9000	0
11	John	8200	200
12	Ismael	7700	700
13	Jose Manuel	7800	800
14	Luis	6900	900

GABARITO EXPLICATIVO

3

Questão 3

A equipe de recrutamento quer exibir os títulos dos cargos com a primeira letra de cada palavra em maiúscula para maior legibilidade.

Crie uma consulta que retorne o nome completo do funcionário em uma única coluna, bem como seu job_id nos moldes acima especificados.

(Utilize a tabela employees)

Explicação e Resultado

Para exibir o nome completo do funcionário e o job_id com a primeira letra de cada palavra em maiúscula, seguimos estes passos:

Criamos a coluna do nome completo usando **CONCAT()**:

- ✓ **CONCAT(first_name, CONCAT(' ', last_name)) AS nome_completo**: unimos as colunas **first_name** e **last_name** da tabela **employees**, adicionando um **espaço** entre nome e sobrenome e renomeando a coluna para **nome_completo**.

Formatamos o **job_id** para ter a primeira letra de cada palavra em maiúscula:

- ✓ **INITCAP(job_id) AS job_id_formatado**: a função **INITCAP()** transforma a primeira letra de cada palavra em maiúscula. Renomeamos a coluna para **job_id_formatado**:

```
SELECT
    CONCAT(first_name, CONCAT(' ', last_name)) nome_completo,
    INITCAP(job_id) AS job_id_formatado
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

A equipe de recrutamento quer exibir os títulos dos cargos com a primeira letra de cada palavra em maiúscula para maior legibilidade.

Crie uma consulta que retorne o nome completo do funcionário em uma única coluna, bem como seu job_id nos moldes acima especificados.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, retornamos o resultado abaixo, que exibe o nome completo em uma única coluna e o job_id com as primeiras letras maiúsculas:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,

	NOME_COMPLETO	JOB_ID_FORMATADO
1	William Gietz	Ac_Account
2	Shelley Higgins	Ac_Mgr
3	Jennifer Whalen	Ad_Asst
4	Steven King	Ad_Pres
5	Neena Kochhar	Ad_Vp
6	Lex De Haan	Ad_Vp
7	Daniel Faviet	Fi_Account
8	John Chen	Fi_Account
9	Ismael Sciarra	Fi_Account
10	Jose Manuel Urman	Fi_Account
11	Luis Popp	Fi_Account
12	Nancy Greenberg	Fi_Mgr
13	Susan Mavris	Hr_Rep
14	Alexander Hunold	It_Prog
15	Bruce Ernst	It_Prog
16	David Austin	It_Prog
17	Valli Pataballa	It_Prog

GABARITO EXPLICATIVO

4

Questão 4

O setor de RH da empresa precisa organizar os contatos dos funcionários por código de área telefônico.

Para isso, devemos criar uma consulta que retorne o nome e sobrenome do funcionário, bem como extraia os três primeiros dígitos dos números de telefone cadastrados no banco de dados, pois esse código representa a região do telefone e facilitará a segmentação dos funcionários por localidade.

(Utilize a tabela employees)

Explicação e Resultado

Para extrair o código de área dos telefones dos funcionários, seguimos estes passos:

Selecionamos os nomes dos funcionários da tabela **employees**:

- ✓ **first_name** e **last_name** para exibir o nome completo.

Extraímos os três primeiros dígitos do telefone:

- ✓ **SUBSTR(phone_number, 1, 3)**: a função **SUBSTR()** captura os caracteres a partir da posição 1, pegando os três primeiros dígitos do **phone_number**. Renomeamos esta coluna para **codigo_de_area**:

```
SELECT
    first_name,
    last_name,
    SUBSTR(phone_number, 1, 3) codigo_de_area
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

O setor de RH da empresa precisa organizar os contatos dos funcionários por código de área telefônico.

Para isso, devemos criar uma consulta que retorne o nome e sobrenome do funcionário, bem como extraia os três primeiros dígitos dos números de telefone cadastrados no banco de dados, pois esse código representa a região do telefone e facilitará a segmentação dos funcionários por localidade.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, retornamos o nome e sobrenome dos 107 funcionários, bem como os códigos de área de seus telefones:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,019 se

	FIRST_NAME	LAST_NAME	CODIGO_DE_AREA
1	Steven	King	515
2	Neena	Kochhar	515
3	Lex	De Haan	515
4	Alexander	Hunold	590
5	Bruce	Ernst	590
6	David	Austin	590
7	Valli	Pataballa	590
8	Diana	Lorentz	590
9	Nancy	Greenberg	515
10	Daniel	Faviet	515
11	John	Chen	515
12	Ismael	Sciarra	515
13	Jose Manuel	Urman	515
14	Luis	Popp	515
15	Den	Raphaely	515
16	Alexander	Khoo	515

GABARITO EXPLICATIVO

5

Questão 5

O departamento de Recursos Humanos está analisando os cargos da empresa para identificar quais funções estão relacionadas a vendas. Para isso, é preciso localizar a posição da palavra "Sales" dentro dos cargos cadastrados no banco de dados. Se o cargo estiver relacionado a vendas, será retornada a posição da palavra 'Sales'; caso contrário, será retornado o valor 0, uma vez que não existe a palavra 'Sales' nos cargos não relacionados a vendas. Essa análise ajudará na organização de treinamentos específicos para a equipe de vendas.

Crie a consulta que retorne o job_title e a posição da palavra 'Sales' dentro de cada job_title.

(Utilize a tabela jobs)

Explicação e Resultado

Para identificar se um cargo está relacionado a vendas, seguimos estes passos:

Selecionamos o cargo na tabela **jobs**:

- ✓ A coluna **job_title** exibe o nome do cargo.

Buscamos a posição da palavra "Sales":

- ✓ **INSTR(job_title, 'Sales')**: retorna a posição da primeira ocorrência da palavra **"Sales"** dentro do **job_title**. Se a palavra estiver presente, retorna a posição inicial dentro do texto. Se não existir, retorna 0. Renomeamos a coluna para **posicao**:

```
SELECT
    job_title,
    INSTR(job_title, 'Sales') posicao
FROM jobs;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

O departamento de Recursos Humanos está analisando os cargos da empresa para identificar quais funções estão relacionadas a vendas. Para isso, é preciso localizar a posição da palavra "Sales" dentro dos cargos cadastrados no banco de dados. Se o cargo estiver relacionado a vendas, será retornada a posição da palavra 'Sales'; caso contrário, será retornado o valor 0, uma vez que não existe a palavra 'Sales' nos cargos não relacionados a vendas. Essa análise ajudará na organização de treinamentos específicos para a equipe de vendas.

Crie a consulta que retorne o job_title e a posição da palavra 'Sales' dentro de cada job_title.

(Utilize a tabela jobs)

Explicação e Resultado

Com o resultado exibido, o RH pode facilmente identificar, entre os 19 cargos, aqueles relacionados a vendas ('Sales Manager' e 'Sales Representative'):

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,003 segundos

	JOB_TITLE	POSICAO
1	President	0
2	Administration Vice President	0
3	Administration Assistant	0
4	Finance Manager	0
5	Accountant	0
6	Accounting Manager	0
7	Public Accountant	0
8	Sales Manager	1
9	Sales Representative	1
10	Purchasing Manager	0
11	Purchasing Clerk	0
12	Stock Manager	0
13	Stock Clerk	0
14	Shipping Clerk	0
15	Programmer	0
16	Marketing Manager	0
17	Marketing Representative	0
18	Human Resources Representative	0
19	Public Relations Representative	0

GABARITO EXPLICATIVO

6

Questão 6

A equipe de controle está preparando um relatório para impressão e deseja que todos os nomes dos departamentos apareçam em letras maiúsculas.

Crie uma consulta que exiba os nomes dos departamentos em caixa alta.

(Utilize a tabela departments)

Explicação e Resultado

Para exibir os nomes dos departamentos em letras maiúsculas, selecionamos a coluna **department_name** da tabela **departments**, utilizando-a como parâmetro da função **UPPER()**, uma vez que essa função transforma todos os caracteres da string em letras maiúsculas. Renomeamos a coluna para **departamento**:

```
SELECT
    UPPER(department_name) AS departamento
FROM departments;
```

Dessa forma, o relatório impresso terá os nomes dos 27 departamentos padronizados em caixa alta, garantindo uma apresentação mais uniforme:

Resultado da Consulta

SQL | Todas as Linhas

	DEPARTAMENTO
1	ADMINISTRATION
2	MARKETING
3	PURCHASING
4	HUMAN RESOURCES
5	SHIPPING
6	IT
7	PUBLIC RELATIONS
8	SALES
9	EXECUTIVE
10	FINANCE
11	ACCOUNTING
12	TREASURY
13	CORPORATE TAX
14	CONTROL AND CREDIT
15	SHAREHOLDER SERVICES
16	BENEFITS
17	MANUFACTURING
18	CONSTRUCTION
19	CONTRACTING
20	OPERATIONS
21	IT SUPPORT
22	NOC
23	IT HELPDESK
24	GOVERNMENT SALES
25	RETAIL SALES
26	RECRUITING
27	PAYROLL

GABARITO EXPLICATIVO

7

Questão 7

A área de Recursos Humanos da empresa está realizando um levantamento sobre o tempo de permanência dos funcionários em seus respectivos cargos. O objetivo é analisar o tempo de experiência acumulado de cada colaborador em suas funções anteriores e utilizá-lo para decisões sobre promoções e treinamentos.

Crie uma consulta que retorne o ID do funcionário, o ID do cargo, a data de início e a data final, além da contagem de quantos meses o funcionário se manteve no respectivo cargo.

(Utilize a tabela job_history)

Explicação e Resultado

Para calcular o tempo de permanência dos funcionários em seus cargos, seguimos estes passos:

Selecionamos as colunas principais da tabela **job_history**:

- ✓ **employee_id**: identifica o funcionário.
- ✓ **job_id**: representa o cargo ocupado.
- ✓ **start_date** e **end_date**: indicam o período em que o funcionário esteve no cargo.

Calculamos o tempo de permanência em meses:

- ✓ **MONTHS_BETWEEN(end_date, start_date)**: retorna a diferença entre as datas em meses.
- ✓ **TRUNC(..., 0)**: remove as casas decimais, exibindo apenas o número inteiro de meses. Renomeamos a coluna para **meses_no_cargo**:

```
SELECT
    employee_id,
    job_id,
    start_date,
    end_date,
    TRUNC(MONTHS_BETWEEN(end_date, start_date), 0) AS meses_no_cargo
FROM job_history;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área de Recursos Humanos da empresa está realizando um levantamento sobre o tempo de permanência dos funcionários em seus respectivos cargos. O objetivo é analisar o tempo de experiência acumulado de cada colaborador em suas funções anteriores e utilizá-lo para decisões sobre promoções e treinamentos.

Crie uma consulta que retorne o ID do funcionário, o ID do cargo, a data de início e a data final, além da contagem de quantos meses o funcionário se manteve no respectivo cargo.

(Utilize a tabela job_history)

Explicação e Resultado

Assim, conseguimos um relatório preciso sobre o tempo de experiência dos funcionários em seus respectivos cargos:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 10 em 0,015 segundos

	EMPLOYEE_ID	JOB_ID	START_DATE	END_DATE	MESES_NO_CARGO
1	102	IT_PROG	13/01/93	24/07/98	66
2	101	AC_ACCOUNT	21/09/89	27/10/93	49
3	101	AC_MGR	28/10/93	15/03/97	40
4	201	MK_REP	17/02/96	19/12/99	46
5	114	ST_CLERK	24/03/98	31/12/99	21
6	122	ST_CLERK	01/01/99	31/12/99	11
7	200	AD_ASST	17/09/87	17/06/93	69
8	176	SA_REP	24/03/98	31/12/98	9
9	176	SA_MAN	01/01/99	31/12/99	11
10	200	AC_ACCOUNT	01/07/94	31/12/98	53

GABARITO EXPLICATIVO

8

Questão 8

A empresa decidiu substituir a sigla "MAN" (de manager) por "DIR" (de director) nos códigos dos cargos para padronização.

Crie a consulta que faça essa substituição.

(Utilize a tabela jobs)

Explicação e Resultado

Para atualizar os códigos dos cargos conforme a nova padronização, seguimos estes passos:

Selecionamos a coluna necessária da tabela **jobs**:

- ✓ **job_id**: exibe o código original do cargo.

Substituímos a sigla **"MAN"** por **"DIR"**:

- ✓ Utilizamos a função **REPLACE(job_id, 'MAN', 'DIR')**, que substitui todas as ocorrências de "MAN" dentro do código do cargo para "DIR". Renomeamos a coluna para **codigo_atualizado**:

```
SELECT
    job_id,
    REPLACE(job_id, 'MAN', 'DIR') AS codigo_atualizado
FROM jobs;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A empresa decidiu substituir a sigla "MAN" (de manager) por "DIR" (de director) nos códigos dos cargos para padronização.

Crie a consulta que faça essa substituição.

(Utilize a tabela jobs)

Explicação e Resultado

Assim, obtemos o seguinte resultado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas:

	JOB_ID	CODIGO_ATUALIZADO
1	AC_ACCOUNT	AC_ACCOUNT
2	AC_MGR	AC_MGR
3	AD_ASST	AD_ASST
4	AD_PRES	AD_PRES
5	AD_VP	AD_VP
6	FI_ACCOUNT	FI_ACCOUNT
7	FI_MGR	FI_MGR
8	HR_REP	HR_REP
9	IT_PROG	IT_PROG
10	MK_MAN	MK_DIR
11	MK_REP	MK_REP
12	PR_REP	PR_REP
13	PU_CLERK	PU_CLERK
14	PU_MAN	PU_DIR
15	SA_MAN	SA_DIR
16	SA_REP	SA_REP
17	SH_CLERK	SH_CLERK
18	ST_CLERK	ST_CLERK
19	ST_MAN	ST_DIR

GABARITO EXPLICATIVO

9

Questão 9

A empresa está planejando uma análise para identificar quando os funcionários completam seis meses de trabalho, a fim de avaliar o momento ideal para promoções ou revisões salariais. Essa análise ajudará a garantir que as promoções sejam feitas de forma estratégica e no momento adequado para cada colaborador.

Crie uma consulta que retorne o nome do funcionário, sua data de contratação e a data prevista para sua promoção, baseada no marco de seis meses de trabalho.

(Utilize a tabela employees)

Explicação e Resultado

Para calcular a data prevista para a promoção de cada funcionário após seis meses de trabalho, seguimos estes passos:

Selecionamos as colunas necessárias da tabela **employees**:

- ✓ **first_name**: exibe o primeiro nome do funcionário.
- ✓ **hire_date**: mostra a data de contratação.

Calculamos a data da promoção:

- ✓ Utilizamos **ADD_MONTHS(hire_date, 6)**, que adiciona seis meses à data de contratação. Renomeamos a coluna para **data_promocao**:

```
SELECT
    first_name,
    hire_date,
    ADD_MONTHS(hire_date, 6) AS data_promocao
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

A empresa está planejando uma análise para identificar quando os funcionários completam seis meses de trabalho, a fim de avaliar o momento ideal para promoções ou revisões salariais. Essa análise ajudará a garantir que as promoções sejam feitas de forma estratégica e no momento adequado para cada colaborador.

Crie uma consulta que retorne o nome do funcionário, sua data de contratação e a data prevista para sua promoção, baseada no marco de seis meses de trabalho.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, obtemos uma visão clara do momento ideal para eventuais promoções ou revisões salariais, facilitando o planejamento estratégico do RH:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,02

	FIRST_NAME	HIRE_DATE	DATA_PROMOCAO
1	Steven	17/06/87	17/12/87
2	Neena	21/09/89	21/03/90
3	Lex	13/01/93	13/07/93
4	Alexander	03/01/90	03/07/90
5	Bruce	21/05/91	21/11/91
6	David	25/06/97	25/12/97
7	Valli	05/02/98	05/08/98
8	Diana	07/02/99	07/08/99
9	Nancy	17/08/94	17/02/95
10	Daniel	16/08/94	16/02/95
11	John	28/09/97	28/03/98
12	Ismael	30/09/97	31/03/98
13	Jose Manuel	07/03/98	07/09/98
14	Luis	07/12/99	07/06/00

GABARITO EXPLICATIVO

10

Questão 10

A empresa deseja analisar o percentual de comissão de seus funcionários. Para isso, é necessário criar um relatório que mostre o nome de cada funcionário, a comissão recebida, a comissão arredondada para uma casa decimal e a comissão truncada para uma casa decimal. O objetivo é entender como essas duas operações (arredondamento e truncamento) impactam no valor final da comissão.

Crie a consulta que retorne as informações desejadas.

(Utilize a tabela employees)

Explicação e Resultado

Para analisar o impacto do arredondamento e do truncamento na comissão dos funcionários, seguimos estes passos:

Selecionamos as colunas necessárias da tabela **employees**:

- ✓ **first_name**: exibe o nome do funcionário.
- ✓ **commission_pct**: mostra o percentual de comissão original.

Aplicamos as funções de arredondamento e truncamento:

- ✓ **ROUND(commission_pct, 1)**: arredonda a comissão para uma casa decimal. Renomeamos a coluna para comissao_arredondada.
- ✓ **TRUNC(commission_pct, 1)**: remove as casas decimais extras, mantendo apenas uma. Renomeamos a coluna para comissao_truncada:

```
SELECT
    first_name,
    commission_pct,
    ROUND(commission_pct, 1) AS comissao_arredondada,
    TRUNC(commission_pct, 1) AS comissao_truncada
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

10

Questão 10

A empresa deseja analisar o percentual de comissão de seus funcionários. Para isso, é necessário criar um relatório que mostre o nome de cada funcionário, a comissão recebida, a comissão arredondada para uma casa decimal e a comissão truncada para uma casa decimal. O objetivo é entender como essas duas operações (arredondamento e truncamento) impactam no valor final da comissão.

Crie a consulta que retorne as informações desejadas.

(Utilize a tabela employees)

Explicação e Resultado

Com essa análise, a empresa pode estudar como cada método afeta o valor final da comissão:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,021 segundos

	FIRST_NAME	COMMISSION_PCT	COMISSAO_ARREDONDADA	COMISSAO_TRUNCADA
44	Randall	(null)	(null)	(null)
45	Peter	(null)	(null)	(null)
46	John	0,4	0,4	0,4
47	Karen	0,3	0,3	0,3
48	Alberto	0,3	0,3	0,3
49	Gerald	0,3	0,3	0,3
50	Eleni	0,2	0,2	0,2
51	Peter	0,3	0,3	0,3
52	David	0,25	0,3	0,2
53	Peter	0,25	0,3	0,2
54	Christopher	0,2	0,2	0,2
55	Nanette	0,2	0,2	0,2
56	Oliver	0,15	0,2	0,1
57	Janette	0,35	0,4	0,3
58	Patrick	0,35	0,4	0,3

GABARITO EXPLICATIVO

11

Questão 11

A equipe de análise precisa extrair informações específicas da data de contratação para estatísticas.

Crie uma consulta que retorne o dia, o mês e o ano da data de contratação em colunas separadas.

(Utilize a tabela employees)

Explicação e Resultado

Para extrair informações detalhadas da data de contratação dos funcionários, seguimos estes passos:

Selecionamos as colunas necessárias da tabela **employees**:

- ✓ **first_name**: exibe o nome do funcionário.
- ✓ **hire_date**: mostra a data completa de contratação.

Utilizamos a função **EXTRACT()** para dividir a data:

- ✓ **EXTRACT(DAY FROM hire_date)**: extrai o dia da contratação. Renomeamos a coluna para **dia_contratacao**.
- ✓ **EXTRACT(MONTH FROM hire_date)**: extrai o mês da contratação. Renomeamos a coluna para **mes_contratacao**.
- ✓ **EXTRACT(YEAR FROM hire_date)**: extrai o ano da contratação. Renomeamos a coluna para **ano_contratacao**:

```
SELECT
    first_name,
    hire_date,
    EXTRACT(DAY FROM hire_date) AS dia_contratacao,
    EXTRACT(MONTH FROM hire_date) AS mes_contratacao,
    EXTRACT(YEAR FROM hire_date) AS ano_contratacao
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

11

Questão 11

A equipe de análise precisa extrair informações específicas da data de contratação para estatísticas.

Crie uma consulta que retorne o dia, o mês e o ano da data de contratação em colunas separadas.

(Utilize a tabela employees)

Explicação e Resultado

Com essa extração, a equipe pode realizar estatísticas detalhadas sobre admissões ao longo do tempo:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,015 segundos

	FIRST_NAME	HIRE_DATE	DIA_CONTRATACAO	MES_CONTRATACAO	ANO_CONTRATACAO
1	Steven	17/06/87	17	6	1987
2	Neena	21/09/89	21	9	1989
3	Lex	13/01/93	13	1	1993
4	Alexander	03/01/90	3	1	1990
5	Bruce	21/05/91	21	5	1991
6	David	25/06/97	25	6	1997
7	Valli	05/02/98	5	2	1998
8	Diana	07/02/99	7	2	1999
9	Nancy	17/08/94	17	8	1994
10	Daniel	16/08/94	16	8	1994
11	John	28/09/97	28	9	1997
12	Ismael	30/09/97	30	9	1997
13	Jose Manuel	07/03/98	7	3	1998
14	Luis	07/12/99	7	12	1999
15	Den	07/12/94	7	12	1994

GABARITO EXPLICATIVO

12

Questão 12

O time de TI está padronizando os números de telefone dos funcionários para garantir que todos tenham o mesmo comprimento no sistema.

Para isso, você precisa criar uma consulta que:

- ✓ Exiba o nome e o sobrenome de cada funcionário.
- ✓ Exiba o número de telefone de cada funcionário.
- ✓ Mostre quantos caracteres cada número possui.
- ✓ Garanta que todos os números tenham 20 caracteres, adicionando o caractere "#" à esquerda caso faltem caracteres.

Essa padronização ajudará na geração de relatórios e integração com outros sistemas.

(Utilize a tabela employees)

Explicação e Resultado

Para padronizar os números de telefone dos funcionários, seguimos estes passos:

Selecionamos as colunas necessárias da tabela **employees**:

- ✓ **first_name** e **last_name**: exibe o nome e sobrenome do funcionário.
- ✓ **phone_number**: exibe o número de telefone original.

Verificamos o tamanho dos números:

- ✓ **LENGTH(phone_number)**: retorna a quantidade de caracteres do telefone atual. Renomeamos a coluna para **tamanho_original**.

Garantimos que todos tenham 20 caracteres:

- ✓ **LPAD(phone_number, 20, '#')**: preenche à esquerda com # até que o número tenha 20 caracteres. Renomeamos a coluna para **telefone_preenchido_esquerda**:

```
SELECT
    first_name,
    last_name,
    phone_number,
    LENGTH(phone_number) AS tamanho_original,
    LPAD(phone_number, 20, '#') AS telefone_preenchido_esquerda
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

12

Questão 12

O time de TI está padronizando os números de telefone dos funcionários para garantir que todos tenham o mesmo comprimento no sistema.

Para isso, você precisa criar uma consulta que:

- ✓ Exiba o nome e o sobrenome de cada funcionário.
- ✓ Exiba o número de telefone de cada funcionário.
- ✓ Mostre quantos caracteres cada número possui.
- ✓ Garanta que todos os números tenham 20 caracteres, adicionando o caractere "#" à esquerda caso faltem caracteres.

Essa padronização ajudará na geração de relatórios e integração com outros sistemas.

(Utilize a tabela employees)

Explicação e Resultado

Com isso, retornamos o resultado com o tamanho original dos telefones, assim como com a padronização de 20 caracteres:

Resultado da Consulta ×

SQL | Todas as Linhas Extraídas: 107 em 0,02 segundos

	FIRST_NAME	LAST_NAME	PHONE_NUMBER	TAMANHO_ORIGINAL	TELEFONE_PREENCHIDO_ESQUERDA
38	Renske	Ladwig	650.121.1234	12	########650.121.1234
39	Stephen	Stiles	650.121.2034	12	########650.121.2034
40	John	Seo	650.121.2019	12	########650.121.2019
41	Joshua	Patel	650.121.1834	12	########650.121.1834
42	Trenna	Rajs	650.121.8009	12	########650.121.8009
43	Curtis	Davies	650.121.2994	12	########650.121.2994
44	Randall	Matos	650.121.2874	12	########650.121.2874
45	Peter	Vargas	650.121.2004	12	########650.121.2004
46	John	Russell	011.44.1344.429268	18	##011.44.1344.429268
47	Karen	Partners	011.44.1344.467268	18	##011.44.1344.467268
48	Alberto	Errazuriz	011.44.1344.429278	18	##011.44.1344.429278
49	Gerald	Cambrault	011.44.1344.619268	18	##011.44.1344.619268
50	Eleni	Zlotkey	011.44.1344.429018	18	##011.44.1344.429018
51	Peter	Tucker	011.44.1344.129268	18	##011.44.1344.129268
52	David	Bernstein	011.44.1344.345268	18	##011.44.1344.345268
53	Peter	Hall	011.44.1344.478968	18	##011.44.1344.478968
54	Christopher	Olsen	011.44.1344.498718	18	##011.44.1344.498718
55	Nanette	Cambrault	011.44.1344.987668	18	##011.44.1344.987668

MÓDULO 7

FUNÇÕES SINGLE ROW (PT. 2)

FUNÇÕES SINGLE ROW (PT. 2)

FUNÇÕES SINGLE ROW (PT. 2)

O QUE VEREMOS NESTE MÓDULO

MÓDULO 7 FUNÇÕES SINGLE ROW: CONVERSÃO e VALORES NULOS

01	FUNÇÕES SINGLE ROW	290
02	FUNÇÕES DE CONVERSÃO	291
03	TO_CHAR (COM DATAS)	292
04	TO_CHAR (COM NÚMEROS)	293
05	TO_NUMBER	294
06	TO_DATE	295
07	FUNÇÕES NULL	296
08	NVL	297
09	NVL2	298
10	COALESCE	299
11	NULLIF	300
12	RESUMO DOS MÓDULOS 6 E 7	301
13	EXERCÍCIOS	302
14	GABARITO EXPLICATIVO	306

Funções Single Row

No módulo anterior trabalhamos com as funções Single Row de números, strings e datas. Neste módulo, vamos trabalhar com as funções de conversão e funções para tratar valores nulos.

NÚMEROS

Funções para trabalhar com **NÚMEROS**.

- ROUND
- TRUNC
- MOD

STRINGS

Funções para trabalhar com **TEXTOS**.

- LOWER
- UPPER
- INITCAP
- CONCAT
- SUBSTR
- LENGTH
- INSTR
- LPAD
- RPAD
- REPLACE

DATAS

Funções para trabalhar com **DATAS**.

- MONTHS_BETWEEN
- ADD_MONTHS
- NEXT_DAY
- LAST_DAY
- EXTRACT

CONVERSÃO

Funções para **converter tipos de dados** (texto para número, texto para data, etc.)

- TO_CHAR
- TO_NUMBER
- TO_DATE

NULL

Funções para se trabalhar com **valores nulos**.

- NVL
- NVL2
- COALESCE
- NULLIF

Funções de Conversão

Agora vamos falar das FUNÇÕES DE CONVERSÃO.

Uma função de conversão nos permite converter um dado, de um determinado tipo, para um outro tipo.

Por exemplo, podemos converter um valor do tipo texto para um número, ou um número para um texto, tudo isso usando as funções de conversão.

A seguir, vamos falar sobre elas.

CONVERSÃO

Funções para **converter tipos de dados** (texto para número, texto para data, etc.)

- TO_CHAR
- TO_NUMBER
- TO_DATE

TO_CHAR (com datas)

A função **TO_CHAR** pode ser usada em duas situações: converter datas em textos, ou números em textos, aplicando uma espécie de formatação.

Ao lado, temos a função TO_CHAR sendo aplicada à sysdate, e em cada linha, retornando a data formatada de um jeito diferente.

```
-- 1. TO_CHAR: Converte números e datas para textos, usando um formato especificado.

-- a)Aplicação com DATAS

SELECT
    sysdate hoje,
    TO_CHAR(sysdate, 'YYYY') ano,
    TO_CHAR(sysdate, 'MM') num_mes,
    TO_CHAR(sysdate, 'MONTH') nome_mes,
    TO_CHAR(sysdate, 'MON') mes_abreviado,
    TO_CHAR(sysdate, 'DAY') nome_dia,
    TO_CHAR(sysdate, 'DD') num_dia,
    TO_CHAR(sysdate, 'DY') dia_abreviado,
    TO_CHAR(sysdate, 'HH12') hora_12,
    TO_CHAR(sysdate, 'HH24') hora_24,
    TO_CHAR(sysdate, 'MI') minutos,
    TO_CHAR(sysdate, 'SS') segundos,
    TO_CHAR(sysdate, 'DAY, DD "de" MONTH "de" YYYY') hora_24
FROM dual;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,013 segundos

	HOJE	ANO	NUM_MES	NOME_MES	MES_ABREVIADO	NOME_DIA	NUM_DIA	DIA_ABREVIADO	HORA_12	HORA_24	MINUTOS	SEGUNDOS	HORA_24_1
1	11/07/22	2022	07	JULY	JUL	MONDAY	11	MON	06	06	30	44	MONDAY ,

TO_CHAR (com números)

Para situações com números, temos exemplos ao lado.

Aplicamos a função TO_CHAR para formatar a coluna salary para 3 tipos de formato diferente.

Os códigos de formatação significam o seguinte:

G: Representa o separador de milhar (o ponto)

D: Representa o separador decimal (a vírgula)

L: Representa o sinal de moeda (o cifrão)

0: Representa qualquer número, mas caso um número não seja representado, ele retorna zero (permite os zeros à esquerda).

9: Representa qualquer número, mas se um número não for representado, ele não retorna nenhum outro número (não permite zeros à esquerda).

```
25  -- a)Aplicação com NÚMEROS
26
27  SELECT
28      first_name,
29      salary,
30      TO_CHAR(salary, '999G999D99') numero_sem_zeros,  --      1.000
31      TO_CHAR(salary, '000G000D00') numero_com_zeros,  --    001.000,00
32      TO_CHAR(salary, 'L999G999D99') moeda
33  FROM employees;
34
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,036 segundos

	FIRST_NAME	SALARY	NUMERO_SEM_ZEROS	NUMERO_COM_ZEROS	MOEDA
1	Donald	2600	2.600,00	002.600,00	R$2.600,00
2	Douglas	2600	2.600,00	002.600,00	R$2.600,00
3	Jennifer	4400	4.400,00	004.400,00	R$4.400,00
4	Michael	13000	13.000,00	013.000,00	R$13.000,00
5	Pat	6000	6.000,00	006.000,00	R$6.000,00
6	Susan	6500	6.500,00	006.500,00	R$6.500,00
7	Hermann	10000	10.000,00	010.000,00	R$10.000,00
8	Shelley	12000	12.000,00	012.000,00	R$12.000,00

TO_NUMBER

A função **TO_NUMBER** converte textos em números sempre que possível.

```
36  -- 2. TO_NUMBER: Converte textos para números, sempre que possível.
37
38  SELECT
39      TO_NUMBER('234,44')
40  FROM dual;
41
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,004 segundos

	TO_NUMBER('234,44')
1	234,44

TO_DATE

A função **TO_DATE** converte textos em datas.

Da mesma forma que na situação anterior, podemos ter datas que são passadas como texto, o que é ruim para o banco. Por isso, essa função ajuda a converter aquela data para o tipo de dados correto.

```
51  -- 3. TO_DATE: Converte textos para datas, sempre que possível.
52
53  SELECT
54      TO_DATE('10/05/2022', 'DD/MM/YYYY')
55  FROM dual;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,008 segundos

	TO_DATE('10/05/2022','DD/MM/YYYY')
1	10/05/22

Funções NULL

Um valor muito comum de encontrarmos em bancos de dados é o valor NULL.

Para trabalhar com ele, o SQL disponibiliza uma série de FUNÇÕES.

Mas, o que é o NULL?

NULL indica que o valor é desconhecido. Um valor nulo é diferente de zero, diferente de vazio e diferente de espaços.

Os valores NULL são usados para indicar que você pode ter um valor, mas ainda não sabe qual deve ser esse valor. Eles são espaços reservados até que você finalmente colete os dados necessários para preencher o campo da tabela com um valor real.

NULL

Funções para se trabalhar com **valores nulos.**

- NVL
- NVL2
- COALESCE
- NULLIF

NVL

A função **NVL** permite substituir o valor NULL por um outro valor, especificado no segundo argumento.

No exemplo ao lado, aplicamos a função NVL na coluna commission_pct. Sempre que o valor é null, ela retorna zero (especificado no segundo argumento da função). E quando o valor é diferente de null, ela retorna o valor original.

```
62  -- 1. NVL: Permite substituir o valor (null) por um outro valor.
63
64
65  SELECT
66      first_name,
67      commission_pct,
68      NVL(commission_pct, 0)
69  FROM employees;
70
```

Resultado da Consulta

SQL | 100 linhas extraídas em 0,087 segundos

	FIRST_NAME	COMMISSION_PCT	NVL(COMMISSION_PCT,0)
52	Curtis	(null)	0
53	Randall	(null)	0
54	Peter	(null)	0
55	John	0,4	0,4
56	Karen	0,3	0,3
57	Alberto	0,3	0,3
58	Gerald	0,3	0,3
59	Eleni	0,2	0,2

NVL2

A função **NVL2** é uma extensão da NVL.

Se o primeiro argumento não for null, retorna o segundo argumento.

Se o primeiro argumento for null, retorna o terceiro argumento.

```
67  /*
68  3. NVL2: É uma extensão do NVL.
69  Se o primeiro argumento não for null, retorna o segundo argumento.
70  Se o primeiro argumento for null, retorna o terceiro.
71  */
72
73  SELECT
74      first_name,
75      commission_pct,
76      NVL2(commission_pct, 0.5, 0)
77  FROM employees;
78
```

Resultado da Consulta

SQL | 100 linhas extraídas em 0,012 segundos

	FIRST_NAME	COMMISSION_PCT	NVL2(COMMISSION_PCT,0.5,0)
38	Renske	(null)	0
39	Stephen	(null)	0
40	John	(null)	0
41	Joshua	(null)	0
42	Trenna	(null)	0
43	Curtis	(null)	0
44	Randall	(null)	0
45	Peter	(null)	0
46	John	0,4	0,5
47	Karen	0,3	0,5
48	Alberto	0,3	0,5
49	Gerald	0,3	0,5
50	Eleni	0,2	0,5

COALESCE

A função **COALESCE** retorna o primeiro valor não nulo que for encontrado dentro de uma lista de valores.

```
94  -- 2. COALESCE: Recebe uma lista de argumentos o primeiro valor não nulo.
95
96  SELECT
97      COALESCE(NULL, NULL, NULL, NULL, 'Hashtag', NULL, 'SQL')
98  FROM dual;
99
100
101
102
103
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,012 segundos

	COALESCE(NULL,NULL,NULL,NULL,'HASHTAG',NULL,'SQL')
1	Hashtag

NULLIF

A função **NULLIF** retorna nulo se os dois argumentos da função forem iguais, e retorna o primeiro argumento caso os dois sejam diferentes.

```
120 -- 4. NULLIF: Recebe dois argumentos. Se os dois forem iguais, retorna nulo, se não retorna o 1º.
121
122 SELECT
123     NULLIF(10, 10),
124     NULLIF(10, 5)
125 FROM dual;
126
127
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,032 segundos

	NULLIF(10,10)	NULLIF(10,5)
1	(null)	10

Resumo dos Módulos 6 e 7

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. FUNÇÕES SINGLE ROW são funções que recebem argumentos e retornam um valor único. As funções Single Row podem ser divididas em funções de: Números, Textos, Datas, Conversão e Valores Nulos.

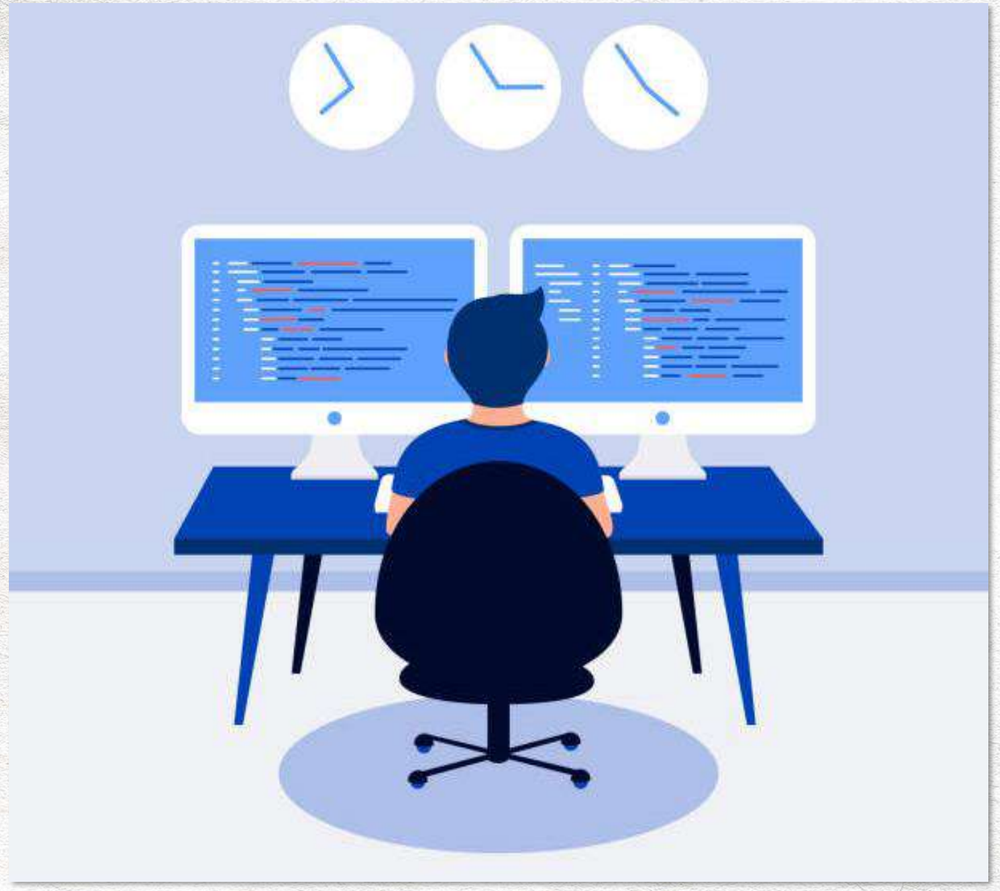
2. As FUNÇÕES DE NÚMEROS são: ROUND, TRUNC e MOD.

3. As FUNÇÕES DE TEXTO são: LOWER, UPPER, INITCAP, CONCAT, SUBSTR, LENGTH, INSTR, LPAD, RPAD e REPLACE.

4. As FUNÇÕES DE DATA são: MONTHS_BETWEEN, ADD_MONTHS, NEXT_DAY, LAST_DAY e EXTRACT.

5. As FUNÇÕES DE CONVERSÃO são: TO_CHAR, TO_NUMBER e TO_DATE.
Já as FUNÇÕES DE VALORES NULOS são: NVL, NVL2, COALESCE e NULLIF.

EXERCÍCIOS



1

Questão 1

A equipe de Recursos Humanos da empresa precisa apresentar um relatório sobre a faixa salarial dos diferentes cargos existentes na organização. Esse relatório será utilizado para auxiliar na revisão das políticas salariais e na definição de novas contratações.

Para isso, devemos elaborar uma consulta que exiba o ID do cargo, o nome do cargo, o piso salarial e o teto salarial formatados corretamente na moeda local, utilizando a função TO_CHAR para garantir a padronização dos valores.

(Utilize a tabela jobs)

2

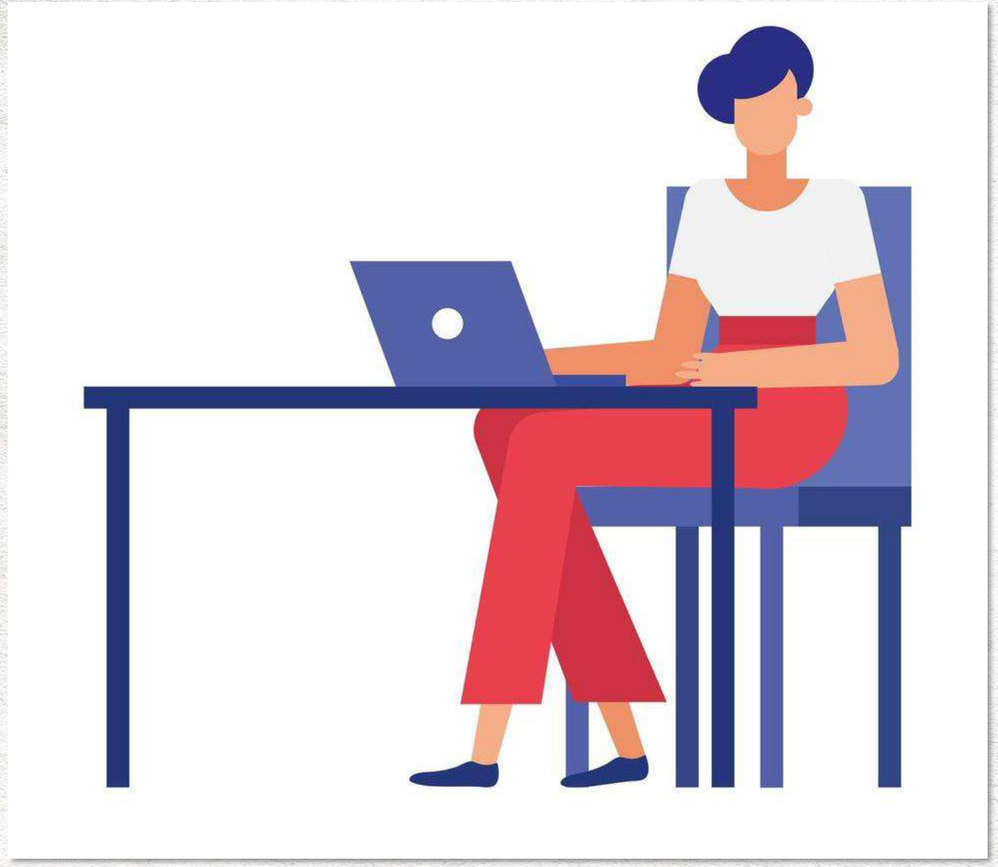
Questão 2

A diretoria da empresa solicitou um levantamento sobre os departamentos para identificar quais possuem um gerente atribuído e quais ainda precisam de um responsável designado. Para facilitar a análise, caso um departamento não tenha um gerente cadastrado, o sistema deve exibir o valor 0 no lugar do identificador do gerente.

Devemos criar uma consulta que liste o ID do departamento, o nome do departamento e o ID do gerente, substituindo valores nulos por 0 para indicar a ausência de um gerente.

(Utilize a tabela departments)

EXERCÍCIOS



3

Questão 3

O time de Logística da empresa precisa gerar um relatório detalhado com os endereços de todas as unidades da organização ao redor do mundo. No entanto, algumas localidades não possuem um estado ou província cadastrados. Para evitar campos vazios no relatório, foi solicitado que, nesses casos, a cidade seja exibida no lugar da província.

Para atender a essa demanda, devemos criar uma consulta que liste o ID da localização, o endereço, a cidade, a província (ou, caso ausente, a cidade como alternativa), e o país correspondente.

(Utilize a tabela locations)

4

Questão 4

O setor de Recursos Humanos está revisando os salários dos funcionários para identificar quais recebem exatamente R$2.500,00. No entanto, para evitar que esses valores sejam considerados em determinados cálculos, foi solicitado que, no relatório, os salários iguais a R$2.500,00 sejam substituídos por NULL.

Para isso, devemos criar uma consulta que exiba o ID do funcionário, seu salário e uma coluna onde os salários de R$2.500,00 sejam transformados em NULL, mantendo os demais valores inalterados.

(Utilize a tabela employees)

EXERCÍCIOS

5

Questão 5

A equipe de Comunicação Interna está organizando uma campanha para celebrar os aniversários de contratação dos colaboradores. Para criar materiais personalizados, eles precisam de um relatório que exiba a data de admissão de cada funcionário de forma clara e amigável.

Devemos elaborar uma consulta que apresente o primeiro nome dos funcionários e a data de contratação formatada no seguinte padrão: "Dia da semana, dia de mês de ano" (exemplo: "segunda-feira, 15 de março de 2023").

(Utilize a tabela employees)

6

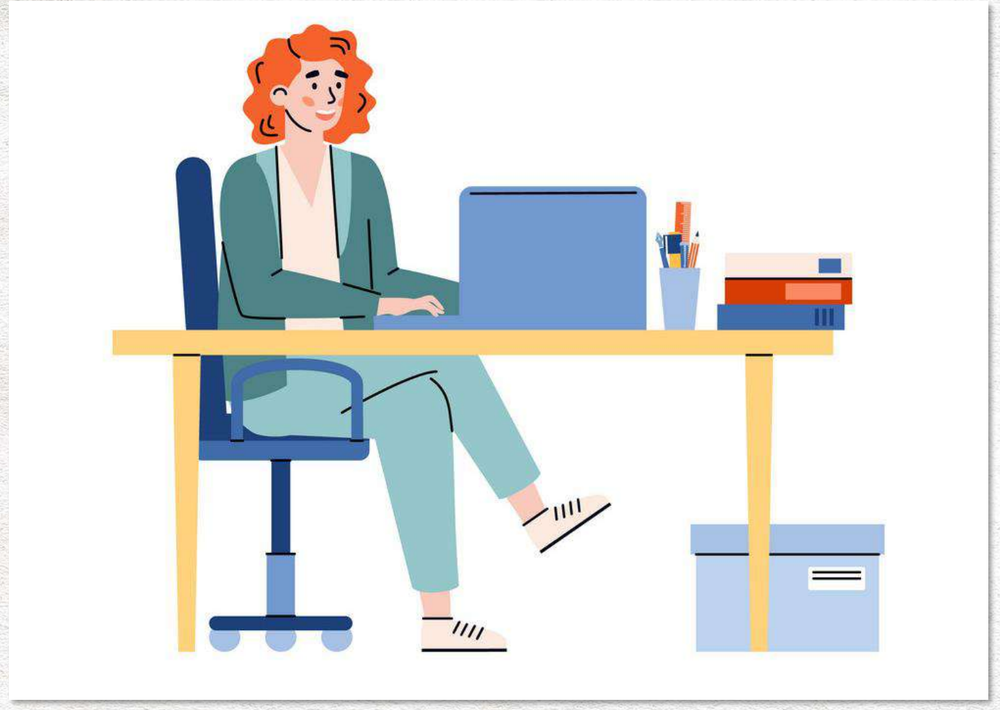
Questão 6

O sistema antigo armazenava as datas de contratação dos funcionários como texto, o que poderia causar inconsistências na migração para o novo banco de dados, onde essas informações devem estar no formato DATE. Para garantir que os dados sejam corretamente convertidos e processados no sistema atual, a equipe de TI precisa validar a conversão das datas migradas.

Devemos criar uma consulta que exiba o primeiro nome dos funcionários, a data de contratação registrada no novo sistema e a conversão da mesma para o formato DATE, assegurando que a migração ocorreu corretamente.

(Utilize a tabela employees)

EXERCÍCIOS



7

Questão 7

O Departamento de Vendas precisa gerar um relatório para verificar quais funcionários recebem comissão. No entanto, a informação sobre a comissão pode estar ausente para alguns. Foi solicitado que, no relatório, os funcionários com comissão recebam a indicação "Possui comissão" e os sem comissão, "Sem comissão".

Devemos criar uma consulta que exiba o nome do funcionário, o valor da comissão e uma coluna adicional com o status de comissão, conforme solicitado.

(Utilize a tabela employees)

8

Questão 8

O sistema de integração da empresa com um aplicativo de gestão de contatos exige que todos os números de telefone sejam enviados sem formatação, contendo apenas os números. Foi solicitado que, no relatório, os números de telefone dos funcionários sejam apresentados no formato simples, sem qualquer caractere especial.

Devemos criar uma consulta que exiba o número de telefone original e uma coluna adicional com o número de telefone contendo apenas os números, conforme solicitado.

(Utilize a tabela employees)

GABARITO EXPLICATIVO

1

Questão 1

A equipe de Recursos Humanos da empresa precisa apresentar um relatório sobre a faixa salarial dos diferentes cargos existentes na organização. Esse relatório será utilizado para auxiliar na revisão das políticas salariais e na definição de novas contratações.

Para isso, devemos elaborar uma consulta que exiba o ID do cargo, o nome do cargo, o piso salarial e o teto salarial formatados corretamente na moeda local, utilizando a função TO_CHAR para garantir a padronização dos valores.

(Utilize a tabela jobs)

Explicação e Resultado

Para apresentar a faixa salarial dos cargos com formatação monetária, seguimos estes passos:

Selecionamos os dados relevantes:

- ✓ **job_id** (como id_cargo): identifica o cargo.
- ✓ **job_title** (como cargo): exibe o nome do cargo.

Formatamos os valores salariais:

- ✓ **TO_CHAR(min_salary, 'L999G999D99')**: exibe o piso salarial com separadores de milhar (G), decimal (D) e símbolo de moeda (L).
- ✓ **TO_CHAR(max_salary, 'L999G999D99')**: exibe o teto salarial no mesmo formato:

```
SELECT
    job_id id_cargo,
    job_title cargo,
    TO_CHAR(min_salary, 'L999G999D99') AS piso_salarial,
    TO_CHAR(max_salary, 'L999G999D99') AS teto_salarial
FROM jobs;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

A equipe de Recursos Humanos da empresa precisa apresentar um relatório sobre a faixa salarial dos diferentes cargos existentes na organização. Esse relatório será utilizado para auxiliar na revisão das políticas salariais e na definição de novas contratações.

Para isso, devemos elaborar uma consulta que exiba o ID do cargo, o nome do cargo, o piso salarial e o teto salarial formatados corretamente na moeda local, utilizando a função TO_CHAR para garantir a padronização dos valores.

(Utilize a tabela jobs)

Explicação e Resultado

Dessa forma, exibimos o seguinte resultado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,024 segundos

	ID_CARGO	CARGO	PISO_SALARIAL	TETO_SALARIAL
1	AD_PRES	President	R$20.000,00	R$40.000,00
2	AD_VP	Administration Vice President	R$15.000,00	R$30.000,00
3	AD_ASST	Administration Assistant	R$3.000,00	R$6.000,00
4	FI_MGR	Finance Manager	R$8.200,00	R$16.000,00
5	FI_ACCOUNT	Accountant	R$4.200,00	R$9.000,00
6	AC_MGR	Accounting Manager	R$8.200,00	R$16.000,00
7	AC_ACCOUNT	Public Accountant	R$4.200,00	R$9.000,00
8	SA_MAN	Sales Manager	R$10.000,00	R$20.000,00
9	SA_REP	Sales Representative	R$6.000,00	R$12.000,00
10	PU_MAN	Purchasing Manager	R$8.000,00	R$15.000,00
11	PU_CLERK	Purchasing Clerk	R$2.500,00	R$5.500,00
12	ST_MAN	Stock Manager	R$5.500,00	R$8.500,00
13	ST_CLERK	Stock Clerk	R$2.000,00	R$5.000,00
14	SH_CLERK	Shipping Clerk	R$2.500,00	R$5.500,00
15	IT_PROG	Programmer	R$4.000,00	R$10.000,00
16	MK_MAN	Marketing Manager	R$9.000,00	R$15.000,00
17	MK_REP	Marketing Representative	R$4.000,00	R$9.000,00
18	HR_REP	Human Resources Representative	R$4.000,00	R$9.000,00
19	PR_REP	Public Relations Representative	R$4.500,00	R$10.500,00

GABARITO EXPLICATIVO

2

Questão 2

A diretoria da empresa solicitou um levantamento sobre os departamentos para identificar quais possuem um gerente atribuído e quais ainda precisam de um responsável designado. Para facilitar a análise, caso um departamento não tenha um gerente cadastrado, o sistema deve exibir o valor 0 no lugar do identificador do gerente.

Devemos criar uma consulta que liste o ID do departamento, o nome do departamento e o ID do gerente, substituindo valores nulos por 0 para indicar a ausência de um gerente.

(Utilize a tabela departments)

Explicação e Resultado

Para listar os departamentos e indicar se possuem um gerente atribuído, seguimos estes passos:

Selecionamos os dados relevantes:

- ✓ **department_id** (como id_departamento): identifica o departamento.
- ✓ **department_name** (como departamento): exibe o nome do departamento.

Substituímos valores nulos no ID do gerente:

- ✓ **NVL(manager_id, 0) AS id_gerente**: caso o departamento não tenha gerente, exibimos 0 no lugar de NULL:

```
SELECT
    department_id id_departamento,
    department_name departamento,
    NVL(manager_id, 0) AS id_gerente
FROM departments;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A diretoria da empresa solicitou um levantamento sobre os departamentos para identificar quais possuem um gerente atribuído e quais ainda precisam de um responsável designado. Para facilitar a análise, caso um departamento não tenha um gerente cadastrado, o sistema deve exibir o valor 0 no lugar do identificador do gerente.

Devemos criar uma consulta que liste o ID do departamento, o nome do departamento e o ID do gerente, substituindo valores nulos por 0 para indicar a ausência de um gerente.

(Utilize a tabela departments)

Explicação e Resultado

Dessa forma, a diretoria pode facilmente identificar quais departamentos precisam de um gerente:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 27 em 0,043 segundos

	ID_DEPARTAMENTO	DEPARTAMENTO	ID_GERENTE
1	10	Administration	200
2	20	Marketing	201
3	30	Purchasing	114
4	40	Human Resources	203
5	50	Shipping	121
6	60	IT	103
7	70	Public Relations	204
8	80	Sales	145
9	90	Executive	100
10	100	Finance	108
11	110	Accounting	205
12	120	Treasury	0
13	130	Corporate Tax	0
14	140	Control And Credit	0
15	150	Shareholder Services	0
16	160	Benefits	0
17	170	Manufacturing	0
18	180	Construction	0
19	190	Contracting	0
20	200	Operations	0
21	210	IT Support	0
22	220	NOC	0
23	230	IT Helpdesk	0
24	240	Government Sales	0
25	250	Retail Sales	0
26	260	Recruiting	0
27	270	Payroll	0

GABARITO EXPLICATIVO

3

Questão 3

O time de Logística da empresa precisa gerar um relatório detalhado com os endereços de todas as unidades da organização ao redor do mundo. No entanto, algumas localidades não possuem um estado ou província cadastrados. Para evitar campos vazios no relatório, foi solicitado que, nesses casos, a cidade seja exibida no lugar da província.

Para atender a essa demanda, devemos criar uma consulta que liste o ID da localização, o endereço, a cidade, a província (ou, caso ausente, a cidade como alternativa), e o país correspondente.

(Utilize a tabela locations)

Explicação e Resultado

Para exibir corretamente os endereços das unidades da empresa, seguimos estes passos:

Selecionamos os dados principais:

- ✓ **location_id AS id_local**: identifica a unidade.
- ✓ **street_address AS endereco**: mostra o endereço.
- ✓ **city AS cidade**: exibe a cidade.
- ✓ **state_province AS estado_provincia**: mostra a província/estado, quando disponível.
- ✓ **country_id AS pais**: identifica o país.

Substituímos valores nulos na província:

- ✓ **COALESCE(state_province, city) AS provincia_ou_cidade**: se state_province for NULL, exibimos a city no lugar:

```
SELECT
    location_id AS id_local,
    street_address AS endereco,
    city AS cidade,
    state_province AS estado_provincia,
    COALESCE(state_province, city) AS provincia_ou_cidade,
    country_id AS pais
FROM locations;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O time de Logística da empresa precisa gerar um relatório detalhado com os endereços de todas as unidades da organização ao redor do mundo. No entanto, algumas localidades não possuem um estado ou província cadastrados. Para evitar campos vazios no relatório, foi solicitado que, nesses casos, a cidade seja exibida no lugar da província.

Para atender a essa demanda, devemos criar uma consulta que liste o ID da localização, o endereço, a cidade, a província (ou, caso ausente, a cidade como alternativa), e o país correspondente.

(Utilize a tabela locations)

Explicação e Resultado

Dessa forma, garantimos que todas as unidades tenham uma informação preenchida na coluna provincia_ou_cidade, o que facilita a análise logística:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 23 em 0,033 segundos

	ID_LOCAL	ENDERECO	CIDADE	ESTADO_PROVINCIA	PROVINCIA_OU_CIDADE	PAIS
1	1000	1297 Via Cola di Rie	Roma	(null)	Roma	IT
2	1100	93091 Calle della Testa	Venice	(null)	Venice	IT
3	1200	2017 Shinjuku-ku	Tokyo	Tokyo Prefecture	Tokyo Prefecture	JP
4	1300	9450 Kamiya-cho	Hiroshima	(null)	Hiroshima	JP
5	1400	2014 Jabberwocky Rd	Southlake	Texas	Texas	US
6	1500	2011 Interiors Blvd	South San Francisco	California	California	US
7	1600	2007 Zagora St	South Brunswick	New Jersey	New Jersey	US
8	1700	2004 Charade Rd	Seattle	Washington	Washington	US
9	1800	147 Spadina Ave	Toronto	Ontario	Ontario	CA
10	1900	6092 Boxwood St	Whitehorse	Yukon	Yukon	CA
11	2000	40-5-12 Laogianggen	Beijing	(null)	Beijing	CN
12	2100	1298 Vileparle (E)	Bombay	Maharashtra	Maharashtra	IN
13	2200	12-98 Victoria Street	Sydney	New South Wales	New South Wales	AU
14	2300	198 Clementi North	Singapore	(null)	Singapore	SG
15	2400	8204 Arthur St	London	(null)	London	UK
16	2500	Magdalen Centre, The Oxford Science Park	Oxford	Oxford	Oxford	UK
17	2600	9702 Chester Road	Stretford	Manchester	Manchester	UK
18	2700	Schwanthalerstr. 7031	Munich	Bavaria	Bavaria	DE
19	2800	Rua Frei Caneca 1360	Sao Paulo	Sao Paulo	Sao Paulo	BR
20	2900	20 Rue des Corps-Saints	Geneva	Geneve	Geneve	CH
21	3000	Murtenstrasse 921	Bern	BE	BE	CH
22	3100	Pieter Breughelstraat 837	Utrecht	Utrecht	Utrecht	NL
23	3200	Mariano Escobedo 9991	Mexico City	Distrito Federal,	Distrito Federal,	MX

GABARITO EXPLICATIVO

4

Questão 4

O setor de Recursos Humanos está revisando os salários dos funcionários para identificar quais recebem exatamente R$2.500,00. No entanto, para evitar que esses valores sejam considerados em determinados cálculos, foi solicitado que, no relatório, os salários iguais a R$2.500,00 sejam substituídos por NULL.

Para isso, devemos criar uma consulta que exiba o ID do funcionário, seu salário e uma coluna onde os salários de R$2.500,00 sejam transformados em NULL, mantendo os demais valores inalterados.

(Utilize a tabela employees)

Explicação e Resultado

Para exibir corretamente os salários dos funcionários, seguimos estes passos:

Selecionamos os dados principais:

- ✓ **employee_id AS id_funcionario**: identifica o funcionário.
- ✓ **salary AS salario**: exibe o salário original.

Substituímos o salário de R$2.500,00 por NULL:

- ✓ **NULLIF(salary, 2500) AS salario_2500_null**: se o salário for exatamente 2500, retorna NULL; caso contrário, mantém o valor original:

```
SELECT
    employee_id id_funcionario,
    salary salario,
    NULLIF(salary, 2500) AS salario_2500_null
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

O setor de Recursos Humanos está revisando os salários dos funcionários para identificar quais recebem exatamente R$2.500,00. No entanto, para evitar que esses valores sejam considerados em determinados cálculos, foi solicitado que, no relatório, os salários iguais a R$2.500,00 sejam substituídos por NULL.

Para isso, devemos criar uma consulta que exiba o ID do funcionário, seu salário e uma coluna onde os salários de R$2.500,00 sejam transformados em NULL, mantendo os demais valores inalterados.

(Utilize a tabela employees)

Explicação e Resultado

Com isso, o relatório retorna os salários dos 107 funcionários, ocultando os de R$2.500,00, sem modificar os demais valores:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,038 se

	ID_FUNCIONARIO	SALARIO	SALARIO_2500_NULL
19	118	2600	2600
20	119	2500	(null)
21	120	8000	8000
22	121	8200	8200
23	122	7900	7900
24	123	6500	6500
25	124	5800	5800
26	125	3200	3200
27	126	2700	2700
28	127	2400	2400
29	128	2200	2200
30	129	3300	3300
31	130	2800	2800
32	131	2500	(null)
33	132	2100	2100
34	133	3300	3300
35	134	2900	2900
36	135	2400	2400
37	136	2200	2200
38	137	3600	3600
39	138	3200	3200
40	139	2700	2700
41	140	2500	(null)
42	141	3500	3500
43	142	3100	3100

GABARITO EXPLICATIVO

5

Questão 5

A equipe de Comunicação Interna está organizando uma campanha para celebrar os aniversários de contratação dos colaboradores. Para criar materiais personalizados, eles precisam de um relatório que exiba a data de admissão de cada funcionário de forma clara e amigável.

Devemos elaborar uma consulta que apresente o primeiro nome dos funcionários e a data de contratação formatada no seguinte padrão: "Dia da semana, dia de mês de ano" (exemplo: "segunda-feira, 15 de março de 2023").

(Utilize a tabela employees)

Explicação e Resultado

Para exibir a data de contratação em um formato mais amigável, seguimos estes passos:

Selecionamos o nome do funcionário:

- ✓ **first_name AS nome_funcionario**.

Formatamos a data de contratação:

- ✓ **TO_CHAR(hire_date, 'DAY, DD "de" MONTH "de" YYYY') AS data_contratacao**:
  - **DAY**: Nome do dia da semana.
  - **DD "de"**: Dia do mês seguido de "de".
  - **MONTH "de"**: Nome do mês seguido de "de".
  - **YYYY**: Ano com quatro dígitos.

```
SELECT
    first_name nome_funcionario,
    TO_CHAR(hire_date, 'DAY, DD "de" MONTH "de" YYYY') AS data_contratacao
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A equipe de Comunicação Interna está organizando uma campanha para celebrar os aniversários de contratação dos colaboradores. Para criar materiais personalizados, eles precisam de um relatório que exiba a data de admissão de cada funcionário de forma clara e amigável.

Devemos elaborar uma consulta que apresente o primeiro nome dos funcionários e a data de contratação formatada no seguinte padrão: "Dia da semana, dia de mês de ano" (exemplo: "segunda-feira, 15 de março de 2023").

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, exibimos as datas de contratação dos 107 funcionários com o seguinte formato:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,019 segundos

	NOME_FUNCIONARIO	DATA_CONTRATACAO
1	Steven	QUARTA-FEIRA , 17 de JUNHO de 1987
2	Neena	QUINTA-FEIRA , 21 de SETEMBRO de 1989
3	Lex	QUARTA-FEIRA , 13 de JANEIRO de 1993
4	Alexander	QUARTA-FEIRA , 03 de JANEIRO de 1990
5	Bruce	TERÇA-FEIRA , 21 de MAIO de 1991
6	David	QUARTA-FEIRA , 25 de JUNHO de 1997
7	Valli	QUINTA-FEIRA , 05 de FEVEREIRO de 1998
8	Diana	DOMINGO , 07 de FEVEREIRO de 1999
9	Nancy	QUARTA-FEIRA , 17 de AGOSTO de 1994
10	Daniel	TERÇA-FEIRA , 16 de AGOSTO de 1994
11	John	DOMINGO , 28 de SETEMBRO de 1997
12	Ismael	TERÇA-FEIRA , 30 de SETEMBRO de 1997
13	Jose Manuel	SÁBADO , 07 de MARÇO de 1998
14	Luis	TERÇA-FEIRA , 07 de DEZEMBRO de 1999
15	Den	QUARTA-FEIRA , 07 de DEZEMBRO de 1994

GABARITO EXPLICATIVO

6

Questão 6

O sistema antigo armazenava as datas de contratação dos funcionários como texto, o que poderia causar inconsistências na migração para o novo banco de dados, onde essas informações devem estar no formato DATE. Para garantir que os dados sejam corretamente convertidos e processados no sistema atual, a equipe de TI precisa validar a conversão das datas migradas.

Devemos criar uma consulta que exiba o primeiro nome dos funcionários, a data de contratação registrada no novo sistema e a conversão da mesma para o formato DATE, assegurando que a migração ocorreu corretamente.

(Utilize a tabela employees)

Explicação e Resultado

Aqui está a explicação da consulta e os pontos importantes para garantir a correta conversão das datas:

Selecionamos o primeiro nome do funcionário: **first_name AS nome_funcionario**.

Exibimos a data de contratação armazenada no sistema: **hire_date AS data_contratacao** (considerando que ainda pode estar no formato de texto).

Convertendo para o formato DATE:

- ✓ **TO_DATE(hire_date, 'DD-MM-YY') AS data_contratacao_convertida**: TO_DATE() converte uma string em formato de data. O formato 'DD-MM-YY' deve corresponder à estrutura do dado armazenado no banco antigo (caso a data esteja no formato YYYY-MM-DD ou contenha horários, pode ser necessário ajustar o formato na função TO_DATE() para garantir a conversão correta):

```
SELECT
    first_name nome_funcionario,
    hire_date data_contratacao,
    TO_DATE(hire_date, 'DD-MM-YY') AS data_contratacao_convertida
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

6

Questão 6

O sistema antigo armazenava as datas de contratação dos funcionários como texto, o que poderia causar inconsistências na migração para o novo banco de dados, onde essas informações devem estar no formato DATE. Para garantir que os dados sejam corretamente convertidos e processados no sistema atual, a equipe de TI precisa validar a conversão das datas migradas.

Devemos criar uma consulta que exiba o primeiro nome dos funcionários, a data de contratação registrada no novo sistema e a conversão da mesma para o formato DATE, assegurando que a migração ocorreu corretamente.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, exibimos o seguinte resultado, garantindo que, na coluna data_contratacao_convertida, a data de contratação dos 107 funcionários está no formato DATE:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,019 segundos

	NOME_FUNCIONARIO	DATA_CONTRATACAO	DATA_CONTRATACAO_CONVERTIDA
1	Steven	17/06/87	17/06/87
2	Neena	21/09/89	21/09/89
3	Lex	13/01/93	13/01/93
4	Alexander	03/01/90	03/01/90
5	Bruce	21/05/91	21/05/91
6	David	25/06/97	25/06/97
7	Valli	05/02/98	05/02/98
8	Diana	07/02/99	07/02/99
9	Nancy	17/08/94	17/08/94
10	Daniel	16/08/94	16/08/94
11	John	28/09/97	28/09/97
12	Ismael	30/09/97	30/09/97
13	Jose Manuel	07/03/98	07/03/98
14	Luis	07/12/99	07/12/99
15	Den	07/12/94	07/12/94
16	Alexander	18/05/95	18/05/95
17	Shelli	24/12/97	24/12/97
18	Sigal	24/07/97	24/07/97
19	Guy	15/11/98	15/11/98

GABARITO EXPLICATIVO

7

Questão 7

O Departamento de Vendas precisa gerar um relatório para verificar quais funcionários recebem comissão. No entanto, a informação sobre a comissão pode estar ausente para alguns. Foi solicitado que, no relatório, os funcionários com comissão recebam a indicação "Possui comissão" e os sem comissão, "Sem comissão".

Devemos criar uma consulta que exiba o nome do funcionário, o valor da comissão e uma coluna adicional com o status de comissão, conforme solicitado.

(Utilize a tabela employees)

Explicação e Resultado

Para criarmos uma consulta que exiba o nome do funcionário, o valor da comissão e o status de comissão, fazemos o seguinte:

Selecionamos o nome do funcionário: **first_name AS nome_funcionario**.

Exibimos o percentual de comissão: **commission_pct AS percentual_comissao** (pode estar preenchido ou ser NULL).

Definimos um status com base na comissão:

- ✓ **NVL2(commission_pct, 'Possui comissão', 'Sem comissão') AS status_comissao**:
  - NVL2(valor, retorno_se_não_nulo, retorno_se_nulo): se commission_pct não for NULL, retorna "Possui comissão“; caso contrário (se commission_pct for NULL), retorna "Sem comissão":

```
SELECT
    first_name nome_funcionario,
    commission_pct percentual_comissao,
    NVL2(commission_pct, 'Possui comissão', 'Sem comissão') AS status_comissao
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

O Departamento de Vendas precisa gerar um relatório para verificar quais funcionários recebem comissão. No entanto, a informação sobre a comissão pode estar ausente para alguns. Foi solicitado que, no relatório, os funcionários com comissão recebam a indicação "Possui comissão" e os sem comissão, "Sem comissão".

Devemos criar uma consulta que exiba o nome do funcionário, o valor da comissão e uma coluna adicional com o status de comissão, conforme solicitado.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, exibimos uma linha para cada um dos 107 funcionários com as informações solicitadas:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,026 segundos

	NOME_FUNCIONARIO	PERCENTUAL_COMISSAO	STATUS_COMISSAO
37	Hazel	(null)	Sem comissão
38	Renske	(null)	Sem comissão
39	Stephen	(null)	Sem comissão
40	John	(null)	Sem comissão
41	Joshua	(null)	Sem comissão
42	Trenna	(null)	Sem comissão
43	Curtis	(null)	Sem comissão
44	Randall	(null)	Sem comissão
45	Peter	(null)	Sem comissão
46	John	0,4	Possui comissão
47	Karen	0,3	Possui comissão
48	Alberto	0,3	Possui comissão
49	Gerald	0,3	Possui comissão
50	Eleni	0,2	Possui comissão
51	Peter	0,3	Possui comissão
52	David	0,25	Possui comissão
53	Peter	0,25	Possui comissão
54	Christopher	0,2	Possui comissão
55	Nanette	0,2	Possui comissão
56	Oliver	0,15	Possui comissão
57	Janette	0,35	Possui comissão

GABARITO EXPLICATIVO

8

Questão 8

O sistema de integração da empresa com um aplicativo de gestão de contatos exige que todos os números de telefone sejam enviados sem formatação, contendo apenas os números. Foi solicitado que, no relatório, os números de telefone dos funcionários sejam apresentados no formato simples, sem qualquer caractere especial.

Devemos criar uma consulta que exiba o número de telefone original e uma coluna adicional com o número de telefone contendo apenas os números, conforme solicitado.

(Utilize a tabela employees)

Explicação e Resultado

Para retornar os números de telefone dos funcionários sem qualquer caractere especial, fazemos o seguinte:

Selecionamos o número de telefone original:

- ✓ **phone_number AS telefone**: Exibe a coluna phone_number com um alias mais descritivo (telefone).

Convertemos para formato numérico:

- ✓ **TO_NUMBER(phone_number, '99999999999999999') AS telefone_somente_numeros**:
  - A função TO_NUMBER() converte a string phone_number para um valor numérico.
  - O formato '99999999999999999' define que apenas números serão considerados.
  - Utilizamos **18 números 9** no formato porque o maior número de telefone na coluna phone_number contém 18 caracteres (incluindo números e pontos).
  - O parâmetro da função deve conter a quantidade máxima de caracteres possível na coluna, garantindo que todos os números possam ser convertidos corretamente:

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

8

Questão 8

O sistema de integração da empresa com um aplicativo de gestão de contatos exige que todos os números de telefone sejam enviados sem formatação, contendo apenas os números. Foi solicitado que, no relatório, os números de telefone dos funcionários sejam apresentados no formato simples, sem qualquer caractere especial.

Devemos criar uma consulta que exiba o número de telefone original e uma coluna adicional com o número de telefone contendo apenas os números, conforme solicitado.

(Utilize a tabela employees)

Explicação e Resultado

```
SELECT
    phone_number AS telefone,
    TO_NUMBER(phone_number, '99999999999999999') AS telefone_somente_numeros
FROM employees;
```

Dessa forma, exibimos o seguinte resultado:

Resultado da Consulta 1 ×

SQL | Todas as Linhas Extraídas: 107 em 0,013 segundo

	TELEFONE	TELEFONE_SOMENTE_NUMEROS
38	650.121.1234	6501211234
39	650.121.2034	6501212034
40	650.121.2019	6501212019
41	650.121.1834	6501211834
42	650.121.8009	6501218009
43	650.121.2994	6501212994
44	650.121.2874	6501212874
45	650.121.2004	6501212004
46	011.44.1344.429268	11441344429268
47	011.44.1344.467268	11441344467268
48	011.44.1344.429278	11441344429278
49	011.44.1344.619268	11441344619268
50	011.44.1344.429018	11441344429018
51	011.44.1344.129268	11441344129268
52	011.44.1344.345268	11441344345268
53	011.44.1344.478968	11441344478968

MÓDULO 8

CRIANDO AGRUPAMENTOS

CRIANDO AGRUPAMENTOS

CRIANDO AGRUPAMENTOS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 8 FUNÇÕES DE AGREGAÇÃO E AGRUPAMENTOS NO SQL

01	FUNÇÕES DE AGREGAÇÃO: COUNT	324
02	FUNÇÕES DE AGREGAÇÃO: COUNT(*)	327
03	FUNÇÕES DE AGREGAÇÃO: COUNT DISTINCT	328
04	FUNÇÕES DE AGREGAÇÃO: SUM	329
05	FUNÇÕES DE AGREGAÇÃO: AVG	330
06	FUNÇÕES DE AGREGAÇÃO: MAX	331
07	FUNÇÕES DE AGREGAÇÃO: MIN	332
08	FUNÇÕES DE AGREGAÇÃO: CUIDADOS COM VALORES NULOS	333
09	GROUP BY	336
10	GROUP BY: ESTRUTURA	342
11	GROUP BY: APLICAÇÕES	343
12	GROUP BY: FUNÇÕES DE AGRUPAMENTO ANINHADAS	345
13	GROUP BY + WHERE	346
14	GROUP BY + HAVING	347
15	GROUP BY + WHERE VS. GROUP BY + HAVING	348
16	RESUMO DO MÓDULO	349
17	EXERCÍCIOS	350
18	GABARITO EXPLICATIVO	355

Funções de Agregação: COUNT

A função de agregação **COUNT** faz a contagem de valores de uma coluna.

Sua sintaxe é bem simples:

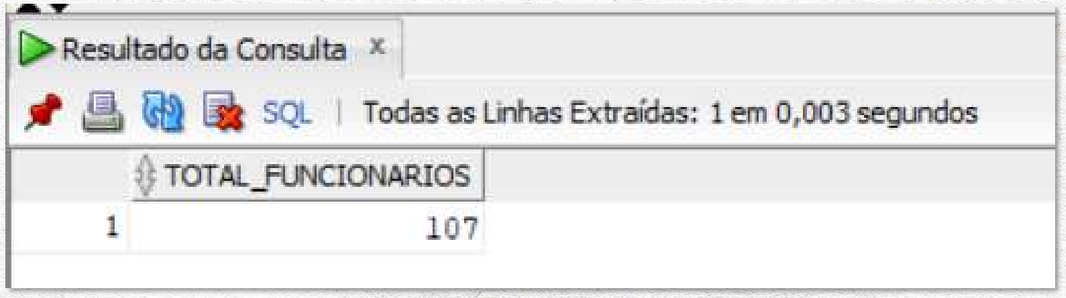
```
SELECT
   COUNT(coluna)
FROM tabela;
```

Repare que, no exemplo ao lado, fizemos um COUNT da coluna employee_id da tabela employees, que retornou o total de 107 funcionários.

Tudo perfeito, então, certo?

Sim, entretanto, tome cuidado...

```
-- a) Descubra o total de funcionários a partir
-- da coluna de EMPLOYEE_ID da tabela EMPLOYEES.

SELECT
    COUNT(employee_id) total_funcionarios
FROM employees;
```



Funções de Agregação: COUNT

Pois...

Veja só este exemplo:

Fizemos o mesmo COUNT na tabela employees, mas, desta vez, informamos a coluna commission_pct.

Agora, foi retornado um total de apenas 35 funcionários.

Por que isso aconteceu?

```
-- c) Descubra o total de funcionários a partir
-- da coluna de COMMISSION_PCT da tabela EMPLOYEES.
-- É possível?

SELECT
    COUNT(commission_pct) total_funcionarios
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,002 segundos

	TOTAL_FUNCIONARIOS
1	35

Funções de Agregação: COUNT

Porque **a função COUNT não considera valores nulos** em sua contagem.

Se você fizer uma consulta às colunas employee_id e commission_pct, verá que:

- A coluna employee_id não tem valores nulos.
- Já a coluna commission_pct tem valores nulos.

Assim, como o COUNT não considera valores nulos, as consultas retornaram valores diferentes.

```
SELECT
    employee_id
FROM employees;
```

Resultado da Consulta

	EMPLOYEE_ID
1	100
2	101
3	102
4	103
5	104
6	105
7	106
8	107
9	108
10	109
11	110
12	111
13	112
14	113
15	114
16	115
17	116
18	117
19	118
20	119

```
SELECT
    commission_pct
FROM employees;
```

Resultado da Consulta

	COMMISSION_PCT
36	(null)
37	(null)
38	(null)
39	(null)
40	(null)
41	(null)
42	(null)
43	(null)
44	(null)
45	(null)
46	0,4
47	0,3
48	0,3
49	0,3
50	0,2
51	0,3
52	0,25
53	0,25
54	0,2
55	0,2

Funções de Agregação: COUNT(*)

Para resolver essa questão dos valores nulos que são desconsiderados pela função COUNT, em vez de especificar qual coluna queremos contar, podemos simplesmente informar um * em seu lugar, conforme exemplo ao lado.

Assim, não precisamos nos preocupar se a coluna informada possui valores nulos, uma vez que a função **COUNT(*)** faz a contagem de todas as linhas de uma tabela, contenham as colunas valores nulos ou não.

```
-- a) Descubra o total de funcionários na tabela EMPLOYEES
-- independente da coluna a ser usada.

SELECT
    COUNT(*) total_funcionarios
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,004 segundos

	TOTAL_FUNCIONARIOS
1	107

Funções de Agregação: COUNT DISTINCT

A função de agregação **COUNT DISTINCT** conta os valores distintos de uma coluna.

No exemplo ao lado, embora a gente já saiba que a tabela employees tem 107 registros (linhas), usando o COUNT DISTINCT obtivemos uma contagem de 19.
Isso aconteceu porque esta função retorna uma contagem apenas dos valores que não são repetidos na coluna informada (no caso, job_id).

Esses 19 registros referem-se ao 19 JOB_IDs diferentes que existem na tabela:

```sql
-- a) Descubra quais são os valores distintos de JOB_ID na tabela EMPLOYEES.
-- Qual é o total de valores distintos?

SELECT
    COUNT(DISTINCT job_id)
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,017 segundos

	COUNT(DISTINCTJOB_ID)
1	19

Resultado da Consul

SQL

	JOB_ID				
1	AC_ACCOUNT	6	FI_ACCOUNT	13	PU_CLERK
2	AC_MGR	7	FI_MGR	14	PU_MAN
3	AD_ASST	8	HR_REP	15	SA_MAN
4	AD_PRES	9	IT_PROG	16	SA_REP
5	AD_VP	10	MK_MAN	17	SH_CLERK
		11	MK_REP	18	ST_CLERK
		12	PR_REP	19	ST_MAN

Funções de Agregação: SUM

A função de agregação **SUM** faz a soma dos valores de uma coluna.

Sintaxe:

```
SELECT
   SUM(coluna)
FROM tabela;
```

No exemplo ao lado, fizemos a soma da coluna salary da tabela employees, que retornou o valor total pago aos funcionários.

```
-- a) Qual é a soma total de salário dos funcionários?

SELECT
    SUM(salary) total_salario
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,003 segundos

	TOTAL_SALARIO
1	691400

Funções de Agregação: AVG

A função de agregação **AVG** faz a média dos valores de uma coluna.

Sintaxe:

```
SELECT
   AVG(coluna)
FROM tabela;
```

No exemplo ao lado (figura 1), fizemos a média da coluna salary da tabela employees.

Dica: Caso a gente queira arredondar as casas decimais, é só utilizar a função ROUND, informando no segundo argumento o número de casas decimais que queremos (figura 2):

```
SELECT
    ROUND(AVG(salary), 2) media_salario
FROM employees;
```

2

Resultado da Consulta

SQL | Todas as L

	TOTAL_SALARIO
1	6461,68

```
-- a) Qual é a média salarial dos funcionários?

SELECT
    AVG(salary) media_salario
FROM employees;
```

1

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,014 segundos

	MEDIA_SALARIO
1	6461,682242990654205607476635514018691589

Funções de Agregação: MAX

A função de agregação **MAX** retorna o maior valor de uma coluna.

Sintaxe:

```
SELECT
   MAX(coluna)
FROM tabela;
```

No exemplo ao lado, retornamos o maior valor (salário) da coluna salary da tabela employees.

```
-- a) Qual é a maior de todos os salários?

SELECT
    MAX(salary) maior_salario
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,004 segundos

	MAIOR_SALARIO
1	24000

Funções de Agregação: MIN

A função de agregação **MIN** retorna o menor valor de uma coluna.

Sintaxe:

```
SELECT
   MIN(coluna)
FROM tabela;
```

No exemplo ao lado, retornamos o menor valor (salário) da coluna salary da tabela employees.

```
-- b) Qual é a menor de todos os salários?

SELECT
    MIN(salary) menor_salario
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,004 segundos

	MENOR_SALARIO
1	2100

Funções de Agregação: Cuidados com valores nulos

Já vimos que a função COUNT retorna valores diferentes caso a coluna utilizada na consulta contenha valores nulos, pois ela não considera este tipo de valores. Lembra-se disso?

Pois bem. Na realidade, não somente a função COUNT ignora valores nulos, mas todas as funções de agregação (COUNT, SUM, AVG, MIN, MAX).

Em relação às funções SUM, MIN e MAX, isso não se torna um problema, já que a soma (SUM) continuará sendo a mesma (pois seria o mesmo que somar “nada” ao resultado); assim como o MIN ainda assim será o menor valor (não nulo) existente na coluna. E o mesmo também vale para a função MAX, que continuará retornando o maior valor da coluna.

Porém, assim como com o COUNT, com a função AVG também devemos tomar cuidado.
Veremos o porquê na próxima página.

Funções de Agregação: Cuidados com valores nulos

Imagine que queiramos saber a média de comissão dos funcionários, e executamos a seguinte consulta:

```
SELECT
    AVG(commission_pct) media_comissao
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,001 segundos

	MEDIA_COMISSAO
1	0,222857142857142857142

Funções de Agregação: Cuidados com valores nulos

Dessa forma, a média está sendo calculada apenas sobre esses 35 registros, não sobre os 107.

Para resolver esse problema, precisamos substituir os valores nulos por 0.

Assim, todas as linhas serão consideradas pela função AVG, que calculará a média correta sobre os 107 registros.

E para isso, utilizamos uma função que já vimos lá no módulo 7, sobre funções Single Row: a NVL, que permite substituir o valor NULL por um outro valor, especificado no segundo argumento:

```
SELECT
    AVG(NVL(commission_pct, 0)) media_comissao
FROM employees;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,014 segundos

	MEDIA_COMISSAO
1	0,072897196261682242990654205607476635514

Na consulta acima, se a coluna commission_pct estiver preenchida, considera seu valor; caso contrário, substitui o valor nulo por 0 e calcula a média dos 107 registros da coluna.

GROUP BY

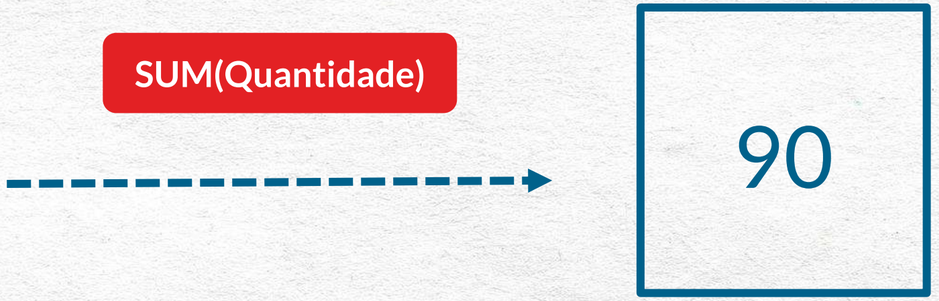
Até agora, vimos como usar funções de agregação para calcular totais.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Até agora, vimos como usar funções de agregação para calcular totais.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30



GROUP BY

Podemos melhorar as nossas análises usando o **GROUP BY**, transformando o total em uma análise mais segmentada.

90

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

GROUP BY

Podemos melhorar as nossas análises usando o **GROUP BY**, transformando o total em uma análise mais segmentada.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

GROUP BY

Podemos melhorar as nossas análises usando o **GROUP BY**, transformando o total em uma análise mais segmentada.

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

GROUP BY

Podemos melhorar as nossas análises usando o **GROUP BY**, transformando o total em uma análise mais segmentada.

90

Data Venda	Produto	Quantidade
2019-01-01	PRODUTO A	10
2019-01-01	PRODUTO B	20
2019-01-01	PRODUTO A	5
2019-01-01	PRODUTO A	15
2019-01-01	PRODUTO B	10
2019-01-01	PRODUTO A	30

GROUP BY

Produto	Quantidade Total
PRODUTO A	60
PRODUTO B	30

```
SELECT
    Produto,
    SUM(Quantidade) “Quantidade Total”
FROM Tabela
GROUP BY Produto;
```

GROUP BY: Estrutura

O GROUP BY agrupa os dados a partir de uma ou mais colunas da tabela.

Sua sintaxe é a seguinte:

```
SELECT
   coluna1,
   funcao_agregacao(coluna2)
FROM tabela
GROUP BY coluna1;
```

Na hora de criar agrupamentos, podemos utilizar uma série de comandos, na seguinte **ordem**:

```
SELECT
        coluna1,
        coluna2,
        funcao_agregacao(coluna3)
FROM tabela
[WHERE condicao]          → Condição para selecionar as linhas desejadas da tabela
GROUP BY coluna1, coluna 2 → Comando para criar o agrupamento
[HAVING condicao]         → Condição para selecionar os agrupamentos desejados
[ORDER BY coluna];        → Comando para ordenar a consulta final
```

GROUP BY: Aplicações

Vamos aos exemplos!

Vamos supor que a gente queira fazer um agrupamento da quantidade total de funcionários por JOB_ID.

Para isso, vamos ter que chamar a coluna job_id e fazer um COUNT(*) da tabela employees, agrupando essa consulta pela coluna job_id por meio do GROUP BY.

Dessa forma, conseguimos segmentar a quantidade de funcionários para cada função, e não somente a quantidade total de funcionários em toda a empresa, conforme estivemos fazendo até agora:

```
SELECT
    job_id,
    COUNT(*) total_funcionarios
FROM employees
GROUP BY job_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas:

	JOB_ID	TOTAL_FUNCIONARIOS
1	AC_ACCOUNT	1
2	AC_MGR	1
3	AD_ASST	1
4	AD_PRES	1
5	AD_VP	2
6	FI_ACCOUNT	5
7	FI_MGR	1
8	HR_REP	1
9	IT_PROG	5
10	MK_MAN	1
11	MK_REP	1
12	PR_REP	1
13	PU_CLERK	5
14	PU_MAN	1
15	SA_MAN	5
16	SA_REP	30
17	SH_CLERK	20
18	ST_CLERK	20
19	ST_MAN	5

GROUP BY: Aplicações

Agora, imagine que queiramos fazer um agrupamento da soma total de salário por DEPARTMENT_ID e JOB_ID.

Repare que, desta vez, vamos precisar agrupar duas colunas da tabela employees (department_id e job_id), para fazer a soma de salário de acordo com cada função de cada departamento.

Podemos, ainda, utilizar o ORDER BY (que vimos no módulo 5) para ordenar a consulta pelo department_id em ordem crescente:

```
SELECT
    department_id,
    job_id,
    SUM(salary) total_salario
FROM employees
GROUP BY department_id, job_id
ORDER BY department_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 20 em 0,04 se

	DEPARTMENT_ID	JOB_ID	TOTAL_SALARIO
1	10	AD_ASST	4400
2	20	MK_MAN	13000
3	20	MK_REP	6000
4	30	PU_CLERK	13900
5	30	PU_MAN	11000
6	40	HR_REP	6500
7	50	SH_CLERK	64300
8	50	ST_CLERK	55700
9	50	ST_MAN	36400
10	60	IT_PROG	28800
11	70	PR_REP	10000
12	80	SA_MAN	61000
13	80	SA_REP	243500
14	90	AD_PRES	24000
15	90	AD_VP	34000
16	100	FI_ACCOUNT	39600
17	100	FI_MGR	12000
18	110	AC_ACCOUNT	8300
19	110	AC_MGR	12000
20	(null)	SA_REP	7000

GROUP BY: Funções de Agrupamento Aninhadas

Podemos utilizar uma função de agrupamento dentro de outra para fazer nossas consultas.

Veja o exemplo a seguir:

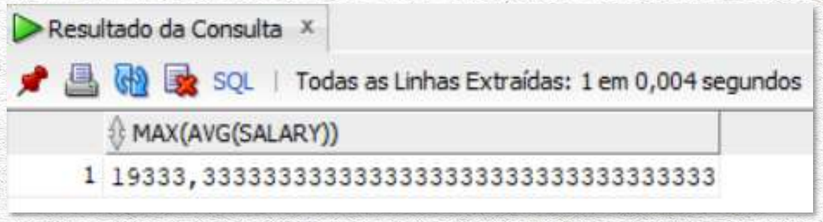
Queremos saber qual é a maior média salarial dentre todos os departamentos.

Para isso, precisaremos calcular a média salarial de todos os departamentos com a função de agrupamento AVG(salary).

Só que, para descobrir qual é a maior de todas essas médias, temos que pegar essa função AVG(salary) e colocá-la dentro de uma função MAX, desta forma: **MAX(AVG(salary))**.

Depois, basta agrupar, por meio do GROUP BY, pela coluna department_id, conforme ao lado, que a maior média salarial entre todos os departamentos será exibida no resultado:

```
SELECT MAX(AVG(salary))
FROM employees
GROUP BY department_id;
```



GROUP BY + WHERE

A combinação **GROUP BY + WHERE** nos permite criar filtros antes de agrupar uma tabela, para depois fazer o agrupamento de dados a partir de uma ou mais colunas dessa tabela.

Vamos a um exemplo:

Faça um agrupamento da quantidade total de funcionários por JOB_ID, mas apenas para os funcionários que foram contratados no ano de 1998.

Neste caso, primeiro a gente vai precisar filtrar a tabela employees para retornar apenas os funcionários que foram contratados em 1998. Para isso, usamos o WHERE.

Assim, podemos fazer uma contagem (COUNT(*)) apenas dos registros retornados (os funcionários contratados em 1998), agrupando-os (GROUP BY) pelo JOB_ID.

Ao lado, veja como fica a nossa consulta e seu resultado:

```
SELECT
    job_id,
    COUNT(*) total_funcionarios
FROM employees
WHERE TO_CHAR(hire_date, 'YYYY')  = '1998'
GROUP BY job_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 6 em 0,004 segundos

	JOB_ID	TOTAL_FUNCIONARIOS
1	IT_PROG	1
2	FI_ACCOUNT	1
3	PU_CLERK	1
4	ST_CLERK	6
5	SA_REP	7
6	SH_CLERK	7

GROUP BY + HAVING

A combinação **GROUP BY + HAVING** nos permite primeiro fazer o agrupamento de dados a partir de uma ou mais colunas de uma tabela, para depois criar filtros a partir desse agrupamento.

Vamos a um exemplo:

Faça um agrupamento da quantidade total de funcionários por JOB_ID. O agrupamento final deve considerar apenas os JOB_IDs com mais de 1 funcionário.

Aqui, primeiro vamos fazer uma contagem (COUNT(*)) do total de funcionários, agrupando-os (GROUP BY) pelo JOB_ID.

Feito isso, utilizamos o HAVING para extrair deste agrupamento criado somente aqueles JOB_IDs cuja contagem retornou um valor maior que 1.

Ao lado, veja como fica a nossa consulta e seu resultado:

```sql
SELECT
    job_id,
    COUNT(*) total_funcionarios
FROM employees
GROUP BY job_id
HAVING COUNT(*) > 1;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 9 em 0,018 segundos

	JOB_ID	TOTAL_FUNCIONARIOS
1	AD_VP	2
2	FI_ACCOUNT	5
3	IT_PROG	5
4	PU_CLERK	5
5	SA_MAN	5
6	SA_REP	30
7	SH_CLERK	20
8	ST_CLERK	20
9	ST_MAN	5

GROUP BY + WHERE vs. GROUP BY + HAVING

Portanto, para saber quando utilizar o **GROUP BY** associado à cláusula **WHERE** ou à cláusula **HAVING**, você deve se perguntar:

Este filtro precisa ser feito na tabela que já existe ou no agrupamento que estou criando?

SE SUA RESPOSTA FOR:

NA TABELA:

UTILIZE O WHERE ANTES DO GROUP BY

NO AGRUPAMENTO:

UTILIZE O HAVING DEPOIS DO GROUP BY

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. Existem diversas FUNÇÕES DE AGREGAÇÃO para criar cálculos no SQL: COUNT, SUM, AVG, MAX e MIN.
2. As FUNÇÕES DE AGREGAÇÃO desconsideram valores nulos no cálculo (tome cuidado com o COUNT e o AVG!).
3. GROUP BY é o comando usado para criar agrupamentos de tabelas.
4. Podemos utilizar o GROUP BY em conjunto com o WHERE para criar filtros ANTES de realizar o agrupamento.
5. Podemos utilizar o GROUP BY em conjunto com o HAVING para criar filtros DEPOIS de realizar o agrupamento.

EXERCÍCIOS

1a

Questão 1-a

A diretoria quer ter uma visão clara dos departamentos que possuem funcionários ativos. Isso ajudará na alocação de recursos, no planejamento estratégico e na reestruturação de setores que possam estar inativos ou precisando de reforço na equipe.

Crie uma consulta que retorne a quantidade de departamentos distintos que possuem ao menos um funcionário alocado.

Dica: para garantir que somente entrarão na contagem os departamentos com funcionários ativos, **utilize a tabela employees**.

1b

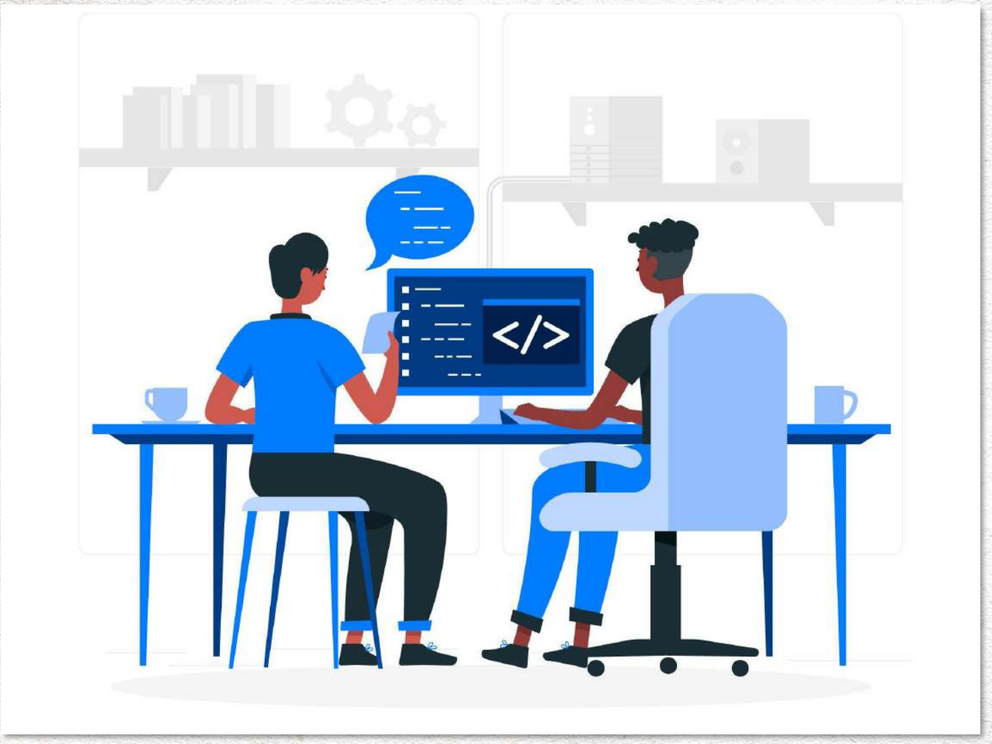
Questão 1-b

Se fizermos a contagem de departamentos distintos na tabela departments, teremos um resultado diferente do apresentado no item A?

O que isto significa?

(Utilize a tabela departments)

EXERCÍCIOS



2

Questão 2

O setor de Recursos Humanos deseja visualizar como os funcionários estão distribuídos entre os departamentos da empresa. Essa análise permitirá identificar quais áreas estão sobrecarregadas ou subutilizadas, auxiliando no planejamento de novas contratações.

Crie uma consulta que retorne o ID do departamento e a contagem de funcionários por departamento. Ordene o resultado pelo total de funcionários em ordem decrescente.

(Utilize a tabela employees)

3

Questão 3

A equipe de planejamento estratégico deseja entender a evolução do quadro de funcionários ao longo do tempo. Com essa informação, será possível identificar tendências de crescimento ou retração da força de trabalho e planejar novas contratações de forma mais eficiente.

Crie uma consulta que retorne o ano e a quantidade de funcionários contratados por ano. Ordene o resultado por ano em ordem crescente.

(Utilize a tabela employees)

EXERCÍCIOS

4

Questão 4

O setor financeiro quer avaliar a diferença entre os menores e os maiores salários dentro de cada departamento. Isso ajudará na criação de políticas de equidade salarial, na identificação de possíveis desigualdades e na estruturação de planos de carreira mais justos.

Crie uma consulta que retorne o ID do departamento, o piso, o teto e a diferença salarial por departamento. Ordene o resultado pela diferença salarial em ordem decrescente.

(Utilize a tabela employees)

5

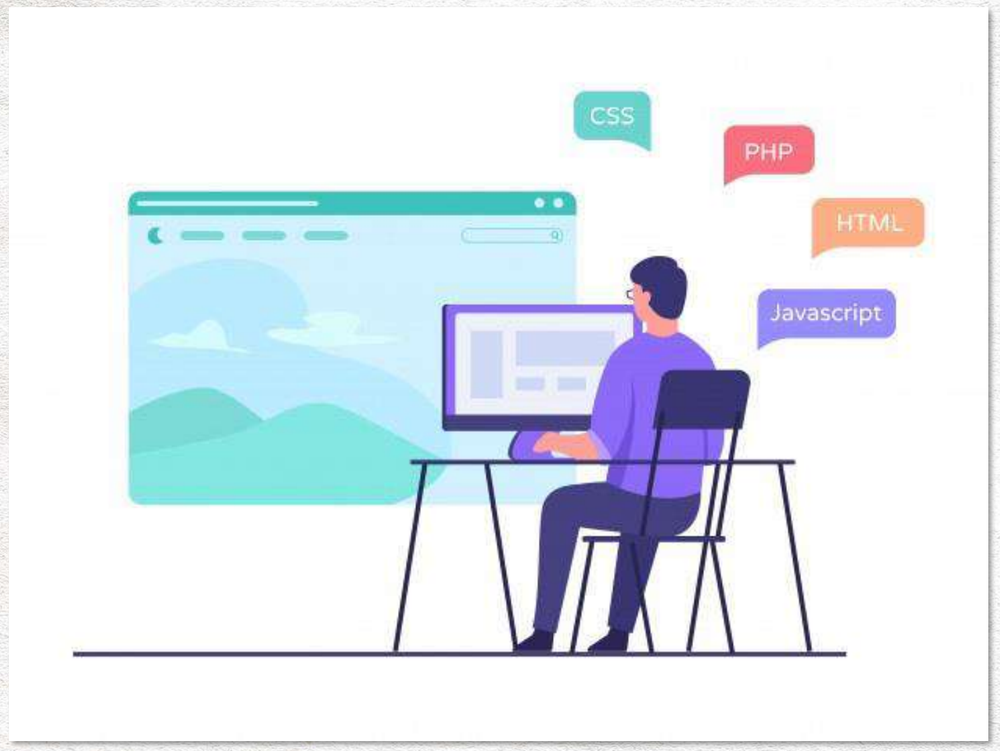
Questão 5

O RH precisa saber quantos funcionários ocupam cargos de gerência na empresa. Essa informação será usada para avaliar a proporção entre gestores e equipes, garantindo uma estrutura organizacional eficiente e equilibrada.

Sabendo-se que os job_id dos gerentes contêm o termo 'MAN' (de manager), crie uma consulta que retorne a quantidade de funcionários que possuem um job_id que contém esse termo.

(Utilize a tabela employees)

EXERCÍCIOS



6

Questão 6

A empresa deseja entender quanto do seu orçamento de folha de pagamento está sendo direcionado para funcionários que recebem comissão. Isso ajudará no planejamento financeiro e na definição de metas de vendas e incentivos.

Crie uma consulta que retorne a soma salarial dos funcionários cuja comissão não seja nula.

(Utilize a tabela employees)

7

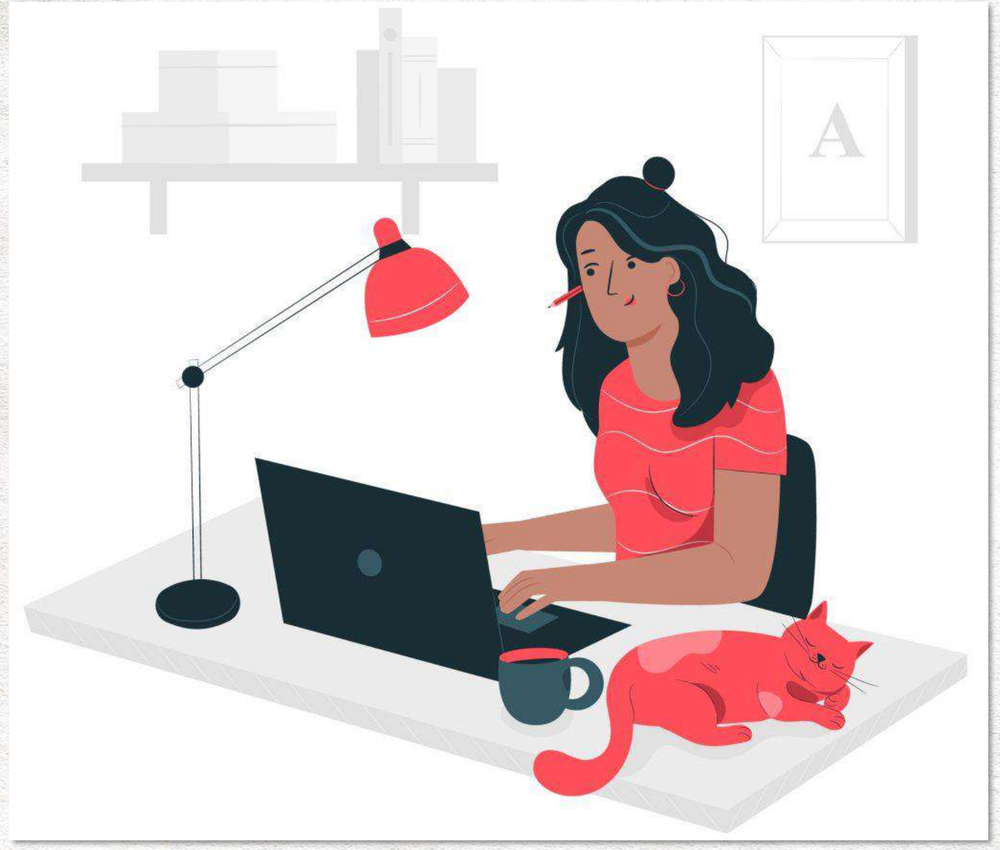
Questão 7

Os gestores dos departamentos 10, 20 e 30 solicitaram um relatório detalhado sobre a distribuição de funcionários por cargo dentro de suas áreas. Essas informações serão usadas para avaliar necessidades de realocação, treinamento e possíveis novas contratações.

Crie uma consulta que retorne o ID do departamento, o ID do cargo e a contagem de funcionários por departamento e cargo, considerando somente os departamentos de ID 10, 20 ou 30.

(Utilize a tabela employees)

EXERCÍCIOS



8

Questão 8

A diretoria quer entender o tamanho dos departamentos que possuem uma média salarial elevada (acima de R$ 8.000). Essa análise ajudará a empresa a verificar se setores estratégicos estão bem estruturados e se os salários estão alinhados com a produtividade e os resultados entregues.

Crie uma consulta que mostre o ID do departamento, a média salarial e a contagem de funcionários por departamento, retornando somente aqueles cuja média salarial ultrapasse R$ 8.000. Ordene o resultado pelo total de funcionários em ordem decrescente.

(Utilize a tabela employees)

9

Questão 9

O RH deseja identificar quais gerentes supervisionam equipes com pelo menos 3 funcionários. Essas informações serão usadas para planejar treinamentos de liderança, definir políticas de promoção e avaliar se alguns gestores estão sobrecarregados.

Crie uma consulta que mostre o ID do gerente e a contagem de funcionários por gerente, retornando somente aqueles cuja quantidade de funcionários supervisionados seja maior que 3. Ordene o resultado pelo total de funcionários gerenciados em ordem decrescente.

(Utilize a tabela employees)

GABARITO EXPLICATIVO

1a

Questão 1-a

A diretoria quer ter uma visão clara dos departamentos que possuem funcionários ativos. Isso ajudará na alocação de recursos, no planejamento estratégico e na reestruturação de setores que possam estar inativos ou precisando de reforço na equipe.

Crie uma consulta que retorne a quantidade de departamentos distintos que possuem ao menos um funcionário alocado.

Dica: para garantir que somente entrarão na contagem os departamentos com funcionários ativos, **utilize a tabela employees**.

Explicação e Resultado

Para retornar a quantidade de departamentos distintos que possuem ao menos um funcionário alocado, fazemos o seguinte:

Identificamos os departamentos distintos que possuem funcionários ativos:

- ✓ A tabela **employees** contém informações sobre os funcionários e os departamentos aos quais pertencem.

Contamos os departamentos distintos:

- ✓ **COUNT(DISTINCT department_id) AS total_departamentos**:
  - DISTINCT department_id garante que contamos apenas departamentos únicos, eliminando repetições.
  - COUNT() retorna a quantidade de departamentos com pelo menos um funcionário cadastrado:

```
SELECT
    COUNT(DISTINCT department_id) AS total_departamentos
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1a

Questão 1-a

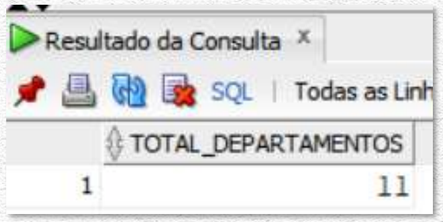
A diretoria quer ter uma visão clara dos departamentos que possuem funcionários ativos. Isso ajudará na alocação de recursos, no planejamento estratégico e na reestruturação de setores que possam estar inativos ou precisando de reforço na equipe.

Crie uma consulta que retorne a quantidade de departamentos distintos que possuem ao menos um funcionário alocado.

Dica: para garantir que somente entrarão na contagem os departamentos com funcionários ativos, **utilize a tabela employees**.

Explicação e Resultado

A consulta retorna o número total de departamentos distintos que possuem pelo menos um funcionário alocado:



Por que utilizamos a tabela employees?

Como queremos contar apenas os departamentos que possuem funcionários ativos, não precisamos consultar a tabela departments. Se um departamento não tem funcionários na tabela employees, ou seja, não tem funcionários, ele não será contado.

GABARITO EXPLICATIVO

1b

Questão 1-b

Se fizermos a contagem de departamentos distintos na tabela departments, teremos um resultado diferente do apresentado no item A?

O que isto significa?

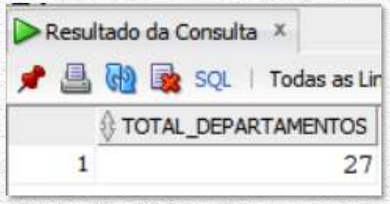
(Utilize a tabela departments)

Explicação e Resultado

Reaproveitando o código do item A e alterando somente a tabela para departments...

```
SELECT
    COUNT(DISTINCT department_id) AS total_departamentos
FROM departments;
```

... o resultado muda de 11 para 27:



Isso acontece porque, na tabela departments temos cadastrados todos os departamentos, inclusive aqueles que não têm funcionários.

Sendo assim, podemos concluir que dos 27 departamentos cadastrados, somente 11 possuem funcionários ativos. Os outros 16 não. Isso pode indicar setores inativos, recém-criados ou que precisam de novas contratações.

GABARITO EXPLICATIVO

2

Questão 2

O setor de Recursos Humanos deseja visualizar como os funcionários estão distribuídos entre os departamentos da empresa. Essa análise permitirá identificar quais áreas estão sobrecarregadas ou subutilizadas, auxiliando no planejamento de novas contratações.

Crie uma consulta que retorne o ID do departamento e a contagem de funcionários por departamento. Ordene o resultado pelo total de funcionários em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna o ID do departamento e a quantidade de funcionários que pertencem a cada um, fazemos o seguinte:

Retornamos a coluna **department_id** e usamos a função de agregação **COUNT(*)** para contar quantos funcionários existem para cada department_id.

Agrupamos por departamento fazendo **GROUP BY department_id**

Ordenamos o resultado mostrando os departamentos do maior para o menor número de funcionários com um **ORDER BY total_funcionarios DESC**:

```
SELECT
    department_id,
    COUNT(*) AS total_funcionarios
FROM employees
GROUP BY department_id
ORDER BY total_funcionarios DESC;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

O setor de Recursos Humanos deseja visualizar como os funcionários estão distribuídos entre os departamentos da empresa. Essa análise permitirá identificar quais áreas estão sobrecarregadas ou subutilizadas, auxiliando no planejamento de novas contratações.

Crie uma consulta que retorne o ID do departamento e a contagem de funcionários por departamento. Ordene o resultado pelo total de funcionários em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, exibimos o relatório abaixo, que ajuda a visualizar quais departamentos têm mais funcionários e quais podem estar subutilizados ou sobrecarregados, auxiliando no planejamento de novas contratações:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 12 em

	DEPARTMENT_ID	TOTAL_FUNCIONARIOS
1	50	45
2	80	34
3	30	6
4	100	6
5	60	5
6	90	3
7	20	2
8	110	2
9	70	1
10	40	1
11	10	1
12	(null)	1

GABARITO EXPLICATIVO

3

Questão 3

A equipe de planejamento estratégico deseja entender a evolução do quadro de funcionários ao longo do tempo. Com essa informação, será possível identificar tendências de crescimento ou retração da força de trabalho e planejar novas contratações de forma mais eficiente.

Crie uma consulta que retorne o ano e a quantidade de funcionários contratados por ano. Ordene o resultado por ano em ordem crescente.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna o ano e a quantidade de funcionários contratados em cada ano, seguimos estes passos:

Extraímos o ano da data de contratação utilizando **EXTRACT(YEAR FROM hire_date)**, pois queremos agrupar os funcionários por ano de admissão.

Contamos quantos funcionários foram contratados em cada ano usando a função de agregação **COUNT(*)**.

Agrupamos os dados pelo ano de contratação com **GROUP BY EXTRACT(YEAR FROM hire_date)**, garantindo que a contagem seja feita separadamente para cada ano.

Ordenamos os resultados em ordem crescente de ano com **ORDER BY ano_contratacao**, para que a evolução do quadro de funcionários seja visualizada de forma cronológica:

```
SELECT
    EXTRACT(YEAR FROM hire_date) AS ano_contratacao,
    COUNT(*) AS total_contratados
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY ano_contratacao;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

A equipe de planejamento estratégico deseja entender a evolução do quadro de funcionários ao longo do tempo. Com essa informação, será possível identificar tendências de crescimento ou retração da força de trabalho e planejar novas contratações de forma mais eficiente.

Crie uma consulta que retorne o ano e a quantidade de funcionários contratados por ano. Ordene o resultado por ano em ordem crescente.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, conseguimos gerar um relatório que mostra a evolução das contratações ao longo do tempo, auxiliando no planejamento estratégico da empresa:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 12 em 0

	ANO_CONTRATACAO	TOTAL_CONTRATADOS
1	1987	2
2	1989	1
3	1990	1
4	1991	1
5	1993	1
6	1994	7
7	1995	4
8	1996	10
9	1997	28
10	1998	23
11	1999	18
12	2000	11

GABARITO EXPLICATIVO

4

Questão 4

O setor financeiro quer avaliar a diferença entre os menores e os maiores salários dentro de cada departamento. Isso ajudará na criação de políticas de equidade salarial, na identificação de possíveis desigualdades e na estruturação de planos de carreira mais justos.

Crie uma consulta que retorne o ID do departamento, o piso, o teto e a diferença salarial por departamento. Ordene o resultado pela diferença salarial em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna o ID do departamento, o menor e o maior salário dentro de cada um, além da diferença entre esses valores, seguimos estes passos:

Selecionamos o **department_id**, pois queremos analisar os salários dentro de cada departamento.

Calculamos o menor salário dentro de cada departamento utilizando **MIN(salary)**, representando o **piso salarial**.

Calculamos o maior salário dentro de cada departamento utilizando **MAX(salary)**, representando o **teto salarial**.

Calculamos a **diferença** entre o maior e o menor salário com **MAX(salary) - MIN(salary)**, permitindo visualizar a disparidade salarial dentro do departamento.

Agrupamos os resultados por departamento com **GROUP BY department_id**, garantindo que os cálculos sejam feitos separadamente para cada setor.

Ordenamos os resultados em ordem decrescente de diferença salarial com **ORDER BY diferenca_salarial DESC**, para destacar os departamentos com maior desigualdade de salários no topo:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

O setor financeiro quer avaliar a diferença entre os menores e os maiores salários dentro de cada departamento. Isso ajudará na criação de políticas de equidade salarial, na identificação de possíveis desigualdades e na estruturação de planos de carreira mais justos.

Crie uma consulta que retorne o ID do departamento, o piso, o teto e a diferença salarial por departamento. Ordene o resultado pela diferença salarial em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

```
SELECT
    department_id,
    MIN(salary) AS piso_salarial,
    MAX(salary) AS teto_salarial,
    MAX(salary) - MIN(salary) AS diferenca_salarial
FROM employees
GROUP BY department_id
ORDER BY diferenca_salarial DESC;
```

Dessa forma, conseguimos visualizar a variação salarial dentro de cada departamento, auxiliando o setor financeiro na criação de políticas mais equitativas.

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 12 em 0,015 segundos

	DEPARTMENT_ID	PISO_SALARIAL	TETO_SALARIAL	DIFERENCA_SALARIAL
1	30	2500	11000	8500
2	80	6100	14000	7900
3	90	17000	24000	7000
4	20	6000	13000	7000
5	50	2100	8200	6100
6	100	6900	12000	5100
7	60	4200	9000	4800
8	110	8300	12000	3700
9	70	10000	10000	0
10	40	6500	6500	0
11	10	4400	4400	0
12	(null)	7000	7000	0

GABARITO EXPLICATIVO

5

Questão 5

O RH precisa saber quantos funcionários ocupam cargos de gerência na empresa. Essa informação será usada para avaliar a proporção entre gestores e equipes, garantindo uma estrutura organizacional eficiente e equilibrada.

Sabendo-se que os job_id dos gerentes contêm o termo 'MAN' (de manager), crie uma consulta que retorne a quantidade de funcionários que possuem um job_id que contém esse termo.

(Utilize a tabela employees)

Explicação e Resultado

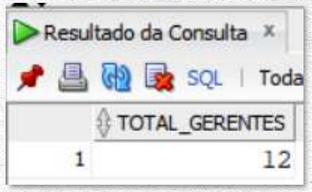
Para criar uma consulta que retorna a quantidade de funcionários que ocupam cargos de gerência, seguimos estes passos:

Contamos os funcionários usando **COUNT(employee_id)**, pois queremos saber quantos funcionários atendem ao critério de serem gerentes.

Filtramos apenas os cargos de gerência com **WHERE job_id LIKE '%MAN%'**, pois os identificadores de cargo (**job_id**) dos gerentes contêm a sigla **"MAN"** (de manager):

```
SELECT
    COUNT(employee_id) AS total_gerentes
FROM employees
WHERE job_id LIKE '%MAN%';
```

Dessa forma, obtemos o número total de funcionários que ocupam cargos de gerência, permitindo que o RH avalie a proporção entre gestores e equipes para garantir uma estrutura organizacional eficiente:



GABARITO EXPLICATIVO

6

Questão 6

A empresa deseja entender quanto do seu orçamento de folha de pagamento está sendo direcionado para funcionários que recebem comissão. Isso ajudará no planejamento financeiro e na definição de metas de vendas e incentivos.

Crie uma consulta que retorne a soma salarial dos funcionários cuja comissão não seja nula.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna a soma dos salários dos funcionários que recebem comissão, seguimos estes passos:

Calculamos o total de salários usando **SUM(salary)**.

Filtramos apenas os funcionários com comissão usando **WHERE commission_pct IS NOT NULL**, garantindo que apenas aqueles que possuem um valor registrado na coluna **commission_pct** sejam considerados:

```
SELECT
    SUM(salary) AS total_salarios_comissionados
FROM employees
WHERE commission_pct IS NOT NULL;
```

Dessa forma, obtemos o total gasto com funcionários comissionados, ajudando a empresa a planejar melhor seu orçamento e estratégias de incentivos:

Resultado da Consulta

SQL | Todas as Linhas Extraíd

	TOTAL_SALARIOS_COMISSIONADOS
1	311500

GABARITO EXPLICATIVO

7

Questão 7

Os gestores dos departamentos 10, 20 e 30 solicitaram um relatório detalhado sobre a distribuição de funcionários por cargo dentro de suas áreas. Essas informações serão usadas para avaliar necessidades de realocação, treinamento e possíveis novas contratações.

Crie uma consulta que retorne o ID do departamento, o ID do cargo e a contagem de funcionários por departamento e cargo, considerando somente os departamentos de ID 10, 20 ou 30.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna a distribuição de funcionários por cargo dentro dos departamentos 10, 20 e 30, seguimos estes passos:

Selecionamos as colunas relevantes: **department_id**, **job_id** e a contagem de funcionários (**COUNT(employee_id)**).

Filtramos apenas os departamentos solicitados com **WHERE department_id IN (10, 20, 30)**, garantindo que apenas esses departamentos sejam considerados.

Agrupamos os resultados por job_id e department_id com **GROUP BY job_id, department_id**, garantindo que a contagem de funcionários seja feita separadamente para cada cargo dentro de cada departamento:

```
SELECT
    department_id,
    job_id,
    COUNT(employee_id) AS total_funcionarios
FROM employees
WHERE department_id IN (10, 20, 30)
GROUP BY job_id, department_id;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Os gestores dos departamentos 10, 20 e 30 solicitaram um relatório detalhado sobre a distribuição de funcionários por cargo dentro de suas áreas. Essas informações serão usadas para avaliar necessidades de realocação, treinamento e possíveis novas contratações.

Crie uma consulta que retorne o ID do departamento, o ID do cargo e a contagem de funcionários por departamento e cargo, considerando somente os departamentos de ID 10, 20 ou 30.

(Utilize a tabela employees)

Explicação e Resultado

Dessa forma, o relatório gerado permite que os gestores analisem a distribuição dos funcionários dentro de suas áreas, ajudando na tomada de decisões sobre realocações, treinamentos e contratações futuras:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 5 em 0,087 segund

	DEPARTMENT_ID	JOB_ID	TOTAL_FUNCIONARIOS
1	10	AD_ASST	1
2	20	MK_MAN	1
3	20	MK_REP	1
4	30	PU_MAN	1
5	30	PU_CLERK	5

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer entender o tamanho dos departamentos que possuem uma média salarial elevada (acima de R$ 8.000). Essa análise ajudará a empresa a verificar se setores estratégicos estão bem estruturados e se os salários estão alinhados com a produtividade e os resultados entregues.

Crie uma consulta que mostre o ID do departamento, a média salarial e a contagem de funcionários por departamento, retornando somente aqueles cuja média salarial ultrapasse R$ 8.000. Ordene o resultado pelo total de funcionários em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna os departamentos com média salarial acima de R$8.000 e a quantidade de funcionários em cada um, seguimos estes passos:

Selecionamos as colunas necessárias: **department_id**, a média salarial (**AVG(salary)**) e a contagem de funcionários (**COUNT(employee_id)**).

Agrupamos os resultados por department_id com **GROUP BY department_id**, garantindo que os cálculos sejam feitos para cada departamento separadamente.

Filtramos os departamentos com média salarial maior que R$ 8.000 usando **HAVING AVG(salary) > 8000**, pois esse filtro precisa ser feito no agrupamento gerado, não na tabela de origem (por isso não podemos usar WHERE).

Ordenamos os resultados pelo total de funcionários em ordem decrescente (**ORDER BY total_funcionarios DESC**) para destacar os departamentos maiores primeiro:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer entender o tamanho dos departamentos que possuem uma média salarial elevada (acima de R$ 8.000). Essa análise ajudará a empresa a verificar se setores estratégicos estão bem estruturados e se os salários estão alinhados com a produtividade e os resultados entregues.

Crie uma consulta que mostre o ID do departamento, a média salarial e a contagem de funcionários por departamento, retornando somente aqueles cuja média salarial ultrapasse R$ 8.000. Ordene o resultado pelo total de funcionários em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

```
SELECT
    department_id,
    AVG(salary) media_salarial,
    COUNT(employee_id) AS total_funcionarios
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 8000
ORDER BY total_funcionarios DESC;
```

Dessa forma, o relatório gerado permite à diretoria identificar quais setores estratégicos possuem altos salários e quantos funcionários fazem parte desses departamentos, auxiliando na avaliação da estrutura organizacional e no alinhamento dos salários com a produtividade e os resultados entregues:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 6 em 0,017 segundos

	DEPARTMENT_ID	MEDIA_SALARIAL	TOTAL_FUNCIONARIOS
1	80	8955,88235294117647058823529411764705882	34
2	100	8600	6
3	90	19333,333333333333333333333333333333333	3
4	110	10150	2
5	20	9500	2
6	70	10000	1

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja identificar quais gerentes supervisionam equipes com pelo menos 3 funcionários. Essas informações serão usadas para planejar treinamentos de liderança, definir políticas de promoção e avaliar se alguns gestores estão sobrecarregados.

Crie uma consulta que mostre o ID do gerente e a contagem de funcionários por gerente, retornando somente aqueles cuja quantidade de funcionários supervisionados seja maior que 3. Ordene o resultado pelo total de funcionários gerenciados em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma consulta que retorna os gerentes que supervisionam mais de 3 funcionários, seguimos os seguintes passos:

Selecionamos o **manager_id** e a contagem de funcionários (**COUNT(employee_id)**) que cada gerente supervisiona.

Agrupamos os resultados por manager_id com o comando **GROUP BY manager_id**, para calcular a quantidade de funcionários supervisionados por cada gerente.

Filtramos os gerentes que supervisionam mais de 3 funcionários usando **HAVING COUNT(employee_id) > 3**. A cláusula HAVING aqui é necessária, pois esse filtro precisa ser aplicado após efetuado o agrupamento.

Ordenamos os resultados pelo total de funcionários em ordem decrescente (**ORDER BY total_funcionarios DESC**) para destacar os gerentes com maior número de funcionários sob sua supervisão:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja identificar quais gerentes supervisionam equipes com pelo menos 3 funcionários. Essas informações serão usadas para planejar treinamentos de liderança, definir políticas de promoção e avaliar se alguns gestores estão sobrecarregados.

Crie uma consulta que mostre o ID do gerente e a contagem de funcionários por gerente, retornando somente aqueles cuja quantidade de funcionários supervisionados seja maior que 3. Ordene o resultado pelo total de funcionários gerenciados em ordem decrescente.

(Utilize a tabela employees)

Explicação e Resultado

```
SELECT
    manager_id,
    COUNT(employee_id) AS total_funcionarios
FROM employees
GROUP BY manager_id
HAVING COUNT(employee_id) > 3
ORDER BY total_funcionarios DESC;
```

Dessa forma, o RH poderá identificar quais gerentes têm equipes maiores e, com base nisso, planejar treinamentos de liderança ou avaliar a sobrecarga de alguns gestores, ajudando na definição de estratégias para o desenvolvimento da liderança na empresa:

Resultado da Consulta

SQL | Todas as Linhas Extraídas:

	MANAGER_ID	TOTAL_FUNCIONARIOS
1	100	14
2	124	8
3	123	8
4	122	8
5	121	8
6	120	8
7	148	6
8	149	6
9	147	6
10	146	6
11	145	6
12	101	5
13	108	5
14	114	5
15	103	4

MÓDULO 9

JOINS E OPERADORES SET

JOINS E OPERADORES SET

JOINS E OPERADORES SET

O QUE VEREMOS NESTE MÓDULO

MÓDULO 9 JOINS e OPERADORES SET

01	JOINS VS. OPERADORES SET	374
02	CHAVE PRIMÁRIA VS. CHAVE ESTRANGEIRA	376
03	TABELA DIMENSÃO VS. TABELA FATO	377
04	INNER JOIN	379
05	ALIAS PARA TABELAS	382
06	INNER JOIN COM MÚLTIPLAS TABELAS	383
07	SELF JOIN	384
08	LEFT JOIN	388
09	RIGHT JOIN	391
10	FULL JOIN	394
11	CROSS JOIN	397
12	NONEQUIJOIN	399
13	UNION ALL	401
14	UNION	404
15	INTERSECT	407
16	MINUS	410
17	RESUMO DO MÓDULO	413
18	EXERCÍCIOS	414
19	GABARITO EXPLICATIVO	418

JOINS vs. OPERADORES SET

JOINS E OPERADORES SET (OPERADORES DE CONJUNTO) são usados para manipular tabelas (conjuntos).

Operadores SET combinam os resultados de duas ou mais consultas em um único conjunto de resultados que inclui todas as linhas que pertencem a todas as consultas na união. De forma bem direta, os operadores SET são usados para se manipular conjuntos.

Já os **JOINs** são usados para relacionar tabelas.
JOINs indicam como o SQL deve usar os dados de uma tabela para selecionar as linhas em outra tabela.

JOINS vs. OPERADORES SET

São eles:

JOINS

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN
- CROSS JOIN
- NONEQUIJOIN

Operadores SET

- UNION
- UNION ALL
- INTERSECT
- MINUS

CHAVE PRIMÁRIA vs. CHAVE ESTRANGEIRA

Uma **Chave Primária** é uma coluna que identifica as informações distintas em uma tabela. Geralmente é uma coluna de ID. Toda tabela terá uma, e somente uma, chave primária. Essa chave é utilizada como identificador único da tabela, sendo representada por uma coluna que não receberá valores repetidos.

Já uma **Chave Estrangeira** é uma coluna que permite relacionar as linhas de uma segunda tabela com a Chave Primária de uma primeira tabela.

Como pode ser visto ao lado, a tabela JOBS possui uma coluna chamada job_id, com valores que não se repetem. Essa será a Chave Primária.

Já na tabela EMPLOYEES, a coluna de job_id também aparece, mas os valores se repetem. Isso porque podemos ter mais de um funcionário com o mesmo cargo.

Na tabela EMPLOYEES, a coluna de job_id vai ser a Chave Estrangeira e vai permitir a gente relacionar os valores dessa coluna com a Chave Primária da tabela JOBS.

TABELA JOBS

	JOB_ID	JOB_TITLE
1	AD_PRES	President
2	AD_VP	Administration Vice President
3	AD_ASST	Administration Assistant
4	FI_MGR	Finance Manager
5	FI_ACCOUNT	Accountant
6	AC_MGR	Accounting Manager
7	AC_ACCOUNT	Public Accountant

Chave Primária

TABELA EMPLOYEES

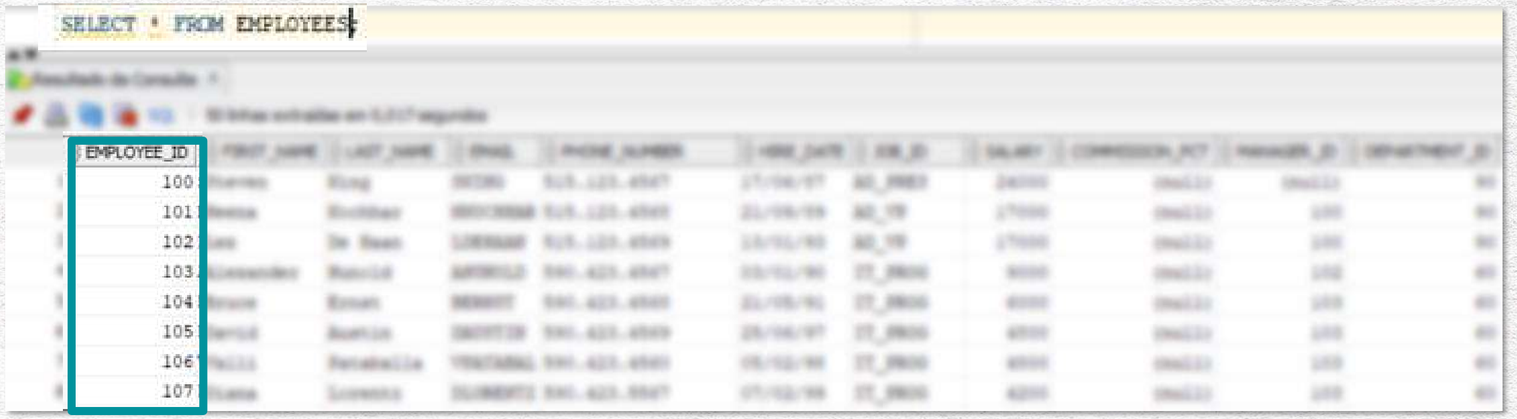
	EMPLOYEE_ID	FIRST_NAME	JOB_ID
1	100	Steven	AD_PRES
2	101	Neena	AD_VP
3	102	Lex	AD_VP
4	103	Alexander	IT_PROG
5	104	Bruce	IT_PROG
6	105	David	IT_PROG
7	106	Valli	IT_PROG
8	107	Diana	IT_PROG
9	108	Nancy	FI_MGR
10	109	Daniel	FI_ACCOUNT

Chave Estrangeira

TABELA DIMENSÃO vs. TABELA FATO

Uma **Tabela Dimensão** é uma tabela que contém características de um determinado elemento: lojas, produtos, funcionários, clientes, etc.

Nessa tabela, nenhum dos elementos principais irá se repetir. **É onde vamos encontrar nossas chaves primárias.**

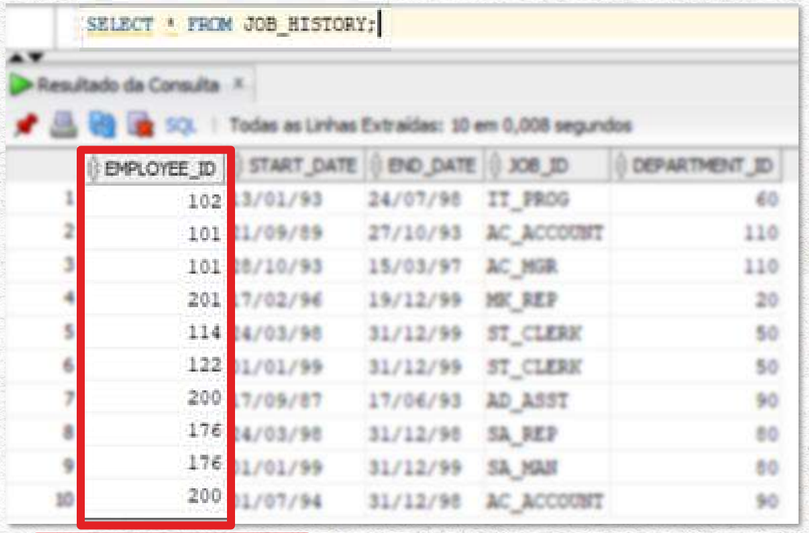


Chave Primária

Já uma **Tabela Fato** é aquela que vai registrar os fatos ou acontecimentos de uma empresa/negócio em determinados períodos de tempo (vendas, devoluções, aberturas de chamados, receitas, despesas, etc.)

Geralmente, é uma tabela com milhares de informações e composta essencialmente por colunas de ID, conhecidas como **chaves estrangeiras**, usadas para buscar as informações complementares de uma tabela dimensão.

No exemplo desta página, a tabela EMPLOYEES é a nossa tabela Dimensão e a JOB_HISTORY é a nossa tabela Fato.



Chave Estrangeira

TABELA DIMENSÃO vs. TABELA FATO

Não necessariamente uma relação acontece entre uma fato e uma dimensão.

Duas tabelas dimensão também podem se relacionar, como é o caso do exemplo ao lado.

O que não fazemos é uma relação entre duas tabelas fato.

```
SELECT * FROM DEPARTMENTS;
```

Resultado da Consulta

Todas as Linhas Extraídas: 27 em 0,011 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700
4	40	Human Resources	203	2400
5	50	Shipping	121	1500

Chave Primária

```
SELECT * FROM EMPLOYEES;
```

Chave Estrangeira

DEPARTMENT_ID
90
90
90
60
60
60
60
60

INNER JOIN

O **(INNER) JOIN** realiza o relacionamento entre as tabelas e retorna apenas as linhas que são comuns entre as duas tabelas.

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

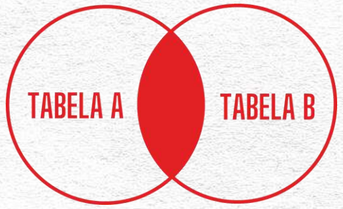


TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão

INNER JOIN

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

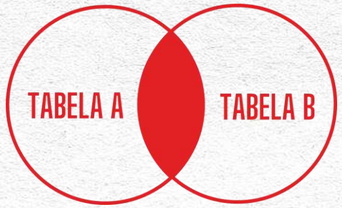


TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão

Sintaxe

```
SELECT colunas

FROM Tabela_A

INNER JOIN  Tabela_B
ON Tabela_A.coluna = Tabela_B.coluna
```

INNER JOIN

Veja o exemplo ao lado:

Nele, utilizamos o **INNER JOIN** para relacionar as tabelas EMPLOYEES e DEPARTMENTS e retornar as colunas employee_id, first_name, salary e department_id da tabela employees, assim como a coluna department_name da tabela departments.

Para isso, fizemos um INNER JOIN entre as duas tabelas, por meio da coluna em comum entre elas: a department_id.

Assim, no resultado, obtivemos uma consulta que retornou todas as linhas em comum entre essas duas tabelas (employees e departments).

```
SELECT
    employees.employee_id,
    employees.first_name,
    employees.salary,
    employees.department_id,
    departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.department_id;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,013 segundos

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_ID	DEPARTMENT_NAME
1	200	Jennifer	4400	10	Administration
2	201	Michael	13000	20	Marketing
3	202	Pat	6000	20	Marketing
4	114	Den	11000	30	Purchasing
5	115	Alexander	3100	30	Purchasing

ALIAS PARA TABELAS

Observe que, para deixar o código mais limpo, podemos dar um “apelido” (alias) para cada uma das tabelas.

O código ao lado retorna exatamente o mesmo resultado da consulta da página anterior, só que neste, utilizamos aliases para renomear as tabelas: a tabela employees foi apelidada de “e”, enquanto que a departments tornou-se “d”.

Obs.: os aliases somente facilitam a codificação da consulta; eles não alteram os nomes das colunas/tabelas no banco de dados!

```
SELECT
    e.employee_id,
    e.first_name,
    e.salary,
    e.department_id,
    d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,013 segundos

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_ID	DEPARTMENT_NAME
1	200	Jennifer	4400	10	Administration
2	201	Michael	13000	20	Marketing
3	202	Pat	6000	20	Marketing
4	114	Den	11000	30	Purchasing
5	115	Alexander	3100	30	Purchasing

INNER JOIN COM MÚLTIPLAS TABELAS

Também é possível relacionarmos várias tabelas para criar nossas consultas.

Vamos supor que queiramos complementar as informações da nossa tabela EMPLOYEES com:

- O nome do departamento em que o funcionário trabalha (department_name, coluna da tabela DEPARTMENTS);
- O cargo desse funcionário (job_title, coluna da tabela JOBS);
- A cidade (city) e o país (country_id) em que ele trabalha (colunas da tabela LOCATIONS).

Para isso, vamos precisar fazer um INNER JOIN relacionando as tabelas EMPLOYEES, DEPARTMENTS, JOBS e LOCATIONS, por meio de suas colunas em comum:

EMPLOYEES e DEPARTMENTS → department_id
EMPLOYEES e JOBS → job_id
DEPARTMENTS e LOCATIONS → location_id

```
SELECT
    e.employee_id,
    e.first_name,
    d.department_name,
    j.job_title,
    l.city,
    l.country_id
FROM employees e
    INNER JOIN departments d  ON e.department_id = d.department_id
    INNER JOIN jobs j         ON e.job_id = j.job_id
    INNER JOIN locations l    ON d.location_id = l.location_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 106 em 0,026 segundos

	EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_NAME	JOB_TITLE	CITY	COUNTRY_ID
94	196	Alana	Shipping	Shipping Clerk	South San Francisco	US
95	197	Kevin	Shipping	Shipping Clerk	South San Francisco	US
96	198	Donald	Shipping	Shipping Clerk	South San Francisco	US
97	199	Douglas	Shipping	Shipping Clerk	South San Francisco	US
98	103	Alexander	IT	Programmer	Southlake	US
99	104	Bruce	IT	Programmer	Southlake	US
100	105	David	IT	Programmer	Southlake	US
101	106	Valli	IT	Programmer	Southlake	US
102	107	Diana	IT	Programmer	Southlake	US
103	201	Michael	Marketing	Marketing Manager	Toronto	CA
104	202	Pat	Marketing	Marketing Representative	Toronto	CA
105	203	Susan	Human Resources	Human Resources Representa...	London	UK
106	204	Hermann	Public Relations	Public Relations Represent	Munich	DE

SELF JOIN

Vamos agora entender como funciona um tipo de JOIN bem interessante: o **SELF JOIN**.

O SELF JOIN, na realidade, é um INNER JOIN que realiza o autorrelacionamento de uma tabela com ela mesma.

Parece confuso? Calma! Vamos ver este exemplo:

Como podemos visualizar a lista de funcionários e seus respectivos gerentes em uma mesma tabela?

Vamos, antes de tudo, fazer uma consulta a nossa tabela EMPLOYEES para relembrar suas colunas:

```
SELECT * FROM employees;
```

SELF JOIN

Repare que na tabela EMPLOYEES, temos, entre outras, as colunas com o ID do funcionário (employee_id), seu nome (first_name) e o ID do seu gerente (manager_id):

Resultado da Consulta

SQL | 50 linhas extraídas em 0,021 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60
6	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
7	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60

SELF JOIN

Analisando a tabela, podemos perceber que o manager_id dos gerentes é, também, o employee_id destes gerentes.

Veja só: se pegarmos a funcionária Neena como exemplo, veremos que o manager_id do gerente dela é o número 100 que, por sua vez, é o employee_id do Steven. Portanto, o Steven é o gerente da Neena.

Resultado da Consulta

SQL | 50 linhas extraídas em 0,021 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60
6	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
7	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60

SELF JOIN

Sendo assim, podemos fazer um SELF JOIN da tabela EMPLOYEES com ela mesma para retornar o nome do gerente ao lado do nome de cada funcionário.

Para isso, devemos pensar como se a tabela employees fosse duas tabelas: uma com as informações dos funcionários, a qual daremos o alias de e; e a outra com as informações dos gerentes, a qual atribuiremos o alias de g.

Assim, em nossa consulta, podemos chamar as colunas employee_id e first_name (do funcionário) da nossa tabela **e**, assim como a coluna first_name (do gerente) da nossa tabela **g**.

Em seguida, estabelecemos um INNER JOIN da tabela **e** com a tabela **g**, fazendo a coluna manager_id da tabela e (de funcionários) referência à coluna employee_id da tabela g (de gerentes).

Veja o resultado ao lado, e repare na primeira linha:
O Steven é, de fato, o gerente da Neena.

```
SELECT
    e.employee_id,
    e.first_name funcionario,
    g.first_name gerente
FROM employees e INNER JOIN employees g
ON e.manager_id = g.employee_id
ORDER BY e.employee_id;
```

Resultado da Consulta

SQL | 50 linhas extraídas em 0,006 segu

	EMPLOYEE_ID	FUNCIONARIO	GERENTE
1	101	Neena	Steven
2	102	Lex	Steven
3	103	Alexander	Lex
4	104	Bruce	Alexander
5	105	David	Alexander
6	106	Valli	Alexander
7	107	Diana	Alexander
8	108	Nancy	Neena
9	109	Daniel	Nancy

LEFT JOIN

O **LEFT JOIN** estabelece o relacionamento entre as tabelas e retorna as linhas que são comuns entre as duas tabelas e também as linhas que existem apenas na tabela da ESQUERDA.

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão
3	C	4	NULL

LEFT JOIN

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão
3	C	4	NULL

Sintaxe

```
SELECT colunas

FROM Tabela_A

LEFT JOIN  Tabela_B
 ON Tabela_A.coluna = Tabela_B.coluna
```

LEFT JOIN

Veja o exemplo ao lado:

Nele, utilizamos o LEFT JOIN para relacionar as tabelas EMPLOYEES e DEPARTMENTS e retornar as colunas employee_id, first_name e salary da tabela employees, assim como a coluna department_name da tabela departments.

Para isso, fizemos um LEFT JOIN entre as duas tabelas, por meio da coluna em comum entre elas: a department_id.

Assim, no resultado, obtivemos uma consulta que retornou todas as linhas em comum entre essas duas tabelas, e também o registro que existia apenas na tabela da esquerda (employees). Perceba que a linha 107 retornou a funcionária Kimberly, que existe na tabela da esquerda (employees), porém, não tem departamento associado a ela (department_name = null). Com o INNER JOIN, a Kimberly não apareceria na nossa consulta; já com o LEFT JOIN, ela aparece.

```
SELECT
    e.employee_id,
    e.first_name,
    e.salary,
    d.department_name
FROM employees e LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,038 segundos

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_NAME
103	112	Jose Manuel	7800	Finance
104	113	Luis	6900	Finance
105	205	Shelley	12000	Accounting
106	206	William	8300	Accounting
107	178	Kimberely	7000	(null)

RIGHT JOIN

O **RIGHT JOIN** estabelece o relacionamento entre as tabelas e retorna as linhas que são comuns entre as duas tabelas e também as linhas que existem apenas na tabela da DIREITA.

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

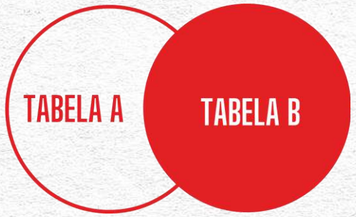


TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão
NULL	NULL	NULL	Notebook

RIGHT JOIN

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

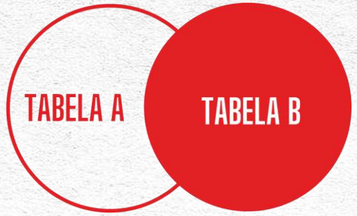


TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão
NULL	NULL	NULL	Notebook

Sintaxe

```
SELECT colunas

FROM Tabela_A

RIGHT JOIN  Tabela_B
 ON Tabela_A.coluna = Tabela_B.coluna
```

Índice Geral
Índice deste Módulo

RIGHT JOIN

Veja o exemplo ao lado:

Nele, utilizamos o RIGHT JOIN para relacionar as tabelas EMPLOYEES e DEPARTMENTS e retornar as colunas employee_id, first_name e salary da tabela employees, assim como a coluna department_name da tabela departments.

Para isso, fizemos um RIGHT JOIN entre as duas tabelas, por meio da coluna em comum entre elas: a department_id.

Assim, no resultado, obtivemos uma consulta que retornou todas as linhas em comum entre essas duas tabelas, e também os registros que existiam apenas na tabela da direita (departments). Perceba que as últimas linhas da consulta retornaram os nomes dos departamentos que existem na tabela departments, entretanto não possuem funcionários vinculados a eles na tabela employees (employee_id, first_name e salary são nulos).

```
SELECT
    e.employee_id,
    e.first_name,
    e.salary,
    d.department_name
FROM employees e RIGHT JOIN departments d
ON e.department_id = d.department_id
ORDER BY employee_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 122 em 0,016 segundos

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_NAME
103	203	Susan	6500	Human Resources
104	204	Hermann	10000	Public Relations
105	205	Shelley	12000	Accounting
106	206	William	8300	Accounting
107	(null)	(null)	(null)	Construction
108	(null)	(null)	(null)	Contracting
109	(null)	(null)	(null)	Operations

FULL JOIN

O **FULL JOIN** estabelece o relacionamento entre as tabelas e retorna TODAS as linhas das tabelas.

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

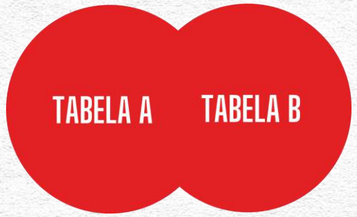


TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão
3	C	4	NULL
NULL	NULL	NULL	Notebook

FULL JOIN

TABELA A

ID_Produto	Produto	ID_Categoria
1	A	1
2	B	2
3	C	4

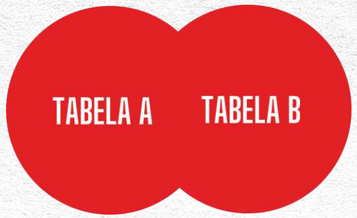


TABELA B

ID_Categoria	Categoria
1	Celular
2	Televisão
3	Notebook

TABELA FINAL

ID_Produto	Produto	ID_Categoria	Categoria
1	A	1	Celular
2	B	2	Televisão
3	C	4	NULL
NULL	NULL	NULL	Notebook

Sintaxe

```
SELECT colunas

FROM Tabela_A

FULL JOIN  Tabela_B
 ON Tabela_A.coluna = Tabela_B.coluna
```

Índice Geral
Índice deste Módulo

FULL JOIN

Veja o exemplo ao lado:

Nele, utilizamos o FULL JOIN para relacionar as tabelas EMPLOYEES e DEPARTMENTS e retornar as colunas employee_id, first_name e salary da tabela employees, assim como a coluna department_name da tabela departments.

Para isso, fizemos um FULL JOIN entre as duas tabelas, por meio da coluna em comum entre elas: a department_id.

Assim, no resultado, obtivemos uma consulta que retornou TODAS as linhas das duas tabelas.
Veja que a funcionária Kimberly, que não tem departamento associado a ela, portanto, existe apenas na tabela da esquerda (employees) apareceu no resultado; e os departamentos que não têm funcionários, que existem somente na tabela da direita (departments) também.

```
SELECT
    e.employee_id,
    e.first_name,
    e.salary,
    d.department_name
FROM employees e FULL JOIN departments d
ON e.department_id = d.department_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 123 em 0,014 segundos

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_NAME
77	176	Jonathon	8600	Sales
78	177	Jack	8400	Sales
79	178	Kimberely	7000	(null)
80				

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_NAME
106	205	Shelley	12000	Accounting
107	206	William	8300	Accounting
108	(null)	(null)	(null)	IT Support
109	(null)	(null)	(null)	Operations
110	(null)	(null)	(null)	Payroll

CROSS JOIN

O **CROSS JOIN** realiza o produto cartesiano entre duas colunas, ou seja: ele faz todas as combinações possíveis entre duas colunas.

Vamos ver um exemplo agora, utilizando o CROSS JOIN para combinar as colunas DEPARTMENT_NAME e JOB_TITLE:

```
SELECT
    d.department_name,
    j.job_title
FROM departments d CROSS JOIN jobs j;
```

CROSS JOIN

Perceba que, para cada item da coluna DEPARTMENT_NAME, o CROSS JOIN realiza uma combinação com cada item da coluna JOB_TITLE:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 513 em 0,079 segundos

	DEPARTMENT_NAME	JOB_TITLE
1	Administration	President
2	Marketing	President
3	Purchasing	President
4	Human Resources	President

	DEPARTMENT_NAME	JOB_TITLE
28	Administration	Administration Vice President
29	Marketing	Administration Vice President
30	Purchasing	Administration Vice President
31	Human Resources	Administration Vice President

	DEPARTMENT_NAME	JOB_TITLE
55	Administration	Administration Assistant
56	Marketing	Administration Assistant
57	Purchasing	Administration Assistant
58	Human Resources	Administration Assistant

	DEPARTMENT_NAME	JOB_TITLE
82	Administration	Finance Manager
83	Marketing	Finance Manager
84	Purchasing	Finance Manager
85	Human Resources	Finance Manager

	DEPARTMENT_NAME	JOB_TITLE
109	Administration	Accountant
110	Marketing	Accountant
111	Purchasing	Accountant
112	Human Resources	Accountant

NONEQUIJOIN

Todos os JOINs que vimos até agora são executados por uma igualdade, pois unimos as tabelas fazendo uso de colunas que estabelecem uma relação entre as tabelas:

```
SELECT T1.coluna_1, T2.coluna_2
FROM tabela_1 T1
JOIN tabela_2 T2
ON T1.nome_da_coluna = T2.nome_da_coluna;
```

Agora, veremos o **NONEQUIJOIN**, que estabelece uma relação entre tabelas não baseada em igualdade, mas sim fazendo uso dos outros diversos operadores lógicos: <, >, <=, >=, <>, BETWEEN, AND, OR, IN, etc.
Para isso, utilizamos a cláusula WHERE. Veja um exemplo de sua sintaxe:

```
SELECT
   T1.coluna_1,
   T1.coluna_2,
   T2.coluna_3
FROM tabela_1 T1, tabela_2 T2
WHERE T2.coluna_3 <> [valor1]
AND T1_coluna_1 <= [valor2];
```

NONEQUIJOIN

Vamos a um exemplo:

Desta vez, devemos retornar o department_id, o department_name, a location e a city das tabelas DEPARTMENTS e LOCATIONS, observando-se que: o intervalo de location_id deve ser entre 1800 e 2500 e o department_id menor ou igual a 30:

Para isso, vamos chamar todas as colunas desejadas de ambas as tabelas no SELECT, listar as tabelas no FROM, estabelecendo, por fim, as condições por meio da cláusula WHERE:

```
SELECT
    d.department_id,
    d.department_name,
    l.location_id,
    l.city
FROM departments d, locations l
WHERE l.location_id BETWEEN 1800 AND 2500
AND d.department_id <= 30;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 24 em 0,017 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID	CITY
1	10	Administration	1800	Toronto
2	10	Administration	1900	Whitehorse
3	10	Administration	2000	Beijing
4	10	Administration	2100	Bombay
5	10	Administration	2200	Sydney
6	10	Administration	2300	Singapore
7	10	Administration	2400	London
8	10	Administration	2500	Oxford
9	20	Marketing	1800	Toronto
10	20	Marketing	1900	Whitehorse
11	20	Marketing	2000	Beijing
12	20	Marketing	2100	Bombay
13	20	Marketing	2200	Sydney
14	20	Marketing	2300	Singapore
15	20	Marketing	2400	London
16	20	Marketing	2500	Oxford
17	30	Purchasing	1800	Toronto
18	30	Purchasing	1900	Whitehorse
19	30	Purchasing	2000	Beijing
20	30	Purchasing	2100	Bombay
21	30	Purchasing	2200	Sydney
22	30	Purchasing	2300	Singapore
23	30	Purchasing	2400	London
24	30	Purchasing	2500	Oxford

UNION ALL

O **UNION ALL** faz a união entre duas tabelas. As linhas inteiramente iguais também são mantidas.

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

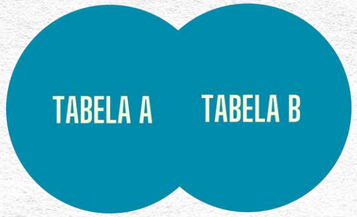


TABELA B

ID	Produto	Preco
2	A	10
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
1	A	10
2	B	50
3	C	20
2	A	10
3	C	20
5	E	15

UNION ALL

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

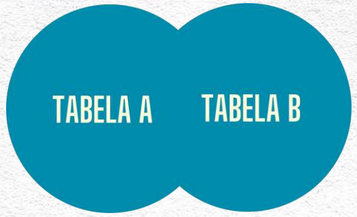


TABELA B

ID	Produto	Preco
1	A	10
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
1	A	10
2	B	50
3	C	20
1	A	10
3	C	20
5	E	15

Sintaxe

```
SELECT *
FROM Tabela_A
UNION ALL
SELECT *
FROM Tabela_B
```

Índice Geral
Índice deste Módulo

UNION ALL

Veja este exemplo:

Nele, utilizamos o UNION ALL para unir duas consultas feitas à tabela EMPLOYEES.

Na primeira consulta, pedimos para retornar apenas os funcionários dos departamentos com department_id 50, 80 ou 100.

Já na segunda consulta, apenas os funcionários com job_id = 'ST_MAN'.

```
SELECT
    employee_id, job_id, salary, department_id
FROM employees
WHERE department_id IN (50, 80, 100)
UNION ALL
SELECT
    employee_id, job_id, salary, department_id
FROM employees
WHERE job_id = 'ST_MAN'
ORDER BY employee_id;
```

Repare no resultado:

Como os funcionários de employee_id 120 a 124 retornaram tanto no primeiro SELECT (pois são do department_id 50), quanto no segundo (por terem job_id = 'ST_MAN'), eles se repetem no resultado, já que o UNION ALL, mantém as linhas que são inteiramente iguais.

Resultado da Consulta

SQL | 50 linhas extraídas em 0,004 segundos

	EMPLOYEE_ID	JOB_ID	SALARY	DEPARTMENT_ID
5	112	FI_ACCOUNT	7800	100
6	113	FI_ACCOUNT	6900	100
7	120	ST_MAN	8000	50
8	120	ST_MAN	8000	50
9	121	ST_MAN	8200	50
10	121	ST_MAN	8200	50
11	122	ST_MAN	7900	50
12	122	ST_MAN	7900	50
13	123	ST_MAN	6500	50
14	123	ST_MAN	6500	50
15	124	ST_MAN	5800	50
16	124	ST_MAN	5800	50
17	125	ST_CLERK	3200	50

UNION

O **UNION** faz a união entre duas tabelas. As linhas inteiramente iguais são excluídas, ficando apenas uma delas.

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

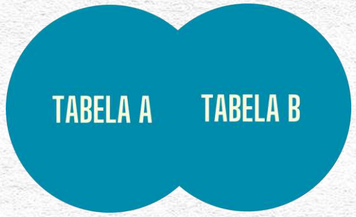


TABELA B

ID	Produto	Preco
2	B	50
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
1	A	10
2	B	50
3	C	20
5	E	15

UNION

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

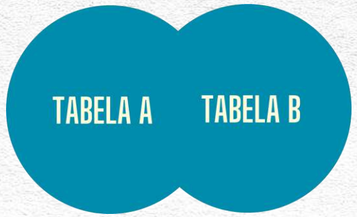


TABELA B

ID	Produto	Preco
2	B	50
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
1	A	10
2	B	50
3	C	20
5	E	15

Sintaxe

```
SELECT *
FROM Tabela_A
UNION
SELECT *
FROM Tabela_B
```

Índice Geral
Índice deste Módulo

UNION

Veja este exemplo:

Nele, utilizamos o UNION para unir duas consultas feitas à tabela EMPLOYEES.

Novamente, na primeira consulta, pedimos para retornar apenas os funcionários dos departamentos com department_id 50, 80 ou 100.

E na segunda consulta, apenas os funcionários com job_id = 'ST_MAN'.

```
SELECT
    employee_id, job_id, salary, department_id
FROM employees
WHERE department_id IN (50, 80, 100)
UNION
SELECT
    employee_id, job_id, salary, department_id
FROM employees
WHERE job_id = 'ST_MAN'
ORDER BY employee_id;
```

Repare no resultado:

Como, desta vez, utilizamos o UNION em vez do UNION ALL, apesar dos funcionários de employee_id 120 a 124 serem do department_id 50 e terem job_id = 'ST_MAN', eles não se repetem no resultado, já que o UNION exclui as linhas que são inteiramente iguais, retornando apenas uma delas.

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 85 em 0,016 segundo

	EMPLOYEE_ID	JOB_ID	SALARY	DEPARTMENT_ID
5	112	FI_ACCOUNT	7800	100
6	113	FI_ACCOUNT	6900	100
7	120	ST_MAN	8000	50
8	121	ST_MAN	8200	50
9	122	ST_MAN	7900	50
10	123	ST_MAN	6500	50
11	124	ST_MAN	5800	50
12	125	ST_CLERK	3200	50

INTERSECT

O **INTERSECT** faz a interseção entre duas tabelas. Ou seja, exclui as linhas que aparecem apenas em uma das tabelas, retornando somente as que são inteiramente iguais nas duas tabelas.

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

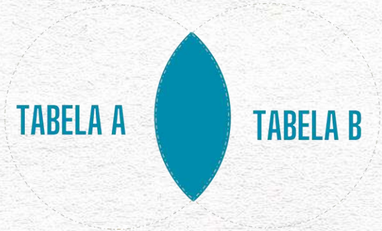


TABELA B

ID	Produto	Preco
2	B	50
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
2	B	50
3	C	20

INTERSECT

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

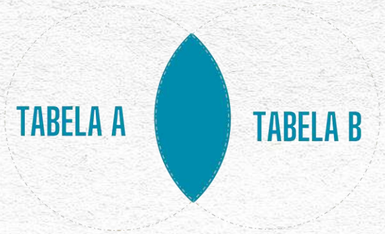


TABELA B

ID	Produto	Preco
2	B	50
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
2	B	50
3	C	20

Sintaxe

```
SELECT *
FROM Tabela_A
INTERSECT
SELECT *
FROM Tabela_B
```

INTERSECT

Veja este exemplo:

Nele, utilizamos o INTERSECT para unir duas consultas feitas à tabela EMPLOYEES.

Aqui, na primeira consulta, pedimos para retornar apenas os funcionários dos departamentos com department_id 30, 50 ou 100.

Já na segunda consulta, apenas os funcionários dos departamentos com department_id 30, 60 ou 90.

```
SELECT
    employee_id, department_id, job_id, salary
FROM employees
WHERE department_id IN (30, 50, 100)
INTERSECT
SELECT
    employee_id, department_id, job_id, salary
FROM employees
WHERE department_id IN (30, 60, 90)
ORDER BY employee_id;
```

Repare no resultado:

Foram retornados apenas os funcionários do department_id 30, já que na interseção do primeiro SELECT com o segundo, somente ele apareceu em comum:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 6 em 0,003 segund

	EMPLOYEE_ID	DEPARTMENT_ID	JOB_ID	SALARY
1	114	30	PU_MAN	11000
2	115	30	PU_CLERK	3100
3	116	30	PU_CLERK	2900
4	117	30	PU_CLERK	2800
5	118	30	PU_CLERK	2600
6	119	30	PU_CLERK	2500

MINUS

O **MINUS** faz a 'subtração' entre duas tabelas. Ou seja, exclui todas as linhas que aparecem na segunda tabela, retornando apenas as que são exclusivamente da primeira tabela.

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

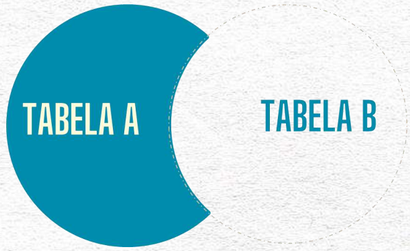


TABELA B

ID	Produto	Preco
2	B	50
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
1	A	10

MINUS

TABELA A

ID	Produto	Preco
1	A	10
2	B	50
3	C	20

TABELA B

ID	Produto	Preco
2	B	50
3	C	20
5	E	15

TABELA FINAL

ID	Produto	Preco
1	A	10

Sintaxe

```
SELECT *
FROM Tabela_A
MINUS
SELECT *
FROM Tabela_B
```

MINUS

Veja este exemplo:

Nele, utilizamos o MINUS para unir duas consultas feitas à tabela EMPLOYEES.

Novamente, na primeira consulta, pedimos para retornar apenas os funcionários dos departamentos com department_id 30, 50 ou 100.

E na segunda consulta, apenas os funcionários dos departamentos com department_id 30, 60 ou 90.

```
SELECT
    employee_id, department_id, job_id, salary
FROM employees
WHERE department_id IN (30, 50, 100)
MINUS
SELECT
    employee_id, department_id, job_id, salary
FROM employees
WHERE department_id IN (30, 60, 90)
ORDER BY employee_id;
```

Repare no resultado:

Foram retornados apenas os funcionários dos department_id 50 e 100, já que são os únicos departamentos que aparecem exclusivamente na primeira consulta:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 51 em 0,016 segundo

	EMPLOYEE_ID	DEPARTMENT_ID	JOB_ID	SALARY
1	108	100	FI_MGR	12000
2	109	100	FI_ACCOUNT	9000
3	110	100	FI_ACCOUNT	8200
4	111	100	FI_ACCOUNT	7700
5	112	100	FI_ACCOUNT	7800
6	113	100	FI_ACCOUNT	6900
7	120	50	ST_MAN	8000
8	121	50	ST_MAN	8200
9	122	50	ST_MAN	7900
10	123	50	ST_MAN	6500
11	124	50	ST_MAN	5800
12	125	50	ST_CLERK	3200
13	126	50	ST_CLERK	2700

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. JOINs são usados para criar relações entre tabelas, enquanto os OPERADORES SET são usados para fazer operações de conjunto entre tabelas.
2. Os OPERADORES SET não criam nenhum tipo de relação entre as tabelas.
3. Temos quatro principais JOINs (INNER, LEFT, RIGHT e FULL) mas os dois mais utilizados são o INNER e o LEFT.
4. Utilizamos ALIASES de tabelas para melhorar a escrita dos nossos códigos, deixando-os mais legíveis.
5. Os OPERADORES SET que podemos usar são: UNION, UNION ALL, INTERSECT e MINUS.

EXERCÍCIOS

1

Questão 1

O setor financeiro está preparando o orçamento anual da empresa e precisa analisar o impacto da folha de pagamento em cada departamento.

Para isso, solicitou um relatório que mostre o nome completo dos funcionários, o salário mensal, o salário anual projetado (12 vezes o salário mensal) e o departamento ao qual estão vinculados.

Essas informações ajudarão a comparar gastos por área e a planejar possíveis contratações ou cortes.

Crie uma consulta com um LEFT JOIN que retorne o resultado esperado.

(Utilize as tabelas employees e departments)

2

Questão 2

A equipe de controladoria está avaliando a competitividade dos salários oferecidos em diferentes cidades onde a empresa possui unidades.

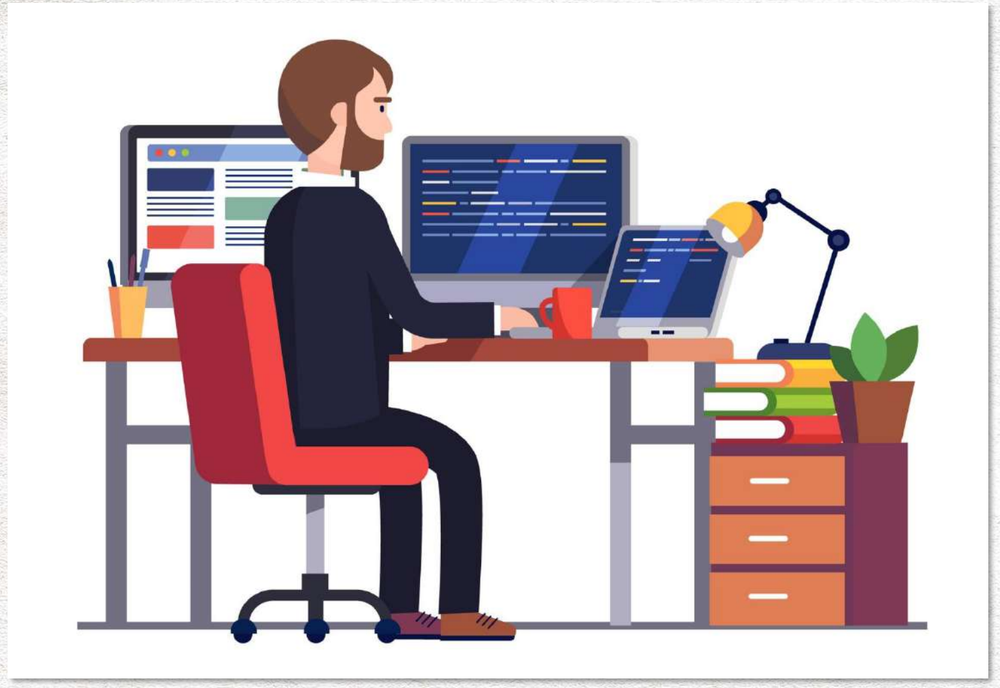
O objetivo é verificar se existem discrepâncias salariais relevantes entre localidades e, com isso, propor ajustes ou bonificações regionais.

Para isso, foi solicitado um levantamento da média salarial dos funcionários agrupados por cidade.

Crie uma consulta com INNER JOINs que retorne o resultado esperado.

(Utilize as tabelas employees, departments e locations)

EXERCÍCIOS



3

Questão 3

A área de expansão internacional está auditando o cadastro de países no sistema. Eles precisam identificar quais países estão registrados, mas que ainda não possuem nenhuma localização (filial ou escritório) associada, o que pode indicar um cadastro incompleto ou uma oportunidade de crescimento. Ao mesmo tempo, é importante exibir as localizações que já existem.

Crie uma consulta com RIGHT JOIN que retorne o ID da localização (se houver), a cidade (se houver) e o país.

(Utilize as tabelas locations e countries)

4

Questão 4

Durante a reformulação da estrutura organizacional, a equipe de Planejamento Estratégico está avaliando como os cargos poderiam ser distribuídos por região.

Para isso, foi solicitado um levantamento com todas as combinações possíveis entre cargos existentes e regiões do mundo, permitindo simular diferentes estruturas e cenários.

Crie uma consulta com um CROSS JOIN que retorne todas as combinações possíveis entre cargo e região.

(Utilize as tabelas jobs e regions)

EXERCÍCIOS

5

Questão 5

O time de Recursos Humanos está revendo a política de remuneração da empresa. Para identificar possíveis candidatos a promoções salariais, eles pediram um relatório com a margem de aumento de cada funcionário — ou seja, quanto ainda pode ser acrescido ao salário atual sem ultrapassar o máximo previsto para o cargo.

Crie uma consulta com um INNER JOIN que retorne o nome completo do funcionário, o ID de seu cargo, seu salário atual, o teto salarial permitido para seu cargo e sua margem para aumento. Considere como margem para aumento a diferença entre o teto salarial do cargo e o atual salário do funcionário.

(Utilize as tabelas employees e jobs)

6

Questão 6

A diretoria solicitou a criação de um painel com a cobertura geográfica completa da empresa, que inclua tanto as regiões onde atua quanto os países cadastrados no sistema. Essa informação será usada em apresentações institucionais e materiais para investidores, sendo necessário consolidar os dados em uma única visualização simples.

Crie uma consulta utilizando UNION ALL para juntar os nomes das regiões e dos países em uma única coluna, com um campo adicional que indique de qual tipo de entidade se trata (“Região” ou “País”).

(Utilize as tabelas regions e countries)

EXERCÍCIOS

7

Questão 7

A equipe de TI está realizando uma auditoria nos dados de infraestrutura para verificar a ocupação dos espaços da empresa. Uma das etapas dessa análise consiste em listar todas as localidades cadastradas que ainda não foram associadas a nenhum departamento.

Essas localidades poderão ser utilizadas futuramente para alocação de novos times ou projetos.

Crie uma consulta com o operador MINUS que liste os IDs das localidades que não estão associadas a nenhum departamento.

(Utilize as tabelas locations e departments)

8

Questão 8

A equipe de Recursos Humanos está preparando um relatório sobre a trajetória dos funcionários dentro da empresa, com foco em mobilidade interna. O objetivo é identificar os profissionais que já passaram por pelo menos uma movimentação de cargo desde que foram contratados.

Para isso, foi solicitado um levantamento com os employee_id que aparecem tanto na tabela de funcionários atuais quanto na tabela de histórico de cargos. Esses dados ajudarão a mapear padrões de promoção e a desenvolver programas de carreira.

Crie uma consulta com o operador INTERSECT que identifique os IDs de funcionários que aparecem nas duas tabelas.

(Utilize as tabelas employees e job_history)

GABARITO EXPLICATIVO

1

Questão 1

O setor financeiro está preparando o orçamento anual da empresa e precisa analisar o impacto da folha de pagamento em cada departamento.

Para isso, solicitou um relatório que mostre o nome completo dos funcionários, o salário mensal, o salário anual projetado (12 vezes o salário mensal) e o departamento ao qual estão vinculados.

Essas informações ajudarão a comparar gastos por área e a planejar possíveis contratações ou cortes.

Crie uma consulta com um LEFT JOIN que retorne o resultado esperado.

(Utilize as tabelas employees e departments)

Explicação e Resultado

Para criar uma consulta que retorna o nome completo dos funcionários, seu salário mensal, o salário anual projetado e o nome do departamento onde trabalham, seguimos os passos abaixo:

- ✓ Selecionamos o **first_name** e o **last_name** do funcionário, unindo-os com o operador || para formar o nome completo.
- ✓ Selecionamos o **salary** (salário mensal) diretamente da tabela **employees**.
- ✓ Calculamos o salário anual **multiplicando** o salário mensal por 12.
- ✓ Incluímos o **department_name**, que está na tabela **departments**.
- ✓ Utilizamos um **LEFT JOIN** entre **employees** e **departments**, relacionando os departamentos mesmo nos casos em que o funcionário ainda não tenha um departamento atribuído. **Isso garante que todos os funcionários apareçam no relatório:**

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O setor financeiro está preparando o orçamento anual da empresa e precisa analisar o impacto da folha de pagamento em cada departamento.

Para isso, solicitou um relatório que mostre o nome completo dos funcionários, o salário mensal, o salário anual projetado (12 vezes o salário mensal) e o departamento ao qual estão vinculados.

Essas informações ajudarão a comparar gastos por área e a planejar possíveis contratações ou cortes.

Crie uma consulta com um LEFT JOIN que retorne o resultado esperado.

(Utilize as tabelas employees e departments)

Explicação e Resultado

```
SELECT
    e.first_name || ' ' || e.last_name AS nome_funcionario,
    e.salary AS salario_mensal,
    e.salary * 12 AS salario_anual,
    d.department_name AS departamento
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Dessa forma, o setor financeiro terá uma visão completa dos gastos com salários por funcionário e por departamento, incluindo aqueles que ainda não estão formalmente alocados em um setor, o que facilita a tomada de decisões para o orçamento anual:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,018 segundos

	NOME_FUNCIONARIO	SALARIO_MENSAL	SALARIO_ANUAL	DEPARTAMENTO
1	Steven King	24000	288000	Executive
2	Neena Kochhar	17000	204000	Executive
3	Lex De Haan	17000	204000	Executive
4	Alexander Hunold	9000	108000	IT
5	Bruce Ernst	6000	72000	IT
6	David Austin	4800	57600	IT
7	Valli Pataballa	4800	57600	IT
8	Diana Lorentz	4200	50400	IT
9	Nancy Greenberg	12000	144000	Finance
10	Daniel Faviet	9000	108000	Finance
11	John Chen	8200	98400	Finance
12	Ismael Sciarra	7700	92400	Finance

GABARITO EXPLICATIVO

2

Questão 2

A equipe de controladoria está avaliando a competitividade dos salários oferecidos em diferentes cidades onde a empresa possui unidades.

O objetivo é verificar se existem discrepâncias salariais relevantes entre localidades e, com isso, propor ajustes ou bonificações regionais.

Para isso, foi solicitado um levantamento da média salarial dos funcionários agrupados por cidade.

Crie uma consulta com INNER JOINs que retorne o resultado esperado.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

Para criar uma consulta que retorna a média salarial dos funcionários agrupados por cidade, seguimos os passos abaixo:

- ✓ Selecionamos a coluna **city** (cidade), que está na tabela **locations**.
- ✓ Calculamos a média salarial dos funcionários com **AVG(e.salary)**.
- ✓ Realizamos um **INNER JOIN** entre **employees** e **departments** para associar cada funcionário ao seu departamento.
- ✓ Fazemos outro **INNER JOIN** entre **departments** e **locations** para associar cada departamento à cidade onde está localizado.
- ✓ Agrupamos os resultados por cidade usando **GROUP BY l.city**:

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

2

Questão 2

A equipe de controladoria está avaliando a competitividade dos salários oferecidos em diferentes cidades onde a empresa possui unidades.

O objetivo é verificar se existem discrepâncias salariais relevantes entre localidades e, com isso, propor ajustes ou bonificações regionais.

Para isso, foi solicitado um levantamento da média salarial dos funcionários agrupados por cidade.

Crie uma consulta com INNER JOINs que retorne o resultado esperado.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

```
SELECT
    l.city AS cidade,
    AVG(e.salary) AS media_salarial
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id
INNER JOIN locations l ON d.location_id = l.location_id
GROUP BY l.city;
```

Dessa forma, a equipe de controladoria poderá analisar se existem discrepâncias entre os salários médios praticados nas diferentes localidades da empresa, possibilitando a criação de políticas mais justas e competitivas para cada região:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 7 em 0,014 segundos

	CIDADE	MEDIA_SALARIAL
1	Seattle	8844,44444444444444444444444444444444444444
2	Southlake	5760
3	South San Francisco	3475,55555555555555555555555555555555555556
4	Oxford	8955,88235294117647058823529411764705882
5	Toronto	9500
6	London	6500
7	Munich	10000

GABARITO EXPLICATIVO

3

Questão 3

A área de expansão internacional está auditando o cadastro de países no sistema. Eles precisam identificar quais países estão registrados, mas que ainda não possuem nenhuma localização (filial ou escritório) associada, o que pode indicar um cadastro incompleto ou uma oportunidade de crescimento. Ao mesmo tempo, é importante exibir as localizações que já existem.

Crie uma consulta com RIGHT JOIN que retorne o ID da localização (se houver), a cidade (se houver) e o país.

(Utilize as tabelas locations e countries)

Explicação e Resultado

Para criar uma consulta que retorna os países cadastrados no sistema e, quando houver, as localizações associadas (como filiais ou escritórios), seguimos os passos abaixo:

- ✓ Selecionamos o ID da localização (**l.location_id**) e a cidade (**l.city**) da tabela **locations**, pois são os dados que queremos exibir se houver uma localização.
- ✓ Selecionamos o nome do país da tabela **countries** (**c.country_name**), pois sempre queremos exibir os países, mesmo que não tenham localizações associadas.
- ✓ Utilizamos um **RIGHT JOIN** entre **locations** e **countries** para garantir que todos os países da tabela **countries** apareçam no resultado — mesmo que não haja correspondência na tabela **locations**.
- ✓ A junção é feita pela coluna **country_id**, que existe em ambas as tabelas:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

A área de expansão internacional está auditando o cadastro de países no sistema. Eles precisam identificar quais países estão registrados, mas que ainda não possuem nenhuma localização (filial ou escritório) associada, o que pode indicar um cadastro incompleto ou uma oportunidade de crescimento. Ao mesmo tempo, é importante exibir as localizações que já existem.

Crie uma consulta com RIGHT JOIN que retorne o ID da localização (se houver), a cidade (se houver) e o país.

(Utilize as tabelas locations e countries)

Explicação e Resultado

```
SELECT
    l.location_id,
    l.city,
    c.country_name
FROM locations l
RIGHT JOIN countries c ON l.country_id = c.country_id;
```

Dessa forma, a área de expansão internacional poderá verificar quais países ainda não possuem localizações cadastradas, identificando possíveis inconsistências no sistema ou oportunidades para abertura de novas unidades:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 34 em 0,027 segundos

	LOCATION_ID	CITY	COUNTRY_NAME
18	2700	Munich	Germany
19	2800	Sao Paulo	Brazil
20	2900	Geneva	Switzerland
21	3000	Bern	Switzerland
22	3100	Utrecht	Netherlands
23	3200	Mexico City	Mexico
24	(null)	(null)	HongKong
25	(null)	(null)	Zambia
26	(null)	(null)	Argentina
27	(null)	(null)	Nigeria
28	(null)	(null)	Egypt
29	(null)	(null)	Denmark
30	(null)	(null)	France
31	(null)	(null)	Israel
32	(null)	(null)	Zimbabwe
33	(null)	(null)	Kuwait
34	(null)	(null)	Belgium

GABARITO EXPLICATIVO

4

Questão 4

Durante a reformulação da estrutura organizacional, a equipe de Planejamento Estratégico está avaliando como os cargos poderiam ser distribuídos por região.

Para isso, foi solicitado um levantamento com todas as combinações possíveis entre cargos existentes e regiões do mundo, permitindo simular diferentes estruturas e cenários.

Crie uma consulta com um CROSS JOIN que retorne todas as combinações possíveis entre cargo e região.

(Utilize as tabelas jobs e regions)

Explicação e Resultado

Para criar uma consulta que retorna todas as combinações possíveis entre cargos e regiões, seguimos os passos abaixo:

- ✓ Selecionamos o título do cargo da tabela **jobs**, usando a coluna **j.job_title**.
- ✓ Selecionamos o nome da região da tabela **regions**, usando a coluna **r.region_name**.
- ✓ Utilizamos um **CROSS JOIN** entre **jobs** e **regions**, que gera o **produto cartesiano** entre as duas tabelas — ou seja, cada cargo será combinado com cada região:

```
SELECT
    j.job_title,
    r.region_name
FROM jobs j
CROSS JOIN regions r;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Durante a reformulação da estrutura organizacional, a equipe de Planejamento Estratégico está avaliando como os cargos poderiam ser distribuídos por região.

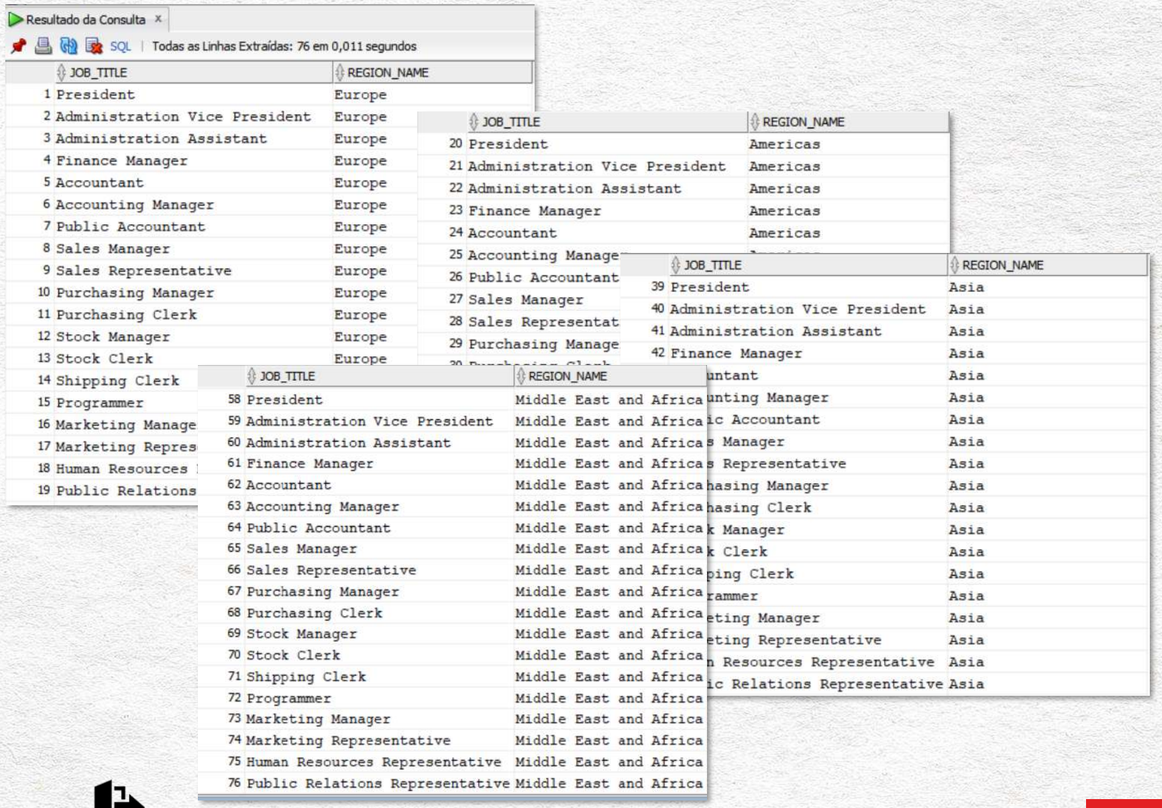
Para isso, foi solicitado um levantamento com todas as combinações possíveis entre cargos existentes e regiões do mundo, permitindo simular diferentes estruturas e cenários.

Crie uma consulta com um CROSS JOIN que retorne todas as combinações possíveis entre cargo e região.

(Utilize as tabelas jobs e regions)

Explicação e Resultado

Dessa forma, geramos todas as combinações possíveis entre cargos e regiões, permitindo à equipe de Planejamento Estratégico simular cenários e propor uma nova estrutura organizacional baseada na distribuição geográfica da empresa:



Resultado da Consulta

Todas as Linhas Extraídas: 76 em 0,011 segundos

	JOB_TITLE	REGION_NAME
1	President	Europe
2	Administration Vice President	Europe
3	Administration Assistant	Europe
4	Finance Manager	Europe
5	Accountant	Europe
6	Accounting Manager	Europe
7	Public Accountant	Europe
8	Sales Manager	Europe
9	Sales Representative	Europe
10	Purchasing Manager	Europe
11	Purchasing Clerk	Europe
12	Stock Manager	Europe
13	Stock Clerk	Europe
14	Shipping Clerk	Europe
15	Programmer	Europe
16	Marketing Manage	Europe
17	Marketing Repres	Europe
18	Human Resources	Europe
19	Public Relations	Europe
20	President	Americas
21	Administration Vice President	Americas
22	Administration Assistant	Americas
23	Finance Manager	Americas
24	Accountant	Americas
25	Accounting Manage	Americas
26	Public Accountant	Americas
27	Sales Manager	Americas
28	Sales Representat	Americas
29	Purchasing Manage	Americas
39	President	Asia
40	Administration Vice President	Asia
41	Administration Assistant	Asia
42	Finance Manager	Asia
	untant	Asia
	unting Manager	Asia
	ic Accountant	Asia
	s Manager	Asia
	s Representative	Asia
	hasing Manager	Asia
	hasing Clerk	Asia
	k Manager	Asia
	k Clerk	Asia
	ping Clerk	Asia
	rammer	Asia
	eting Manager	Asia
	eting Representative	Asia
	n Resources Representative	Asia
	ic Relations Representative	Asia
58	President	Middle East and Africa
59	Administration Vice President	Middle East and Africa
60	Administration Assistant	Middle East and Africa
61	Finance Manager	Middle East and Africa
62	Accountant	Middle East and Africa
63	Accounting Manager	Middle East and Africa
64	Public Accountant	Middle East and Africa
65	Sales Manager	Middle East and Africa
66	Sales Representative	Middle East and Africa
67	Purchasing Manager	Middle East and Africa
68	Purchasing Clerk	Middle East and Africa
69	Stock Manager	Middle East and Africa
70	Stock Clerk	Middle East and Africa
71	Shipping Clerk	Middle East and Africa
72	Programmer	Middle East and Africa
73	Marketing Manager	Middle East and Africa
74	Marketing Representative	Middle East and Africa
75	Human Resources Representative	Middle East and Africa
76	Public Relations Representative	Middle East and Africa

GABARITO EXPLICATIVO

5

Questão 5

O time de Recursos Humanos está revendo a política de remuneração da empresa. Para identificar possíveis candidatos a promoções salariais, eles pediram um relatório com a margem de aumento de cada funcionário — ou seja, quanto ainda pode ser acrescido ao salário atual sem ultrapassar o máximo previsto para o cargo.

Crie uma consulta com um INNER JOIN que retorne o nome completo do funcionário, o ID de seu cargo, seu salário atual, o teto salarial permitido para seu cargo e sua margem para aumento. Considere como margem para aumento a diferença entre o teto salarial do cargo e o atual salário do funcionário.

(Utilize as tabelas employees e jobs)

Explicação e Resultado

Para criar uma consulta que retorne o nome completo do funcionário, o ID de seu cargo, seu salário atual, o teto salarial de seu cargo e a margem que ainda pode ser aumentada, seguimos os passos abaixo:

- ✓ Selecionamos o nome completo do funcionário usando a concatenação de **e.first_name || ' ' || e.last_name**, com o alias **funcionario**.
- ✓ Selecionamos o ID do cargo (**job_id**), o salário atual (**salary**) e o teto salarial (**max_salary**) do cargo a partir das tabelas **employees** e **jobs**.
- ✓ Calculamos a margem para aumento como a **diferença** entre o teto salarial (**max_salary**) e o salário atual (**salary**), com o alias **margem_para_aumento**.
- ✓ Utilizamos um **INNER JOIN** entre as tabelas **employees** e **jobs**, vinculando pelo campo **job_id**, para trazer os dados de ambos os lados:

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

5

Questão 5

O time de Recursos Humanos está revendo a política de remuneração da empresa. Para identificar possíveis candidatos a promoções salariais, eles pediram um relatório com a margem de aumento de cada funcionário — ou seja, quanto ainda pode ser acrescido ao salário atual sem ultrapassar o máximo previsto para o cargo.

Crie uma consulta com um INNER JOIN que retorne o nome completo do funcionário, o ID de seu cargo, seu salário atual, o teto salarial permitido para seu cargo e sua margem para aumento. Considere como margem para aumento a diferença entre o teto salarial do cargo e o atual salário do funcionário.

(Utilize as tabelas employees e jobs)

Explicação e Resultado

```
SELECT
    e.first_name || ' ' || e.last_name AS funcionario,
    e.job_id,
    e.salary,
    j.max_salary,
    j.max_salary - e.salary AS margem_para_aumento
FROM employees e
INNER JOIN jobs j
    ON e.job_id = j.job_id;
```

Dessa forma, geramos um relatório útil para o time de Recursos Humanos avaliar quem está mais próximo do teto salarial do cargo (ou até mesmo já o atingiu) e quem ainda possui margem para promoções ou reajustes:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,019 segundos

	FUNCIONARIO	JOB_ID	SALARY	MAX_SALARY	MARGEM_PARA_AUMENTO
1	Steven King	AD_PRES	24000	40000	16000
2	Neena Kochhar	AD_VP	17000	30000	13000
3	Lex De Haan	AD_VP	17000	30000	13000
4	Alexander Hunold	IT_PROG	9000	10000	1000
5	Bruce Ernst	IT_PROG	6000	10000	4000
6	David Austin	IT_PROG	4800	10000	5200
7	Valli Pataballa	IT_PROG	4800	10000	5200
8	Diana Lorentz	IT_PROG	4200	10000	5800
9	Nancy Greenberg	FI_MGR	12000	16000	4000
10	Daniel Faviet	FI_ACCOUNT	9000	9000	0

GABARITO EXPLICATIVO

6

Questão 6

A diretoria solicitou a criação de um painel com a cobertura geográfica completa da empresa, que inclua tanto as regiões onde atua quanto os países cadastrados no sistema. Essa informação será usada em apresentações institucionais e materiais para investidores, sendo necessário consolidar os dados em uma única visualização simples.

Crie uma consulta utilizando UNION ALL para juntar os nomes das regiões e dos países em uma única coluna, com um campo adicional que indique de qual tipo de entidade se trata (“Região” ou “País”).

(Utilize as tabelas regions e countries)

Explicação e Resultado

Para criar uma consulta que exiba, em uma única lista, os nomes das regiões e dos países cadastrados, junto com a informação do tipo de entidade, seguimos os passos abaixo:

- ✓ 1º SELECT: Selecionamos os nomes das regiões (**region_name**) da tabela **regions**, usando o alias **nome**, e adicionamos uma coluna literal com o valor **'Região'** para identificar o tipo de entidade, com o alias **tipo**.

- ✓ Utilizamos a cláusula **UNION ALL** para unir esse resultado com outra consulta.

- ✓ 2º SELECT: Selecionamos os nomes dos países (**country_name**) da tabela **countries**, também usando o alias **nome**, e uma coluna literal com o valor **'País'** como **tipo**:

```
SELECT
    region_name AS nome,
    'Região' AS tipo
FROM regions

UNION ALL

SELECT
    country_name AS nome,
    'País' AS tipo
FROM countries;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

A diretoria solicitou a criação de um painel com a cobertura geográfica completa da empresa, que inclua tanto as regiões onde atua quanto os países cadastrados no sistema. Essa informação será usada em apresentações institucionais e materiais para investidores, sendo necessário consolidar os dados em uma única visualização simples.

Crie uma consulta utilizando UNION ALL para juntar os nomes das regiões e dos países em uma única coluna, com um campo adicional que indique de qual tipo de entidade se trata (“Região” ou “País”).

(Utilize as tabelas regions e countries)

Explicação e Resultado

Dessa forma, criamos uma visualização única e consolidada com todas as regiões e países em que a empresa possui cadastro, facilitando a construção de painéis e apresentações para investidores:

Resultado da Consulta | SQL | Todas as Linhas Extraídas: 29 em

	NOME	TIPO
1	Europe	Região
2	Americas	Região
3	Asia	Região
4	Middle East and Africa	Região
5	Argentina	País
6	Australia	País
7	Belgium	País
8	Brazil	País
9	Canada	País
10	Switzerland	País
11	China	País
12	Germany	País
13	Denmark	País
14	Egypt	País
15	France	País
16	HongKong	País
17	Israel	País
18	India	País
19	Italy	País
20	Japan	País
21	Kuwait	País
22	Mexico	País
23	Nigeria	País
24	Netherlands	País
25	Singapore	País
26	United Kingdom	País
27	United States of America	País
28	Zambia	País
29	Zimbabwe	País

GABARITO EXPLICATIVO

7

Questão 7

A equipe de TI está realizando uma auditoria nos dados de infraestrutura para verificar a ocupação dos espaços da empresa. Uma das etapas dessa análise consiste em listar todas as localidades cadastradas que ainda não foram associadas a nenhum departamento.

Essas localidades poderão ser utilizadas futuramente para alocação de novos times ou projetos.

Crie uma consulta com o operador MINUS que liste os IDs das localidades que não estão associadas a nenhum departamento.

(Utilize as tabelas locations e departments)

Explicação e Resultado

Para criar uma consulta que retorne os IDs das localidades cadastradas no sistema, mas que ainda não estão associadas a nenhum departamento, seguimos os passos abaixo:

- ✓ **1º SELECT:** Selecionamos os **location_id** da tabela **locations**, ou seja, todas as localidades cadastradas.
- ✓ Usamos o operador **MINUS**, que retorna os valores da primeira consulta que não aparecem na segunda.
- ✓ **2º SELECT:** Selecionamos os **location_id** da tabela **departments**, ou seja, as localidades que já estão em uso por algum departamento:

```
SELECT
    location_id
FROM locations

MINUS

SELECT
    location_id
FROM departments;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A equipe de TI está realizando uma auditoria nos dados de infraestrutura para verificar a ocupação dos espaços da empresa. Uma das etapas dessa análise consiste em listar todas as localidades cadastradas que ainda não foram associadas a nenhum departamento.

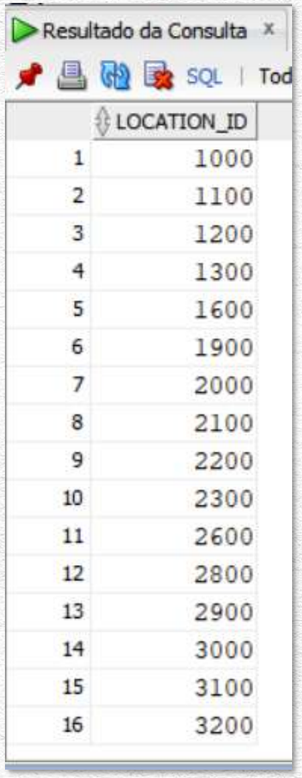
Essas localidades poderão ser utilizadas futuramente para alocação de novos times ou projetos.

Crie uma consulta com o operador MINUS que liste os IDs das localidades que não estão associadas a nenhum departamento.

(Utilize as tabelas locations e departments)

Explicação e Resultado

Dessa forma, obtemos um relatório com os IDs das localidades disponíveis para expansão, ajudando a equipe de TI a identificar espaços livres para novas alocações:


Resultado da Consulta

	LOCATION_ID
1	1000
2	1100
3	1200
4	1300
5	1600
6	1900
7	2000
8	2100
9	2200
10	2300
11	2600
12	2800
13	2900
14	3000
15	3100
16	3200

GABARITO EXPLICATIVO

8

Questão 8

A equipe de Recursos Humanos está preparando um relatório sobre a trajetória dos funcionários dentro da empresa, com foco em mobilidade interna. O objetivo é identificar os profissionais que já passaram por pelo menos uma movimentação de cargo desde que foram contratados.

Para isso, foi solicitado um levantamento com os employee_id que aparecem tanto na tabela de funcionários atuais quanto na tabela de histórico de cargos. Esses dados ajudarão a mapear padrões de promoção e a desenvolver programas de carreira.

Crie uma consulta com o operador INTERSECT que identifique os IDs de funcionários que aparecem nas duas tabelas.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

Para identificar os funcionários que já passaram por ao menos uma movimentação de cargo, ou seja, que aparecem tanto na tabela de funcionários atuais (**employees**) quanto na tabela de histórico de cargos (**job_history**), seguimos os passos abaixo:

1º SELECT: Selecionamos os **employee_id** da tabela **employees**, que representa os funcionários em seus cargos atuais na empresa.

Utilizamos o operador **INTERSECT**, que retorna apenas os valores que estão presentes nas duas consultas.

2º SELECT: Selecionamos os **employee_id** da tabela **job_history**, que representa os funcionários que tiveram algum histórico de movimentação de cargo:

```
SELECT
    employee_id
FROM employees

INTERSECT

SELECT
    employee_id
FROM job_history;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A equipe de Recursos Humanos está preparando um relatório sobre a trajetória dos funcionários dentro da empresa, com foco em mobilidade interna. O objetivo é identificar os profissionais que já passaram por pelo menos uma movimentação de cargo desde que foram contratados.

Para isso, foi solicitado um levantamento com os employee_id que aparecem tanto na tabela de funcionários atuais quanto na tabela de histórico de cargos. Esses dados ajudarão a mapear padrões de promoção e a desenvolver programas de carreira.

Crie uma consulta com o operador INTERSECT que identifique os IDs de funcionários que aparecem nas duas tabelas.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

Dessa forma, obtemos a lista de IDs dos funcionários que já passaram por uma mudança de cargo, o que permite ao RH acompanhar casos de mobilidade interna e traçar planos de carreira mais efetivos:

Resultado da Consulta

SQL

	EMPLOYEE_ID
1	101
2	102
3	114
4	122
5	176
6	200
7	201

MÓDULO 10

C R U D

C R U D

C R U D

O QUE VEREMOS NESTE MÓDULO

MÓDULO 10 CRUD (Comandos DDL, DML e TCL)

01	CRUD (COMANDO DDL, DML e TCL)	436
02	TIPOS DE DADOS	437
03	COMANDOS DDL	438
04	COMANDOS DDL: CREATE	439
05	COMANDOS DDL: ALTER (ADICIONANDO COLUNAS)	440
06	COMANDOS DDL: ALTER (MODIFICANDO COLUNAS)	441
07	COMANDOS DDL: ALTER (RENOMEANDO E EXCLUINDO COLUNAS)	442
08	COMANDOS DDL: TRUNCATE	443
09	COMANDOS DDL: DROP	444
10	COMANDOS DDL: COMMENT (TABELA)	445
11	COMANDOS DDL: COMMENT (COLUNA)	446
12	COMANDOS DML	447
13	COMANDOS DML: INSERT INTO	448
14	COMANDOS DML: UPDATE	449
15	COMANDOS DML: DELETE	450
16	COMANDOS TCL	451
17	COMANDOS TCL: COMMIT	452
18	COMANDOS TCL: ROLLBACK	453
19	COMANDOS TCL: SAVEPOINT	454
20	RESUMO DO MÓDULO	456
21	EXERCÍCIOS	457
22	GABARITO EXPLICATIVO	462

CRUD (Comandos DDL, DML e TCL)

CRUD é um nome dado às operações realizadas dentro de um banco de dados. É a sigla para: CREATE, READ, UPDATE e DELETE. De forma mais geral, podemos dividir os comandos dentro de um banco de dados em alguns grupos:

DDL

Data Definition Language

São comandos para definição de objetos em um banco de dados.

CREATE
Cria uma nova tabela, view ou outro objeto no banco de dados.

ALTER
Modifica um objeto no banco de dados (tabela, view, etc).

TRUNCATE
Exclui os registros de um objeto, mantendo o objeto no banco de dados.

DROP
Exclui um objeto do banco de dados (tabela, view, etc).

COMMENT
Cria comentários para objetos do banco de dados.

DML

Data Manipulation Language

São comandos para manipulação de dados em um banco de dados.

INSERT INTO
Adiciona uma nova linha em uma tabela.

UPDATE
Atualiza os valores das linhas de uma tabela.

DELETE
Exclui linhas de uma tabela.

TCL

Transaction Control Language

São comandos para controle de transações DML em um banco de dados.

COMMIT
Salva as alterações no banco.

ROLLBACK
Desfaz alterações no banco.

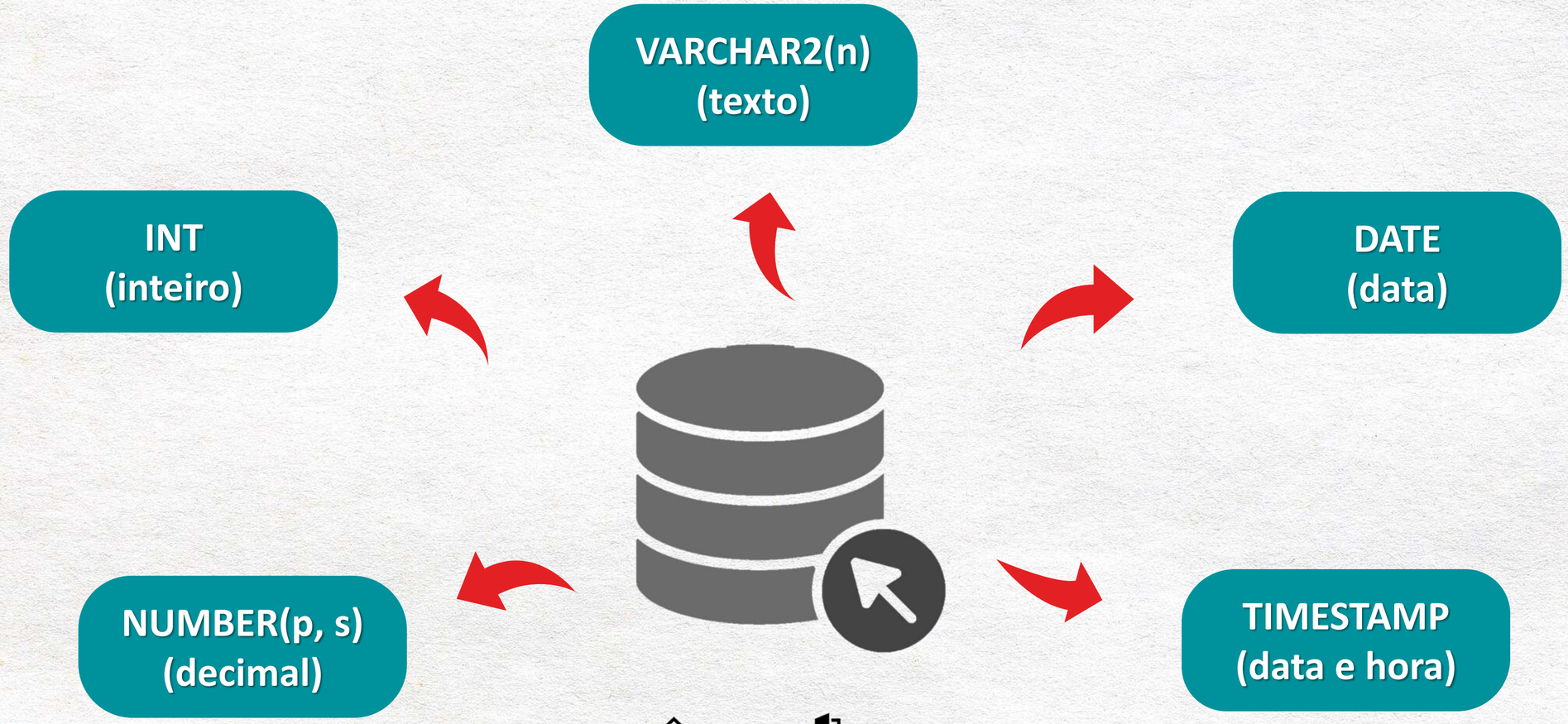
SAVEPOINT
Define um ponto de salvamento na transação.

Índice Geral

Índice deste Módulo

Tipos de Dados

Os dados armazenados dentro de um banco de dados poderão ser de uma série de tipos. Aqui, estão os mais comuns:



Comandos DDL:

Os comandos DDL (Data Definition Language) servem para definir os objetos de um banco de dados.
São eles: CREATE, ALTER, TRUNCATE, DROP e COMMENT. Veremos agora como funciona cada um deles.

DDL

Data Definition Language

CREATE
Cria uma nova tabela, view ou outro objeto no banco de dados.

ALTER
Modifica um objeto no banco de dados (tabela, view, etc).

TRUNCATE
Exclui os registros de um objeto, mantendo o objeto no banco de dados.

DROP
Exclui um objeto do banco de dados (tabela, view, etc).

COMMENT
Cria comentários para objetos do banco de dados.

Comandos DDL: CREATE

O comando DDL CREATE serve para criar novos objetos (tabelas, views, etc.) em um banco de dados.

SINTAXE:

```
CREATE TABLE nome_da_tabela(
        nome_da_coluna_1 tipo_de_dado,
        nome_da_coluna_2 tipo_de_dado,
        nome_da_coluna_3 tipo_de_dado,
        nome_da_coluna_n tipo_de_dado);
```

```
-- A tabela CLIENTES deve conter as seguintes colunas:

-- Coluna 1: id_cliente do tipo INT
-- Coluna 2: nome_cliente do tipo VARCHAR2
-- Coluna 3: sexo do tipo VARCHAR2
-- Coluna 4: email do tipo VARCHAR2
-- Coluna 5: data_nascimento do tipo DATE
-- Coluna 6: cpf do tipo VARCHAR2

CREATE TABLE clientes(
    id_cliente INT,
    nome_cliente VARCHAR2(100),
    sexo VARCHAR2(100),
    email VARCHAR2(100),
    data_nascimento DATE,
    cpf VARCHAR2(100)
);
```

Comandos DDL: ALTER (adicionando colunas)

O comando DDL ALTER permite que a gente faça alterações em nossas tabelas, tais como: adicionar, modificar, renomear ou excluir colunas.

Veja como adicionamos uma coluna:

```
ALTER TABLE nome_da_tabela
        ADD nome_da_nova_coluna tipo_de_dado;
```

```
-- Adicionar uma coluna de telefone na tabela CLIENTES
ALTER TABLE clientes
    ADD telefone VARCHAR2(100);
```

Para adicionar mais de uma coluna ao mesmo tempo, fazemos assim:

```
ALTER TABLE nome_da_tabela
        ADD (nome_da_1ª_nova_coluna tipo_de_dado,
             nome_da_2ª_nova_coluna tipo_de_dado,
             nome_da_nª_nova_coluna tipo_de_dado);
```

```
-- Adicionar mais de uma coluna
ALTER TABLE clientes
    ADD (endereco VARCHAR2(100),
         salario NUMBER(10, 2),
         qtd_filhos INT);
```

Comandos DDL: ALTER (modificando colunas)

O comando DDL ALTER permite que a gente faça alterações em nossas tabelas, tais como: adicionar, modificar, renomear ou excluir colunas.

Veja como alterar o tipo de dado de uma coluna:

```
ALTER TABLE nome_da_tabela
        MODIFY nome_da_coluna novo_tipo_de_dado;
```

```
-- Modificar uma coluna
ALTER TABLE clientes
    MODIFY salario INT;
```

Para alterar o tipo de dado de mais de uma coluna ao mesmo tempo, fazemos assim:

```
ALTER TABLE nome_da_tabela
        MODIFY
                (nome_da_coluna_1 novo_tipo_de_dado,
                 nome_da_coluna_2 novo_tipo_de_dado,
                 nome_da_coluna_n novo_tipo_de_dado);
```

```
-- Modificar mais de uma coluna
ALTER TABLE clientes
    MODIFY
        (endereco INT,
        salario INT,
        qtd_filhos DATE);
```

Comandos DDL: ALTER (renomeando e excluindo colunas)

O comando DDL ALTER permite que a gente faça alterações em nossas tabelas, tais como: adicionar, modificar, renomear ou excluir colunas.

Veja como renomear uma coluna:

```
ALTER TABLE nome_da_tabela
        RENAME COLUMN nome_atual_da_coluna TO novo_nome_da_coluna;
```

```
-- 3. Como renomear uma coluna
ALTER TABLE clientes
    RENAME COLUMN sexo TO genero;
```

Por fim, veja como fazemos para excluir uma coluna de uma tabela:

```
ALTER TABLE nome_da_tabela
        DROP COLUMN nome_da_coluna;
```

```
-- 4. Como excluir uma coluna
ALTER TABLE clientes
    DROP COLUMN endereco;
```

Comandos DDL: TRUNCATE

O comando DDL TRUNCATE exclui todos os registros de um objeto (tabela, view, etc.), mas mantém o objeto (vazio) no banco de dados.

SINTAXE:

TRUNCATE TABLE nome_da_tabela;

```
/* Exclua todas as linhas da tabela VENDAS,
mas mantenha a tabela no banco de dados. */

TRUNCATE TABLE vendas;
```

Comandos DDL: DROP

O comando DDL DROP exclui o objeto (tabela, view, etc.) do banco de dados.

SINTAXE:

DROP TABLE nome_da_tabela;

```
-- Exclua a tabela VENDAS do banco de dados.

DROP TABLE vendas;
```

Comandos DDL: COMMENT (tabela)

O comando DDL COMMENT cria comentários para objetos do banco de dados. Podemos utilizá-lo para criar comentários tanto nas tabelas, quanto em suas colunas.

Vejamos, primeiro, como criar, visualizar e excluir um comentário em uma TABELA:

SINTAXE:

1 Criando um comentário em uma tabela:

```
COMMENT ON TABLE nome_da_tabela
IS ‘Comentario’;
```

2 Visualizando um comentário de uma tabela:

```
SELECT * FROM USER_TAB_COMMENTS;
```

3 Excluindo um comentário de uma tabela:

```
COMMENT ON TABLE nome_da_tabela
IS ‘’;
```

1

```
/* 1) Na tabela produtos, crie o seguinte
comentário: 'Informações de Produtos'. */

COMMENT ON TABLE produtos
IS 'Informação de Produtos';
```

2

```
SELECT * FROM USER_TAB_COMMENTS;
```

TABLE_NAME	TABLE_TYPE	COMMENTS	ORIGIN_CON_
CLIENTES	TABLE	(null)	3
PRODUTOS	TABLE	Informação de Produtos	3
VENDAS	TABLE	(null)	3

3

```
-- Exclui um comentário na tabela
COMMENT ON TABLE produtos
IS '';
```

Índice Geral

Índice deste Módulo

Comandos DDL: COMMENT (coluna)

O comando DDL COMMENT cria comentários para objetos do banco de dados. Podemos utilizá-lo para criar comentários tanto nas tabelas, quanto em suas colunas.

Agora, vejamos como criar, visualizar e excluir um comentário em uma COLUNA:

SINTAXE:

1 Criando um comentário em uma coluna:

```
COMMENT ON COLUMN nome_da_tabela.nome_da_coluna
IS ‘Comentario’;
```

2 Visualizando um comentário de uma coluna:

```
SELECT * FROM USER_COL_COMMENTS
WHERE table_name = ‘nome_da_tabela’;
```

3 Excluindo um comentário de uma coluna:

```
COMMENT ON COLUMN nome_da_tabela.nome_da_coluna
IS ‘’;
```

1

```
/* 2) Crie um comentário na coluna CATEGORIA da
tabela PRODUTOS: 'Descreve a categoria do produto'. */

COMMENT ON COLUMN produtos.categoria
IS 'Descreve a categoria do produto';
```

2

```
SELECT * FROM USER_COL_COMMENTS
WHERE table_name = 'PRODUTOS';
```

TABLE_NAME	COLUMN_NAME	COMMENTS	ORIGIN_CON_
PRODUTOS	MARCA	(null)	3
PRODUTOS	CATEGORIA	Descreve a categoria do produto	3
PRODUTOS	PRECO_UNIT	(null)	3

3

```
-- Exclui um comentário na coluna
COMMENT ON COLUMN produtos.categoria
IS '';
```

Índice Geral

Índice deste Módulo

Comandos DML:

Os comandos DML (Data Manipulation Language) servem para manipular os objetos de um banco de dados.
São eles: INSERT INTO, UPDATE e DELETE. Veremos agora como funciona cada um deles.

DML

Data Manipulation Language

INSERT INTO
Adiciona uma nova linha em uma tabela.

UPDATE
Atualiza os valores das linhas de uma tabela.

DELETE
Exclui linhas de uma tabela.

Comandos DML: INSERT INTO

O comando DML INSERT INTO adiciona uma nova linha em uma tabela.

SINTAXE:

INSERT INTO nome_da_tabela(nome_da_coluna_1, nome_da_coluna_2, nome_da_coluna_n) **VALUES**
(dado_da_coluna_1, dado_da_coluna_2, dado_da_coluna_n);

```
-- Preencha a tabela CLIENTES

INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
    (1,  'André Martins',   'M', 'andre.m@gmail.com', '12/02/1989', '839.283.190-00');
INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
    (2,  'Bárbara Campos',  'F', 'barb_campos@hotmail.com', '07/05/1992', '351.391.410-02');
INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
    (3,  'Carol Freitas',   'F', 'carol@gmail.com', '23/04/1985', '139.274.921-12');
```

Comandos DML: UPDATE

O comando DML UPDATE atualiza os valores das linhas de uma tabela.

SINTAXE:

UPDATE nome_da_tabela
SET nome_da_coluna = novo_valor
WHERE nome_da_coluna = valor_atual

```
-- Atualize o preço unitário do produto de ID = 1 para R$3900.

UPDATE produtos
SET preco_unit = 3900
WHERE id_produto = 1;
```

Comandos DML: DELETE

O comando DML DELETE exclui linhas de uma tabela.

SINTAXE:

```
DELETE FROM nome_da_tabela
WHERE nome_da_coluna = valor_da_coluna
```

Obs.: com o WHERE podem ser usados quaisquer operadores lógicos que melhor atendam sua necessidade: =, <, >, <=, >=, <>, AND, OR, IN, BETWEEN, etc.

```
-- Exclua da tabela CLIENTES o cliente de ID = 10.

DELETE FROM clientes
WHERE id_cliente = 10;
```

Comandos TCL:

Os comandos TCL (Transaction Control Language) servem para controlar as transações DML efetuadas em um banco de dados. São eles: COMMIT, ROLLBACK e SAVEPOINT. Veremos agora como funciona cada um deles.

TCL

Transaction Control Language

COMMIT
Salva as alterações no banco.

ROLLBACK
Desfaz alterações no banco.

SAVEPOINT
Define um ponto de salvamento na transação.

Comandos TCL: COMMIT

O comando TCL COMMIT salva as alterações feitas por um comando DML (INSERT INTO, UPDATE, DELETE) no banco de dados.

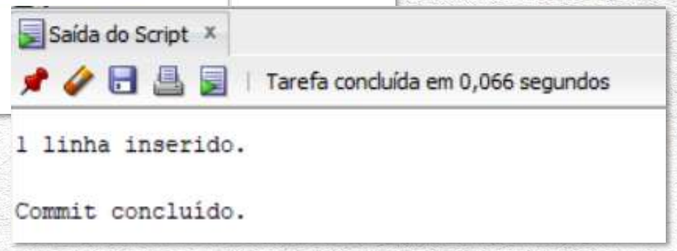
Quando manipulamos os dados constantes em nosso banco de dados por meio de um comando DML, as alterações efetuadas ficam registradas no SGBD do nosso computador.

Para que essas alterações fiquem visíveis para os demais usuários, temos que salvá-la com um COMMIT.

Para isso, basta inserir o comando DML e acrescentar o COMMIT na linha seguinte do código.

No exemplo abaixo, inserimos o cliente Leonardo à tabela de clientes, fazendo uso do comando TCL COMMIT:

```
INSERT INTO CLIENTES(ID_CLIENTE, NOME_CLIENTE, SEXO, EMAIL, DATA_NASCIMENTO, CPF) VALUES
    (20,  'Leonardo', 'M', 'leo@hotmail.com', '07/05/1992', '928.285.129-12');

COMMIT;
```



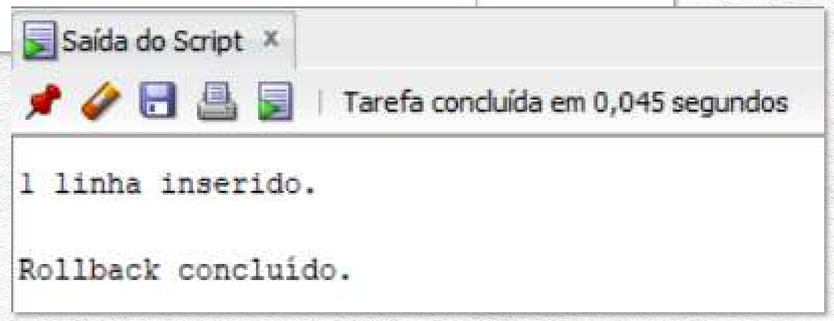
Comandos TCL: ROLLBACK

O comando TCL ROLLBACK desfaz as alterações feitas por um comando DML (INSERT INTO, UPDATE, DELETE) no banco de dados.

Para isso, basta, após executada a operação com o comando DML, utilizarmos o comando ROLLBACK.

No exemplo abaixo, inserimos a cliente Maria à tabela de clientes, e, em seguida, desfizemos essa inserção:

```
INSERT INTO CLIENTES(ID_CLIENTE, NOME_CLIENTE, SEXO, EMAIL, DATA_NASCIMENTO, CPF) VALUES
    (21,  'Maria', 'F', 'maria@hotmail.com', '17/08/1994', '111.285.129-12');

ROLLBACK;
```



Obs.: Não é possível desfazer uma operação por meio de um ROLLBACK após o COMMIT já ter sido efetuado.

Comandos TCL: SAVEPOINT

O comando TCL SAVEPOINT define um ponto de salvamento na transação.

Assim, caso seja necessário desfazer uma alteração efetuada (mas ainda não “commitada”), podemos, junto ao ROLLBACK, informar o nome dado ao SAVEPOINT criado.

Dessa forma, todas as alterações efetuadas após aquele SAVEPOINT serão desfeitas, preservando-se as efetuadas anteriormente ao SAVEPOINT.

No exemplo ao lado, inserimos os clientes Bárbara, Carol e Diego à tabela de clientes, definindo um SAVEPOINT após cada inserção, dando os nomes de P1, P2 e P3 a eles:

```
INSERT INTO CLIENTES(ID_CLIENTE, NOME_CLIENTE, SEXO, EMAIL, DATA_NASCIMENTO) VALUES
    (2,  'Bárbara Campos',  'F', 'barb_campos@hotmail.com', '07/05/1992');

SAVEPOINT P1;

INSERT INTO CLIENTES(ID_CLIENTE, NOME_CLIENTE, SEXO, EMAIL, DATA_NASCIMENTO) VALUES
    (3,  'Carol Freitas',   'F', 'carol@gmail.com', '23/04/1985');

SAVEPOINT P2;

INSERT INTO CLIENTES(ID_CLIENTE, NOME_CLIENTE, SEXO, EMAIL, DATA_NASCIMENTO) VALUES
    (4,  'Diego Cardoso', 'M', 'dcardoso@gmail.com', '11/10/1994');

SAVEPOINT P3;
```

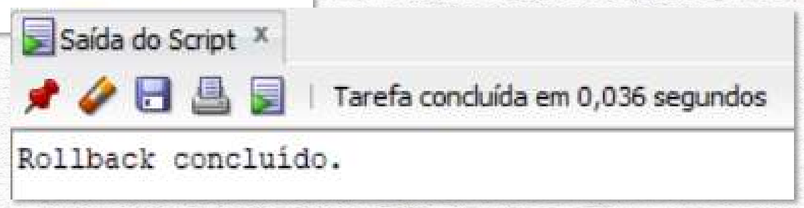
Comandos TCL: SAVEPOINT

Por algum motivo, precisamos desfazer a inclusão da Carol e do Diego na tabela Clientes.

Para isso, utilizamos o comando “ROLLBACK TO SAVEPOINT P1”

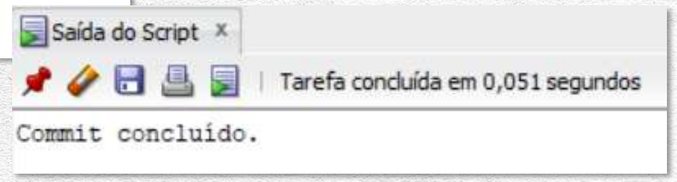
E assim, a inserção da Carol e do Diego foram desfeitas, preservando somente a inclusão da Bárbara, cujo comando estava antes do SAVEPOINT P1:

```
ROLLBACK TO SAVEPOINT P1;
```



Agora, para salvar de fato a alteração efetuada antes do SAVEPOINT P1 (inclusão da Bárbara à tabela Clientes), basta dar um COMMIT:

```
COMMIT;
```

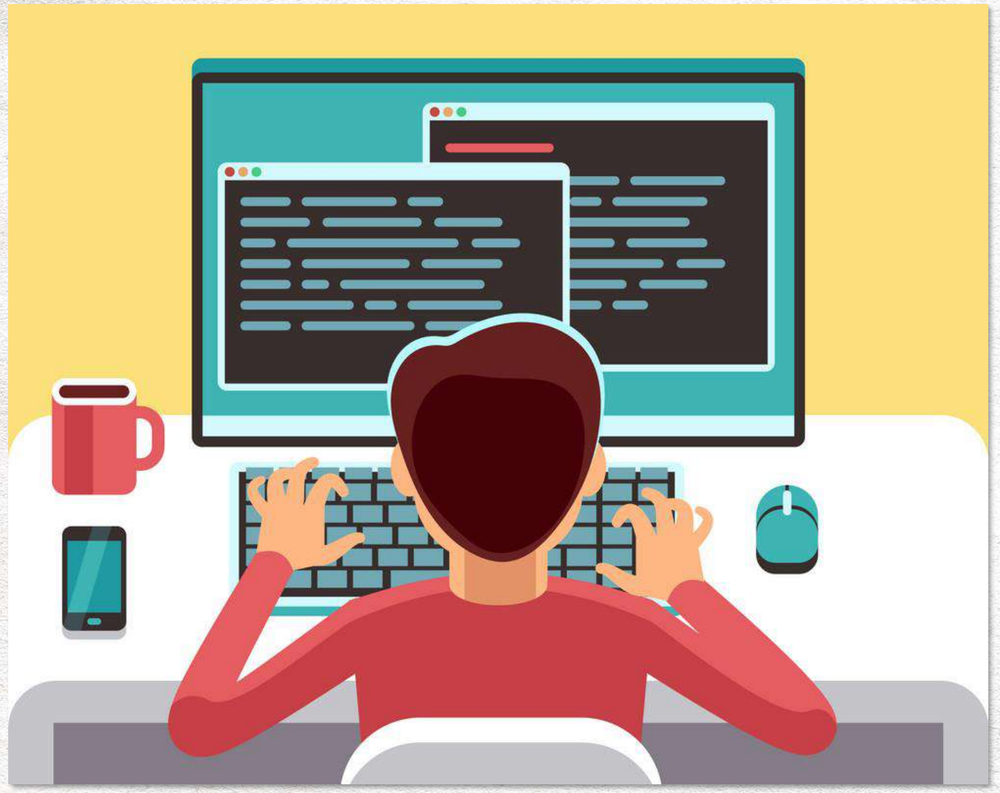


Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. Os comandos SQL podem ser divididos em grupos. Exemplos: DDL, DML e TCL.
2. CRUD é a sigla para CREATE, READ, UPDATE e DELETE, operações básicas em bancos de dados.
3. Podemos criar comentários em tabelas e colunas usando o comando COMMENT.
4. Podemos controlar transações DML (INSERT INTO, UPDATE e DELETE) através dos comandos TCL: COMMIT, ROLLBACK e SAVEPOINT.
5. Podemos alterar tabelas usando o ALTER TABLE.

EXERCÍCIOS



1

Questão 1

O departamento de Recursos Humanos da empresa decidiu criar um controle específico dos benefícios oferecidos aos colaboradores, como vale-alimentação, plano de saúde, entre outros.

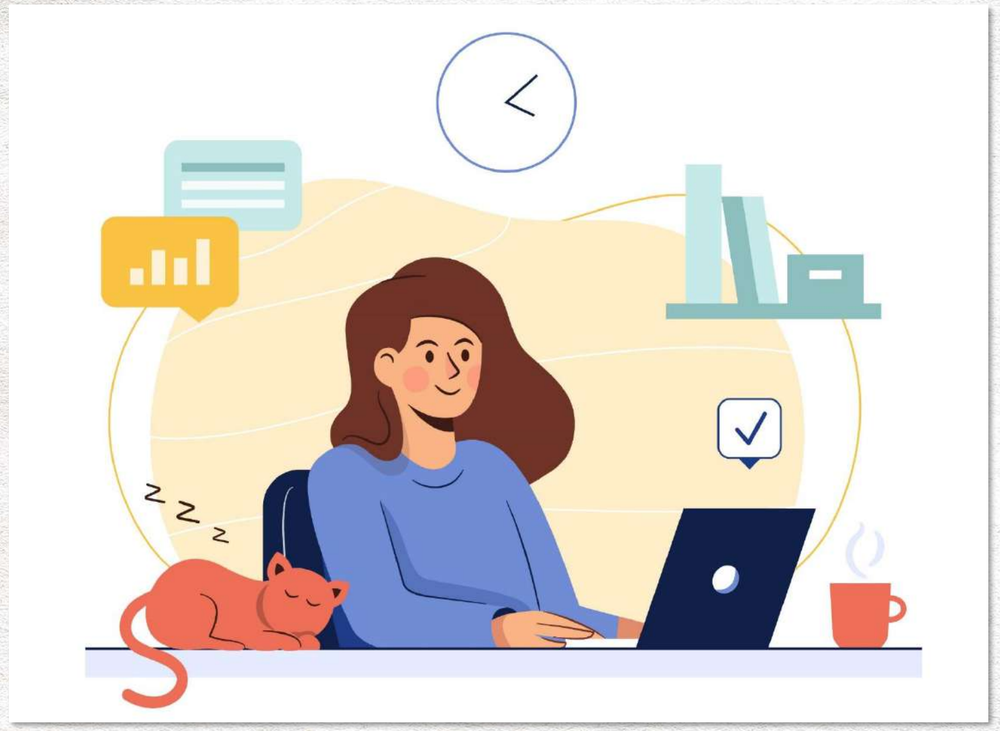
Essa nova tabela armazenará os tipos de benefícios e o período em que foram disponibilizados a cada funcionário.

Crie a tabela BENEFITS no banco de dados RH com as colunas necessárias para armazenar o ID de benefício, o ID do funcionário, o tipo de benefício e sua descrição, além das datas de início e fim.

Siga as instruções abaixo:

- ✓ A coluna do ID do benefício deverá se chamar benefit_id para receber números inteiros;
- ✓ A coluna do ID do funcionário deverá se chamar employee_id para receber números inteiros;
- ✓ A coluna do tipo de benefício deverá se chamar benefit_type para receber textos de até 30 caracteres;
- ✓ A coluna da descrição do benefício deverá se chamar benefit_description para receber textos de até 50 caracteres;
- ✓ A coluna da data de início do benefício deverá se chamar start_date para receber uma data;
- ✓ A coluna da data final do benefício deverá se chamar end_date para receber uma data.

EXERCÍCIOS



2

Questão 2

O setor financeiro solicitou que o RH inclua na tabela BENEFITS o valor mensal estimado para cada benefício concedido. Essa informação será usada futuramente em relatórios contábeis.

Altere a tabela BENEFITS para adicionar uma nova coluna chamada monthly_value, do tipo NUMBER com até duas casas decimais.

3

Questão 3

O RH iniciou o preenchimento da nova tabela com os benefícios vigentes para alguns colaboradores. Após cada inserção, um ponto de controle será criado para garantir que o progresso possa ser revertido com segurança, se necessário.

Insira 5 registros na tabela BENEFITS, utilizando um SAVEPOINT após cada inserção (chame os savepoints de sp1, sp2, sp3, sp4 e sp5). Em seguida, faça um SELECT para consultar os dados inseridos.

EXERCÍCIOS

4

Questão 4

Após uma revisão, o RH identificou um erro nos últimos dois registros inseridos. Para corrigir, decidiu-se realizar um ROLLBACK até o ponto de controle anterior à inserção desses dados.

Realize um ROLLBACK até o savepoint sp3 e consulte os dados atuais da tabela.

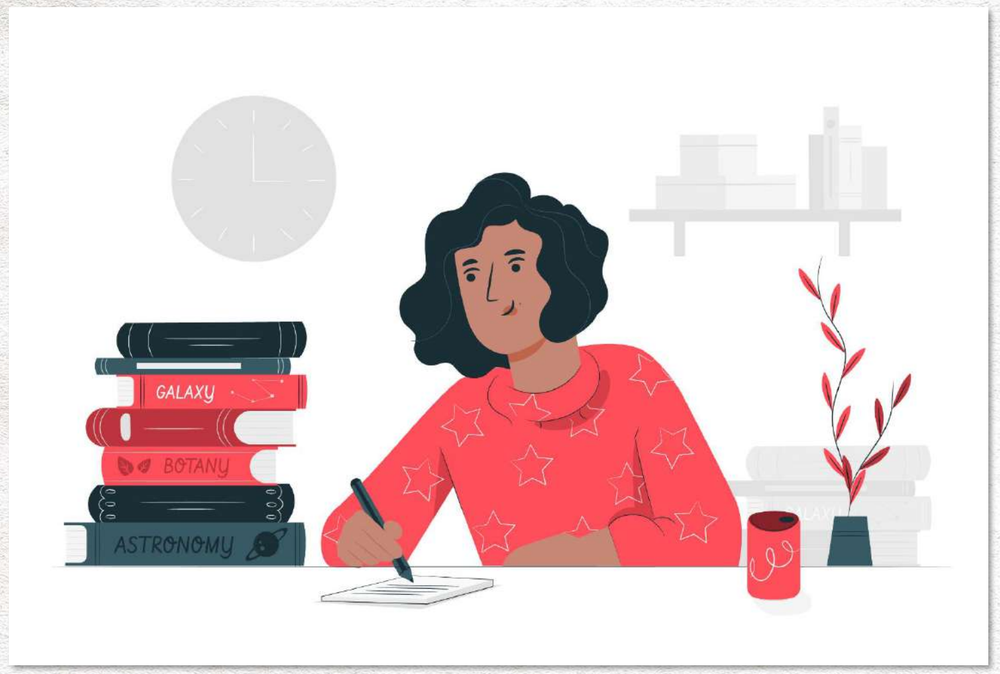
5

Questão 5

Após a verificação, o RH confirmou que os dados restantes estão corretos e decidiu consolidar essas alterações no banco de dados.

Efetive as alterações feitas até o momento utilizando o COMMIT e faça uma nova consulta na tabela.

EXERCÍCIOS



6

Questão 6

Foi identificado que o valor mensal do benefício de vale-refeição foi reajustado para R$ 500,00. A equipe de RH precisa atualizar essa informação na tabela.

Atualize o valor do benefício Vale Refeição para R$ 500,00, confirme a transação e consulte os dados atualizados.

7

Questão 7

Houve um equívoco e o colaborador de employee_id = 145 não deveria ter sido registrado com plano de saúde.

Exclua o benefício do colaborador de employee_id = 145 e consulte os dados atualizados.

8

Questão 8

Após a exclusão do benefício do employee_id = 145, percebeu-se que a informação era válida e precisava ser mantida.

Reverta a exclusão efetuada no exercício anterior com um ROLLBACK. Depois, consulte a tabela.

EXERCÍCIOS

9

Questão 9

O sistema de gestão de benefícios foi migrado para um novo módulo integrado ao ERP da empresa. O time de TI precisa limpar a tabela BENEFITS, mas mantê-la estruturada para eventual uso futuro.

Remova todos os registros da tabela BENEFITS utilizando o comando TRUNCATE e consulte a tabela.

10

Questão 10

Como a gestão de benefícios agora é realizada por um sistema terceirizado, a tabela BENEFITS não será mais utilizada no banco de dados da empresa.

Remova completamente a tabela BENEFITS do banco de dados. Em seguida, tente consultá-la para verificar a remoção.

GABARITO EXPLICATIVO

1

Questão 1

O departamento de Recursos Humanos da empresa decidiu criar um controle específico dos benefícios oferecidos aos colaboradores, como vale-alimentação, plano de saúde, entre outros. Essa nova tabela armazenará os tipos de benefícios e o período em que foram disponibilizados a cada funcionário. Crie a tabela BENEFITS no banco de dados RH com as colunas necessárias para armazenar o ID de benefício, o ID do funcionário, o tipo de benefício e sua descrição, além das datas de início e fim. Siga as instruções abaixo:

- ✓ A coluna do ID do benefício deverá se chamar benefit_id para receber números inteiros;
- ✓ A coluna do ID do funcionário deverá se chamar employee_id para receber números inteiros;
- ✓ A coluna do tipo de benefício deverá se chamar benefit_type para receber textos de até 30 caracteres;
- ✓ A coluna da descrição do benefício deverá se chamar benefit_description para receber textos de até 50 caracteres;
- ✓ A coluna da data de início do benefício deverá se chamar start_date para receber uma data;
- ✓ A coluna da data final do benefício deverá se chamar end_date para receber uma data.

Explicação e Resultado

Objetivo: Criar a tabela **BENEFITS** com as colunas necessárias.

Como o código foi desenvolvido:

```
CREATE TABLE benefits (
  benefit_id INT,
  employee_id INT,
  benefit_type VARCHAR2(30),
  benefit_description VARCHAR2(50),
  start_date DATE,
  end_date DATE
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O departamento de Recursos Humanos da empresa decidiu criar um controle específico dos benefícios oferecidos aos colaboradores, como vale-alimentação, plano de saúde, entre outros. Essa nova tabela armazenará os tipos de benefícios e o período em que foram disponibilizados a cada funcionário. Crie a tabela BENEFITS no banco de dados RH com as colunas necessárias para armazenar o ID de benefício, o ID do funcionário, o tipo de benefício e sua descrição, além das datas de início e fim. Siga as instruções abaixo:

- ✓ A coluna do ID do benefício deverá se chamar benefit_id para receber números inteiros;
- ✓ A coluna do ID do funcionário deverá se chamar employee_id para receber números inteiros;
- ✓ A coluna do tipo de benefício deverá se chamar benefit_type para receber textos de até 30 caracteres;
- ✓ A coluna da descrição do benefício deverá se chamar benefit_description para receber textos de até 50 caracteres;
- ✓ A coluna da data de início do benefício deverá se chamar start_date para receber uma data;
- ✓ A coluna da data final do benefício deverá se chamar end_date para receber uma data.

Explicação e Resultado

Explicação passo a passo:

- ✓ **CREATE TABLE benefits**: Inicia a criação de uma nova tabela chamada benefits.
- ✓ **(...)** : Os parênteses delimitam a lista de colunas e seus tipos de dados.
- ✓ **benefit_id INT**: Cria a coluna benefit_id, do tipo inteiro (INT), que armazenará o identificador único de cada benefício.
- ✓ **employee_id INT**: Cria a coluna employee_id, também do tipo inteiro, que armazenará o ID do funcionário relacionado ao benefício.
- ✓ **benefit_type VARCHAR2(30)**: Cria a coluna benefit_type, que recebe textos com até 30 caracteres para indicar o tipo do benefício.
- ✓ **benefit_description VARCHAR2(50)**: Cria a coluna benefit_description, usada para armazenar uma descrição mais detalhada do benefício (até 50 caracteres).
- ✓ **start_date DATE**: Cria a coluna start_date, que armazenará a data de início do benefício.
- ✓ **end_date DATE**: Cria a coluna end_date, que armazenará a data de término do benefício (caso exista).
- ✓ **;** : O ponto e vírgula finaliza o comando SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O departamento de Recursos Humanos da empresa decidiu criar um controle específico dos benefícios oferecidos aos colaboradores, como vale-alimentação, plano de saúde, entre outros. Essa nova tabela armazenará os tipos de benefícios e o período em que foram disponibilizados a cada funcionário. Crie a tabela BENEFITS no banco de dados RH com as colunas necessárias para armazenar o ID de benefício, o ID do funcionário, o tipo de benefício e sua descrição, além das datas de início e fim. Siga as instruções abaixo:

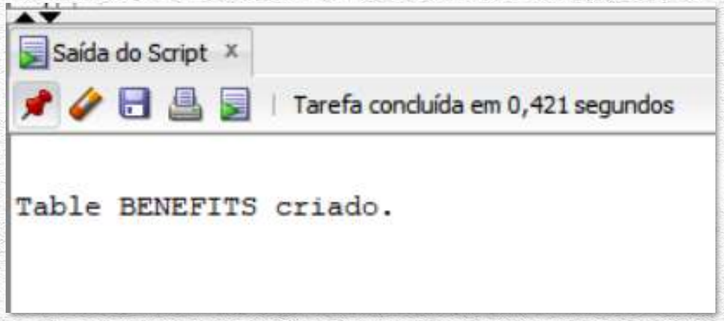
- ✓ A coluna do ID do benefício deverá se chamar benefit_id para receber números inteiros;
- ✓ A coluna do ID do funcionário deverá se chamar employee_id para receber números inteiros;
- ✓ A coluna do tipo de benefício deverá se chamar benefit_type para receber textos de até 30 caracteres;
- ✓ A coluna da descrição do benefício deverá se chamar benefit_description para receber textos de até 50 caracteres;
- ✓ A coluna da data de início do benefício deverá se chamar start_date para receber uma data;
- ✓ A coluna da data final do benefício deverá se chamar end_date para receber uma data.

Explicação e Resultado

Resultado:

A tabela **BENEFITS** será criada com a estrutura definida.

Ao executar esse comando no SQL Developer, a mensagem de sucesso será exibida:



GABARITO EXPLICATIVO

2

Questão 2

O setor financeiro solicitou que o RH inclua na tabela BENEFITS o valor mensal estimado para cada benefício concedido. Essa informação será usada futuramente em relatórios contábeis.

Altere a tabela BENEFITS para adicionar uma nova coluna chamada monthly_value, do tipo NUMBER com até duas casas decimais.

Explicação e Resultado

Objetivo: Alterar a tabela **BENEFITS** para adicionar uma nova coluna que armazenará o valor mensal do benefício.

Como o código foi desenvolvido:

```
ALTER TABLE benefits
ADD monthly_value NUMBER(10, 2);
```

Explicação passo a passo:

- ✓ **ALTER TABLE benefits**: Utilizamos o comando ALTER TABLE para modificar a estrutura da tabela existente chamada benefits.
- ✓ **ADD monthly_value NUMBER(10, 2)**: Usamos a cláusula ADD para incluir uma nova coluna chamada monthly_value. O tipo de dado NUMBER(10, 2) indica que a coluna aceitará números com até 10 dígitos no total, sendo 2 casas decimais. Isso permite armazenar valores como 500.00, 12345.67, etc., ideais para representar valores monetários.
- ✓ ; : O ponto e vírgula encerra o comando.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

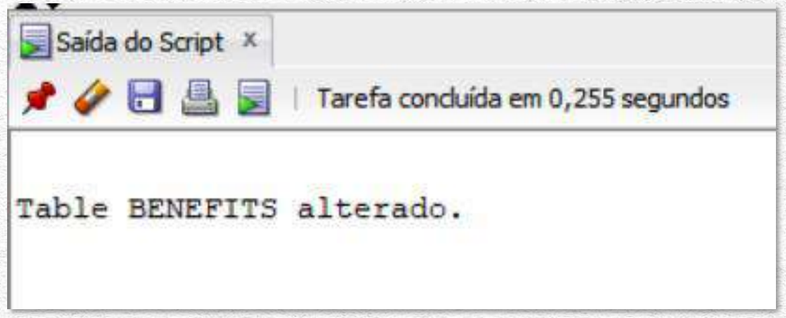
O setor financeiro solicitou que o RH inclua na tabela BENEFITS o valor mensal estimado para cada benefício concedido. Essa informação será usada futuramente em relatórios contábeis.

Altere a tabela BENEFITS para adicionar uma nova coluna chamada monthly_value, do tipo NUMBER com até duas casas decimais.

Explicação e Resultado

A tabela **BENEFITS** passa a contar com uma nova coluna chamada **monthly_value**, que poderá ser usada para armazenar o valor mensal de cada benefício concedido.

Ao executar o comando, a seguinte mensagem será exibida no SQL Developer:



GABARITO EXPLICATIVO

3

Questão 3

O RH iniciou o preenchimento da nova tabela com os benefícios vigentes para alguns colaboradores. Após cada inserção, um ponto de controle será criado para garantir que o progresso possa ser revertido com segurança, se necessário.

Insira 5 registros na tabela BENEFITS, utilizando um SAVEPOINT após cada inserção (chame os savepoints de sp1, sp2, sp3, sp4 e sp5). Em seguida, faça um SELECT para consultar os dados inseridos.

Explicação e Resultado

Objetivo: Inserir 5 registros na tabela **BENEFITS**, criando um **SAVEPOINT** após cada **INSERT**, e consultar os dados inseridos.

Como o código foi desenvolvido:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (1, 145, 'Plano de Saúde', 'Plano Ouro da Hashmed', '01/01/1997', NULL, 550.00);
SAVEPOINT sp1;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (2, 146, 'Vale Refeição', 'Cartão Hashcard', '05/03/1997', NULL, 450.00);
SAVEPOINT sp2;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (3, 147, 'Vale Transporte', 'Bilhete Único da Hashtag', '10/05/1997', NULL, 180.00);
SAVEPOINT sp3;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (4, 148, 'Auxílio Educação', 'Bolsa de Especialização Impressionadora', '15/12/1999', NULL, 800.00);
SAVEPOINT sp4;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (5, 149, 'Gympass', 'Plano HashGold', '29/03/2000', NULL, 120.00);
SAVEPOINT sp5;

SELECT * FROM benefits;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O RH iniciou o preenchimento da nova tabela com os benefícios vigentes para alguns colaboradores. Após cada inserção, um ponto de controle será criado para garantir que o progresso possa ser revertido com segurança, se necessário.

Insira 5 registros na tabela BENEFITS, utilizando um SAVEPOINT após cada inserção (chame os savepoints de sp1, sp2, sp3, sp4 e sp5). Em seguida, faça um SELECT para consultar os dados inseridos.

Explicação e Resultado

Vamos entender o primeiro **INSERT**:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (1, 145, 'Plano de Saúde', 'Plano Ouro da Hashmed', '01/01/1997', NULL, 550.00);
SAVEPOINT sp1;
```

- ✓ **INSERT INTO benefits (...)**: Esse comando insere um novo registro na tabela BENEFITS. Especificamos as colunas a serem preenchidas entre parênteses.
- ✓ **VALUES (...)**: Informamos os valores correspondentes às colunas, na mesma ordem em que foram listadas, separados por vírgulas.
- ✓ Repare que:
  - Os valores numéricos são passados sem aspas. Havendo casas decimais, elas são separadas por um ponto (.);
  - As strings (textos) são informadas entre aspas simples;
  - As datas são informadas entre aspas simples, no formato padrão DD/MM/YYYY;
  - Como o benefício ainda está vigente, o end_date está indefinido, por isso usamos NULL.
- ✓ **SAVEPOINT sp1**: Criamos um ponto de controle chamado sp1, permitindo que a transação possa ser revertida até esse ponto, se necessário.
- ✓ ; : O ponto e vírgula encerra o comando.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O RH iniciou o preenchimento da nova tabela com os benefícios vigentes para alguns colaboradores. Após cada inserção, um ponto de controle será criado para garantir que o progresso possa ser revertido com segurança, se necessário.

Insira 5 registros na tabela BENEFITS, utilizando um SAVEPOINT após cada inserção (chame os savepoints de sp1, sp2, sp3, sp4 e sp5). Em seguida, faça um SELECT para consultar os dados inseridos.

Explicação e Resultado

Esse mesmo padrão se repete para os próximos registros:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (2, 146, 'Vale Refeição', 'Cartão Hashcard', '05/03/1997', NULL, 450.00);
SAVEPOINT sp2;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (3, 147, 'Vale Transporte', 'Bilhete Único da Hashtag', '10/05/1997', NULL, 180.00);
SAVEPOINT sp3;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (4, 148, 'Auxílio Educação', 'Bolsa de Especialização Impressionadora', '15/12/1999', NULL, 800.00);
SAVEPOINT sp4;

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (5, 149, 'Gympass', 'Plano HashGold', '29/03/2000', NULL, 120.00);
SAVEPOINT sp5;
```

Cada **SAVEPOINT** (**sp1 a sp5**) é criado após uma inserção, para permitir um controle maior em uma possível reversão com **ROLLBACK**.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

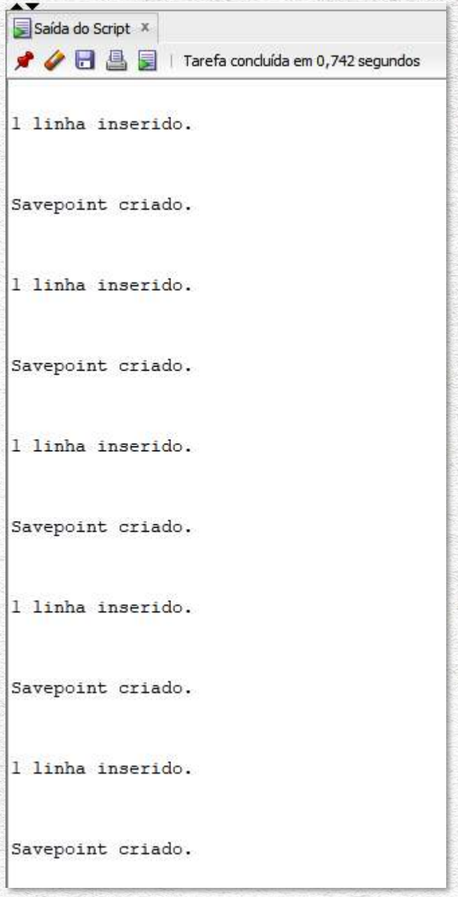
Questão 3

O RH iniciou o preenchimento da nova tabela com os benefícios vigentes para alguns colaboradores. Após cada inserção, um ponto de controle será criado para garantir que o progresso possa ser revertido com segurança, se necessário.

Insira 5 registros na tabela BENEFITS, utilizando um SAVEPOINT após cada inserção (chame os savepoints de sp1, sp2, sp3, sp4 e sp5). Em seguida, faça um SELECT para consultar os dados inseridos.

Explicação e Resultado

Ao executar os comandos, as linhas são inseridas e os savepoints criados, retornando no SQL Developer as mensagens mostradas ao lado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O RH iniciou o preenchimento da nova tabela com os benefícios vigentes para alguns colaboradores. Após cada inserção, um ponto de controle será criado para garantir que o progresso possa ser revertido com segurança, se necessário.

Insira 5 registros na tabela BENEFITS, utilizando um SAVEPOINT após cada inserção (chame os savepoints de sp1, sp2, sp3, sp4 e sp5). Em seguida, faça um SELECT para consultar os dados inseridos.

Explicação e Resultado

Ao consultarmos a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Temos o seguinte resultado:

Saída do Script × Resultado da Consulta ×

SQL | Todas as Linhas Extraídas: 5 em 0,029 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	550
2	2	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
3	3	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
4	4	148	Auxílio Educação	Bolsa de Especialização Impressionadora	15/12/99	(null)	800
5	5	149	Gympass	Plano HashGold	29/03/00	(null)	120

GABARITO EXPLICATIVO

4

Questão 4

Após uma revisão, o RH identificou um erro nos últimos dois registros inseridos. Para corrigir, decidiu-se realizar um ROLLBACK até o ponto de controle anterior à inserção desses dados.

Realize um ROLLBACK até o savepoint sp3 e consulte os dados atuais da tabela.

Explicação e Resultado

Objetivo: Reverter os dois últimos **INSERTs** utilizando um **ROLLBACK TO** até o **SAVEPOINT sp3**, e consultar os dados resultantes.

Como o código foi desenvolvido:

```
ROLLBACK TO SAVEPOINT sp3;
```

Esse comando desfaz todas as alterações realizadas após o ponto de controle **sp3**.

No caso, serão revertidas as inserções feitas **após sp3**, ou seja, os registros relacionados aos **savepoints sp4 e sp5**.

Isso significa que os dois últimos registros (benefícios com **benefit_id 4 e 5**) não estarão mais na tabela após esse comando.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

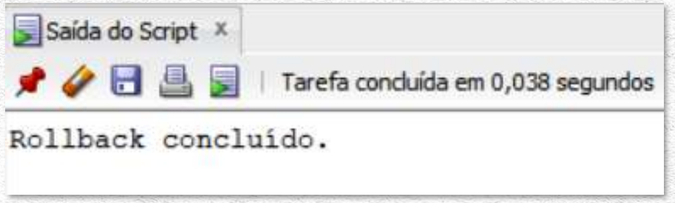
Questão 4

Após uma revisão, o RH identificou um erro nos últimos dois registros inseridos. Para corrigir, decidiu-se realizar um ROLLBACK até o ponto de controle anterior à inserção desses dados.

Realize um ROLLBACK até o savepoint sp3 e consulte os dados atuais da tabela.

Explicação e Resultado

Ao executar o comando, o **ROLLBACK** até o **SAVEPOINT sp3** é efetuado, retornando no SQL Developer a mensagem mostrada abaixo:



Dessa forma, ao consultarmos novamente a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Vemos que os dois últimos registros foram apagados:

Saída do Script | Resultado da Consulta

Todas as Linhas Extraídas: 3 em 0,012 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	550
2	2	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
3	3	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180

GABARITO EXPLICATIVO

5

Questão 5

Após a verificação, o RH confirmou que os dados restantes estão corretos e decidiu consolidar essas alterações no banco de dados.

Efetive as alterações feitas até o momento utilizando o COMMIT e faça uma nova consulta na tabela.

Explicação e Resultado

Objetivo: Confirmar as alterações feitas até o momento utilizando o **COMMIT** e consultar os dados da tabela.

Como o código foi desenvolvido:

```
COMMIT;
```

Esse comando confirma definitivamente todas as alterações realizadas desde o último **COMMIT** ou **ROLLBACK**.

No caso dos nossos exercícios, as 3 primeiras inserções feitas nos registros da tabela (**benefit_id 1, 2 e 3**) foram mantidas após o **ROLLBACK TO SAVEPOINT sp3**, e agora estão sendo oficialmente gravadas no banco de dados.

Isso significa que, mesmo que um **ROLLBACK** seja executado posteriormente, **essas 3 linhas não serão mais desfeitas**, pois já foram persistidas (armazenadas de forma global no banco de dados).

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

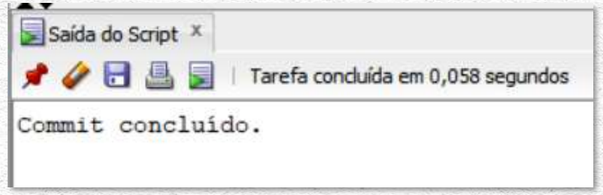
Questão 5

Após a verificação, o RH confirmou que os dados restantes estão corretos e decidiu consolidar essas alterações no banco de dados.

Efetive as alterações feitas até o momento utilizando o COMMIT e faça uma nova consulta na tabela.

Explicação e Resultado

Ao executar o comando, o **COMMIT** é efetuado, retornando no SQL Developer a mensagem mostrada abaixo:



Dessa forma, ao consultarmos novamente a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Vemos que os três primeiros registros permanecem na tabela (de forma definitiva):

Saída do Script | Resultado da Consulta

Todas as Linhas Extraídas: 3 em 0,002 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	550
2	2	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
3	3	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180

GABARITO EXPLICATIVO

6

Questão 6

Foi identificado que o valor mensal do benefício de vale-refeição foi reajustado para R$ 500,00. A equipe de RH precisa atualizar essa informação na tabela.

Atualize o valor do benefício Vale Refeição para R$ 500,00, confirme a transação e consulte os dados atualizados.

Explicação e Resultado

Objetivo: Atualizar o valor mensal do benefício de vale-refeição para R$ 500,00, confirmar a transação e consultar os dados atualizados.

Como o código foi desenvolvido:

```
UPDATE benefits
SET monthly_value = 500.00
WHERE benefit_type = 'Vale Refeição';

COMMIT;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

Foi identificado que o valor mensal do benefício de vale-refeição foi reajustado para R$ 500,00. A equipe de RH precisa atualizar essa informação na tabela.

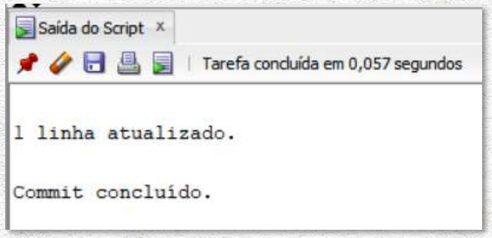
Atualize o valor do benefício Vale Refeição para R$ 500,00, confirme a transação e consulte os dados atualizados.

Explicação e Resultado

Explicação passo a passo:

- ✓ **UPDATE benefits**: Inicia o comando de atualização na tabela BENEFITS.
- ✓ **SET monthly_value = 500.00**: Indica que o valor da coluna monthly_value será alterado para 500.00.
- ✓ **WHERE benefit_type = 'Vale Refeição'**: Aplica a alteração apenas nos registros cujo tipo de benefício (benefit_type) seja exatamente 'Vale Refeição'. Esse WHERE é fundamental para garantir que apenas o benefício correto seja atualizado, evitando alterações indesejadas em outros registros.
- ✓ **;** : O ponto e vírgula encerra o comando UPDATE.
- ✓ **COMMIT;** : Esse comando confirma a alteração feita no UPDATE, salvando a modificação de forma definitiva no banco de dados. Ele também é encerrado com um ponto e vírgula.

Ao executarmos o código no SQL Developer, são retornadas as mensagens de confirmação mostradas ao lado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

Foi identificado que o valor mensal do benefício de vale-refeição foi reajustado para R$ 500,00. A equipe de RH precisa atualizar essa informação na tabela.

Atualize o valor do benefício Vale Refeição para R$ 500,00, confirme a transação e consulte os dados atualizados.

Explicação e Resultado

Dessa forma, ao consultarmos novamente a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Vemos que o registro do 'Vale Refeição' (benefit_id = 2) agora aparece com monthly_value igual a 500.00:

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas: 3 em 0,001 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	550
2	2	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	500
3	3	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180

GABARITO EXPLICATIVO

7

Questão 7

Houve um equívoco e o colaborador de employee_id = 145 não deveria ter sido registrado com plano de saúde.

Exclua o benefício do colaborador de employee_id = 145 e consulte os dados atualizados.

Explicação e Resultado

Objetivo: Excluir o registro do benefício associado ao colaborador de **employee_id = 145** e consultar os dados atualizados.

Como o código foi desenvolvido:

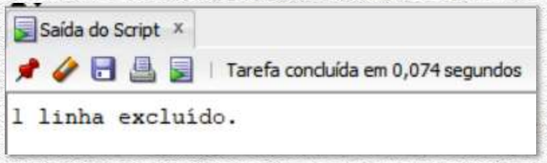
```
DELETE FROM benefits
WHERE employee_id = 145;
```

Explicação passo a passo:

DELETE FROM benefits: Indica que queremos excluir um ou mais registros da tabela BENEFITS.

WHERE employee_id = 145: Especifica qual registro será excluído: aquele em que a coluna employee_id contém o valor 145. Essa cláusula WHERE é extremamente importante, pois sem ela todos os registros da tabela seriam excluídos.

Ao executarmos o código no SQL Developer, é retornada a mensagem mostrada ao lado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Houve um equívoco e o colaborador de employee_id = 145 não deveria ter sido registrado com plano de saúde.

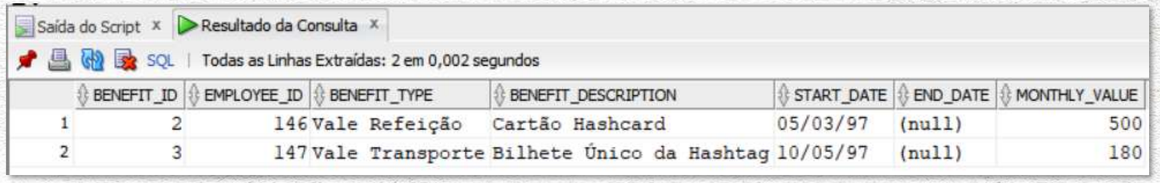
Exclua o benefício do colaborador de employee_id = 145 e consulte os dados atualizados.

Explicação e Resultado

Dessa forma, ao consultarmos novamente a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Vemos que o registro com employee_id = 145 foi excluído:


Saída do Script | Resultado da Consulta

Todas as Linhas Extraídas: 2 em 0,002 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	2	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	500
2	3	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180

Lembre-se: como ainda não executamos um COMMIT ou um ROLLBACK, **esta transação continua em aberto**, podendo ser confirmada ou revertida posteriormente.

GABARITO EXPLICATIVO

8

Questão 8

Após a exclusão do benefício do employee_id = 145, percebeu-se que a informação era válida e precisava ser mantida.

Reverta a exclusão efetuada no exercício anterior com um ROLLBACK. Depois, consulte a tabela.

Explicação e Resultado

Objetivo: Reverter a exclusão realizada no exercício anterior, restaurando o registro do colaborador com **employee_id = 145**.

Como o código foi desenvolvido:

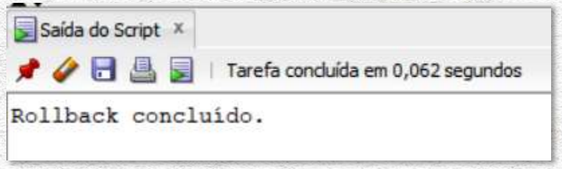
```
ROLLBACK;
```

Esse comando **desfaz todas as alterações** feitas na sessão atual **desde o último COMMIT**.

Como o comando DELETE do exercício anterior não foi seguido por um COMMIT, ele ainda está "pendente".

Ao usar o ROLLBACK, cancelamos essa exclusão e o registro é restaurado.

Ao executarmos o código no SQL Developer, é retornada a mensagem mostrada ao lado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Após a exclusão do benefício do employee_id = 145, percebeu-se que a informação era válida e precisava ser mantida.

Reverta a exclusão efetuada no exercício anterior com um ROLLBACK. Depois, consulte a tabela.

Explicação e Resultado

Dessa forma, ao consultarmos novamente a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Vemos que o registro com employee_id = 145 reaparece na tabela, pois a exclusão foi desfeita com sucesso:

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas: 3 em 0,002 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	550
2	2	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	500
3	3	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180

GABARITO EXPLICATIVO

9

Questão 9

O sistema de gestão de benefícios foi migrado para um novo módulo integrado ao ERP da empresa. O time de TI precisa limpar a tabela BENEFITS, mas mantê-la estruturada para eventual uso futuro.

Remova todos os registros da tabela BENEFITS utilizando o comando TRUNCATE e consulte a tabela.

Explicação e Resultado

Objetivo: Remover todos os registros da tabela **BENEFITS**, mantendo sua estrutura para uso futuro.

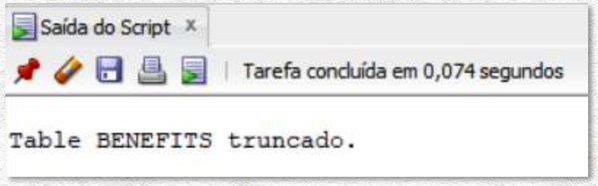
Como o código foi desenvolvido:

```
TRUNCATE TABLE benefits;
```

Esse comando **remove todos os registros de uma tabela, mas mantém sua estrutura** (colunas, tipos de dados, etc.).

Ao contrário do DELETE, o TRUNCATE não permite ROLLBACK (a operação é irreversível).

Ao executarmos o código no SQL Developer, é retornada a mensagem mostrada ao lado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O sistema de gestão de benefícios foi migrado para um novo módulo integrado ao ERP da empresa. O time de TI precisa limpar a tabela BENEFITS, mas mantê-la estruturada para eventual uso futuro.

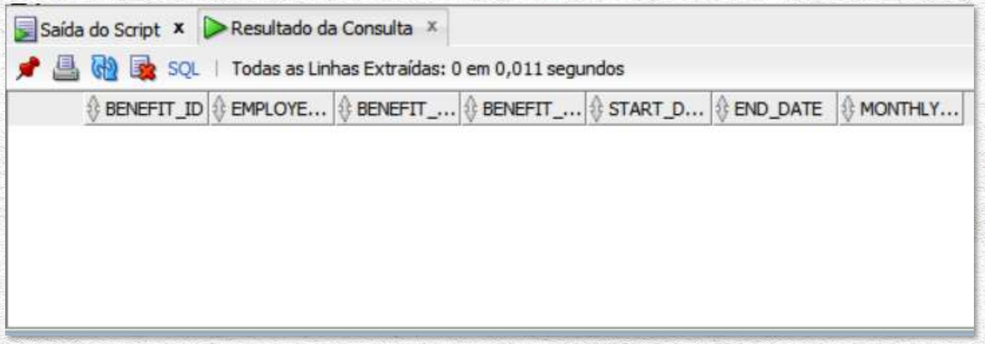
Remova todos os registros da tabela BENEFITS utilizando o comando TRUNCATE e consulte a tabela.

Explicação e Resultado

Dessa forma, ao consultarmos novamente a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

Vemos que esse SELECT retorna nenhum registro, já que todos foram removidos, mas a tabela ainda existe. Em outras palavras, a estrutura da tabela BENEFITS continua no banco, porém vazia:



GABARITO EXPLICATIVO

10

Questão 10

Como a gestão de benefícios agora é realizada por um sistema terceirizado, a tabela BENEFITS não será mais utilizada no banco de dados da empresa.

Remova completamente a tabela BENEFITS do banco de dados. Em seguida, tente consultá-la para verificar a remoção.

Explicação e Resultado

Objetivo: Remover completamente a tabela **BENEFITS** do banco de dados, inclusive sua estrutura.

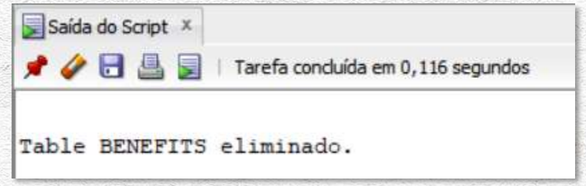
Como o código foi desenvolvido:

```
DROP TABLE benefits;
```

Esse comando apaga completamente a tabela do banco de dados, ou seja:

- ✓ Remove todos os registros;
- ✓ Remove toda a estrutura da tabela (colunas, tipos, restrições, etc.);
- ✓ Não pode ser desfeito (não é possível aplicar ROLLBACK).

Ao executarmos o código no SQL Developer, é retornada a mensagem mostrada ao lado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

10

Questão 10

Como a gestão de benefícios agora é realizada por um sistema terceirizado, a tabela BENEFITS não será mais utilizada no banco de dados da empresa.

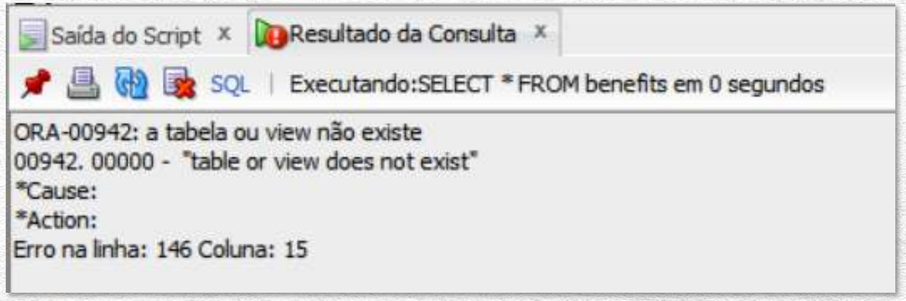
Remova completamente a tabela BENEFITS do banco de dados. Em seguida, tente consultá-la para verificar a remoção.

Explicação e Resultado

Dessa forma, se tentarmos consultar a tabela **BENEFITS**...

```
SELECT * FROM benefits;
```

O Oracle retornará o erro mostrado abaixo, que informa que a tabela não existe:



MÓDULO 11

CONSTRAINTS, SEQUENCES e ÍNDICES

CONSTRAINTS, SEQUENCES e ÍNDICES

CONSTRAINTS, SEQUENCES e ÍNDICES

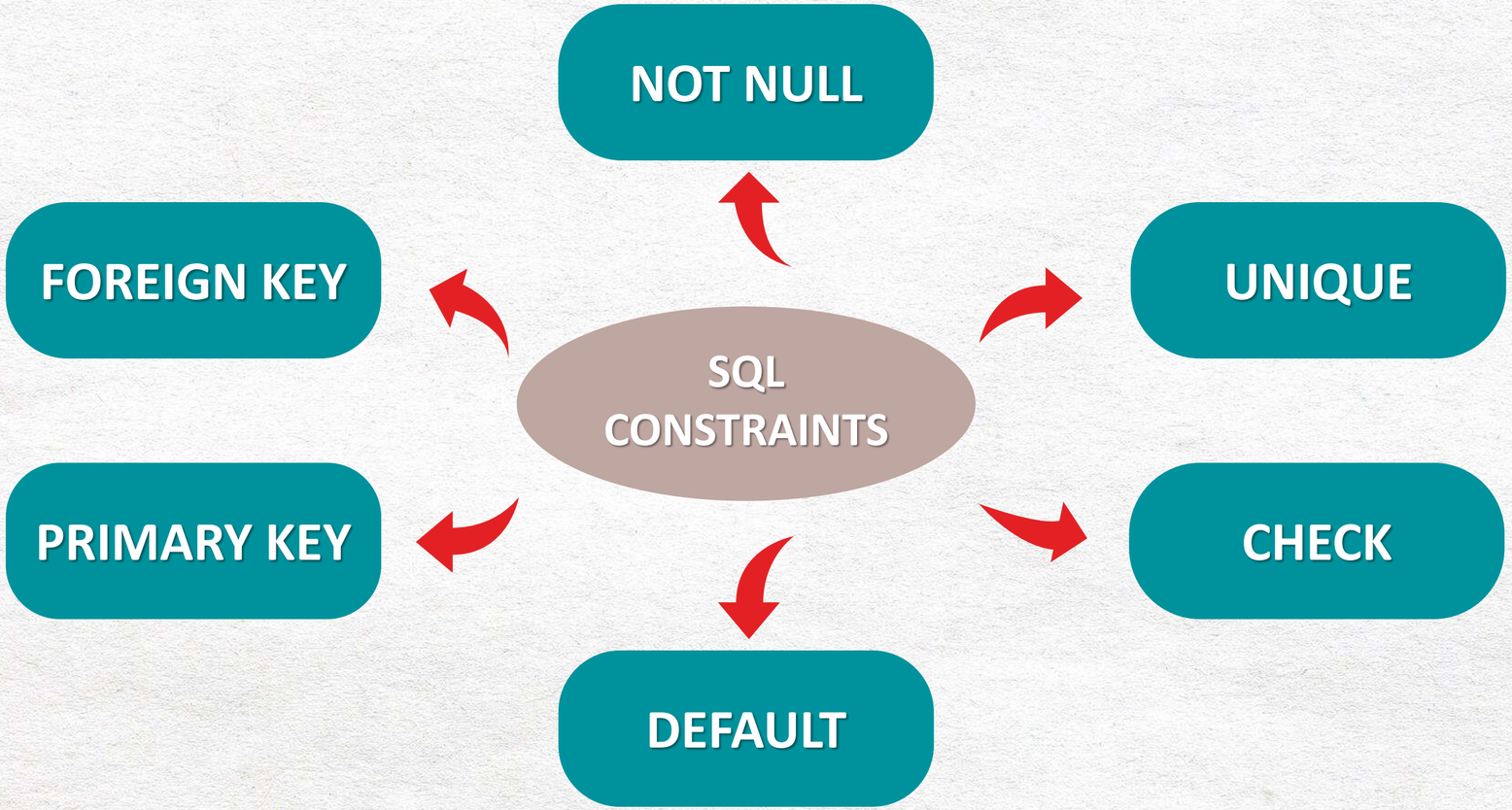
O QUE VEREMOS NESTE MÓDULO

MÓDULO 11 CONSTRAINTS, SEQUENCES e ÍNDICES

01	O QUE SÃO CONSTRAINTS?	489
02	CONSTRAINTS: NOT NULL	490
03	CONSTRAINTS: UNIQUE	491
04	CONSTRAINTS: CHECK	492
05	CONSTRAINTS: DEFAULT	493
06	CONSTRAINTS: PRIMARY KEY	494
07	CONSTRAINTS: FOREIGN KEY	495
08	VIOLAÇÃO DE CONSTRAINTS: NOT NULL	496
09	VIOLAÇÃO DE CONSTRAINTS: CHECK	498
10	VIOLAÇÃO DE CONSTRAINTS: UNIQUE	500
11	VIOLAÇÃO DE CONSTRAINTS: PRIMARY KEY e FOREIGN KEY	502
12	GERENCIANDO CONSTRAINTS: ADICIONAR	505
13	GERENCIANDO CONSTRAINTS: RENOMEAR	506
14	GERENCIANDO CONSTRAINTS: REMOVER	507
15	GERENCIANDO CONSTRAINTS: REMOVER (CASCADE)	508
16	CUIDADOS AO DELETAR TABELAS COM CONSTRAINTS	509
17	O QUE SÃO SEQUENCES?	511
18	SEQUENCES: DICAS	516
19	ALTERANDO SEQUENCES	517
20	CUIDADOS AO CRIAR SEQUENCES	518
21	ÍNDICES	519
22	RESUMO DO MÓDULO	524
23	EXERCÍCIOS	525
24	GABARITO EXPLICATIVO	532

O que são CONSTRAINTS?

CONSTRAINTS no SQL são regras (restrições) que podemos definir para as colunas de uma tabela. Essas regras garantem integridade ao Banco de Dados, pois é através delas que garantimos que apenas os valores que atendam às regras pré-estabelecidas sejam incluídos em cada coluna.



Constraints: NOT NULL

A CONSTRAINT NOT NULL não permite que sejam adicionados valores nulos à coluna.

Repare ao lado na criação da tabela CLIENTES:

Definimos a restrição NOT NULL para as colunas nome_cliente, sexo, email e cpf.

Isso significa que, quando formos inserir linhas na tabela CLIENTES, não podemos deixar de informar dados para estas colunas.

```
CREATE TABLE clientes(
    id_cliente INT,
    nome_cliente VARCHAR2(100) NOT NULL,
    sexo VARCHAR2(100) NOT NULL,
    email VARCHAR2(100) NOT NULL,
    data_nascimento DATE,
    cpf VARCHAR2(100) NOT NULL,
    CONSTRAINT clientes_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT clientes_cpf_un UNIQUE(cpf)
);
```

Constraints: UNIQUE

A CONSTRAINT UNIQUE identifica uma coluna de forma única, sem permitir valores duplicados (mas permite NULL).

Repare ao lado na criação da tabela CLIENTES:

Definimos a restrição UNIQUE para a coluna cpf.

Isso significa que, quando formos inserir linhas na tabela CLIENTES, não podemos informar o mesmo CPF para dois ou mais clientes.

```
CREATE TABLE clientes(
    id_cliente INT,
    nome_cliente VARCHAR2(100) NOT NULL,
    sexo VARCHAR2(100) NOT NULL,
    email VARCHAR2(100) NOT NULL,
    data_nascimento DATE,
    cpf VARCHAR2(100) NOT NULL,
    CONSTRAINT clientes_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT clientes_cpf_un UNIQUE(cpf)
);
```

Constraints: CHECK

A CONSTRAINT CHECK verifica se o valor adicionado na coluna atende a uma determinada condição.

Repare ao lado na criação da tabela PRODUTOS:

Definimos a restrição CHECK para as colunas preco_unit, custo_unit e estoque, para verificar se os valores informados são maiores ou iguais a 0.

Isso significa que, quando formos inserir linhas na tabela PRODUTOS, não podemos informar valores negativos para as colunas preco_unit, custo_unit e estoque.

```
CREATE TABLE produtos(
    id_produto INT,
    nome_produto VARCHAR2(100) NOT NULL,
    marca VARCHAR2(100) NOT NULL,
    categoria VARCHAR2(100) NOT NULL,
    preco_unit NUMBER(10, 2) NOT NULL,
    custo_unit NUMBER(10, 2) NOT NULL,
    estoque INT NOT NULL,
    CONSTRAINT produtos_id_produto_pk PRIMARY KEY(id_produto),
    CONSTRAINT produtos_preco_unit_ck CHECK(preco_unit >= 0),
    CONSTRAINT produtos_custo_unit_ck CHECK(custo_unit >= 0),
    CONSTRAINT produtos_estoque_ck CHECK(estoque >= 0)
);
```

Constraints: DEFAULT

A CONSTRAINT DEFAULT retorna um valor default (padrão) caso a coluna não seja preenchida.

Repare na criação da tabela VENDAS:

Definimos a restrição DEFAULT SYSDATE para a coluna data_venda.

Isso significa que, quando formos inserir linhas na tabela VENDAS, caso não seja informada nenhuma data para a coluna data_venda, esta será preenchida, por padrão, com a data atual.

```
CREATE TABLE vendas(
    id_venda INT,
    data_venda DATE DEFAULT SYSDATE,
    id_cliente INT,
    id_produto INT,
    quantidade NUMBER(10, 2) NOT NULL,
    CONSTRAINT vendas_id_venda_pk PRIMARY KEY(id_venda),
    CONSTRAINT vendas_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente),
    CONSTRAINT vendas_id_produto_fk FOREIGN KEY(id_produto) REFERENCES produtos(id_produto)
);
```

Constraints: PRIMARY KEY

A CONSTRAINT PRIMARY KEY torna uma coluna a chave primária da tabela. É uma coluna que identifica de forma única as linhas de uma tabela.

Nenhum dos valores de uma coluna de chave primária deve ser nulo ou se repetir. Será através dessa coluna que criaremos relações entre as tabelas.

Repare na criação da tabela VENDAS:

Definimos a restrição PRIMARY KEY para a coluna id_venda.

Isso significa que esta coluna é a chave primária da tabela VENDAS e será utilizada para se relacionar com outras tabelas, que farão referência a ela como uma FOREIGN KEY (chave estrangeira).

```
CREATE TABLE vendas(
    id_venda INT,
    data_venda DATE DEFAULT SYSDATE,
    id_cliente INT,
    id_produto INT,
    quantidade NUMBER(10, 2) NOT NULL,
    CONSTRAINT vendas_id_venda_pk PRIMARY KEY(id_venda),
    CONSTRAINT vendas_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente),
    CONSTRAINT vendas_id_produto_fk FOREIGN KEY(id_produto) REFERENCES produtos(id_produto)
);
```

Constraints: FOREIGN KEY

A CONSTRAINT FOREIGN KEY torna uma coluna uma chave estrangeira da tabela, que será relacionada à chave primária (PRIMARY KEY) de outra tabela.

Repare na criação da tabela VENDAS:

Definimos a restrição FOREIGN KEY para as colunas id_cliente e id_produto.

Isso significa que estas colunas se relacionarão com as PRIMARY KEYS (chaves primárias) id_cliente e id_produto das tabelas CLIENTES e PRODUTOS, respectivamente.

```
CREATE TABLE vendas(
    id_venda INT,
    data_venda DATE DEFAULT SYSDATE,
    id_cliente INT,
    id_produto INT,
    quantidade NUMBER(10, 2) NOT NULL,
    CONSTRAINT vendas_id_venda_pk PRIMARY KEY(id_venda),
    CONSTRAINT vendas_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente),
    CONSTRAINT vendas_id_produto_fk FOREIGN KEY(id_produto) REFERENCES produtos(id_produto)
);
```

Violação de Constraints: NOT NULL

Recapitulando, a CONSTRAINT NOT NULL não permite que sejam adicionados valores nulos à coluna.

Agora, repare na criação da nossa tabela PRODUTOS:

Podemos observar que definimos a CONSTRAINT NOT NULL para diversas colunas, entre elas a coluna ESTOQUE.

Isto significa que a coluna ESTOQUE precisa ser devidamente preenchida quando inserimos um novo registro (linha) na tabela PRODUTOS.

```
CREATE TABLE produtos(
    id_produto INT,
    nome_produto VARCHAR2(100) NOT NULL,
    marca VARCHAR2(100) NOT NULL,
    categoria VARCHAR2(100) NOT NULL,
    preco_unit NUMBER(10, 2) NOT NULL,
    custo_unit NUMBER(10, 2) NOT NULL,
    estoque INT NOT NULL,
    CONSTRAINT produtos_id_produto_pk PRIMARY KEY(id_produto),
    CONSTRAINT produtos_preco_unit_ck CHECK(preco_unit >= 0),
    CONSTRAINT produtos_custo_unit_ck CHECK(custo_unit >= 0),
    CONSTRAINT produtos_estoque_ck CHECK(estoque >= 0)
);
```

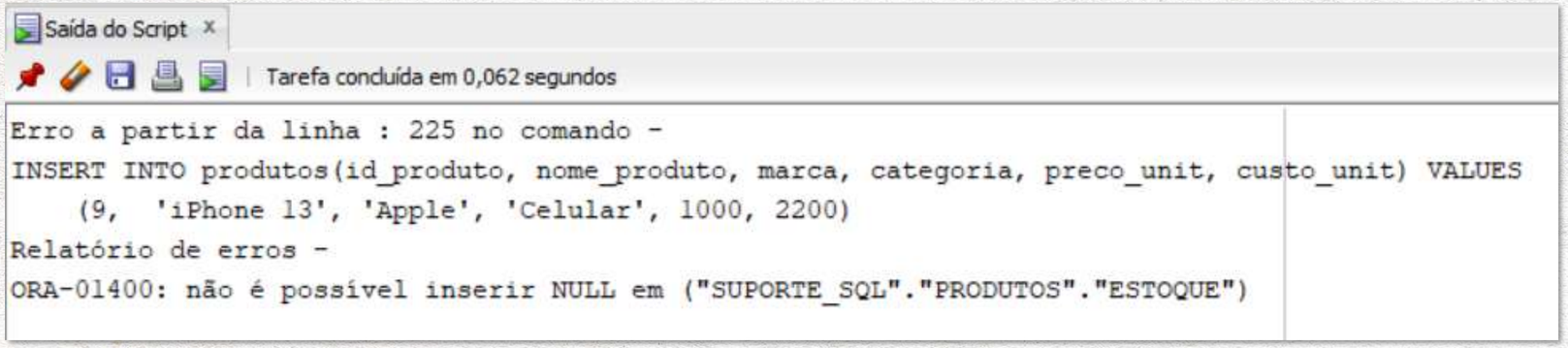
Violação de Constraints: NOT NULL

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos um problema. Consegue perceber por quê?

```
INSERT INTO produtos(id_produto, nome_produto, marca, categoria, preco_unit, custo_unit) VALUES
    (9,  'iPhone 13', 'Apple', 'Celular', 1000, 2200);
```

Se você respondeu que é porque não especificamos a coluna estoque no nosso código, acertou!

Quando deixamos de informar uma coluna na hora de inserir novos dados, o sistema entende que aquela coluna deve ser deixada vazia (preenchida com null). Como definimos, quando criamos nossa tabela PRODUTOS, a restrição NOT NULL para a coluna ESTOQUE, o sistema vai retornar o seguinte erro ao tentar executar o código acima:



Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Violação de Constraints: CHECK

Recapitulando, a CONSTRAINT CHECK verifica se o valor adicionado na coluna atende a uma determinada condição.

Agora, repare na criação da nossa tabela PRODUTOS:

Podemos observar que definimos a CONSTRAINT CHECK para diversas colunas, entre elas a coluna PRECO_UNIT, estabelecendo a condição:

preco_unit >= 0.

Isto significa que a coluna PRECO_UNIT não pode ser preenchida com um número negativo (< 0) quando inserimos um novo registro (linha) na tabela PRODUTOS.

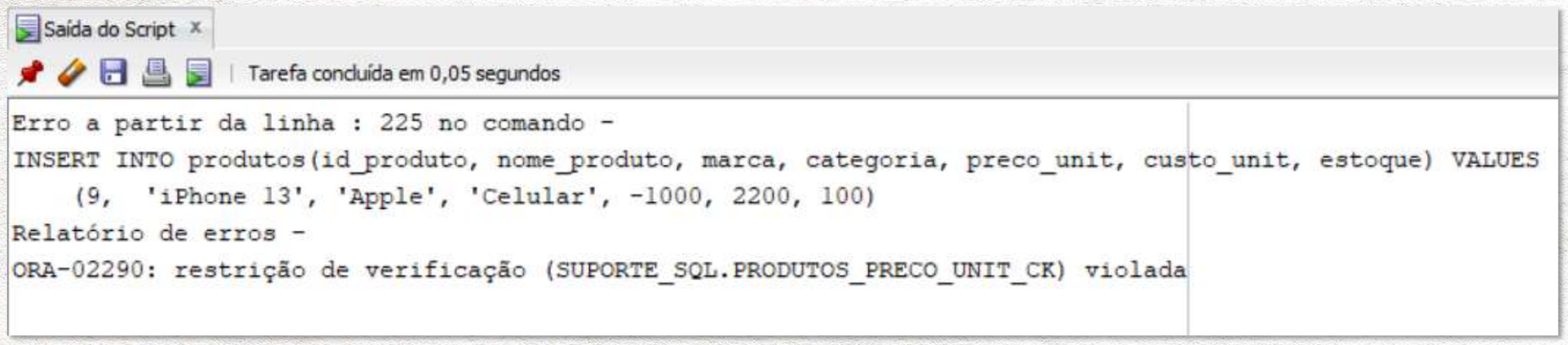
```
CREATE TABLE produtos(
    id_produto INT,
    nome_produto VARCHAR2(100) NOT NULL,
    marca VARCHAR2(100) NOT NULL,
    categoria VARCHAR2(100) NOT NULL,
    preco_unit NUMBER(10, 2) NOT NULL,
    custo_unit NUMBER(10, 2) NOT NULL,
    estoque INT NOT NULL,
    CONSTRAINT produtos_id_produto_pk PRIMARY KEY(id_produto),
    CONSTRAINT produtos_preco_unit_ck CHECK(preco_unit >= 0),
    CONSTRAINT produtos_custo_unit_ck CHECK(custo_unit >= 0),
    CONSTRAINT produtos_estoque_ck CHECK(estoque >= 0)
);
```

Violação de Constraints: CHECK

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos outro problema. Só que desta vez será porque tentamos cadastrar um valor menor que 0 na coluna preco_unit:

```
INSERT INTO produtos(id_produto, nome_produto, marca, categoria, preco_unit, custo_unit, estoque) VALUES
    (9,  'iPhone 13', 'Apple', 'Celular', -1000, 2200, 100);
```

Quando violamos uma restrição CHECK, o sistema retorna um erro informando os nomes da conexão, da tabela e da constraint violada:



Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Violação de Constraints: UNIQUE

Recapitulando, a CONSTRAINT UNIQUE identifica uma coluna de forma única, sem permitir valores duplicados.

Agora, repare na criação da nossa tabela CLIENTES:

Podemos observar que definimos a CONSTRAINT UNIQUE para a coluna CPF.

Isto significa que a coluna CPF não pode receber um valor que já conste na coluna como pertencente a algum outro registro (cliente) da tabela CLIENTES.

```
CREATE TABLE clientes(
    id_cliente INT,
    nome_cliente VARCHAR2(100) NOT NULL,
    sexo VARCHAR2(100) NOT NULL,
    email VARCHAR2(100) NOT NULL,
    data_nascimento DATE,
    cpf VARCHAR2(100) NOT NULL,
    CONSTRAINT clientes_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT clientes_cpf_un UNIQUE(cpf)
);
```

Violação de Constraints: UNIQUE

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos mais um problema...

```
INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
    (11, 'Karine Dias', 'F', 'kdias@gmail.com', '15/06/1993', '416.209.192-47');
```

... pois o CPF informado já foi informado como sendo de outro cliente. Veja:

SQL | Todas as Linhas Extraídas: 10 em 0,006 segundos

	ID_CLIENTE	NOME_CLIENTE	SEXO	EMAIL	DATA_NASCIMENTO	CPF
10	10	Juliana Pires	F	jupires@gmail.com	13/01/91	416.209.192-47

Quando violamos uma restrição UNIQUE, o sistema retorna um erro informando os nomes da conexão, da tabela e da constraint violada:

Saída do Script

Tarefa concluída em 0,048 segundos

```
Erro a partir da linha : 230 no comando -
INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
    (11, 'Karine Dias', 'F', 'kdias@gmail.com', '15/06/1993', '416.209.192-47')
Relatório de erros -
ORA-00001: restrição exclusiva (SUPORTE_SQL.CLIENTES_CPF_UN) violada
```

Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Violação de Constraints: PRIMARY KEY e FOREIGN KEY

Recapitulando, a CONSTRAINT PRIMARY KEY torna uma coluna a chave primária da tabela, ao passo que a CONSTRAINT FOREIGN KEY torna uma coluna uma chave estrangeira da tabela, que será relacionada à chave primária (PRIMARY KEY) de outra tabela.

Agora, repare na criação da nossa tabela VENDAS:

Podemos observar que definimos a CONSTRAINT FOREIGN KEY para a coluna id_produto, que faz referência à PRIMARY KEY id_produto estabelecida na tabela PRODUTOS.

Isto significa que a coluna id_produto da tabela VENDAS não pode receber um ID de um produto que não exista na coluna id_produto da tabela PRODUTOS.

```
CREATE TABLE vendas(
    id_venda INT,
    data_venda DATE DEFAULT SYSDATE,
    id_cliente INT,
    id_produto INT,
    quantidade NUMBER(10, 2) NOT NULL,
    CONSTRAINT vendas_id_venda_pk PRIMARY KEY(id_venda),
    CONSTRAINT vendas_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente),
    CONSTRAINT vendas_id_produto_fk FOREIGN KEY(id_produto) REFERENCES produtos(id_produto)
);
```

FOREIGN KEY DA TABELA VENDAS

REFERÊNCIA À PRIMARY KEY DA TABELA PRODUTOS

Violação de Constraints: PRIMARY KEY e FOREIGN KEY

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos outro problema...

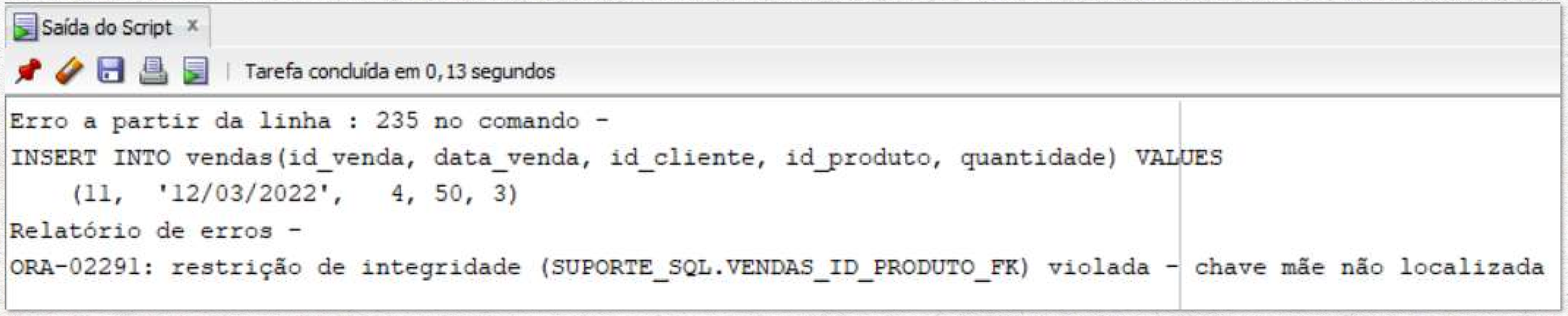
```
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (11,  '12/03/2022',   4,  50,  3);
```

... pois o id_produto informado (50) não existe na tabela PRODUTOS. Veja:

ID_PRODUTO	NOME_PRODUTO	MARCA	CATEGORIA	PRECO_UNIT	CUSTO_UNIT	ESTOQUE
1	iPhone 11	Apple	Celular	3500	2200	100
2	iPhone 12	Apple	Celular	4100	3000	100
3	Moto G60	Motorola	Celular	1600	800	100
4	Samsung S22	Samsung	Celular	5000	2800	100
5	Aspire 5	Acer	Notebook	3300	1800	100
6	Inspiron 15 3000	Dell	Notebook	2800	1100	100
7	Smart TV 4K	LG	Televisão	1800	700	100
8	Smart TV Crystal UHD 4K	Samsung	Televisão	3000	1200	100

Violação de Constraints: PRIMARY KEY e FOREIGN KEY

Quando violamos as restrições PRIMARY KEY e FOREIGN KEY, o sistema retorna um erro informando os nomes da conexão, da tabela e da constraint violada:



Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Gerenciando Constraints: ADICIONAR

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para ADICIONAR uma nova CONSTRAINT a uma tabela, utilizamos a seguinte sintaxe:

```
ALTER TABLE   nome_da_tabela
ADD CONSTRAINT   nome_da_nova_constraint   tipo_da_nova_constraint (nome_da_coluna);
```

Exemplos:

```
-- Adicione a constraint PK id_venda na tabela VENDAS.
ALTER TABLE vendas
ADD CONSTRAINT vendas_id_venda_pk PRIMARY KEY(id_venda);
```

```
-- Adicione a constraint FK id_cliente na tabela VENDAS.
ALTER TABLE vendas
ADD CONSTRAINT vendas_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente);
```

Gerenciando Constraints: RENOMEAR

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para RENOMEAR uma CONSTRAINT de uma tabela, utilizamos a seguinte sintaxe:

```
ALTER TABLE   nome_da_tabela
RENAME CONSTRAINT   nome_atual_da_constraint  TO  novo_nome_da_constraint;
```

Exemplo:

```
-- Altera o nome de uma constraint criada.
ALTER TABLE clientes
RENAME CONSTRAINT clientes_cpg_un TO clientes_cpf_un;
```

Gerenciando Constraints: REMOVER

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para REMOVER uma CONSTRAINT de uma tabela, utilizamos a seguinte sintaxe:

```
ALTER TABLE   nome_da_tabela
DROP CONSTRAINT   nome_da_constraint;
```

Exemplo:

```
-- Remova a constraint PK da tabela VENDAS.
ALTER TABLE vendas
DROP CONSTRAINT vendas_id_venda_pk;
```

Gerenciando Constraints: REMOVER (CASCADE)

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para REMOVER uma CONSTRAINT PRIMARY KEY de uma tabela que foi referenciada como FOREIGN KEY em outra(s) tabela(s), utilizamos o termo CASCADE ao final do código.

Desta forma, conseguimos removê-la da tabela principal (na qual ela é uma PRIMARY KEY), assim como excluir todas as CONSTRAINTS que a referenciaram como FOREIGN KEY na(s) outra(s) tabela(s).

Utilizamos a seguinte sintaxe:

```
ALTER TABLE   nome_da_tabela
DROP CONSTRAINT   nome_da_constraint CASCADE;
```

Exemplo:

```
-- Remova a constraint PK id_produto da tabela PRODUTOS (CASCADE).
ALTER TABLE produtos
DROP CONSTRAINT produtos_id_produto_pk CASCADE;
```

Cuidados ao deletar tabelas com CONSTRAINTS

Quando, utilizando o código “DROP TABLE nome_da_tabela;”, tentamos DELETAR uma tabela com uma CONSTRAINT que foi referenciada em outra(s) tabela(s), é retornado o seguinte erro:

```
DROP TABLE clientes;
```

Saída do Script

Tarefa concluída em 0,037 segundos

```
Erro a partir da linha : 252 no comando -
DROP TABLE clientes
Relatório de erros -
ORA-02449: chaves primárias/exclusivas na tabela referenciadas por chaves externas
02449. 00000 -  "unique/primary keys in table referenced by foreign keys"
*Cause:    An attempt was made to drop a table with unique or
           primary keys referenced by foreign keys in another table.
*Action:   Before performing the above operations the table, drop the
           foreign key constraints in other tables. You can see what
           constraints are referencing a table by issuing the following
           command:
           SELECT * FROM USER_CONSTRAINTS WHERE TABLE_NAME = "tabnam";
```

Cuidados ao deletar tabelas com CONSTRAINTS

Se quisermos forçar a exclusão da tabela mesmo assim, precisamos utilizar o termo CASCADE CONSTRAINTS:

Desta forma, conseguimos DELETAR a tabela desejada, assim como todas as CONSTRAINTS que a referenciaram na(s) outra(s) tabela(s).

Exemplo:

```
DROP TABLE clientes CASCADE CONSTRAINTS;
DROP TABLE produtos;
DROP TABLE vendas;
```

O que são SEQUENCES?

Uma sequência (SEQUENCE) é um objeto que utilizamos para criação de números sequenciais automáticos.

São usados especialmente para gerar valores sequenciais únicos para as chaves primárias das tabelas.

Dessa forma, não precisamos ficar preenchendo a sequência de IDs manualmente (como fizemos até então). Podemos gerá-los automaticamente por meio de uma SEQUENCE.

Nas próximas páginas, vamos entender sua sintaxe e como utilizá-la.

O que são SEQUENCES?

SINTAXE:

CREATE SEQUENCE nome_sequencia	Criar sequência
INCREMENT BY n	Incrementar de quantos em quantos números (de 1 em 1, de 10 em 10, etc.)
START WITH n	Começar a sequência a partir de qual número
MAXVALUE n | NOMAXVALUE	A sequência terá valor máximo ou não
MINVALUE n | NOMINVALUE	A sequência terá valor mínimo ou não
CYCLE | NOCYCLE	Quando atingir o valor máximo (se houver), deve voltar ao começo ou não
CACHE n | NOCACHE;	Os valores da sequência devem ser armazenados na memória cache ou não

O que são SEQUENCES?

EXEMPLO:

Vamos criar uma sequência para utilizar na tabela VENDAS que:

```
CREATE SEQUENCE vendas_seq
START WITH 1
INCREMENT BY 1
NOMAXVALUE
NOCYCLE
NOCACHE;
```

Será chamada de vendas_seq;

Começará pelo número 1;

Será incrementada de 1 em 1;

Não terá valor máximo;

Não voltará ao início;

Não terá valores armazenados na memória cache.

O que são SEQUENCES?

EXEMPLO:

Para fazer uso da nossa SEQUENCE vendas_seq na tabela VENDAS, quando formos inserir dados nesta tabela, em vez de preencher a coluna id_venda manualmente, digitamos, em seu lugar, “vendas_seq.NEXTVAL”:

```
INSERT INTO vendas(id_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,   5, 1, 2);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '10/01/2022',   2, 3, 1);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '13/01/2022',   1, 7, 1);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '02/02/2022',   7, 1, 1);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '17/02/2022',   4, 1, 3);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '21/02/2022',   6, 5, 3);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '05/03/2022',   7, 5, 1);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '07/03/2022',   10, 2, 1);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '09/03/2022',   9, 1, 1);
INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
    (vendas_seq.NEXTVAL,  '12/03/2022',   4, 3, 3);
```

O que são SEQUENCES?

EXEMPLO:

Assim, teremos a coluna id_venda preenchida automaticamente:

ID_VENDA	DATA_VENDA	ID_CLIENTE	ID_PRODUTO	QUANTIDADE
1	10/08/22	5	1	2
2	10/01/22	2	3	1
3	13/01/22	1	7	1
4	02/02/22	7	1	1
5	17/02/22	4	1	3
6	21/02/22	6	5	3
7	05/03/22	7	5	1
8	07/03/22	10	2	1
9	09/03/22	9	1	1
10	12/03/22	4	3	3

SEQUENCES: Dicas

DICAS:

Para consultar o valor atual de uma SEQUENCE, utilize o comando CURRVAL:

```
-- Valor atual da sequência

SELECT vendas_seq.CURRVAL
FROM dual;
```

Para consultar o próximo valor de uma SEQUENCE, utilize o comando NEXTVAL:

```
-- Próximo valor da sequência

SELECT vendas_seq.NEXTVAL
FROM dual;
```

Para consultar as informações de todas as SEQUENCES do usuário, utilize o comando SELECT * FROM user_sequences;

```
SELECT * FROM user_sequences;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 3 em 0,026 segundos

	SEQUENCE_NAME	MIN_VALUE	MAX_VALUE	INCREMENT_BY	LAST_NUMBER
1	CLIENTES_SEQ	1	9999999999999999999999999999	1	11
2	PRODUTOS_SEQ	1	9999999999999999999999999999	1	9
3	VENDAS_SEQ	1	9999999999999999999999999999	1	11

Por fim, para excluir uma SEQUENCE, utilize o comando DROP SEQUENCE:

```
-- Excluir uma sequence
DROP SEQUENCE vendas_seq;
```

Alterando SEQUENCES

Para alterar uma SEQUENCE, utilize o comando ALTER SEQUENCE:

SINTAXE:

ALTER SEQUENCE nome_sequencia
INCREMENT BY n
MAXVALUE n | **NOMAXVALUE**
MINVALUE n | **NOMINVALUE**
CYCLE | **NOCYCLE**
CACHE n | **NOCACHE**;

```
ALTER SEQUENCE vendas_seq
INCREMENT BY 1
NOMAXVALUE
NOCYCLE
NOCACHE;
```

Cuidados ao criar SEQUENCES

1. Se tivermos um ROLLBACK, as transações são desfeitas, mas o NEXTVAL não volta atrás.

2. Não utilize uma mesma sequência em mais de uma tabela.

ÍNDICES

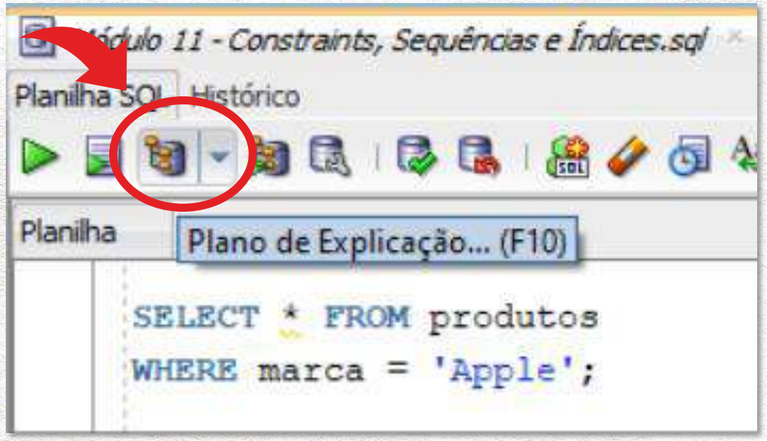
Um ÍNDICE (INDEX) é um recurso do SGBD que possibilita que uma consulta seja retornada com uma maior rapidez, otimizando a busca dos dados no banco de dados.

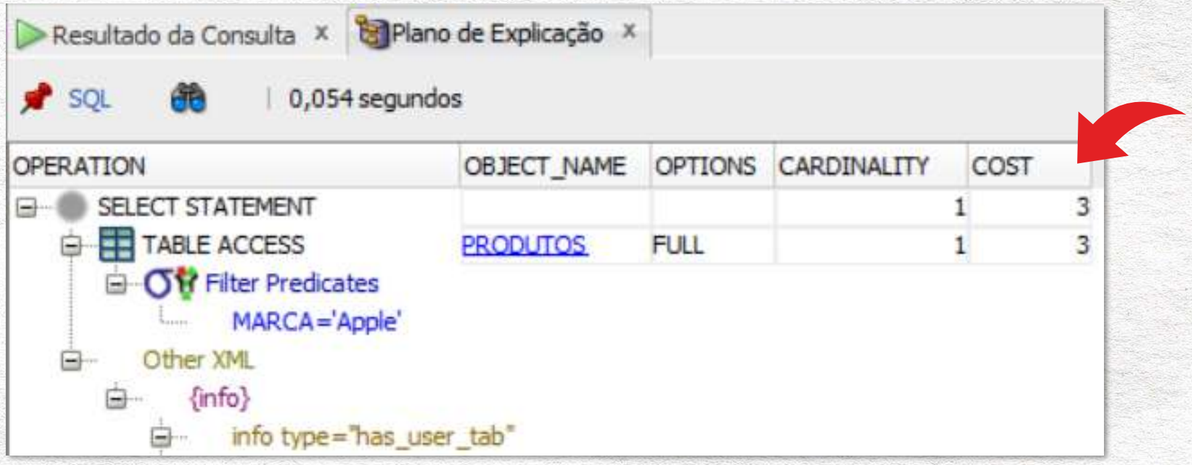
Para entender na prática, inicialmente, vamos fazer uma consulta à tabela PRODUTOS para trazer todos os produtos da marca Apple:

```
SELECT * FROM produtos
WHERE marca = 'Apple';
```

	ID_PRODUTO	NOME_PRODUTO	MARCA	CATEGORIA	PRECO_UNIT	CUSTO_UNIT	ESTOQUE
1	1	iPhone 11	Apple	Celular	3500	2200	100
2	2	iPhone 12	Apple	Celular	4100	3000	100

Se clicarmos no ícone “Plano de Explicação”, veremos que o sistema retornará o número 3 para o custo (COST) desta operação (consulta):

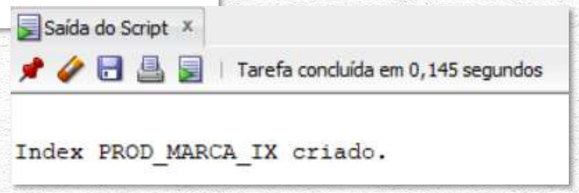




ÍNDICES

Agora, vamos criar um índice para a coluna MARCA da nossa tabela PRODUTOS:

```
CREATE INDEX prod_marca_ix
ON produtos(marca);
```

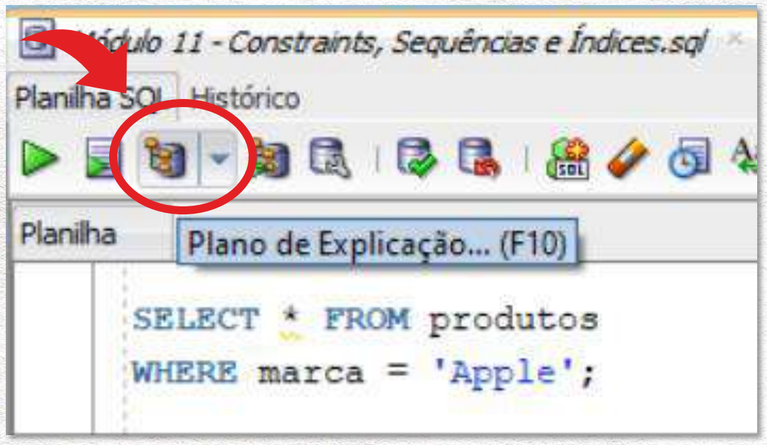


Com o índice criado, vamos efetuar a mesma consulta aos produtos da marca Apple:

```
SELECT * FROM produtos
WHERE marca = 'Apple';
```

	ID_PRODUTO	NOME_PRODUTO	MARCA	CATEGORIA	PRECO_UNIT	CUSTO_UNIT	ESTOQUE
1	1	iPhone 11	Apple	Celular	3500	2200	100
2	2	iPhone 12	Apple	Celular	4100	3000	100

Se clicarmos novamente no ícone “Plano de Explicação”, veremos que, agora, o sistema retornará o número 2 para o custo (COST) desta operação (consulta):

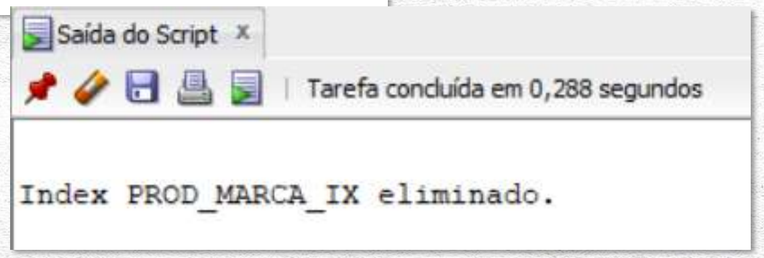


Isto porque, com o índice que criamos, nossa consulta foi otimizada. Neste exemplo, a diferença é pequena, pois a tabela que utilizamos não tem muitos registros. Mas em um BD robusto, a criação de índices pode fazer uma grande diferença!

DICA:

Para excluir o índice criado do banco de dados, utilize o comando DROP INDEX:

```
DROP INDEX prod_marca_ix;
```

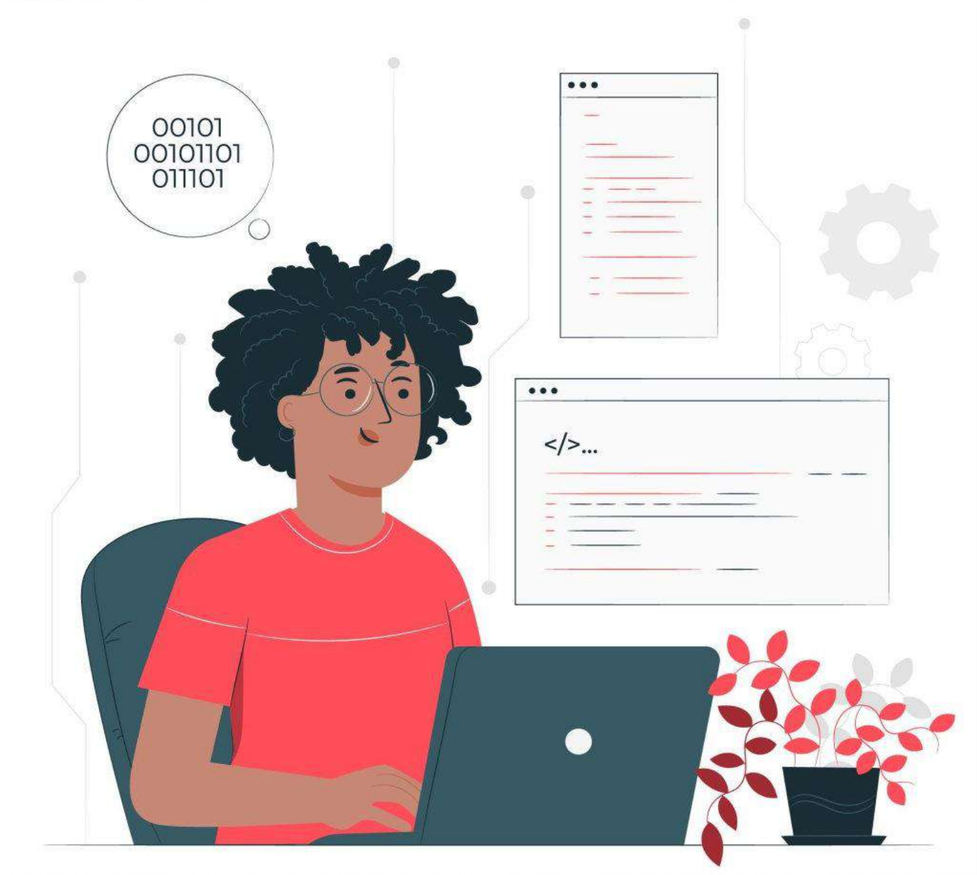


Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. CONSTRAINTS (restrições) são regras criadas para as colunas de uma tabela. Elas garantem uma maior integridade/coerência ao nosso banco de dados na hora de adicionar valores, pois caso elas sejam violadas, o banco de dados não permite a inserção / modificação dos dados.
2. Trabalhamos com as CONSTRAINTS: NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY e FOREIGN KEY.
3. Criar CONSTRAINTS com nomes garante a organização do nosso banco de dados.
4. SEQUENCES facilitam o processo de criação de colunas de IDs no banco de dados.
5. Índices (INDEX) tornam as consultas aos bancos de dados mais otimizadas. No entanto, não são obrigatórios.

EXERCÍCIOS



1

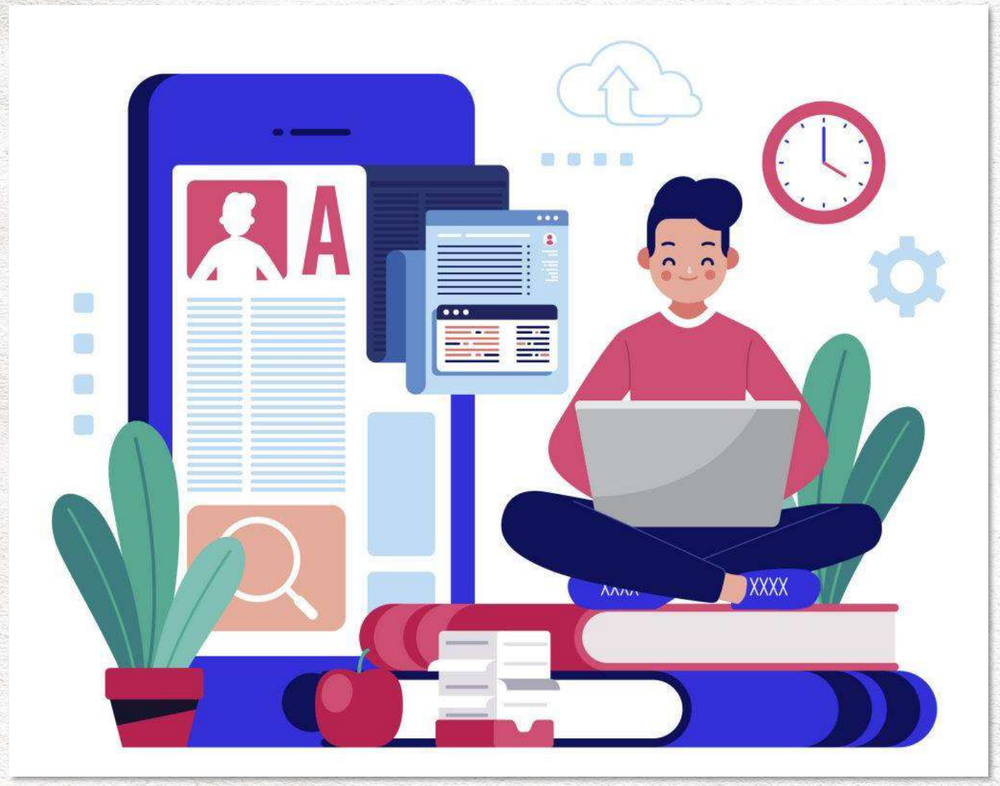
Questão 1

A empresa decidiu registrar os benefícios oferecidos a seus colaboradores em uma nova tabela chamada **BENEFITS**. Crie a tabela BENEFITS no banco de dados **RH** com as colunas necessárias para armazenar o ID do benefício, o ID do funcionário, o tipo de benefício e sua descrição, as datas de início e fim, além do valor mensal. Siga as instruções abaixo:

- **Em relação aos nomes e tipos de dados das colunas:**
  - ✓ A coluna do ID do benefício deverá se chamar **benefit_id** para receber números inteiros;
  - ✓ A coluna do ID do funcionário deverá se chamar **employee_id** para receber números inteiros;
  - ✓ A coluna do tipo de benefício deverá se chamar **benefit_type** para receber textos de até 30 caracteres;
  - ✓ A coluna da descrição do benefício deverá se chamar **benefit_description** para receber textos de até 50 caracteres;
  - ✓ A coluna da data de início do benefício deverá se chamar **start_date** para receber uma data;
  - ✓ A coluna da data final do benefício deverá se chamar **end_date** para receber uma data;
  - ✓ A coluna do valor mensal do benefício deverá se chamar **monthly_value** para receber um valor com até duas casas decimais.

CONTINUA NA PRÓXIMA PÁGINA

EXERCÍCIOS

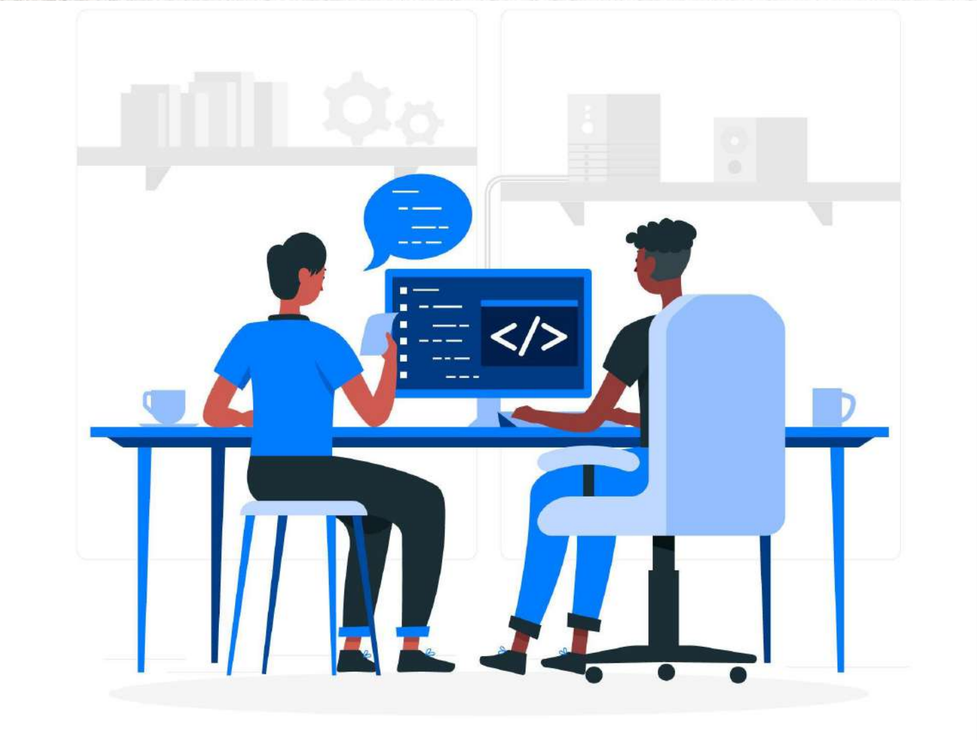


1

Questão 1

- **Sobre as constraints:**
  - ✓ A coluna benefit_id deve ser a chave primária (chame esta constraint de **benefits_benefit_id_pk**);
  - ✓ A coluna employee_id deve ser uma chave estrangeira que faz referência à tabela employees (chame esta constraint de **benefits_employee_id_fk**);
  - ✓ As colunas benefit_type e benefit_description não podem ser nulas;
  - ✓ Caso não seja informada uma data para a coluna start_date, por padrão, deve ser armazenada a data atual;
  - ✓ A coluna end_date pode ser nula ou conter uma data maior que a data inserida na coluna start_date (chame esta constraint de **benefits_end_date_ck**);
  - ✓ A coluna monthly_value não deve aceitar valor negativos (chame esta constraint de **benefits_monthly_value_ck**).

EXERCÍCIOS



2

Questão 2

A área de controladoria da empresa alterou sua política de benefícios e definiu que o valor mínimo mensal para qualquer benefício deverá ser de R$100,00. Altere a constraint existente na tabela BENEFITS para refletir essa nova regra.

Dica: se necessário, exclua a constraint e recrie-a com a nova condição.

3

Questão 3

Crie uma sequence chamada benefits_seq para ser utilizada na geração automática dos identificadores dos benefícios registrados na tabela BENEFITS.

Ela deve iniciar com o valor 1, incrementar de 1 em 1, não ter valor máximo, não voltar ao início nem armazenar valores na memória cache.

EXERCÍCIOS

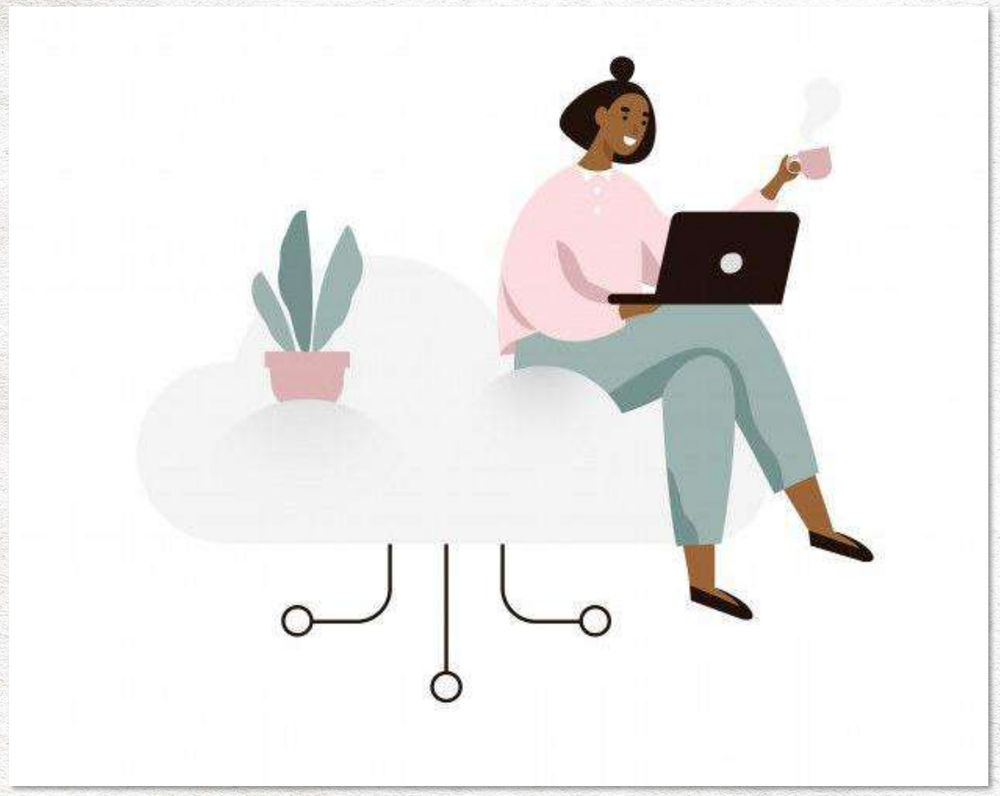
4

Questão 4

Com base em uma listagem de benefícios fornecida pela área de RH, insira os 20 registros correspondentes na tabela BENEFITS, utilizando a sequence criada no exercício anterior para preencher a coluna benefit_id.

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

EXERCÍCIOS



5

Questão 5

Insira o novo benefício especificado abaixo na tabela BENEFITS, omitindo os valores das colunas start_date e end_date. Em seguida, observe o que é atribuído automaticamente para essas colunas. Explique o motivo de cada resultado com base na definição da tabela:

- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 131;
- ✓ Tipo do benefício: Auxílio Educação;
- ✓ Descrição do benefício: Bolsa de Especialização Impressionadora;
- ✓ Valor mensal: R$ 800,00.

6

Questão 6

Tente atualizar o campo end_date do benefit_id = 1 com a data '10/12/2000'. Observe o que aconteceu e explique por que ocorreu.

EXERCÍCIOS

7

Questão 7

Tente inserir um novo registro na tabela BENEFITS com os dados especificados abaixo:

- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 132;
- ✓ Tipo do benefício: Gympass;
- ✓ Descrição do benefício: Plano HashSilver;
- ✓ Data de início: 05/06/2010;
- ✓ Data final: Nula;
- ✓ Valor mensal: R$ 90,00.

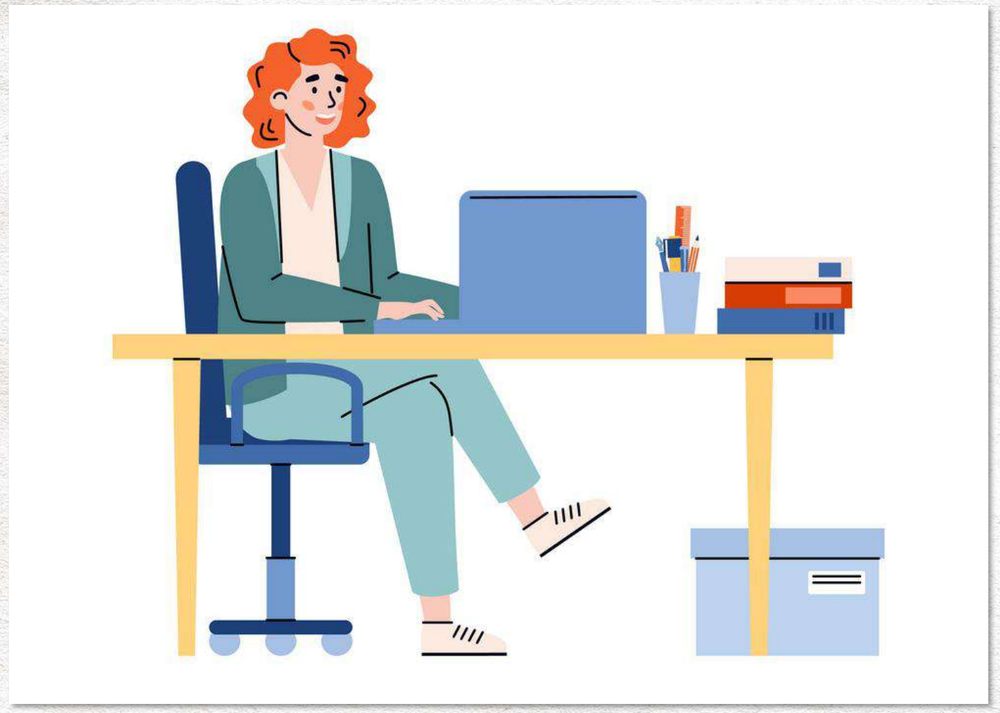
O que acontece com essa operação? Por quê?

8

Questão 8

Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130. Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação. O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

EXERCÍCIOS



9

Questão 9

Remova o índice criado no exercício anterior (benefit_emp_ix) da tabela BENEFITS.

10

Questão 10

Remova a sequence benefits_seq (criada anteriormente para geração dos IDs) da tabela BENEFITS.

11

Questão 11

Exclua a tabela BENEFITS garantindo que quaisquer constraints que possam estar referenciando essa tabela também sejam removidas automaticamente.

GABARITO EXPLICATIVO

1

Questão 1

A empresa decidiu registrar os benefícios oferecidos a seus colaboradores em uma nova tabela chamada **BENEFITS**. Crie a tabela BENEFITS no banco de dados **RH** com as colunas necessárias para armazenar o ID do benefício, o ID do funcionário, o tipo de benefício e sua descrição, as datas de início e fim, além do valor mensal. Siga as instruções abaixo:

- **Em relação aos nomes e tipos de dados das colunas:**
  - ✓ A coluna do ID do benefício deverá se chamar **benefit_id** para receber números inteiros;
  - ✓ A coluna do ID do funcionário deverá se chamar **employee_id** para receber números inteiros;
  - ✓ A coluna do tipo de benefício deverá se chamar **benefit_type** para receber textos de até 30 caracteres;
  - ✓ A coluna da descrição do benefício deverá se chamar **benefit_description** para receber textos de até 50 caracteres;
  - ✓ A coluna da data de início do benefício deverá se chamar **start_date** para receber uma data;
  - ✓ A coluna da data final do benefício deverá se chamar **end_date** para receber uma data;
  - ✓ A coluna do valor mensal do benefício deverá se chamar **monthly_value** para receber um valor com até duas casas decimais.

- **Sobre as constraints:**
  - ✓ A coluna benefit_id deve ser a chave primária (chame esta constraint de **benefits_benefit_id_pk**);
  - ✓ A coluna employee_id deve ser uma chave estrangeira que faz referência à tabela employees (chame esta constraint de **benefits_employee_id_fk**);
  - ✓ As colunas benefit_type e benefit_description não podem ser nulas;
  - ✓ Caso não seja informada uma data para a coluna start_date, por padrão, deve ser armazenada a data atual;
  - ✓ A coluna end_date pode ser nula ou conter uma data maior que a data inserida na coluna start_date (chame esta constraint de **benefits_end_date_ck**);
  - ✓ A coluna monthly_value não deve aceitar valor negativos (chame esta constraint de **benefits_monthly_value_ck**).

Explicação e Resultado

Objetivo:

- ✓ Criar a tabela **BENEFITS** com as colunas e constraints necessárias.

Como o código foi desenvolvido:

```
CREATE TABLE benefits (
  benefit_id INT,
  employee_id INT,
  benefit_type VARCHAR2(30) NOT NULL,
  benefit_description VARCHAR2(50) NOT NULL,
  start_date DATE DEFAULT SYSDATE,
  end_date DATE,
  monthly_value NUMBER(10, 2),
  CONSTRAINT benefits_benefit_id_pk PRIMARY KEY(benefit_id),
  CONSTRAINT benefits_employee_id_fk FOREIGN KEY(employee_id) REFERENCES employees(employee_id),
  CONSTRAINT benefits_end_date_ck CHECK (end_date IS NULL OR end_date > start_date),
  CONSTRAINT benefits_monthly_value_ck CHECK(monthly_value >= 0)
);
```

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

1 Questão 1

A empresa decidiu registrar os benefícios oferecidos a seus colaboradores em uma nova tabela chamada **BENEFITS**. Crie a tabela BENEFITS no banco de dados **RH** com as colunas necessárias para armazenar o ID do benefício, o ID do funcionário, o tipo de benefício e sua descrição, as datas de início e fim, além do valor mensal. Siga as instruções abaixo:

- **Em relação aos nomes e tipos de dados das colunas:**
  - ✓ A coluna do ID do benefício deverá se chamar **benefit_id** para receber números inteiros;
  - ✓ A coluna do ID do funcionário deverá se chamar **employee_id** para receber números inteiros;
  - ✓ A coluna do tipo de benefício deverá se chamar **benefit_type** para receber textos de até 30 caracteres;
  - ✓ A coluna da descrição do benefício deverá se chamar **benefit_description** para receber textos de até 50 caracteres;
  - ✓ A coluna da data de início do benefício deverá se chamar **start_date** para receber uma data;
  - ✓ A coluna da data final do benefício deverá se chamar **end_date** para receber uma data;
  - ✓ A coluna do valor mensal do benefício deverá se chamar **monthly_value** para receber um valor com até duas casas decimais.

- **Sobre as constraints:**
  - ✓ A coluna benefit_id deve ser a chave primária (chame esta constraint de **benefits_benefit_id_pk**);
  - ✓ A coluna employee_id deve ser uma chave estrangeira que faz referência à tabela employees (chame esta constraint de **benefits_employee_id_fk**);
  - ✓ As colunas benefit_type e benefit_description não podem ser nulas;
  - ✓ Caso não seja informada uma data para a coluna start_date, por padrão, deve ser armazenada a data atual;
  - ✓ A coluna end_date pode ser nula ou conter uma data maior que a data inserida na coluna start_date (chame esta constraint de **benefits_end_date_ck**);
  - ✓ A coluna monthly_value não deve aceitar valor negativos (chame esta constraint de **benefits_monthly_value_ck**).

Explicação e Resultado

Explicação passo a passo:

- ✓ **CREATE TABLE benefits**: Inicia a criação de uma nova tabela chamada BENEFITS.
- ✓ **(...)** : Os parênteses delimitam a lista de colunas, seus tipos de dados e constraints.
- ✓ **benefit_id INT**: Cria a coluna benefit_id, do tipo inteiro (INT), que armazenará o identificador único de cada benefício.
- ✓ **employee_id INT**: Cria a coluna employee_id, também do tipo inteiro, que armazenará o ID do funcionário relacionado ao benefício.
- ✓ **benefit_type VARCHAR2(30) NOT NULL**: Cria a coluna benefit_type, que recebe textos com até 30 caracteres para indicar o tipo do benefício. Incluímos a cláusula NOT NULL para impedir que essa coluna aceite valores nulos.
- ✓ **benefit_description VARCHAR2(50) NOT NULL**: Cria a coluna benefit_description, usada para armazenar uma descrição mais detalhada do benefício (até 50 caracteres). Também utilizamos NOT NULL para garantir que essa informação sempre seja preenchida.
- ✓ **start_date DATE DEFAULT SYSDATE**: Cria a coluna start_date, que armazenará a data de início do benefício. Com o DEFAULT SYSDATE, definimos que, se nenhum valor for informado no momento da inserção, a data atual será usada automaticamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

A empresa decidiu registrar os benefícios oferecidos a seus colaboradores em uma nova tabela chamada **BENEFITS**. Crie a tabela BENEFITS no banco de dados **RH** com as colunas necessárias para armazenar o ID do benefício, o ID do funcionário, o tipo de benefício e sua descrição, as datas de início e fim, além do valor mensal. Siga as instruções abaixo:

- **Em relação aos nomes e tipos de dados das colunas:**
  - ✓ A coluna do ID do benefício deverá se chamar **benefit_id** para receber números inteiros;
  - ✓ A coluna do ID do funcionário deverá se chamar **employee_id** para receber números inteiros;
  - ✓ A coluna do tipo de benefício deverá se chamar **benefit_type** para receber textos de até 30 caracteres;
  - ✓ A coluna da descrição do benefício deverá se chamar **benefit_description** para receber textos de até 50 caracteres;
  - ✓ A coluna da data de início do benefício deverá se chamar **start_date** para receber uma data;
  - ✓ A coluna da data final do benefício deverá se chamar **end_date** para receber uma data;
  - ✓ A coluna do valor mensal do benefício deverá se chamar **monthly_value** para receber um valor com até duas casas decimais.

- **Sobre as constraints:**
  - ✓ A coluna benefit_id deve ser a chave primária (chame esta constraint de **benefits_benefit_id_pk**);
  - ✓ A coluna employee_id deve ser uma chave estrangeira que faz referência à tabela employees (chame esta constraint de **benefits_employee_id_fk**);
  - ✓ As colunas benefit_type e benefit_description não podem ser nulas;
  - ✓ Caso não seja informada uma data para a coluna start_date, por padrão, deve ser armazenada a data atual;
  - ✓ A coluna end_date pode ser nula ou conter uma data maior que a data inserida na coluna start_date (chame esta constraint de **benefits_end_date_ck**);
  - ✓ A coluna monthly_value não deve aceitar valor negativos (chame esta constraint de **benefits_monthly_value_ck**).

Explicação e Resultado

- ✓ **end_date DATE**: Cria a coluna end_date, que armazenará a data de término do benefício (caso exista). Essa coluna permite valores nulos.
- ✓ **monthly_value NUMBER(10, 2):** Cria a coluna monthly_value, que armazenará o valor mensal do benefício. O tipo NUMBER(10, 2) indica até 10 dígitos no total, sendo 2 deles reservados para as casas decimais.
- ✓ **CONSTRAINT benefits_benefit_id_pk PRIMARY KEY(benefit_id):** Define a constraint benefits_benefit_id_pk como chave primária da tabela, garantindo que benefit_id seja único e não nulo.
- ✓ **CONSTRAINT benefits_employee_id_fk FOREIGN KEY(employee_id) REFERENCES employees(employee_id):** Cria a constraint benefits_employee_id_fk, que define employee_id como chave estrangeira. Essa coluna faz referência à chave primária (coluna employee_id) da tabela employees.
- ✓ **CONSTRAINT benefits_end_date_ck CHECK (end_date IS NULL OR end_date > start_date):** Define a constraint benefits_end_date_ck, que garante que a end_date seja nula ou maior que a start_date.
- ✓ **CONSTRAINT benefits_monthly_value_ck CHECK(monthly_value >= 0):** Cria a constraint benefits_monthly_value_ck, que impede a inserção de valores negativos na coluna monthly_value.
- ✓ **;** : O ponto e vírgula finaliza o comando SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

A empresa decidiu registrar os benefícios oferecidos a seus colaboradores em uma nova tabela chamada **BENEFITS**. Crie a tabela BENEFITS no banco de dados **RH** com as colunas necessárias para armazenar o ID do benefício, o ID do funcionário, o tipo de benefício e sua descrição, as datas de início e fim, além do valor mensal. Siga as instruções abaixo:

- **Em relação aos nomes e tipos de dados das colunas:**
  - ✓ A coluna do ID do benefício deverá se chamar **benefit_id** para receber números inteiros;
  - ✓ A coluna do ID do funcionário deverá se chamar **employee_id** para receber números inteiros;
  - ✓ A coluna do tipo de benefício deverá se chamar **benefit_type** para receber textos de até 30 caracteres;
  - ✓ A coluna da descrição do benefício deverá se chamar **benefit_description** para receber textos de até 50 caracteres;
  - ✓ A coluna da data de início do benefício deverá se chamar **start_date** para receber uma data;
  - ✓ A coluna da data final do benefício deverá se chamar **end_date** para receber uma data;
  - ✓ A coluna do valor mensal do benefício deverá se chamar **monthly_value** para receber um valor com até duas casas decimais.

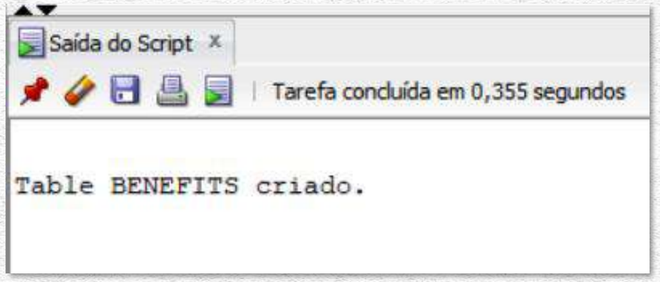
- **Sobre as constraints:**
  - ✓ A coluna benefit_id deve ser a chave primária (chame esta constraint de **benefits_benefit_id_pk**);
  - ✓ A coluna employee_id deve ser uma chave estrangeira que faz referência à tabela employees (chame esta constraint de **benefits_employee_id_fk**);
  - ✓ As colunas benefit_type e benefit_description não podem ser nulas;
  - ✓ Caso não seja informada uma data para a coluna start_date, por padrão, deve ser armazenada a data atual;
  - ✓ A coluna end_date pode ser nula ou conter uma data maior que a data inserida na coluna start_date (chame esta constraint de **benefits_end_date_ck**);
  - ✓ A coluna monthly_value não deve aceitar valor negativos (chame esta constraint de **benefits_monthly_value_ck**).

Explicação e Resultado

Resultado:

A tabela BENEFITS será criada com a estrutura definida.

Ao executar esse comando no SQL Developer, a mensagem de sucesso será exibida:



GABARITO EXPLICATIVO

2

Questão 2

A área de controladoria da empresa alterou sua política de benefícios e definiu que o valor mínimo mensal para qualquer benefício deverá ser de R$100,00.

Altere a constraint existente na tabela BENEFITS para refletir essa nova regra.

Dica: se necessário, exclua a constraint e recrie-a com a nova condição.

Explicação e Resultado

Objetivo:

- ✓ Atualizar a constraint benefits_monthly_value_ck da tabela BENEFITS para exigir que o valor mensal de um benefício seja, no mínimo, R$100,00.

Como o código foi desenvolvido:

```
ALTER TABLE benefits
DROP CONSTRAINT benefits_monthly_value_ck;

ALTER TABLE benefits
ADD CONSTRAINT benefits_monthly_value_ck CHECK(monthly_value >= 100);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A área de controladoria da empresa alterou sua política de benefícios e definiu que o valor mínimo mensal para qualquer benefício deverá ser de R$100,00.

Altere a constraint existente na tabela BENEFITS para refletir essa nova regra.

Dica: se necessário, exclua a constraint e recrie-a com a nova condição.

Explicação e Resultado

Explicação passo a passo:

- ✓ **ALTER TABLE benefits:** Inicia a modificação da estrutura da tabela BENEFITS.
- ✓ **DROP CONSTRAINT benefits_monthly_value_ck;:** Remove a constraint existente que validava o valor da coluna monthly_value. Essa etapa é necessária porque constraints do tipo CHECK não podem ser alteradas diretamente.

- ✓ **ALTER TABLE benefits:** Começa um novo comando para modificar novamente a tabela.
- ✓ **ADD CONSTRAINT benefits_monthly_value_ck CHECK(monthly_value >= 100);:** Adiciona uma nova constraint com o mesmo nome, exigindo agora que o valor da coluna monthly_value seja maior ou igual a 100.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A área de controladoria da empresa alterou sua política de benefícios e definiu que o valor mínimo mensal para qualquer benefício deverá ser de R$100,00.

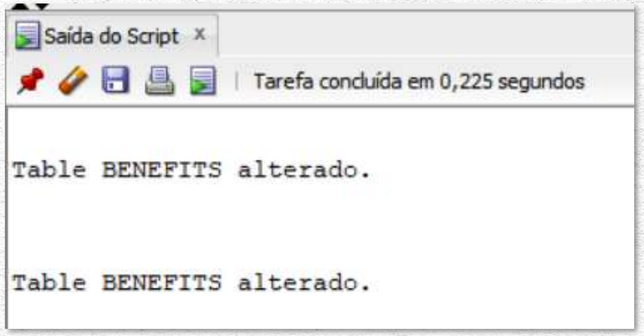
Altere a constraint existente na tabela BENEFITS para refletir essa nova regra.

Dica: se necessário, exclua a constraint e recrie-a com a nova condição.

Explicação e Resultado

Resultado:

A constraint anterior, que permitia valores a partir de 0, foi substituída:



A partir de agora, qualquer tentativa de inserir ou atualizar um valor menor que R$100,00 na coluna monthly_value será rejeitada.

GABARITO EXPLICATIVO

3

Questão 3

Crie uma sequence chamada benefits_seq para ser utilizada na geração automática dos identificadores dos benefícios registrados na tabela BENEFITS.

Ela deve iniciar com o valor 1, incrementar de 1 em 1, não ter valor máximo, não voltar ao início nem armazenar valores na memória cache.

Explicação e Resultado

Objetivo:

Criar uma sequence chamada benefits_seq para gerar valores automáticos para a coluna benefit_id da tabela BENEFITS.

Como o código foi desenvolvido:

```
CREATE SEQUENCE benefits_seq
    START WITH 1
    INCREMENT BY 1
    NOMAXVALUE
    NOCYCLE
    NOCACHE;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Crie uma sequence chamada benefits_seq para ser utilizada na geração automática dos identificadores dos benefícios registrados na tabela BENEFITS.

Ela deve iniciar com o valor 1, incrementar de 1 em 1, não ter valor máximo, não voltar ao início nem armazenar valores na memória cache.

Explicação e Resultado

Explicação passo a passo:

- ✓ **CREATE SEQUENCE benefits_seq:** Cria uma nova sequence com o nome benefits_seq, que será usada para gerar números sequenciais.
- ✓ **START WITH 1:** Define que a sequence começará no valor 1.
- ✓ **INCREMENT BY 1:** Define que cada novo valor gerado aumentará de 1 em relação ao anterior.
- ✓ **NOMAXVALUE:** Define que a sequence não terá um valor máximo, ou seja, continuará gerando valores indefinidamente até atingir o limite do tipo de dado.
- ✓ **NOCACHE:** Indica que os valores da sequence não serão armazenados em cache, garantindo que cada valor seja gerado individualmente.
- ✓ **NOCYCLE:** Garante que a sequence não será reiniciada automaticamente quando atingir seu valor máximo.
- ✓ **; :** O ponto e vírgula finaliza o comando SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

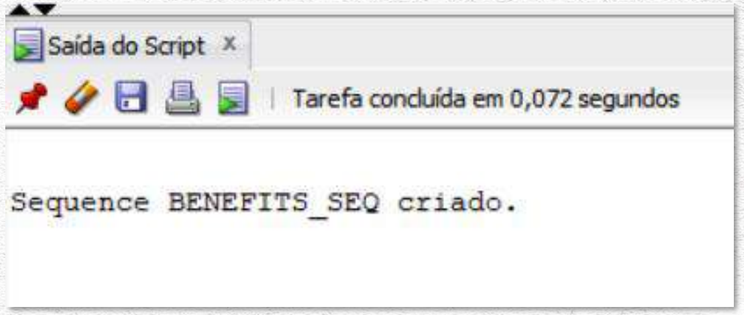
Crie uma sequence chamada benefits_seq para ser utilizada na geração automática dos identificadores dos benefícios registrados na tabela BENEFITS.

Ela deve iniciar com o valor 1, incrementar de 1 em 1, não ter valor máximo, não voltar ao início nem armazenar valores na memória cache.

Explicação e Resultado

Resultado:

A sequence benefits_seq será criada com sucesso:



Ela poderá ser usada para gerar automaticamente valores únicos para a coluna benefit_id nas operações de inserção na tabela BENEFITS.

GABARITO EXPLICATIVO

4 Questão 4

Com base em uma listagem de benefícios fornecida pela área de RH, insira os 20 registros correspondentes na tabela BENEFITS, utilizando a sequence criada no exercício anterior para preencher a coluna benefit_id.

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

Explicação e Resultado

Objetivo:

Inserir 20 registros na tabela BENEFITS utilizando a sequence benefits_seq para preencher automaticamente o campo benefit_id.

Como o código foi desenvolvido:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 118, 'Plano de Saúde', 'Plano Prata da Hashmed', '10/01/2001', NULL, 480.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 118, 'Vale Refeição', 'Cartão Hashcard', '10/01/2001', NULL, 420.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 119, 'Vale Transporte', 'Bilhete Único da Hashtag', '01/02/2002', NULL, 180.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 120, 'Auxílio Educação', 'Bolsa Graduação Hashtalent', '01/03/2002', '28/02/2006', 700.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 121, 'Gympass', 'Plano HashFit', '15/04/2002', NULL, 150.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 122, 'Vale Refeição', 'Cartão Hashcard', '20/06/2003', NULL, 430.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 122, 'Vale Transporte', 'Bilhete Único da Hashtag', '20/06/2003', NULL, 180.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 123, 'Plano de Saúde', 'Plano Ouro da Hashmed', '01/07/2003', NULL, 550.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 124, 'Auxílio Educação', 'Bolsa MBA Hashstrategic', '10/08/2004', '10/08/2005', 900.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 125, 'Vale Refeição', 'Cartão Hashcard', '01/09/2004', NULL, 450.00);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Com base em uma listagem de benefícios fornecida pela área de RH, insira os 20 registros correspondentes na tabela BENEFITS, utilizando a sequence criada no exercício anterior para preencher a coluna benefit_id.

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

Explicação e Resultado

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 126, 'Gympass', 'Plano HashGold', '12/10/2005', '12/10/2006', 130.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 126, 'Plano de Saúde', 'Plano Bronze da Hashmed', '12/10/2005', NULL, 400.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 126, 'Vale Transporte', 'Bilhete Único da Hashtag', '05/01/2006', NULL, 180.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 127, 'Plano de Saúde', 'Plano Prata da Hashmed', '01/02/2006', '09/10/2010', 490.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 128, 'Auxílio Educação', 'Bolsa Hashtag para Pós-Tech', '15/03/2007', '14/03/2009', 750.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 128, 'Vale Refeição', 'Cartão Hashcard', '15/03/2007', NULL, 460.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 129, 'Vale Transporte', 'Bilhete Único da Hashtag', '01/04/2007', NULL, 180.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 130, 'Gympass', 'Plano HashFit', '05/06/2008', NULL, 140.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 130, 'Plano de Saúde', 'Plano Ouro da Hashmed', '05/06/2008', NULL, 550.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 130, 'Auxílio Educação', 'Bolsa Hashtag para Líderes', '10/10/2009', '10/10/2011', 820.00);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Com base em uma listagem de benefícios fornecida pela área de RH, insira os 20 registros correspondentes na tabela BENEFITS, utilizando a sequence criada no exercício anterior para preencher a coluna benefit_id.

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

Explicação e Resultado

Vamos entender o primeiro INSERT:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 118, 'Plano de Saúde', 'Plano Prata da Hashmed', '10/01/2001', NULL, 480.00);
```

- ✓ **INSERT INTO benefits (...)**: Esse comando insere um novo registro na tabela BENEFITS. Especificamos as colunas a serem preenchidas entre parênteses.
- ✓ **VALUES (...)**: Informamos os valores correspondentes às colunas, na mesma ordem em que foram listadas, separados por vírgulas.
- ✓ Repare que:
  - O benefit_id será preenchido automaticamente com o próximo valor da sequence benefits_seq (benenifs_seq.NEXTVAL);
  - Os valores numéricos são passados sem aspas. Havendo casas decimais, elas são separadas por um ponto (.);
  - As strings (textos) são informadas entre aspas simples;
  - As datas são informadas entre aspas simples, no formato padrão DD/MM/YYYY;
  - Como o benefício ainda está vigente, o end_date está indefinido, por isso usamos NULL.
- ✓ ; : O ponto e vírgula encerra o comando.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4 Questão 4

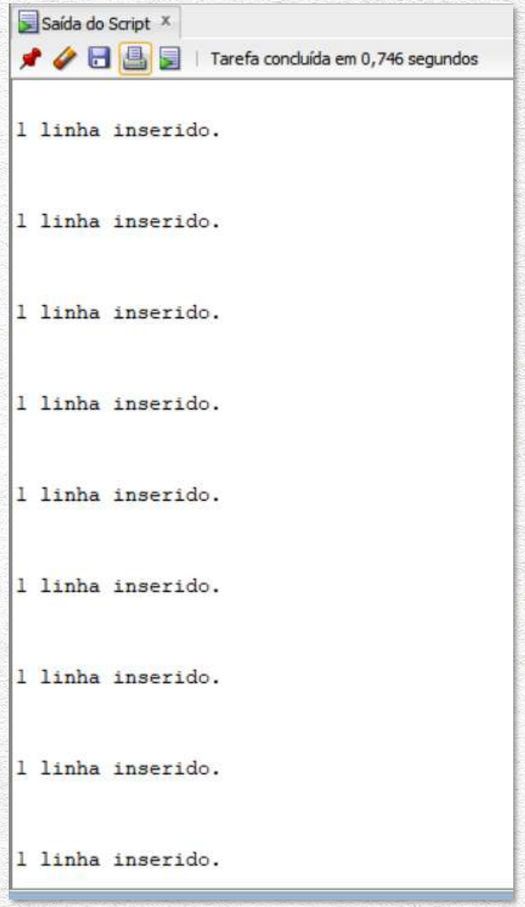
Com base em uma listagem de benefícios fornecida pela área de RH, insira os 20 registros correspondentes na tabela BENEFITS, utilizando a sequence criada no exercício anterior para preencher a coluna benefit_id.

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

Explicação e Resultado

Resultado:

Ao executarmos os 20 INSERTs, 20 registros (linhas) serão inseridos na tabela BENEFITS com todos os dados especificados:



CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

4 Questão 4

Com base em uma listagem de benefícios fornecida pela área de RH, insira os 20 registros correspondentes na tabela BENEFITS, utilizando a sequence criada no exercício anterior para preencher a coluna benefit_id.

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

Explicação e Resultado

Se fizermos uma consulta à tabela BENEFITS, notaremos isso:

```
SELECT * FROM benefits;
```

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480
2	118	Vale Refeição	Cartão Hashcard	10/01/01	(null)	420
3	119	Vale Transporte	Bilhete Único da Hashtag	01/02/02	(null)	180
4	120	Auxílio Educação	Bolsa Graduação Hashtalent	01/03/02	28/02/06	700
5	121	Gympass	Plano HashFit	15/04/02	(null)	150
6	122	Vale Refeição	Cartão Hashcard	20/06/03	(null)	430
7	122	Vale Transporte	Bilhete Único da Hashtag	20/06/03	(null)	180
8	123	Plano de Saúde	Plano Ouro da Hashmed	01/07/03	(null)	550
9	124	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	125	Vale Refeição	Cartão Hashcard	01/09/04	(null)	450
11	126	Gympass	Plano HashGold	12/10/05	12/10/06	130
12	126	Plano de Saúde	Plano Bronze da Hashmed	12/10/05	(null)	400
13	126	Vale Transporte	Bilhete Único da Hashtag	05/01/06	(null)	180
14	127	Plano de Saúde	Plano Prata da Hashmed	01/02/06	09/10/10	490
15	128	Auxílio Educação	Bolsa Hashtag para Pós-Tech	15/03/07	14/03/09	750
16	128	Vale Refeição	Cartão Hashcard	15/03/07	(null)	460
17	129	Vale Transporte	Bilhete Único da Hashtag	01/04/07	(null)	180
18	130	Gympass	Plano HashFit	05/06/08	(null)	140
19	130	Plano de Saúde	Plano Ouro da Hashmed	05/06/08	(null)	550
20	130	Auxílio Educação	Bolsa Hashtag para Líderes	10/10/09	10/10/11	820

GABARITO EXPLICATIVO

5

Questão 5

Insira o novo benefício especificado abaixo na tabela BENEFITS, omitindo os valores das colunas start_date e end_date. Em seguida, observe o que é atribuído automaticamente para essas colunas. Explique o motivo de cada resultado com base na definição da tabela:

- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 131;
- ✓ Tipo do benefício: Auxílio Educação;
- ✓ Descrição do benefício: Bolsa de Especialização Impressionadora;
- ✓ Valor mensal: R$ 800,00.

Explicação e Resultado

Objetivo:

Inserir um novo benefício na tabela BENEFITS, omitindo as colunas start_date e end_date, para observar como esses campos se comportam automaticamente com base na definição da tabela.

Como o código foi desenvolvido:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, monthly_value)
VALUES (benefits_seq.NEXTVAL, 131, 'Auxílio Educação', 'Bolsa de Especialização Impressionadora', 800.00);
```

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

5

Questão 5

Insira o novo benefício especificado abaixo na tabela BENEFITS, omitindo os valores das colunas start_date e end_date. Em seguida, observe o que é atribuído automaticamente para essas colunas. Explique o motivo de cada resultado com base na definição da tabela:

- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 131;
- ✓ Tipo do benefício: Auxílio Educação;
- ✓ Descrição do benefício: Bolsa de Especialização Impressionadora;
- ✓ Valor mensal: R$ 800,00.

Explicação e Resultado

Explicação passo a passo:

- ✓ **INSERT INTO benefits (...):** Inicia a inserção de um novo registro na tabela BENEFITS. As colunas a serem preenchidas são listadas entre parênteses.
- ✓ **benefit_id:** Está sendo preenchido automaticamente com o próximo valor da sequence benefits_seq, por meio do uso de benefits_seq.NEXTVAL.
- ✓ **employee_id:** Está sendo definido com o valor 131, correspondente ao ID do funcionário que receberá o benefício.
- ✓ **benefit_type:** Informado como 'Auxílio Educação', especificando o tipo do benefício.
- ✓ **benefit_description:** Informado como 'Bolsa de Especialização Impressionadora', descrevendo com mais detalhes o benefício.
- ✓ **monthly_value:** Informado como 800.00, valor mensal do benefício.
- ✓ **; :** O ponto e vírgula finaliza o comando SQL.

Repare que start_date e end_date foram omitidas no comando.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

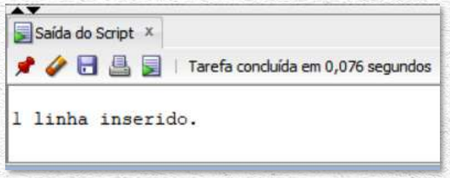
Insira o novo benefício especificado abaixo na tabela BENEFITS, omitindo os valores das colunas start_date e end_date. Em seguida, observe o que é atribuído automaticamente para essas colunas. Explique o motivo de cada resultado com base na definição da tabela:

- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 131;
- ✓ Tipo do benefício: Auxílio Educação;
- ✓ Descrição do benefício: Bolsa de Especialização Impressionadora;
- ✓ Valor mensal: R$ 800,00.

Explicação e Resultado

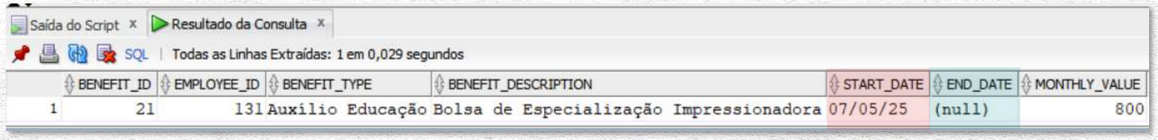
Resultado:

Ao executar o comando, uma nova linha será inserida na tabela BENEFITS com os dados fornecidos:



Ao fazermos um SELECT na tabela BENEFITS, percebemos que a coluna start_date foi preenchida automaticamente com a data atual e end_date ficou como NULL:

```
SELECT * FROM benefits
WHERE employee_id = 131;
```



	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	21	131	Auxílio Educação	Bolsa de Especialização Impressionadora	07/05/25	(null)	800

Isso ocorreu porque a coluna start_date tem a cláusula DEFAULT SYSDATE, que preenche automaticamente com a data atual se nenhum valor for especificado. Já a coluna end_date permite NULL, e como não foi informada no comando, permanece sem valor.

GABARITO EXPLICATIVO

6

Questão 6

Tente atualizar o campo end_date do benefit_id = 1 com a data '10/12/2000'.

Observe o que aconteceu e explique por que ocorreu.

Explicação e Resultado

Objetivo:

Tentar atualizar a coluna end_date do registro com benefit_id = 1 para a data '10/12/2000', e analisar o que acontece com base nas regras da tabela.

Como o código foi desenvolvido:

```
UPDATE benefits
SET end_date = '10/12/2000'
WHERE benefit_id = 1;
```

Explicação passo a passo:

- ✓ **UPDATE benefits:** Inicia o comando para atualizar registros na tabela BENEFITS.
- ✓ **SET end_date = '10/12/2000':** Define que o campo end_date deve receber o valor '10/12/2000'.
- ✓ **WHERE benefit_id = 1:** Especifica que a atualização deve ocorrer somente para o registro com benefit_id igual a 1.
- ✓ **; :** O ponto e vírgula finaliza o comando SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

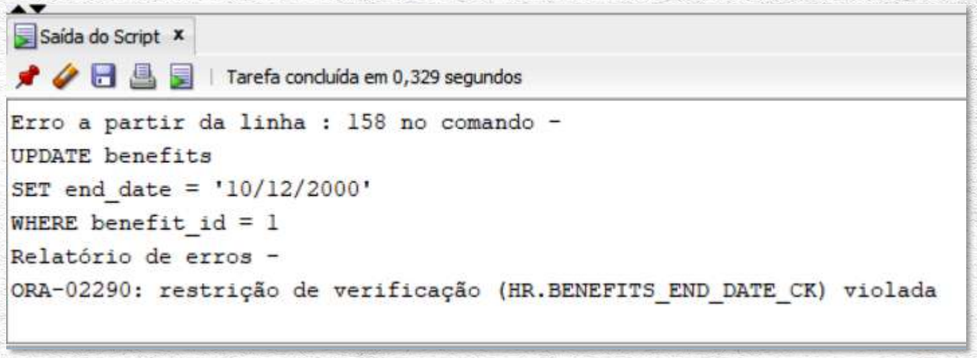
Tente atualizar o campo end_date do benefit_id = 1 com a data '10/12/2000'.

Observe o que aconteceu e explique por que ocorreu.

Explicação e Resultado

Resultado:

Ao executar esse comando, o SQL Developer exibiu uma mensagem de erro indicando que a atualização violou uma constraint:



Por que isso ocorreu?

A tabela BENEFITS possui uma constraint CHECK chamada benefits_end_date_ck, que garante a regra mostrada abaixo. Ou seja: o campo da coluna end_date pode ser nulo ou maior que o valor existente na coluna start_date.

```
CONSTRAINT benefits_end_date_ck CHECK (end_date IS NULL OR end_date > start_date),
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

Tente atualizar o campo end_date do benefit_id = 1 com a data '10/12/2000'.

Observe o que aconteceu e explique por que ocorreu.

Explicação e Resultado

Se verificarmos o registro que tentamos alterar (do benefit_id = 1), veremos que a start_date dele é '10/01/2001':

```
SELECT * FROM benefits
WHERE benefit_id = 1;
```

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,012 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	118	Plano de Saúde	Plano Prata da Hashmed	10/01/01	(null)	480

Portanto, a data que tentamos inserir na coluna end_date ('10/12/2000') é anterior à data armazenada na coluna start_date. Sendo assim, a regra da constraint benefits_end_date_ck foi violada, por isso a atualização não foi efetuada.

GABARITO EXPLICATIVO

7

Questão 7

Tente inserir um novo registro na tabela BENEFITS com os dados especificados abaixo:

- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 132;
- ✓ Tipo do benefício: Gympass;
- ✓ Descrição do benefício: Plano HashSilver;
- ✓ Data de início: 05/06/2010;
- ✓ Data final: Nula;
- ✓ Valor mensal: R$ 90,00.

O que acontece com essa operação? Por quê?

Explicação e Resultado

Objetivo: Tentar inserir um novo benefício com valor mensal de R$ 90,00, para entender o comportamento da constraint.

Como o código foi desenvolvido:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (benefits_seq.NEXTVAL, 132, 'Gympass', 'Plano HashSilver', '05/06/2010', NULL, 90.00);
```

Explicação passo a passo:

- ✓ **INSERT INTO benefits (...):** Inicia o comando de inserção de um novo registro na tabela BENEFITS, listando as colunas que serão preenchidas.
- ✓ **VALUES (...):** Define os valores das colunas informadas, na mesma ordem.
- ✓ **benefit_id:** Será preenchido automaticamente com o próximo valor da sequence benefits_seq.
- ✓ **employee_id:** Recebe o valor 132, identificando o funcionário que receberá o benefício.
- ✓ **benefit_type:** Informado como 'Gympass'.
- ✓ **benefit_description:** Informado como 'Plano HashSilver'.
- ✓ **start_date:** Definida como '05/06/2010'.
- ✓ **end_date:** Informada como NULL, já que o benefício ainda está ativo.
- ✓ **monthly_value:** Definido como 90.00, valor mensal do benefício.
- ✓ **; :** O ponto e vírgula finaliza o comando SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Tente inserir um novo registro na tabela BENEFITS com os dados especificados abaixo:

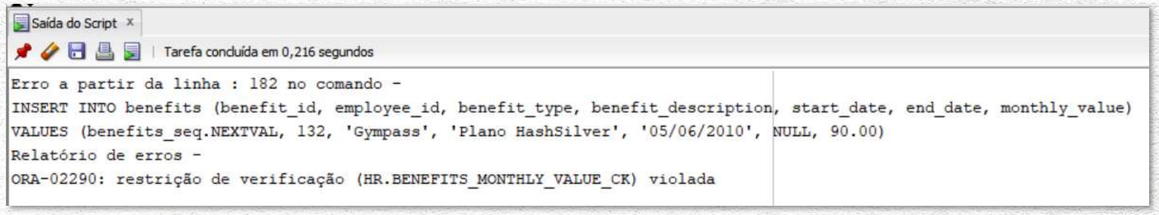
- ✓ ID do benefício: Utilize a sequence;
- ✓ ID do funcionário: 132;
- ✓ Tipo do benefício: Gympass;
- ✓ Descrição do benefício: Plano HashSilver;
- ✓ Data de início: 05/06/2010;
- ✓ Data final: Nula;
- ✓ Valor mensal: R$ 90,00.

O que acontece com essa operação? Por quê?

Explicação e Resultado

Resultado:

Ao executar o comando, ocorre um erro no SQL Developer, e o registro não é inserido na tabela.



Por que isso ocorreu?

A tabela BENEFITS possui uma constraint CHECK chamada benefits_monthly_value_ck, que impõe a seguinte regra:

```
CONSTRAINT benefits_monthly_value_ck CHECK(monthly_value >= 100);
```

Como tentamos inserir o valor 90.00, que é menor do que o valor mínimo permitido, a constraint foi violada e a operação foi cancelada automaticamente para preservar a integridade dos dados.

GABARITO EXPLICATIVO

8

Questão 8

Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130.

Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação.

O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

Explicação e Resultado

Objetivo:

Avaliar o impacto da criação de um índice na performance de uma consulta na tabela BENEFITS.

Como o código foi desenvolvido:

```
-- Consulta antes da criação do índice (utilizando o plano de explicação)
SELECT * FROM benefits
WHERE employee_id IN (126, 130);

-- Criação do índice
CREATE INDEX benefit_emp_ix
ON benefits(employee_id);

-- Consulta após a criação do índice (utilizando o plano de explicação)
SELECT * FROM benefits
WHERE employee_id IN (126, 130);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130.

Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação.

O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

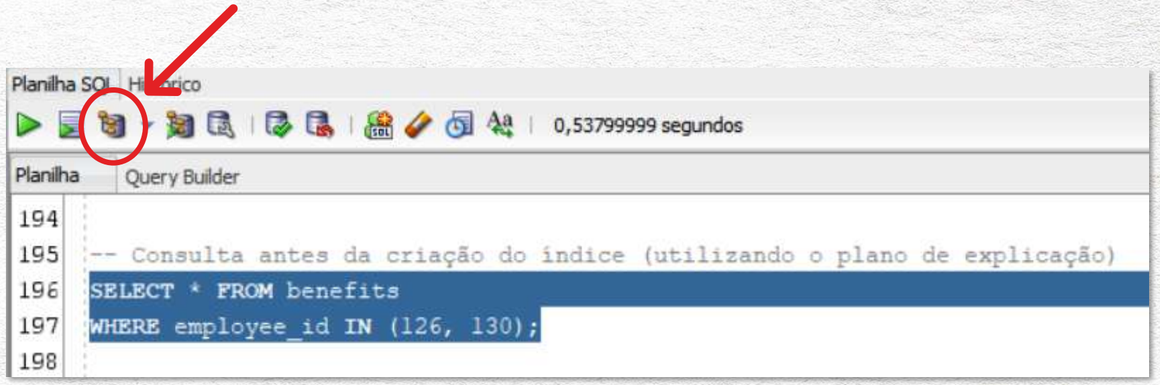
Explicação e Resultado

Explicação passo a passo:

Consulta inicial:

Busca todos os registros da tabela BENEFITS onde o employee_id é 126 ou 130.

Devemos executá-la clicando no ícone Plano de Explicação (ou pressionando a tecla F10):



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

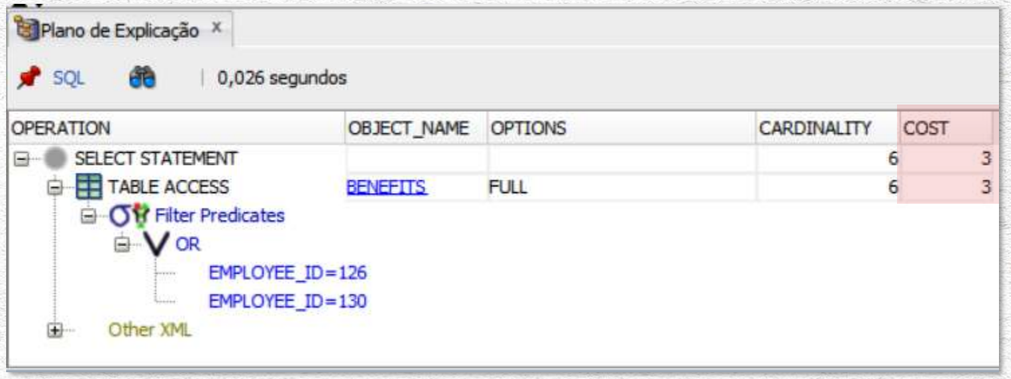
Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130.

Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação.

O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

Explicação e Resultado

Neste momento, a coluna employee_id ainda não possui índice, então o Oracle fará a leitura completa da tabela para encontrar os resultados, o que resultará em um COST de valor 3 para o banco:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130.

Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação.

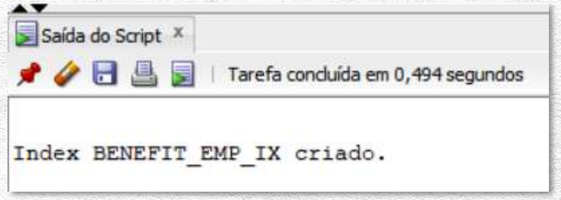
O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

Explicação e Resultado

Criação do índice:

Então, criamos o índice que permitirá que o Oracle acesse os registros mais rapidamente quando a consulta fizer filtros com base na coluna employee_id:

```
-- Criação do índice
CREATE INDEX benefit_emp_ix
ON benefits(employee_id);
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130.

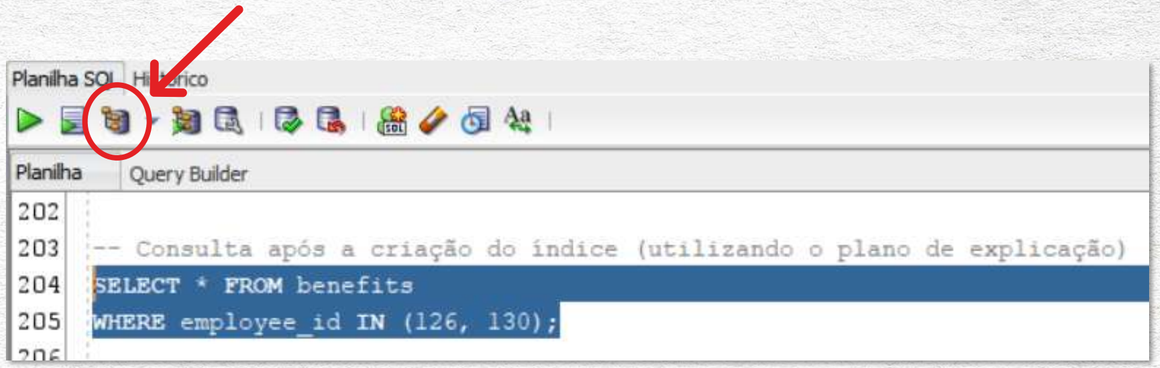
Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação.

O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

Explicação e Resultado

Consulta final:

Em seguida, executamos a mesma consulta novamente, mais uma vez clicando em Plano de Explicação (ou pressionando a tecla F10):



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Realize uma consulta filtrando a tabela BENEFITS pelos funcionários com employee_id 126 e 130.

Em seguida, crie um índice na coluna employee_id chamado benefit_emp_ix, repita a mesma consulta e compare o custo da operação nas duas execuções utilizando o plano de explicação.

O que você pode observar? O índice benefit_emp_ix conseguiu otimizar a consulta?

Explicação e Resultado

Agora, como existe um índice, o Oracle pode usá-lo para acessar diretamente os registros desejados, evitando a leitura completa da tabela. E isso faz com que o COST da operação caia de 3 (sem o índice) para 2 (com o índice):

Portanto: sim, a criação do índice benefit_emp_ix otimizou a performance da consulta. Isso é confirmado pela redução do custo no plano de explicação, que caiu de 3 para 2. Com o índice, o Oracle conseguiu acessar os dados de forma mais eficiente, o que é especialmente vantajoso em tabelas com grande volume de dados.

GABARITO EXPLICATIVO

9

Questão 9

Remova o índice criado no exercício anterior (benefit_emp_ix) da tabela BENEFITS.

Explicação e Resultado

Objetivo:

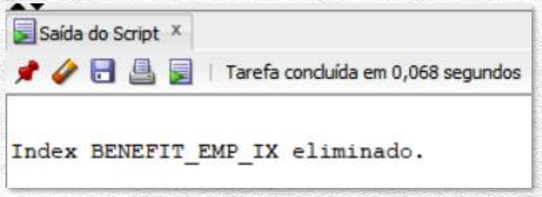
Remover o índice benefit_emp_ix da tabela BENEFITS.

Como o código foi desenvolvido:

```
DROP INDEX benefit_emp_ix;
```

Esse comando remove o índice chamado benefit_emp_ix da base de dados.

Ao executar o comando acima no SQL Developer, a mensagem confirmando a remoção do índice é exibida:



Após sua remoção, a coluna employee_id deixa de ter acesso otimizado via esse índice, e o Oracle volta a precisar fazer a leitura completa da tabela para encontrar os resultados.

GABARITO EXPLICATIVO

10

Questão 10

Remova a sequence benefits_seq (criada anteriormente para geração dos IDs) da tabela BENEFITS.

Explicação e Resultado

Objetivo:

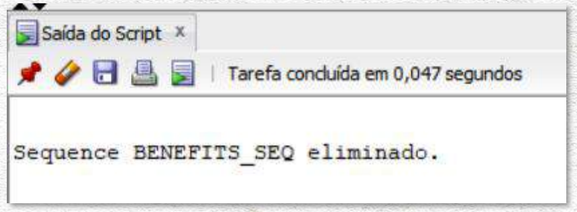
Remover a sequence benefits_seq do banco de dados.

Como o código foi desenvolvido:

```
DROP SEQUENCE benefits_seq;
```

Esse comando exclui do banco a sequence chamada benefits_seq, que era usada para gerar valores automáticos para a coluna benefit_id.

Ao executar o comando no SQL Developer, a mensagem de remoção da sequence será exibida:



GABARITO EXPLICATIVO

11

Questão 11

Exclua a tabela BENEFITS garantindo que quaisquer constraints que possam estar referenciando essa tabela também sejam removidas automaticamente.

Explicação e Resultado

Objetivo:

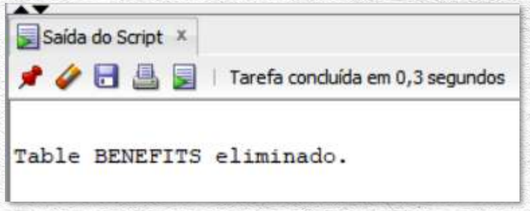
Excluir a tabela BENEFITS e remover automaticamente quaisquer constraints que a referenciem.

Como o código foi desenvolvido:

```
DROP TABLE benefits CASCADE CONSTRAINTS;
```

Esse comando exclui completamente a tabela BENEFITS do banco de dados e, com o uso da cláusula CASCADE CONSTRAINTS, garante que todas as constraints eventualmente existentes em outras tabelas que estejam associadas a ela (como chaves estrangeiras) também sejam removidas automaticamente.

Ao executar o comando no SQL Developer, será exibida a mensagem de confirmação da exclusão:



MÓDULO 12

VIEWS

VIEWS

VIEWS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 12 VIEWS

01	VIEWS	566
02	CRIANDO / SUBSTITUINDO VIEWS	567
03	EXCLUINDO VIEWS	570
04	RESUMO DO MÓDULO	571
05	EXERCÍCIOS	572
06	GABARITO EXPLICATIVO	577

VIEWS

Uma VIEW nada mais é do que um objeto salvo dentro de um banco de dados para armazenar o resultado de um SELECT.

Vantagens de se utilizar VIEWS:

1) RESTRINGIR ACESSO AOS DADOS:

Por exemplo: se quisermos dar acesso à tabela EMPLOYEES a um usuário, mas não à coluna de salário, podemos criar uma VIEW a partir desta tabela, sem selecionar a coluna de salário, e dar a este usuário acesso apenas à VIEW.

2) REUTILIZAR CONSULTAS COMPLEXAS:

Se utilizamos com frequência uma consulta que possui muitos Joins, Where, Group By, etc., em vez de recriar o código toda vez que precisarmos efetuar essa consulta, podemos criar uma VIEW e reaproveitar esse código de forma mais simples.

3) REPRESENTAR DIFERENTES VISÕES DE UMA MESMA TABELA:

Podemos fazer diferentes filtros em uma mesma tabela, para que diferentes visões sejam compartilhadas entre diferentes usuários.

Criando / Substituindo VIEWS

Para criarmos ou substituirmos uma VIEW, utilizamos o comando CREATE OR REPLACE VIEW.

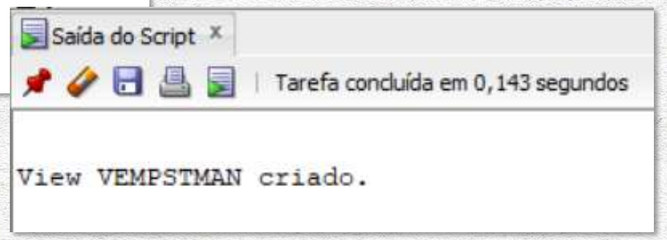
Utilizamos o CREATE para criar e o REPLACE para substituir uma VIEW.

Se utilizamos os dois comandos no mesmo código, fica mais prático, pois, caso a VIEW ainda não exista em nosso banco de dados, ela é criada; se ela já existir, é substituída.

Veja este exemplo:

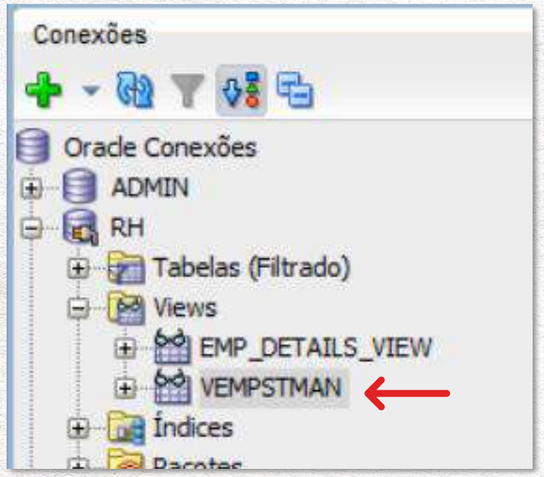
Vamos criar uma VIEW armazenando o resultado dos funcionários cujo JOB_ID = 'ST_MAN'.

```
CREATE OR REPLACE VIEW vEmpSTMan AS
SELECT *
FROM employees
WHERE job_id = 'ST_MAN';
```



Criando / Substituindo VIEWS

Uma vez criada, podemos ter acesso aos dados da VIEW clicando sobre o submenu “Views” do menu lateral à esquerda da tela:



Com isso, teremos acesso a diversas informações sobre a VIEW criada, como as colunas relacionadas, os registros (linhas), as dependências (originou-se a partir de qual(is) tabela(s)), o código SQL utilizado, etc.):

VEMPSTMAN

Colunas | Dados | Concessões | Dependências | Detalhes | Triggers | SQL | Erros

Ações...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS	INSERTABLE	UPDATABLE	DELETABLE
1	EMPLOYEE_ID	NUMBER(6)	No	(null)	1	(null)	YES	YES	YES
2	FIRST_NAME	VARCHAR2(20)	Yes	(null)	2	(null)	YES	YES	YES
3	LAST_NAME	VARCHAR2(25)	No	(null)	3	(null)	YES	YES	YES
4	EMAIL	VARCHAR2(25)	No	(null)	4	(null)	YES	YES	YES
5	PHONE_NUMBER	VARCHAR2(20)	Yes	(null)	5	(null)	YES	YES	YES
6	HIRE_DATE	DATE	No	(null)	6	(null)	YES	YES	YES
7	JOB_ID	VARCHAR2(10)	No	(null)	7	(null)	YES	YES	YES
8	SALARY	NUMBER(8,2)	Yes	(null)	8	(null)	YES	YES	YES
9	COMMISSION_PCT	NUMBER(2,2)	Yes	(null)	9	(null)	YES	YES	YES
10	MANAGER_ID	NUMBER(6)	Yes	(null)	10	(null)	YES	YES	YES
11	DEPARTMENT_ID	NUMBER(4)	Yes	(null)	11	(null)	YES	YES	YES

Criando / Substituindo VIEWS

Para consultar uma VIEW, utilizamos o comando SELECT e informamos o nome da VIEW:

```
SELECT * FROM vEmpSTMan;
```

Resultado da Consulta

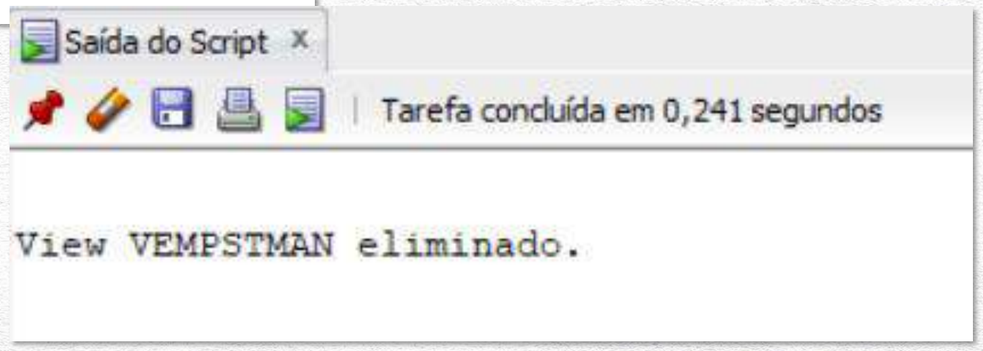
SQL | Todas as Linhas Extraídas: 5 em 0,01 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	120	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	(null)	100	50
2	121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	50
3	122	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	(null)	100	50
4	123	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	(null)	100	50
5	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16/11/99	ST_MAN	5800	(null)	100	50

Excluindo VIEWS

Para excluir uma VIEW, utilizamos o comando DROP VIEW e informamos o nome da VIEW:

```
DROP VIEW vEmpSTMan;
```



Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. VIEWS são objetos dentro de um banco de dados que armazenam o resultado de um SELECT.

2. Criamos VIEWS para: restringir acesso aos dados, reutilizar consultas complexas e criar diferentes visões de uma mesma tabela.

3. Usamos o comando CREATE OR REPLACE VIEW para criar/substituir VIEWS.

4. Informações de uma VIEW (como colunas, dependências e o código SQL utilizado) podem ser vistas quando clicamos na VIEW dentro da pasta VIEWS do banco de dados em que ela foi criada, no menu lateral à esquerda da tela.

5. Usamos o comando DROP VIEW para excluir VIEWS.

EXERCÍCIOS

1

Questão 1

O diretor do departamento de Executivos (ID 90) participa de reuniões frequentes com o RH e solicita relatórios recorrentes sobre os colaboradores sob sua supervisão. Para evitar a repetição de consultas manuais, o setor de RH deseja uma visualização reutilizável com todos os funcionários desse departamento.

Crie uma view chamada *v_func_depto_90* para atender a essa demanda.

(Utilize a tabela employees)

2

Questão 2

A área de compliance está conduzindo uma auditoria para verificar a conformidade dos processos de admissão realizados ao longo dos anos. Para isso, o RH precisa de uma lista prática e atualizada dos funcionários contratados após o ano 1995.

Crie uma view chamada *v_contratados_apos_1995* com os funcionários contratados a partir de 01 de janeiro de 1995.

(Utilize a tabela employees)

EXERCÍCIOS

3

Questão 3

A controladoria está analisando a distribuição salarial da empresa e precisa identificar os cinco maiores salários para avaliar o impacto desses cargos no orçamento da folha de pagamento.

Crie uma view chamada *v_top5_salarios* com os 5 funcionários mais bem pagos, exibindo nome, sobrenome, salário e ID do cargo.

(Utilize a tabela employees)

4

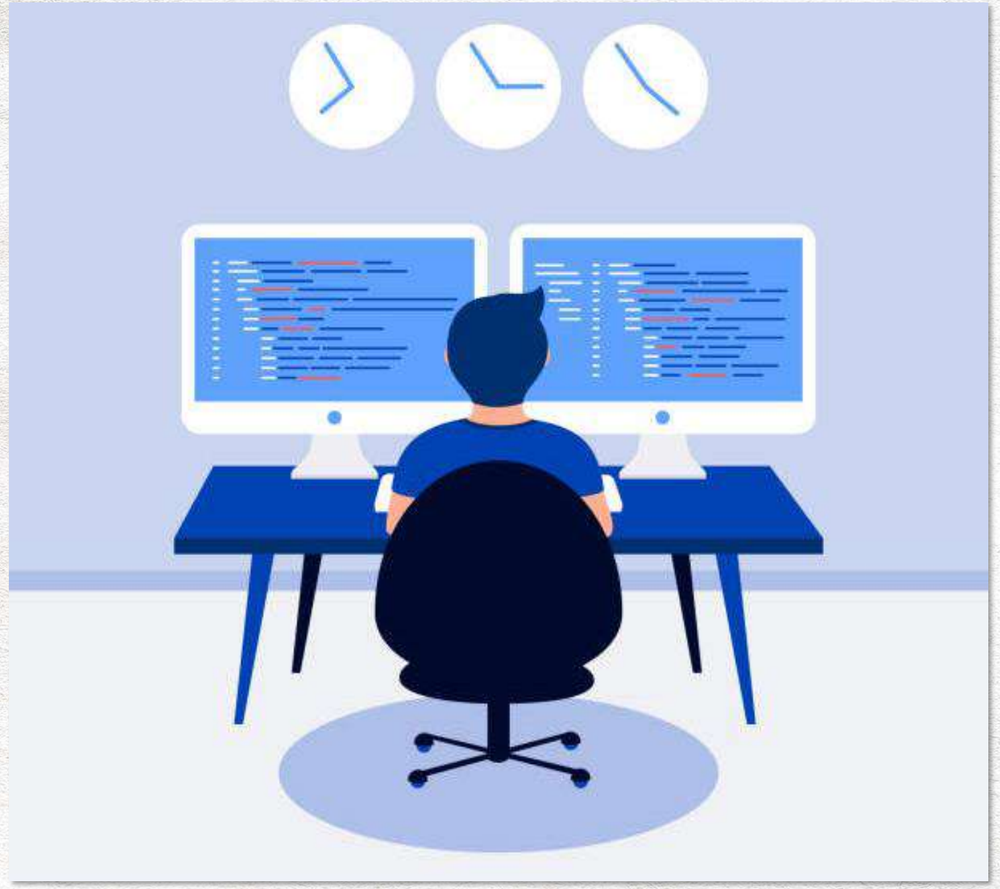
Questão 4

O time de planejamento estratégico solicitou um levantamento com a quantidade de funcionários por departamento, com o objetivo de realocar recursos humanos conforme a demanda de cada setor.

Crie uma view chamada *v_qtd_func_por_depto*, mostrando o ID do departamento, o nome do departamento e a quantidade de funcionários por departamento.

(Utilize as tabelas employees e departments)

EXERCÍCIOS



5

Questão 5

A equipe de remuneração está elaborando uma nova política salarial. Para isso, precisa de uma visualização com o teto salarial de cada cargo, facilitando a análise comparativa entre as funções com maior potencial de remuneração.

Crie uma view chamada *v_teto_salarial_cargos* com o título do cargo e o teto salarial de cada cargo, ordenando do maior para o menor salário.

(Utilize a tabela jobs)

6

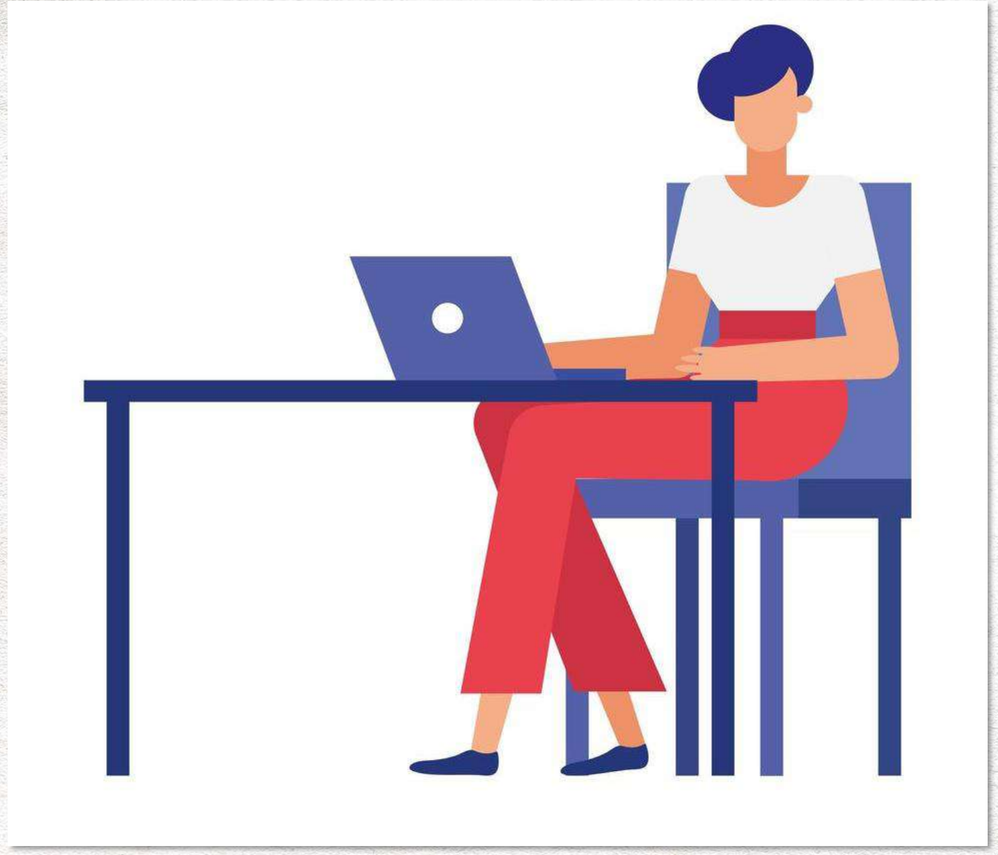
Questão 6

A diretoria internacional deseja revisar a distribuição geográfica das operações. Como ponto de partida, pediu ao setor de operações um relatório com o número total de localizações da empresa em cada país.

Crie uma view chamada *v_qtd_loc_por_pais* que exiba, para cada país, o número total de localizações cadastradas.

(Utilize a tabela locations)

EXERCÍCIOS



7

Questão 7

O setor de desenvolvimento organizacional está criando um mapa de lideranças. Para isso, precisa de uma visualização que relacione cada funcionário ao seu gerente direto e ao departamento onde trabalha.

Crie uma view chamada *v_func_gerente_depto* que mostre o nome completo do funcionário e de seu gerente, assim como o nome do departamento onde trabalha.

(Utilize as tabelas employees e departments)

8

Questão 8

A equipe de Relações Internacionais está preparando um relatório sobre os colaboradores alocados em filiais fora dos Estados Unidos, com foco na estrutura organizacional e distribuição global.

Crie uma view chamada *v_func_fora_eua* com todos os funcionários que trabalham fora dos Estados Unidos. Considere que os países fora dos EUA têm country_id diferente de 'US'. Retorne todas as informações armazenadas desses funcionários, os nomes dos departamentos em que trabalham e os IDs de seus países.

(Utilize as tabelas employees, departments e locations)

EXERCÍCIOS

9

Questão 9

O setor de Facilities está revisando a ocupação dos espaços físicos e precisa identificar quais departamentos possuem mais de 3 funcionários para planejar alocações de salas e estações de trabalho.

Crie uma view chamada *v_deptos_mais_3_func* com os nomes dos departamentos que possuem mais de 3 funcionários alocados e suas respectivas quantidades de funcionários.

(Utilize as tabelas departments e employees)

10

Questão 10

Após a apresentação dos relatórios solicitados, os dados foram aprovados e arquivados. Para manter o ambiente de banco de dados limpo e otimizado, decidiu-se excluir as views criadas para essas análises pontuais.

Exclua todas as views criadas nos exercícios anteriores.

GABARITO EXPLICATIVO

1

Questão 1

O diretor do departamento de Executivos (ID 90) participa de reuniões frequentes com o RH e solicita relatórios recorrentes sobre os colaboradores sob sua supervisão. Para evitar a repetição de consultas manuais, o setor de RH deseja uma visualização reutilizável com todos os funcionários desse departamento.

Crie uma view chamada *v_func_depto_90* para atender a essa demanda.

(Utilize a tabela employees)

Explicação e Resultado

Para criar uma visualização reutilizável com todos os funcionários do departamento 90, seguimos os seguintes passos:

- ✓ Utilizamos o comando CREATE OR REPLACE VIEW para criar (ou atualizar) uma view chamada v_func_depto_90.
- ✓ Definimos o conteúdo da view como o resultado de uma consulta na tabela employees, filtrando apenas os registros cujo department_id seja 90:

```
CREATE OR REPLACE VIEW v_func_depto_90 AS
SELECT *
FROM employees
WHERE department_id = 90;

SELECT * FROM v_func_depto_90;
```

```
Saída do Script ×
Tarefa concluída em 0,123 segundos

View V_FUNC_DEPTO_90 criado.
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O diretor do departamento de Executivos (ID 90) participa de reuniões frequentes com o RH e solicita relatórios recorrentes sobre os colaboradores sob sua supervisão. Para evitar a repetição de consultas manuais, o setor de RH deseja uma visualização reutilizável com todos os funcionários desse departamento.

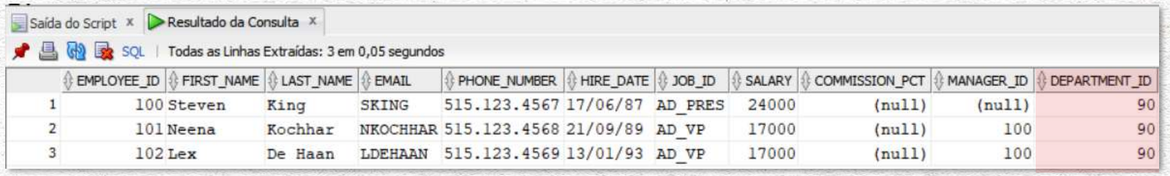
Crie uma view chamada *v_func_depto_90* para atender a essa demanda.

(Utilize a tabela employees)

Explicação e Resultado

Para visualizar os dados da view criada, utilizamos um SELECT * sobre a v_func_depto_90:

```
SELECT * FROM v_func_depto_90;
```


Saída do Script | Resultado da Consulta

Todas as Linhas Extraídas: 3 em 0,05 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90

Dessa forma, o RH poderá acessar rapidamente essa visualização sempre que precisar consultar os funcionários do departamento 90, sem reescrever a consulta. Isso agiliza a rotina de geração de relatórios e mantém a consistência da informação.

GABARITO EXPLICATIVO

2

Questão 2

A área de compliance está conduzindo uma auditoria para verificar a conformidade dos processos de admissão realizados ao longo dos anos. Para isso, o RH precisa de uma lista prática e atualizada dos funcionários contratados após o ano 1995.

Crie uma view chamada *v_contratados_apos_1995* com os funcionários contratados a partir de 01 de janeiro de 1995.

(Utilize a tabela employees)

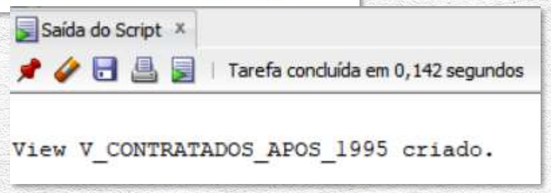
Explicação e Resultado

Para criar uma view com todos os funcionários contratados a partir de 1º de janeiro de 1995, seguimos os passos abaixo:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar uma view chamada **v_contratados_apos_1995**.

Dentro da view, fazemos uma seleção completa da tabela **employees**, aplicando o filtro **WHERE hire_date >= TO_DATE('01/01/1995', 'DD/MM/YYYY')** para considerar somente os funcionários contratados a partir da data especificada:

```
CREATE OR REPLACE VIEW v_contratados_apos_1995 AS
SELECT *
FROM employees
WHERE hire_date >= TO_DATE('01/01/1995', 'DD/MM/YYYY');
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A área de compliance está conduzindo uma auditoria para verificar a conformidade dos processos de admissão realizados ao longo dos anos. Para isso, o RH precisa de uma lista prática e atualizada dos funcionários contratados após o ano 1995.

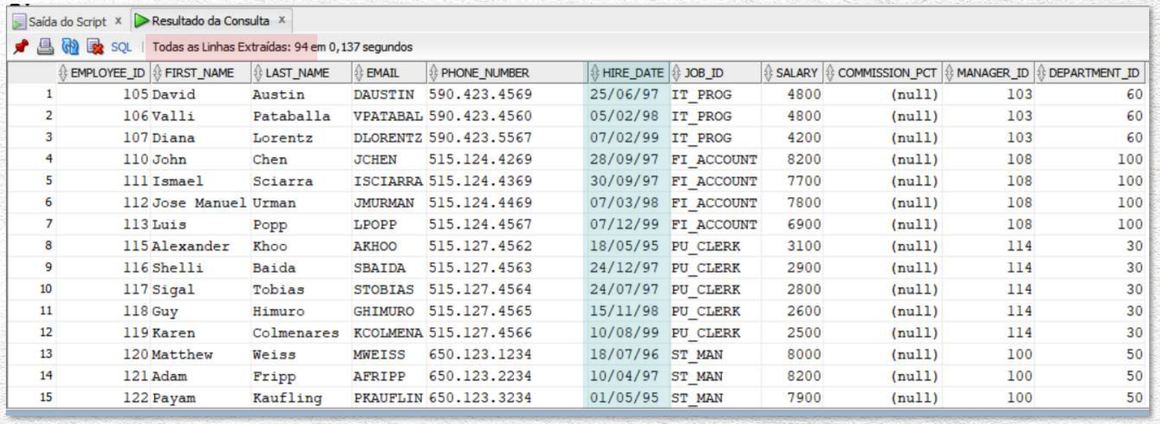
Crie uma view chamada *v_contratados_apos_1995* com os funcionários contratados a partir de 01 de janeiro de 1995.

(Utilize a tabela employees)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_contratados_apos_1995**:

```
SELECT * FROM v_contratados_apos_1995;
```



	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
2	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
3	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60
4	110	John	Chen	JCHEN	515.124.4269	28/09/97	FI_ACCOUNT	8200	(null)	108	100
5	111	Ismael	Sciarra	ISCIARRA	515.124.4369	30/09/97	FI_ACCOUNT	7700	(null)	108	100
6	112	Jose Manuel	Urman	JMURMAN	515.124.4469	07/03/98	FI_ACCOUNT	7800	(null)	108	100
7	113	Luis	Popp	LPOPP	515.124.4567	07/12/99	FI_ACCOUNT	6900	(null)	108	100
8	115	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	(null)	114	30
9	116	Shelli	Baida	SBAIDA	515.127.4563	24/12/97	PU_CLERK	2900	(null)	114	30
10	117	Sigal	Tobias	STOBIAS	515.127.4564	24/07/97	PU_CLERK	2800	(null)	114	30
11	118	Guy	Himuro	GHIMURO	515.127.4565	15/11/98	PU_CLERK	2600	(null)	114	30
12	119	Karen	Colmenares	KCOLMENA	515.127.4566	10/08/99	PU_CLERK	2500	(null)	114	30
13	120	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	(null)	100	50
14	121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	50
15	122	Payam	Kaufling	PKAUFLIN	650.123.3234	01/05/95	ST_MAN	7900	(null)	100	50

Com essa view, a equipe de RH pode acessar rapidamente a lista de contratações a partir de 1995 sempre que necessário, facilitando a auditoria da área de compliance sem a necessidade de refazer a consulta.

GABARITO EXPLICATIVO

3

Questão 3

A controladoria está analisando a distribuição salarial da empresa e precisa identificar os cinco maiores salários para avaliar o impacto desses cargos no orçamento da folha de pagamento.

Crie uma view chamada *v_top5_salarios* com os 5 funcionários mais bem pagos, exibindo nome, sobrenome, salário e ID do cargo.

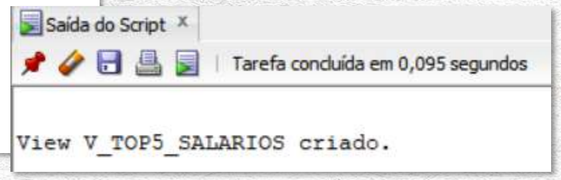
(Utilize a tabela employees)

Explicação e Resultado

Para criar uma view com os 5 funcionários mais bem pagos, mostrando seus nomes, salários e cargos, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_top5_salarios AS**: cria ou substitui uma view chamada **v_top5_salarios**.
- ✓ **SELECT first_name AS nome, last_name AS sobrenome, salary AS salario, job_id AS id_cargo**: seleciona os campos relevantes da tabela employees, renomeando-os para facilitar a leitura no relatório.
- ✓ **FROM employees**: indica que os dados estão sendo extraídos da tabela employees.
- ✓ **ORDER BY salary DESC**: ordena os funcionários em ordem decrescente de salário, do maior para o menor.
- ✓ **FETCH NEXT 5 ROWS ONLY**: limita o resultado aos 5 primeiros registros após a ordenação, ou seja, os 5 salários mais altos:

```
CREATE OR REPLACE VIEW v_top5_salarios AS
SELECT
    first_name AS nome,
    last_name AS sobrenome,
    salary AS salario,
    job_id AS id_cargo
FROM employees
ORDER BY salary DESC
FETCH NEXT 5 ROWS ONLY;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

A controladoria está analisando a distribuição salarial da empresa e precisa identificar os cinco maiores salários para avaliar o impacto desses cargos no orçamento da folha de pagamento.

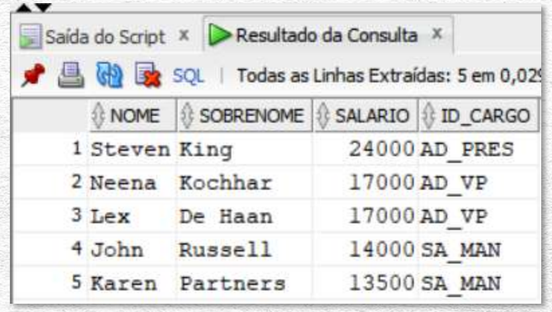
Crie uma view chamada *v_top5_salarios* com os 5 funcionários mais bem pagos, exibindo nome, sobrenome, salário e ID do cargo.

(Utilize a tabela employees)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_top5_salarios**:

```
SELECT * FROM v_top5_salarios;
```



	NOME	SOBRENOME	SALARIO	ID_CARGO
1	Steven	King	24000	AD_PRES
2	Neena	Kochhar	17000	AD_VP
3	Lex	De Haan	17000	AD_VP
4	John	Russell	14000	SA_MAN
5	Karen	Partners	13500	SA_MAN

Com essa view, a controladoria pode consultar rapidamente os maiores salários da empresa sem reescrever a lógica, o que agiliza análises de impacto na folha de pagamento.

GABARITO EXPLICATIVO

4

Questão 4

O time de planejamento estratégico solicitou um levantamento com a quantidade de funcionários por departamento, com o objetivo de realocar recursos humanos conforme a demanda de cada setor.

Crie uma view chamada *v_qtd_func_por_depto*, mostrando o ID do departamento, o nome do departamento e a quantidade de funcionários por departamento.

(Utilize as tabelas employees e departments)

Explicação e Resultado

Para criar uma view que exiba a quantidade de funcionários por departamento, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_qtd_func_por_depto AS**: cria (ou substitui) uma view chamada **v_qtd_func_por_depto**.
- ✓ **SELECT:** define os campos que serão retornados:
  - **e.department_id AS id_departamento**: ID do departamento, vindo da tabela employees.
  - **d.department_name AS departamento**: nome do departamento, vindo da tabela departments.
  - **COUNT(*) AS qtd_funcionarios**: conta o número de funcionários por departamento.
- ✓ **FROM employees e**: a tabela employees é usada como base da consulta e recebe o alias e.
- ✓ **INNER JOIN departments d ON e.department_id = d.department_id**: junta as tabelas employees e departments com base no department_id, associando os dados do funcionário ao nome do departamento.
- ✓ **GROUP BY e.department_id, d.department_name**: agrupa os dados por departamento, garantindo que cada linha da view represente um departamento único:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4 Questão 4

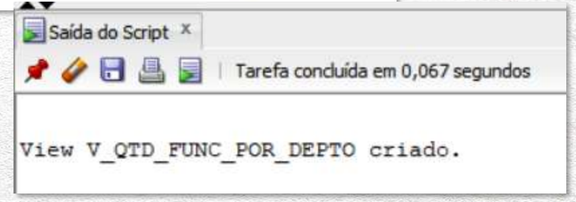
O time de planejamento estratégico solicitou um levantamento com a quantidade de funcionários por departamento, com o objetivo de realocar recursos humanos conforme a demanda de cada setor.

Crie uma view chamada *v_qtd_func_por_depto*, mostrando o ID do departamento, o nome do departamento e a quantidade de funcionários por departamento.

(Utilize as tabelas employees e departments)

Explicação e Resultado

```
CREATE OR REPLACE VIEW v_qtd_func_por_depto AS
SELECT
    e.department_id AS id_departamento,
    d.department_name AS departamento,
    COUNT(*) AS qtd_funcionarios
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
GROUP BY e.department_id, d.department_name;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

O time de planejamento estratégico solicitou um levantamento com a quantidade de funcionários por departamento, com o objetivo de realocar recursos humanos conforme a demanda de cada setor.

Crie uma view chamada *v_qtd_func_por_depto*, mostrando o ID do departamento, o nome do departamento e a quantidade de funcionários por departamento.

(Utilize as tabelas employees e departments)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_qtd_func_por_depto**:

```
SELECT * FROM v_qtd_func_por_depto;
```

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,063 segundos

	ID_DEPARTAMENTO	DEPARTAMENTO	QTD_FUNCIONARIOS
1	10	Administration	1
2	20	Marketing	2
3	30	Purchasing	6
4	40	Human Resources	1
5	50	Shipping	45
6	60	IT	5
7	70	Public Relations	1
8	80	Sales	34
9	90	Executive	3
10	100	Finance	6
11	110	Accounting	2

Essa view fornece uma visão consolidada da distribuição de pessoal na empresa, essencial para o planejamento estratégico e decisões de realocação de equipe.

GABARITO EXPLICATIVO

5

Questão 5

A equipe de remuneração está elaborando uma nova política salarial. Para isso, precisa de uma visualização com o teto salarial de cada cargo, facilitando a análise comparativa entre as funções com maior potencial de remuneração.

Crie uma view chamada *v_teto_salarial_cargos* com o título do cargo e o teto salarial de cada cargo, ordenando do maior para o menor salário.

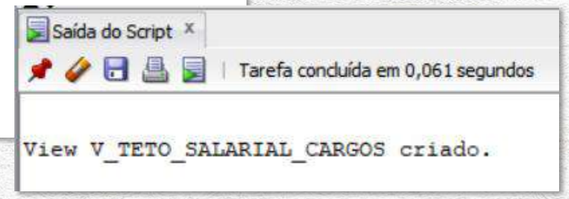
(Utilize a tabela jobs)

Explicação e Resultado

Para criar uma view ordenada com o teto salarial máximo por cargo, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_teto_salarial_cargos AS**: cria (ou substitui) uma view com o nome **v_teto_salarial_cargos**.
- ✓ **SELECT**: define os campos retornados:
  - **job_title AS cargo**: Título do cargo.
  - **max_salary AS teto_salarial**: Valor máximo de salário definido para o cargo.
- ✓ **FROM jobs**: os dados são extraídos da tabela jobs, que contém as informações de cargos e seus limites salariais.
- ✓ **ORDER BY max_salary DESC**: ordena os resultados do maior para o menor teto salarial, priorizando os cargos com maior remuneração potencial no topo da listagem:

```
CREATE OR REPLACE VIEW v_teto_salarial_cargos AS
SELECT
    job_title AS cargo,
    max_salary AS teto_salarial
FROM jobs
ORDER BY max_salary DESC;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A equipe de remuneração está elaborando uma nova política salarial. Para isso, precisa de uma visualização com o teto salarial de cada cargo, facilitando a análise comparativa entre as funções com maior potencial de remuneração.

Crie uma view chamada *v_teto_salarial_cargos* com o título do cargo e o teto salarial de cada cargo, ordenando do maior para o menor salário.

(Utilize a tabela jobs)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_teto_salarial_cargos**:

```
SELECT * FROM v_teto_salarial_cargos;
```

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas: 19 em 0,028 segundos

	CARGO	TETO_SALARIAL
1	President	40000
2	Administration Vice President	30000
3	Sales Manager	20000
4	Finance Manager	16000
5	Accounting Manager	16000
6	Purchasing Manager	15000
7	Marketing Manager	15000
8	Sales Representative	12000
9	Public Relations Representative	10500
10	Programmer	10000
11	Accountant	9000
12	Marketing Representative	9000
13	Human Resources Representative	9000
14	Public Accountant	9000
15	Stock Manager	8500
16	Administration Assistant	6000
17	Shipping Clerk	5500
18	Purchasing Clerk	5500
19	Stock Clerk	5000

Essa view permite comparar rapidamente os cargos com base no teto salarial, servindo como base para decisões estratégicas de remuneração, promoções e retenção de talentos.

GABARITO EXPLICATIVO

6

Questão 6

A diretoria internacional deseja revisar a distribuição geográfica das operações. Como ponto de partida, pediu ao setor de operações um relatório com o número total de localizações da empresa em cada país.

Crie uma view chamada *v_qtd_loc_por_pais* que exiba, para cada país, o número total de localizações cadastradas.

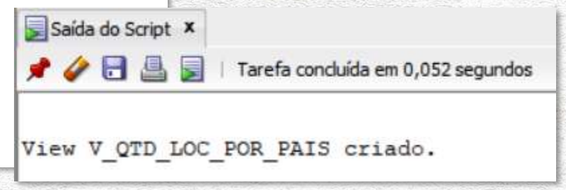
(Utilize a tabela locations)

Explicação e Resultado

Para criar uma view que mostre quantas localizações existem em cada país, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_qtd_loc_por_pais AS**: cria ou substitui uma view chamada v_qtd_loc_por_pais.
- ✓ **SELECT**: define os campos retornados:
  - **country_id AS id_pais**: retorna o ID do país vinculado à localização.
  - **COUNT(*) AS qtd_localizacoes**: conta quantas localizações estão registradas para cada país.
- ✓ **FROM locations**: os dados são obtidos da tabela locations, que contém os endereços das unidades da empresa.
- ✓ **GROUP BY country_id**: agrupa os registros por país para que o COUNT(*) funcione corretamente, somando as localizações por país:

```
CREATE OR REPLACE VIEW v_qtd_loc_por_pais AS
SELECT
    country_id AS id_pais,
    COUNT(*) AS qtd_localizacoes
FROM locations
GROUP BY country_id;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

A diretoria internacional deseja revisar a distribuição geográfica das operações. Como ponto de partida, pediu ao setor de operações um relatório com o número total de localizações da empresa em cada país.

Crie uma view chamada *v_qtd_loc_por_pais* que exiba, para cada país, o número total de localizações cadastradas.

(Utilize a tabela locations)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_qtd_loc_por_pais**:

```
SELECT * FROM v_qtd_loc_por_pais;
```

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas:

	ID_PAIS	QTD_LOCALIZACOES
1	IT	2
2	JP	2
3	US	4
4	CA	2
5	CN	1
6	IN	1
7	AU	1
8	SG	1
9	UK	3
10	DE	1
11	BR	1
12	CH	2
13	NL	1
14	MX	1

Essa view ajuda a diretoria a visualizar rapidamente a presença da empresa em cada país, sendo uma base útil para decisões de expansão, realocação ou encerramento de unidades.

GABARITO EXPLICATIVO

7

Questão 7

O setor de desenvolvimento organizacional está criando um mapa de lideranças. Para isso, precisa de uma visualização que relacione cada funcionário ao seu gerente direto e ao departamento onde trabalha.

Crie uma view chamada *v_func_gerente_depto* que mostre o nome completo do funcionário e de seu gerente, assim como o nome do departamento onde trabalha.

(Utilize as tabelas employees e departments)

Explicação e Resultado

Para criar uma view que relacione cada funcionário ao seu gerente e ao departamento em que trabalha, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_func_gerente_depto AS**: cria ou substitui uma view chamada v_func_gerente_depto.
- ✓ **SELECT**: define os campos retornados:
  - **e.first_name || ' ' || e.last_name AS funcionario**: concatena o primeiro e último nome do funcionário.
  - **m.first_name || ' ' || m.last_name AS gerente**: concatena o primeiro e último nome do gerente (autorrelacionamento na mesma tabela).
  - **d.department_name AS departamento**: retorna o nome do departamento onde o funcionário trabalha.
- ✓ **FROM employees e**: os dados principais vêm da tabela employees, apelidada como e.
- ✓ **LEFT JOIN employees m ON e.manager_id = m.employee_id**: faz um self-join com a própria tabela employees para obter os dados do gerente direto.
- ✓ **LEFT JOIN departments d ON e.department_id = d.department_id**: relaciona cada funcionário ao seu departamento:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

O setor de desenvolvimento organizacional está criando um mapa de lideranças. Para isso, precisa de uma visualização que relacione cada funcionário ao seu gerente direto e ao departamento onde trabalha.

Crie uma view chamada *v_func_gerente_depto* que mostre o nome completo do funcionário e de seu gerente, assim como o nome do departamento onde trabalha.

(Utilize as tabelas employees e departments)

Explicação e Resultado

```
CREATE OR REPLACE VIEW v_func_gerente_depto AS
SELECT
    e.first_name || ' ' || e.last_name AS funcionario,
    m.first_name || ' ' || m.last_name AS gerente,
    d.department_name AS departamento
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
LEFT JOIN departments d ON e.department_id = d.department_id;
```

```
Saída do Script
Tarefa concluída em 0,081 segundos

View V_FUNC_GERENTE_DEPTO criado.
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

O setor de desenvolvimento organizacional está criando um mapa de lideranças. Para isso, precisa de uma visualização que relacione cada funcionário ao seu gerente direto e ao departamento onde trabalha.

Crie uma view chamada *v_func_gerente_depto* que mostre o nome completo do funcionário e de seu gerente, assim como o nome do departamento onde trabalha.

(Utilize as tabelas employees e departments)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_func_gerente_depto**:

```
SELECT * FROM v_func_gerente_depto;
```

Saída do Script | Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,047 segundos

	FUNCIONARIO	GERENTE	DEPARTAMENTO
1	Steven King		Executive
2	Neena Kochhar	Steven King	Executive
3	Lex De Haan	Steven King	Executive
4	Alexander Hunold	Lex De Haan	IT
5	Bruce Ernst	Alexander Hunold	IT
6	David Austin	Alexander Hunold	IT
7	Valli Pataballa	Alexander Hunold	IT
8	Diana Lorentz	Alexander Hunold	IT
9	Nancy Greenberg	Neena Kochhar	Finance
10	Daniel Faviet	Nancy Greenberg	Finance
11	John Chen	Nancy Greenberg	Finance
12	Ismael Sciarra	Nancy Greenberg	Finance
13	Jose Manuel Urman	Nancy Greenberg	Finance
14	Luis Popp	Nancy Greenberg	Finance
15	Den Raphaely	Steven King	Purchasing

Essa view permite identificar rapidamente a cadeia de liderança dentro da empresa, ajudando no mapeamento organizacional, análise de estruturas hierárquicas e planejamento de sucessões.

GABARITO EXPLICATIVO

8

Questão 8

A equipe de Relações Internacionais está preparando um relatório sobre os colaboradores alocados em filiais fora dos Estados Unidos, com foco na estrutura organizacional e distribuição global.

Crie uma view chamada *v_func_fora_eua* com todos os funcionários que trabalham fora dos Estados Unidos. Considere que os países fora dos EUA têm country_id diferente de 'US'. Retorne todas as informações armazenadas desses funcionários, os nomes dos departamentos em que trabalham e os IDs de seus países.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

Para criar uma view que mostre todos os funcionários alocados fora dos Estados Unidos, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_func_fora_eua AS**: cria ou substitui uma view chamada **v_func_fora_eua**.
- ✓ **SELECT**: define os campos retornados:
  - **e.***: retorna todas as colunas da tabela de funcionários (employees).
  - **d.department_name**: exibe o nome do departamento em que o funcionário trabalha.
  - **l.country_id**: retorna o ID do país da localização do departamento.
- ✓ **FROM employees e**: os dados principais vêm da tabela employees, apelidada como e.
- ✓ **INNER JOIN departments d ON e.department_id = d.department_id**: relaciona cada funcionário ao seu departamento.
- ✓ **INNER JOIN locations l ON d.location_id = l.location_id**: relaciona o departamento à localização correspondente.
- ✓ **WHERE l.country_id <> 'US'**: filtra os resultados para retornar apenas os funcionários alocados em países diferentes dos Estados Unidos:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

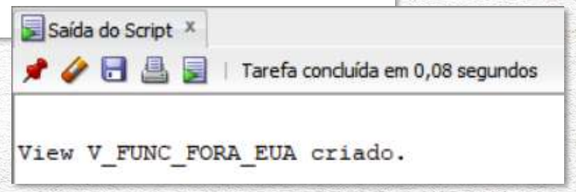
A equipe de Relações Internacionais está preparando um relatório sobre os colaboradores alocados em filiais fora dos Estados Unidos, com foco na estrutura organizacional e distribuição global.

Crie uma view chamada *v_func_fora_eua* com todos os funcionários que trabalham fora dos Estados Unidos. Considere que os países fora dos EUA têm country_id diferente de 'US'. Retorne todas as informações armazenadas desses funcionários, os nomes dos departamentos em que trabalham e os IDs de seus países.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

```
CREATE OR REPLACE VIEW v_func_fora_eua AS
SELECT
    e.*,
    d.department_name,
    l.country_id
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id
INNER JOIN locations l ON d.location_id = l.location_id
WHERE l.country_id <> 'US';
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A equipe de Relações Internacionais está preparando um relatório sobre os colaboradores alocados em filiais fora dos Estados Unidos, com foco na estrutura organizacional e distribuição global.

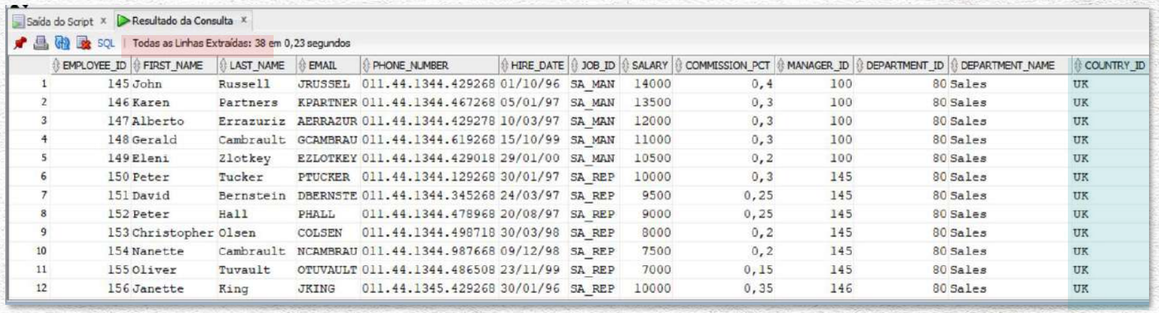
Crie uma view chamada *v_func_fora_eua* com todos os funcionários que trabalham fora dos Estados Unidos. Considere que os países fora dos EUA têm country_id diferente de 'US'. Retorne todas as informações armazenadas desses funcionários, os nomes dos departamentos em que trabalham e os IDs de seus países.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_func_fora_eua**:

```
SELECT * FROM v_func_fora_eua;
```



Saída do Script × Resultado da Consulta ×

SQL | Todas as Linhas Extraídas: 38 em 0,23 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	DEPARTMENT_NAME	COUNTRY_ID
1	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80	Sales	UK
2	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80	Sales	UK
3	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	80	Sales	UK
4	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100	80	Sales	UK
5	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29/01/00	SA_MAN	10500	0,2	100	80	Sales	UK
6	150	Peter	Tucker	PTUCKER	011.44.1344.129268	30/01/97	SA_REP	10000	0,3	145	80	Sales	UK
7	151	David	Bernstein	DBERNSTE	011.44.1344.345268	24/03/97	SA_REP	9500	0,25	145	80	Sales	UK
8	152	Peter	Hall	PHALL	011.44.1344.478968	20/08/97	SA_REP	9000	0,25	145	80	Sales	UK
9	153	Christopher	Olsen	COLSEN	011.44.1344.498718	30/03/98	SA_REP	8000	0,2	145	80	Sales	UK
10	154	Nanette	Cambrault	NCAMBRAU	011.44.1344.987668	09/12/98	SA_REP	7500	0,2	145	80	Sales	UK
11	155	Oliver	Tuvault	OTUVAULT	011.44.1344.486508	23/11/99	SA_REP	7000	0,15	145	80	Sales	UK
12	156	Janette	King	JKING	011.44.1345.429268	30/01/96	SA_REP	10000	0,35	146	80	Sales	UK

Essa view fornece uma base consolidada sobre os funcionários alocados fora dos EUA, facilitando análises sobre presença internacional, estrutura organizacional global e tomada de decisões estratégicas para atuação externa.

GABARITO EXPLICATIVO

9

Questão 9

O setor de Facilities está revisando a ocupação dos espaços físicos e precisa identificar quais departamentos possuem mais de 3 funcionários para planejar alocações de salas e estações de trabalho.

Crie uma view chamada *v_deptos_mais_3_func* com os nomes dos departamentos que possuem mais de 3 funcionários alocados e suas respectivas quantidades de funcionários.

(Utilize as tabelas departments e employees)

Explicação e Resultado

Para criar uma view que mostre os departamentos com mais de 3 funcionários, fazemos o seguinte:

- ✓ **CREATE OR REPLACE VIEW v_deptos_mais_3_func AS**: cria ou substitui uma view chamada **v_deptos_mais_3_func**.
- ✓ **SELECT**: define os campos retornados:
  - **d.department_name AS departamento**: exibe o nome do departamento.
  - **COUNT(e.employee_id) AS qtd_funcionarios**: conta quantos funcionários estão alocados em cada departamento.
- ✓ **FROM departments d**: os dados principais vêm da tabela departments, apelidada como d.
- ✓ **INNER JOIN employees e ON d.department_id = e.department_id**: associa cada departamento aos funcionários que pertencem a ele.
- ✓ **GROUP BY d.department_name**: agrupa os dados por nome de departamento para permitir a contagem de funcionários por grupo.
- ✓ **HAVING COUNT(e.employee_id) > 3**: filtra os grupos para manter apenas aqueles com mais de 3 funcionários:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

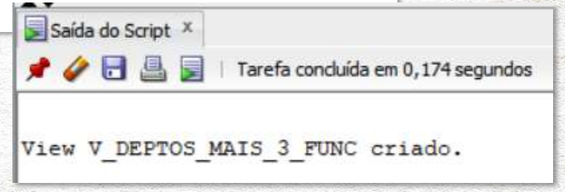
O setor de Facilities está revisando a ocupação dos espaços físicos e precisa identificar quais departamentos possuem mais de 3 funcionários para planejar alocações de salas e estações de trabalho.

Crie uma view chamada *v_deptos_mais_3_func* com os nomes dos departamentos que possuem mais de 3 funcionários alocados e suas respectivas quantidades de funcionários.

(Utilize as tabelas departments e employees)

Explicação e Resultado

```
CREATE OR REPLACE VIEW v_deptos_mais_3_func AS
SELECT
    d.department_name AS departamento,
    COUNT(e.employee_id) AS qtd_funcionarios
FROM departments d
INNER JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING COUNT(e.employee_id) > 3;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O setor de Facilities está revisando a ocupação dos espaços físicos e precisa identificar quais departamentos possuem mais de 3 funcionários para planejar alocações de salas e estações de trabalho.

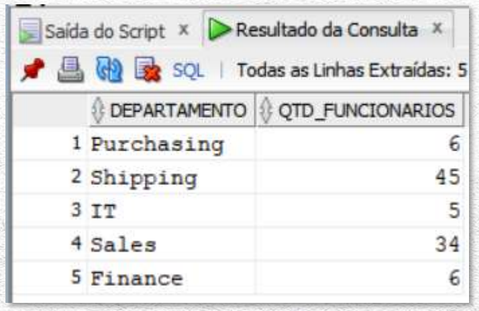
Crie uma view chamada *v_deptos_mais_3_func* com os nomes dos departamentos que possuem mais de 3 funcionários alocados e suas respectivas quantidades de funcionários.

(Utilize as tabelas departments e employees)

Explicação e Resultado

Para consultar os dados da view, utilizamos um **SELECT * FROM v_deptos_mais_3_func**:

```
SELECT * FROM v_deptos_mais_3_func;
```



	DEPARTAMENTO	QTD_FUNCIONARIOS
1	Purchasing	6
2	Shipping	45
3	IT	5
4	Sales	34
5	Finance	6

Essa view é útil para o setor de Facilities planejar alocações físicas de forma eficiente, identificando quais departamentos necessitam de mais espaço ou recursos devido ao número elevado de colaboradores.

GABARITO EXPLICATIVO

10

Questão 10

Após a apresentação dos relatórios solicitados, os dados foram aprovados e arquivados. Para manter o ambiente de banco de dados limpo e otimizado, decidiu-se excluir as views criadas para essas análises pontuais.

Exclua todas as views criadas nos exercícios anteriores.

Explicação e Resultado

Para excluir as views que criamos anteriormente, aplicamos um **DROP VIEW** com o nome de cada view:

```
DROP VIEW v_func_depto_90;
```
→
```
View V_FUNC_DEPTO_90 eliminado.
```

```
DROP VIEW v_contratados_apos_1995;
```
→
```
View V_CONTRATADOS_APOS_1995 eliminado.
```

```
DROP VIEW v_top5_salarios;
```
→
```
View V_TOP5_SALARIOS eliminado.
```

```
DROP VIEW v_qtd_func_por_depto;
```
→
```
View V_QTD_FUNC_POR_DEPTO eliminado.
```

```
DROP VIEW v_teto_salarial_cargos;
```
→
```
View V_TETO_SALARIAL_CARGOS eliminado.
```

```
DROP VIEW v_qtd_loc_por_pais;
```
→
```
View V_QTD_LOC_POR_PAIS eliminado.
```

```
DROP VIEW v_func_gerente_depto;
```
→
```
View V_FUNC_GERENTE_DEPTO eliminado.
```

```
DROP VIEW v_func_fora_eua;
```
→
```
View V_FUNC_FORA_EUA eliminado.
```

```
DROP VIEW v_deptos_mais_3_func;
```
→
```
View V_DEPTOS_MAIS_3_FUNC eliminado.
```

MÓDULO 13

SUBQUERIES

SUBQUERIES

SUBQUERIES

O QUE VEREMOS NESTE MÓDULO

MÓDULO 13 SUBQUERIES

01	O QUE É UMA SUBQUERY?	602
02	TIPOS DE SUBQUERIES: ESCALARES (SINGLE ROW) NA CLÁUSULA WHERE	603
03	TIPOS DE SUBQUERIES: DE LINHA MÚLTIPLA NA CLÁUSULA WHERE	606
04	SUBQUERIES DE LINHA MÚLTIPLA NA CLÁUSULA WHERE (IN)	607
05	SUBQUERIES DE LINHA MÚLTIPLA NA CLÁUSULA WHERE (NOT IN)	610
06	SUBQUERIES DE LINHA MÚLTIPLA NA CLÁUSULA WHERE (ANY)	611
07	SUBQUERIES DE LINHA MÚLTIPLA NA CLÁUSULA WHERE (ALL)	613
08	SUBQUERIES DE LINHA MÚLTIPLA E COLUNA MÚLTIPLA	615
09	EXISTS E NOT EXISTS	617
10	TIPOS DE SUBQUERIES: CORRELACIONADAS	620
11	SUBQUERIES CORRELACIONADAS	621
12	SUBQUERIES: SITUAÇÕES DE USO	624
13	SUBQUERIES DENTRO DA CLÁUSULA SELECT	625
14	SUBQUERIES DENTRO DA CLÁUSULA FROM	627
15	RESUMO DO MÓDULO	629
16	EXERCÍCIOS	630
17	GABARITO EXPLICATIVO	634

O que é uma SUBQUERY?

SUBQUERY é um recurso dentro do SQL que nos permite criar queries (consultas) mais otimizadas, reaproveitando o resultado de uma query dentro de outra query.

Elas podem ser usadas em três **SITUAÇÕES**:

Dentro da cláusula WHERE, como um **filtro**;

Dentro da cláusula SELECT, como uma **nova coluna**;

Dentro da cláusula FROM, como uma **nova tabela**.

E podem ser divididas em três **TIPOS**:

Subqueries **escalares** (single row);

Subqueries de **linha múltipla** (multiple row);

Subqueries **correlacionadas**.

TIPOS DE SUBQUERIES: ESCALARES (SINGLE ROW) na cláusula WHERE

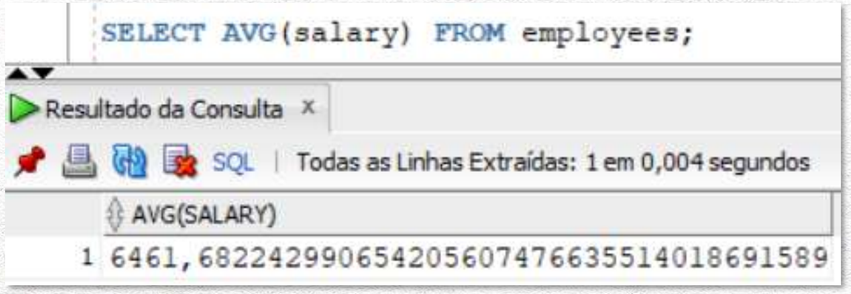
Iniciaremos este módulo com as SUBQUERIES ESCALARES, também chamadas de SINGLE ROW.

Subqueries Escalares retornam valores únicos, sempre 1 linha e 1 coluna.

Vejamos um exemplo:

Vamos supor que a gente queira saber quais funcionários recebem um salário acima da média de todos os funcionários.

Para isso, precisamos descobrir a média salarial:



Repare que o resultado retornado para média salarial (6461,682242990654205607476635514018691589) é um valor único (1 linha e 1 coluna).

TIPOS DE SUBQUERIES: ESCALARES (SINGLE ROW) na cláusula WHERE

Sem aplicar uma subquery, conseguiríamos descobrir quais são os funcionários que recebem um salário acima da média, se filtrássemos a nossa query com o resultado do SELECT anterior:

```
SELECT
    employee_id,
    first_name,
    salary
FROM employees
WHERE salary >= 6461.682242990654205607476635514018691589
ORDER BY salary;
```

	EMPLOYEE_ID	FIRST_NAME	SALARY
1	123	Shanta	6500
2	203	Susan	6500
3	165	David	6800
4	113	Luis	6900
5	155	Oliver	7000
6	161	Sarath	7000

O problema em se resolver a questão desta forma, é que o código não fica dinâmico.

Se houver qualquer alteração na média salarial dos funcionários, o nosso SELECT não será automaticamente atualizado, pois fizemos um filtro com um valor estático.

Para resolver isso de maneira dinâmica, podemos, então, utilizar uma subquery escalar na cláusula WHERE.

TIPOS DE SUBQUERIES: ESCALARES (SINGLE ROW) na cláusula WHERE

Para tanto, em vez de informarmos na cláusula WHERE o resultado do primeiro SELECT que fizemos para descobrir a média salarial, informamos o próprio SELECT, desta forma:

```
SELECT                      -- outer query (query externa, query principal)
    employee_id,
    first_name,
    salary
FROM employees
WHERE salary >= (SELECT AVG(salary) FROM employees)    -- inner query (query interna, subquery)
ORDER BY salary;
```

Um detalhe importante sobre a subquery acima é que ela é executada de dentro pra fora: primeiro é executada a query mais interna, para só depois executar a query mais externa. Dessa forma, não existe nenhuma dependência da query interna em relação à query externa. Ela é calculada de forma independente.

(Em alguns casos, no entanto, a query mais interna depende da query externa. A esse tipo de subquery, chamamos de correlacionada. Veremos sua aplicação mais a frente ainda neste módulo).

Resultado da Consulta

SQL | 50 linhas extraídas em 0,009 se

	EMPLOYEE_ID	FIRST_NAME	SALARY
1	123	Shanta	6500
2	203	Susan	6500
3	165	David	6800
4	113	Luis	6900
5	155	Oliver	7000
6	161	Sarath	7000

Só foram retornados os funcionários que ganham mais que a média salarial: 6461,68...

TIPOS DE SUBQUERIES: DE LINHA MÚLTIPLA na cláusula WHERE

Agora, vamos às SUBQUERIES DE LINHA MÚLTIPLA, também chamadas de MULTIPLE ROW.

Subqueries de Linha Múltipla retornam uma lista de valores.

Para filtrar nossas SUBQUERIES de linha múltipla, podemos utilizar os operadores: **IN**, **NOT IN**, **ANY** e **ALL**.

Vejamos os exemplos a seguir para entendermos melhor.

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (IN)

Vamos imaginar que a gente queira saber quais funcionários recebem um salário igual a qualquer uma das médias salariais agrupadas por departamento.

Para isso, precisamos descobrir, primeiramente, a média salarial de cada departamento:

```
SELECT AVG(salary)
FROM employees
GROUP BY department_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 12 em 0,005 segundos

	AVG(SALARY)
1	19333,333333333333333333333333333333333
2	5760
3	8600
4	4150
5	3475,555555555555555555555555555555555556
6	8955,882352941176470588235294117647058824
7	7000
8	4400
9	9500
10	6500
11	10000
12	10150

Repare que o resultado retornado é uma lista de valores.

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (IN)

Sem aplicar uma subquery, conseguiríamos descobrir quais são os funcionários que recebem um salário igual a qualquer uma das médias salariais agrupadas por departamento, se filtrássemos a nossa query com a lista apresentada como resultado do SELECT anterior:

```
SELECT *
FROM employees
WHERE salary IN (19333.33, 5760, 8600, 4150, 3475.56, 8955.88, 7000, 4400, 9500, 6500, 10000, 10150);
```

O problema em se resolver a questão desta forma, é que o código não fica dinâmico.

Se houver qualquer alteração na média salarial dos funcionários por departamento, o nosso SELECT não será automaticamente atualizado, pois fizemos um filtro com valores estáticos.

Para resolver isso de maneira dinâmica, podemos, então, utilizar uma subquery de linha múltipla na cláusula WHERE.

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 14 em 0,003 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	123	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	(null)	100	50
2	150	Peter	Tucker	PTUCKER	011.44.1344.129268	30/01/97	SA_REP	10000	0,3	145	80
3	151	David	Bernstein	DBERNSTE	011.44.1344.345268	24/03/97	SA_REP	9500	0,25	145	80
4	155	Oliver	Tuvault	OTUVAULT	011.44.1344.486508	23/11/99	SA_REP	7000	0,15	145	80
5	156	Janette	King	JKING	011.44.1345.429268	30/01/96	SA_REP	10000	0,35	146	80
6	157	Patrick	Sully	PSULLY	011.44.1345.929268	04/03/96	SA_REP	9500	0,35	146	80
7	161	Sarath	Sewall	SSEWALL	011.44.1345.529268	03/11/98	SA_REP	7000	0,25	146	80
8	163	Danielle	Greene	DGREENE	011.44.1346.229268	19/03/99	SA_REP	9500	0,15	147	80
9	169	Harrison	Bloom	HBLOOM	011.44.1343.829268	23/03/98	SA_REP	10000	0,2	148	80
10	176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	24/03/98	SA_REP	8600	0,2	149	80
11	178	Kimberely	Grant	KGRANT	011.44.1644.429263	24/05/99	SA_REP	7000	0,15	149	(null)
12	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
13	203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	(null)	101	40
14	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	70

Índice Geral

Índice deste Módulo

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (IN)

Para tanto, em vez de informarmos na cláusula WHERE o resultado do primeiro SELECT que fizemos para descobrir a média salarial de cada departamento, informamos o próprio SELECT, utilizando o operador IN, desta forma:

```
SELECT *
FROM employees
WHERE salary IN (SELECT AVG(salary) FROM employees GROUP BY department_id);
```

Perceba que o resultado retornou, de fato, todos os funcionários com salário igual a uma das médias salariais apresentadas no SELECT da página anterior (que é a subquery utilizada na cláusula WHERE da nossa consulta acima).

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 14 em 0,007 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	123	Shanta	Vollman	SVOLLMAN	650.123.4234	10/10/97	ST_MAN	6500	(null)	100	50
2	150	Peter	Tucker	PTUCKER	011.44.1344.129268	30/01/97	SA_REP	10000	0,3	145	80
3	151	David	Bernstein	DBERNSTE	011.44.1344.345268	24/03/97	SA_REP	9500	0,25	145	80
4	155	Oliver	Tuvault	OTUVAULT	011.44.1344.486508	23/11/99	SA_REP	7000	0,15	145	80
5	156	Janette	King	JKING	011.44.1345.429268	30/01/96	SA_REP	10000	0,35	146	80
6	157	Patrick	Sully	PSULLY	011.44.1345.929268	04/03/96	SA_REP	9500	0,35	146	80
7	161	Sarath	Sewall	SSEWALL	011.44.1345.529268	03/11/98	SA_REP	7000	0,25	146	80
8	163	Danielle	Greene	DGREENE	011.44.1346.229268	19/03/99	SA_REP	9500	0,15	147	80
9	169	Harrison	Bloom	HBLOOM	011.44.1343.829268	23/03/98	SA_REP	10000	0,2	148	80
10	176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	24/03/98	SA_REP	8600	0,2	149	80
11	178	Kimberely	Grant	KGRANT	011.44.1644.429263	24/05/99	SA_REP	7000	0,15	149	(null)
12	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
13	203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	(null)	101	40
14	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	70

Índice Geral

Índice deste Módulo

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (NOT IN)

Agora, vamos supor que a gente queira saber exatamente o contrário da nossa consulta anterior: quais funcionários recebem um salário diferente de qualquer uma das médias salariais agrupadas por departamento.

Para isso, basta substituirmos o operador IN pelo NOT IN:

```
SELECT *
FROM employees
WHERE salary NOT IN (SELECT AVG(salary) FROM employees GROUP BY department_id);
```

Perceba que o resultado retornou todos os funcionários com salário diferente de uma das médias salariais listadas na subquery utilizada na cláusula WHERE da nossa consulta acima.

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 93 em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
5	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60
6	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
7	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60
9	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
10	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (ANY)

Continuando, vamos imaginar que, agora, a gente queira saber quais funcionários recebem um salário menor que qualquer uma das médias salariais agrupadas por departamento (ou seja, menor do que a maior média salarial).

Para isso, podemos utilizar o operador ANY precedido do sinal de menor (<):

```
SELECT *
FROM employees
WHERE salary < ANY (SELECT AVG(salary) FROM employees GROUP BY department_id)
ORDER BY salary DESC;
```

Repare que o resultado retornado apresenta todos os funcionários que recebem um salário menor do que a maior de todas as médias salariais retornadas pela subquery utilizada na cláusula WHERE do nosso SELECT acima (19333,3333333333333333333 33333333333333).

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 106 em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21/09/89	AD_VP	17000	(null)	100	90
2	102	Lex	De Haan	LDEHAAN	515.123.4569	13/01/93	AD_VP	17000	(null)	100	90
3	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
4	146	Karen	Partners	KPARTNER	011.44.1344.467268	05/01/97	SA_MAN	13500	0,3	100	80
5	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
6	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110
7	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
8	147	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	10/03/97	SA_MAN	12000	0,3	100	80
9	168	Lisa	Ozer	LOZER	011.44.1343.929268	11/03/97	SA_REP	11500	0,25	148	80
10	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30
11	174	Ellen	Abel	EABEL	011.44.1644.429267	11/05/96	SA_REP	11000	0,3	149	80
12	148	Gerald	Cambrault	GCAMBRAU	011.44.1344.619268	15/10/99	SA_MAN	11000	0,3	100	80

Índice Geral

Índice deste Módulo

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (ANY)

Vamos imaginar que também queiramos saber quais funcionários recebem um salário maior que qualquer uma das médias salariais agrupadas por departamento (ou seja, maior do que a menor média salarial).

Para isso, podemos utilizar o operador ANY precedido do sinal de maior (>):

```
SELECT *
FROM employees
WHERE salary > ANY (SELECT AVG(salary) FROM employees GROUP BY department_id)
ORDER BY salary;
```

Repare que o resultado retornado apresenta todos os funcionários que recebem um salário maior do que a menor de todas as médias salariais retornadas pela subquery utilizada na cláusula WHERE do nosso SELECT acima (3475,555555555555555555555555555).

Resultado da Consulta

SQL | 50 linhas extraídas em 0,017 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	141	Trenna	Rajs	TRAJS	650.121.8009	17/10/95	ST_CLERK	3500	(null)	124	50
2	189	Jennifer	Dilly	JDILLY	650.505.2876	13/08/97	SH_CLERK	3600	(null)	122	50
3	137	Renske	Ladwig	RLADWIG	650.121.1234	14/07/95	ST_CLERK	3600	(null)	123	50
4	188	Kelly	Chung	KCHUNG	650.505.1876	14/06/97	SH_CLERK	3800	(null)	122	50
5	193	Britney	Everett	BEVERETT	650.501.2876	03/03/97	SH_CLERK	3900	(null)	123	50
6	192	Sarah	Bell	SBELL	650.501.1876	04/02/96	SH_CLERK	4000	(null)	123	50
7	185	Alexis	Bull	ABULL	650.509.2876	20/02/97	SH_CLERK	4100	(null)	121	50
8	107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/99	IT_PROG	4200	(null)	103	60
9	184	Nandita	Sarchand	NSARCHAN	650.509.1876	27/01/96	SH_CLERK	4200	(null)	121	50
10	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
11	105	David	Austin	DAUSTIN	590.423.4569	25/06/97	IT_PROG	4800	(null)	103	60
12	106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/98	IT_PROG	4800	(null)	103	60

Índice Geral

Índice deste Módulo

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (ALL)

Agora, vamos supor que a gente queira saber quais funcionários recebem um salário menor que todas as médias salariais agrupadas por departamento (ou seja, menor do que a menor média salarial).

Para isso, podemos utilizar o operador ALL precedido do sinal de menor (<):

```
SELECT *
FROM employees
WHERE salary < ALL (SELECT AVG(salary) FROM employees GROUP BY department_id)
ORDER BY salary DESC;
```

Repare que o resultado retornado apresenta todos os funcionários que recebem um salário menor do que a menor de todas as médias salariais retornadas pela subquery utilizada na cláusula WHERE do nosso SELECT acima (3475,55555555555555555555555555555555555556).

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 37 em 0,008 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	186	Julia	Dellinger	JDELLING	650.509.3876	24/06/98	SH_CLERK	3400	(null)	121	50
2	133	Jason	Mallin	JMALLIN	650.127.1934	14/06/96	ST_CLERK	3300	(null)	122	50
3	129	Laura	Bissot	LBISSOT	650.124.5234	20/08/97	ST_CLERK	3300	(null)	121	50
4	125	Julia	Nayer	JNAYER	650.124.1214	16/07/97	ST_CLERK	3200	(null)	120	50
5	194	Samuel	McCain	SMCCAIN	650.501.3876	01/07/98	SH_CLERK	3200	(null)	123	50
6	180	Winston	Taylor	WTAYLOR	650.507.9876	24/01/98	SH_CLERK	3200	(null)	120	50
7	138	Stephen	Stiles	SSTILES	650.121.2034	26/10/97	ST_CLERK	3200	(null)	123	50
8	115	Alexander	Khoo	AKHOO	515.127.4562	18/05/95	PU_CLERK	3100	(null)	114	30
9	196	Alana	Walsh	AWALSH	650.507.9811	24/04/98	SH_CLERK	3100	(null)	124	50
10	181	Jean	Fleaur	JFLEAUR	650.507.9877	23/02/98	SH_CLERK	3100	(null)	120	50
11	142	Curtis	Davies	CDAVIES	650.121.2994	29/01/97	ST_CLERK	3100	(null)	124	50
12	197	Kevin	Feeney	KFEENEY	650.507.9822	23/05/98	SH_CLERK	3000	(null)	124	50
13	187	Anthony	Cabrio	ACABRIO	650.509.4876	07/02/99	SH_CLERK	3000	(null)	121	50

Índice Geral

Índice deste Módulo

SUBQUERIES DE LINHA MÚLTIPLA na cláusula WHERE (ALL)

Vamos imaginar que também queiramos saber quais funcionários recebem um salário maior que todas as médias salariais agrupadas por departamento (ou seja, maior do que a maior média salarial).

Para isso, podemos utilizar o operador ALL precedido do sinal de maior (>):

```
SELECT *
FROM employees
WHERE salary > ALL (SELECT AVG(salary) FROM employees GROUP BY department_id)
ORDER BY salary DESC;
```

Repare que o resultado retornado apresenta o funcionário que recebe um salário maior do que a maior de todas as médias salariais retornadas pela subquery utilizada na cláusula WHERE do nosso SELECT acima (19333,3333333333333333333333333333333).

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,003 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90

SUBQUERIES DE LINHA MÚLTIPLA E COLUNA MÚLTIPLA

Neste próximo exemplo queremos encontrar os funcionários que ganham o maior salário de cada departamento.

Para isso, vamos descobrir o MAX(salary) → maior salário por departamento (department_id):

```
SELECT department_id, MAX(salary)
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
ORDER BY department_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extra

	DEPARTMENT_ID	MAX(SALARY)
1	10	4400
2	20	13000
3	30	11000
4	40	6500
5	50	8200
6	60	9000
7	70	10000
8	80	14000
9	90	24000
10	100	12000
11	110	12000

SUBQUERIES DE LINHA MÚLTIPLA E COLUNA MÚLTIPLA

Aplicando a consulta anterior dentro de um outro SELECT na tabela EMPLOYEES, como uma SUBQUERY, conseguimos, por fim, encontrar as informações dos funcionários que ganham o maior salário de cada departamento:

```
SELECT *
FROM employees
WHERE (department_id, salary) IN (SELECT department_id, MAX(salary)
                                  FROM employees
                                  GROUP BY department_id)
ORDER BY department_id;
```

Repare que, agora, estamos retornando nesta consulta DUAS LISTAS: uma da coluna **department_id**, com todos os departamentos que possuem funcionários vinculados a ele 1, outra da função **MAX(salary)** com os maiores salários de cada um desses departamentos 2

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,004 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY 2	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID 1
1	200	Jennifer	Whalen	JWHALEN	515.123.4444	17/09/87	AD_ASST	4400	(null)	101	10
2	201	Michael	Hartstein	MHARTSTE	515.123.5555	17/02/96	MK_MAN	13000	(null)	100	20
3	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30
4	203	Susan	Mavris	SMAVRIS	515.123.7777	07/06/94	HR_REP	6500	(null)	101	40
5	121	Adam	Fripp	AFRIPP	650.123.2234	10/04/97	ST_MAN	8200	(null)	100	50
6	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
7	204	Hermann	Baer	HBAER	515.123.8888	07/06/94	PR_REP	10000	(null)	101	70
8	145	John	Russell	JRUSSEL	011.44.1344.429268	01/10/96	SA_MAN	14000	0,4	100	80
9	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
10	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
11	205	Shelley	Higgins	SHIGGINS	515.123.8080	07/06/94	AC_MGR	12000	(null)	101	110

Por isso usamos o termo Subquery de Linha Múltipla (lista) e Coluna Múltipla (mais de uma coluna).

EXISTS e NOT EXISTS

O operador EXISTS é usado para testar a existência de qualquer registro (linha) em uma subconsulta.

Pensando em um exemplo prático, imagine que a gente queira descobrir quais são os departamentos que possuem funcionários.

Sabemos que existem 27 departamentos na tabela DEPARTMENTS:

```
SELECT DISTINCT * FROM departments;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 27 em 0,009 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700

Porém, apenas 11 deles estão relacionados a funcionários na tabela EMPLOYEES:

```
SELECT DISTINCT department_id
FROM employees
WHERE department_id IS NOT NULL;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,005 segundos

	DEPARTMENT_ID
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90
10	100
11	110

Portanto, queremos descobrir quais são estes 11 departamentos.

EXISTS e NOT EXISTS

Para isso, podemos utilizar o operador EXISTS, juntamente com uma subquery que faz um filtro comparando o department_id das tabelas DEPARTMENTS e EMPLOYEES:

```
SELECT * FROM departments d
WHERE EXISTS(SELECT * FROM employees e WHERE e.department_id = d.department_id);
```

Repare que o resultado retornou os 11 departamentos vinculados a funcionários que estávamos procurando:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 11 em 0,002 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700
4	40	Human Resources	203	2400
5	50	Shipping	121	1500
6	60	IT	103	1400
7	70	Public Relations	204	2700
8	80	Sales	145	2500
9	90	Executive	100	1700
10	100	Finance	108	1700
11	110	Accounting	205	1700

EXISTS e NOT EXISTS

Se quisermos descobrir exatamente o contrário, ou seja, quais são os departamentos que NÃO possuem funcionários vinculados, utilizamos o mesmo código, porém trocamos o operador EXISTS por NOT EXISTS:

```
SELECT * FROM departments d
WHERE NOT EXISTS(SELECT * FROM employees e WHERE e.department_id = d.department_id);
```

Repare que o resultado retornou os outros 16 departamentos que não estão vinculados a funcionários na tabela EMPLOYEES:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 16 em 0,011 segundos

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	120	Treasury	(null)	1700
2	130	Corporate Tax	(null)	1700
3	140	Control And Credit	(null)	1700
4	150	Shareholder Services	(null)	1700
5	160	Benefits	(null)	1700
6	170	Manufacturing	(null)	1700
7	180	Construction	(null)	1700
8	190	Contracting	(null)	1700
9	200	Operations	(null)	1700
10	210	IT Support	(null)	1700
11	220	NOC	(null)	1700
12	230	IT Helpdesk	(null)	1700
13	240	Government Sales	(null)	1700
14	250	Retail Sales	(null)	1700
15	260	Recruiting	(null)	1700
16	270	Payroll	(null)	1700

TIPOS DE SUBQUERIES: CORRELACIONADAS

Vamos entender agora o que são as SUBQUERIES CORRELACIONADAS.

Uma Subquery Correlacionada é aquela em que a subquery interna possui uma dependência com a query externa.

Acabamos de ver uma subquery correlacionada com o operador EXISTS, uma vez que, no último exemplo, a subquery interna dependeu da externa para retornar valores.

Entretanto, as subqueries correlacionadas não se limitam apenas ao uso do EXISTS, por isso, vamos falar um pouco mais sobre elas agora.

Vamos imaginar que queremos descobrir quais são os funcionários que ganham acima da média salarial do seu próprio departamento.

Já vimos na explicação sobre subqueries de linha múltipla na cláusula WHERE que podemos resolver isso fazendo uso do GROUP BY na subquery interna.

Entretanto, temos também outra forma de solucionar este desafio por meio de uma subquery correlacionada.

Vejamos na próxima página.

SUBQUERIES CORRELACIONADAS

Se nós quiséssemos apenas saber quais são os funcionários que recebem um salário maior ou igual à média salarial geral, de toda a empresa, seria bem simples, pois bastaria utilizarmos uma subquery na cláusula WHERE que calculasse a média salarial (AVG(salary)) da tabela EMPLOYEES, comparando-a com o salário do funcionário chamado na query externa:

```
SELECT
    *
FROM employees
WHERE salary >= (
                SELECT
                    AVG(salary)
                FROM employees
                );
```

SUBQUERIES CORRELACIONADAS

Porém, não queremos saber quais são os funcionários que recebem um salário maior ou igual à média salarial geral, mas sim os que recebem um salário maior ou igual à média salarial DE SEU PRÓPRIO DEPARTAMENTO.

Para isso, precisaremos pensar na tabela EMPLOYEES como se fosse duas tabelas: podemos chama-las de e1 e e2.

Vamos utilizar a e1 na query externa para chamar os funcionários cujo salário seja maior ou igual à média salarial (AVG(salary)) do e2.department_id que será retornado na subquery interna.

Esse e2.department_id, por sua vez, estará relacionado ao e1.department_id do funcionário que estiver na consulta externa.

```
SELECT
    *
FROM employees e1
WHERE e1.salary >= (
                    SELECT
                        AVG(salary)
                    FROM employees e2
                    WHERE e2.department_id = e1.department_id
                    );
```

SUBQUERIES CORRELACIONADAS

Assim, retornaremos 41 dos 107 funcionários registrados na tabela EMPLOYEES, pois são estes que recebem acima da média de seu próprio departamento:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 41 em 0,009 segundos

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90
2	103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/90	IT_PROG	9000	(null)	102	60
3	104	Bruce	Ernst	BERNST	590.423.4568	21/05/91	IT_PROG	6000	(null)	103	60
4	108	Nancy	Greenberg	NGREENBE	515.124.4569	17/08/94	FI_MGR	12000	(null)	101	100
5	109	Daniel	Faviet	DFAVIET	515.124.4169	16/08/94	FI_ACCOUNT	9000	(null)	108	100
6	114	Den	Raphaely	DRAPHEAL	515.127.4561	07/12/94	PU_MAN	11000	(null)	100	30
7	120	Matthew	Weiss	MWEISS	650.123.1234	18/07/96	ST_MAN	8000	(null)	100	50

A título de exemplo, veja o primeiro funcionário retornado na nossa consulta: Steven King, do department_id = 90.

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	17/06/87	AD_PRES	24000	(null)	(null)	90

Na nossa consulta, ele foi retornado porque a média salarial do departamento 90 é $19333,333333333333333333333333333333333333, e o salário dele é de $24000,00.

Portanto: o e1.salary (24000) é >= (SELECT AVG(salary) FROM employees e2 WHERE e2.department_id = e1.department_id) (19333,333333333333333333333333333333333333)

SUBQUERIES: SITUAÇÕES DE USO

Por enquanto, neste módulo, vimos diversos exemplos de uso das subqueries dentro da cláusula WHERE, como um **filtro**.

Agora, veremos como utilizá-la dentro da cláusula SELECT como uma **nova coluna**.

E, por fim, dentro da cláusula FROM, como uma **nova tabela**.

SUBQUERIES dentro da cláusula SELECT

SUBQUERIES também podem ser utilizadas dentro de um SELECT como uma **nova coluna** a ser exibida no resultado da consulta.

Vejamos um exemplo:

Imagine que a gente queira fazer uma consulta à tabela EMPLOYEES para retornar as seguintes colunas:

1. employee_id
2. first_name
3. job_id
4. salary
5. média salarial

SUBQUERIES dentro da cláusula SELECT

Sabemos que as quatro primeiras colunas já existem na tabela EMPLOYEES, porém a última (média salarial) não.

Para retorná-la, devemos utilizar uma subconsulta dentro do SELECT, da seguinte forma:

```
SELECT
    employee_id,
    first_name,
    job_id,
    salary,
    ROUND((SELECT AVG(salary) FROM employees), 2) AS media_salario
FROM employees;
```

Obs.: Apesar de não ser obrigatório, aqui utilizamos a função ROUND também para podermos retornar apenas duas casas decimais no resultado da coluna criada com a nossa subquery.

Resultado da Consulta

SQL | 50 linhas extraídas em 0,018 segundos

	EMPLOYEE_ID	FIRST_NAME	JOB_ID	SALARY	MEDIA_SALARIO
1	100	Steven	AD_PRES	24000	6461,68
2	101	Neena	AD_VP	17000	6461,68
3	102	Lex	AD_VP	17000	6461,68
4	103	Alexander	IT_PROG	9000	6461,68
5	104	Bruce	IT_PROG	6000	6461,68
6	105	David	IT_PROG	4800	6461,68
7	106	Valli	IT_PROG	4800	6461,68

SUBQUERIES dentro da cláusula FROM

SUBQUERIES também podem ser utilizadas dentro de uma cláusula FROM como uma **nova tabela**.

Vejamos um exemplo:

Imagine que a gente queira, considerando a quantidade de funcionários por JOB_ID, saber qual é a maior, a menor e a média da quantidade de funcionários.

Sabemos que para, inicialmente, descobrirmos quantos funcionários temos para cada JOB_ID, basta fazermos uma contagem, desta forma:

```
SELECT
    job_id,
    COUNT(*) qtd_funcionarios
FROM employees
GROUP BY job_id;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas

	JOB_ID	QTD_FUNCIONARIOS
1	AC_ACCOUNT	1
2	AC_MGR	1
3	AD_ASST	1
4	AD_PRES	1
5	AD_VP	2
6	FI_ACCOUNT	5
7	FI_MGR	1
8	HR_REP	1
9	IT_PROG	5
10	MK_MAN	1
11	MK_REP	1
12	PR_REP	1
13	PU_CLERK	5
14	PU_MAN	1
15	SA_MAN	5
16	SA_REP	30
17	SH_CLERK	20
18	ST_CLERK	20
19	ST_MAN	5

SUBQUERIES dentro da cláusula FROM

Para descobrirmos qual é a maior, a menor e a média da quantidade de funcionários de acordo com o JOB_ID, podemos utilizar esta contagem que efetuamos na página anterior dentro de uma cláusula FROM, como se fosse uma tabela:

```
SELECT
    MAX(qtd_funcionarios) max_funcionarios,
    MIN(qtd_funcionarios) min_funcionarios,
    AVG(qtd_funcionarios) avg_funcionarios
FROM (
    SELECT
        job_id,
        COUNT(*) qtd_funcionarios
    FROM employees
    GROUP BY job_id) t;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,003 segundos

	MAX_FUNCIONARIOS	MIN_FUNCIONARIOS	AVG_FUNCIONARIOS
1	30	1	5,631578947368421052631578947368421052 63

Perceba que, neste caso, devemos dar um nome a nossa SUBQUERY que faz as vezes de tabela. Neste exemplo, a chamamos de t.

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. SUBQUERIES (subconsultas) são um SELECT dentro de outro SELECT, usadas para tornar o cálculo das nossas consultas mais automático e dinâmico, menos sujeito a erros.

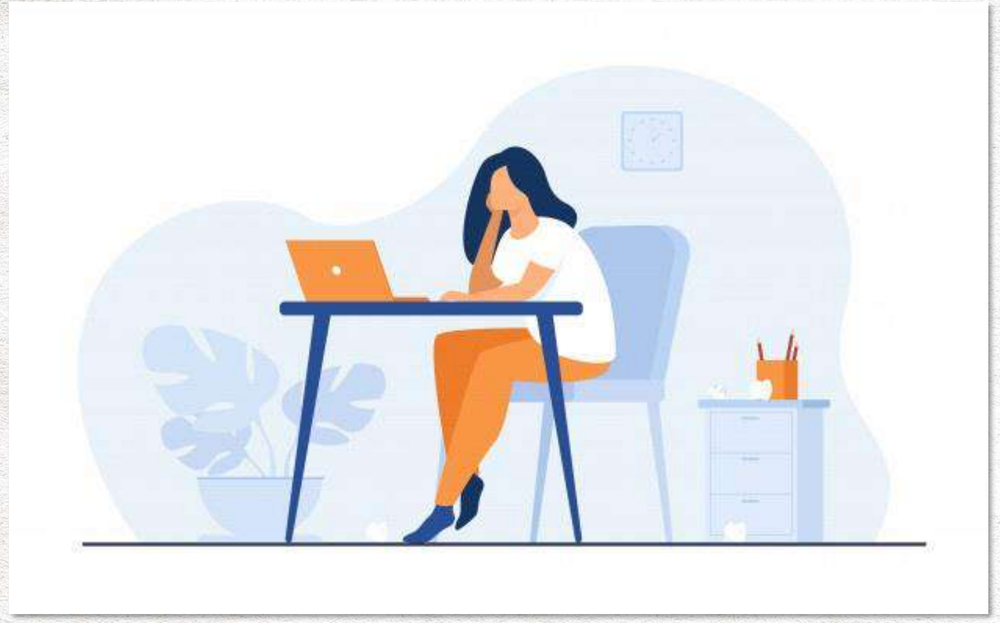
2. As subconsultas podem ser usadas nas cláusulas WHERE, SELECT e FROM.

3. Existem 3 tipos de subconsultas: escalares, linhas/colunas múltiplas e correlacionadas.

4. Usamos os operadores ANY e ALL para fazer operações diferenciadas com listas de valores (mais abrangentes que o IN).

5. Utilizamos o EXISTS para verificar a existência de registros nas nossas consultas.

EXERCÍCIOS



1

Questão 1

O setor de Recursos Humanos está realizando um estudo para redistribuição de equipe e orçamento. Para isso, é necessário identificar os departamentos com maior número de funcionários, visando priorizar ações estratégicas de gestão de pessoas.

Crie uma consulta que liste os IDs e nomes dos departamentos que possuem mais de 5 funcionários. **(Utilize as tabelas departments e employees)**

2

Questão 2

A diretoria está avaliando uma possível reestruturação das unidades operacionais nos Estados Unidos. Para embasar essa decisão, foi solicitado um levantamento com os nomes dos funcionários que trabalham em departamentos localizados nesse país.

Crie um código que liste os nomes e sobrenomes dos funcionários que trabalham em departamentos localizados nos Estados Unidos e os IDs desses departamentos. **(Utilize as tabelas employees, departments e locations)**

EXERCÍCIOS

3

Questão 3

O RH está estudando perfis de funcionários que já passaram por diferentes cargos, com o objetivo de mapear potenciais candidatos para promoções internas e planos de carreira mais versáteis.

Crie um código que liste os IDs, os nomes e os sobrenomes dos funcionários cujo cargo atual é diferente de algum cargo que já ocuparam anteriormente. **(Utilize as tabelas employees e job_history)**

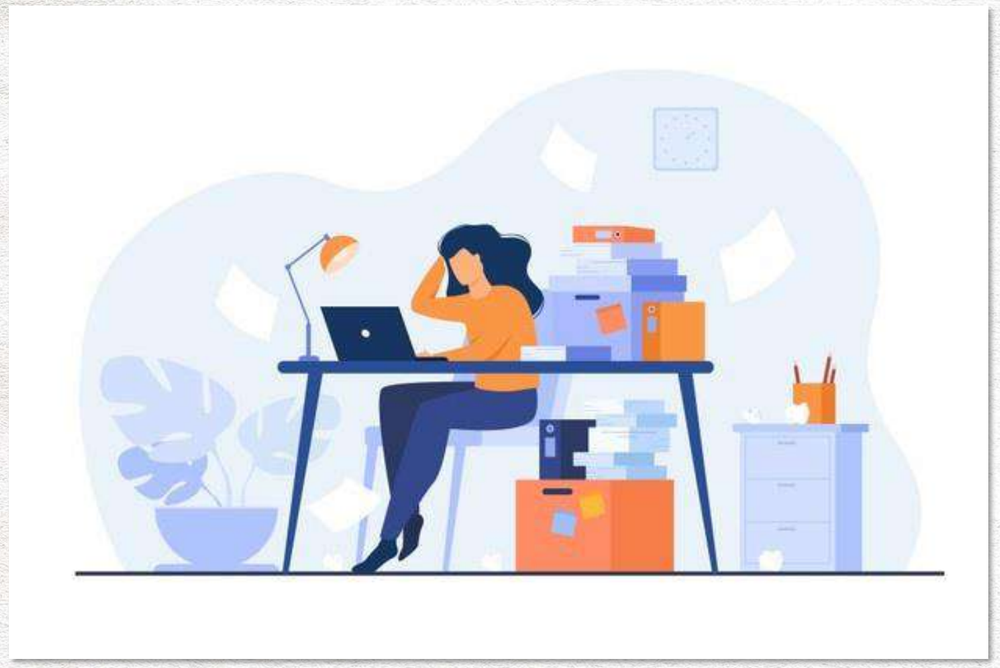
4

Questão 4

A equipe de gestão de desempenho está avaliando o cargo ocupado por Peter Tucker, que teve destaque em avaliações recentes. O objetivo é identificar colegas que exercem o mesmo cargo, para aplicar a mesma política de treinamentos e metas.

Crie um código que liste os nomes e sobrenomes dos funcionários que possuem o mesmo cargo que Peter Tucker e o ID de seu cargo. **(Utilize a tabela employees)**

EXERCÍCIOS



5

Questão 5

O RH deseja identificar funcionários que permanecem no mesmo departamento desde a contratação, pois esses casos podem representar estabilidade ou, em alguns casos, necessidade de novos desafios para evitar estagnação.

Crie um código que liste os nomes e sobrenomes dos funcionários que nunca trabalharam em departamentos diferentes do atual. **(Utilize as tabelas employees e job_history)**

6

Questão 6

A área de expansão imobiliária está estudando a concentração de departamentos por cidade. O objetivo é avaliar a viabilidade de unificar unidades administrativas em uma única sede nas cidades com mais de um departamento.

Crie um código que liste os nomes das cidades que eventualmente possuam mais de um departamento. **(Utilize as tabelas locations e departments)**

EXERCÍCIOS

7

Questão 7

A área financeira precisa identificar quais departamentos têm políticas de comissão em vigor. Esse levantamento é essencial para revisar os impactos dessas comissões nos custos operacionais de cada área.

Crie um código que liste os IDs e nomes dos departamentos que possuem funcionários com comissão. **(Utilize as tabelas departments e employees)**

8

Questão 8

O setor de planejamento de carreira está analisando os cargos mais populares da empresa para desenvolver trilhas de capacitação e programas de crescimento profissional voltados a essas funções.

Crie um código que liste os títulos dos cargos que são ocupados por mais de 3 funcionários. **(Utilize as tabelas jobs e employees)**

GABARITO EXPLICATIVO

1

Questão 1

O setor de Recursos Humanos está realizando um estudo para redistribuição de equipe e orçamento. Para isso, é necessário identificar os departamentos com maior número de funcionários, visando priorizar ações estratégicas de gestão de pessoas.

Crie uma consulta que liste os IDs e nomes dos departamentos que possuem mais de 5 funcionários.

(Utilize as tabelas departments e employees)

Explicação e Resultado

Para identificar os departamentos com mais de 5 funcionários, utilizamos uma subquery na cláusula **WHERE department_id IN (...)**. Veja a explicação passo a passo:

1. Subquery:

- ✓ Agrupa os funcionários por departamento (**GROUP BY department_id**).
- ✓ Filtra apenas os grupos que têm mais de 5 funcionários (**HAVING COUNT(*) > 5**).
- ✓ Retorna os IDs desses departamentos:

```
SELECT
    department_id,
    department_name
FROM departments
WHERE department_id IN (
    SELECT
        department_id
    FROM employees
    GROUP BY department_id
    HAVING COUNT(*) > 5
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O setor de Recursos Humanos está realizando um estudo para redistribuição de equipe e orçamento. Para isso, é necessário identificar os departamentos com maior número de funcionários, visando priorizar ações estratégicas de gestão de pessoas.

Crie uma consulta que liste os IDs e nomes dos departamentos que possuem mais de 5 funcionários.

(Utilize as tabelas departments e employees)

Explicação e Resultado

2. Consulta principal:

- ✓ Retorna o ID e o nome dos departamentos (**department_id, department_name**) cujo ID aparece no resultado da subquery.
- ✓ A cláusula **IN** garante que apenas os departamentos com mais de 5 funcionários sejam listados:

```
SELECT
    department_id,
    department_name
FROM departments
WHERE department_id IN (
    SELECT
        department_id
    FROM employees
    GROUP BY department_id
    HAVING COUNT(*) > 5
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

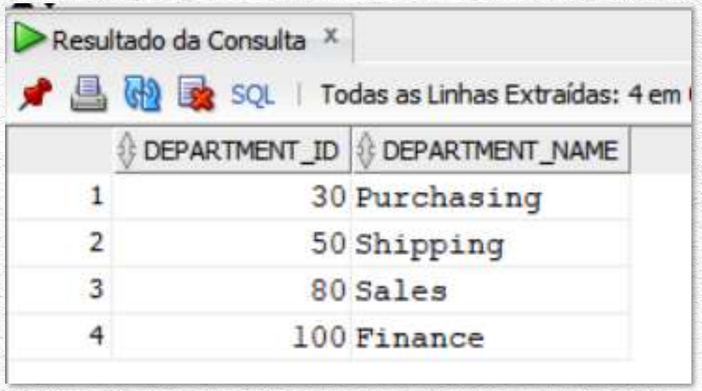
O setor de Recursos Humanos está realizando um estudo para redistribuição de equipe e orçamento. Para isso, é necessário identificar os departamentos com maior número de funcionários, visando priorizar ações estratégicas de gestão de pessoas.

Crie uma consulta que liste os IDs e nomes dos departamentos que possuem mais de 5 funcionários.

(Utilize as tabelas departments e employees)

Explicação e Resultado

Resultado retornado:


Resultado da Consulta

SQL | Todas as Linhas Extraídas: 4 em

	DEPARTMENT_ID	DEPARTMENT_NAME
1	30	Purchasing
2	50	Shipping
3	80	Sales
4	100	Finance

Essa consulta ajuda o RH a identificar onde estão concentrados os maiores times, permitindo alocar orçamento, lideranças e recursos com mais eficiência.

GABARITO EXPLICATIVO

2

Questão 2

A diretoria está avaliando uma possível reestruturação das unidades operacionais nos Estados Unidos. Para embasar essa decisão, foi solicitado um levantamento com os nomes dos funcionários que trabalham em departamentos localizados nesse país.

Crie um código que liste os nomes e sobrenomes dos funcionários que trabalham em departamentos localizados nos Estados Unidos e os IDs desses departamentos.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

Para listar os nomes e sobrenomes dos funcionários que trabalham em departamentos localizados nos Estados Unidos, usamos uma subquery aninhada para filtrar os **location_id** vinculados ao país **'US'**. Abaixo está a explicação passo a passo:

1. Subquery mais interna:

- ✓ Retorna os IDs de localizações que estão nos Estados Unidos, filtrando pelo **country_id = 'US'**:

```
SELECT
    first_name,
    last_name,
    department_id
FROM employees
WHERE department_id IN (
    SELECT
        department_id
    FROM departments
    WHERE location_id IN (
        SELECT
            location_id
        FROM locations
        WHERE country_id = 'US'
    )
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A diretoria está avaliando uma possível reestruturação das unidades operacionais nos Estados Unidos. Para embasar essa decisão, foi solicitado um levantamento com os nomes dos funcionários que trabalham em departamentos localizados nesse país.

Crie um código que liste os nomes e sobrenomes dos funcionários que trabalham em departamentos localizados nos Estados Unidos e os IDs desses departamentos.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

2. Subquery intermediária:

- ✓ Usa os **location_id** obtidos anteriormente para encontrar os **department_id** localizados nos EUA:

```
SELECT
    first_name,
    last_name,
    department_id
FROM employees
WHERE department_id IN (
    SELECT
        department_id
    FROM departments
    WHERE location_id IN (
        SELECT
            location_id
        FROM locations
        WHERE country_id = 'US'
    )
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A diretoria está avaliando uma possível reestruturação das unidades operacionais nos Estados Unidos. Para embasar essa decisão, foi solicitado um levantamento com os nomes dos funcionários que trabalham em departamentos localizados nesse país.

Crie um código que liste os nomes e sobrenomes dos funcionários que trabalham em departamentos localizados nos Estados Unidos e os IDs desses departamentos.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

3. Consulta principal:

- ✓ Seleciona os nomes, sobrenomes e IDs de departamento dos funcionários que pertencem a departamentos localizados nos EUA:

```
SELECT
    first_name,
    last_name,
    department_id
FROM employees
WHERE department_id IN (
    SELECT
        department_id
    FROM departments
    WHERE location_id IN (
        SELECT
            location_id
        FROM locations
        WHERE country_id = 'US'
    )
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

A diretoria está avaliando uma possível reestruturação das unidades operacionais nos Estados Unidos. Para embasar essa decisão, foi solicitado um levantamento com os nomes dos funcionários que trabalham em departamentos localizados nesse país.

Crie um código que liste os nomes e sobrenomes dos funcionários que trabalham em departamentos localizados nos Estados Unidos e os IDs desses departamentos.

(Utilize as tabelas employees, departments e locations)

Explicação e Resultado

Resultado retornado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 68 em 0,05 se

	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Steven	King	90
2	Neena	Kochhar	90
3	Lex	De Haan	90
4	Alexander	Hunold	60
5	Bruce	Ernst	60
6	David	Austin	60
7	Valli	Pataballa	60
8	Diana	Lorentz	60
9	Nancy	Greenberg	100
10	Daniel	Faviet	100
11	John	Chen	100
12	Ismael	Sciarra	100
13	Jose Manuel	Urman	100
14	Luis	Popp	100
15	Den	Raphaely	30
16	Alexander	Khoo	30
17	Shelli	Baida	30
18	Sigal	Tobias	30
19	Guy	Himuro	30

Essa consulta é essencial para apoiar a diretoria na tomada de decisão sobre reorganizações específicas nos Estados Unidos, ao mostrar exatamente quais funcionários estão alocados em unidades desse país.

GABARITO EXPLICATIVO

3

Questão 3

O RH está estudando perfis de funcionários que já passaram por diferentes cargos, com o objetivo de mapear potenciais candidatos para promoções internas e planos de carreira mais versáteis.

Crie um código que liste os IDs, os nomes e os sobrenomes dos funcionários cujo cargo atual é diferente de algum cargo que já ocuparam anteriormente.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

Para listar os funcionários cujo cargo atual é diferente de algum cargo que já ocuparam anteriormente, utilizamos uma subquery correlacionada com a função **EXISTS**. A consulta é composta por uma única subquery aninhada. Abaixo está o detalhamento:

1. Subquery:

Essa subquery verifica, para cada funcionário da tabela **employees**, se existe algum registro na tabela **job_history** com o mesmo **employee_id**, mas com um **job_id** (cargo) diferente do atual (**e.job_id**):

- ✓ **j.employee_id = e.employee_id**: garante que estamos comparando o histórico do mesmo funcionário analisado na consulta principal.
- ✓ **j.job_id <> e.job_id**: filtra apenas os casos em que o cargo atual é diferente do cargo registrado anteriormente.
- ✓ Se essa condição for satisfeita, a subquery retorna uma linha, e a função **EXISTS** considera a condição como verdadeira:

```
SELECT
    employee_id,
    first_name,
    last_name
FROM employees e
WHERE EXISTS (
    SELECT
        employee_id
    FROM job_history j
    WHERE j.employee_id = e.employee_id
    AND j.job_id <> e.job_id
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O RH está estudando perfis de funcionários que já passaram por diferentes cargos, com o objetivo de mapear potenciais candidatos para promoções internas e planos de carreira mais versáteis.

Crie um código que liste os IDs, os nomes e os sobrenomes dos funcionários cujo cargo atual é diferente de algum cargo que já ocuparam anteriormente.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

2. Consulta principal:

Essa parte da consulta seleciona o ID, o nome e o sobrenome de todos os funcionários para os quais existe pelo menos um registro na tabela **job_history** com cargo diferente do atual:

```
SELECT
    employee_id,
    first_name,
    last_name
FROM employees e
WHERE EXISTS (
    SELECT
        employee_id
    FROM job_history j
    WHERE j.employee_id = e.employee_id
    AND j.job_id <> e.job_id
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O RH está estudando perfis de funcionários que já passaram por diferentes cargos, com o objetivo de mapear potenciais candidatos para promoções internas e planos de carreira mais versáteis.

Crie um código que liste os IDs, os nomes e os sobrenomes dos funcionários cujo cargo atual é diferente de algum cargo que já ocuparam anteriormente.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

Resultado retornado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 7 em (

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	101	Neena	Kochhar
2	102	Lex	De Haan
3	114	Den	Raphaely
4	122	Payam	Kaufling
5	176	Jonathon	Taylor
6	200	Jennifer	Whalen
7	201	Michael	Hartstein

Essa consulta identifica com precisão os funcionários que já exerceram cargos diferentes do atual, sendo ideal para o RH mapear perfis com experiência diversificada dentro da empresa. É uma ferramenta eficaz para fundamentar decisões em programas de promoção, mobilidade interna ou desenvolvimento de carreira.

GABARITO EXPLICATIVO

4

Questão 4

A equipe de gestão de desempenho está avaliando o cargo ocupado por Peter Tucker, que teve destaque em avaliações recentes. O objetivo é identificar colegas que exercem o mesmo cargo, para aplicar a mesma política de treinamentos e metas.

Crie um código que liste os nomes e sobrenomes dos funcionários que possuem o mesmo cargo que Peter Tucker e o ID de seu cargo.

(Utilize a tabela employees)

Explicação e Resultado

Para listar os nomes, sobrenomes e o cargo de todos os funcionários que ocupam o mesmo cargo de **Peter Tucker**, utilizamos uma subquery que retorna o **job_id** dele. Abaixo está o detalhamento:

1. Subquery:

Essa subquery busca o valor do **job_id** referente ao funcionário chamado Peter Tucker. Como esperamos que o nome seja único, a subquery retorna um único valor, que será utilizado como critério de comparação na consulta principal:

```
SELECT
    first_name,
    last_name,
    job_id
FROM employees
WHERE job_id = (
    SELECT job_id
    FROM employees
    WHERE first_name = 'Peter' AND last_name = 'Tucker'
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A equipe de gestão de desempenho está avaliando o cargo ocupado por Peter Tucker, que teve destaque em avaliações recentes. O objetivo é identificar colegas que exercem o mesmo cargo, para aplicar a mesma política de treinamentos e metas.

Crie um código que liste os nomes e sobrenomes dos funcionários que possuem o mesmo cargo que Peter Tucker e o ID de seu cargo.

(Utilize a tabela employees)

Explicação e Resultado

2. Consulta principal:

Aqui, a consulta busca todos os funcionários cujo **job_id** seja igual ao valor retornado pela subquery, ou seja, todos que ocupam o mesmo cargo que **Peter Tucker**:

```
SELECT
    first_name,
    last_name,
    job_id
FROM employees
WHERE job_id = (
    SELECT job_id
    FROM employees
    WHERE first_name = 'Peter' AND last_name = 'Tucker'
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A equipe de gestão de desempenho está avaliando o cargo ocupado por Peter Tucker, que teve destaque em avaliações recentes. O objetivo é identificar colegas que exercem o mesmo cargo, para aplicar a mesma política de treinamentos e metas.

Crie um código que liste os nomes e sobrenomes dos funcionários que possuem o mesmo cargo que Peter Tucker e o ID de seu cargo.

(Utilize a tabela employees)

Explicação e Resultado

Resultado retornado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 30

	FIRST_NAME	LAST_NAME	JOB_ID
1	Peter	Tucker	SA_REP
2	David	Bernstein	SA_REP
3	Peter	Hall	SA_REP
4	Christopher	Olsen	SA_REP
5	Nanette	Cambrault	SA_REP
6	Oliver	Tuvault	SA_REP
7	Janette	King	SA_REP
8	Patrick	Sully	SA_REP
9	Allan	McEwen	SA_REP
10	Lindsey	Smith	SA_REP
11	Louise	Doran	SA_REP
12	Sarath	Sewall	SA_REP
13	Clara	Vishney	SA_REP
14	Danielle	Greene	SA_REP
15	Mattea	Marvins	SA_REP
16	David	Lee	SA_REP

Essa consulta identifica rapidamente todos os colegas de Peter Tucker que ocupam a mesma função, o que facilita a aplicação de políticas de treinamento, metas e desenvolvimento de competências alinhadas ao cargo. É uma base prática e eficaz para ações de gestão de desempenho com foco em cargos específicos.

GABARITO EXPLICATIVO

5

Questão 5

O RH deseja identificar funcionários que permanecem no mesmo departamento desde a contratação, pois esses casos podem representar estabilidade ou, em alguns casos, necessidade de novos desafios para evitar estagnação.

Crie um código que liste os nomes e sobrenomes dos funcionários que nunca trabalharam em departamentos diferentes do atual.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

Para listar os nomes e sobrenomes dos funcionários que nunca trabalharam em departamentos diferentes do atual, ou seja, que permaneceram no mesmo departamento desde a contratação, fazemos o seguinte:

1. Subquery:

- ✓ Essa subquery busca no histórico de cargos (**job_history**) todos os registros do funcionário atual (**j.employee_id = e.employee_id**) que tenham um departamento diferente do atual (**j.department_id <> e.department_id**).
- ✓ Se retornar algum registro, significa que o funcionário já trabalhou em outro departamento diferente do atual:

```sql
SELECT
    first_name,
    last_name
FROM employees e
WHERE NOT EXISTS (
    SELECT employee_id
    FROM job_history j
    WHERE j.employee_id = e.employee_id
    AND j.department_id <> e.department_id
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

O RH deseja identificar funcionários que permanecem no mesmo departamento desde a contratação, pois esses casos podem representar estabilidade ou, em alguns casos, necessidade de novos desafios para evitar estagnação.

Crie um código que liste os nomes e sobrenomes dos funcionários que nunca trabalharam em departamentos diferentes do atual.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

2. Consulta principal:

- ✓ A consulta principal seleciona os nomes e sobrenomes dos funcionários (**employees**).
- ✓ A cláusula **WHERE NOT EXISTS** verifica se não existe nenhum registro na subquery interna para aquele funcionário.
- ✓ Ou seja, traz apenas os funcionários para os quais não existem registros no **job_history** indicando departamento diferente do atual:

```
SELECT
    first_name,
    last_name
FROM employees e
WHERE NOT EXISTS (
    SELECT employee_id
    FROM job_history j
    WHERE j.employee_id = e.employee_id
    AND j.department_id <> e.department_id
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

O RH deseja identificar funcionários que permanecem no mesmo departamento desde a contratação, pois esses casos podem representar estabilidade ou, em alguns casos, necessidade de novos desafios para evitar estagnação.

Crie um código que liste os nomes e sobrenomes dos funcionários que nunca trabalharam em departamentos diferentes do atual.

(Utilize as tabelas employees e job_history)

Explicação e Resultado

Resultado retornado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 103

	FIRST_NAME	LAST_NAME
1	Steven	King
2	Alexander	Hunold
3	Bruce	Ernst
4	David	Austin
5	Valli	Pataballa
6	Diana	Lorentz
7	Nancy	Greenberg
8	Daniel	Faviet
9	John	Chen
10	Ismael	Sciarra
11	Jose Manuel	Urman
12	Luis	Popp
13	Alexander	Khoo
14	Shelli	Baida
15	Sigal	Tobias
16	Guy	Himuro

Essa consulta é útil para o RH identificar funcionários que permanecem no mesmo departamento desde a contratação, indicando estabilidade ou, eventualmente, a necessidade de novos desafios para evitar estagnação na carreira. Assim, o RH pode tomar decisões mais assertivas sobre movimentações internas e planos de desenvolvimento.

GABARITO EXPLICATIVO

6

Questão 6

A área de expansão imobiliária está estudando a concentração de departamentos por cidade. O objetivo é avaliar a viabilidade de unificar unidades administrativas em uma única sede nas cidades com mais de um departamento.

Crie um código que liste os nomes das cidades que eventualmente possuam mais de um departamento.

(Utilize as tabelas locations e departments)

Explicação e Resultado

Para listar os nomes das cidades que possuem mais de um departamento, com o intuito de avaliar a concentração administrativa e a viabilidade de unificação, fazemos o seguinte:

1. Subquery:

- ✓ Agrupa os departamentos pelo **location_id**, ou seja, por localidade.
- ✓ Utiliza a cláusula **HAVING** para filtrar apenas os **location_id** que têm mais de um departamento associado.
- ✓ O resultado é uma lista de localidades que concentram múltiplos departamentos:

```
SELECT city
FROM locations
WHERE location_id IN (
    SELECT location_id
    FROM departments
    GROUP BY location_id
    HAVING COUNT(department_id) > 1
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

A área de expansão imobiliária está estudando a concentração de departamentos por cidade. O objetivo é avaliar a viabilidade de unificar unidades administrativas em uma única sede nas cidades com mais de um departamento.

Crie um código que liste os nomes das cidades que eventualmente possuam mais de um departamento.

(Utilize as tabelas locations e departments)

Explicação e Resultado

2. Consulta principal:

- ✓ Seleciona os nomes das cidades da tabela **locations**.
- ✓ Usa o **location_id** da subquery para filtrar apenas as cidades que possuem mais de um departamento:

```
SELECT city
FROM locations
WHERE location_id IN (
    SELECT location_id
    FROM departments
    GROUP BY location_id
    HAVING COUNT(department_id) > 1
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6 Questão 6

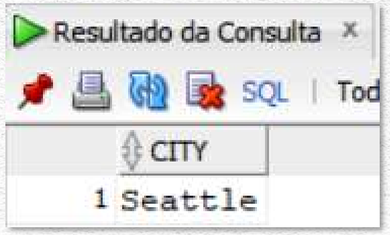
A área de expansão imobiliária está estudando a concentração de departamentos por cidade. O objetivo é avaliar a viabilidade de unificar unidades administrativas em uma única sede nas cidades com mais de um departamento.

Crie um código que liste os nomes das cidades que eventualmente possuam mais de um departamento.

(Utilize as tabelas locations e departments)

Explicação e Resultado

Resultado retornado:



Essa consulta oferece à área de expansão imobiliária uma visão clara das cidades com maior concentração de departamentos. Essas informações são fundamentais para avaliar a viabilidade de centralizar operações em uma única sede, otimizando recursos e estrutura administrativa.

GABARITO EXPLICATIVO

7

Questão 7

A área financeira precisa identificar quais departamentos têm políticas de comissão em vigor. Esse levantamento é essencial para revisar os impactos dessas comissões nos custos operacionais de cada área.

Crie um código que liste os IDs e nomes dos departamentos que possuem funcionários com comissão. **(Utilize as tabelas departments e employees)**

Explicação e Resultado

Para listar os IDs e nomes dos departamentos que possuem funcionários com comissão, para que a área financeira possa revisar os custos operacionais relacionados a essas políticas, fazemos o seguinte:

1. Subquery:

- ✓ Essa subquery busca os **department_id** de funcionários que possuem valor de comissão (**commission_pct**) preenchido.
- ✓ A condição **IS NOT NULL** garante que somente funcionários com comissão ativa sejam considerados.
- ✓ O resultado é uma lista de departamentos que têm ao menos um funcionário com comissão:

```
SELECT
    department_id,
    department_name
FROM departments
WHERE department_id IN (
    SELECT
        department_id
    FROM employees
    WHERE commission_pct IS NOT NULL
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área financeira precisa identificar quais departamentos têm políticas de comissão em vigor. Esse levantamento é essencial para revisar os impactos dessas comissões nos custos operacionais de cada área.

Crie um código que liste os IDs e nomes dos departamentos que possuem funcionários com comissão. **(Utilize as tabelas departments e employees)**

Explicação e Resultado

2. Consulta principal:

- ✓ A consulta principal seleciona os **department_id** e **department_name** da tabela **departments**.
- ✓ Usa a subquery para filtrar apenas os departamentos que possuem funcionários com comissão cadastrada:

```
SELECT
    department_id,
    department_name
FROM departments
WHERE department_id IN (
    SELECT
        department_id
    FROM employees
    WHERE commission_pct IS NOT NULL
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área financeira precisa identificar quais departamentos têm políticas de comissão em vigor. Esse levantamento é essencial para revisar os impactos dessas comissões nos custos operacionais de cada área.

Crie um código que liste os IDs e nomes dos departamentos que possuem funcionários com comissão. **(Utilize as tabelas departments e employees)**

Explicação e Resultado

Resultado retornado:

Essa consulta permite que a área financeira identifique com precisão quais departamentos estão aplicando políticas de comissão. Isso é essencial para revisar os impactos financeiros dessas práticas e apoiar decisões relacionadas a orçamento e incentivos internos.

GABARITO EXPLICATIVO

8

Questão 8

O setor de planejamento de carreira está analisando os cargos mais populares da empresa para desenvolver trilhas de capacitação e programas de crescimento profissional voltados a essas funções.

Crie um código que liste os títulos dos cargos que são ocupados por mais de 3 funcionários.

(Utilize as tabelas jobs e employees)

Explicação e Resultado

Para listar os títulos dos cargos que são ocupados por mais de 3 funcionários, para que o setor de planejamento de carreira possa focar seus programas de capacitação nas funções mais populares, fazemos o seguinte:

1. Subquery:

- ✓ Agrupa os funcionários pela coluna **job_id**, ou seja, por cargo.
- ✓ A cláusula **HAVING COUNT(*) > 3** filtra apenas os cargos que têm mais de três funcionários associados.
- ✓ O resultado é uma lista de **job_id** representando os cargos mais populares da empresa:

```
SELECT
    job_title
FROM jobs
WHERE job_id IN (
    SELECT
        job_id
    FROM employees
    GROUP BY job_id
    HAVING COUNT(*) > 3
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

O setor de planejamento de carreira está analisando os cargos mais populares da empresa para desenvolver trilhas de capacitação e programas de crescimento profissional voltados a essas funções.

Crie um código que liste os títulos dos cargos que são ocupados por mais de 3 funcionários.

(Utilize as tabelas jobs e employees)

Explicação e Resultado

2. Consulta principal:

- ✓ A consulta principal seleciona os **job_title** da tabela **jobs**.
- ✓ Filtra apenas os títulos cujos **job_id** aparecem na lista da subquery, ou seja, os cargos ocupados por mais de três funcionários:

```
SELECT
    job_title
FROM jobs
WHERE job_id IN (
    SELECT
        job_id
    FROM employees
    GROUP BY job_id
    HAVING COUNT(*) > 3
);
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

O setor de planejamento de carreira está analisando os cargos mais populares da empresa para desenvolver trilhas de capacitação e programas de crescimento profissional voltados a essas funções.

Crie um código que liste os títulos dos cargos que são ocupados por mais de 3 funcionários.

(Utilize as tabelas jobs e employees)

Explicação e Resultado

Resultado retornado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 8

	JOB_TITLE
1	Accountant
2	Programmer
3	Purchasing Clerk
4	Sales Manager
5	Sales Representative
6	Shipping Clerk
7	Stock Clerk
8	Stock Manager

Essa consulta fornece ao setor de planejamento de carreira uma base concreta para identificar os cargos mais recorrentes na empresa. Com essas informações, é possível desenvolver trilhas de capacitação e programas de crescimento profissional focados nas funções que impactam um maior número de colaboradores.

MÓDULO 14

BLOCOS ANÔNIMOS

BLOCOS ANÔNIMOS

BLOCOS ANÔNIMOS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 14 BLOCOS ANÔNIMOS

01	O QUE É UM BLOCO ANÔNIMO?	661
02	COMO CRIAR UM BLOCO ANÔNIMO	662
03	UTILIZANDO FUNÇÕES EM BLOCOS ANÔNIMOS	665
04	BLOCOS ANÔNIMOS ANINHADOS	670
05	RESUMO DO MÓDULO	672
06	EXERCÍCIOS	673
07	GABARITO EXPLICATIVO	678

O que é um BLOCO ANÔNIMO?

Um BLOCO ANÔNIMO é um bloco de código sem nome.

Aprender sua estrutura será muito importante no momento em que falarmos de Functions e Procedures.

Um bloco anônimo é composto por três partes:

BLOCO DE EXECUÇÃO:

- É o único bloco obrigatório para um bloco anônimo.
- Inicia com a palavra BEGIN e termina com a palavra END.
- Dentro dele, escrevemos as instruções que queremos executar.

BLOCO DE DECLARAÇÃO:

- É onde faremos, por exemplo, a declaração de variáveis.

BLOCO DE EXCEÇÃO:

- Contém tratamentos de erros.

SINTAXE:

```
[DECLARE
   -- área de declarações
BEGIN
   -- blocos de código
[EXCEPTION
   -- tratamento de erros
END;
```

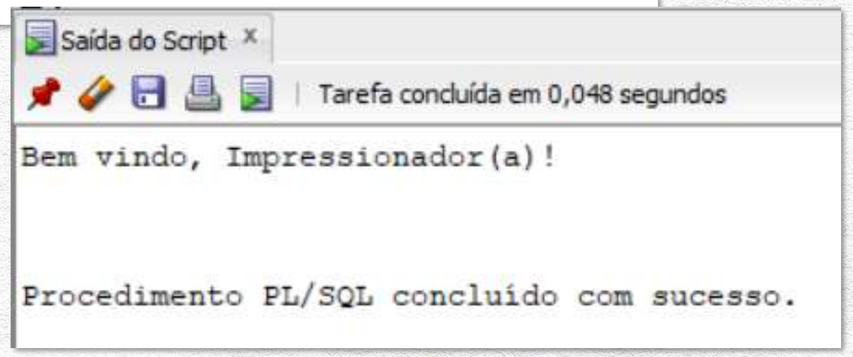
Como criar um BLOCO ANÔNIMO

Vamos iniciar com uma estrutura bem básica de bloco anônimo para entender o conceito:

Neste exemplo, teremos apenas o bloco de execução, que exibirá na tela uma mensagem de boas vindas:

Repare que um bloco de execução inicia com um BEGIN e termina com um END. Para exibir na tela a mensagem, utilizamos o comando dbms_output.put_line().

```
SET SERVEROUTPUT ON -- inicia o serviço de impressão de resultado na tela

BEGIN
    dbms_output.put_line('Bem vindo, Impressionador(a)!');
END;
```



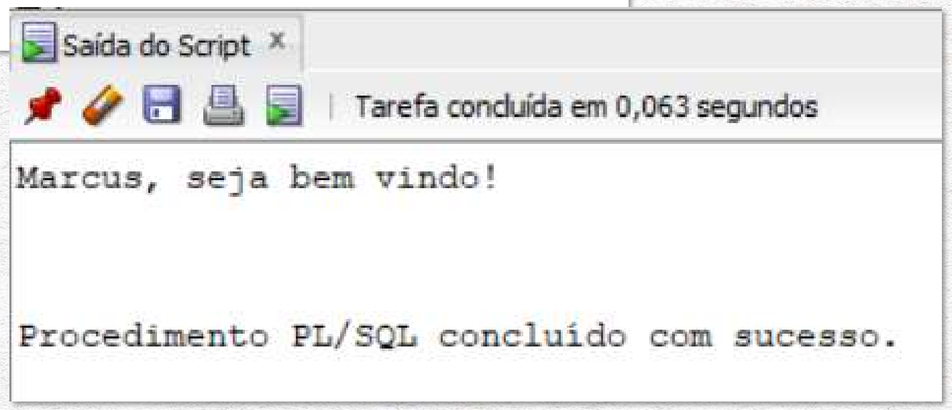
Como criar um BLOCO ANÔNIMO

Vamos aprofundar mais um pouquinho agora, acrescentando um bloco de declaração antes do nosso bloco de execução para poder utilizar uma variável:

Perceba que no bloco de declaração criamos a variável nome, e a utilizamos dentro do bloco de execução para atribuir um texto a ela (Marcus). Em seguida, informamos essa variável dentro do nosso print.

A título de ilustração, concatenamos no print, logo após a variável, o texto ‘, seja bem vindo!’, utilizando o comando ||.

```
DECLARE
    nome VARCHAR2(50);
BEGIN
    nome := 'Marcus';
    dbms_output.put_line(nome || ', seja bem vindo!');
END;
```



Como criar um BLOCO ANÔNIMO

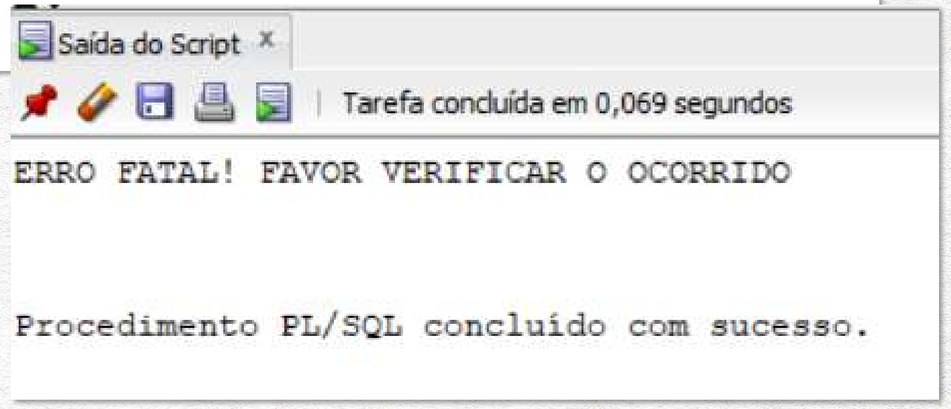
Por fim, vejamos como podemos utilizar um bloco de exceção (instrução EXCEPTION) dentro do bloco de execução para tratar erros:

Repare que no bloco de declaração criamos duas variáveis (var1 e var2 do tipo inteiro (INT) 1

Porém, no bloco de execução atribuímos um texto à var2 (A), em vez de um número do tipo inteiro, para, no print, efetuarmos a soma da var1 com a var2 2

```
DECLARE 1
    var1 INT;
    var2 INT;
BEGIN
    var1 := 100;
    var2 := 'A'; 2
    dbms_output.put_line(var1 + var2);
    EXCEPTION 3
    WHEN OTHERS THEN
        dbms_output.put_line('ERRO FATAL! FAVOR VERIFICAR O OCORRIDO');
END;
```

Isso fatalmente resultaria em um erro. Para que isso não aconteça, criamos, dentro do bloco de execução, um bloco de exceção que, ao checar a tentativa de execução de um erro, retorna a mensagem que inserimos no nosso print (Erro Fatal! Favor verificar o ocorrido) 3



Utilizando funções em BLOCOS ANÔNIMOS

Veremos agora como podemos fazer uso das funções Single Row (de Número, String e Data) dentro de blocos anônimos para formatarmos nossos resultados.

Vamos a um exemplo: imagine que em um determinado mês, uma empresa enviou 137 propostas no valor de R$1.000 e conseguiu fechar 25 contratos. Queremos calcular a Receita Total (em R$) e a Taxa de Conversão (com 4 casas decimais).

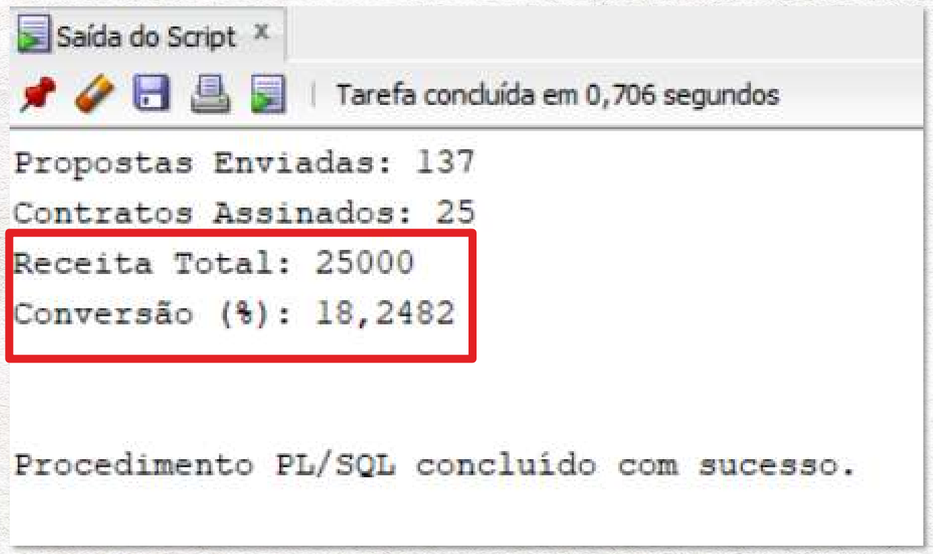
Para isso, declaramos nossas variáveis no bloco de declaração 1 , informamos os valores das variáveis no bloco de execução, fizemos os cálculos para descobrir a Receita Total e a Taxa de Conversão 2 , assim como imprimimos na tela as informações: Propostas Enviadas, Contratos Assinados, Receita Total e Conversão (%) 3 .

```
DECLARE 1
    vPropostasEnviadas INT;
    vContratosAssinados INT;
    vValorContrato NUMBER(10, 2);
    vConversao NUMBER(10, 4);
    vReceita NUMBER(10, 2);
BEGIN
    vPropostasEnviadas := 137; 2
    vContratosAssinados := 25;
    vValorContrato := 1000;
    vReceita := vContratosAssinados * vValorContrato;
    vConversao := (vContratosAssinados / vPropostasEnviadas) * 100;
    dbms_output.put_line('Propostas Enviadas: ' || vPropostasEnviadas); 3
    dbms_output.put_line('Contratos Assinados: ' || vContratosAssinados);
    dbms_output.put_line('Receita Total: ' || vReceita);
    dbms_output.put_line('Conversão (%): ' || vConversao);
END;
```

Utilizando funções em BLOCOS ANÔNIMOS

Ao imprimirmos na tela, tivemos o resultado abaixo.

Repare na Receita Total e na Conversão (%):



E se quisermos que esse trecho do resultado apareça formatado conforme abaixo?

```
Receita Total: R$25.000,00
Conversão (%): 18,25
```

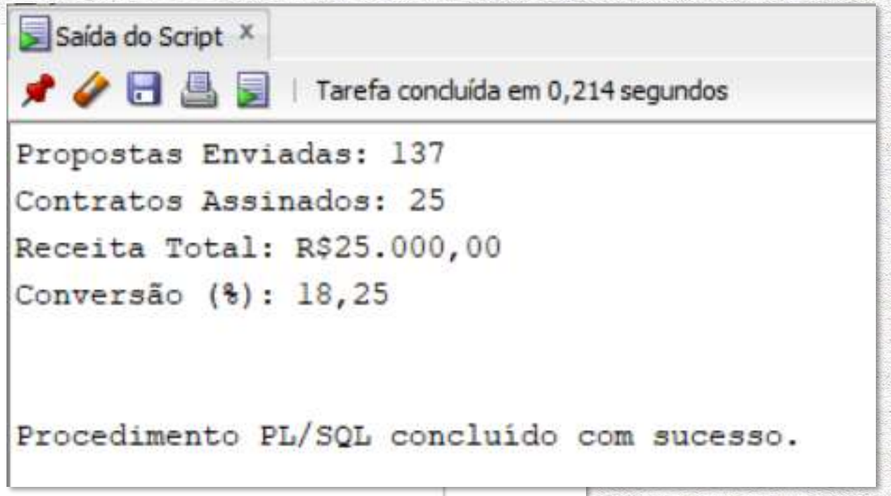
Para isso, precisamos aplicar algumas Funções Single Row ao nosso código.

Vejamos na próxima página.

Utilizando funções em BLOCOS ANÔNIMOS

Repare na parte final do nosso código: aplicamos as funções TRIM, TO_CHAR e ROUND para chegar ao resultado desejado:

```
DECLARE
    vPropostasEnviadas INT;
    vContratosAssinados INT;
    vValorContrato NUMBER(10, 2);
    vConversao NUMBER(10, 4);
    vReceita NUMBER(10, 2);
BEGIN
    vPropostasEnviadas := 137;
    vContratosAssinados := 25;
    vValorContrato := 1000;
    vReceita := vContratosAssinados * vValorContrato;
    vConversao := (vContratosAssinados / vPropostasEnviadas) * 100;
    dbms_output.put_line('Propostas Enviadas: ' || vPropostasEnviadas);
    dbms_output.put_line('Contratos Assinados: ' || vContratosAssinados);
    dbms_output.put_line('Receita Total: ' || TRIM(TO_CHAR(vReceita, 'L999G999D99')));
    dbms_output.put_line('Conversão (%): ' || ROUND(vConversao, 2));
END;
```



Utilizando funções em BLOCOS ANÔNIMOS

Usamos o TRIM para remover espaços desnecessários;
E o TO_CHAR para mostrar o valor no formato de moeda, em R$, utilizando a formatação 'L999G999D99' ao final, sendo:

L = R$;
9 = um número que serve como padrão para configurar quantos dígitos numéricos podem compor o trecho;
G = O ponto (.) que separa o milhar;
D = a vírgula (,) que separa as casas decimais;

```
dbms_output.put_line('Receita Total: ' || TRIM(TO_CHAR(vReceita, 'L999G999D99')));
```

```
Receita Total: R$25.000,00
```

Utilizando funções em BLOCOS ANÔNIMOS

Usamos o ROUND para arredondar o número e mostrar somente a quantidade de casas decimais informada no segundo argumento (no caso, 2):

```
dbms_output.put_line('Conversão (%): ' || ROUND(vConversao, 2));
```

```
Conversão (%): 18,25
```

BLOCOS ANÔNIMOS ANINHADOS

BLOCOS ANÔNIMOS ANINHADOS é como chamamos quando criamos bloco(s) interno(s) dentro de um bloco externo.

Veja só este exemplo:

Repare que criamos um bloco anônimo mais externo, declarando a variável vBloco1 em seu bloco de declaração 1; em seguida, em seu bloco de execução, imprimimos essa variável na tela 2.

Depois, dentro desse bloco externo, criamos um novo bloco anônimo para declarar a variável vBloco2 e imprimir na tela novamente a vBloco1 criada no bloco externo, bem como a recém criada vBloco2 3.

Fechamos o bloco interno (END) 4 para, logo após, imprimir novamente a vBloco1.

```
DECLARE
    vBloco1 INT; 1
BEGIN
    vBloco1 := 10; 2
    dbms_output.put_line('Variável do bloco 1: ' || vBloco1);

    DECLARE 3
        vBloco2 INT;
    BEGIN
        vBloco2 := 20;
        dbms_output.put_line('Variável do bloco 1: ' || vBloco1);
        dbms_output.put_line('Variável do bloco 2: ' || vBloco2);
    END; 4

    dbms_output.put_line('Variável do bloco 1: ' || vBloco1);

END;
```

Veja o resultado:

```
Saída do Script
Tarefa concluída em 0,21 segundos
Variável do bloco 1: 10
Variável do bloco 1: 10
Variável do bloco 2: 20
Variável do bloco 1: 10


Procedimento PL/SQL concluído com sucesso.
```

BLOCOS ANÔNIMOS ANINHADOS

Agora, preste atenção no que aconteceria se resolvêssemos mudar o último print da tela 1 e, em vez de imprimir a vBloco1 novamente, tentássemos imprimir a vBloco2 (dentro do bloco externo, mas fora do bloco interno em que ela foi declarada):

```
DECLARE
    vBloco1 INT;
BEGIN
    vBloco1 := 10;
    dbms_output.put_line('Variável do bloco 1: ' || vBloco1);

    DECLARE
        vBloco2 INT;
    BEGIN
        vBloco2 := 20;
        dbms_output.put_line('Variável do bloco 1: ' || vBloco1);
        dbms_output.put_line('Variável do bloco 2: ' || vBloco2);
    END;

    dbms_output.put_line('Variável do bloco 1: ' || vBloco2);

END;
```

1

Repare abaixo no resultado: o sistema retornaria o erro “o identificador ‘VBLOCO2’ deve ser declarado” 2 .

Isso acontece porque tudo que se encontra dentro de um bloco interno só existe dentro dele.

Portanto, no nosso exemplo, para o bloco externo, é como se a variável vBloco2 não existisse.

Mas o contrário é possível: utilizar uma variável criada no bloco externo dentro do interno, pois o bloco interno reconhece tudo que foi criado no bloco mais externo.

Saída do Script ×

Tarefa concluída em 0,115 segundos

```
END;
Relatório de erros -
ORA-06550: linha 15, coluna 54:
PLS-00201: o identificador 'VBLOCO2' deve ser declarado
ORA-06550: linha 15, coluna 5:
PL/SQL: Statement ignored
06550. 00000 -  "line %s, column %s:\n%s"
*Cause:    Usually a PL/SQL compilation error.
*Action:
```

2

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. BLOCOS ANÔNIMOS são blocos de códigos sem nome, que executam uma série de comandos.
2. BLOCOS ANÔNIMOS são divididos em 3 partes: Bloco de Declaração, Bloco de Execução e Bloco de Exceção.
3. Apenas o Bloco de Execução (BEGIN ... END) é obrigatório.
4. Comentários nos blocos de códigos são muito úteis para clareza do que o código faz.
5. BLOCOS ANÔNIMOS são a introdução para Functions e Procedures.

EXERCÍCIOS

1

Questão 1

Ao iniciar o expediente, é comum exibir uma mensagem de boas-vindas no sistema interno da empresa.

Crie um bloco anônimo que exiba a mensagem "Bom dia, equipe!" utilizando *dbms_output.put_line*.

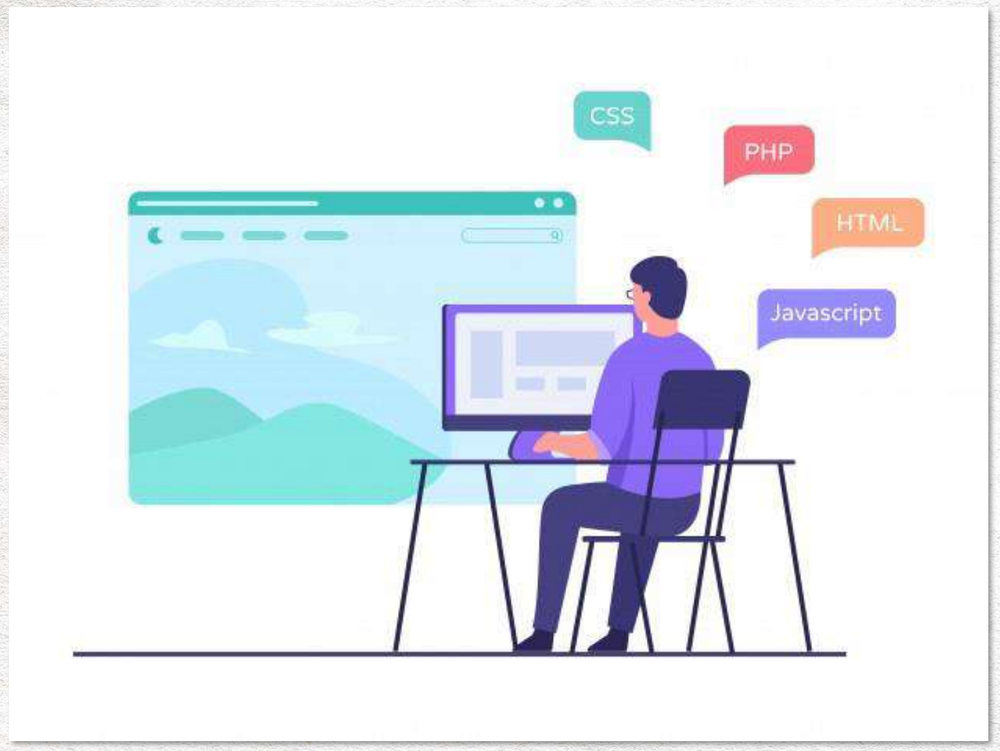
2

Questão 2

É necessário exibir a data atual no formato 'DD/MM/YYYY' no relatório diário.

Crie um bloco anônimo que declare uma variável para armazenar a data atual e exiba-a formatada.

EXERCÍCIOS



3

Questão 3

Um funcionário que ganha R$ 3.000,00 recebeu um bônus de 10% sobre seu salário.

Crie um bloco anônimo que declare variáveis para o salário e o bônus, calcule o valor do bônus e exiba o resultado.

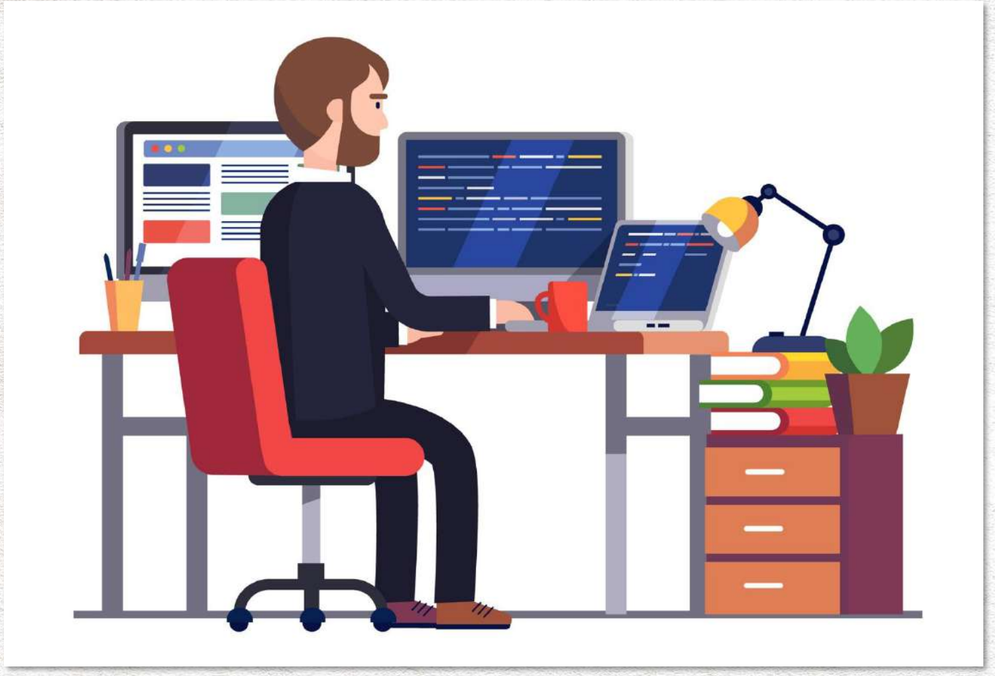
4

Questão 4

A empresa realiza transações em dólares e precisa converter valores para reais.

Crie um bloco anônimo que converta $10.000 (dólares) para reais, considerando a cotação de R$5,25.

EXERCÍCIOS



5

Questão 5

Para fins de benefícios, é necessário calcular a idade de um funcionário com base na data de nascimento.

Crie um bloco anônimo que calcule e exiba a idade de um funcionário nascido em 15/08/1990.

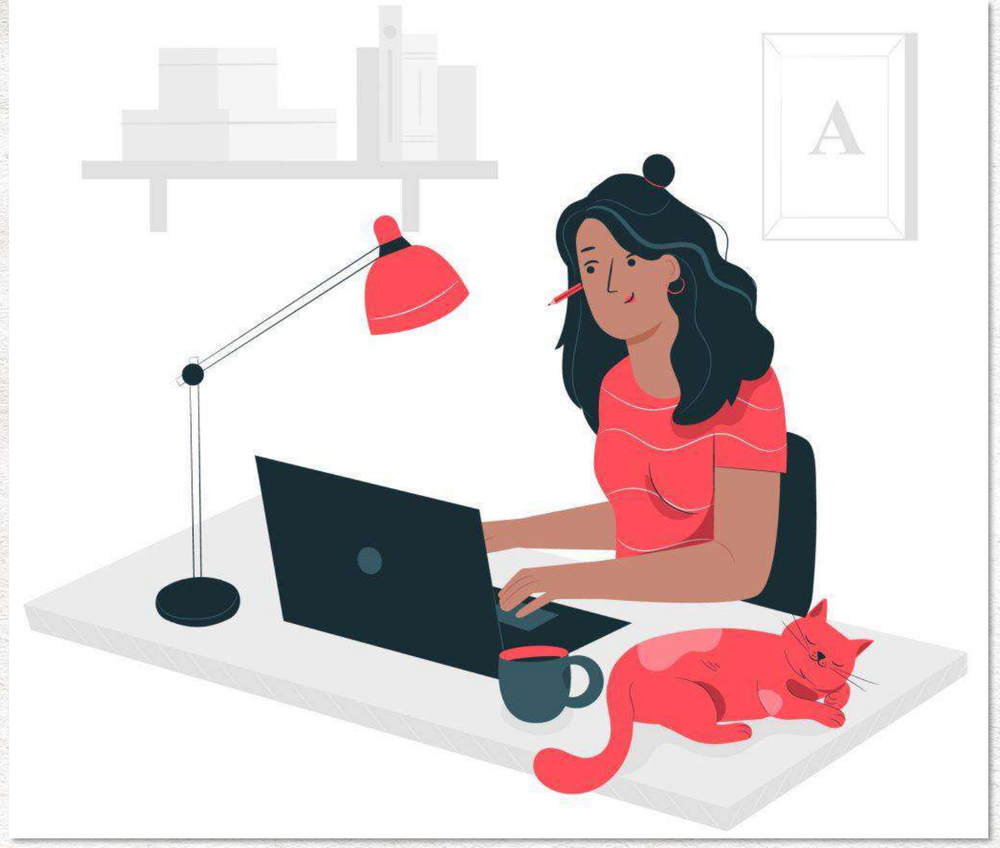
6

Questão 6

Ao gerar crachás, é necessário exibir o nome completo do funcionário em letras maiúsculas.

Crie um bloco anônimo que declare variáveis para o primeiro e último nome, concatene-os e exiba o nome completo em maiúsculas.

EXERCÍCIOS



7

Questão 7

Durante cálculos financeiros, pode ocorrer divisão por zero.

Crie um bloco anônimo que tente dividir um número por outro e trate a exceção, exibindo uma mensagem de erro, caso ocorra divisão por zero.

Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção ZERO_DIVIDE para tratar este tipo de erro.

8

Questão 8

A empresa precisa converter temperaturas de Celsius (°C) para Fahrenheit (°F) em relatórios técnicos.

Crie um bloco anônimo que converta 30 graus Celsius para Fahrenheit e exiba o resultado.

Dica: a fórmula para conversão de °C para °F é: F = (C * 9/5) + 32.

EXERCÍCIOS

9

Questão 9

Um sistema tenta converter texto para número e precisa evitar falhas se o valor não for numérico.

Crie um bloco anônimo que tente converter uma string para número e, em caso de falha, trate a exceção exibindo uma mensagem de erro.

Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção VALUE_ERROR para tratar este tipo de erro.

10

Questão 10

Uma aplicação educacional precisa calcular a média de um aluno.

Crie um bloco anônimo que calcule a média entre três notas e a exiba na tela com duas casas decimais.

GABARITO EXPLICATIVO

1 Questão 1

Ao iniciar o expediente, é comum exibir uma mensagem de boas-vindas no sistema interno da empresa.

Crie um bloco anônimo que exiba a mensagem "Bom dia, equipe!" utilizando *dbms_output.put_line*.

Explicação e Resultado

Para exibir uma mensagem de boas-vindas no sistema interno ao iniciar o expediente, utilizamos um bloco anônimo PL/SQL com o procedimento **dbms_output.put_line**, responsável por imprimir textos no console do Oracle.

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
BEGIN
    dbms_output.put_line('Bom dia, equipe!');
END;
```

- ✓ **BEGIN e END:** delimitam o corpo do bloco anônimo.
- ✓ **dbms_output.put_line('Bom dia, equipe!'):** imprime a mensagem no console Oracle.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

Ao iniciar o expediente, é comum exibir uma mensagem de boas-vindas no sistema interno da empresa.

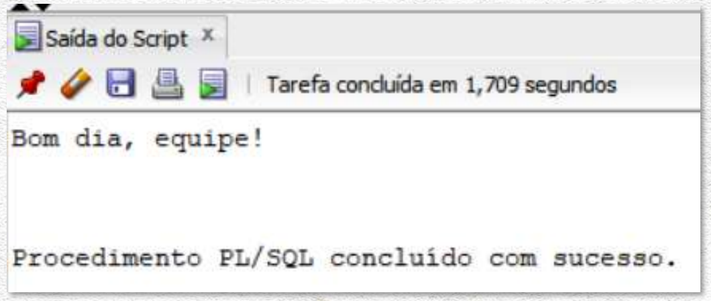
Crie um bloco anônimo que exiba a mensagem "Bom dia, equipe!" utilizando *dbms_output.put_line*.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

BEGIN
    dbms_output.put_line('Bom dia, equipe!');
END;
```

Exibimos o seguinte resultado:



GABARITO EXPLICATIVO

2 Questão 2

É necessário exibir a data atual no formato 'DD/MM/YYYY' no relatório diário.

Crie um bloco anônimo que declare uma variável para armazenar a data atual e exiba-a formatada.

Explicação e Resultado

Para exibir a data atual no formato 'DD/MM/YYYY' em um relatório diário, utilizamos um bloco anônimo PL/SQL com declaração de variável e formatação de data.

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_data_atual DATE := SYSDATE;
BEGIN
    dbms_output.put_line('Data atual: ' || TO_CHAR(v_data_atual, 'DD/MM/YYYY'));
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

É necessário exibir a data atual no formato 'DD/MM/YYYY' no relatório diário.

Crie um bloco anônimo que declare uma variável para armazenar a data atual e exiba-a formatada.

Explicação e Resultado

- ✓ **DECLARE**: abre a seção de declaração de variáveis.
- ✓ **v_data_atual DATE := SYSDATE;**: cria a variável v_data_atual e armazena nela a data e hora atual do sistema com a função SYSDATE.
- ✓ **BEGIN**: inicia o bloco de execução.
- ✓ **dbms_output.put_line(...);**: imprime a mensagem formatada com a data.
- ✓ **TO_CHAR(v_data_atual, 'DD/MM/YYYY')**: converte o valor da data para o formato desejado com dia, mês e ano separados por barra.
- ✓ **END;**: finaliza o bloco PL/SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

É necessário exibir a data atual no formato 'DD/MM/YYYY' no relatório diário.

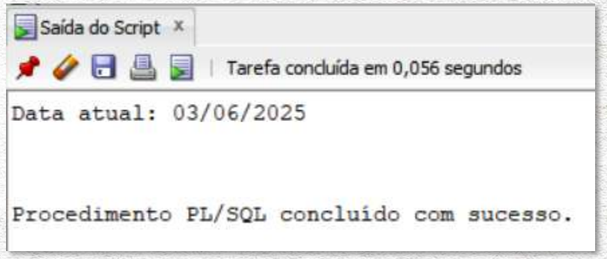
Crie um bloco anônimo que declare uma variável para armazenar a data atual e exiba-a formatada.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_data_atual DATE := SYSDATE;
BEGIN
    dbms_output.put_line('Data atual: ' || TO_CHAR(v_data_atual, 'DD/MM/YYYY'));
END;
```

Exibimos o resultado ao lado, muito útil para exibir dinamicamente a data atual nos relatórios diários da empresa, padronizando a apresentação das datas no formato brasileiro e garantindo clareza nas informações:



Observação: A data exibida será diferente quando você executar, pois exibirá a data atual no momento da execução.

GABARITO EXPLICATIVO

3 Questão 3

Um funcionário que ganha R$ 3.000,00 recebeu um bônus de 10% sobre seu salário.

Crie um bloco anônimo que declare variáveis para o salário e o bônus, calcule o valor do bônus e exiba o resultado.

Explicação e Resultado

Para calcular o valor de um bônus de 10% sobre um salário de R$ 3.000,00, utilizamos um bloco anônimo PL/SQL com declaração de variáveis, cálculo e exibição do valor formatado.

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_salario NUMBER := 3000.00;
    v_bonus NUMBER;
BEGIN
    v_bonus := v_salario * 0.10;
    dbms_output.put_line('Valor do bônus: R$' || TO_CHAR(v_bonus, '9999.99'));
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Um funcionário que ganha R$ 3.000,00 recebeu um bônus de 10% sobre seu salário.

Crie um bloco anônimo que declare variáveis para o salário e o bônus, calcule o valor do bônus e exiba o resultado.

Explicação e Resultado

- ✓ **DECLARE**: define as variáveis do bloco.
- ✓ **v_salario NUMBER := 3000.00;**: variável que armazena o valor do salário.
- ✓ **v_bonus NUMBER;**: variável que armazenará o valor calculado do bônus.
- ✓ **BEGIN**: inicia a execução do bloco.
- ✓ **v_bonus := v_salario * 0.10;**: realiza o cálculo de 10% sobre o salário.
- ✓ **dbms_output.put_line(...);**: exibe o valor do bônus formatado.
- ✓ **TO_CHAR(v_bonus, '9999.99')**: formata o número com duas casas decimais, para melhor apresentação como valor monetário.
- ✓ **END;**: finaliza o bloco PL/SQL.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Um funcionário que ganha R$ 3.000,00 recebeu um bônus de 10% sobre seu salário.

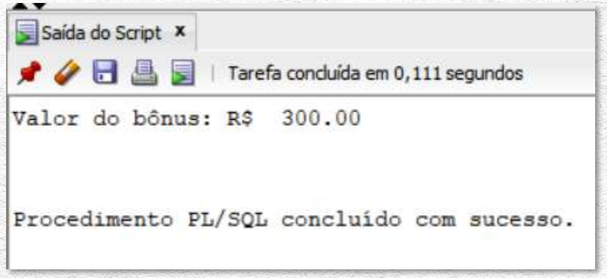
Crie um bloco anônimo que declare variáveis para o salário e o bônus, calcule o valor do bônus e exiba o resultado.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_salario NUMBER := 3000.00;
    v_bonus NUMBER;
BEGIN
    v_bonus := v_salario * 0.10;
    dbms_output.put_line('Valor do bônus: R$' || TO_CHAR(v_bonus, '9999.99'));
END;
```

Exibimos o resultado ao lado, muito útil para simular ou registrar cálculos simples de remuneração variável, como bônus salariais. Pode ser reutilizado em rotinas de folha de pagamento ou relatórios gerenciais internos:



GABARITO EXPLICATIVO

4

Questão 4

A empresa realiza transações em dólares e precisa converter valores para reais.

Crie um bloco anônimo que converta $10.000 (dólares) para reais, considerando a cotação de R$5,25.

Explicação e Resultado

Para realizar a conversão de $10.000 dólares para reais, com uma cotação de R$5,25, utilizamos um bloco anônimo PL/SQL com variáveis para armazenar os valores e uma operação de multiplicação para calcular a conversão.

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_dolar NUMBER := 10000;
    v_cotacao NUMBER := 5.25;
    v_reais NUMBER;
BEGIN
    v_reais := v_dolar * v_cotacao;
    dbms_output.put_line('Valor em reais: R$' || TO_CHAR(v_reais, 'L999G999D99'));
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A empresa realiza transações em dólares e precisa converter valores para reais.

Crie um bloco anônimo que converta $10.000 (dólares) para reais, considerando a cotação de R$5,25.

Explicação e Resultado

Declaração das variáveis:

- ✓ **v_dolar**: armazena o valor original em dólares (10.000).
- ✓ **v_cotacao**: representa a taxa de conversão (5,25 reais por dólar).
- ✓ **v_reais**: variável que armazenará o resultado da conversão.

Bloco de execução:

- ✓ **v_reais := v_dolar * v_cotacao;**: realiza a conversão multiplicando o valor em dólares pela cotação.
- ✓ **dbms_output.put_line(...);**: exibe o valor convertido formatado:
- ✓ **TO_CHAR(v_reais, 'L999G999D99')**: formata o número como valor monetário (usando símbolo de moeda local com separadores de milhar e decimais).

Obs.: O formato **'L999G999D99'** usa **L** para símbolo de moeda, **G** como separador de milhar e **D** como separador decimal, respeitando a configuração regional do banco Oracle.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A empresa realiza transações em dólares e precisa converter valores para reais.

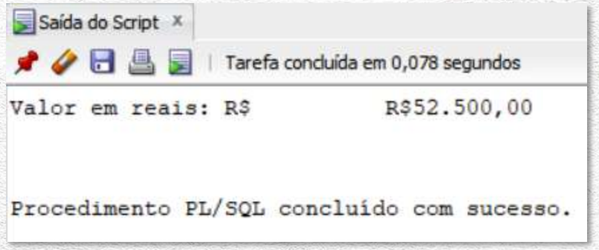
Crie um bloco anônimo que converta $10.000 (dólares) para reais, considerando a cotação de R$5,25.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_dolar NUMBER := 10000;
    v_cotacao NUMBER := 5.25;
    v_reais NUMBER;
BEGIN
    v_reais := v_dolar * v_cotacao;
    dbms_output.put_line('Valor em reais: R$' || TO_CHAR(v_reais, 'L999G999D99'));
END;
```

Exibimos o resultado ao lado, muito útil em relatórios financeiros, simulações de custos internacionais e sistemas de controle de câmbio:



GABARITO EXPLICATIVO

5

Questão 5

Para fins de benefícios, é necessário calcular a idade de um funcionário com base na data de nascimento.

Crie um bloco anônimo que calcule e exiba a idade de um funcionário nascido em 15/08/1990.

Explicação e Resultado

Para calcular e exibir a idade de um funcionário com base na data de nascimento (15/08/1990), utilizamos um bloco anônimo PL/SQL da seguinte forma:

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_data_nascimento DATE := TO_DATE('15/02/1990', 'DD/MM/YYYY');
    v_idade NUMBER;
BEGIN
    v_idade := TRUNC(MONTHS_BETWEEN(SYSDATE, v_data_nascimento) / 12);
    dbms_output.put_line('Idade: ' || v_idade || ' anos');
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Para fins de benefícios, é necessário calcular a idade de um funcionário com base na data de nascimento.

Crie um bloco anônimo que calcule e exiba a idade de um funcionário nascido em 15/08/1990.

Explicação e Resultado

Declaração de variáveis:

- ✓ **v_data_nascimento**: armazena a data de nascimento convertida para o tipo DATE com TO_DATE('15/08/1990', 'DD/MM/YYYY').
- ✓ **v_idade**: variável que armazenará o resultado do cálculo da idade.

Bloco de execução:

- ✓ **v_idade := TRUNC(MONTHS_BETWEEN(SYSDATE, v_data_nascimento) / 12);**
  - **MONTHS_BETWEEN(SYSDATE, v_data_nascimento)**: calcula o número total de meses entre a data atual e a data de nascimento.
  - A divisão por **12** converte meses em anos.
  - **TRUNC(...)** remove a parte decimal, resultando na idade completa.
- ✓ **dbms_output.put_line('Idade: ' || v_idade || ' anos');: concatena e exibe a idade no formato textual.**

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Para fins de benefícios, é necessário calcular a idade de um funcionário com base na data de nascimento.

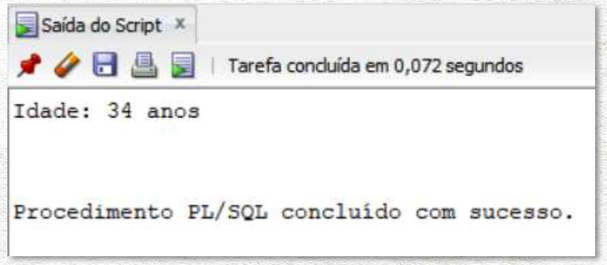
Crie um bloco anônimo que calcule e exiba a idade de um funcionário nascido em 15/08/1990.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_data_nascimento DATE := TO_DATE('15/08/1990', 'DD/MM/YYYY');
    v_idade NUMBER;
BEGIN
    v_idade := TRUNC(MONTHS_BETWEEN(SYSDATE, v_data_nascimento) / 12);
    dbms_output.put_line('Idade: ' || v_idade || ' anos');
END;
```

Exibimos o resultado ao lado, muito útil para cálculos automatizados de idade no contexto de recursos humanos, como concessão de benefícios ou enquadramento em faixas etárias:



Observação: A idade exibida pode ser diferente quando você executar, pois será calculada considerando-se a data atual no momento da execução.

GABARITO EXPLICATIVO

6

Questão 6

Ao gerar crachás, é necessário exibir o nome completo do funcionário em letras maiúsculas.

Crie um bloco anônimo que declare variáveis para o primeiro e último nome, concatene-os e exiba o nome completo em maiúsculas.

Explicação e Resultado

Para concatenar o primeiro e o último nome de um funcionário, converter o resultado para letras maiúsculas e exibi-lo para uso em crachás, utilizamos um bloco anônimo PL/SQL da seguinte forma:

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_primeiro_nome VARCHAR2(20) := 'marcus';
    v_ultimo_nome VARCHAR2(20) := 'cavalcanti';
    v_nome_completo VARCHAR2(40);
BEGIN
    v_nome_completo := UPPER(v_primeiro_nome || ' ' || v_ultimo_nome);
    dbms_output.put_line('Nome completo: ' || v_nome_completo);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

Ao gerar crachás, é necessário exibir o nome completo do funcionário em letras maiúsculas.

Crie um bloco anônimo que declare variáveis para o primeiro e último nome, concatene-os e exiba o nome completo em maiúsculas.

Explicação e Resultado

Declaração de variáveis:

- ✓ **v_primeiro_nome**: armazena o primeiro nome como string ('marcus').
- ✓ **v_ultimo_nome**: armazena o último nome como string ('cavalcanti').
- ✓ **v_nome_completo**: variável onde será armazenado o nome completo resultante da concatenação e conversão para maiúsculas.

Bloco de execução:

- ✓ **v_nome_completo := UPPER(v_primeiro_nome || ' ' || v_ultimo_nome);**:
  - Concatena o primeiro e último nome com um espaço entre eles.
  - Aplica a função **UPPER** para transformar todo o texto resultante em letras maiúsculas.
- ✓ **dbms_output.put_line('Nome completo: ' || v_nome_completo);**: exibe a string final formatada com o nome completo em caixa alta.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

6

Questão 6

Ao gerar crachás, é necessário exibir o nome completo do funcionário em letras maiúsculas.

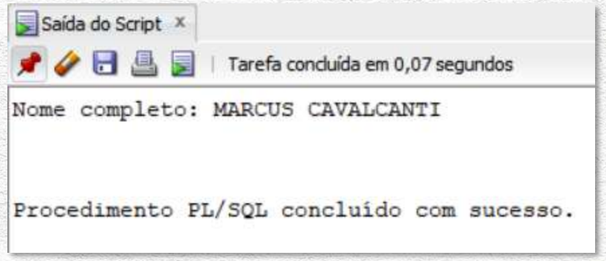
Crie um bloco anônimo que declare variáveis para o primeiro e último nome, concatene-os e exiba o nome completo em maiúsculas.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_primeiro_nome VARCHAR2(20) := 'marcus';
    v_ultimo_nome VARCHAR2(20) := 'cavalcanti';
    v_nome_completo VARCHAR2(40);
BEGIN
    v_nome_completo := UPPER(v_primeiro_nome || ' ' || v_ultimo_nome);
    dbms_output.put_line('Nome completo: ' || v_nome_completo);
END;
```

Exibimos o resultado ao lado, ideal para padronizar a exibição de nomes em crachás ou sistemas onde a leitura em letras maiúsculas é desejada. Ele garante clareza e uniformidade na apresentação dos nomes dos colaboradores:



GABARITO EXPLICATIVO

7

Questão 7

Durante cálculos financeiros, pode ocorrer divisão por zero.

Crie um bloco anônimo que tente dividir um número por outro e trate a exceção, exibindo uma mensagem de erro, caso ocorra divisão por zero.

Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção ZERO_DIVIDE para tratar este tipo de erro.

Explicação e Resultado

Para realizar uma divisão entre dois números e tratar adequadamente a exceção caso ocorra uma tentativa de divisão por zero, exibindo uma mensagem de erro amigável, utilizamos um bloco anônimo PL/SQL da seguinte forma:

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_num1 NUMBER := 100;
    v_num2 NUMBER := 0;
    v_resultado NUMBER;
BEGIN
    v_resultado := v_num1 / v_num2;
    dbms_output.put_line('Resultado: ' || v_resultado);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        dbms_output.put_line('Erro: Divisão por zero não é permitida.');
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Durante cálculos financeiros, pode ocorrer divisão por zero.

Crie um bloco anônimo que tente dividir um número por outro e trate a exceção, exibindo uma mensagem de erro, caso ocorra divisão por zero.

Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção ZERO_DIVIDE para tratar este tipo de erro.

Explicação e Resultado

Declaração de variáveis:

- ✓ **v_num1**: recebe o valor 100, numerador da divisão.
- ✓ **v_num2**: recebe 0, que causará a exceção de divisão por zero.
- ✓ **v_resultado**: variável que armazenaria o resultado da operação, se fosse válida.

Bloco BEGIN:

- ✓ **v_resultado := v_num1 / v_num2;**: aqui ocorre a tentativa de divisão. Como **v_num2** é igual a **0**, a exceção **ZERO_DIVIDE** será disparada automaticamente pelo Oracle.

Bloco EXCEPTION:

- ✓ **WHEN ZERO_DIVIDE THEN**: a exceção ZERO_DIVIDE captura o erro específico de divisão por zero.
- ✓ **dbms_output.put_line('Erro: Divisão por zero não é permitida.');**: em vez de gerar falha no programa, o erro é tratado com uma mensagem amigável ao usuário.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Durante cálculos financeiros, pode ocorrer divisão por zero.

Crie um bloco anônimo que tente dividir um número por outro e trate a exceção, exibindo uma mensagem de erro, caso ocorra divisão por zero.

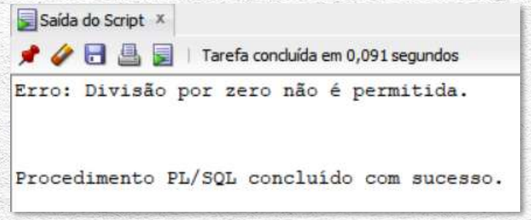
Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção ZERO_DIVIDE para tratar este tipo de erro.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_num1 NUMBER := 100;
    v_num2 NUMBER := 0;
    v_resultado NUMBER;
BEGIN
    v_resultado := v_num1 / v_num2;
    dbms_output.put_line('Resultado: ' || v_resultado);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        dbms_output.put_line('Erro: Divisão por zero não é permitida.');
END;
```

Exibimos o resultado ao lado, útil para garantir robustez em cálculos financeiros e evitar que o sistema pare por erros previsíveis como a divisão por zero. O uso da exceção ZERO_DIVIDE torna o tratamento mais claro e específico, melhorando a manutenção e legibilidade do código:



CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

7

Questão 7

Durante cálculos financeiros, pode ocorrer divisão por zero.

Crie um bloco anônimo que tente dividir um número por outro e trate a exceção, exibindo uma mensagem de erro, caso ocorra divisão por zero.

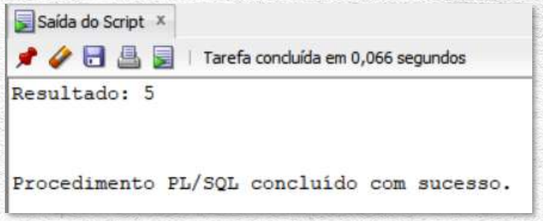
Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção ZERO_DIVIDE para tratar este tipo de erro.

Explicação e Resultado

Agora, se não ocorrer uma divisão por zero...

```
SET SERVEROUTPUT ON;

DECLARE
    v_num1 NUMBER := 100;
    v_num2 NUMBER := 20;
    v_resultado NUMBER;
BEGIN
    v_resultado := v_num1 / v_num2;
    dbms_output.put_line('Resultado: ' || v_resultado);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        dbms_output.put_line('Erro: Divisão por zero não é permitida.');
END;
```

A divisão é efetuada normalmente e seu resultado é armazenado na variável **v_resultado**, sendo exibido pelo **dbms_output.put_line('Resultado: ' || v_resultado);** do bloco **BEGIN**, encerrando a execução do bloco anônimo (neste caso, o bloco EXCEPTION não chega a ser executado, pois não houve divisão por zero):



GABARITO EXPLICATIVO

8

Questão 8

A empresa precisa converter temperaturas de Celsius (°C) para Fahrenheit (°F) em relatórios técnicos.

Crie um bloco anônimo que converta 30 graus Celsius para Fahrenheit e exiba o resultado.

Dica: a fórmula para conversão de °C para °F é: F = (C * 9/5) + 32.

Explicação e Resultado

Para converter uma temperatura de 30°C para Fahrenheit utilizando a fórmula apropriada e exibir o resultado, utilizamos um bloco anônimo PL/SQL da seguinte forma:

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_celsius NUMBER := 30;
    v_fahrenheit NUMBER;
BEGIN
    v_fahrenheit := (v_celsius * 9/5) + 32;
    dbms_output.put_line('Temperatura em Fahrenheit: ' || v_fahrenheit);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A empresa precisa converter temperaturas de Celsius (°C) para Fahrenheit (°F) em relatórios técnicos.

Crie um bloco anônimo que converta 30 graus Celsius para Fahrenheit e exiba o resultado.

Dica: a fórmula para conversão de °C para °F é: F = (C * 9/5) + 32.

Explicação e Resultado

Declaração de variáveis:

- ✓ **v_celsius**: recebe o valor 30, representando a temperatura original em graus Celsius.
- ✓ **v_fahrenheit**: variável que armazenará o resultado da conversão para Fahrenheit.

Bloco de execução:

- ✓ **v_fahrenheit := (v_celsius * 9/5) + 32;**: aplica a fórmula padrão de conversão de Celsius para Fahrenheit. Neste caso: (30 * 9 / 5) + 32 = 86.
- ✓ **dbms_output.put_line('Temperatura em Fahrenheit: ' || v_fahrenheit);**: exibe a temperatura convertida com a mensagem apropriada.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

A empresa precisa converter temperaturas de Celsius (°C) para Fahrenheit (°F) em relatórios técnicos.

Crie um bloco anônimo que converta 30 graus Celsius para Fahrenheit e exiba o resultado.

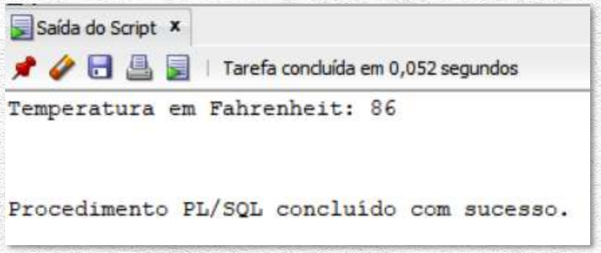
Dica: a fórmula para conversão de °C para °F é: F = (C * 9/5) + 32.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_celsius NUMBER := 30;
    v_fahrenheit NUMBER;
BEGIN
    v_fahrenheit := (v_celsius * 9/5) + 32;
    dbms_output.put_line('Temperatura em Fahrenheit: ' || v_fahrenheit);
END;
```

Exibimos o resultado ao lado, ideal para aplicações técnicas e científicas em ambientes corporativos:



GABARITO EXPLICATIVO

9

Questão 9

Um sistema tenta converter texto para número e precisa evitar falhas se o valor não for numérico.

Crie um bloco anônimo que tente converter uma string para número e, em caso de falha, trate a exceção exibindo uma mensagem de erro.

Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção VALUE_ERROR para tratar este tipo de erro.

Explicação e Resultado

Para converter um texto para número e tratar a exceção caso o conteúdo não seja numérico, evitando falhas no sistema, utilizamos um bloco anônimo PL/SQL da seguinte forma:

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_texto VARCHAR2(10) := 'ABC123';
    v_numero NUMBER;
BEGIN
    v_numero := TO_NUMBER(v_texto);
    dbms_output.put_line('Valor convertido: ' || v_numero);
EXCEPTION
    WHEN VALUE_ERROR THEN
        dbms_output.put_line('Erro: valor inválido para conversão numérica.');
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

Um sistema tenta converter texto para número e precisa evitar falhas se o valor não for numérico.

Crie um bloco anônimo que tente converter uma string para número e, em caso de falha, trate a exceção exibindo uma mensagem de erro.

Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção VALUE_ERROR para tratar este tipo de erro.

Explicação e Resultado

Declaração de variáveis:

- ✓ **v_texto**: armazena o valor 'ABC123', uma string que contém letras e números.
- ✓ **v_numero**: variável do tipo NUMBER que receberia o resultado da conversão se não ocorresse nenhuma falha.

Bloco BEGIN:

- ✓ **v_numero := TO_NUMBER(v_texto);**: tenta converter a string da variável **v_texto** para número. Mas como **'ABC123'** contém letras, a função **TO_NUMBER** falha e dispara uma exceção do tipo **VALUE_ERROR**.

Tratamento de exceção:

- ✓ **EXCEPTION WHEN VALUE_ERROR THEN**: captura especificamente a exceção **VALUE_ERROR**, que ocorre quando há erro de tipo em tempo de execução (como conversão inválida).
- ✓ **dbms_output.put_line('Erro: valor inválido para conversão numérica.');**: exibe uma mensagem de erro personalizada informando o motivo da falha.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

9

Questão 9

Um sistema tenta converter texto para número e precisa evitar falhas se o valor não for numérico.

Crie um bloco anônimo que tente converter uma string para número e, em caso de falha, trate a exceção exibindo uma mensagem de erro.

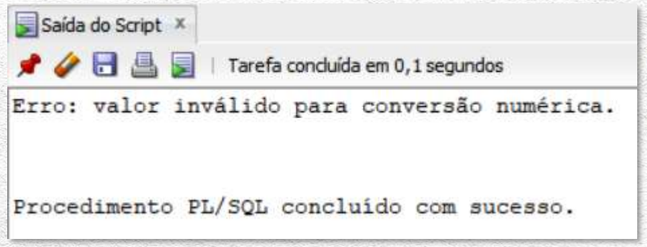
Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção VALUE_ERROR para tratar este tipo de erro.

Explicação e Resultado

Portanto, com este código...

```
DECLARE
    v_texto VARCHAR2(10) := 'ABC123';
    v_numero NUMBER;
BEGIN
    v_numero := TO_NUMBER(v_texto);
    dbms_output.put_line('Valor convertido: ' || v_numero);
EXCEPTION
    WHEN VALUE_ERROR THEN
        dbms_output.put_line('Erro: valor inválido para conversão numérica.');
END;
```

Exibimos o resultado ao lado, que simula uma conversão de string para número e trata corretamente a exceção quando a entrada não é válida, o que é fundamental para garantir estabilidade e controle de erros em sistemas que processam dados de entrada externos:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

Um sistema tenta converter texto para número e precisa evitar falhas se o valor não for numérico.

Crie um bloco anônimo que tente converter uma string para número e, em caso de falha, trate a exceção exibindo uma mensagem de erro.

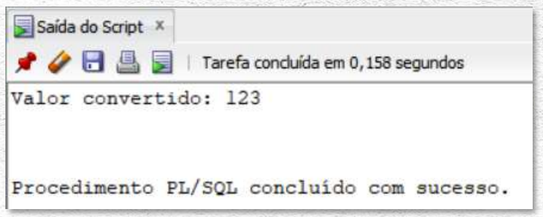
Dica: no bloco de exceção, você pode usar a exceção OTHERS. Porém, se quiser ser mais específico, você pode usar a exceção VALUE_ERROR para tratar este tipo de erro.

Explicação e Resultado

Agora, se a string contém apenas números...

```
SET SERVEROUTPUT ON;

DECLARE
    v_texto VARCHAR2(10) := '123';
    v_numero NUMBER;
BEGIN
    v_numero := TO_NUMBER(v_texto);
    dbms_output.put_line('Valor convertido: ' || v_numero);
EXCEPTION
    WHEN VALUE_ERROR THEN
        dbms_output.put_line('Erro: valor inválido para conversão numérica.');
END;
```

A conversão é efetuada sem falhas e o valor numérico resultante é armazenado na variável **v_numero**, sendo exibido pelo **dbms_output.put_line('Valor convertido: ' || v_numero);** do bloco **BEGIN**, encerrando a execução do bloco anônimo (neste caso, o bloco EXCEPTION não chega a ser executado, pois não houve falha de tipo durante a conversão):



GABARITO EXPLICATIVO

10

Questão 10

Uma aplicação educacional precisa calcular a média de um aluno.

Crie um bloco anônimo que calcule a média entre três notas e a exiba na tela com duas casas decimais.

Explicação e Resultado

Para calcular a média entre três notas e exibir o resultado com duas casas decimais, utilizamos um bloco anônimo PL/SQL da seguinte forma:

Comando de ativação da saída:

```
SET SERVEROUTPUT ON;
```

- ✓ Esse comando é necessário para que o Oracle exiba as mensagens enviadas pelo dbms_output.put_line. Sem ele, a saída do bloco não será visível no console.

Bloco anônimo:

```
DECLARE
    v_notal NUMBER := 7.5;
    v_nota2 NUMBER := 6.9;
    v_nota3 NUMBER := 10;
    v_media NUMBER;
BEGIN
    v_media := (v_notal + v_nota2 + v_nota3) / 3;

    dbms_output.put_line('Média: ' || ROUND(v_media, 2));
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

10

Questão 10

Uma aplicação educacional precisa calcular a média de um aluno.

Crie um bloco anônimo que calcule a média entre três notas e a exiba na tela com duas casas decimais.

Explicação e Resultado

Declaração de variáveis:

- ✓ **v_nota1, v_nota2, v_nota3**: armazenam as três notas do aluno.
- ✓ **v_media**: variável que guardará o resultado da média.

Bloco de execução:

- ✓ **v_media := (v_nota1 + v_nota2 + v_nota3) / 3;**: soma as três notas e divide o total por 3.
- ✓ **dbms_output.put_line('Média: ' || ROUND(v_media, 2));**: utiliza a função ROUND para arredondar a média com duas casas decimais, e então é exibida formatada na tela.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

10

Questão 10

Uma aplicação educacional precisa calcular a média de um aluno.

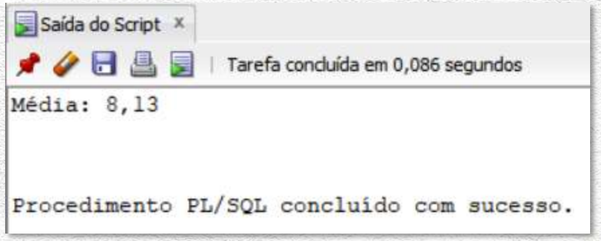
Crie um bloco anônimo que calcule a média entre três notas e a exiba na tela com duas casas decimais.

Explicação e Resultado

Portanto, com este código...

```
SET SERVEROUTPUT ON;

DECLARE
    v_notal NUMBER := 7.5;
    v_nota2 NUMBER := 6.9;
    v_nota3 NUMBER := 10;
    v_media NUMBER;
BEGIN
    v_media := (v_notal + v_nota2 + v_nota3) / 3;

    dbms_output.put_line('Média: ' || ROUND(v_media, 2));
END;
```

Exibimos o resultado ao lado, que realiza de forma simples e eficaz o cálculo da média de três notas e apresenta o valor final com precisão de duas casas decimais, o que é útil em contextos educacionais para manter clareza e padronização na exibição de resultados:



MÓDULO 15

ESTRUTURAS DE CONTROLE

ESTRUTURAS DE CONTROLE

ESTRUTURAS DE CONTROLE

O QUE VEREMOS NESTE MÓDULO

MÓDULO 15 ESTRUTURAS DE CONTROLE

01	ESTRUTURAS DE CONTROLE	711
02	ESTRUTURAS CONDICIONAIS: IF	712
03	ESTRUTURAS CONDICIONAIS: CASE	716
04	ESTRUTURAS DE REPETIÇÃO: LOOP	720
05	ESTRUTURAS DE REPETIÇÃO: FOR LOOP	723
06	ESTRUTURAS DE REPETIÇÃO: WHILE LOOP	726
07	RESUMO DO MÓDULO	729
08	EXERCÍCIOS	730
09	GABARITO EXPLICATIVO	735

ESTRUTURAS DE CONTROLE

As ESTRUTURAS DE CONTROLE podem ser divididas em:

Estruturas condicionais: IF, CASE.

Estruturas de repetição: LOOP, FOR LOOP, WHILE LOOP.

Estruturas Condicionais: IF

A estrutura de controle condicional IF serve para avaliar uma condição e, dependendo do resultado dessa condição, se ela for verdadeira ou falsa, executamos determinado(s) comando(s) ou outro(s).

SINTAXE:

```
IF condição THEN                    -- Se esta condição for verdadeira:
        comandos                    -- Execute estes comandos;
ELSE                                -- Caso contrário:
        comandos                    -- Execute estes comandos;
END IF;                             -- Finalize a condição.
```

Estruturas Condicionais: IF

Vamos ver um exemplo:

Existe um indicador chamado NPS que mede a satisfação de um cliente em relação a um produto/serviço.

Este indicador se trata de uma nota de 0 a 10, em que o cliente pode se encaixar em uma das 3 categorias abaixo, de acordo com a sua nota:

Promotor: Nota entre 9-10

Neutro: Nota entre 7-8

Detrator: Nota entre 0-6

Vamos agora criar um programa que receba um determinado NPS de um cliente e o classifique de acordo com a sua nota.

Estruturas Condicionais: IF

1. Inicializamos o serviço de impressão;
2. Criamos uma caixinha para o usuário digitar o NPS;
3. Em um bloco anônimo, declaramos uma variável (vNPS) para receber a resposta do usuário (&resposta);
4. Iniciamos uma estrutura de controle condicional IF para comparar se o valor informado pelo usuário é maior ou igual a 9:
   Se for, imprimimos na tela a mensagem: “Este cliente é um PROMOTOR. Nota: [vNPS]”;
   Caso contrário, criamos um novo IF (mais interno) para verificar se a nota é maior ou igual a 7:
   Se for, imprimimos na tela a mensagem “Este cliente é NEUTRO. Nota: [vNPS]”;
   Caso contrário, imprimimos na tela a mensagem “Este cliente é um DETRATOR. Nota: [vNPS]”;
5. Finalizamos o IF interno;
6. Finalizamos o IF externo;
7. Finalizamos o bloco anônimo.

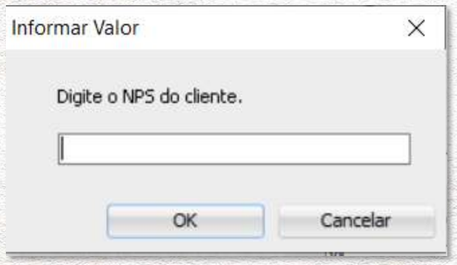
```
SET SERVEROUTPUT ON; 1

ACCEPT resposta PROMPT 'Digite o NPS do cliente.'; 2

DECLARE 3
    vNPS NUMBER(10, 2) := &resposta;
BEGIN

    IF vNPS >= 9 THEN 4
        dbms_output.put_line('Este cliente é um PROMOTOR. Nota: ' || vNPS);
    ELSE
        IF vNPS >= 7 THEN
            dbms_output.put_line('Este cliente é NEUTRO. Nota: ' || vNPS);
        ELSE
            dbms_output.put_line('Este cliente é um DETRATOR. Nota: ' || vNPS);
        END IF; 5
    END IF; 6

END; 7
```

Índice Geral

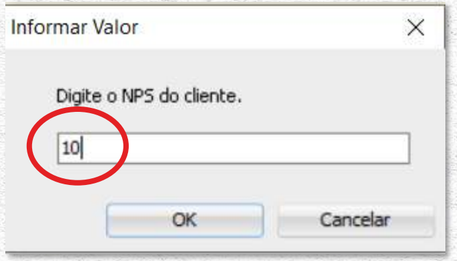
Índice deste Módulo

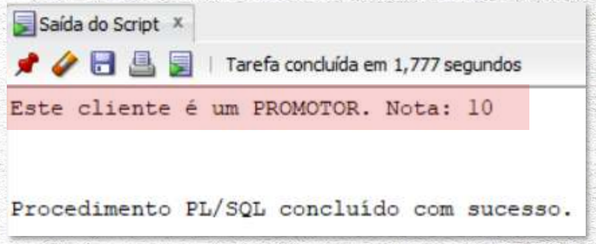
Estruturas Condicionais: IF

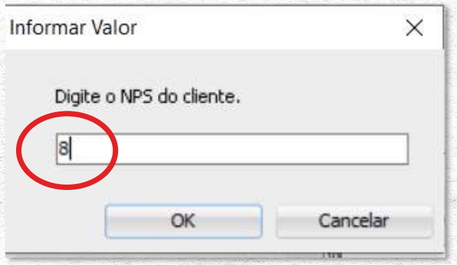
Ao executarmos o código, aparecerá na tela uma caixinha para digitarmos o NPS:

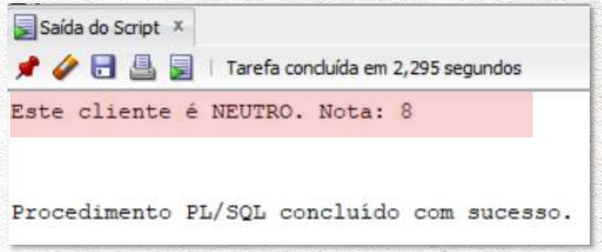


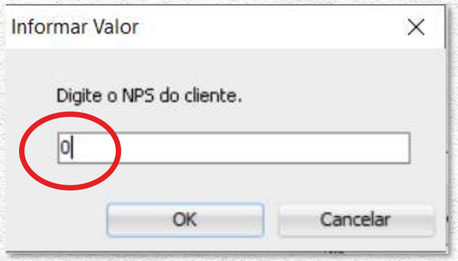
A depender do NPS informado, o sistema retornará os seguintes resultados:

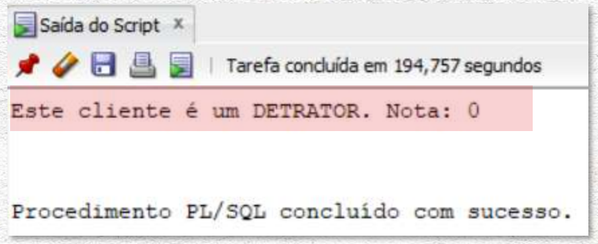












Estruturas Condicionais: CASE

A estrutura de controle condicional CASE é uma alternativa ao IF, e também serve para avaliar uma condição. Dependendo do resultado dessa condição, se ela for verdadeira ou falsa, executamos determinado(s) comando(s) ou outro(s).

SINTAXE:

```
CASE                        -- Caso:
WHEN condicao               -- Quando esta condição for verdadeira:
   comandos                 -- Execute estes comandos;
WHEN condicao               -- Já quando a condição anterior for falsa e esta condição for verdadeira:
   comandos                 -- Execute estes comandos;
ELSE                        -- Caso contrário:
   comandos                 -- Execute estes comandos;
END CASE;                   -- Finalize a condição.
```

Estruturas Condicionais: CASE

Veremos o mesmo exemplo anterior, agora aplicando a estrutura de controle condicional CASE.

Vamos relembrar o enunciado:

Existe um indicador chamado NPS que mede a satisfação de um cliente em relação a um produto/serviço.

Este indicador se trata de uma nota de 0 a 10, em que o cliente pode se encaixar em uma das 3 categorias abaixo, de acordo com a sua nota:

Promotor: Nota entre 9-10

Neutro: Nota entre 7-8

Detrator: Nota entre 0-6

Vamos agora criar um programa que receba um determinado NPS de um cliente e o classifique de acordo com a sua nota.

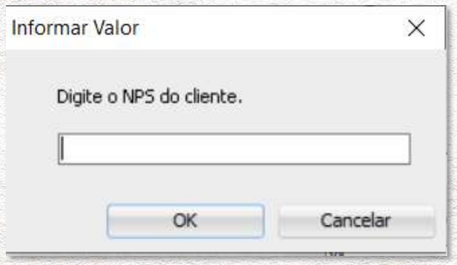
Estruturas Condicionais: CASE

1. Inicializamos o serviço de impressão;
2. Criamos uma caixinha para o usuário digitar o NPS;
3. Em um bloco anônimo, declaramos uma variável (vNPS) para receber a resposta do usuário (&resposta);
4. Iniciamos uma estrutura de controle condicional CASE para comparar se o valor informado pelo usuário é maior ou igual a 9:
   Quando for, imprimimos na tela a mensagem: “Este cliente é PROMOTOR. Nota: [vNPS]”;
   Caso contrário, comparamos novamente para saber se a nota atribuída é maior ou igual a 7:
   Quando for, imprimimos na tela a mensagem “Este cliente é NEUTRO. Nota: [vNPS]”;
   Caso contrário, imprimimos na tela a mensagem “Este cliente é DETRATOR. Nota: [vNPS]”;
5. Finalizamos o CASE;
6. Finalizamos o bloco anônimo.

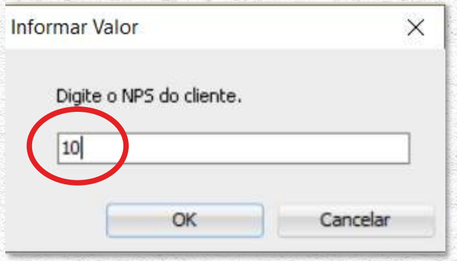
```
SET SERVEROUTPUT ON; 1

ACCEPT resposta PROMPT 'Digite o NPS do cliente.'; 2

DECLARE 3
    vNPS NUMBER(10, 2) := &resposta;
BEGIN

    CASE 4
        WHEN vNPS >= 9 THEN
            dbms_output.put_line('Este cliente é PROMOTOR. Nota: ' || vNPS);
        WHEN vNPS >= 7 THEN
            dbms_output.put_line('Este cliente é NEUTRO. Nota: ' || vNPS);
        ELSE
            dbms_output.put_line('Este cliente é DETRATOR. Nota: ' || vNPS);
    END CASE; 5

END; 6
```

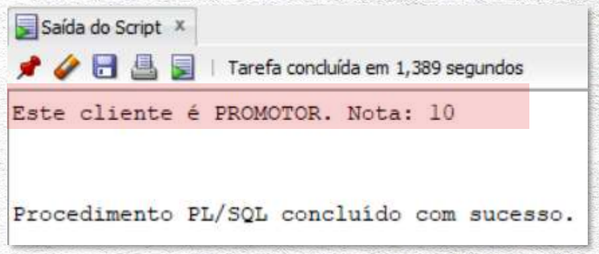
Estruturas Condicionais: CASE

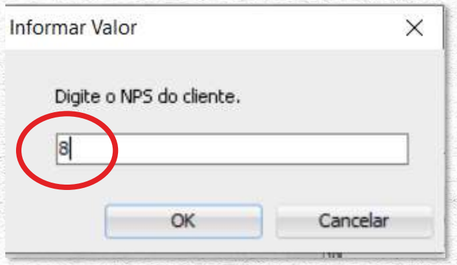
Ao executarmos o código, aparecerá na tela uma caixinha para digitarmos o NPS:

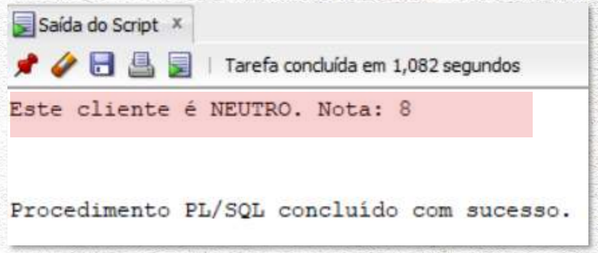


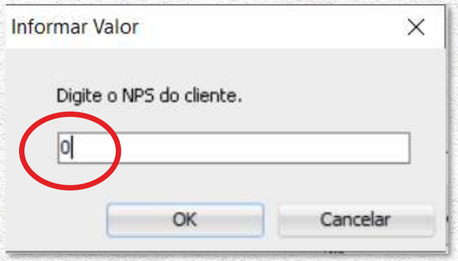
A depender do NPS informado, o sistema retornará os seguintes resultados:

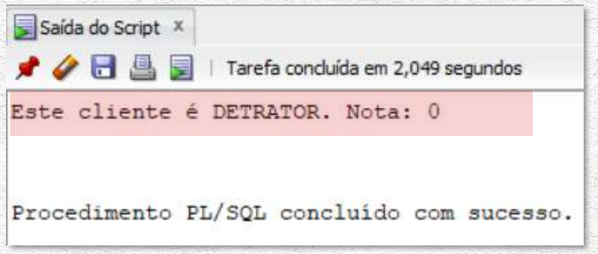












Estruturas de Repetição: LOOP

A estrutura de controle LOOP nos permite criar uma estrutura de repetição.

SINTAXE:

```
LOOP                                    -- Repita:
   comandos                             -- Execute este(s) comando(s);
EXIT WHEN condicao                      -- Pare quando a condição for verdadeira;
   incrementa contador                  -- Incremente o contador;
END LOOP;                               -- Finalize a repetição.
```

Estruturas de Repetição: LOOP

Vejamos o exemplo abaixo:

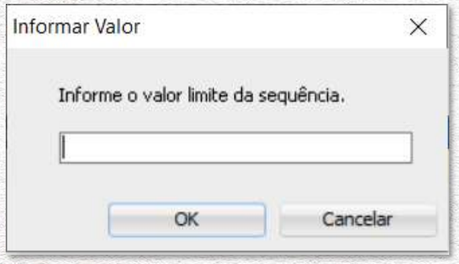
Queremos fazer um contador que imprima na tela valores desde um valor inicial até um valor limite, que deverá ser passado através de uma variável (usando o comando ACCEPT):

1. Inicializamos o serviço de impressão;
2. Criamos uma caixinha para o usuário digitar o valor limite da sequência;
3. Em um bloco anônimo, declaramos uma variável que inicializará o contador com valor 1 (vContador), e outra para receber a resposta do usuário quanto ao valor limite (vFinal);
4. Iniciamos uma estrutura de repetição com o comando LOOP;
   Imprimimos na tela a mensagem: "O contador é: [vContador]";
   Determinamos a interrupção do LOOP quando a variável vContador for igual à vFinal (EXIT WHEN);
   Incrementamos o contador de 1 em 1;
5. Finalizamos o LOOP;
6. Finalizamos o bloco anônimo.

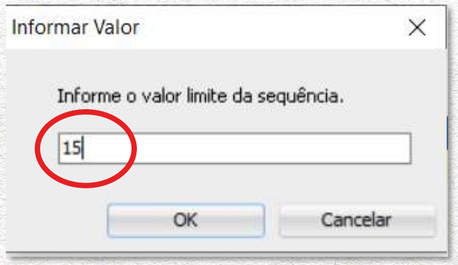
```
SET SERVEROUTPUT ON; 1

ACCEPT resposta PROMPT 'Informe o valor limite da sequência.'; 2

DECLARE 3
    vContador INT := 1;
    vFinal INT := &resposta;
BEGIN

    LOOP 4
        dbms_output.put_line('O contador é: ' || vContador);
        EXIT WHEN vContador = vFinal;
            vContador := vContador + 1;
    END LOOP; 5
END; 6
```

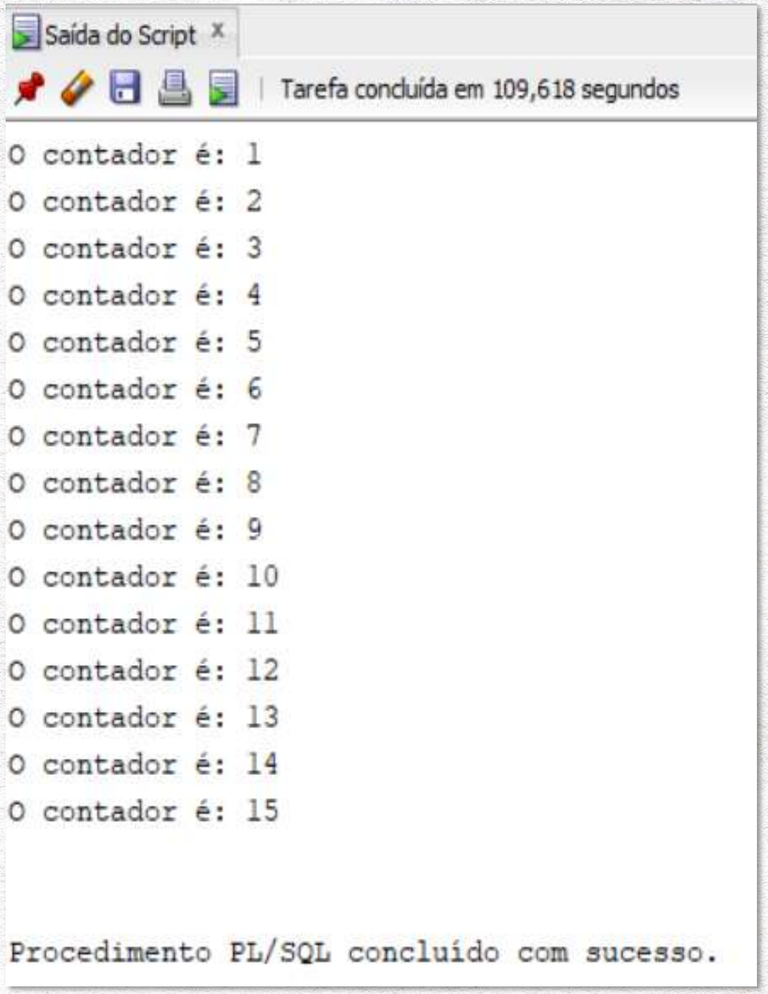
Estruturas de Repetição: LOOP

Ao executarmos o código, aparecerá na tela uma caixinha para digitarmos o valor limite da sequência:



A depender da valor informado, o contador irá mostrar na tela toda a sequência até chegar a esse valor limite:





Estruturas de Repetição: FOR LOOP

A estrutura de controle FOR LOOP também nos permite criar uma estrutura de repetição.

Geralmente é a mais utilizada entre as estruturas de repetição.

Repare em sua sintaxe que, diferentemente do LOOP, com o FOR LOOP não precisamos incrementar o contador:

SINTAXE:

```
FOR i IN condição LOOP                -- Para um valor que esteja dentro da condição, repita:
   comandos                           -- Execute este(s) comando(s);
END LOOP;                             -- Finalize a repetição.
```

Estruturas de Repetição: FOR LOOP

Vejamos o mesmo exemplo anterior, só que agora aplicando o FOR LOOP:

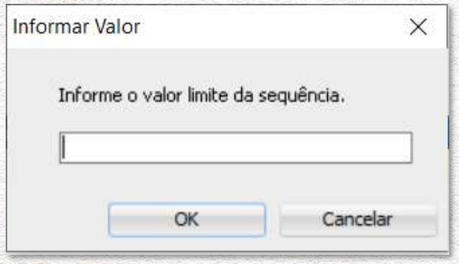
Queremos fazer um contador que imprima na tela valores desde um valor inicial até um valor limite, que deverá ser passado através de uma variável (usando o comando ACCEPT):

1. Inicializamos o serviço de impressão;
2. Criamos uma caixinha para o usuário digitar o valor limite da sequência;
3. Em um bloco anônimo, declaramos uma variável que inicializará o contador com valor 1 (vInicial), e outra para receber a resposta do usuário quanto ao valor limite (vFinal);
4. Iniciamos uma estrutura de repetição com o comando FOR LOOP;
   Imprimimos na tela a mensagem: “O valor é: [i]”;
5. Finalizamos o LOOP;
6. Finalizamos o bloco anônimo.

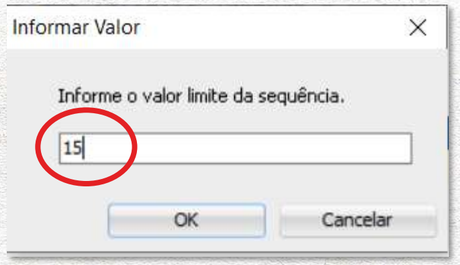
```
SET SERVEROUTPUT ON; 1
ACCEPT resposta PROMPT 'Informe o valor limite da sequência.' 2

DECLARE 3
    vInicial INT := 1;
    vFinal INT := &resposta;
BEGIN

    FOR i IN vInicial..vFinal LOOP 4
        dbms_output.put_line('O valor é: ' || i);
    END LOOP; 5

END; 6
```

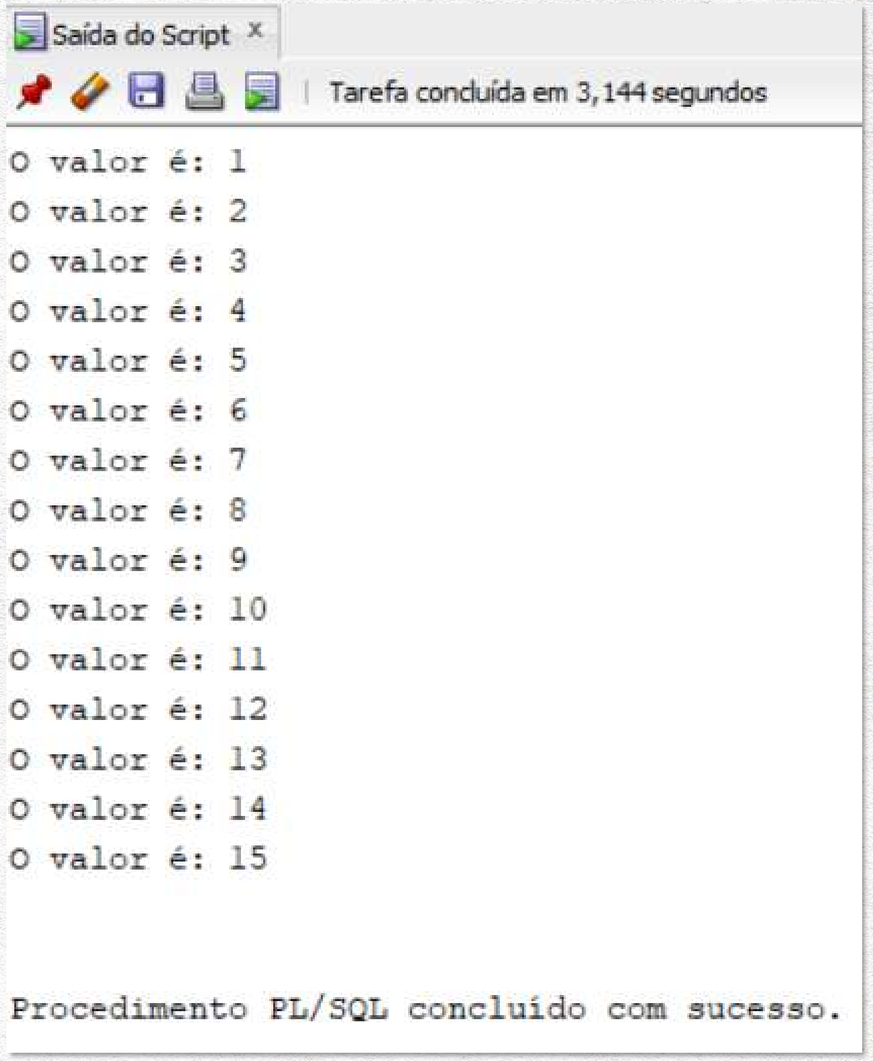
Estruturas de Repetição: FOR LOOP

Ao executarmos o código, aparecerá na tela uma caixinha para digitarmos o valor limite da sequência:



A depender da valor informado, o sistema irá mostrar na tela toda a sequência até chegar a esse valor limite:





Estruturas de Repetição: WHILE LOOP

A estrutura de controle WHILE LOOP também nos permite criar uma estrutura de repetição.

Repare em sua sintaxe que, diferentemente do FOR LOOP, precisamos incrementar o contador (assim como com o LOOP):

SINTAXE:

```
WHILE condição LOOP                -- Enquanto a condição for verdadeira, repita:
   comandos                        -- Execute este(s) comando(s);
   incrementa contador             -- Incremente o contador;
END LOOP;                          -- Finalize a repetição.
```

Estruturas de Repetição: WHILE LOOP

Vejamos o mesmo exemplo anterior, só que agora aplicando o WHILE LOOP:

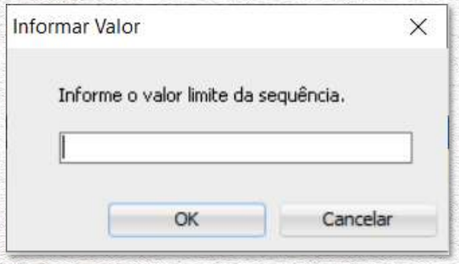
Queremos fazer um contador que imprima na tela valores desde um valor inicial até um valor limite, que deverá ser passado através de uma variável (usando o comando ACCEPT):

1. Inicializamos o serviço de impressão;
2. Criamos uma caixinha para o usuário digitar o valor limite da sequência;
3. Em um bloco anônimo, declaramos uma variável que inicializará o contador com valor 1 (vContador), e outra para receber a resposta do usuário quanto ao valor limite (vFinal);
4. Iniciamos uma estrutura de repetição com o comando WHILE LOOP;
   Imprimimos na tela a mensagem: “O valor é: [vContador]”;
   Incrementamos o contador de 1 em 1;
5. Finalizamos o LOOP;
6. Finalizamos o bloco anônimo.

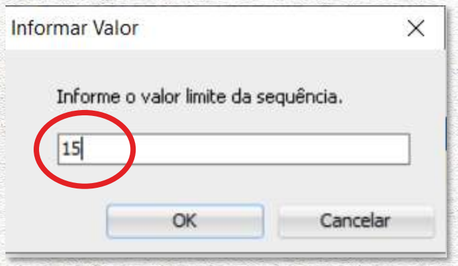
```
SET SERVEROUTPUT ON; 1
ACCEPT resposta PROMPT 'Informe o valor limite da sequência.' 2

DECLARE 3
    vContador INT := 1;
    vFinal INT := &resposta;
BEGIN

    WHILE vContador <= vFinal LOOP 4
        dbms_output.put_line('O valor é: ' || vContador);
        vContador := vContador + 1;
    END LOOP; 5

END; 6
```

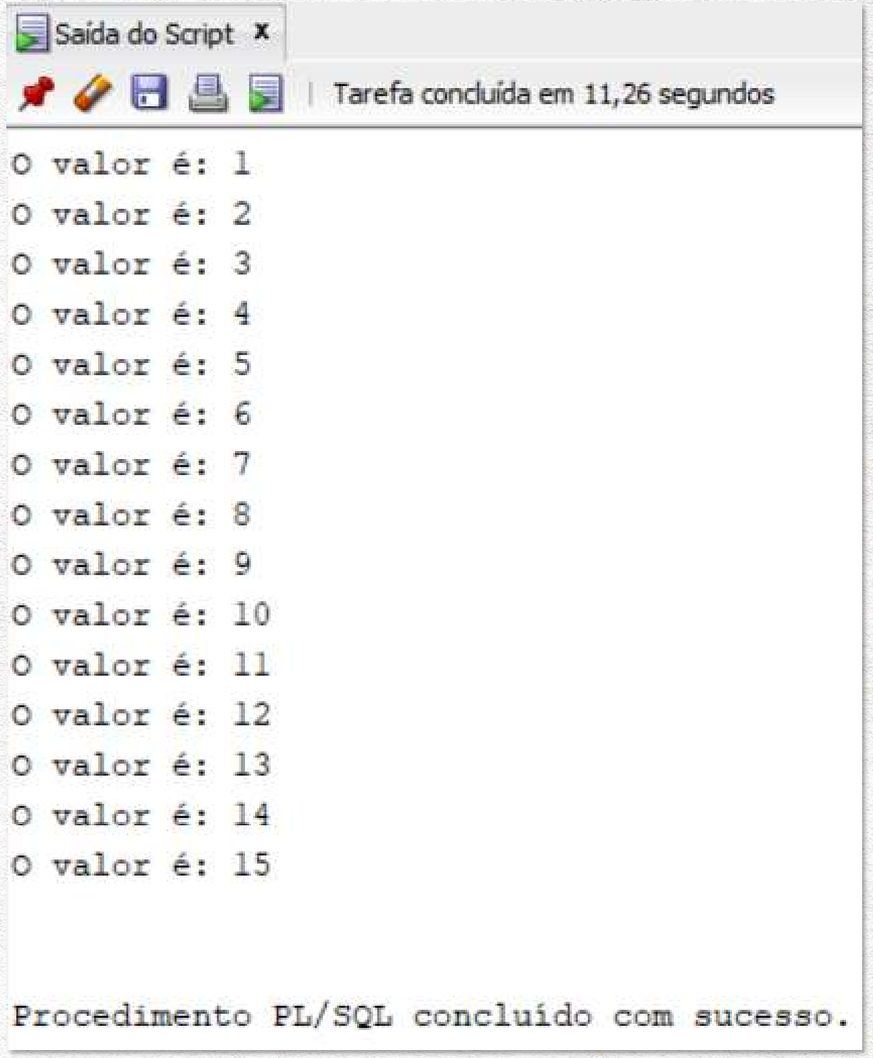
Estruturas de Repetição: WHILE LOOP

Ao executarmos o código, aparecerá na tela uma caixinha para digitarmos o valor limite da sequência:



A depender da valor informado, o sistema irá mostrar na tela toda a sequência até chegar a esse valor limite:





Índice Geral

Índice deste Módulo

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. As ESTRUTURAS DE CONTROLE podem ser divididas em estruturas condicionais e de repetição.

2. As estruturas condicionais são: IF e CASE.

3. As estruturas de repetição são: LOOP, FOR LOOP e WHILE LOOP.

4. Se você usar as estruturas LOOP ou WHILE, lembre-se de incrementar o seu contador.
Para a estrutura FOR LOOP o contador é automático.

EXERCÍCIOS

1

Questão 1

Crie um bloco anônimo que receba via ACCEPT a idade de uma pessoa e, usando IF, exiba "Maior de idade" se ela tiver 18 anos ou mais, ou "Menor de idade" se ela tiver menos de 18 anos.

2

Questão 2

Crie um bloco anônimo que receba via ACCEPT um número e, usando IF, se ele for par, exiba "Número par". Caso contrário, exiba "Número ímpar".

EXERCÍCIOS

3

Questão 3

Crie um bloco anônimo que receba via ACCEPT a nota de um aluno e, usando CASE, exiba:

- ✓ Se a nota for maior ou igual a 7: "Aprovado";
- ✓ Se estiver entre 5 (inclusive) e 7: "Recuperação";
- ✓ Caso contrário: "Reprovado".

4

Questão 4

Crie um bloco anônimo que receba via ACCEPT as siglas dos estados da Região Sudeste ("SP", "RJ", "MG", "ES") e exiba o nome completo do estado usando CASE. Se for informada a sigla de outro estado não pertencente à Região Sudeste, exiba "Estado não cadastrado".

Para evitar resultado incorreto, lembre-se de usar uma função que converta a sigla informada para letras maiúsculas antes de compará-la, caso o usuário tenha informando-a com letras minúsculas.

EXERCÍCIOS

5

Questão 5

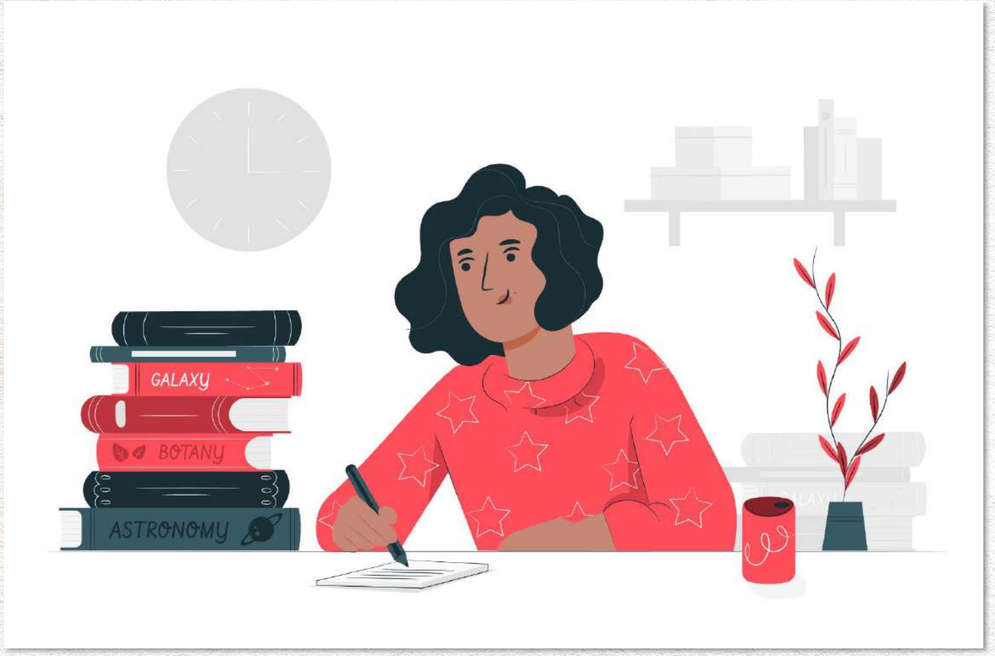
Crie um bloco anônimo que utilize um LOOP para somar números inteiros a partir de 1. O bloco deve parar quando a soma acumulada ultrapassar 100. Ao final, exiba a soma e o número que fez a soma ultrapassar esse limite.

6

Questão 6

Um produto custa R$ 500. A cada mês, ele recebe um desconto de R$ 30. Crie um bloco que utilize WHILE LOOP para simular a aplicação mensal do desconto até que o valor do produto seja menor que R$ 300. Exiba o valor final e quantos meses de desconto foram aplicados.

EXERCÍCIOS



7

Questão 7

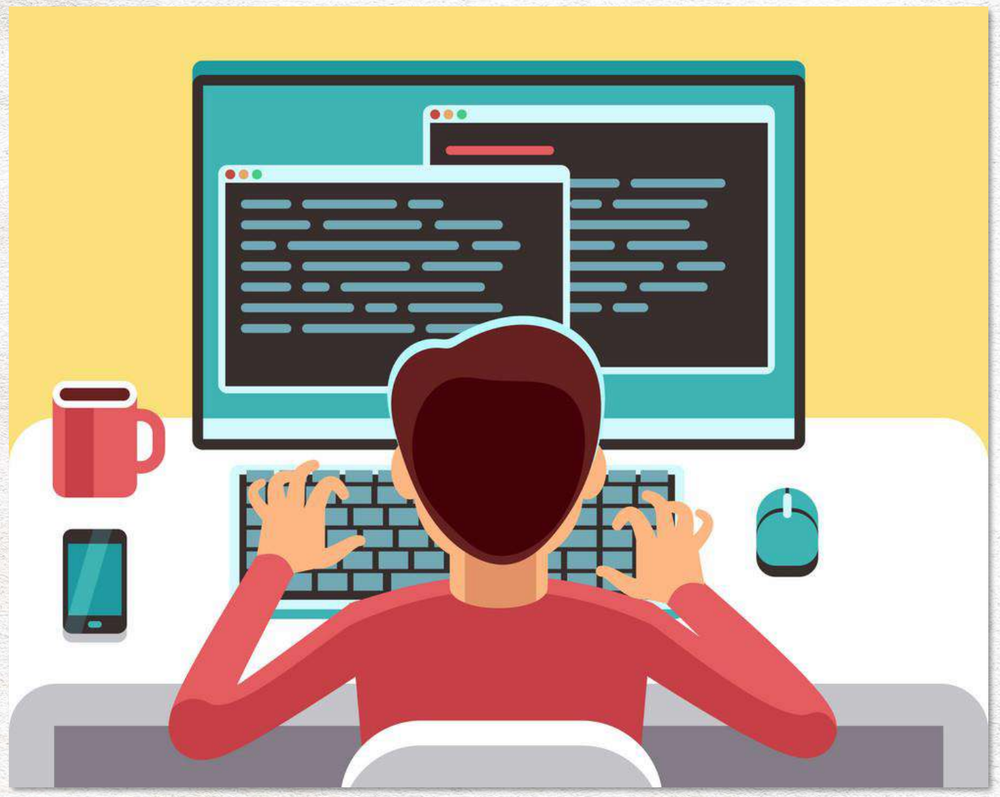
Crie um bloco que utilize FOR LOOP para simular o pagamento de um valor de R$ 1200 em 4 parcelas iguais. Para cada parcela, exiba o número da parcela e o valor pago.

8

Questão 8

Um usuário decide guardar R$ 150 por mês. Crie um bloco LOOP que simule essa economia mensal até que o valor acumulado atinja ou ultrapasse R$ 1000. Exiba o total acumulado e o número de meses necessários.

EXERCÍCIOS



9

Questão 9

Uma loja possui 30 unidades de um produto em estoque. A cada venda, 3 unidades são retiradas. Crie um bloco com WHILE LOOP que simule essas vendas até o estoque ficar abaixo de 5 unidades. Exiba o número de vendas realizadas e o estoque final.

10

Questão 10

Um salário inicial é de R$ 2000. Crie um bloco com FOR LOOP que simule um aumento de 5% ao ano, durante 3 anos. Exiba o salário ao final de cada ano.

GABARITO EXPLICATIVO

1

Questão 1

Crie um bloco anônimo que receba via ACCEPT a idade de uma pessoa e, usando IF, exiba "Maior de idade" se ela tiver 18 anos ou mais, ou "Menor de idade" se ela tiver menos de 18 anos.

Explicação e Resultado

Para receber a idade de uma pessoa via ACCEPT e, usando uma estrutura condicional IF, determinar se é maior ou menor de idade, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

ACCEPT idade PROMPT 'Informe a idade: '

DECLARE
    vIdade NUMBER := &idade;
BEGIN
    IF vIdade >= 18 THEN
        dbms_output.put_line('Maior de idade');
    ELSE
        dbms_output.put_line('Menor de idade');
    END IF;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

Crie um bloco anônimo que receba via ACCEPT a idade de uma pessoa e, usando IF, exiba "Maior de idade" se ela tiver 18 anos ou mais, ou "Menor de idade" se ela tiver menos de 18 anos.

Explicação e Resultado

Explicação passo a passo:

1. Ativação da saída de mensagens:

- ✓ **SET SERVEROUTPUT ON;**: Permite que mensagens escritas com **dbms_output.put_line** sejam exibidas na tela.

2. Entrada de dados pelo usuário:

- ✓ **ACCEPT idade PROMPT 'Informe a idade: '**: Solicita ao usuário que informe a idade. O valor digitado será atribuído ao substituível **&idade**.

3. Bloco anônimo:

- ✓ **DECLARE vIdade NUMBER := &idade;**: Cria a variável **vIdade** e atribui o valor digitado pelo usuário.
- ✓ **Estrutura condicional IF**: Verifica se a idade é maior ou igual a 18. Se for, exibe **“Maior de idade”**; caso contrário, exibe **“Menor de idade”**.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

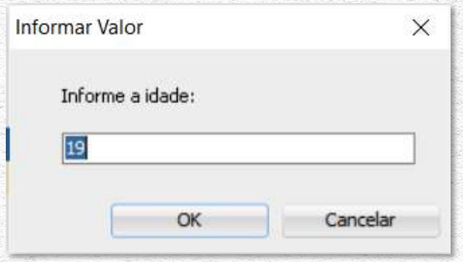
Questão 1

Crie um bloco anônimo que receba via ACCEPT a idade de uma pessoa e, usando IF, exiba "Maior de idade" se ela tiver 18 anos ou mais, ou "Menor de idade" se ela tiver menos de 18 anos.

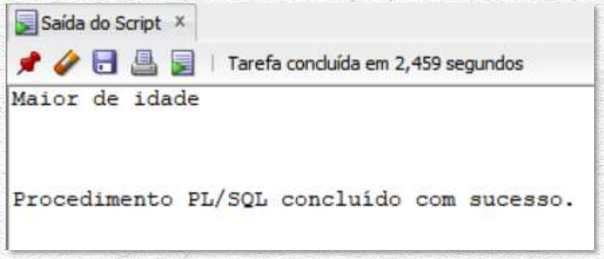
Explicação e Resultado

Assim, quando executamos o código, aparece uma janelinha na tela para digitarmos a idade.

Se informarmos um valor maior ou igual a 18...



Retornamos o resultado “Maior de Idade”:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

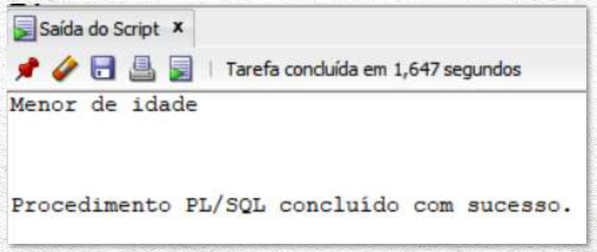
1 Questão 1

Crie um bloco anônimo que receba via ACCEPT a idade de uma pessoa e, usando IF, exiba "Maior de idade" se ela tiver 18 anos ou mais, ou "Menor de idade" se ela tiver menos de 18 anos.

Explicação e Resultado

Mas se informarmos um valor menor que 18...

Retornamos o resultado “Menor de Idade”:



Esse bloco anônimo é útil para decisões simples baseadas em condições, como validar se uma pessoa é maior de idade. Utiliza a estrutura IF para bifurcar a lógica com base em uma entrada fornecida pelo usuário, sendo uma construção fundamental em scripts interativos.

GABARITO EXPLICATIVO

2

Questão 2

Crie um bloco anônimo que receba via ACCEPT um número e, usando IF, se ele for par, exiba "Número par". Caso contrário, exiba "Número ímpar".

Explicação e Resultado

Para receber um número do usuário e, usando IF, exibir "Número par" se ele for divisível por 2, ou "Número ímpar" caso contrário, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

ACCEPT numero PROMPT 'Informe um número: '

DECLARE
    vNumero NUMBER := &numero;
BEGIN
    IF MOD(vNumero, 2) = 0 THEN
        dbms_output.put_line('Número par');
    ELSE
        dbms_output.put_line('Número ímpar');
    END IF;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

Crie um bloco anônimo que receba via ACCEPT um número e, usando IF, se ele for par, exiba "Número par". Caso contrário, exiba "Número ímpar".

Explicação e Resultado

Explicação passo a passo:

1. Ativação da saída de mensagens:

- ✓ **SET SERVEROUTPUT ON;**: Habilita a exibição de mensagens via **dbms_output.put_line**.

2. Entrada de dados:

- ✓ **ACCEPT numero PROMPT 'Informe um número: '**: Solicita que o usuário digite um número. O valor digitado será usado com o substituível **&numero**.

3. Bloco anônimo:

- ✓ **DECLARE vNumero NUMBER := №**: Atribui à variável vNumero o valor digitado pelo usuário.
- ✓ **IF MOD(vNumero, 2) = 0 THEN (...)**: Usa a função **MOD** para obter o resto da divisão por 2. Se o resto for 0, o número é par. Caso contrário, é ímpar.
- ✓ O **dbms_output.put_line** exibe a mensagem de acordo com o resultado da verificação.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

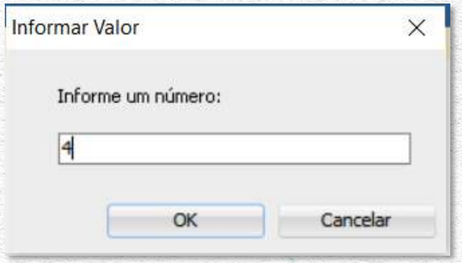
Questão 2

Crie um bloco anônimo que receba via ACCEPT um número e, usando IF, se ele for par, exiba "Número par". Caso contrário, exiba "Número ímpar".

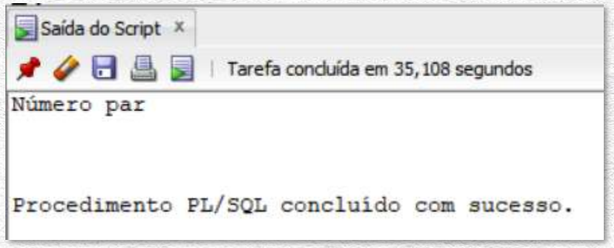
Explicação e Resultado

Assim, quando executamos o código, aparece uma janelinha na tela para digitarmos um número.

Se informarmos um valor divisível por 2...



Retornamos o resultado “Número par”:



CONTINUA NA PRÓXIMA PÁGINA..

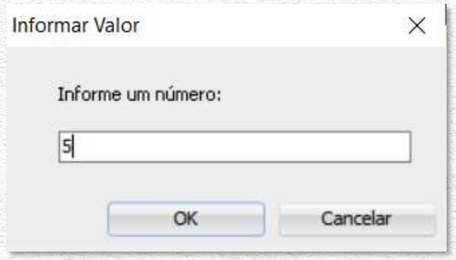
GABARITO EXPLICATIVO

2 Questão 2

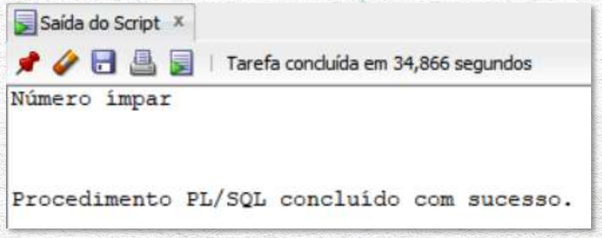
Crie um bloco anônimo que receba via ACCEPT um número e, usando IF, se ele for par, exiba "Número par". Caso contrário, exiba "Número ímpar".

Explicação e Resultado

Mas se informarmos um valor que não seja divisível por 2...



Retornamos o resultado “Número ímpar”:



Esse bloco é um exemplo clássico de uso do IF com condição numérica. A função MOD permite verificar facilmente se o número é par ou ímpar, tornando o código simples e eficiente para esse tipo de validação.

GABARITO EXPLICATIVO

3

Questão 3

Crie um bloco anônimo que receba via ACCEPT a nota de um aluno e, usando CASE, exiba:

- ✓ Se a nota for maior ou igual a 7: "Aprovado";
- ✓ Se estiver entre 5 (inclusive) e 7: "Recuperação";
- ✓ Caso contrário: "Reprovado".

Explicação e Resultado

Para receber a nota de um aluno e, utilizando CASE, exibir se ele está "Aprovado", em "Recuperação" ou "Reprovado", com base no valor da nota, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

ACCEPT nota PROMPT 'Informe a nota do aluno: '

DECLARE
    vNota NUMBER := &nota;
BEGIN
    CASE WHEN vNota >= 7 THEN
        dbms_output.put_line('Aprovado');
    WHEN vNota >= 5 THEN
        dbms_output.put_line('Recuperação');
    ELSE
        dbms_output.put_line('Reprovado');
    END CASE;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Crie um bloco anônimo que receba via ACCEPT a nota de um aluno e, usando CASE, exiba:

- ✓ Se a nota for maior ou igual a 7: "Aprovado";
- ✓ Se estiver entre 5 (inclusive) e 7: "Recuperação";
- ✓ Caso contrário: "Reprovado".

Explicação e Resultado

Explicação passo a passo:

1. Ativação da saída de mensagens:

- ✓ **SET SERVEROUTPUT ON;**: Habilita a exibição de mensagens na tela com **dbms_output.put_line**.

2. Entrada de dados:

- ✓ **ACCEPT nota PROMPT 'Informe a nota do aluno: '**: Solicita que o usuário digite a nota. O valor será referenciado como **&nota**.

3. Bloco anônimo:

- ✓ **DECLARE vNota NUMBER := &nota;**: Armazena a nota digitada pelo usuário.
- ✓ **Estrutura CASE:** Avalia as condições de forma sequencial.
  - Primeira verificação: **vNota >= 7**: Se verdadeira, exibe **"Aprovado"**.
  - Segunda verificação: **vNota >= 5**: Se não foi aprovado, mas essa condição for verdadeira, exibe **"Recuperação"**.
  - Caso nenhuma das anteriores se aplique, entra no **ELSE** e exibe **"Reprovado"**.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

3

Questão 3

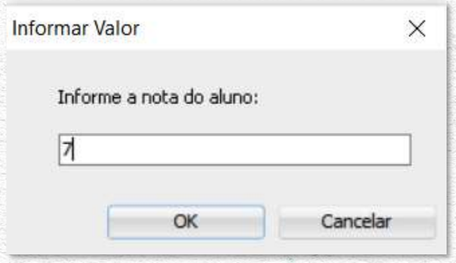
Crie um bloco anônimo que receba via ACCEPT a nota de um aluno e, usando CASE, exiba:

- ✓ Se a nota for maior ou igual a 7: "Aprovado";
- ✓ Se estiver entre 5 (inclusive) e 7: "Recuperação";
- ✓ Caso contrário: "Reprovado".

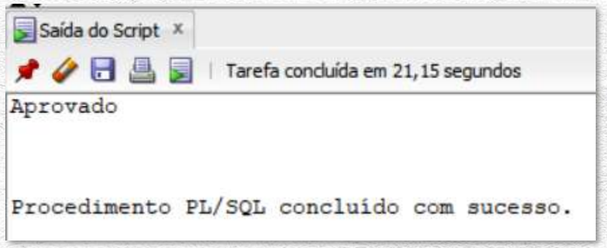
Explicação e Resultado

Assim, quando executamos o código, aparece uma janelinha na tela para digitarmos uma nota.

Se informarmos um valor maior ou igual a 7...



Retornamos o resultado “Aprovado”:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

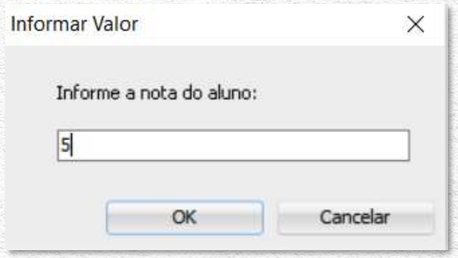
Questão 3

Crie um bloco anônimo que receba via ACCEPT a nota de um aluno e, usando CASE, exiba:

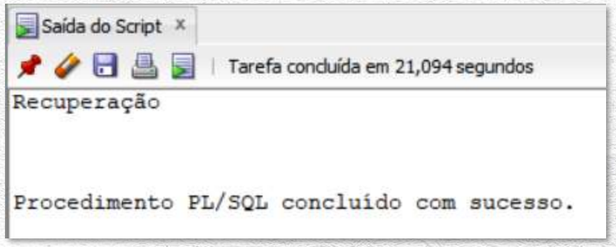
- ✓ Se a nota for maior ou igual a 7: "Aprovado";
- ✓ Se estiver entre 5 (inclusive) e 7: "Recuperação";
- ✓ Caso contrário: "Reprovado".

Explicação e Resultado

Se informarmos 5 ou 6...



Retornamos o resultado “Recuperação”:



CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

3

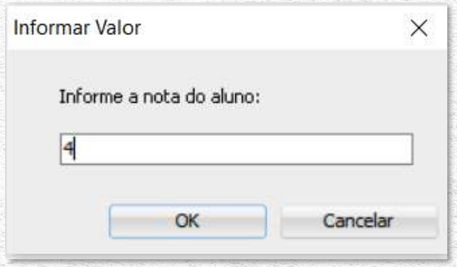
Questão 3

Crie um bloco anônimo que receba via ACCEPT a nota de um aluno e, usando CASE, exiba:

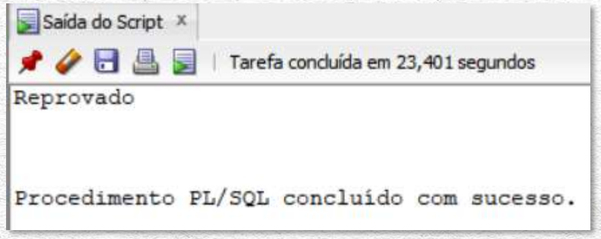
- ✓ Se a nota for maior ou igual a 7: "Aprovado";
- ✓ Se estiver entre 5 (inclusive) e 7: "Recuperação";
- ✓ Caso contrário: "Reprovado".

Explicação e Resultado

E, por fim, se informarmos uma nota menor que 5...



Retornamos o resultado “Reprovado”:



Este bloco utiliza CASE para organizar múltiplas condições de forma clara e direta. A estrutura é especialmente útil quando temos várias faixas de valores que exigem tratamentos diferentes, como é comum em avaliações escolares.

GABARITO EXPLICATIVO

4

Questão 4

Crie um bloco anônimo que receba via ACCEPT as siglas dos estados da Região Sudeste ("SP", "RJ", "MG", "ES") e exiba o nome completo do estado usando CASE. Se for informada a sigla de outro estado não pertencente à Região Sudeste, exiba "Estado não cadastrado".

Para evitar resultado incorreto, lembre-se de usar uma função que converta a sigla informada para letras maiúsculas antes de compará-la, caso o usuário tenha informando-a com letras minúsculas.

Explicação e Resultado

Para receber uma sigla de estado via ACCEPT, converter para letras maiúsculas e exibir o nome do estado se for da Região Sudeste, caso contrário, exibir "Estado não cadastrado", criamos o seguinte código:

```
SET SERVEROUTPUT ON;

ACCEPT estado PROMPT 'Informe a sigla do estado: '

DECLARE
    vSigla VARCHAR2(2) := '&estado';
    vNomeEstado VARCHAR2(50);
BEGIN
    vNomeEstado := CASE
        WHEN UPPER(vSigla) = 'SP' THEN 'São Paulo'
        WHEN UPPER(vSigla) = 'RJ' THEN 'Rio de Janeiro'
        WHEN UPPER(vSigla) = 'MG' THEN 'Minas Gerais'
        WHEN UPPER(vSigla) = 'ES' THEN 'Espírito Santo'
        ELSE 'Estado não cadastrado'
    END;

    dbms_output.put_line(vNomeEstado);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Crie um bloco anônimo que receba via ACCEPT as siglas dos estados da Região Sudeste ("SP", "RJ", "MG", "ES") e exiba o nome completo do estado usando CASE. Se for informada a sigla de outro estado não pertencente à Região Sudeste, exiba "Estado não cadastrado".

Para evitar resultado incorreto, lembre-se de usar uma função que converta a sigla informada para letras maiúsculas antes de compará-la, caso o usuário tenha informando-a com letras minúsculas.

Explicação e Resultado

Explicação passo a passo:

1. Ativação da saída:

- ✓ **SET SERVEROUTPUT ON;**: Habilita o uso do **dbms_output.put_line** para exibir mensagens.

2. Entrada de dados:

- ✓ **ACCEPT estado PROMPT 'Informe a sigla do estado: '**: Solicita que o usuário digite a sigla do estado (ex: sp, RJ, etc.). O valor digitado será passado como **&estado**.

3. Bloco anônimo:

- ✓ **DECLARE vSigla VARCHAR2(2) := '&estado';**: Armazena a sigla digitada.
- ✓ **vNomeEstado VARCHAR2(50);**: declara a variável que receberá o nome do estado, dependendo da condição do CASE.
- ✓ **Estrutura CASE**: Compara a sigla com os estados da Região Sudeste. Se a sigla não corresponder a nenhuma das opções válidas, a cláusula ELSE retorna "Estado não cadastrado".
- ✓ **Uso de UPPER**: Garante que comparações funcionem mesmo que o usuário digite em minúsculas (ex: sp vira SP).
- ✓ **dbms_output.put_line(vNomeEstado);**: Exibe o resultado armazenado na variável vNomeEstado, após atendida uma das condições do CASE.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

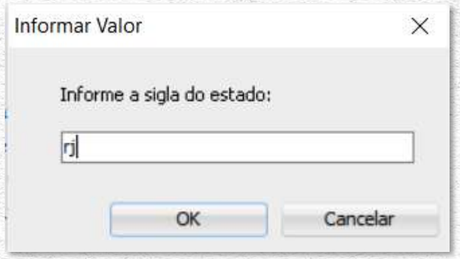
Crie um bloco anônimo que receba via ACCEPT as siglas dos estados da Região Sudeste ("SP", "RJ", "MG", "ES") e exiba o nome completo do estado usando CASE. Se for informada a sigla de outro estado não pertencente à Região Sudeste, exiba "Estado não cadastrado".

Para evitar resultado incorreto, lembre-se de usar uma função que converta a sigla informada para letras maiúsculas antes de compará-la, caso o usuário tenha informando-a com letras minúsculas.

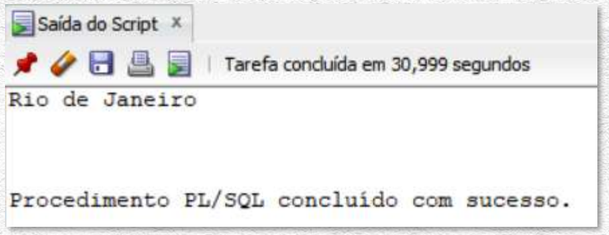
Explicação e Resultado

Assim, quando executamos o código, aparece uma janelinha na tela para digitarmos uma sigla de estado.

Se informarmos a sigla de algum estado da Região Sudeste, em letras maiúsculas ou minúsculas...



Retornamos o resultado o nome do referido estado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

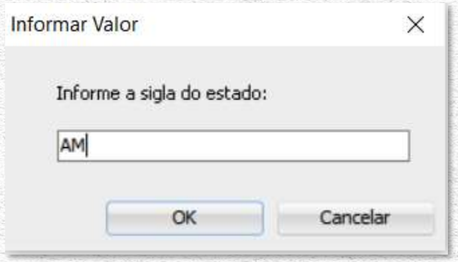
4 Questão 4

Crie um bloco anônimo que receba via ACCEPT as siglas dos estados da Região Sudeste ("SP", "RJ", "MG", "ES") e exiba o nome completo do estado usando CASE. Se for informada a sigla de outro estado não pertencente à Região Sudeste, exiba "Estado não cadastrado".

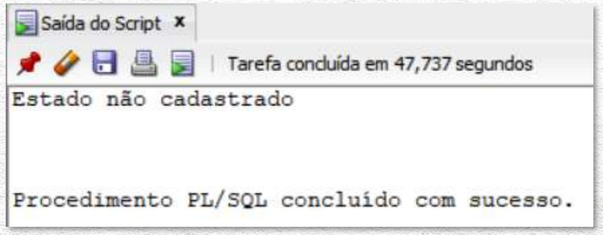
Para evitar resultado incorreto, lembre-se de usar uma função que converta a sigla informada para letras maiúsculas antes de compará-la, caso o usuário tenha informando-a com letras minúsculas.

Explicação e Resultado

Mas se informarmos a sigla de algum estado que não pertença à Região Sudeste...



Retornamos como resultado “Estado não cadastrado”:



O uso de UPPER com CASE torna o código robusto para diferentes formatos de entrada. Esse tipo de lógica é comum em sistemas que precisam validar ou identificar siglas de forma segura e amigável para o usuário.

GABARITO EXPLICATIVO

5

Questão 5

Crie um bloco anônimo que utilize um LOOP para somar números inteiros a partir de 1. O bloco deve parar quando a soma acumulada ultrapassar 100. Ao final, exiba a soma e o número que fez a soma ultrapassar esse limite.

Explicação e Resultado

Para usar um LOOP que acumule a soma dos inteiros a partir de 1 até que o total ultrapasse 100 e, ao final, exibir a soma final e o número responsável por ultrapassar esse limite, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    vSoma NUMBER := 0;
    vNumero NUMBER := 1;
BEGIN
    LOOP
        vSoma := vSoma + vNumero;

        EXIT WHEN vSoma > 100;

        vNumero := vNumero + 1;
    END LOOP;

    dbms_output.put_line('Soma final: ' || vSoma);
    dbms_output.put_line('Último número somado: ' || vNumero);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Crie um bloco anônimo que utilize um LOOP para somar números inteiros a partir de 1. O bloco deve parar quando a soma acumulada ultrapassar 100. Ao final, exiba a soma e o número que fez a soma ultrapassar esse limite.

Explicação e Resultado

Explicação passo a passo:

SET SERVEROUTPUT ON;: Ativa a exibição de mensagens com **dbms_output.put_line**.

DECLARE vSoma NUMBER := 0;: Declara a variável que acumula os valores.

vNumero NUMBER := 1;: Declara a variável com o número a ser somado. No caso, começa em 1 e será somado iterativamente.

O LOOP:

- ✓ Soma **vNumero** à **vSoma**.
- ✓ Verifica se a **vSoma** ultrapassou **100**.
- ✓ Se sim, o **LOOP** é encerrado com **EXIT**.
- ✓ Caso contrário, incrementa **vNumero** em **1** e repete.

Após o **LOOP**, o **dbms_output.put_line** exibe a soma acumulada e o último número somado que fez a soma ultrapassar 100.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

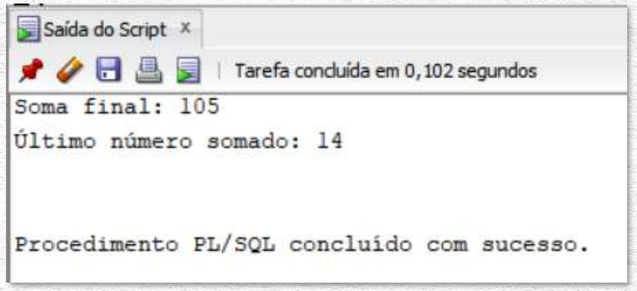
5

Questão 5

Crie um bloco anônimo que utilize um LOOP para somar números inteiros a partir de 1. O bloco deve parar quando a soma acumulada ultrapassar 100. Ao final, exiba a soma e o número que fez a soma ultrapassar esse limite.

Explicação e Resultado

Ao executarmos o código, **exibimos o seguinte resultado**:



Ou seja, o número **14** foi o responsável por fazer a soma passar de **100** (1 + 2 + ... + 14 = 105).

Esse tipo de estrutura com LOOP é útil para executar repetições controladas por condições, em vez de um número fixo de vezes. No exemplo, usamos um loop para acumular valores até atingir um limite, algo comum em situações como:

- Controle de metas (ex: somar vendas até atingir o faturamento desejado);
- Simulações financeiras;
- Processamento de registros até uma condição ser satisfeita.

A estrutura LOOP permite flexibilidade na lógica de parada, sendo ideal quando não se sabe previamente quantas iterações serão necessárias. É uma ferramenta essencial na automação de tarefas repetitivas.

GABARITO EXPLICATIVO

6

Questão 6

Um produto custa R$ 500. A cada mês, ele recebe um desconto de R$ 30. Crie um bloco que utilize WHILE LOOP para simular a aplicação mensal do desconto até que o valor do produto seja menor que R$ 300. Exiba o valor final e quantos meses de desconto foram aplicados.

Explicação e Resultado

Para simular a aplicação mensal de um desconto fixo (R$ 30) sobre o preço de um produto (inicialmente de R$ 500), até que ele fique abaixo de um valor mínimo (R$ 300), criamos o código abaixo, que utiliza um WHILE LOOP para repetir o cálculo enquanto a condição for verdadeira, acumulando o número de meses necessários:

```
SET SERVEROUTPUT ON;

DECLARE
    vPreco NUMBER := 500;
    vMeses NUMBER := 0;
BEGIN
    WHILE vPreco >= 300 LOOP
        vPreco := vPreco - 30;
        vMeses := vMeses + 1;
    END LOOP;

    dbms_output.put_line('Preço final: R$' || vPreco);
    dbms_output.put_line('Meses de desconto: ' || vMeses);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

Um produto custa R$ 500. A cada mês, ele recebe um desconto de R$ 30. Crie um bloco que utilize WHILE LOOP para simular a aplicação mensal do desconto até que o valor do produto seja menor que R$ 300. Exiba o valor final e quantos meses de desconto foram aplicados.

Explicação e Resultado

Explicação passo a passo:

SET SERVEROUTPUT ON;: Permite que o **dbms_output.put_line** funcione e exiba mensagens no console.

DECLARE vPreco NUMBER := 500;: Armazena o preço atual do produto, começando em R$ 500.

vMeses NUMBER := 0;: Contador que indica quantos meses de desconto foram aplicados.

O WHILE LOOP faz o seguinte:

- ✓ Enquanto o preço for maior ou igual a R$ 300, o loop continua.
- ✓ A cada iteração, o preço é reduzido em R$ 30 e o contador de meses é incrementado.
- ✓ Quando o preço fica abaixo de R$ 300, o loop é encerrado.

O **dbms_output.put_line** exibe o preço final e quantos meses foram necessários para que o preço caísse abaixo de R$ 300.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

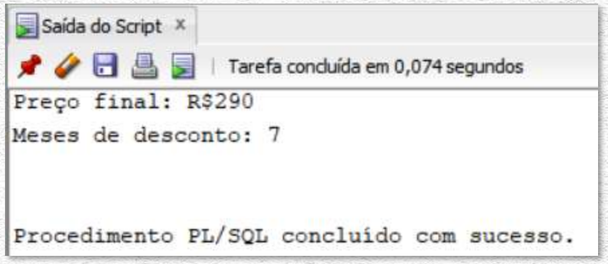
6

Questão 6

Um produto custa R$ 500. A cada mês, ele recebe um desconto de R$ 30. Crie um bloco que utilize WHILE LOOP para simular a aplicação mensal do desconto até que o valor do produto seja menor que R$ 300. Exiba o valor final e quantos meses de desconto foram aplicados.

Explicação e Resultado

Ao executarmos o código, **exibimos o seguinte resultado**:



Ou seja, foram necessários **7 meses** de descontos para que o preço caísse para menos de R$ 300 (no caso, **R$ 290**).

A estrutura WHILE LOOP é indicada quando a repetição depende de uma condição que deve ser verificada antes de cada execução. Neste exemplo, usamos esse tipo de loop para simular a redução progressiva de um valor, cenário comum em:

- Cálculo de depreciação de produtos;
- Aplicação de descontos em parcelas;
- Simulações financeiras ou de estoques.

Esse tipo de controle é essencial para automatizar processos com regras de negócio variáveis, especialmente quando não se conhece o número exato de iterações de antemão.

GABARITO EXPLICATIVO

7

Questão 7

Crie um bloco que utilize FOR LOOP para simular o pagamento de um valor de R$ 1200 em 4 parcelas iguais. Para cada parcela, exiba o número da parcela e o valor pago.

Explicação e Resultado

Para simular o pagamento parcelado de um valor total fixo (R$ 1200) utilizando a estrutura de repetição FOR LOOP, exibindo o número de cada parcela e o valor correspondente, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    vTotal NUMBER := 1200;
    vParcela NUMBER;
BEGIN
    FOR i IN 1..4 LOOP
        vParcela := vTotal / 4;
        dbms_output.put_line('Parcela ' || i || ': R$' || vParcela
    END LOOP;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Crie um bloco que utilize FOR LOOP para simular o pagamento de um valor de R$ 1200 em 4 parcelas iguais. Para cada parcela, exiba o número da parcela e o valor pago.

Explicação e Resultado

Explicação passo a passo:

SET SERVEROUTPUT ON;: Permite que o **dbms_output.put_line** exiba resultados na tela do SQL Developer.

DECLARE vTotal NUMBER := 1200;: Declara a variável que recebe o total da dívida (R$ 1200).

vParcela NUMBER;: Declara a variável que será usada para calcular o valor de cada parcela.

O FOR LOOP:

- ✓ Percorre de 1 até 4, simulando quatro parcelas.
- ✓ Em cada repetição, calcula-se o valor da parcela como 1200 / 4.
- ✓ Exibe o número da parcela e o valor.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

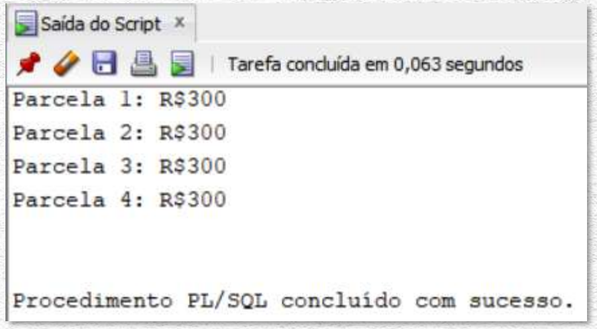
7

Questão 7

Crie um bloco que utilize FOR LOOP para simular o pagamento de um valor de R$ 1200 em 4 parcelas iguais. Para cada parcela, exiba o número da parcela e o valor pago.

Explicação e Resultado

Ao executarmos o código, **exibimos o seguinte resultado**:



Ou seja, foram pagos **R$ 300** em cada uma das **4 parcelas** para atingir o valor de **R$ 1200**.

O FOR LOOP é ideal quando se sabe de antemão o número exato de repetições, como neste caso com 4 parcelas. Esse tipo de estrutura é útil em situações como:

- Parcelamentos fixos de produtos ou serviços;
- Geração de boletos mensais;
- Cálculos repetitivos em quantidade conhecida.

Ele proporciona clareza e controle simples sobre iterações com limite fixo, facilitando automações previsíveis em sistemas administrativos e financeiros.

GABARITO EXPLICATIVO

8

Questão 8

Um usuário decide guardar R$ 150 por mês. Crie um bloco LOOP que simule essa economia mensal até que o valor acumulado atinja ou ultrapasse R$ 1000. Exiba o total acumulado e o número de meses necessários.

Explicação e Resultado

Para simular o processo de economia mensal (R$ 150) de uma pessoa utilizando um LOOP, até que o valor acumulado atinja ou ultrapasse R$ 1000, mostrando o total economizado e quantos meses foram necessários, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    vAcumulado NUMBER := 0;
    vMeses NUMBER := 0;
BEGIN
    LOOP
        vAcumulado := vAcumulado + 150;
        vMeses := vMeses + 1;

        EXIT WHEN vAcumulado >= 1000;
    END LOOP;

    dbms_output.put_line('Total acumulado: R$' || vAcumulado);
    dbms_output.put_line('Meses necessários: ' || vMeses);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Um usuário decide guardar R$ 150 por mês. Crie um bloco LOOP que simule essa economia mensal até que o valor acumulado atinja ou ultrapasse R$ 1000. Exiba o total acumulado e o número de meses necessários.

Explicação e Resultado

Explicação passo a passo:

SET SERVEROUTPUT ON;: Permite que os resultados sejam exibidos com **dbms_output.put_line**.

DECLARE vAcumulado NUMBER := 0;: Declara a variável que armazenará o total economizado ao longo dos meses, iniciando-a com o valor 0.

vMeses NUMBER := 0;: Declara a variável que contará quantos meses foram necessários, iniciando-a com o valor 0.

O LOOP faz o seguinte:

- ✓ A cada repetição, soma R$ 150 ao valor acumulado.
- ✓ Incrementa o número de meses.
- ✓ Termina quando o total acumulado for maior ou igual a R$ 1000.

O **dbms_output.put_line** exibe o total acumulado e a quantidade de meses de economia necessários para atingir o referido montante.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

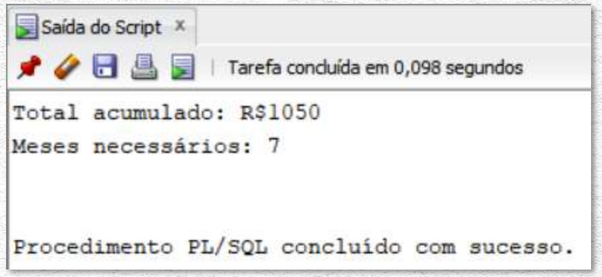
8

Questão 8

Um usuário decide guardar R$ 150 por mês. Crie um bloco LOOP que simule essa economia mensal até que o valor acumulado atinja ou ultrapasse R$ 1000. Exiba o total acumulado e o número de meses necessários.

Explicação e Resultado

Ao executarmos o código, **exibimos o seguinte resultado**:



Ou seja, foram necessários **7 meses** de economia de R$ 150 para acumular mais de R$ 1000 (no caso, **R$ 1050**).

Esse exercício mostra como usar o LOOP para simular um processo repetitivo com condição de parada baseada em valor acumulado. Esse tipo de estrutura é útil em diversos cenários do dia a dia, como:

- Planejamento financeiro e metas de economia;
- Simulações de metas progressivas;
- Acúmulo de pontos, milhas ou bonificações.

O LOOP fornece flexibilidade quando o número de repetições não é conhecido de antemão, apenas a condição de parada, o que torna seu uso ideal nesse tipo de problema.

GABARITO EXPLICATIVO

9

Questão 9

Uma loja possui 30 unidades de um produto em estoque. A cada venda, 3 unidades são retiradas. Crie um bloco com WHILE LOOP que simule essas vendas até o estoque ficar abaixo de 5 unidades. Exiba o número de vendas realizadas e o estoque final.

Explicação e Resultado

Para simular o controle de estoque de um produto utilizando a estrutura WHILE LOOP, reduzindo o estoque a cada venda (de 3 em 3 unidades) até restarem menos de 5 unidades, exibindo quantas vendas foram feitas e qual o estoque final, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    vEstoque NUMBER := 30;
    vVendas NUMBER := 0;
BEGIN
    WHILE vEstoque >= 5 LOOP
        vEstoque := vEstoque - 3;
        vVendas := vVendas + 1;
    END LOOP;

    dbms_output.put_line('Vendas realizadas: ' || vVendas);
    dbms_output.put_line('Estoque restante: ' || vEstoque);
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

Uma loja possui 30 unidades de um produto em estoque. A cada venda, 3 unidades são retiradas. Crie um bloco com WHILE LOOP que simule essas vendas até o estoque ficar abaixo de 5 unidades. Exiba o número de vendas realizadas e o estoque final.

Explicação e Resultado

Explicação passo a passo:

SET SERVEROUTPUT ON;: Garante que as mensagens geradas com **dbms_output.put_line** serão exibidas.

DECLARE vEstoque NUMBER := 30;: Declara a variável que armazena a quantidade atual de produtos em estoque (inicialmente 30).

vVendas NUMBER := 0;: Declara a variável que conta quantas vendas foram realizadas, iniciando-a com o valor 0.

O WHILE LOOP faz o seguinte:

- ✓ Enquanto o estoque for igual ou maior que 5, a loja realiza uma venda de 3 unidades.
- ✓ A cada venda, incrementa-se o contador vVendas.
- ✓ Quanto o estoque ficar abaixo de 5 unidades, o WHILE LOOP é encerrado.

O **dbms_output.put_line** exibe quantas vendas foram realizadas até que o estoque ficasse com menos de 5 unidades, assim como o estoque restante.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

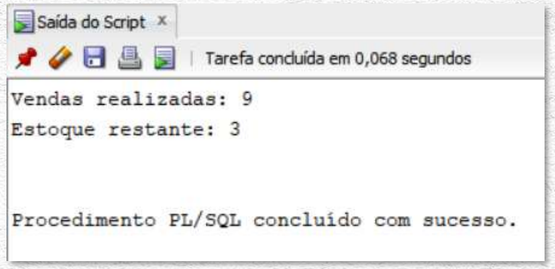
9

Questão 9

Uma loja possui 30 unidades de um produto em estoque. A cada venda, 3 unidades são retiradas. Crie um bloco com WHILE LOOP que simule essas vendas até o estoque ficar abaixo de 5 unidades. Exiba o número de vendas realizadas e o estoque final.

Explicação e Resultado

Ao executarmos o código, **exibimos o seguinte resultado**:



Ou seja, foram necessárias **9 vendas** (de 3 em 3 unidades) para que a quantidade de unidades disponíveis em estoque ficasse abaixo de 5 (no caso, restaram **3 unidades**).

Esse exercício demonstra como o WHILE LOOP pode ser aplicado em cenários de controle de estoque, onde é necessário repetir uma ação (venda) até que se atinja uma condição de parada (estoque mínimo).

Essa estrutura é útil em rotinas empresariais como:

- Simulações de consumo de materiais;
- Controle automatizado de vendas e alertas de estoque mínimo;
- Planejamento de reabastecimento com base em movimentações.

O WHILE LOOP é ideal quando o número de repetições não é fixo, e depende de uma condição dinâmica que deve ser verificada a cada ciclo.

GABARITO EXPLICATIVO

10

Questão 10

Um salário inicial é de R$ 2000. Crie um bloco com FOR LOOP que simule um aumento de 5% ao ano, durante 3 anos. Exiba o salário ao final de cada ano.

Explicação e Resultado

Para simular um aumento salarial anual de 5% utilizando a estrutura FOR LOOP, durante um período de 3 anos sobre um salário inicial de R$ 2000, exibindo o salário ao final de cada ano, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    vSalario NUMBER := 2000;
BEGIN
    FOR ano IN 1..3 LOOP
        vSalario := vSalario * 1.05;
        dbms_output.put_line('Salário após ano ' || ano || ': R$' || ROUND(vSalario, 2));
    END LOOP;
END;
```

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

10

Questão 10

Um salário inicial é de R$ 2000. Crie um bloco com FOR LOOP que simule um aumento de 5% ao ano, durante 3 anos. Exiba o salário ao final de cada ano.

Explicação e Resultado

Explicação passo a passo:

SET SERVEROUTPUT ON;: Permite que as saídas com **dbms_output.put_line** sejam visualizadas.

DECLARE vSalario NUMBER := 2000;: Declara a variável que armazena o salário, definindo-o inicialmente como R$ 2000.

O FOR LOOP:

- ✓ É executado de 1 até 3, representando os 3 anos.
- ✓ Em cada iteração, o salário recebe um aumento de 5% multiplicando por 1.05.
- ✓ O dbms_output.put_line exibe o salário de cada ano com duas casas decimais usando ROUND.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

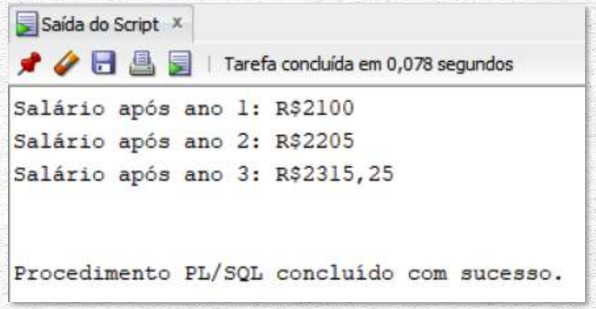
10

Questão 10

Um salário inicial é de R$ 2000. Crie um bloco com FOR LOOP que simule um aumento de 5% ao ano, durante 3 anos. Exiba o salário ao final de cada ano.

Explicação e Resultado

Ao executarmos o código, **exibimos o seguinte resultado**:



Ou seja, após **1 ano** o salário passou a ser de **R$ 2100**; após **2 anos**, de **R$ 2205**; e após **3 anos**, de **R$ 2315,25**.

Esse exercício mostra como o FOR LOOP pode ser aplicado em simulações de crescimento financeiro com prazo definido, como aumentos salariais, rendimentos de investimentos ou reajustes anuais.

A estrutura FOR LOOP é ideal quando se conhece de antemão o número de repetições, neste caso, 3 anos. Esse tipo de código é útil para:

- Previsão de gastos com folha salarial;
- Projeções de orçamento;
- Simulação de crescimento de rendimentos ao longo do tempo.

MÓDULO 16

FUNCTIONS

FUNCTIONS

FUNCTIONS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 16 FUNCTIONS

01	O QUE É UMA FUNCTION?	772
02	COMO CRIAR FUNCTIONS	773
03	UTILIZANDO FUNCTIONS DENTRO DE UM SELECT	776
04	COMO EXCLUIR FUNCTIONS	777
05	RESUMO DO MÓDULO	778
06	EXERCÍCIOS	779
07	GABARITO EXPLICATIVO	783

O que é uma FUNCTION?

Uma FUNCTION é uma sub-rotina, um conjunto de instruções que utilizamos para executar algum cálculo no banco de dados.

As FUNCTIONS criadas têm o mesmo papel das funções single row: receber argumentos e retornar um valor único.

SINTAXE:

```
CREATE OR REPLACE FUNCTION nome_function (arg_1 tipo_dado_1, arg_n tipo_dado_n)   -- Crie ou substitua a função 'X':
RETURN NUMBER IS                                                                  -- Para retornar valor:
        variáveis                                                                 -- Declare as variáveis;
BEGIN                                                                             -- Inicie:
        instruções da function                                                    -- Execute estas instruções;
        RETURN valor;                                                             -- Retorne o valor;
END;                                                                              -- Finalize a função.
```

Como criar FUNCTIONS

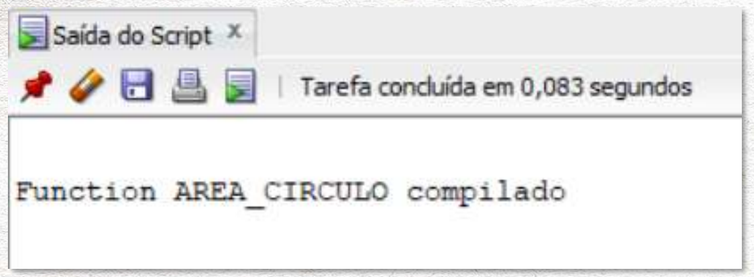
Vejamos um exemplo de como criar uma FUNCTION:

Queremos criar uma função que calcule e retorne a área de um círculo:

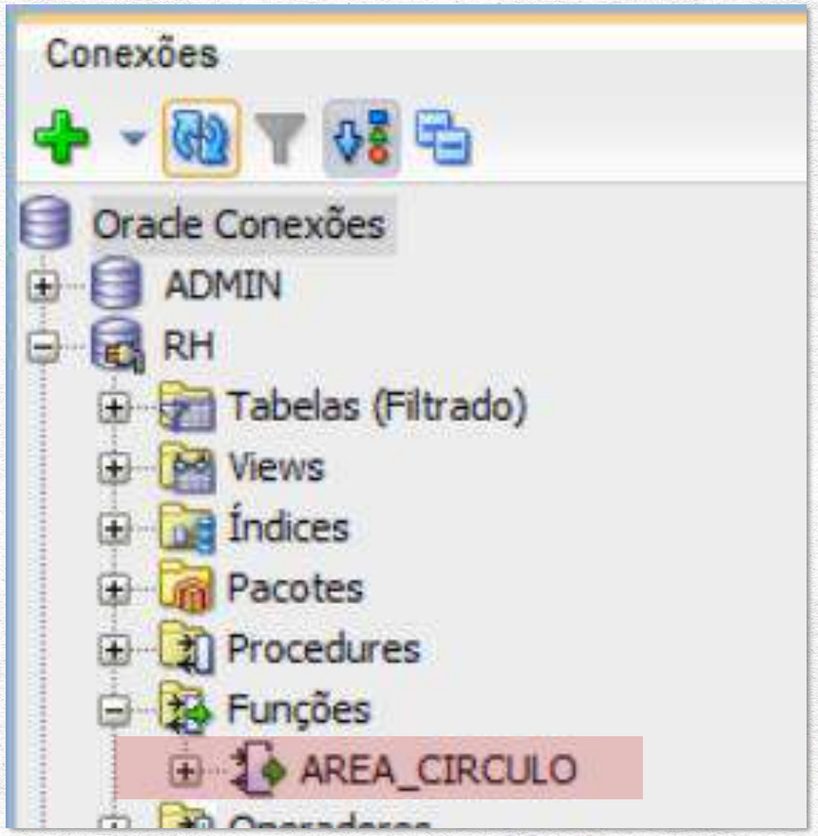
1. Iniciamos com o comando CREATE OR REPLACE FUNCTION (para criar a função, caso não exista; ou substituí-la, caso já exista), definimos um nome para ela (area_circulo), e quais serão seus argumentos (neste caso, apenas um: raio do tipo NUMBER);
2. Começamos a declaração das variáveis com RETURN NUMBER IS;
3. Declaramos as variáveis pi e area;
4. Iniciamos o bloco que receberá as instruções da função;
   Definimos o valor da variável pi;
   Definimos o cálculo a ser efetuado e armazenado na variável area;
   Retornamos o valor da variável area;
5. Finalizamos o bloco de instruções e a função.

```
CREATE OR REPLACE FUNCTION area_circulo (raio NUMBER) 1
RETURN NUMBER 2
IS

pi NUMBER(10, 4);
area NUMBER(10, 4); 3

BEGIN 4
    -- Área do circulo: pi * r * r
    pi := 3.1413;
    area := pi * (raio * raio);
    RETURN area;
END; 5
```

Como criar FUNCTIONS



Criada a FUNCTION, ela aparecerá no submenu Funções da conexão que estivermos utilizando:



Como criar FUNCTIONS

Para retornar o resultado de uma FUNCTION na tela, podemos criar um bloco anônimo:

1. Inicializamos o serviço de impressão;
2. Em um bloco anônimo, declaramos uma variável (teste);
3. Iniciamos o bloco de execução com BEGIN;
   Atribuímos à variável teste criada acima a function area_circulo, passando um argumento (raio) para a function (no exemplo, o valor 3);
   Imprimimos na tela a mensagem: “A área do círculo é: [teste]”;
4. Finalizamos o bloco de execução e o bloco anônimo com END.

```
SET SERVEROUTPUT ON; 1

DECLARE 2
    teste NUMBER(10, 4);
BEGIN 3
    teste := area_circulo(3);
    dbms_output.put_line('A área do círculo é: ' || teste);
END; 4
```

Veja só o resultado:

```
Saída do Script ×
Tarefa concluída em 0,068 segundos
A área do círculo é: 28,2717

Procedimento PL/SQL concluído com sucesso.
```

Utilizando FUNCTIONS dentro de um SELECT

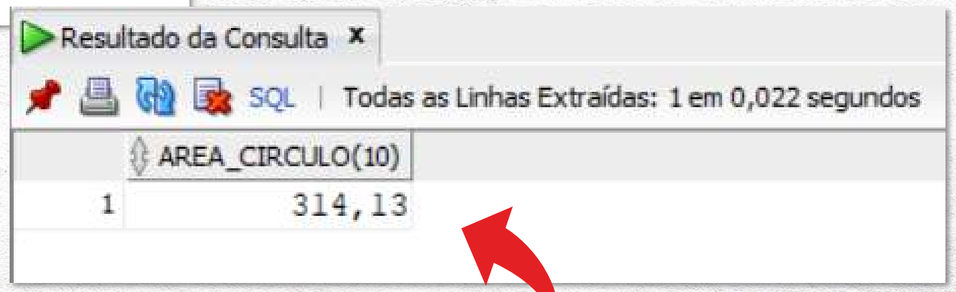
Também podemos retornar o valor calculado por uma FUNCTION dentro de uma consulta (SELECT).

Veja:

```
SELECT area_circulo(10)
FROM dual;
```

Desta vez, atribuímos o valor 10 ao raio.

Precisamos utilizar a tabela dual na nossa consulta.



Veja o resultado acima.

Como excluir FUNCTIONS

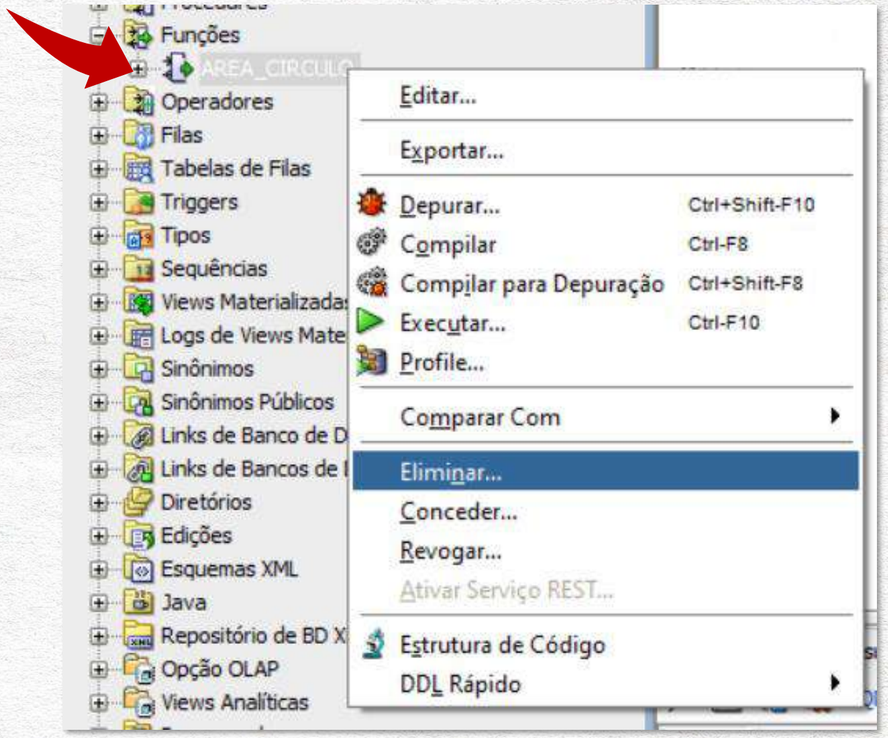
Para excluir uma FUNCTION, basta utilizarmos o comando:

DROP FUNCTION nome_funcao;

```
DROP FUNCTION area_circulo;
```

```
Saída do Script
Tarefa concluída em 0,051 segundos

Function AREA_CIRCULO eliminado.
```

Ou, se preferirmos, podemos também clicar com o botão direito sobre o nome da FUNCTION e selecionar Eliminar:



Resumo do Módulo

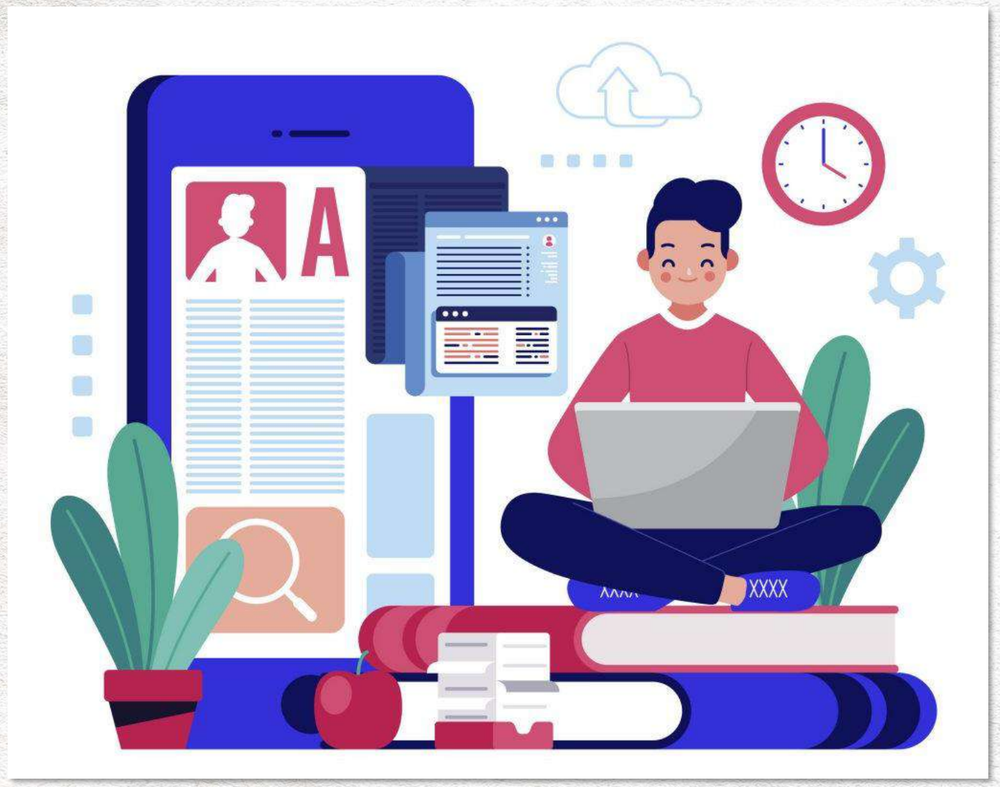
Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. FUNCTION é um conjunto de instruções que utilizamos para executar algum cálculo no banco de dados.
   As FUNCTIONS criadas têm o mesmo papel das funções single row: receber argumentos e retornar um valor único.

2. As FUNCTIONS sempre retornam um único valor.

3. As FUNCTIONS criadas podem ser utilizadas tanto dentro de blocos anônimos quanto em SELECTs.

EXERCÍCIOS



1

Questão 1

Sua equipe de RH foi contratada para fazer a gestão de funcionários de uma multinacional presente em países com diferentes unidades de temperatura. Para alguns relatórios locais, é necessário converter temperaturas de Celsius para Fahrenheit.

Crie uma function que receba a temperatura em Celsius e retorne o valor correspondente em Fahrenheit.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

2

Questão 2

Sua empresa está revisando os benefícios de funcionários conforme a idade.

Para isso, crie uma function que receba a data de nascimento de um funcionário e retorne a idade em anos completos com base na data atual do sistema.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

EXERCÍCIOS

3

Questão 3

No processo de confecção de crachás, é necessário apresentar o nome completo do funcionário em uma única linha.

Crie uma function que receba o primeiro e último nome de um funcionário e retorne o nome completo com espaço entre eles.

Teste essa function com os nomes (first_name e last_name) dos funcionários registrados na tabela employees.

4

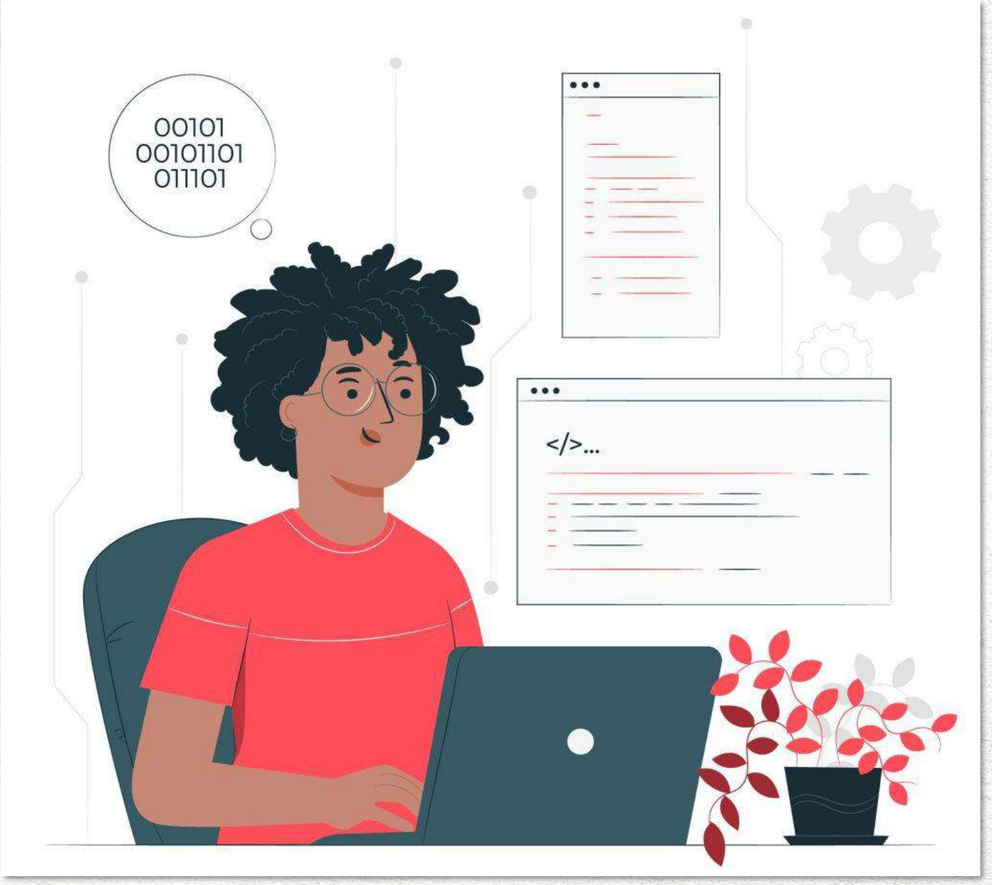
Questão 4

Sua equipe de RH está avaliando a distribuição salarial.

Crie uma function que receba o salário de um funcionário e o valor médio de salários da empresa, e retorne 'ACIMA' se for maior que a média ou 'DENTRO/ABAIXO' caso contrário.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os salários dos funcionários, bem como uma coluna com o status verificado pela function. Dica: neste SELECT, você pode usar uma subquery que calcule a média salarial e passe esse valor como parâmetro para sua function.

EXERCÍCIOS



5

Questão 5

Você precisa analisar há quanto tempo cada funcionário trabalha na empresa para fins de reconhecimento e premiação.

Crie uma function que receba a data de admissão e retorne a quantidade de meses completos de trabalho.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e as datas de contratação dos funcionários, além de uma coluna com a quantidade de meses completos trabalhados, verificados usando a function.

6

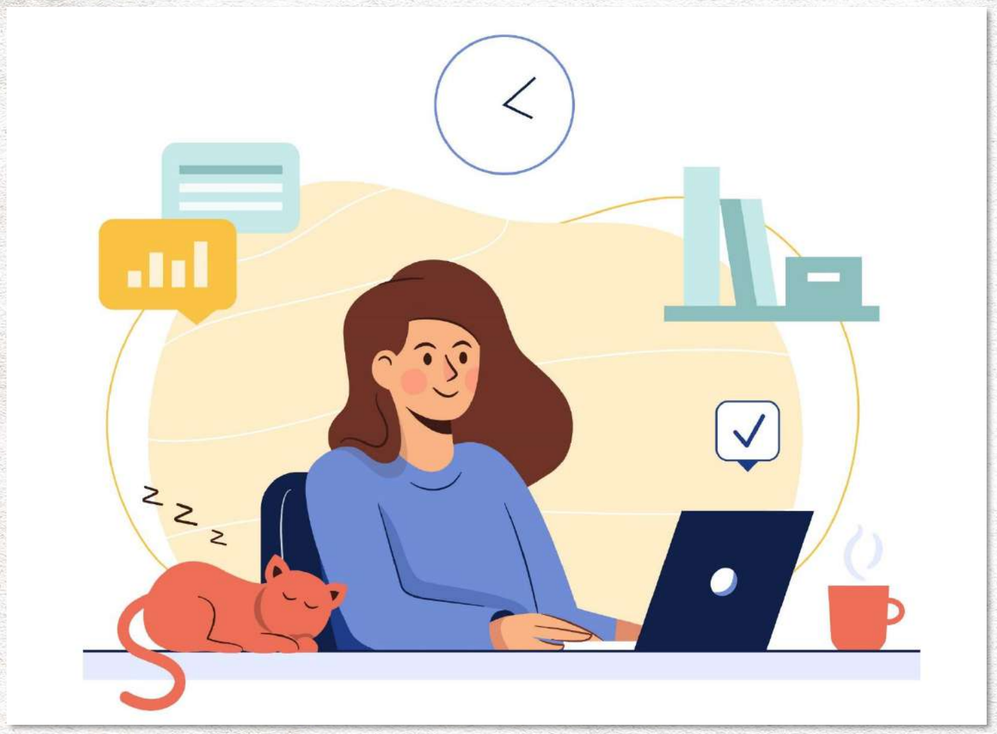
Questão 6

O RH deseja gerar uma lista de salários formatados com o símbolo "R$" para envio em PDF a gestores brasileiros.

Crie uma function que receba um salário (número) e retorne o mesmo valor em formato "R$ 9.999,99".

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs dos funcionários e seus respectivos salários formatados pela function.

EXERCÍCIOS



7

Questão 7

Os gestores solicitaram que o percentual de comissão dos funcionários seja apresentado em relatórios com o símbolo %, para melhor leitura.

Crie uma function que receba o valor decimal e retorne o percentual formatado como texto (Exemplo: 10%, 15%, 20%). E para aqueles que não recebem comissão, que seja exibida a informação 'SEM COMISSÃO'.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os percentuais de comissão (sem formatação) dos funcionários, bem como os percentuais formatados pela function.

8

Questão 8

Exclua todas as functions criadas nos exercícios anteriores.

GABARITO EXPLICATIVO

1

Questão 1

Sua equipe de RH foi contratada para fazer a gestão de funcionários de uma multinacional presente em países com diferentes unidades de temperatura. Para alguns relatórios locais, é necessário converter temperaturas de Celsius para Fahrenheit.

Crie uma function que receba a temperatura em Celsius e retorne o valor correspondente em Fahrenheit.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

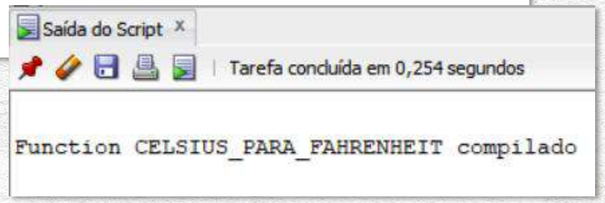
Objetivo:

Criar uma function que receba uma temperatura em Celsius e retorne o valor correspondente em Fahrenheit, utilizando a fórmula de conversão.

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION celsius_para_fahrenheit(temp_celsius NUMBER)
RETURN NUMBER IS
    temp_fahrenheit NUMBER;
BEGIN
    temp_fahrenheit := (temp_celsius * 9/5) + 32;
    RETURN temp_fahrenheit;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

Sua equipe de RH foi contratada para fazer a gestão de funcionários de uma multinacional presente em países com diferentes unidades de temperatura. Para alguns relatórios locais, é necessário converter temperaturas de Celsius para Fahrenheit.

Crie uma function que receba a temperatura em Celsius e retorne o valor correspondente em Fahrenheit.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION celsius_para_fahrenheit(temp_celsius NUMBER)**: Inicia a definição da function chamada celsius_para_fahrenheit. Ela recebe um parâmetro de entrada chamado temp_celsius, do tipo NUMBER, que representa a temperatura em Celsius.
- ✓ **RETURN NUMBER**: Indica que a function retornará um valor numérico (NUMBER), que será o resultado da conversão para Fahrenheit.
- ✓ **IS**: Palavra-chave que inicia a seção de declaração interna da function, onde são declaradas as variáveis locais.
- ✓ **temp_fahrenheit NUMBER;**: Declara uma variável local chamada temp_fahrenheit, também do tipo NUMBER. Ela armazenará o resultado da conversão antes de ser retornado.
- ✓ **BEGIN ... END;**: Delimita o bloco de execução da function. Todo o código responsável por processar os dados e gerar o resultado fica dentro desse bloco.
- ✓ **temp_fahrenheit := (temp_celsius * 9/5) + 32;**: Aplica a fórmula de conversão de Celsius para Fahrenheit. O resultado é armazenado na variável temp_fahrenheit.
- ✓ **RETURN temp_fahrenheit;**: Devolve o valor calculado como resultado final da function. Esse é o valor que será recebido por quem chamar a function.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

1 Questão 1

Sua equipe de RH foi contratada para fazer a gestão de funcionários de uma multinacional presente em países com diferentes unidades de temperatura. Para alguns relatórios locais, é necessário converter temperaturas de Celsius para Fahrenheit.

Crie uma function que receba a temperatura em Celsius e retorne o valor correspondente em Fahrenheit.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

2. Teste da function com bloco anônimo:

```
SET SERVEROUTPUT ON;

DECLARE
    resultado NUMBER;
BEGIN
    resultado := celsius_para_fahrenheit(30);
    dbms_output.put_line('30 graus Celsius equivalem a ' || resultado || ' Fahrenheit.');
END;
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON;**: Habilita a exibição de mensagens no console com dbms_output.put_line.
- ✓ **DECLARE resultado NUMBER;**: Declara uma variável chamada resultado, que será usada para armazenar o valor retornado pela function.
- ✓ **BEGIN resultado := celsius_para_fahrenheit(30);**: Chama a function celsius_para_fahrenheit, passando a temperatura em Celsius (por exemplo, 30) como parâmetro, armazenando o resultado (a temperatura em Fahrenheit) na variável resultado.
- ✓ **dbms_output.put_line(...)**: Exibe a temperatura em Fahrenheit no console.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

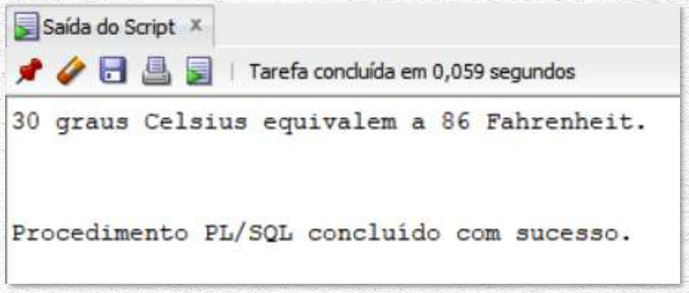
Sua equipe de RH foi contratada para fazer a gestão de funcionários de uma multinacional presente em países com diferentes unidades de temperatura. Para alguns relatórios locais, é necessário converter temperaturas de Celsius para Fahrenheit.

Crie uma function que receba a temperatura em Celsius e retorne o valor correspondente em Fahrenheit.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

Veja o resultado do nosso teste:



Esse exercício demonstra como encapsular uma lógica reutilizável dentro de uma function, permitindo que diferentes partes do sistema possam chamá-la sempre que for necessário converter temperaturas.

Esse tipo de function é especialmente útil para:

- ✓ Padronizar cálculos usados em relatórios internacionais;
- ✓ Evitar repetição de código;
- ✓ Facilitar manutenções futuras, centralizando a lógica em um único local.

GABARITO EXPLICATIVO

2

Questão 2

Sua empresa está revisando os benefícios de funcionários conforme a idade.

Para isso, crie uma function que receba a data de nascimento de um funcionário e retorne a idade em anos completos com base na data atual do sistema.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

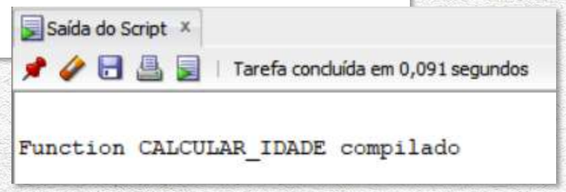
Objetivo:

Criar uma function que receba a data de nascimento de um funcionário e retorne a idade em anos completos, calculada com base na data atual do sistema.

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION calcular_idade(data_nascimento DATE)
RETURN NUMBER IS
    idade NUMBER;
BEGIN
    idade := FLOOR(MONTHS_BETWEEN(SYSDATE, data_nascimento)/12);
    RETURN idade;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Sua empresa está revisando os benefícios de funcionários conforme a idade.

Para isso, crie uma function que receba a data de nascimento de um funcionário e retorne a idade em anos completos com base na data atual do sistema.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION calcular_idade(data_nascimento DATE)**: Inicia a definição da function chamada calcular_idade, que recebe como parâmetro de entrada data_nascimento, do tipo DATE. Esse parâmetro representa a data de nascimento do funcionário.
- ✓ **RETURN NUMBER**: Indica que a function retornará um valor numérico (do tipo NUMBER), que corresponderá à idade calculada em anos completos.
- ✓ **IS**: Palavra-chave que marca o início da parte interna da function, onde são declaradas as variáveis locais.
- ✓ **idade NUMBER;**: Declara uma variável local chamada idade, do tipo NUMBER, que armazenará o valor final da idade antes de ser retornado.
- ✓ **BEGIN ... END;**: Delimita o bloco executável da function, ou seja, o trecho onde ocorre o processamento da lógica.
- ✓ **idade := FLOOR(MONTHS_BETWEEN(SYSDATE, data_nascimento)/12);**: Calcula a quantidade de meses entre a data atual (SYSDATE) e a data de nascimento informada, divide esse valor por 12 para converter em anos, e utiliza a função FLOOR para obter apenas a parte inteira do número, correspondente aos anos completos (ignorando meses e dias).
- ✓ **RETURN idade;**: Retorna o valor calculado da idade como resultado da function.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

Sua empresa está revisando os benefícios de funcionários conforme a idade.

Para isso, crie uma function que receba a data de nascimento de um funcionário e retorne a idade em anos completos com base na data atual do sistema.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

2. Teste da function com bloco anônimo:

```
SET SERVEROUTPUT ON;

DECLARE
    idade_funcionario NUMBER;
BEGIN
    idade_funcionario := calcular_idade(TO_DATE('1990-06-01', 'YYYY-MM-DD'));
    dbms_output.put_line('Idade do funcionário: ' || idade_funcionario || ' anos');
END;
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON;**: Habilita a exibição de mensagens no console com dbms_output.put_line.
- ✓ **DECLARE idade_funcionario NUMBER;**: Declara uma variável chamada idade_funcionario, que será usada para armazenar o valor retornado pela function.
- ✓ **idade_funcionario := calcular_idade(TO_DATE('1990-06-01', 'YYYY-MM-DD'));**: Chama a function calcular_idade, passando a data de nascimento no formato DATE (no exemplo, usamos a data 01/06/1990), e armazena o resultado (a idade) na variável idade_funcionario.
- ✓ **dbms_output.put_line(...)**: Exibe a idade calculada no console.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

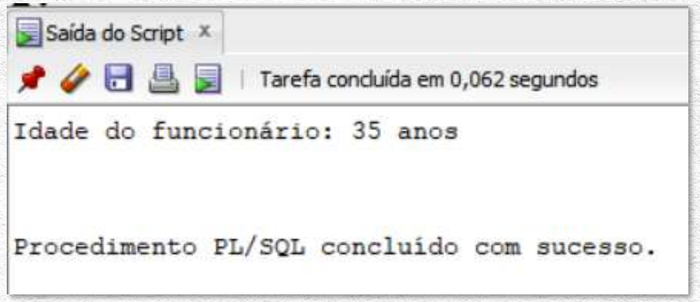
Sua empresa está revisando os benefícios de funcionários conforme a idade.

Para isso, crie uma function que receba a data de nascimento de um funcionário e retorne a idade em anos completos com base na data atual do sistema.

Lembre-se de, ao final, testar o funcionamento da function em um bloco anônimo.

Explicação e Resultado

Veja o resultado do nosso teste:



Esse exercício mostra como encapsular uma lógica de cálculo de idade dentro de uma function, permitindo reutilização e centralização do cálculo em sistemas de RH ou outros módulos que necessitem dessa informação.

Esse tipo de function é especialmente útil para:

- ✓ Automatizar regras baseadas em idade (ex: aposentadoria, benefícios);
- ✓ Garantir consistência no cálculo da idade em diferentes partes da aplicação;
- ✓ Reduzir redundância e facilitar manutenção futura da regra de cálculo.

GABARITO EXPLICATIVO

3

Questão 3

No processo de confecção de crachás, é necessário apresentar o nome completo do funcionário em uma única linha.

Crie uma function que receba o primeiro e último nome de um funcionário e retorne o nome completo com espaço entre eles.

Teste essa function com os nomes (first_name e last_name) dos funcionários registrados na tabela employees.

Explicação e Resultado

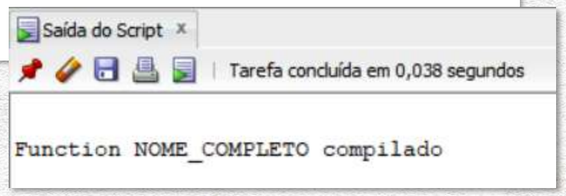
Objetivo:

Criar uma function que receba o primeiro e o último nome de um funcionário e retorne o nome completo com espaço entre eles.

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION nome_completo(p_nome VARCHAR2, u_nome VARCHAR2)
RETURN VARCHAR2 IS
    nome_full VARCHAR2(50);
BEGIN
    nome_full := p_nome || ' ' || u_nome;
    RETURN nome_full;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

No processo de confecção de crachás, é necessário apresentar o nome completo do funcionário em uma única linha.

Crie uma function que receba o primeiro e último nome de um funcionário e retorne o nome completo com espaço entre eles.

Teste essa function com os nomes (first_name e last_name) dos funcionários registrados na tabela employees.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION nome_completo(p_nome VARCHAR2, u_nome VARCHAR2)**: Define a function chamada nome_completo, que recebe dois parâmetros de entrada:
  - **p_nome**: representa o primeiro nome do funcionário (do tipo VARCHAR2);
  - **u_nome**: representa o último nome do funcionário (também VARCHAR2).
- ✓ **RETURN VARCHAR2**: Indica que a function retornará uma cadeia de caracteres (VARCHAR2) contendo o nome completo.
- ✓ **IS**: Inicia a seção de declaração de variáveis da function.
- ✓ **nome_full VARCHAR2(50);**: Declara uma variável local chamada nome_full, que armazenará o nome completo antes de ser retornado.
- ✓ **BEGIN ... END;**: Delimita o bloco executável onde ocorre a lógica da function.
- ✓ **nome_full := p_nome || ' ' || u_nome;**: Realiza a concatenação do primeiro nome com o último nome, separados por um espaço. Lembre-se: o operador || é utilizado para concatenar strings no Oracle.
- ✓ **RETURN nome_full;**: Retorna o nome completo resultante.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

No processo de confecção de crachás, é necessário apresentar o nome completo do funcionário em uma única linha.

Crie uma function que receba o primeiro e último nome de um funcionário e retorne o nome completo com espaço entre eles.

Teste essa function com os nomes (first_name e last_name) dos funcionários registrados na tabela employees.

Explicação e Resultado

2. Teste da function com um SELECT:

```
SELECT
    nome_completo(first_name, last_name) AS nome_completo
FROM employees;
```

Explicação do teste:

- ✓ O comando **SELECT** percorre todos os registros da tabela **employees**.
- ✓ Para cada linha, a function **nome_completo** é chamada, utilizando as colunas **first_name** e **last_name** como parâmetros.
- ✓ O resultado da function é exibido na coluna **nome_completo**.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

No processo de confecção de crachás, é necessário apresentar o nome completo do funcionário em uma única linha.

Crie uma function que receba o primeiro e último nome de um funcionário e retorne o nome completo com espaço entre eles.

Teste essa function com os nomes (first_name e last_name) dos funcionários registrados na tabela employees.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,064 seg

	NOME_COMPLETO
1	Ellen Abel
2	Sundar Ande
3	Mozhe Atkinson
4	David Austin
5	Hermann Baer
6	Shelli Baida
7	Amit Banda
8	Elizabeth Bates
9	Sarah Bell
10	David Bernstein
11	Laura Bissot
12	Harrison Bloom

Esse exercício demonstra como utilizar uma function para unificar dados relacionados (neste caso, nomes) de forma prática e reutilizável. Essa abordagem evita repetição de lógica de concatenação em diferentes partes do sistema e garante padronização na apresentação dos dados.

Esse tipo de function é especialmente útil para:

- ✓ Formatação de dados em relatórios ou telas do sistema;
- ✓ Geração de identificações (como crachás, certificados, listagens);
- ✓ Centralização da lógica de concatenação para facilitar manutenções futuras.

GABARITO EXPLICATIVO

4

Questão 4

Sua equipe de RH está avaliando a distribuição salarial.

Crie uma function que receba o salário de um funcionário e o valor médio de salários da empresa, e retorne 'ACIMA' se for maior que a média ou 'DENTRO/ABAIXO' caso contrário.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os salários dos funcionários, bem como uma coluna com o status verificado pela function. Dica: neste SELECT, você pode usar uma subquery que calcule a média salarial e passe esse valor como parâmetro para sua function.

Explicação e Resultado

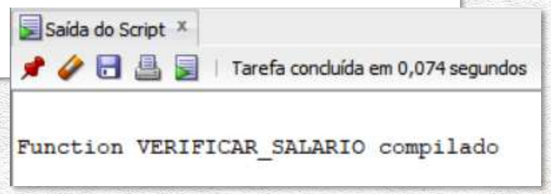
Objetivo:

Criar uma function que avalie se o salário de um funcionário está acima da média da empresa ou não, retornando um status padronizado.

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION verificar_salario(salario NUMBER, media NUMBER)
RETURN VARCHAR2 IS
BEGIN
    IF salario > media THEN
        RETURN 'ACIMA';
    ELSE
        RETURN 'DENTRO/ABAIXO';
    END IF;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Sua equipe de RH está avaliando a distribuição salarial.

Crie uma function que receba o salário de um funcionário e o valor médio de salários da empresa, e retorne 'ACIMA' se for maior que a média ou 'DENTRO/ABAIXO' caso contrário.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os salários dos funcionários, bem como uma coluna com o status verificado pela function. Dica: neste SELECT, você pode usar uma subquery que calcule a média salarial e passe esse valor como parâmetro para sua function.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION verificar_salario(salario NUMBER, media NUMBER)**: Define a function verificar_salario, que recebe dois parâmetros:
- ✓ **salario**: valor do salário individual do funcionário (tipo NUMBER);
- ✓ **media**: média geral dos salários da empresa (tipo NUMBER).
- ✓ **RETURN VARCHAR2**: Indica que a function retornará um texto (VARCHAR2) com o status da análise.
- ✓ **IS**: Inicia a seção de declaração de variáveis da function.
- ✓ **BEGIN ... END;**: Delimita o bloco executável onde será feita a comparação.
- ✓ **IF salario > media THEN**: Verifica se o salário está acima da média.
- ✓ **RETURN 'ACIMA';**: Retorna a palavra 'ACIMA' se a condição for verdadeira.
- ✓ **ELSE**: Caso contrário...
- ✓ **RETURN 'DENTRO/ABAIXO';**: Retorna 'DENTRO/ABAIXO' se o salário for igual ou inferior à média.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4 Questão 4

Sua equipe de RH está avaliando a distribuição salarial.

Crie uma function que receba o salário de um funcionário e o valor médio de salários da empresa, e retorne 'ACIMA' se for maior que a média ou 'DENTRO/ABAIXO' caso contrário.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os salários dos funcionários, bem como uma coluna com o status verificado pela function. Dica: neste SELECT, você pode usar uma subquery que calcule a média salarial e passe esse valor como parâmetro para sua function.

Explicação e Resultado

2. Teste da function com um SELECT:

```
SELECT employee_id,
       salary,
       verificar_salario(salary, (SELECT AVG(salary) FROM employees)) AS status
FROM employees;
```

Explicação do teste:

- ✓ A consulta retorna os **employee_id**, **salary** e um **status** baseado na média geral.
- ✓ A chamada **verificar_salario(salary, (SELECT AVG(salary) FROM employees))** compara cada salário individual com a média salarial geral.
- ✓ A subquery **(SELECT AVG(salary) FROM employees)** calcula a média dos salários na tabela **employees** em tempo real.
- ✓ Para cada linha da tabela, a function é chamada com os valores corretos.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Sua equipe de RH está avaliando a distribuição salarial.

Crie uma function que receba o salário de um funcionário e o valor médio de salários da empresa, e retorne 'ACIMA' se for maior que a média ou 'DENTRO/ABAIXO' caso contrário.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os salários dos funcionários, bem como uma coluna com o status verificado pela function. Dica: neste SELECT, você pode usar uma subquery que calcule a média salarial e passe esse valor como parâmetro para sua function.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em

	EMPLOYEE_ID	SALARY	STATUS
1	100	24000	ACIMA
2	101	17000	ACIMA
3	102	17000	ACIMA
4	103	9000	ACIMA
5	104	6000	DENTRO/ABAIXO
6	105	4800	DENTRO/ABAIXO
7	106	4800	DENTRO/ABAIXO
8	107	4200	DENTRO/ABAIXO
9	108	12000	ACIMA
10	109	9000	ACIMA
11	110	8200	ACIMA
12	111	7700	ACIMA
13	112	7800	ACIMA
14	113	6900	ACIMA
15	114	11000	ACIMA
16	115	3100	DENTRO/ABAIXO

Esse exercício mostra como criar functions personalizadas para análises comparativas em tempo real, promovendo clareza e padronização nos relatórios.

Esse tipo de function é especialmente útil para:

- ✓ Avaliar automaticamente indicadores de desempenho ou distribuição;
- ✓ Automatizar comparações com métricas corporativas (como metas ou médias);
- ✓ Criar dashboards ou relatórios com lógica incorporada, sem a necessidade de replicar comparações manuais.

GABARITO EXPLICATIVO

5

Questão 5

Você precisa analisar há quanto tempo cada funcionário trabalha na empresa para fins de reconhecimento e premiação.

Crie uma function que receba a data de admissão e retorne a quantidade de meses completos de trabalho.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e as datas de contratação dos funcionários, além de uma coluna com a quantidade de meses completos trabalhados, verificados usando a function.

Explicação e Resultado

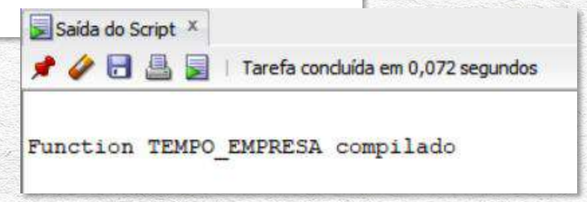
Objetivo:

Criar uma function que calcule a quantidade de meses completos que um funcionário está na empresa com base na sua data de contratação (hire_date).

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION tempo_empresa(hire_date DATE)
RETURN NUMBER IS
BEGIN
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, hire_date));
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Você precisa analisar há quanto tempo cada funcionário trabalha na empresa para fins de reconhecimento e premiação.

Crie uma function que receba a data de admissão e retorne a quantidade de meses completos de trabalho.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e as datas de contratação dos funcionários, além de uma coluna com a quantidade de meses completos trabalhados, verificados usando a function.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION tempo_empresa(hire_date DATE)**: Define a function chamada tempo_empresa, que recebe como parâmetro uma data (hire_date), representando a data de contratação do funcionário.
- ✓ **RETURN NUMBER**: Indica que a function retornará um número inteiro, que representa a quantidade de meses completos de trabalho.
- ✓ **IS**: Inicia a seção de declaração de variáveis da function.
- ✓ **BEGIN ... END;**: Delimita o bloco de execução da function.
- ✓ **RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, hire_date));**:
  - A função **MONTHS_BETWEEN** calcula a diferença entre duas datas em meses (com decimais).
  - **SYSDATE** representa a data atual do sistema.
  - **TRUNC(...)** remove a parte decimal, retornando somente os meses completos de trabalho.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Você precisa analisar há quanto tempo cada funcionário trabalha na empresa para fins de reconhecimento e premiação.

Crie uma function que receba a data de admissão e retorne a quantidade de meses completos de trabalho.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e as datas de contratação dos funcionários, além de uma coluna com a quantidade de meses completos trabalhados, verificados usando a function.

Explicação e Resultado

2. Teste da function com um SELECT:

```
SELECT
    employee_id,
    hire_date,
    tempo_empresa(hire_date) AS meses_empresa
FROM employees;
```

Explicação do teste:

- ✓ A consulta seleciona três colunas da tabela **employees**:
  - **employee_id**: ID do funcionário.
  - **hire_date**: Data em que o funcionário foi contratado.
  - **tempo_empresa(hire_date) AS meses_empresa**: Chamada da function, que calcula e retorna os meses completos desde a contratação até a data atual.
- ✓ O resultado será uma lista com os IDs, datas de admissão e o tempo de empresa em meses.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Você precisa analisar há quanto tempo cada funcionário trabalha na empresa para fins de reconhecimento e premiação.

Crie uma function que receba a data de admissão e retorne a quantidade de meses completos de trabalho.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e as datas de contratação dos funcionários, além de uma coluna com a quantidade de meses completos trabalhados, verificados usando a function.

Explicação e Resultado

Veja o resultado do nosso teste:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,

	EMPLOYEE_ID	HIRE_DATE	MESES_EMPRESA
1	100	17/06/87	455
2	101	21/09/89	428
3	102	13/01/93	389
4	103	03/01/90	425
5	104	21/05/91	408
6	105	25/06/97	335
7	106	05/02/98	328
8	107	07/02/99	316
9	108	17/08/94	369
10	109	16/08/94	370
11	110	28/09/97	332
12	111	30/09/97	332
13	112	07/03/98	327

Esse exercício mostra como encapsular cálculos temporais reutilizáveis dentro de uma function, tornando os relatórios de tempo de casa mais simples, padronizados e automáticos.

Esse tipo de function é especialmente útil para:

- ✓ Premiações por tempo de serviço;
- ✓ Gestão de benefícios com base em tempo de empresa;
- ✓ Campanhas de reconhecimento e relatórios sazonais do RH;
- ✓ Evitar duplicação da lógica de cálculo em várias consultas e facilitar futuras manutenções.

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja gerar uma lista de salários formatados com o símbolo "R$" para envio em PDF a gestores brasileiros.

Crie uma function que receba um salário (número) e retorne o mesmo valor em formato "R$ 9.999,99".

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs dos funcionários e seus respectivos salários formatados pela function.

Explicação e Resultado

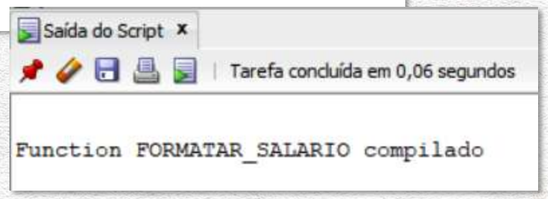
Objetivo:

Criar uma function que receba um salário numérico e retorne esse valor formatado no padrão brasileiro de moeda, incluindo o símbolo "R$" e os separadores de milhares e decimais.

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION formatar_salario(salario NUMBER)
RETURN VARCHAR2 IS
BEGIN
    RETURN 'R$ ' || TO_CHAR(salario, '999G999D99');
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja gerar uma lista de salários formatados com o símbolo "R$" para envio em PDF a gestores brasileiros.

Crie uma function que receba um salário (número) e retorne o mesmo valor em formato "R$ 9.999,99".

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs dos funcionários e seus respectivos salários formatados pela function.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION formatar_salario(salario NUMBER)**: Define a function chamada formatar_salario, que recebe um parâmetro salario do tipo numérico.
- ✓ **RETURN VARCHAR2**: Indica que a function retornará uma string formatada, ou seja, um valor textual.
- ✓ **IS**: Inicia a seção de declaração de variáveis da function.
- ✓ **BEGIN ... END;**: Delimita o bloco de execução da function.
- ✓ **RETURN 'R$ ' || TO_CHAR(salario, '999G999D99');**: Concatena o texto "R$ " ao valor formatado do salário. A função TO_CHAR converte o número para uma string formatada usando o modelo '999G999D99':
  - **G** representa o separador de milhares de acordo com a localidade do banco (ponto no Brasil).
  - **D** representa o separador decimal (vírgula no Brasil).
  - Assim, por exemplo, o número 1500.5 é formatado como "1.500,50", e concatenado com "R$ " fica "R$ 1.500,50".

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja gerar uma lista de salários formatados com o símbolo "R$" para envio em PDF a gestores brasileiros.

Crie uma function que receba um salário (número) e retorne o mesmo valor em formato "R$ 9.999,99".

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs dos funcionários e seus respectivos salários formatados pela function.

Explicação e Resultado

2. Teste da function com um SELECT:

```
SELECT
    employee_id,
    formatar_salario(salary) AS salario_formatado
FROM employees;
```

Explicação do teste:

- ✓ O comando seleciona o **employee_id** e o **salário formatado** da tabela **employees**.
- ✓ Para cada salário, chama a function **formatar_salario**, que retorna o valor com o símbolo "**R$**" e a **formatação brasileira**.
- ✓ O resultado apresenta uma coluna com salários já formatados para uso em relatórios ou documentos.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja gerar uma lista de salários formatados com o símbolo "R$" para envio em PDF a gestores brasileiros.

Crie uma function que receba um salário (número) e retorne o mesmo valor em formato "R$ 9.999,99".

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs dos funcionários e seus respectivos salários formatados pela function.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0

	EMPLOYEE_ID	SALARIO_FORMATADO
1	100	R$ 24.000,00
2	101	R$ 17.000,00
3	102	R$ 17.000,00
4	103	R$ 9.000,00
5	104	R$ 6.000,00
6	105	R$ 4.800,00
7	106	R$ 4.800,00
8	107	R$ 4.200,00
9	108	R$ 12.000,00
10	109	R$ 9.000,00
11	110	R$ 8.200,00
12	111	R$ 7.700,00
13	112	R$ 7.800,00

Esse exercício demonstra como usar functions para formatar dados numéricos para apresentação final, garantindo consistência visual em relatórios financeiros.

Esse tipo de function é especialmente útil para:

- ✓ Preparar dados para relatórios financeiros e comunicados oficiais;
- ✓ Facilitar a visualização e conferência dos valores pelos gestores;
- ✓ Evitar repetição de código de formatação em diferentes partes do sistema;
- ✓ Garantir padronização em diferentes sistemas e relatórios.

GABARITO EXPLICATIVO

7

Questão 7

Os gestores solicitaram que o percentual de comissão dos funcionários seja apresentado em relatórios com o símbolo %, para melhor leitura.

Crie uma function que receba o valor decimal e retorne o percentual formatado como texto (Exemplo: 10%, 15%, 20%). E para aqueles que não recebem comissão, que seja exibida a informação 'SEM COMISSÃO'.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os percentuais de comissão (sem formatação) dos funcionários, bem como os percentuais formatados pela function.

Explicação e Resultado

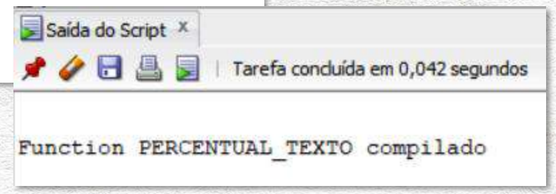
Objetivo:

Criar uma function que receba o valor decimal da comissão de um funcionário e retorne uma string formatada com o percentual e o símbolo "%". Caso o funcionário não receba comissão (valor NULL), a função deve retornar o texto "SEM COMISSÃO".

Passo a passo:

1. Criação da function:

```
CREATE OR REPLACE FUNCTION percentual_texto(perc NUMBER)
RETURN VARCHAR2 IS
BEGIN
    IF perc IS NULL THEN
        RETURN 'SEM COMISSÃO';
    ELSE
        RETURN TO_CHAR(perc * 100) || '%';
    END IF;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Os gestores solicitaram que o percentual de comissão dos funcionários seja apresentado em relatórios com o símbolo %, para melhor leitura.

Crie uma function que receba o valor decimal e retorne o percentual formatado como texto (Exemplo: 10%, 15%, 20%). E para aqueles que não recebem comissão, que seja exibida a informação 'SEM COMISSÃO'.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os percentuais de comissão (sem formatação) dos funcionários, bem como os percentuais formatados pela function.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE FUNCTION percentual_texto(perc NUMBER)**: Define a function chamada percentual_texto que recebe um parâmetro perc do tipo numérico, representando a comissão em formato decimal (ex: 0.10 para 10%).
- ✓ **RETURN VARCHAR2**: Especifica que a function retornará uma string, para exibir texto com o percentual ou mensagem.
- ✓ **IS**: Inicia a seção de declaração de variáveis da function.
- ✓ **BEGIN ... END;**: Bloco de execução da function.
- ✓ **IF perc IS NULL THEN**: Verifica se o valor da comissão é NULL (ou seja, funcionário não recebe comissão).
- ✓ **RETURN 'SEM COMISSÃO';**: Caso a comissão seja nula, retorna a mensagem "SEM COMISSÃO".
- ✓ **ELSE**: Caso contrário...
- ✓ **RETURN TO_CHAR(perc * 100) || '%';**: Multiplica o valor decimal por 100 para converter em percentual (ex: 0.10 * 100 = 10), converte o número em texto com TO_CHAR e concatena o símbolo "%".

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Os gestores solicitaram que o percentual de comissão dos funcionários seja apresentado em relatórios com o símbolo %, para melhor leitura.

Crie uma function que receba o valor decimal e retorne o percentual formatado como texto (Exemplo: 10%, 15%, 20%). E para aqueles que não recebem comissão, que seja exibida a informação 'SEM COMISSÃO'.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os percentuais de comissão (sem formatação) dos funcionários, bem como os percentuais formatados pela function.

Explicação e Resultado

2. Teste da function com um SELECT:

```
SELECT
    employee_id,
    commission_pct,
    percentual_texto(commission_pct) AS comissao_texto
FROM employees;
```

Explicação do teste:

- ✓ A consulta seleciona o **employee_id** do funcionário, o percentual de comissão sem formatação (**commission_pct**), e a **versão formatada** com texto, através da function **percentual_texto**.
- ✓ Para funcionários sem comissão (**commission_pct = NULL**), a function retorna **"SEM COMISSÃO"**.
- ✓ Para os demais, retorna o valor percentual com símbolo **"%"**.
- ✓ Assim, o relatório mostra claramente quem recebe comissão, exibindo o percentual formatado, e indica quem não recebe, evitando dúvidas e erros de interpretação.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

Os gestores solicitaram que o percentual de comissão dos funcionários seja apresentado em relatórios com o símbolo %, para melhor leitura.

Crie uma function que receba o valor decimal e retorne o percentual formatado como texto (Exemplo: 10%, 15%, 20%). E para aqueles que não recebem comissão, que seja exibida a informação 'SEM COMISSÃO'.

Teste essa function fazendo um SELECT na tabela employees. Retorne os IDs e os percentuais de comissão (sem formatação) dos funcionários, bem como os percentuais formatados pela function.

Explicação e Resultado

Veja o resultado do nosso teste:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 107 em 0,03 segu

	EMPLOYEE_ID	COMMISSION_PCT	COMISSAO_TEXTO
40	139	(null)	SEM COMISSÃO
41	140	(null)	SEM COMISSÃO
42	141	(null)	SEM COMISSÃO
43	142	(null)	SEM COMISSÃO
44	143	(null)	SEM COMISSÃO
45	144	(null)	SEM COMISSÃO
46	145	0,4	40%
47	146	0,3	30%
48	147	0,3	30%
49	148	0,3	30%
50	149	0,2	20%
51	150	0,3	30%
52	151	0,25	25%

Esse exercício exemplifica como tratar valores opcionais e apresentar dados de forma amigável e padronizada, melhorando a comunicação nos relatórios empresariais.

Esse tipo de function é especialmente útil para:

- ✓ Apresentar dados financeiros com clareza;
- ✓ Tratar valores ausentes (NULL) de forma personalizada;
- ✓ Centralizar a lógica de formatação em um único ponto do sistema;
- ✓ Evitar repetição e inconsistência na apresentação de dados.

GABARITO EXPLICATIVO

8

Questão 8

Exclua todas as functions criadas nos exercícios anteriores.

Explicação e Resultado

Para excluir as functions que criamos anteriormente, aplicamos um **DROP FUNCTION** com o nome de cada function:

```
DROP FUNCTION celsius_para_fahrenheit;
```
→
```
Function CELSIUS_PARA_FAHRENHEIT eliminado.
```

```
DROP FUNCTION calcular_idade;
```
→
```
Function CALCULAR_IDADE eliminado.
```

```
DROP FUNCTION nome_completo;
```
→
```
Function NOME_COMPLETO eliminado.
```

```
DROP FUNCTION verificar_salario;
```
→
```
Function VERIFICAR_SALARIO eliminado.
```

```
DROP FUNCTION tempo_empresa;
```
→
```
Function TEMPO_EMPRESA eliminado.
```

```
DROP FUNCTION formatar_salario;
```
→
```
Function FORMATAR_SALARIO eliminado.
```

```
DROP FUNCTION percentual_texto;
```
→
```
Function PERCENTUAL_TEXTO eliminado.
```

MÓDULO 17

PROCEDURES

PROCEDURES

PROCEDURES

O QUE VEREMOS NESTE MÓDULO

MÓDULO 17 PROCEDURES

01	O QUE É UMA PROCEDURE?	814
02	COMO CRIAR PROCEDURES	815
03	COMO CRIAR PROCEDURES SEM PARÂMETROS	816
04	COMO CRIAR PROCEDURES COM PARÂMETROS	822
05	PROCEDURES COM PARÂMETROS E ESTRUTURA DE CONTROLE	829
06	COMO EXCLUIR PROCEDURES	837
07	FUNCTIONS vs. PROCEDURES	838
08	RESUMO DO MÓDULO	839
09	EXERCÍCIOS	840
10	GABARITO EXPLICATIVO	848

O que é uma PROCEDURE?

Uma PROCEDURE é um bloco de código que possui um nome e pode ser armazenado no banco de dados, incluindo uma série de comandos SQL para executar alguma tarefa.

PROCEDURES são usadas para fazer tarefas repetitivas que não são possíveis em queries do SQL, podendo incluir estruturas de controle e comandos DDL, DML, TCL.

SINTAXE:

```
CREATE OR REPLACE PROCEDURE nome_procedure (param_1 tipo_dado_1, param_n tipo_dado_n)   -- Crie ou substitua a procedure 'X':
AS                                                                                      -- Como:
        variáveis                                                                       -- Declare as variáveis;
BEGIN                                                                                   -- Inicie:
        instruções da procedure                                                         -- Execute estas instruções;
END nome_procedure;                                                                     -- Finalize a procedure.
```

Observações:

- ✓ Nem toda procedure terá parâmetros;
- ✓ Informar o nome da procedure após o END não é obrigatório, mas é uma boa prática.

Como criar PROCEDURES

Existem duas formas de se criar uma PROCEDURE: **sem parâmetros** e **com parâmetros**.

Na PROCEDURE sem parâmetros, todas as informações necessárias para sua execução serão atribuídas dentro da própria PROCEDURE.

Já na PROCEDURE com parâmetros, essas informações serão fornecidas pelo usuário.

Vejamos a seguir exemplos de cada uma delas.

Como criar PROCEDURES SEM PARÂMETROS

Vejamos um exemplo de como criar uma PROCEDURE SEM PARÂMETROS:

Queremos criar uma PROCEDURE para cadastrar a nova venda abaixo:

Produto: iPhone 11 / Cliente: Bárbara Campos / Quantidade: 1

Para isso, seguiremos o seguinte passo a passo:

1º Passo: Vamos definir as variáveis a serem utilizadas;

2º Passo: Vamos atribuir valores às variáveis que receberão o nome do produto, o nome do cliente e a quantidade vendida;

3º Passo: Armazenaremos o valor de id_produto de acordo com o produto vendido;

4º Passo: Armazenaremos o valor de id_cliente de acordo com o nome do cliente;

5º Passo: Vamos utilizar o INSERT INTO para inserir os dados na tabela VENDAS;

6º Passo: Atualizaremos a tabela PRODUTOS com o estoque final.

Como criar PROCEDURES SEM PARÂMETROS

Abaixo, temos o código completo de nossa PROCEDURE:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE registro_venda
AS
    -- 1° Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vNomeProduto VARCHAR2(100);
    vIdCliente INT;
    vNomeCliente VARCHAR2(100);
    vQuantidadeVendida INT;

BEGIN
    -- 2° Passo: Atribuir valores às variáveis que receberão o nome do produto, o nome do cliente e a quantidade vendida
    vNomeProduto := 'iPhone 11';
    vNomeCliente := 'Bárbara Campos';
    vQuantidadeVendida := 1;

    -- 3° Passo: Armazenar o valor de id_produto de acordo com o produto vendido
    SELECT id_produto
    INTO vIdProduto
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 4° Passo: Armazenar o valor de id_cliente de acordo com o nome do cliente
    SELECT id_cliente
    INTO vIdCliente
    FROM clientes
    WHERE nome_cliente = vNomeCliente;

    -- 5° Passo: Utilizar o INSERT INTO para inserir os dados na tabela vendas
    INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
        (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

    -- 6° Passo: Atualizar a tabela produtos com o estoque final
    UPDATE produtos
    SET estoque = (estoque - vQuantidadeVendida)
    WHERE id_produto = vIdProduto;

    COMMIT;

    dbms_output.put_line('Venda registrada com sucesso!');

END registro_venda;


EXEC registro_venda;
```

Fique tranquilo(a), nas próximas páginas veremos o passo a passo dele!

Como criar PROCEDURES SEM PARÂMETROS

1. Iniciamos o serviço de impressão;
2. Começamos com o comando CREATE OR REPLACE PROCEDURE e atribuímos um nome para nossa procedure (registro_venda);
3. Declaramos as variáveis, iniciando essa declaração com o comando AS;

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE PROCEDURE registro_venda 2
AS 3
    -- 1º Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vNomeProduto VARCHAR2(100);
    vIdCliente INT;
    vNomeCliente VARCHAR2(100);
    vQuantidadeVendida INT;
```

Como criar PROCEDURES SEM PARÂMETROS

4 Começamos a definir as instruções da procedure com o comando BEGIN;

5 Atribuímos os valores de nome do produto, nome do cliente e quantidade vendida às variáveis vNomeProduto, vNomeCliente e vQuantidadeVendida, respectivamente;

6 Armazenamos o valor de id_produto na variável vIdProduto de acordo com o nome do produto vendido;

7 Armazenamos o valor de id_cliente na variável vIdCliente de acordo com o nome do cliente que efetuou a compra;

```
BEGIN 4
    -- 2° Passo: Atribuir valores às variáveis que receberão
    -- o nome do produto, o nome do cliente e a quantidade vendida
    vNomeProduto := 'iPhone 11'; 5
    vNomeCliente := 'Bárbara Campos';
    vQuantidadeVendida := 1;

    -- 3° Passo: Armazenar o valor de id_produto de acordo com o produto vendido
    SELECT id_produto 6
    INTO vIdProduto
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 4° Passo: Armazenar o valor de id_cliente de acordo com o nome do cliente
    SELECT id_cliente 7
    INTO vIdCliente
    FROM clientes
    WHERE nome_cliente = vNomeCliente;
```

Como criar PROCEDURES SEM PARÂMETROS

8 Inserimos na tabela VENDAS o registro da nova venda efetuada;

9 Atualizamos na tabela PRODUTOS a quantidade disponível em estoque do produto vendido;

10 Efetuamos um COMMIT para salvar as inserções/alterações realizadas;

11 Imprimimos na tela a informação “Venda registrada com sucesso!”;

12 Finalizamos as instruções da PROCEDURE com o comando END;

13 Executamos a PROCEDURE registro_venda.

```
    -- 5° Passo: Utilizar o INSERT INTO para inserir os dados na tabela vendas
    INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES 8
        (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

    -- 6° Passo: Atualizar a tabela produtos com o estoque final
    UPDATE produtos 9
    SET estoque = (estoque - vQuantidadeVendida)
    WHERE id_produto = vIdProduto;

    COMMIT; 10

    dbms_output.put_line('Venda registrada com sucesso!'); 11

END registro_venda; 12

EXEC registro_venda; 13
```

```
Saída do Script
Tarefa concluída em 0,07 segundos

Procedure REGISTRO_VENDA compilado

Venda registrada com sucesso!


Procedimento PL/SQL concluído com sucesso.
```

Índice Geral

Índice deste Módulo

Como criar PROCEDURES SEM PARÂMETROS

Repare que na tela VENDAS o registro foi devidamente inserido na sequência (id_venda = 11):

Resultado da Consulta | Resultado da Consulta 1

SQL | Todas as Linhas Extraídas: 11 em 0,009 segundos

	ID_VE...	DATA_VENDA	ID_CLIENTE	ID_PRODUTO	QUANTIDADE
1	1	10/08/22	5	1	2
2	2	10/01/22	2	3	1
3	3	13/01/22	1	7	1
4	4	02/02/22	7	1	1
5	5	17/02/22	4	1	3
6	6	21/02/22	6	5	3
7	7	05/03/22	7	5	1
8	8	07/03/22	10	2	1
9	9	09/03/22	9	1	1
10	10	12/03/22	4	3	3
11	11	23/08/22	2	1	1

Assim como a quantidade em estoque do produto vendido (iPhone 11) também foi atualizada: foi vendida uma unidade, passando seu estoque de 100 para 99 unidades:

Resultado da Consulta | Resultado da Consulta 1

SQL | Todas as Linhas Extraídas: 8 em 0,002 segundos

	ID_PRODUTO	NOME_PRODUTO	MARCA	CATEGORIA	PRECO_UNIT	CUSTO_UNIT	ESTOQUE
1	1	iPhone 11	Apple	Celular	3500	2200	99
2	2	iPhone 12	Apple	Celular	4100	3000	100
3	3	Moto G60	Motorola	Celular	1600	800	100
4	4	Samsung S22	Samsung	Celular	5000	2800	100
5	5	Aspire 5	Acer	Notebook	3300	1800	100
6	6	Inspiron 15 3000	Dell	Notebook	2800	1100	100
7	7	Smart TV 4K	LG	Televisão	1800	700	100
8	8	Smart TV Crystal UHD 4K	Samsung	Televisão	3000	1200	100

Como criar PROCEDURES COM PARÂMETROS

Vejamos um exemplo de como criar uma PROCEDURE COM PARÂMETROS.

Para tanto, vamos adaptar o código utilizado no nosso exemplo anterior de PROCEDURE sem parâmetros.

Assim, desta vez, queremos criar uma PROCEDURE para cadastrar a nova venda abaixo, informada pelo usuário:

Produto: Samsung 22 / Cliente: Juliana Pires / Quantidade: 10

Para isso, seguiremos o seguinte passo a passo:

1º Passo: Vamos definir as variáveis a serem utilizadas;

2º Passo: Armazenaremos o valor de id_produto de acordo com o produto vendido;

3º Passo: Armazenaremos o valor de id_cliente de acordo com o nome do cliente;

4º Passo: Vamos utilizar o INSERT INTO para inserir os dados na tabela VENDAS;

5º Passo: Atualizaremos a tabela PRODUTOS com o estoque final.

Como criar PROCEDURES COM PARÂMETROS

Este é o código completo de nossa PROCEDURE COM PARÂMETROS:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE registro_venda(vNomeProduto VARCHAR2, vNomeCliente VARCHAR2, vQuantidadeVendida INT)
AS
    -- 1° Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vIdCliente INT;

BEGIN

    -- 2° Passo: Armazenar o valor de id_produto de acordo com o produto vendido
    SELECT id_produto
    INTO vIdProduto
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 3° Passo: Armazenar o valor de id_cliente de acordo com o nome do cliente
    SELECT id_cliente
    INTO vIdCliente
    FROM clientes
    WHERE nome_cliente = vNomeCliente;

    -- 4° Passo: Utilizar o INSERT INTO para inserir os dados na tabela vendas
    INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
        (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

    -- 5° Passo: Atualizar a tabela produtos com o estoque final
    UPDATE produtos
    SET estoque = (estoque - vQuantidadeVendida)
    WHERE id_produto = vIdProduto;

    COMMIT;

    dbms_output.put_line('Venda registrada com sucesso!');

END registro_venda;


EXEC registro_venda('Samsung S22', 'Juliana Pires', 10);
```

Como criar PROCEDURES COM PARÂMETROS


```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE registro_venda(vNomeProduto VARCHAR2, vNomeCliente VARCHAR2, vQuantidadeVendida INT)
AS
    -- 1° Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vIdCliente INT;

BEGIN

    -- 2° Passo: Armazenar o valor de id_produto de acordo com o produto vendido
    SELECT id_produto
    INTO vIdProduto
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 3° Passo: Armazenar o valor de id_cliente de acordo com o nome do cliente
    SELECT id_cliente
    INTO vIdCliente
    FROM clientes
    WHERE nome_cliente = vNomeCliente;

    -- 4° Passo: Utilizar o INSERT INTO para inserir os dados na tabela vendas
    INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
        (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

    -- 5° Passo: Atualizar a tabela produtos com o estoque final
    UPDATE produtos
    SET estoque = (estoque - vQuantidadeVendida)
    WHERE id_produto = vIdProduto;

    COMMIT;

    dbms_output.put_line('Venda registrada com sucesso!');

END registro_venda;


EXEC registro_venda('Samsung S22', 'Juliana Pires', 10);
```

Repare que, desta vez, informamos as variáveis vNomeProduto, vNomeCliente e vQuantidadeVendida na primeira linha de código, junto aos comandos CREATE OR REPLACE PROCEDURE (e não dentro do código – 1º passo, como fizemos com a PROCEDURE sem parâmetros):

Desta forma, os dados que deverão ser armazenados nestas variáveis poderão ser informados pelo usuário no momento da execução da PROCEDURE:

Índice Geral

Índice deste Módulo

Como criar PROCEDURES COM PARÂMETROS

Portanto:

1. Iniciamos o serviço de impressão;

2. Começamos com o comando CREATE OR REPLACE PROCEDURE, atribuímos um nome para nossa procedure (registro_venda) e definimos quais serão os parâmetros (variáveis) que deverão ser informados pelo usuário no momento da execução da PROCEDURE;

3. Iniciamos a declaração das demais variáveis com o comando AS;

4. Declaramos as demais variáveis (as que não servirão de parâmetros a serem informados pelo usuário).
Essas variáveis irão buscar internamente as informações nas tabelas (id_produto e id_cliente);

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE PROCEDURE registro_venda(vNomeProduto VARCHAR2, vNomeCliente VARCHAR2, vQuantidadeVendida INT) 2
AS 3
    -- 1° Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vIdCliente INT; 4
```

Como criar PROCEDURES COM PARÂMETROS

5 Começamos a definir as instruções da procedure com o comando BEGIN;

6 Armazenamos o valor de id_produto na variável vIdProduto de acordo com o nome do produto vendido;

7 Armazenamos o valor de id_cliente na variável vIdCliente de acordo com o nome do cliente que efetuou a compra;

```
BEGIN 5

    -- 2º Passo: Armazenar o valor de id_produto de acordo com o produto vendido
    SELECT id_produto 6
    INTO vIdProduto
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 3º Passo: Armazenar o valor de id_cliente de acordo com o nome do cliente
    SELECT id_cliente 7
    INTO vIdCliente
    FROM clientes
    WHERE nome_cliente = vNomeCliente;
```

Como criar PROCEDURES COM PARÂMETROS

8 Inserimos na tabela VENDAS o registro da nova venda efetuada;

9 Atualizamos na tabela PRODUTOS a quantidade disponível em estoque do produto vendido;

10 Efetuamos um COMMIT para salvar as inserções/alterações realizadas;

11 Imprimimos na tela a informação “Venda registrada com sucesso!”;

12 Finalizamos as instruções da PROCEDURE com o comando END;

13 Executamos a PROCEDURE registro_venda, informando os dados dos parâmetros vNomeProduto (Samsung S22), vNomeCliente (Juliana Pires) e vQuantidadeVendida (10).

```
    -- 4° Passo: Utilizar o INSERT INTO para inserir os dados na tabela vendas
    INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES 8
        (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

    -- 5° Passo: Atualizar a tabela produtos com o estoque final
    UPDATE produtos 9
    SET estoque = (estoque - vQuantidadeVendida)
    WHERE id_produto = vIdProduto;

    COMMIT; 10

    dbms_output.put_line('Venda registrada com sucesso!'); 11

END registro_venda; 12


EXEC registro_venda('Samsung S22', 'Juliana Pires', 10); 13
```

```
Saída do Script x
Tarefa concluída em 0,118 segundos

Procedure REGISTRO_VENDA compilado

Venda registrada com sucesso!


Procedimento PL/SQL concluído com sucesso.
```

Índice Geral

Índice deste Módulo

Como criar PROCEDURES COM PARÂMETROS

Repare que na tela VENDAS o registro foi devidamente inserido na sequência (id_venda = 12):

Resultado da Consulta | Resultado da Consulta 1

SQL | Todas as Linhas Extraídas: 12 em 0,009 segundos

	ID_VE...	DATA_VENDA	ID_CLIENTE	ID_PRODUTO	QUANTIDADE
1	1	10/08/22	5	1	2
2	2	10/01/22	2	3	1
3	3	13/01/22	1	7	1
4	4	02/02/22	7	1	1
5	5	17/02/22	4	1	3
6	6	21/02/22	6	5	3
7	7	05/03/22	7	5	1
8	8	07/03/22	10	2	1
9	9	09/03/22	9	1	1
10	10	12/03/22	4	3	3
11	11	23/08/22	2	1	1
12	12	23/08/22	10	4	10

E a quantidade em estoque do produto vendido (Samsung S22) também foi atualizada: foram vendidas 10 unidades, passando seu estoque de 100 para 90 unidades:

Resultado da Consulta | Resultado da Consulta 1

SQL | Todas as Linhas Extraídas: 8 em 0,004 segundos

	ID_PRODUTO	NOME_PRODUTO	MARCA	CATEGORIA	PRECO_UNIT	CUSTO_UNIT	ESTOQUE
1	1	iPhone 11	Apple	Celular	3500	2200	99
2	2	iPhone 12	Apple	Celular	4100	3000	100
3	3	Moto G60	Motorola	Celular	1600	800	100
4	4	Samsung S22	Samsung	Celular	5000	2800	90
5	5	Aspire 5	Acer	Notebook	3300	1800	100
6	6	Inspiron 15 3000	Dell	Notebook	2800	1100	100
7	7	Smart TV 4K	LG	Televisão	1800	700	100
8	8	Smart TV Crystal UHD 4K	Samsung	Televisão	3000	1200	100

PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

Vimos na página anterior que agora o nosso estoque do produto Samsung S22 está com 90 UNIDADES disponíveis para venda.

Se alguém tentar efetuar uma compra de 100 unidades deste produto, essa venda não deveria ocorrer por falta de estoque, concorda?

Porém, da forma que a nossa PROCEDURE está configurada, não existe nada que impeça essa venda sem estoque disponível de acontecer.

Portanto, para que possamos sanar esta questão, precisamos acrescentar uma ESTRUTURA DE CONTROLE ao código da nossa PROCEDURE para que, antes de efetuar a venda, seja verificado o seguinte:

Há estoque suficiente para esta venda?

Sim: informe que a venda foi efetuada e registre no banco de dados.

Não: informe que não há estoque suficiente e não registre a venda no banco de dados.

PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

Assim, vamos acrescentar um IF ao nosso código, que vai ficar assim:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE registro_venda(vNomeProduto VARCHAR2, vNomeCliente VARCHAR2, vQuantidadeVendida INT)
AS
    -- 1° Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vIdCliente INT;
    vEstoque INT;

BEGIN
    -- 2° Passo: Armazenar o valor do estoque na variável vEstoque
    -- de acordo com o produto vendido
    SELECT estoque
    INTO vEstoque
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 3° Passo: Iniciar o IF: Se a quantidade existente em estoque
    -- for maior que a quantidade vendida, execute:
    IF vEstoque >= vQuantidadeVendida THEN

        -- Armazenar o valor de id_produto de acordo com o produto vendido
        SELECT id_produto
        INTO vIdProduto
        FROM produtos
        WHERE nome_produto = vNomeproduto;

        -- Armazenar o valor de id_cliente de acordo com o nome do cliente
        SELECT id_cliente
        INTO vIdCliente
        FROM clientes
        WHERE nome_cliente = vNomeCliente;

        -- Utilizar o INSERT INTO para inserir os dados na tabela vendas
        INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
            (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

        -- Atualizar a tabela produtos com o estoque final
        UPDATE produtos
        SET estoque = (estoque - vQuantidadeVendida)
        WHERE id_produto = vIdProduto;

        COMMIT;
        dbms_output.put_line('Venda registrada com sucesso!');

    -- Caso contrário, informe:
    ELSE
        dbms_output.put_line('Venda não realizada por falta de estoque!');
    END IF;
END registro_venda;

EXEC registro_venda('Samsung S22', 'Juliana Pires', 100);
```

Índice Geral

Índice deste Módulo

PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

1. Iniciamos o serviço de impressão;

2. Começamos com o comando CREATE OR REPLACE PROCEDURE, atribuímos um nome para nossa procedure (registro_venda) e definimos quais serão os parâmetros (variáveis) que deverão ser informados pelo usuário no momento da execução da PROCEDURE;

3. Iniciamos a declaração das demais variáveis com o comando AS;

4. Declaramos as demais variáveis (as que não servirão de parâmetros a serem informados pelo usuário). Essas variáveis irão buscar internamente as informações nas tabelas (id_produto, id_cliente e estoque);

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE PROCEDURE registro_venda(vNomeProduto VARCHAR2, vNomeCliente VARCHAR2, vQuantidadeVendida INT) 2
AS 3
    -- 1° Passo: Definir as variáveis a serem utilizadas.
    vIdProduto INT;
    vIdCliente INT; 4
    vEstoque INT;
```

PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

5 Começamos a definir as instruções da procedure com o comando BEGIN;

6 Armazenamos o valor de estoque na variável vEstoque de acordo com o nome do produto vendido;

7 Iniciamos a estrutura de controle IF para verificar se a quantidade do produto em estoque é igual ou maior do que a quantidade vendida. Se sim (THEN):

8 Armazenamos o valor de id_produto na variável vIdProduto de acordo com o nome do produto vendido;

9 Armazenamos o valor de id_cliente na variável vIdCliente de acordo com o nome do cliente que efetuou a compra;

```
BEGIN 5
    -- 2° Passo: Armazenar o valor do estoque na variável vEstoque
    -- de acordo com o produto vendido
    SELECT estoque 6
    INTO vEstoque
    FROM produtos
    WHERE nome_produto = vNomeproduto;

    -- 3° Passo: Iniciar o IF: Se a quantidade existente em estoque
    -- for maior que a quantidade vendida, execute:
    IF vEstoque >= vQuantidadeVendida THEN 7

        -- Armazenar o valor de id_produto de acordo com o produto vendido
        SELECT id_produto 8
        INTO vIdProduto
        FROM produtos
        WHERE nome_produto = vNomeproduto;

        -- Armazenar o valor de id_cliente de acordo com o nome do cliente
        SELECT id_cliente 9
        INTO vIdCliente
        FROM clientes
        WHERE nome_cliente = vNomeCliente;
```

PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

10 Inserimos na tabela VENDAS o registro da nova venda;

11 Atualizamos na tabela PRODUTOS a quantidade disponível em estoque do produto vendido;

12 Efetuamos um COMMIT para salvar as alterações realizadas;

13 Imprimimos “Venda registrada com sucesso!”;

14 Caso contrário (se a quantidade do produto em estoque for menor do que a quantidade vendida), imprimimos “Venda não realizada por falta de estoque!”;

15 Finalizamos a estrutura de controle IF;

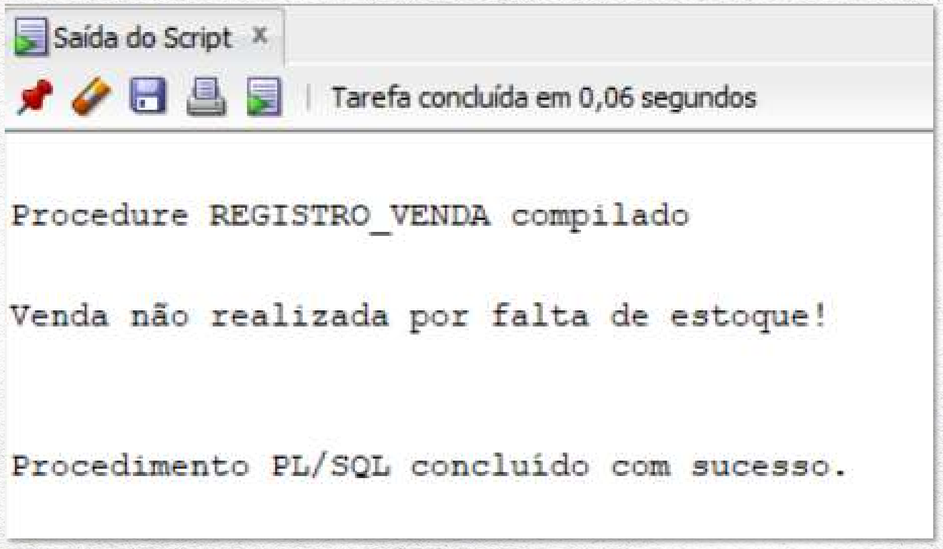
16 Finalizamos as instruções da PROCEDURE com o comando END;

17 Executamos a PROCEDURE, informando os parâmetros vNomeProduto (Samsung S22), vNomeCliente (Juliana Pires) e vQuantidadeVendida (100).

```
        -- Utilizar o INSERT INTO para inserir os dados na tabela vendas
        INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES 10
            (vendas_seq.NEXTVAL, sysdate, vIdCliente, vIdProduto, vQuantidadeVendida);

        -- Atualizar a tabela produtos com o estoque final
        UPDATE produtos 11
        SET estoque = (estoque - vQuantidadeVendida)
        WHERE id_produto = vIdProduto;

        COMMIT; 12

        dbms_output.put_line('Venda registrada com sucesso!'); 13

    -- Caso contrário, informe:
    ELSE 14
        dbms_output.put_line('Venda não realizada por falta de estoque!');
    END IF; 15

END registro_venda; 16

EXEC registro_venda('Samsung S22', 'Juliana Pires', 100); 17
```

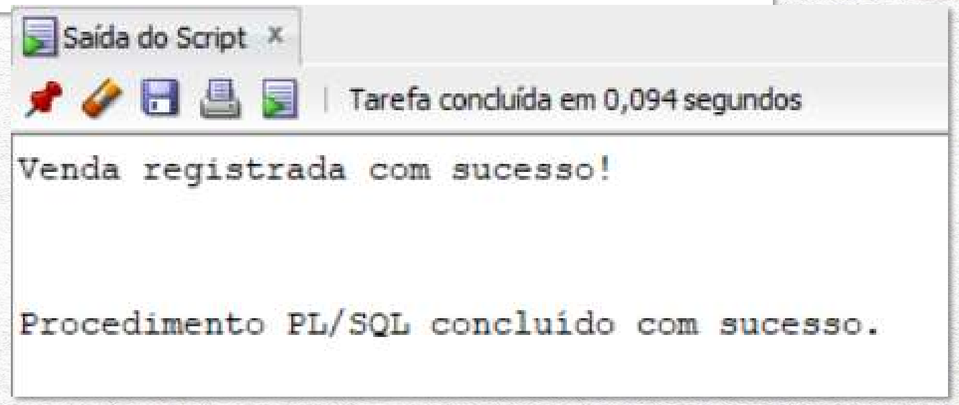
PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

Repare que, como só temos 90 unidades do produto em estoque e tentamos efetuar uma venda de 100 unidades, a PROCEDURE, agora com uma estrutura de controle IF inserida, foi capaz de evitar a efetivação dessa venda por falta de estoque:



Se tentarmos efetuar uma venda dentro dos limites do estoque, por exemplo, de 20 unidades, conforme abaixo, a PROCEDURE a registrará normalmente:

```
EXEC registro_venda('Samsung S22', 'Juliana Pires', 20);
```



PROCEDURES COM PARÂMETROS e ESTRUTURA DE CONTROLE

Repare que na tela VENDAS o registro foi devidamente inserido na sequência (id_venda = 13):

Resultado da Consulta | Resultado da Consulta 1

SQL | Todas as Linhas Extraídas: 13 em 0,007 segundos

	ID_VENDA	DATA_VENDA	ID_CLIENTE	ID_PRODUTO	QUANTIDADE
1	11	23/08/22	2	1	1
2	1	10/08/22	5	1	2
3	2	10/01/22	2	3	1
4	3	13/01/22	1	7	1
5	4	02/02/22	7	1	1
6	5	17/02/22	4	1	3
7	6	21/02/22	6	5	3
8	7	05/03/22	7	5	1
9	8	07/03/22	10	2	1
10	9	09/03/22	9	1	1
11	10	12/03/22	4	3	3
12	12	23/08/22	10	4	10
13	13	24/08/22	10	4	20

Assim como a quantidade em estoque do produto vendido (Samsung S22) também foi atualizada: foram vendidas 20 unidades, passando seu estoque de 90 para 70 unidades:

Resultado da Consulta | Resultado da Consulta 1

SQL | Todas as Linhas Extraídas: 8 em 0,002 segundos

	ID_PRODUTO	NOME_PRODUTO	MARCA	CATEGORIA	PRECO_UNIT	CUSTO_UNIT	ESTOQUE
1	1	iPhone 11	Apple	Celular	3500	2200	99
2	2	iPhone 12	Apple	Celular	4100	3000	100
3	3	Moto G60	Motorola	Celular	1600	800	100
4	4	Samsung S22	Samsung	Celular	5000	2800	70
5	5	Aspire 5	Acer	Notebook	3300	1800	100
6	6	Inspiron 15 3000	Dell	Notebook	2800	1100	100
7	7	Smart TV 4K	LG	Televisão	1800	700	100
8	8	Smart TV Crystal UHD 4K	Samsung	Televisão	3000	1200	100

Como excluir PROCEDURES

Para excluir uma PROCEDURE, basta utilizarmos o comando:

DROP PROCEDURE nome_procedure;

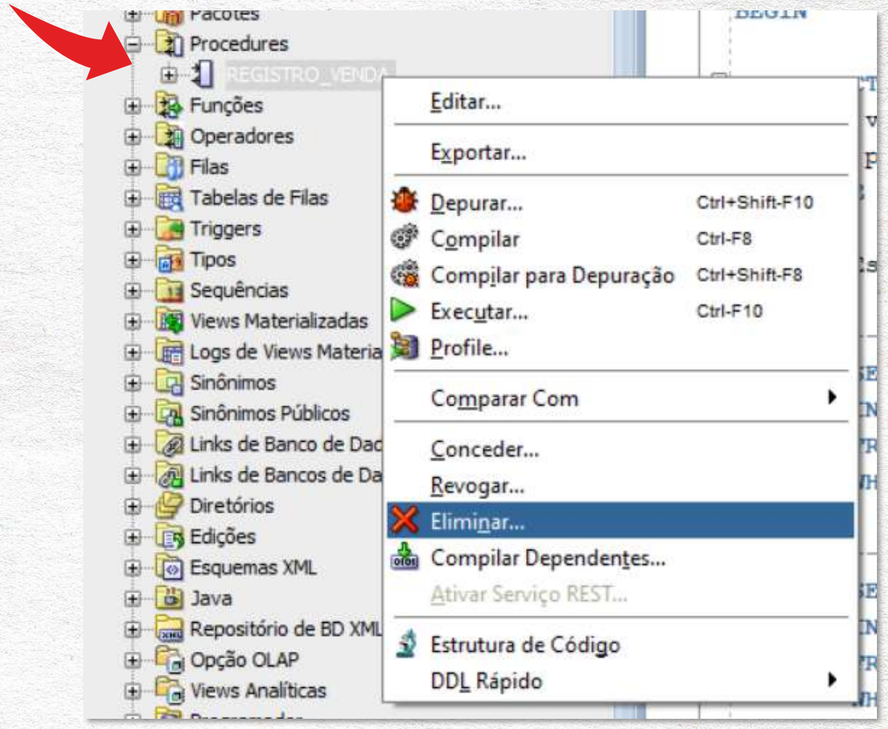
```
DROP PROCEDURE registro_venda;
```

Saída do Script

Tarefa concluída em 0,331 segundos

```
Procedure REGISTRO_VENDA eliminado.
```

Ou, se preferirmos, podemos também clicar com o botão direito sobre o nome da PROCEDURE e selecionar Eliminar:



Índice Geral

Índice deste Módulo

FUNCTIONS vs. PROCEDURES

Muita gente se pergunta quais são as diferenças entre FUNCTIONS e PROCEDURES.

Abaixo, listamos as principais delas:

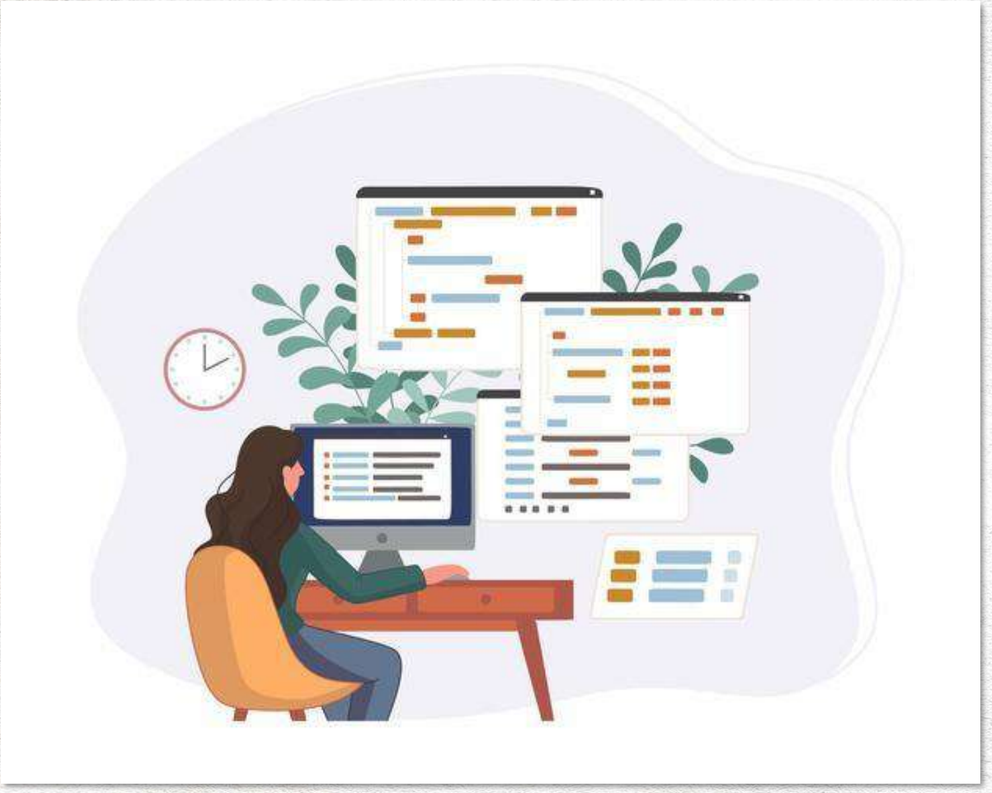
DIFERENÇAS	PROCEDURES	FUNCTIONS
1	São usadas para executar um processo, uma sequência de comandos e blocos SQL.	São usadas para fazer cálculos.
2	Não podem ser “chamadas” dentro da cláusula SELECT.	Podem ser “chamadas” dentro da cláusula SELECT (desde que não contenham comandos SELECT).
3	Não precisam retornar valor.	Precisam retornar algum valor.

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. PROCEDURES são blocos de códigos que executam uma série de ações dentro de um banco de dados.
2. Podemos usar comandos DDL, DML e TCL ou estruturas de controle para realizar as tarefas.
3. PROCEDURES podem ou não ter parâmetros de entrada.
4. PROCEDURES não necessariamente retornam algum valor.
5. Se você precisar de uma estrutura que retorne um valor como resultado, pense em usar uma FUNCTION.

EXERCÍCIOS



Para resolvermos os exercícios deste módulo, utilizaremos a tabela benefits (semelhante a que usamos no módulo de CRUD).

Primeiramente, execute o comando **DROP TABLE** abaixo. Assim, caso a tabela já exista, ela será excluída (e se não existir, aparecerá a informação que a tabela não existe):

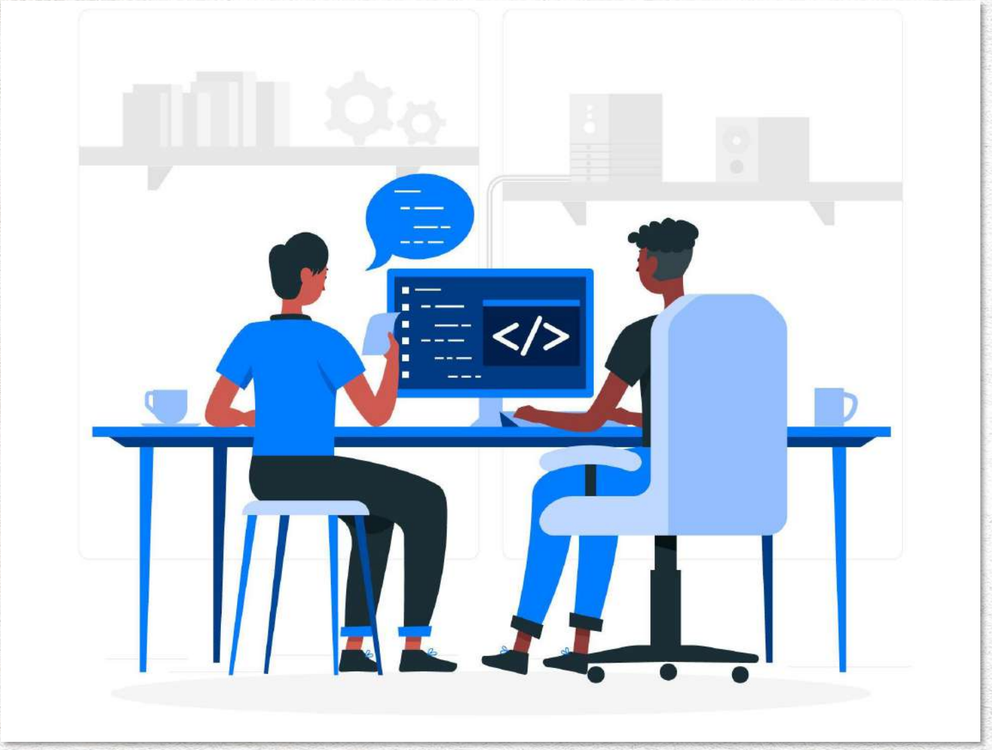
```
DROP TABLE benefits;
```

Após, crie a tabela benefits executando o **CREATE TABLE** abaixo:

```
CREATE TABLE benefits (
  benefit_id INT,
  employee_id INT,
  benefit_type VARCHAR2(30),
  benefit_description VARCHAR2(50),
  start_date DATE,
  end_date DATE,
  monthly_value NUMBER(10, 2)
);
```

CONTINUA NA PRÓXIMA PÁGINA..

EXERCÍCIOS



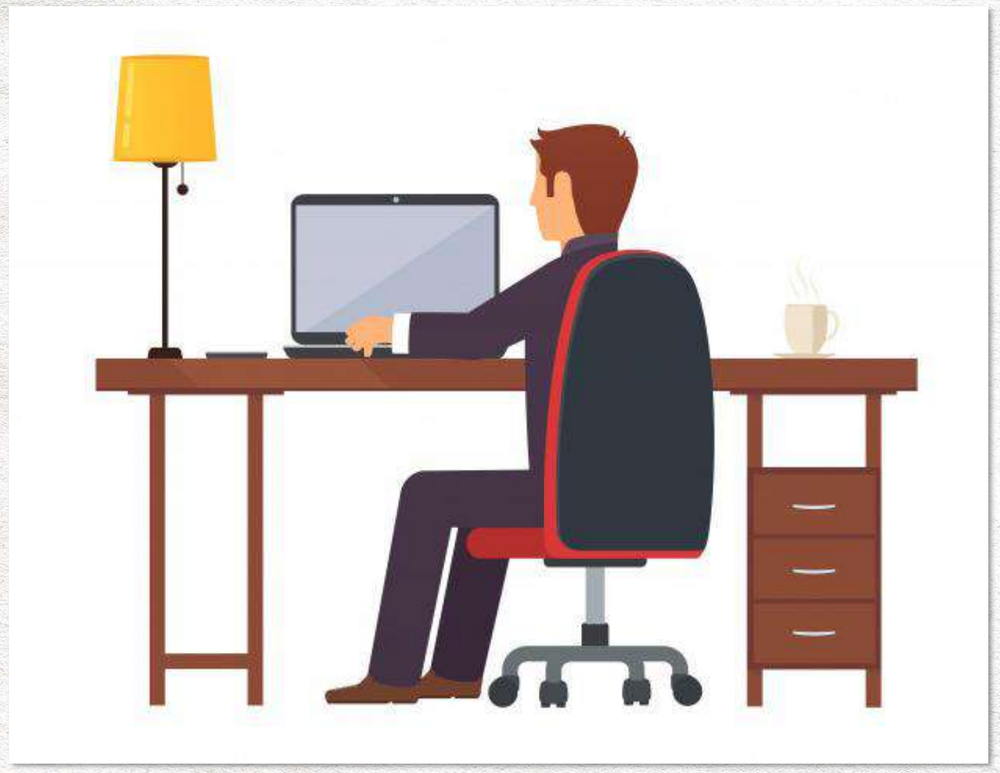
Para resolvermos os exercícios deste módulo, utilizaremos a tabela benefits (semelhante a que usamos no módulo de CRUD).

Por fim, insira suas **10 linhas** com os **INSERT INTO** abaixo:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (1, 145, 'Plano de Saúde', 'Plano Ouro da Hashmed', '01/01/1997', NULL, 850.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (2, 145, 'Vale Refeição', 'Cartão Hashcard', '01/01/1997', NULL, 450.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (3, 146, 'Vale Refeição', 'Cartão Hashcard', '05/03/1997', NULL, 450.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (4, 147, 'Vale Transporte', 'Bilhete Único da Hashtag', '10/05/1997', NULL, 180.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (5, 148, 'Auxílio Educação', 'Bolsa de Especialização Impressionadora', '15/12/1999', NULL, 800.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (6, 148, 'Gympass', 'Plano HashGold', '17/10/2001', '16/10/2002', 120.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (7, 149, 'Gympass', 'Plano HashGold', '29/03/2000', NULL, 120.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (8, 150, 'Plano de Saúde', 'Plano Silver da Hashmed', '01/07/1998', '30/06/2003', 550.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (9, 151, 'Auxílio Educação', 'Bolsa MBA Hashstrategic', '10/08/2004', '10/08/2005', 900.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (10, 151, 'Plano de Saúde', 'Plano Diamante da Hashmed', '20/04/2002', NULL, 1500.00);
```

CONTINUA NA PRÓXIMA PÁGINA...

EXERCÍCIOS

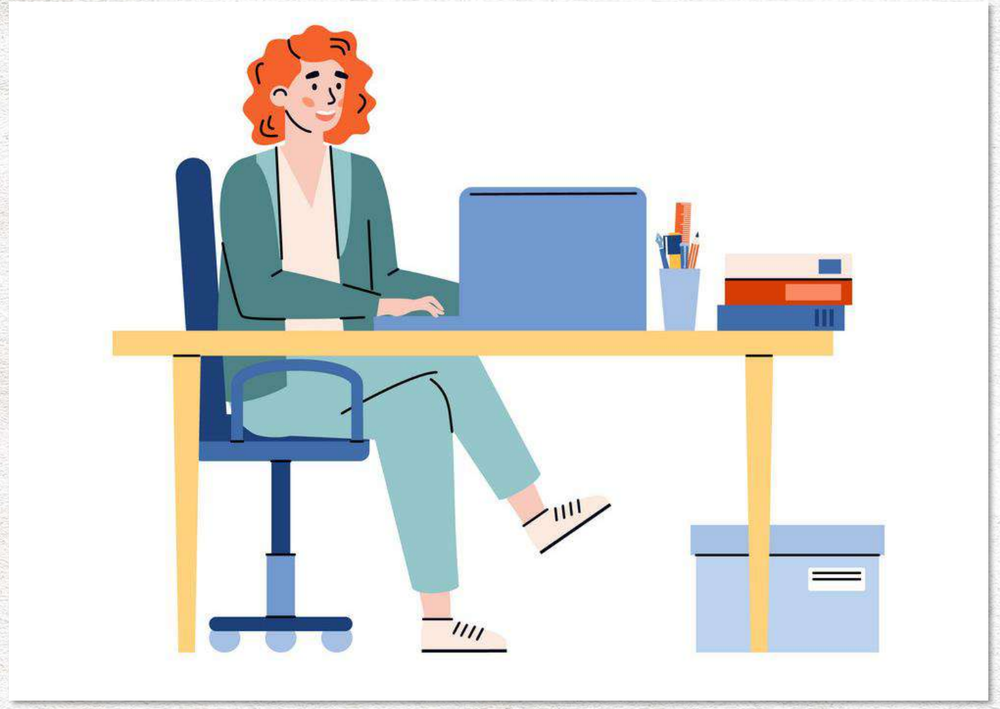


Para resolvermos os exercícios deste módulo, utilizaremos a tabela benefits (semelhante a que usamos no módulo de CRUD).

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

Agora, aos exercícios!

EXERCÍCIOS



1

Questão 1

O time de RH solicitou a criação de uma procedure simples que mostre uma mensagem de boas-vindas ao acessar o sistema de gerenciamento de benefícios.

Crie uma procedure chamada mensagem_boas_vindas que imprima a mensagem 'Bem-vindo(a) ao sistema de gestão de benefícios!' na tela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

2

Questão 2

Um gestor precisa verificar rapidamente o valor mensal de determinado benefício fornecido a um colaborador.

Crie uma procedure chamada consultar_valor_beneficio que receba o ID do benefício como parâmetro e exiba o valor mensal correspondente.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

EXERCÍCIOS

3

Questão 3

O setor financeiro quer automatizar a verificação de quais benefícios ainda estão ativos (sem data de término).

Crie uma procedure chamada verificar_beneficio_ativo que receba o ID do benefício e informe se ele está ou não ativo.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

4

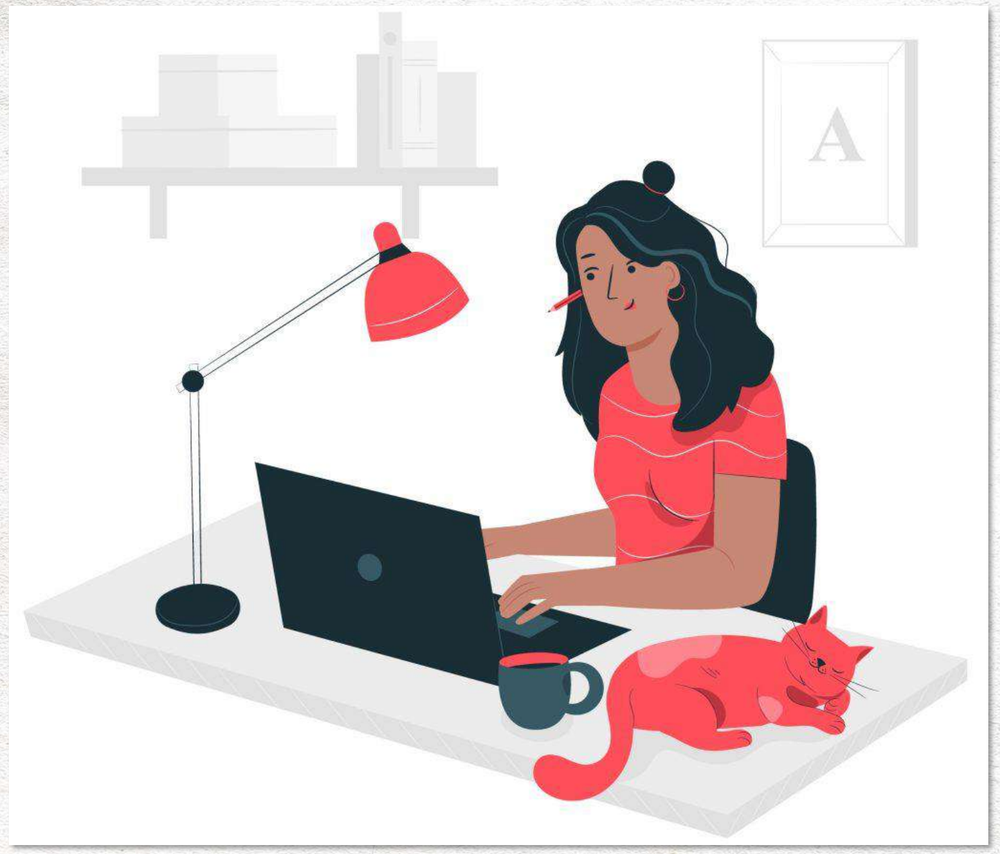
Questão 4

A empresa vai reajustar o valor mensal de um benefício.

Crie uma procedure chamada atualizar_valor_beneficio que receba o ID do benefício e o novo valor, e atualize esse valor na tabela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

EXERCÍCIOS



5

Questão 5

A equipe de RH quer saber quantos benefícios ainda estão ativos (sem data de encerramento) para apresentar em uma reunião de desempenho.

Crie uma procedure chamada contar_beneficios_ativos que exiba o total de benefícios ativos no momento.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

EXERCÍCIOS

7

Questão 7

A área de planejamento financeiro aprovou um reajuste para todos os benefícios do tipo "Plano de Saúde".

Crie uma procedure chamada reajustar_plano_saude que aumente o valor mensal desses benefícios em um valor percentual informado como parâmetro.

Considere apenas os benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

8

Questão 8

Ao sair da empresa, o colaborador precisa ter todos os benefícios encerrados.

Crie uma procedure chamada encerrar_beneficios_funcionario que receba o employee_id e defina a data atual (SYSDATE) como data de término para todos os seus benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

EXERCÍCIOS

9

Questão 9

O RH deseja exibir a descrição detalhada de um benefício pelo seu ID.

Crie uma procedure chamada exibir_descricao_beneficio que receba o benefit_id e retorne a descrição armazenada na coluna benefit_description.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

10

Questão 10

Exclua todas as procedures criadas nos exercícios anteriores.

GABARITO EXPLICATIVO

1 Questão 1

O time de RH solicitou a criação de uma procedure simples que mostre uma mensagem de boas-vindas ao acessar o sistema de gerenciamento de benefícios.

Crie uma procedure chamada mensagem_boas_vindas que imprima a mensagem 'Bem-vindo(a) ao sistema de gestão de benefícios!' na tela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

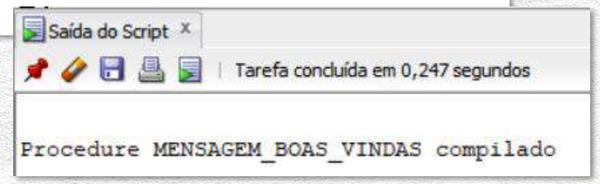
Objetivo:

Criar uma procedure chamada **mensagem_boas_vindas** que exibe uma mensagem padrão ao usuário, usada para dar boas-vindas ao acessar o sistema de gestão de benefícios da empresa.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE mensagem_boas_vindas
AS
BEGIN
  dbms_output.put_line('Bem-vindo(a) ao sistema de gestão de benefícios!');
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O time de RH solicitou a criação de uma procedure simples que mostre uma mensagem de boas-vindas ao acessar o sistema de gerenciamento de benefícios.

Crie uma procedure chamada mensagem_boas_vindas que imprima a mensagem 'Bem-vindo(a) ao sistema de gestão de benefícios!' na tela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE mensagem_boas_vindas:** Inicia a criação de uma procedure chamada mensagem_boas_vindas. A cláusula OR REPLACE permite substituir a procedure se ela já existir.
- ✓ **AS:** Palavra-chave que inicia a definição da procedure.
- ✓ **BEGIN ... END;**: Bloco que delimita o corpo da procedure, ou seja, onde o código será executado.
- ✓ **dbms_output.put_line(...)**: Utiliza a funcionalidade dbms_output para imprimir a mensagem desejada no console SQL. Neste caso, exibe: "Bem-vindo(a) ao sistema de gestão de benefícios!"

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O time de RH solicitou a criação de uma procedure simples que mostre uma mensagem de boas-vindas ao acessar o sistema de gerenciamento de benefícios.

Crie uma procedure chamada mensagem_boas_vindas que imprima a mensagem 'Bem-vindo(a) ao sistema de gestão de benefícios!' na tela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC mensagem_boas_vindas;
```

Explicação do teste:

- ✓ A instrução **EXEC** executa a procedure criada.
- ✓ Antes de executá-la, é necessário ativar a exibição de mensagens com o comando **SET SERVEROUTPUT ON**.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

O time de RH solicitou a criação de uma procedure simples que mostre uma mensagem de boas-vindas ao acessar o sistema de gerenciamento de benefícios.

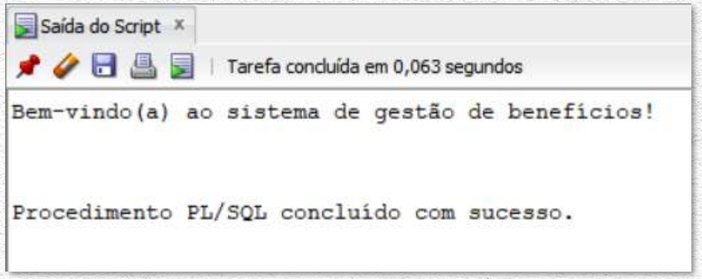
Crie uma procedure chamada mensagem_boas_vindas que imprima a mensagem 'Bem-vindo(a) ao sistema de gestão de benefícios!' na tela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja o resultado do nosso teste:

Ao executar a procedure, a mensagem "Bem-vindo(a) ao sistema de gestão de benefícios!" é impressa no console. Isso confirma que a procedure funciona corretamente.



Este exercício mostra como encapsular uma mensagem fixa dentro de uma procedure, permitindo reaproveitamento do código sempre que necessário.

Esse tipo de procedure é especialmente útil para:

- ✓ Automatizar mensagens institucionais no sistema;
- ✓ Melhorar a comunicação com o usuário;
- ✓ Centralizar textos-padrão que podem ser exibidos em múltiplos pontos do sistema.

GABARITO EXPLICATIVO

2 Questão 2

Um gestor precisa verificar rapidamente o valor mensal de determinado benefício fornecido a um colaborador.

Crie uma procedure chamada consultar_valor_beneficio que receba o ID do benefício como parâmetro e exiba o valor mensal correspondente.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

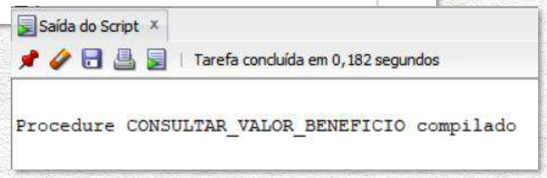
Objetivo:

Criar uma procedure chamada **consultar_valor_beneficio** que receba como parâmetro o ID de um benefício e exiba o valor mensal correspondente.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE consultar_valor_beneficio(vIdBeneficio INT)
AS
  vValorBeneficio NUMBER(10,2);
BEGIN
  SELECT monthly_value
  INTO vValorBeneficio
  FROM benefits
  WHERE benefit_id = vIdBeneficio;

  dbms_output.put_line('Valor mensal: ' || TO_CHAR(vValorBeneficio, 'L999G999D99'));
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

Um gestor precisa verificar rapidamente o valor mensal de determinado benefício fornecido a um colaborador.

Crie uma procedure chamada consultar_valor_beneficio que receba o ID do benefício como parâmetro e exiba o valor mensal correspondente.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE consultar_valor_beneficio(vIdBeneficio INT)**: Cria ou substitui uma procedure chamada consultar_valor_beneficio, que recebe como parâmetro de entrada um número inteiro (vIdBeneficio), representando o ID do benefício.
- ✓ **AS:** Inicia a definição da procedure.
- ✓ **vValorBeneficio NUMBER(10,2);**: Declara uma variável local chamada vValorBeneficio, com até 10 dígitos e 2 casas decimais. Ela armazenará o valor do benefício.
- ✓ **SELECT monthly_value INTO vValorBeneficio FROM benefits WHERE benefit_id = vIdBeneficio;**: Realiza uma consulta na tabela benefits, buscando o valor mensal (monthly_value) do benefício cujo ID corresponde ao informado. O resultado é armazenado na variável vValorBeneficio com o uso do comando INTO.
- ✓ **dbms_output.put_line('Valor mensal: ' || TO_CHAR(vValorBeneficio, 'L999G999D99'));**: Exibe o valor mensal formatado em moeda, com separador de milhar (, ou .) e símbolo de moeda (L, que adapta ao local definido no banco de dados).

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Um gestor precisa verificar rapidamente o valor mensal de determinado benefício fornecido a um colaborador.

Crie uma procedure chamada consultar_valor_beneficio que receba o ID do benefício como parâmetro e exiba o valor mensal correspondente.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC consultar_valor_beneficio(4);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Ativa a exibição de mensagens geradas por dbms_output.put_line.
- ✓ **EXEC consultar_valor_beneficio(4);**: Executa a procedure, passando, por exemplo, o ID 4 como parâmetro para buscar o valor mensal correspondente ao benefício de ID 4.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Um gestor precisa verificar rapidamente o valor mensal de determinado benefício fornecido a um colaborador.

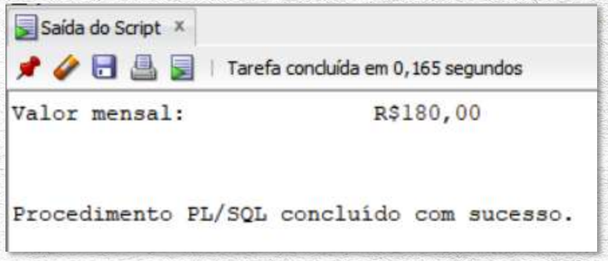
Crie uma procedure chamada consultar_valor_beneficio que receba o ID do benefício como parâmetro e exiba o valor mensal correspondente.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

Ao chamar a procedure, o sistema exibe o valor mensal do benefício com ID 4, formatado com símbolo de moeda e separadores adequados:



Se consultarmos a tabela benefits, veremos que o monthly_value do benefity_id = 4 é exatamente 180:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

Portanto, a procedure está funcionando corretamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

Um gestor precisa verificar rapidamente o valor mensal de determinado benefício fornecido a um colaborador.

Crie uma procedure chamada consultar_valor_beneficio que receba o ID do benefício como parâmetro e exiba o valor mensal correspondente.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure facilita consultas rápidas sobre valores de benefícios específicos, permitindo que gestores e analistas tenham acesso direto e formatado às informações financeiras necessárias.

Esse tipo de procedure é especialmente útil para:

- ✓ Automatizar consultas frequentes a dados específicos;
- ✓ Padronizar o formato de exibição;
- ✓ Agilizar o atendimento a demandas pontuais da gestão de pessoas.

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro quer automatizar a verificação de quais benefícios ainda estão ativos (sem data de término).

Crie uma procedure chamada verificar_beneficio_ativo que receba o ID do benefício e informe se ele está ou não ativo.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

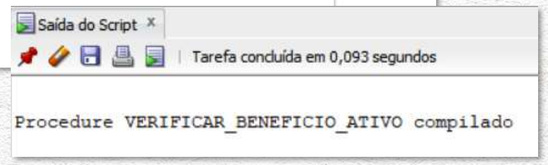
Objetivo:

Criar uma procedure chamada **verificar_beneficio_ativo** que receba o ID de um benefício e informe se ele ainda está ativo (sem data de término) ou se já foi encerrado, exibindo a data de fim, se houver.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE verificar_beneficio_ativo(vIdBeneficio INT)
AS
  vDataFim DATE;
BEGIN
  SELECT end_date
  INTO vDataFim
  FROM benefits
  WHERE benefit_id = vIdBeneficio;

  IF vDataFim IS NULL THEN
    dbms_output.put_line('Benefício ainda está ativo.');
  ELSE
    dbms_output.put_line('Benefício encerrado em: ' || TO_CHAR(vDataFim, 'DD/MM/YYYY'));
  END IF;
END;
```



Índice Geral
Índice deste Módulo
CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro quer automatizar a verificação de quais benefícios ainda estão ativos (sem data de término).

Crie uma procedure chamada verificar_beneficio_ativo que receba o ID do benefício e informe se ele está ou não ativo.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE verificar_beneficio_ativo(vIdBeneficio INT)**: Cria ou substitui uma procedure chamada verificar_beneficio_ativo, que recebe como parâmetro o ID de um benefício.
- ✓ **AS:** Inicia o bloco da procedure.
- ✓ **vDataFim DATE;**: Declara uma variável local chamada vDataFim, do tipo DATE, que será usada para armazenar a data de término do benefício.
- ✓ **SELECT end_date INTO vDataFim FROM benefits WHERE benefit_id = vIdBeneficio;**: Recupera da tabela benefits a coluna end_date (data de término) do benefício com o ID informado, armazenando o valor na variável vDataFim.
- ✓ **IF vDataFim IS NULL THEN ... ELSE ... END IF;**: Verifica se a data de término está nula:
  - Se NULL, significa que o benefício ainda está ativo, portanto exibe a informação 'Benefício ainda está ativo.' na tela.
  - Caso contrário, exibe a data de encerramento do benefício formatada como DD/MM/YYYY.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro quer automatizar a verificação de quais benefícios ainda estão ativos (sem data de término).

Crie uma procedure chamada verificar_beneficio_ativo que receba o ID do benefício e informe se ele está ou não ativo.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC verificar_beneficio_ativo(8);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON:** Ativa a exibição de mensagens geradas com dbms_output.put_line.
- ✓ **EXEC verificar_beneficio_ativo(8);**: Executa a procedure, passando, por exemplo, o ID 8 como parâmetro. O sistema buscará a data de término do benefício de ID 8 e mostrará a situação atual.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro quer automatizar a verificação de quais benefícios ainda estão ativos (sem data de término).

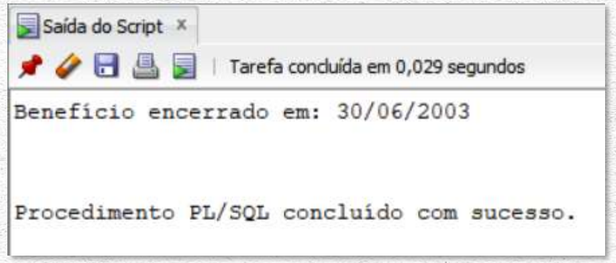
Crie uma procedure chamada verificar_beneficio_ativo que receba o ID do benefício e informe se ele está ou não ativo.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

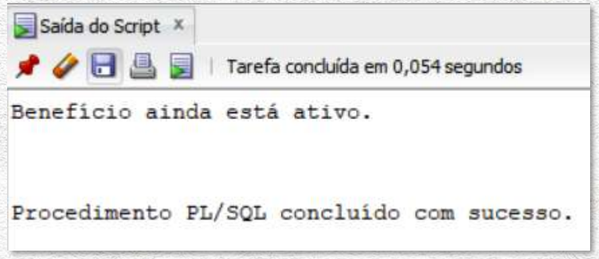
Veja o resultado do nosso teste:

Como o benefit_id = 8 já foi encerrado, ou seja, a coluna end_date não é nula, pois está preenchida com a data do término, aparece na tela a mensagem de benefício encerrado, mostrando a data em que isso ocorreu:



No entanto, quando executamos a procedure informando o ID de um benefício ainda ativo (por exemplo, 1), a mensagem retornada informa exatamente isso:

```
EXEC verificar_beneficio_ativo(1);
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro quer automatizar a verificação de quais benefícios ainda estão ativos (sem data de término).

Crie uma procedure chamada verificar_beneficio_ativo que receba o ID do benefício e informe se ele está ou não ativo.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure automatiza a checagem de status dos benefícios, eliminando a necessidade de fazer a verificação manual no banco.

Ela é especialmente útil para:

- ✓ Relatórios periódicos de controle de benefícios ativos;
- ✓ Alertas de renovação ou encerramento;
- ✓ Integração com sistemas de folha de pagamento ou RH.

GABARITO EXPLICATIVO

4

Questão 4

A empresa vai reajustar o valor mensal de um benefício.

Crie uma procedure chamada atualizar_valor_beneficio que receba o ID do benefício e o novo valor, e atualize esse valor na tabela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

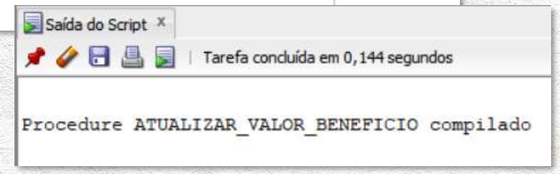
Objetivo:

Criar uma procedure chamada **atualizar_valor_beneficio** que receba o ID de um benefício e um novo valor mensal, atualizando esse valor na tabela benefits.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE atualizar_valor_beneficio(vIdBeneficio INT, vNovoValor NUMBER)
AS
BEGIN
  UPDATE benefits
  SET monthly_value = vNovoValor
  WHERE benefit_id = vIdBeneficio;

  COMMIT;

  dbms_output.put_line('Valor atualizado com sucesso!');
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A empresa vai reajustar o valor mensal de um benefício.

Crie uma procedure chamada atualizar_valor_beneficio que receba o ID do benefício e o novo valor, e atualize esse valor na tabela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE atualizar_valor_beneficio(vIdBeneficio INT, vNovoValor NUMBER)**: Cria ou substitui a procedure chamada atualizar_valor_beneficio. Ela recebe dois parâmetros:
  - **vIdBeneficio**: o ID do benefício a ser atualizado.
  - **vNovoValor**: o novo valor mensal a ser atribuído.
- ✓ **AS**: Inicia o bloco da procedure.
- ✓ **BEGIN ... END**: Delimita o corpo da procedure, onde serão definidos os comandos executáveis.
- ✓ **UPDATE benefits SET monthly_value = vNovoValor WHERE benefit_id = vIdBeneficio;**: Atualiza a coluna monthly_value com o valor informado em vNovoValor apenas para o registro cujo benefit_id seja igual ao valor de vIdBeneficio.
- ✓ **COMMIT;**: Confirma a transação, salvando permanentemente a alteração no banco de dados.
- ✓ **dbms_output.put_line('Valor atualizado com sucesso!');**: Exibe uma mensagem para indicar que o processo foi concluído corretamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A empresa vai reajustar o valor mensal de um benefício.

Crie uma procedure chamada atualizar_valor_beneficio que receba o ID do benefício e o novo valor, e atualize esse valor na tabela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC atualizar_valor_beneficio(7, 150);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Garante que a mensagem gerada com dbms_output.put_line será exibida na tela.
- ✓ **EXEC atualizar_valor_beneficio(7, 150);**: Executa a procedure, informando que o benefício com ID 7, por exemplo (**que atualmente é de R$ 120**), deverá ter seu valor mensal alterado para **R$ 150**.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4 Questão 4

A empresa vai reajustar o valor mensal de um benefício.

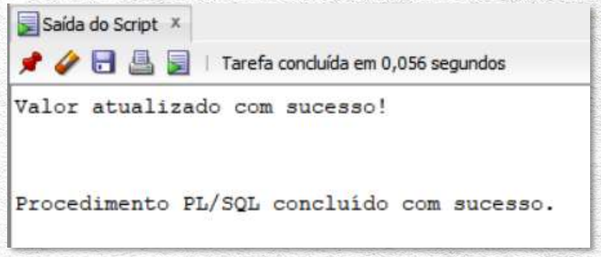
Crie uma procedure chamada atualizar_valor_beneficio que receba o ID do benefício e o novo valor, e atualize esse valor na tabela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

Ao executar a procedure com os parâmetros informados, será exibida a mensagem 'Valor atualizado com sucesso!':



A alteração pode ser confirmada com um SELECT na tabela benefits para verificar que o novo valor do benefício de ID 7 é R$ 150:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	150
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

Portanto, a procedure está funcionando corretamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

A empresa vai reajustar o valor mensal de um benefício.

Crie uma procedure chamada atualizar_valor_beneficio que receba o ID do benefício e o novo valor, e atualize esse valor na tabela.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure automatiza o processo de atualização dos valores mensais dos benefícios, garantindo mais agilidade e padronização no controle financeiro da empresa.

Ela é especialmente útil para:

- ✓ Aplicar reajustes periódicos nos valores de benefícios;
- ✓ Facilitar alterações massivas via scripts;
- ✓ Evitar erros manuais em atualizações feitas diretamente via comandos SQL.

GABARITO EXPLICATIVO

5

Questão 5

A equipe de RH quer saber quantos benefícios ainda estão ativos (sem data de encerramento) para apresentar em uma reunião de desempenho.

Crie uma procedure chamada contar_beneficios_ativos que exiba o total de benefícios ativos no momento.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

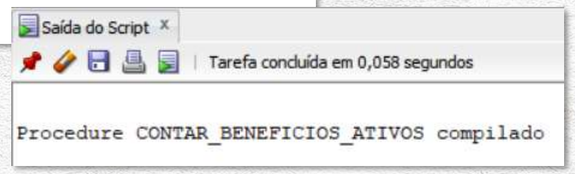
Objetivo:

Criar uma procedure chamada **contar_beneficios_ativos** que conte e exiba a quantidade total de benefícios que ainda estão ativos (ou seja, que não possuem uma data de encerramento registrada na tabela benefits).

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE contar_beneficios_ativos
AS
  vTotal INT;
BEGIN
  SELECT COUNT(*)
  INTO vTotal
  FROM benefits
  WHERE end_date IS NULL;

  dbms_output.put_line('Total de beneficios ativos: ' || vTotal);
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A equipe de RH quer saber quantos benefícios ainda estão ativos (sem data de encerramento) para apresentar em uma reunião de desempenho.

Crie uma procedure chamada contar_beneficios_ativos que exiba o total de benefícios ativos no momento.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE contar_beneficios_ativos**: Cria ou substitui a procedure chamada contar_beneficios_ativos, que não recebe parâmetros de entrada.
- ✓ **AS**: Inicia a definição do corpo da procedure.
- ✓ **vTotal INT;**: Declara uma variável local chamada vTotal, do tipo INT, que será usada para armazenar o total de benefícios ativos.
- ✓ **BEGIN ... END**: Delimita o bloco executável da procedure, onde fica a lógica principal.
- ✓ **SELECT COUNT(*) INTO vTotal FROM benefits WHERE end_date IS NULL;**: Realiza a contagem dos registros na tabela benefits que não possuem data de término (end_date IS NULL) e armazena o total na variável vTotal.
- ✓ **dbms_output.put_line('Total de benefícios ativos: ' || vTotal);**: Exibe o total calculado na tela com uma mensagem explicativa.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A equipe de RH quer saber quantos benefícios ainda estão ativos (sem data de encerramento) para apresentar em uma reunião de desempenho.

Crie uma procedure chamada contar_beneficios_ativos que exiba o total de benefícios ativos no momento.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC contar_beneficios_ativos;
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Ativa a exibição de mensagens geradas com dbms_output.put_line.
- ✓ **EXEC contar_beneficios_ativos;**: Executa a procedure, exibindo a quantidade total de benefícios que ainda estão ativos.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A equipe de RH quer saber quantos benefícios ainda estão ativos (sem data de encerramento) para apresentar em uma reunião de desempenho.

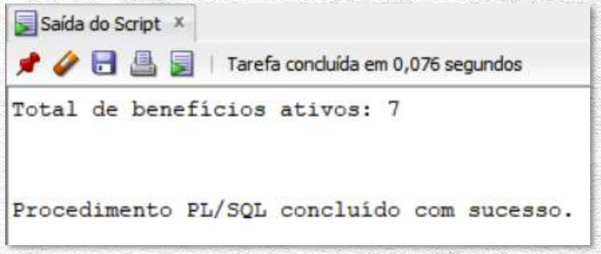
Crie uma procedure chamada contar_beneficios_ativos que exiba o total de benefícios ativos no momento.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

A execução da procedure retorna uma mensagem como a mostrada ao lado, com o total de benefícios atualmente ativos:



Se consultarmos a tabela benefits, veremos que, de fato, temos 7 registros em que a coluna end_date está nula, ou seja, cujos benefícios ainda não foram encerrados:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	150
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

Portanto, a procedure está funcionando corretamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A equipe de RH quer saber quantos benefícios ainda estão ativos (sem data de encerramento) para apresentar em uma reunião de desempenho.

Crie uma procedure chamada contar_beneficios_ativos que exiba o total de benefícios ativos no momento.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure fornece uma forma rápida e prática de visualizar a quantidade de benefícios em vigor, sem necessidade de executar consultas manuais.

Esse tipo de procedure é especialmente útil para:

- ✓ Relatórios gerenciais em reuniões de desempenho;
- ✓ Indicadores de uso de recursos da empresa;
- ✓ Monitoramento contínuo de benefícios ativos no sistema.

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

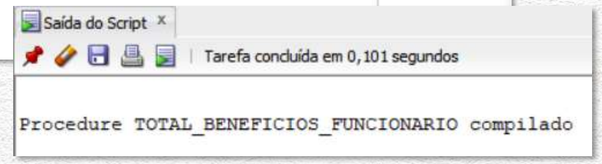
Objetivo:

Criar uma procedure chamada **total_beneficios_funcionario** que receba o employee_id de um funcionário e exiba o total mensal que ele recebe em benefícios ativos. Caso o funcionário não possua nenhum benefício ativo cadastrado, uma mensagem específica deve ser exibida.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE total_beneficios_funcionario(vIdFuncionario INT)
AS
  vTotal NUMBER(10,2);
BEGIN
  SELECT SUM(monthly_value)
  INTO vTotal
  FROM benefits
  WHERE employee_id = vIdFuncionario
  AND end_date IS NULL;

  IF vTotal IS NOT NULL THEN
    dbms_output.put_line('Total mensal de beneficios: ' || TO_CHAR(vTotal, 'L999G999D99'));
  ELSE
    dbms_output.put_line('Funcionário sem benefícios ativos cadastrados.');
  END IF;
END;
```



Índice Geral
Índice deste Módulo
CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE total_beneficios_funcionario(vIdFuncionario INT)**: Cria ou substitui a procedure chamada total_beneficios_funcionario, que recebe como parâmetro o ID do funcionário (vIdFuncionario).
- ✓ **AS**: Inicia a declaração do corpo da procedure.
- ✓ **vTotal NUMBER(10,2);**: Declara a variável local vTotal, que armazenará o valor total dos benefícios ativos recebidos pelo funcionário.
- ✓ **BEGIN ... END**: Delimita o bloco de execução da procedure, onde será implementada a lógica principal.
- ✓ **SELECT SUM(monthly_value) INTO vTotal FROM benefits WHERE employee_id = vIdFuncionario AND end_date IS NULL;**: Soma todos os valores mensais dos benefícios ativos (com end_date IS NULL) do funcionário informado e armazena o resultado em vTotal.
- ✓ **IF vTotal IS NOT NULL THEN dbms_output.put_line('Total mensal de benefícios: ' || TO_CHAR(vTotal, 'L999G999D99'));**: Verifica se o valor retornado pela soma não é nulo (ou seja, se há pelo menos um benefício ativo) e, se verdadeira a condição, exibe o total de benefícios formatado com símbolo de moeda.
- ✓ **ELSE dbms_output.put_line('Funcionário sem benefícios ativos cadastrados.');**: Caso a soma seja nula, exibe uma mensagem indicando que o funcionário não possui benefícios ativos.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC total_beneficios_funcionario(148);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Ativa a exibição de mensagens geradas por dbms_output.put_line.
- ✓ **EXEC total_beneficios_funcionario(148);**: Executa a procedure, passando o ID do funcionário como parâmetro (neste exemplo, 148).

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

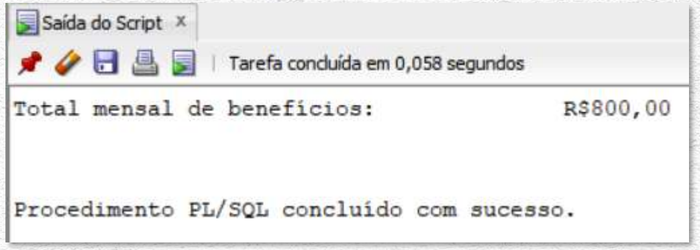
Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja o resultado do nosso teste:

Ao executar a procedure, a procedure retorna o valor R$ 800,00 para a somatória de benefícios ativos:



Isso está correto, pois se consultarmos a tabela benefits, veremos que temos dois benefícios cadastrados para o employee_id = 148: o Auxílio Educação no valor de R$ 800 e o Gympass de R$ 120. No entanto, apenas o benefício de R$ 800 está ativo, pois o de R$ 120 foi encerrado em 16/10/2002, não sendo considerado na somatória:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

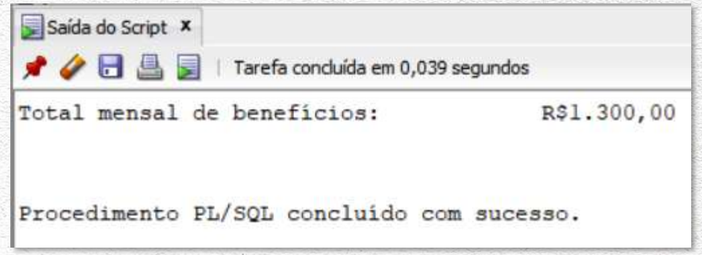
Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja o resultado do nosso teste:

Agora, se executarmos a procedure para o employee_id = 145, somaremos R$ 1.300,00:

```
EXEC total_beneficios_funcionario(145);
```



Isso porque este funcionário possui dois benefícios ativos: o Plano de Saúde de R$ 850 e o Vale Refeição de 450:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6 Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

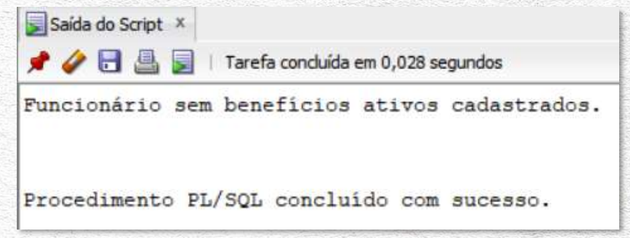
Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja o resultado do nosso teste:

Por fim, se executarmos a procedure para o employee_id = 152, retornaremos a mensagem 'Funcionário sem benefícios ativos cadastrados.':

```
EXEC total_beneficios_funcionario(152);
```



Consultando a tabela benefits, vemos que realmente não existem benefícios ativos cadastrados para o funcionário de ID 152:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	150
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

Portanto, com estes testes, verificamos que a procedure está funcionando corretamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

6

Questão 6

O RH deseja calcular quanto está sendo gasto mensalmente com benefícios de um funcionário específico.

Crie uma procedure chamada total_beneficios_funcionario que receba o employee_id e exiba o valor total mensal recebido em benefícios.

Considere apenas os benefícios ativos.

Quando for informado o employee_id de um funcionário que não recebe benefícios, apresente na tela a mensagem 'Funcionário sem benefícios ativos cadastrados.'

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure permite verificar, de forma prática, o valor mensal gasto com benefícios por funcionário, considerando apenas os benefícios vigentes.

Esse tipo de procedure é útil para:

- ✓ Relatórios financeiros e orçamentários individualizados;
- ✓ Apoio à tomada de decisão em promoções, bonificações e cortes;
- ✓ Automatização de sistemas de gestão de RH, com respostas rápidas por funcionário.

GABARITO EXPLICATIVO

7

Questão 7

A área de planejamento financeiro aprovou um reajuste para todos os benefícios do tipo "Plano de Saúde".

Crie uma procedure chamada reajustar_plano_saude que aumente o valor mensal desses benefícios em um valor percentual informado como parâmetro.

Considere apenas os benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Objetivo:

Criar uma procedure chamada **reajustar_plano_saude** que aplique um reajuste percentual (informado como parâmetro) sobre o valor mensal de todos os benefícios do tipo "Plano de Saúde", considerando apenas aqueles que ainda estão ativos.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE reajustar_plano_saude(vPercentualReajuste NUMBER)
AS
BEGIN
  UPDATE benefits
  SET monthly_value = monthly_value + (monthly_value * vPercentualReajuste / 100)
  WHERE benefit_type = 'Plano de Saúde'
  AND end_date IS NULL;

  COMMIT;

  dbms_output.put_line('Reajuste de ' || vPercentualReajuste || '% aplicado aos benefícios do tipo Plano de Saúde.');
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área de planejamento financeiro aprovou um reajuste para todos os benefícios do tipo "Plano de Saúde".

Crie uma procedure chamada reajustar_plano_saude que aumente o valor mensal desses benefícios em um valor percentual informado como parâmetro.

Considere apenas os benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE reajustar_plano_saude(vPercentualReajuste NUMBER)**: Cria ou substitui a procedure chamada reajustar_plano_saude, que recebe como parâmetro o percentual de reajuste a ser aplicado (vPercentualReajuste).
- ✓ **AS**: Inicia a declaração do corpo da procedure.
- ✓ **BEGIN ... END**: Delimita o bloco de execução da procedure, onde a lógica de reajuste é implementada.
- ✓ **UPDATE benefits SET monthly_value = monthly_value + (monthly_value * vPercentualReajuste / 100) WHERE benefit_type = 'Plano de Saúde' AND end_date IS NULL;**: Atualiza o valor mensal (monthly_value) dos benefícios, somando o percentual de reajuste ao valor atual. O filtro WHERE garante que o reajuste será aplicado apenas aos benefícios do tipo "Plano de Saúde" e que ainda estão ativos (end_date IS NULL).
- ✓ **COMMIT;**: Confirma as alterações de forma definitiva no banco de dados.
- ✓ **dbms_output.put_line(...)**: Exibe uma mensagem informando o percentual aplicado com sucesso.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área de planejamento financeiro aprovou um reajuste para todos os benefícios do tipo "Plano de Saúde".

Crie uma procedure chamada reajustar_plano_saude que aumente o valor mensal desses benefícios em um valor percentual informado como parâmetro.

Considere apenas os benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC reajustar_plano_saude(10);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Habilita a exibição de mensagens geradas por dbms_output.put_line.

- ✓ **EXEC reajustar_plano_saude(10);**: Executa a procedure, aplicando um reajuste de 10% (neste exemplo) aos benefícios ativos do tipo "Plano de Saúde".

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área de planejamento financeiro aprovou um reajuste para todos os benefícios do tipo "Plano de Saúde".

Crie uma procedure chamada reajustar_plano_saude que aumente o valor mensal desses benefícios em um valor percentual informado como parâmetro.

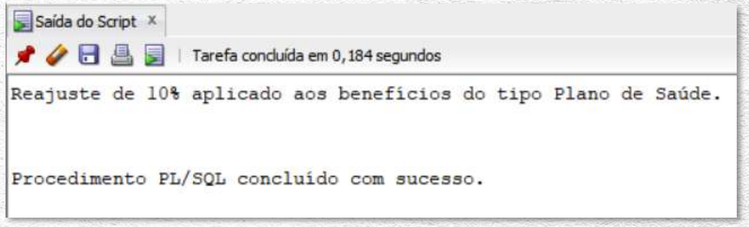
Considere apenas os benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja ao lado o resultado do nosso teste:

Ao executar a procedure, a mensagem "Reajuste de 10% aplicado aos benefícios do tipo Plano de Saúde." é impressa no console:



Consultando a tabela benefits, vemos que os benefícios de benefit_id = 1 e 10 realmente receberam um ajuste de 10%, pois antes seus monthly_value eram de R$ 850 e R$ 1500, e agora passaram a R$ 935 e R$ 1650, respectivamente.

Note que o benefit_id = 8 não sofreu o ajuste, pois está encerrado (e a procedure foi configurada para atualizar somente os benefícios ativos):

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	935
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	150
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1650

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

7

Questão 7

A área de planejamento financeiro aprovou um reajuste para todos os benefícios do tipo "Plano de Saúde".

Crie uma procedure chamada reajustar_plano_saude que aumente o valor mensal desses benefícios em um valor percentual informado como parâmetro.

Considere apenas os benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure facilita a atualização em massa de valores de benefícios específicos, garantindo agilidade e padronização nas ações do setor de planejamento financeiro.

Ela é especialmente útil para:

- ✓ Aplicação de reajustes anuais ou periódicos;
- ✓ Redução de erros manuais em atualizações múltiplas;
- ✓ Manutenção de consistência nos dados de benefícios corporativos.

GABARITO EXPLICATIVO

8

Questão 8

Ao sair da empresa, o colaborador precisa ter todos os benefícios encerrados.

Crie uma procedure chamada encerrar_beneficios_funcionario que receba o employee_id e defina a data atual (SYSDATE) como data de término para todos os seus benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

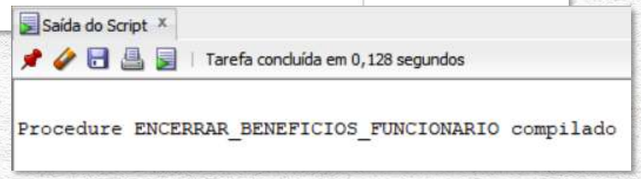
Objetivo:

Criar uma procedure **chamada encerrar_beneficios_funcionario** que receba o ID de um funcionário e atualize a data de término (end_date) de todos os seus benefícios ativos para a data atual (SYSDATE), encerrando-os.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE encerrar_beneficios_funcionario(vIdFuncionario INT)
AS
BEGIN
  UPDATE benefits
  SET end_date = SYSDATE
  WHERE employee_id = vIdFuncionario
  AND end_date IS NULL;

  COMMIT;

  dbms_output.put_line('Benefícios encerrados com sucesso para o funcionário ' || vIdFuncionario);
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Ao sair da empresa, o colaborador precisa ter todos os benefícios encerrados.

Crie uma procedure chamada encerrar_beneficios_funcionario que receba o employee_id e defina a data atual (SYSDATE) como data de término para todos os seus benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE encerrar_beneficios_funcionário (vIdFuncionario INT)**: Cria ou substitui uma procedure chamada encerrar_beneficios_funcionario que recebe um parâmetro vIdFuncionario, representando o ID do funcionário.
- ✓ **AS**: Inicia a definição da procedure.
- ✓ **BEGIN ... END**: Delimita o bloco onde está a lógica principal da procedure.
- ✓ **UPDATE benefits SET end_date = SYSDATE WHERE employee_id = vIdFuncionario AND end_date IS NULL;**: Atualiza o campo end_date para a data atual, encerrando o benefício. O filtro WHERE garante que apenas os benefícios ativos (sem data de término) do funcionário informado sejam atualizados.
- ✓ **COMMIT;**: Confirma a transação no banco de dados, salvando as alterações.
- ✓ **dbms_output.put_line(...)**: Exibe uma mensagem informando que os benefícios foram encerrados com sucesso para o funcionário especificado.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Ao sair da empresa, o colaborador precisa ter todos os benefícios encerrados.

Crie uma procedure chamada encerrar_beneficios_funcionario que receba o employee_id e defina a data atual (SYSDATE) como data de término para todos os seus benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC encerrar_beneficios_funcionario(149);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Habilita a visualização de mensagens de saída do dbms_output.put_line.
- ✓ **EXEC encerrar_beneficios_funcionario(149);**: Executa a procedure para o funcionário com employee_id 149 (que neste exemplo, foi hipoteticamente desligado da empresa), encerrando todos os seus benefícios ativos.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Ao sair da empresa, o colaborador precisa ter todos os benefícios encerrados.

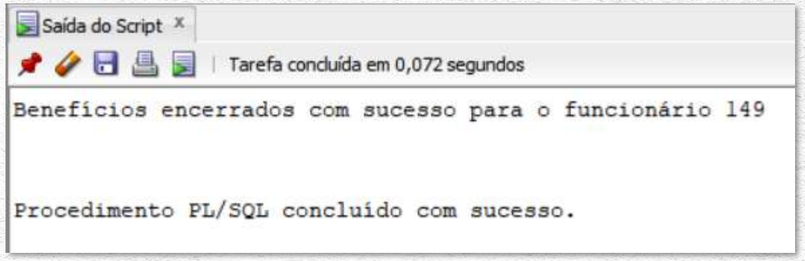
Crie uma procedure chamada encerrar_beneficios_funcionario que receba o employee_id e defina a data atual (SYSDATE) como data de término para todos os seus benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja o resultado do nosso teste:

Ao executar a procedure, a mensagem "Benefícios encerrados com sucesso para o funcionário 149" é impressa no console:



Consultando a tabela benefits, vemos que o benefício vinculado ao funcionário de ID 149 (benefit_id = 7), antes ativo, foi encerrado com a execução da procedure, armazenando na coluna monthly_value a data atual do sistema (que será diferente quando você executar o seu código em outro dia):

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
7	149	Gympass	Plano HashGold	29/03/00	19/06/25	150

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

8

Questão 8

Ao sair da empresa, o colaborador precisa ter todos os benefícios encerrados.

Crie uma procedure chamada encerrar_beneficios_funcionario que receba o employee_id e defina a data atual (SYSDATE) como data de término para todos os seus benefícios ativos.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure automatiza a tarefa crítica de encerramento de benefícios ao desligar um colaborador.

Esse tipo de procedure é útil para garantir:

- ✓ Que não haja benefícios indevidamente ativos após o desligamento;
- ✓ Padronização e agilidade no processo de desligamento;
- ✓ Redução de falhas manuais em processos de RH.

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja exibir a descrição detalhada de um benefício pelo seu ID.

Crie uma procedure chamada exibir_descricao_beneficio que receba o benefit_id e retorne a descrição armazenada na coluna benefit_description.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

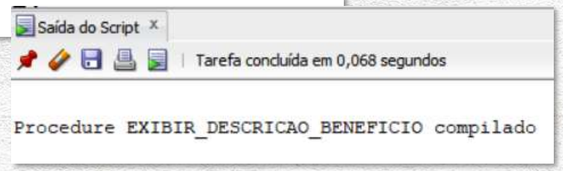
Objetivo:

Criar uma procedure chamada **exibir_descricao_beneficio** que receba o ID de um benefício (benefit_id) e exiba sua descrição completa, armazenada na coluna benefit_description.

Passo a passo:

1. Criação da procedure:

```
CREATE OR REPLACE PROCEDURE exibir_descricao_beneficio(vIdBeneficio INT)
AS
  vDescricaoBeneficio VARCHAR2(50);
BEGIN
  SELECT benefit_description
  INTO vDescricaoBeneficio
  FROM benefits
  WHERE benefit_id = vIdBeneficio;

  dbms_output.put_line('Descrição: ' || vDescricaoBeneficio);
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja exibir a descrição detalhada de um benefício pelo seu ID.

Crie uma procedure chamada exibir_descricao_beneficio que receba o benefit_id e retorne a descrição armazenada na coluna benefit_description.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Vamos analisar comando por comando:

- ✓ **CREATE OR REPLACE PROCEDURE exibir_descricao_beneficio(vIdBeneficio INT)**: Cria ou substitui uma procedure chamada exibir_descricao_beneficio, que recebe no parâmetro vIdBeneficio um número inteiro correspondente ao benefit_id.
- ✓ **AS**: Inicia a definição do corpo da procedure.
- ✓ **vDescricaoBeneficio VARCHAR2(50);**: Declara uma variável local que armazenará temporariamente a descrição do benefício consultado.
- ✓ **BEGIN ... END;**: Delimita o bloco de execução da procedure, contendo a lógica que será executada.
- ✓ **SELECT benefit_description INTO vDescricaoBeneficio FROM benefits WHERE benefit_id = vIdBeneficio;**: Realiza a consulta na tabela benefits, buscando a descrição do benefício com o ID fornecido, e armazena o resultado na variável vDescricaoBeneficio.
- ✓ **dbms_output.put_line('Descrição: ' || vDescricaoBeneficio);**: Exibe a descrição obtida no console, precedida do texto "Descrição: ".

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja exibir a descrição detalhada de um benefício pelo seu ID.

Crie uma procedure chamada exibir_descricao_beneficio que receba o benefit_id e retorne a descrição armazenada na coluna benefit_description.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

2. Teste da procedure:

```
SET SERVEROUTPUT ON;

EXEC exibir_descricao_beneficio(10);
```

Explicação do teste:

- ✓ **SET SERVEROUTPUT ON**: Habilita a exibição de mensagens no console usando dbms_output.put_line.

- ✓ **EXEC exibir_descricao_beneficio(10);**: Executa a procedure passando, neste exemplo, o número 10 como parâmetro, o que faz com que seja buscada a descrição (benefit_description) do benefício com benefit_id = 10.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja exibir a descrição detalhada de um benefício pelo seu ID.

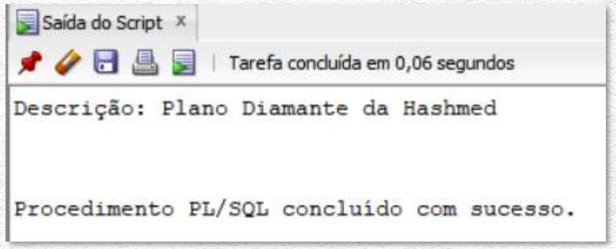
Crie uma procedure chamada exibir_descricao_beneficio que receba o benefit_id e retorne a descrição armazenada na coluna benefit_description.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Veja o resultado do nosso teste:

Ao executar a procedure, a descrição 'Plano Diamante da Hashmed' é exibida na tela:



Ao consultar a tabela benefits, vemos que a benefit_description do benefit_id = 10 é essa mesmo:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1650

Portanto, a procedure está funcionando corretamente.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

9

Questão 9

O RH deseja exibir a descrição detalhada de um benefício pelo seu ID.

Crie uma procedure chamada exibir_descricao_beneficio que receba o benefit_id e retorne a descrição armazenada na coluna benefit_description.

Lembre-se de executar a procedure após criá-la para testar seu funcionamento.

Explicação e Resultado

Essa procedure oferece uma forma rápida e padronizada de consultar e exibir a descrição de um benefício com base no seu ID.

Esse tipo de procedure é útil em:

- ✓ Telas administrativas;
- ✓ Geração de relatórios detalhados;
- ✓ Suporte técnico ou atendimento ao colaborador.

GABARITO EXPLICATIVO

10

Questão 10

Exclua todas as procedures criadas nos exercícios anteriores.

Explicação e Resultado

Para excluir as procedures que criamos anteriormente, aplicamos um **DROP PROCEDURE** com o nome de cada procedure:

```
DROP PROCEDURE mensagem_boas_vindas;
```
→
```
Procedure MENSAGEM_BOAS_VINDAS eliminado.
```

```
DROP PROCEDURE consultar_valor_beneficio;
```
→
```
Procedure CONSULTAR_VALOR_BENEFICIO eliminado.
```

```
DROP PROCEDURE verificar_beneficio_ativo;
```
→
```
Procedure VERIFICAR_BENEFICIO_ATIVO eliminado.
```

```
DROP PROCEDURE atualizar_valor_beneficio;
```
→
```
Procedure ATUALIZAR_VALOR_BENEFICIO eliminado.
```

```
DROP PROCEDURE contar_beneficios_ativos;
```
→
```
Procedure CONTAR_BENEFICIOS_ATIVOS eliminado.
```

```
DROP PROCEDURE total_beneficios_funcionario;
```
→
```
Procedure TOTAL_BENEFICIOS_FUNCIONARIO eliminado.
```

```
DROP PROCEDURE reajustar_plano_saude;
```
→
```
Procedure REAJUSTAR_PLANO_SAUDE eliminado.
```

```
DROP PROCEDURE encerrar_beneficios_funcionario;
```
→
```
Procedure ENCERRAR_BENEFICIOS_FUNCIONARIO eliminado.
```

```
DROP PROCEDURE exibir_descricao_beneficio;
```
→
```
Procedure EXIBIR_DESCRICAO_BENEFICIO eliminado.
```

MÓDULO 18

TRIGGERS

TRIGGERS

TRIGGERS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 18 TRIGGERS

01	O QUE É UMA TRIGGER?	897
02	COMO CRIAR TRIGGERS	898
03	COMO GERENCIAR TRIGGERS	906
04	COMO EXCLUIR TRIGGERS	907
05	RESUMO DO MÓDULO	908
06	EXERCÍCIOS	909
07	GABARITO EXPLICATIVO	917

O que é uma TRIGGER?

Uma TRIGGER é um gatilho que será disparado automaticamente quando acontecer um evento.

Uma TRIGGER DML é disparada quando um comando INSERT, UPDATE ou DELETE é executado em uma Tabela ou View.

Na hora de criar uma TRIGGER, podemos definir alguns **elementos**:

TEMPO DE EXECUÇÃO (em relação ao evento): BEFORE (antes) ou AFTER (depois);

EVENTO: INSERT, UPDATE, DELETE;

CORPO: Bloco de Comandos.

Como criar TRIGGERS

Para se criar uma TRIGGER, usamos a seguinte SINTAXE:

CREATE OR REPLACE TRIGGER nome_da_trigger	Cria ou substitui Trigger “X”
[BEFORE ou AFTER] tipo_do_evento	Será disparada antes ou depois do evento “y” acontecer
ON nome_do_objeto	nome da tabela/view
BEGIN	Inicia as instruções da Trigger
Corpo da Trigger;	Comandos da Trigger
END;	Finaliza a Trigger

Como criar TRIGGERS

Vamos a um exemplo:

Precisamos criar uma TRIGGER que seja disparada sempre antes (BEFORE) que um comando (evento) INSERT, UPDATE ou DELETE for feito na tabela PRODUTOS:

Caso o dia em que o INSERT, UPDATE ou DELETE esteja sendo efetuado seja SÁBADO ou DOMINGO ou, sendo dia útil, o horário não seja o comercial (entre 9h e 18h), então o evento deverá ser abortado.

Portanto, teremos que fazer duas verificações:

Como criar TRIGGERS

Para isso, criaremos a seguinte TRIGGER:

```
CREATE OR REPLACE TRIGGER TG_PRODUTOS_BEFORE_INSERT_UPDATE_DELETE
BEFORE INSERT OR UPDATE OR DELETE
ON produtos
BEGIN
    IF TRIM(TO_CHAR(SYSDATE, 'DAY')) IN ('SÁBADO', 'DOMINGO')
    OR
    (TRIM(TO_CHAR(SYSDATE, 'DAY')) NOT IN ('SÁBADO', 'DOMINGO') AND TO_CHAR(SYSDATE, 'HH24') NOT BETWEEN 9 AND 18)
    THEN
        CASE
            WHEN INSERTING THEN
                RAISE_APPLICATION_ERROR(-20001, 'O cadastro de produtos deve ser feito de seg. à sex., entre 9 e 18h.');
            WHEN UPDATING THEN
                RAISE_APPLICATION_ERROR(-20001, 'A atualização de produtos deve ser feito de seg. à sex., entre 9 e 18h.');
        ELSE
                RAISE_APPLICATION_ERROR(-20001, 'A exclusão de produtos deve ser feito de seg. à sex., entre 9 e 18h.');
        END CASE;
    END IF;
END;
```

Como criar TRIGGERS

Vejamos o passo a passo:

1. Iniciamos o serviço de impressão;
2. Começamos com o comando CREATE OR REPLACE TRIGGER e atribuímos um nome para nossa trigger (TG_PRODUTOS_BEFORE_INSERT_UPDATE_DELETE);
3. Definimos que esta TRIGGER será disparada antes (BEFORE) os eventos de INSERT ou UPDATE ou DELETE acontecerem;
4. Informamos que esta TRIGGER deverá ser disparada sempre que tentarem modificar a tabela PRODUTOS;

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE TRIGGER TG_PRODUTOS_BEFORE_INSERT_UPDATE_DELETE 2
BEFORE INSERT OR UPDATE OR DELETE 3
ON produtos 4
```

Como criar TRIGGERS

5 Iniciamos o bloco de instruções da TRIGGER com o comando BEGIN;

6 Por meio da estrutura de controle IF, estabelecemos nossa verificação: Se o dia da semana for sábado ou domingo; Ou, se o dia da semana for um dia útil (de segunda à sexta), porém estiver fora do horário comercial (ou seja, não entre 9h e 18h), então:

```
BEGIN 5
    IF TRIM(TO_CHAR(SYSDATE, 'DAY')) IN ('SÁBADO', 'DOMINGO') 6
    OR
    (TRIM(TO_CHAR(SYSDATE, 'DAY')) NOT IN ('SÁBADO', 'DOMINGO') AND TO_CHAR(SYSDATE, 'HH24') NOT BETWEEN 9 AND 18)
    THEN
```

Como criar TRIGGERS

7 Fazemos outra verificação com o CASE;

Quando for tentada a inserção de novo registro:

Imprimimos na tela o erro: 'O cadastro de produtos deve ser feito de seg. à sex., entre 9 e 18h.';

Quando for tentada a alteração de algum registro existente:

Imprimimos na tela o erro: 'A atualização de produtos deve ser feito de seg. à sex., entre 9 e 18h.';

Caso contrário (ou seja, quando for tentada a exclusão de um registro existente):

Imprimimos na tela o erro: 'A exclusão de produtos deve ser feita de seg. à sex., entre 9 e 18h.';

```
CASE 7
    WHEN INSERTING THEN
        RAISE_APPLICATION_ERROR(-20001, 'O cadastro de produtos deve ser feito de seg. à sex., entre 9 e 18h.');
    WHEN UPDATING THEN
        RAISE_APPLICATION_ERROR(-20001, 'A atualização de produtos deve ser feito de seg. à sex., entre 9 e 18h.');
ELSE
        RAISE_APPLICATION_ERROR(-20001, 'A exclusão de produtos deve ser feito de seg. à sex., entre 9 e 18h.');
```

Como criar TRIGGERS

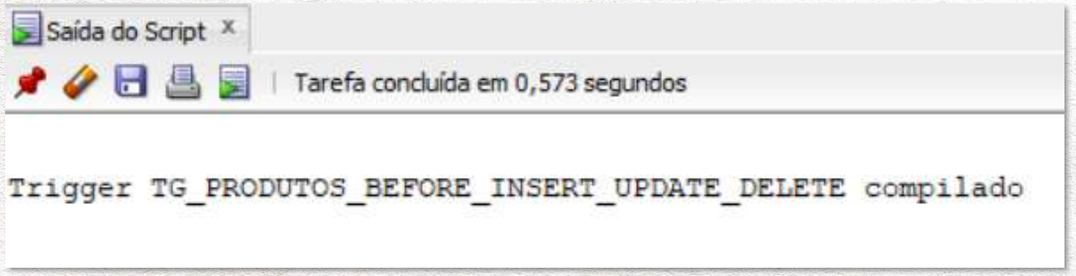
8 Finalizamos o CASE;

9 Finalizamos o IF;

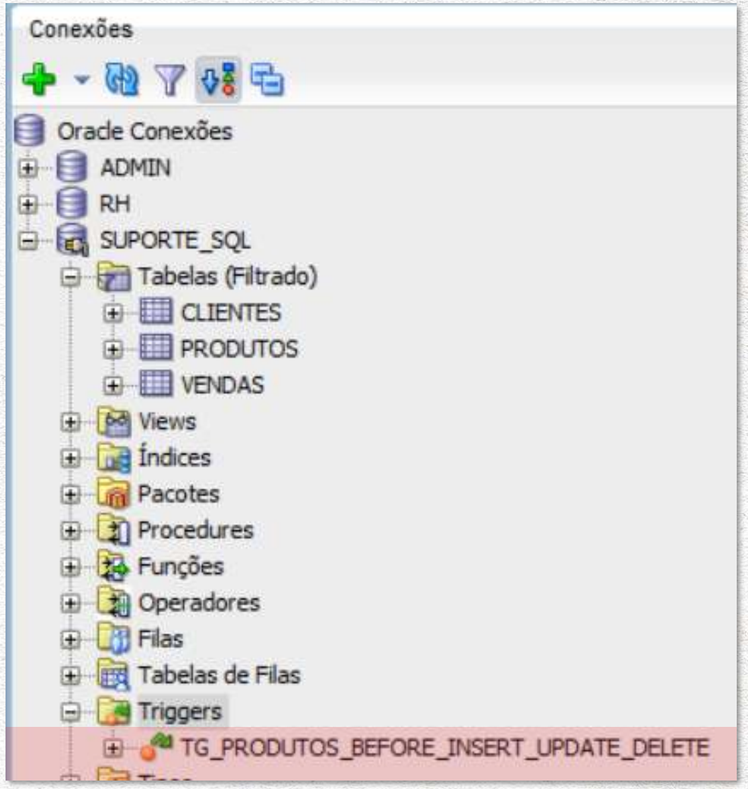
10 Finalizamos a TRIGGER.

```
        END CASE; 8
    END IF; 9
END; 10
```

Executamos a TRIGGER para criá-la:



Repare que agora ela aparecerá na pasta de Triggers da conexão na qual ela foi criada:

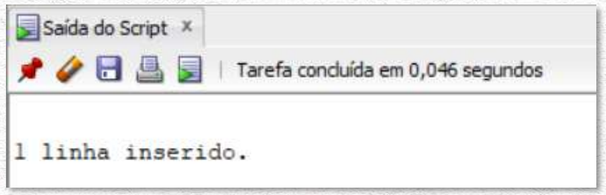


Como criar TRIGGERS

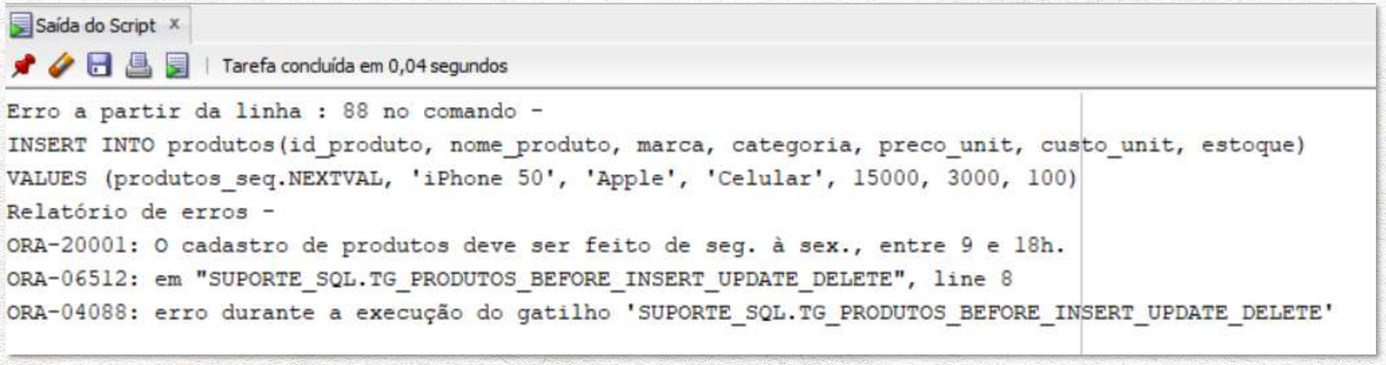
Agora, quando tentarmos efetuar um INSERT, UPDATE ou DELETE na tabela PRODUTOS:

```
INSERT INTO produtos(id_produto, nome_produto, marca, categoria, preco_unit, custo_unit, estoque)
VALUES (produtos_seq.NEXTVAL, 'iPhone 40', 'Apple', 'Celular', 15000, 3000, 100);
```

Se não for sábado ou domingo e, sendo dia útil (de segunda à sexta), estiver dentro do horário comercial (das 9h às 18h), a inserção, alteração ou exclusão será efetuada normalmente:



Caso contrário, a TRIGGER será disparada e retornará o erro específico para o evento de INSERT, UPDATE ou DELETE:



Como gerenciar TRIGGERS

Não precisamos excluir uma TRIGGER caso não queiramos que ela seja executada.

Podemos simplesmente desabilitá-la (seja individualmente ou todas de uma vez) e mantê-la no nosso banco de dados para uso futuro.

SINTAXE:

Para habilitar uma TRIGGER:
ALTER TRIGGER nome_da_trigger **ENABLE**;

```
ALTER TRIGGER TG_PRODUTOS_BEFORE_INSERT_UPDATE_DELETE ENABLE;
```

Para desabilitar uma TRIGGER
ALTER TRIGGER nome_da_trigger **DISABLE**;

```
ALTER TRIGGER TG_PRODUTOS_BEFORE_INSERT_UPDATE_DELETE DISABLE;
```

Para habilitar TODAS as TRIGGERS de uma tabela:
ALTER TABLE nome_da_tabela **ENABLE ALL TRIGGERS**;

```
ALTER TABLE produtos ENABLE ALL TRIGGERS;
```

Para desabilitar TODAS as TRIGGERS de uma tabela:
ALTER TABLE nome_da_tabela **DISABLE ALL TRIGGERS**;

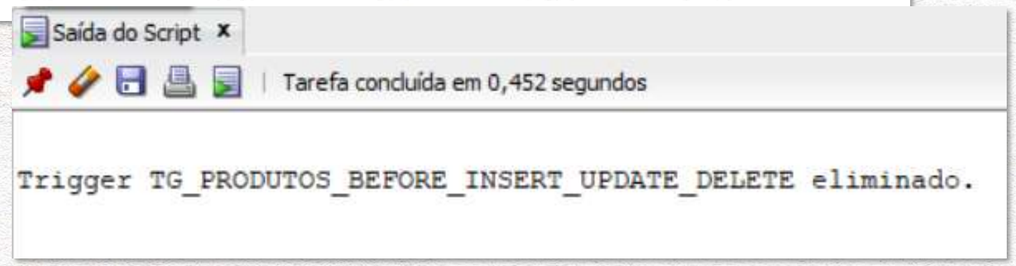
```
ALTER TABLE produtos DISABLE ALL TRIGGERS;
```

Como excluir TRIGGERS

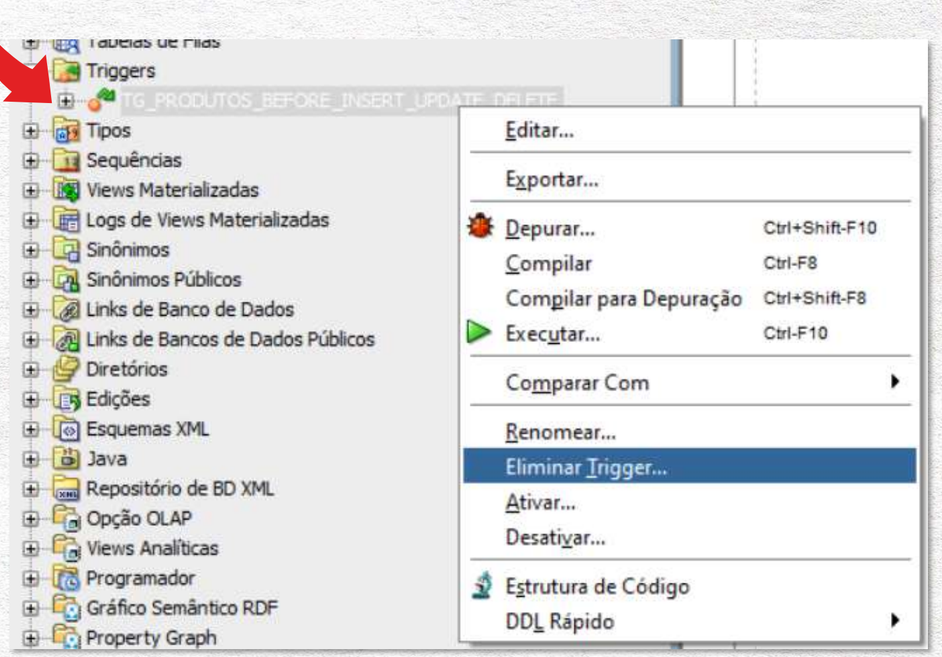
Para excluir uma TRIGGER, basta utilizarmos o comando:

DROP TRIGGER nome_da_trigger;

```
DROP TRIGGER TG_PRODUTOS_BEFORE_INSERT_UPDATE_DELETE;
```



Ou, se preferirmos, podemos também clicar com o botão direito sobre o nome da TRIGGER e selecionar Eliminar Trigger:



Resumo do Módulo

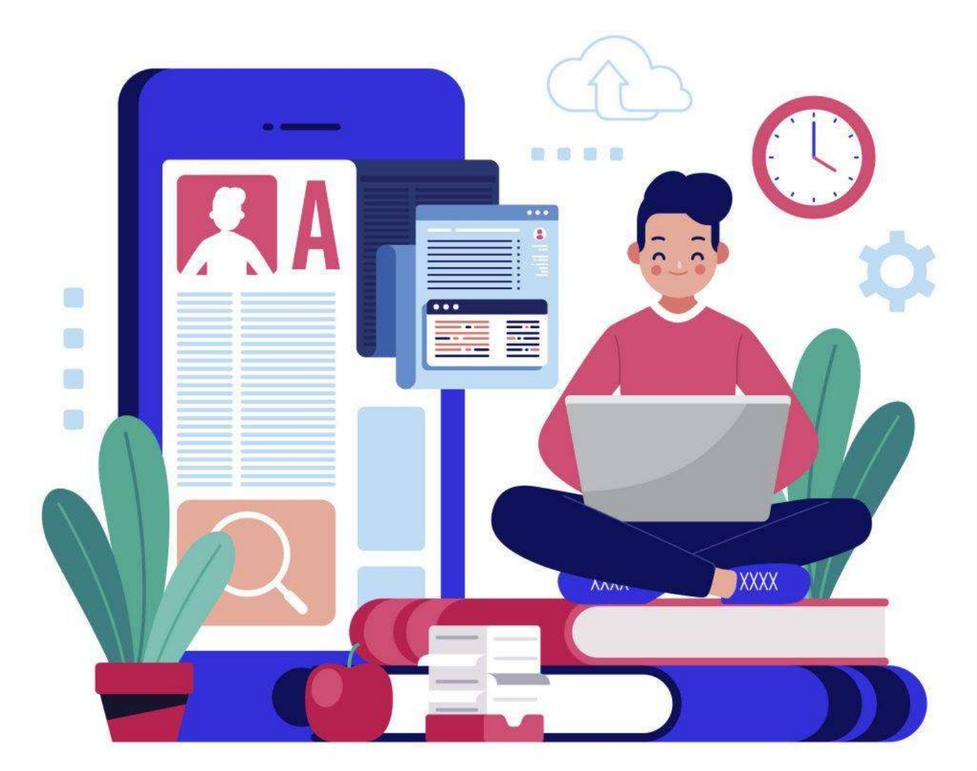
Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. TRIGGERS são eventos que acontecem caso algum outro evento ocorra.

2. Podemos criar TRIGGERS que serão executadas antes (BEFORE) ou depois (AFTER) de comandos DML: INSERT, UPDATE, DELETE.

3. Não precisamos excluir uma TRIGGER caso a gente não queira que ela seja executada. Podemos simplesmente desabilitar a TRIGGER individualmente (ou todas as TRIGGERS) e mantê-la no nosso banco de dados para uso futuro.

EXERCÍCIOS



Para resolvermos os exercícios deste módulo, utilizaremos a tabela benefits (semelhante a que usamos no módulo de CRUD).

Primeiramente, execute o comando **DROP TABLE** abaixo. Assim, caso a tabela já exista, ela será excluída (e se não existir, aparecerá a informação que a tabela não existe):

```
DROP TABLE benefits;
```

Após, crie a tabela benefits executando o **CREATE TABLE** abaixo:

```
CREATE TABLE benefits (
    benefit_id INT,
    employee_id INT,
    benefit_type VARCHAR2(30),
    benefit_description VARCHAR2(50),
    start_date DATE,
    end_date DATE,
    monthly_value NUMBER(10, 2)
);
```

CONTINUA NA PRÓXIMA PÁGINA..

EXERCÍCIOS

Para resolvermos os exercícios deste módulo, utilizaremos a tabela benefits (semelhante a que usamos no módulo de CRUD).

Por fim, insira suas **10 linhas** com os **INSERT INTO** abaixo:

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (1, 145, 'Plano de Saúde', 'Plano Ouro da Hashmed', '01/01/1997', NULL, 850.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (2, 145, 'Vale Refeição', 'Cartão Hashcard', '01/01/1997', NULL, 450.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (3, 146, 'Vale Refeição', 'Cartão Hashcard', '05/03/1997', NULL, 450.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (4, 147, 'Vale Transporte', 'Bilhete Único da Hashtag', '10/05/1997', NULL, 180.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (5, 148, 'Auxílio Educação', 'Bolsa de Especialização Impressionadora', '15/12/1999', NULL, 800.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (6, 148, 'Gympass', 'Plano HashGold', '17/10/2001', '16/10/2002', 120.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (7, 149, 'Gympass', 'Plano HashGold', '29/03/2000', NULL, 120.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (8, 150, 'Plano de Saúde', 'Plano Silver da Hashmed', '01/07/1998', '30/06/2003', 550.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (9, 151, 'Auxílio Educação', 'Bolsa MBA Hashstrategic', '10/08/2004', '10/08/2005', 900.00);

INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (10, 151, 'Plano de Saúde', 'Plano Diamante da Hashmed', '20/04/2002', NULL, 1500.00);
```

CONTINUA NA PRÓXIMA PÁGINA...

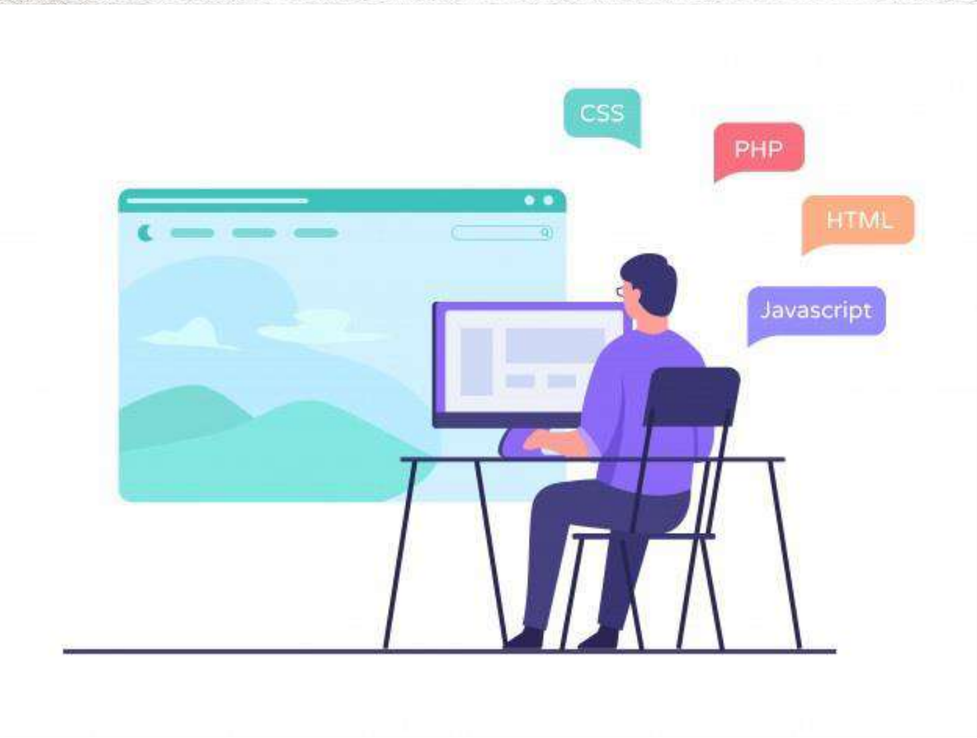
EXERCÍCIOS

Para resolvermos os exercícios deste módulo, utilizaremos a tabela benefits (semelhante a que usamos no módulo de CRUD).

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

Agora, aos exercícios!

EXERCÍCIOS



1

Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes**.

2

Questão 2

Embora a trigger de bloqueio no último dia do mês seja essencial para a segurança dos dados, o setor de RH identificou, de forma excepcional, uma inconsistência grave no cadastro de um benefício que precisa ser corrigida ainda hoje, justamente no último dia do mês.

Como a trigger impede alterações nesta data, a equipe de TI foi acionada para desabilitar temporariamente a trigger, possibilitando a correção imediata e evitando prejuízos financeiros e operacionais.

Desabilite a trigger **tg_bloqueia_benefits_fechamento_mes**.

EXERCÍCIOS

3

Questão 3

Após a correção emergencial realizada pelo RH no último dia do mês, a trigger que bloqueia alterações na tabela benefits precisa ser reativada imediatamente, garantindo que o controle continue funcionando corretamente nos fechamentos dos próximos meses.

Essa reativação faz parte das boas práticas de governança de dados e evita que a exceção se torne regra.

Habilite a trigger **tg_bloqueia_benefits_fechamento_mes**.

4

Questão 4

A empresa está implementando um novo sistema de folha de pagamento integrado ao ERP, que centraliza as regras de bloqueio e validações diretamente na aplicação. Com isso, a trigger de banco que bloqueia alterações no último dia do mês se tornou redundante e desnecessária.

A equipe de TI decidiu, então, remover a trigger do banco de dados, mantendo a lógica exclusivamente na nova aplicação para facilitar a manutenção futura.

Exclua a trigger **tg_bloqueia_benefits_fechamento_mes**.

EXERCÍCIOS

5

Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

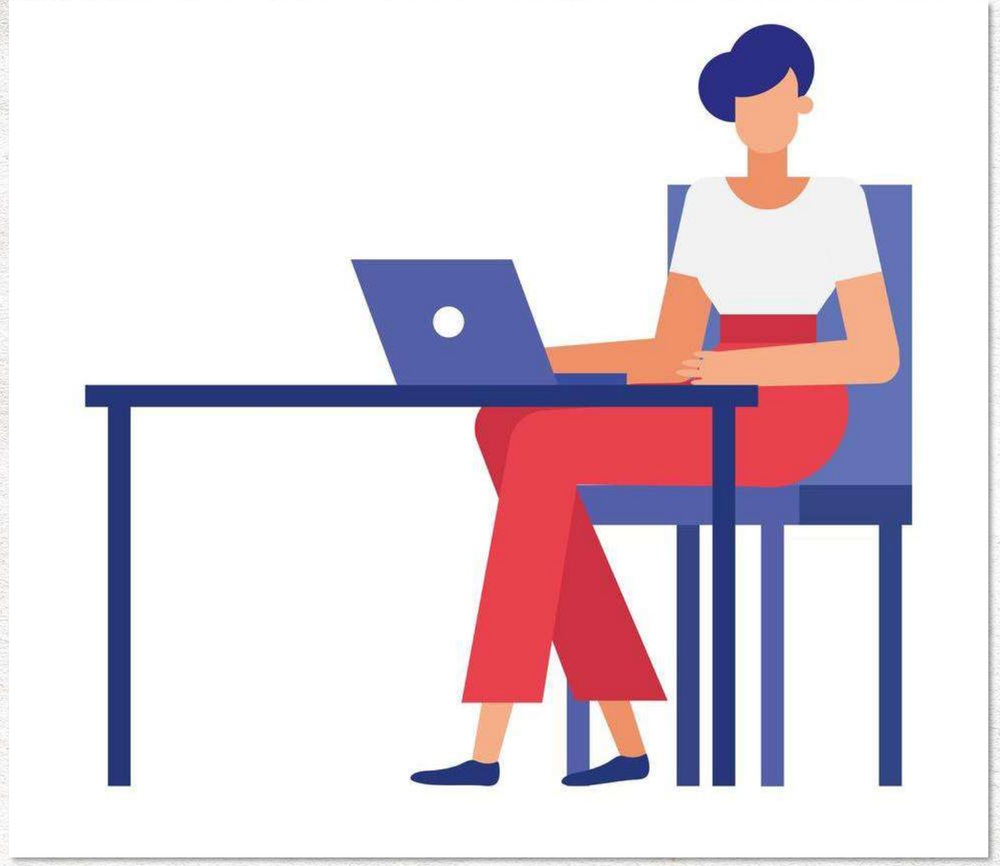
- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

EXERCÍCIOS



6

Questão 6

Devido a uma revisão geral nos registros de benefícios, o RH planeja realizar um mutirão de ajustes que envolve operações em massa fora dos dias tradicionais definidos pela rotina semanal.

Para não precisar esperar os dias corretos e não correr o risco de bloqueio, a trigger precisa ser temporariamente desabilitada, permitindo que as equipes façam as correções necessárias em lote.

Desabilite a trigger **tg_rotinas_diarias_benefits**.

7

Questão 7

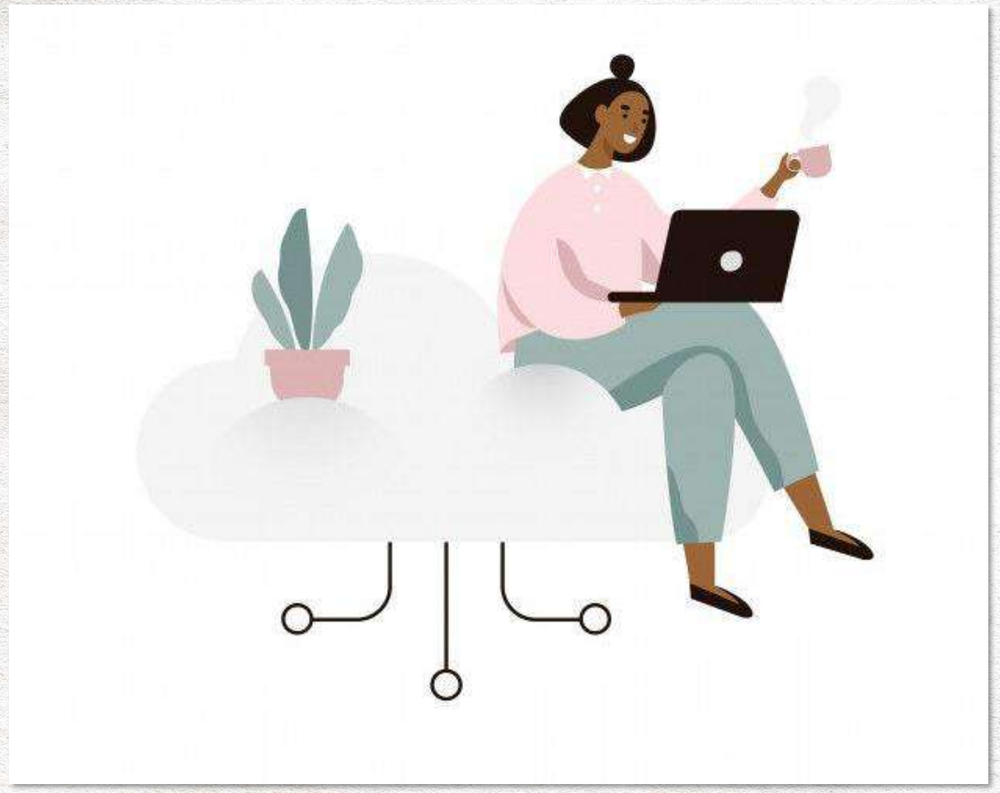
Com a finalização da revisão em massa dos dados de benefícios, o RH solicitou que a rotina semanal de controle por tipo de operação seja reativada normalmente.

A trigger que controla os dias permitidos deve ser habilitada novamente, garantindo que o fluxo regular de manutenção de dados continue sendo seguido conforme a política interna da empresa.

Habilite a trigger **tg_rotinas_diarias_benefits**.

EXERCÍCIOS

8



Questão 8

Com o lançamento de um novo módulo no sistema de RH, que bloqueia automaticamente as operações de cadastro, edição e exclusão com base no dia da semana, a lógica implementada no banco por meio da trigger tg_rotinas_diarias_benefits tornou-se obsoleta.

Para evitar duplicidade de regras e reduzir a complexidade de manutenção, a equipe decidiu remover a trigger do banco de dados.

Exclua a trigger **tg_rotinas_diarias_benefits**.

GABARITO EXPLICATIVO

1 Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes.**

Explicação e Resultado

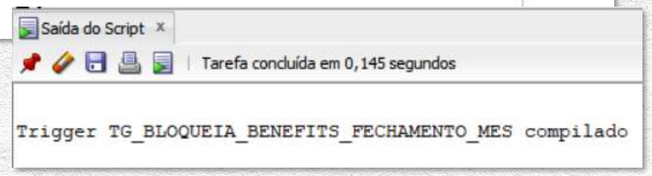
Objetivo:

Devemos criar uma trigger chamada tg_bloqueia_benefits_fechamento_mes que impeça qualquer alteração (inserção, atualização ou exclusão) na tabela benefits durante o último dia do mês, garantindo a integridade dos dados no processo de fechamento contábil.

Passo a passo:

1. Criação da trigger:

```
CREATE OR REPLACE TRIGGER tg_bloqueia_benefits_fechamento_mes
BEFORE INSERT OR UPDATE OR DELETE ON benefits
BEGIN
  IF TRUNC(SYSDATE) = LAST_DAY(SYSDATE) THEN
    RAISE_APPLICATION_ERROR(-20001, 'Operações não permitidas no último dia do mês.');
  END IF;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes.**

Explicação e Resultado

Análise linha por linha:

CREATE OR REPLACE TRIGGER tg_bloqueia_benefits_fechamento_mes: cria ou substitui uma trigger com o nome tg_bloqueia_benefits_fechamento_mes.

BEFORE INSERT OR UPDATE OR DELETE ON benefits: define que a trigger será disparada antes de qualquer operação de inserção, atualização ou exclusão na tabela benefits.

BEGIN ... END: delimita o corpo da trigger, onde a lógica será executada:

- ✓ **IF TRUNC(SYSDATE) = LAST_DAY(SYSDATE) THEN**: compara se a data atual (sem a parte de horário) é igual ao último dia do mês:
  - **TRUNC(SYSDATE)**: elimina a parte da hora da data atual.
  - **LAST_DAY(SYSDATE)**: retorna o último dia do mês da data atual.
- ✓ **RAISE_APPLICATION_ERROR(-20001, 'Operações não permitidas no último dia do mês.');**: caso a condição acima seja verdadeira, a trigger bloqueia a operação levantando um erro personalizado com código -20001 e uma mensagem explicativa.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes.**

Explicação e Resultado

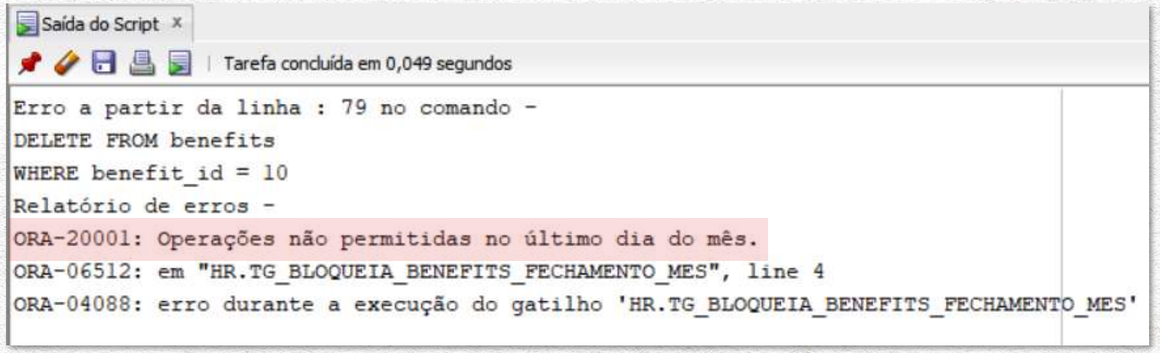
2. Teste da trigger:

Podemos testar fazendo um INSERT, UPDATE ou DELETE:

```
DELETE FROM benefits
WHERE benefit_id = 10;
```

Essa instrução tenta excluir um benefício com benefit_id = 10.

Se o teste for feito no último dia do mês, a trigger bloqueia a operação e exibe a mensagem de erro personalizada:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes.**

Explicação e Resultado

E consultando a tabela benefits...

```
SELECT * FROM benefits;
```

... vemos que o registro do benefit_id = 10 permanece lá:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 10 em 0,009 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900
10	10	151	Plano de Saúde	Plano Diamante da Hashmed	20/04/02	(null)	1500

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

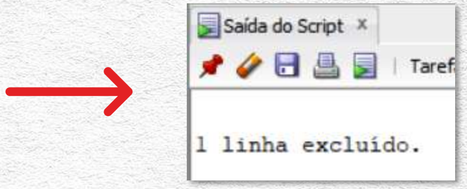
Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes.**

Explicação e Resultado

Mas se executamos um INSERT, UPDATE ou DELETE em qualquer outro dia do mês...

```
DELETE FROM benefits
WHERE benefit_id = 10;
```



... a inserção, atualização ou exclusão é efetuada (no caso do exemplo, a exclusão da linha referente ao benefit_id = 10). Podemos confirmar consultando a tabela benefits:

```
SELECT * FROM benefits;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 9 em 0,003 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 Questão 1

Durante o último dia útil do mês, o setor de Folha de Pagamento realiza o fechamento contábil dos valores de benefícios. Nesse momento, qualquer alteração na tabela benefits pode gerar inconsistências nos relatórios enviados para a contabilidade e no cálculo dos valores a serem pagos.

Para garantir a integridade dos dados e evitar alterações acidentais, o time de TI recebeu a demanda de criar uma trigger que bloqueie todas as operações (INSERT, UPDATE e DELETE) na tabela de benefícios no último dia do mês.

Crie essa trigger, chamando-a de **tg_bloqueia_benefits_fechamento_mes.**

Explicação e Resultado

Essa trigger é uma medida preventiva que protege a integridade dos dados sensíveis durante o fechamento contábil mensal.

Ela:

- ✓ Impede alterações acidentais ou indevidas no último dia do mês;
- ✓ Garante consistência nos relatórios gerados nesse período crítico;
- ✓ Contribui para a segurança e confiabilidade do sistema de benefícios da empresa.

GABARITO EXPLICATIVO

2

Questão 2

Embora a trigger de bloqueio no último dia do mês seja essencial para a segurança dos dados, o setor de RH identificou, de forma excepcional, uma inconsistência grave no cadastro de um benefício que precisa ser corrigida ainda hoje, justamente no último dia do mês.

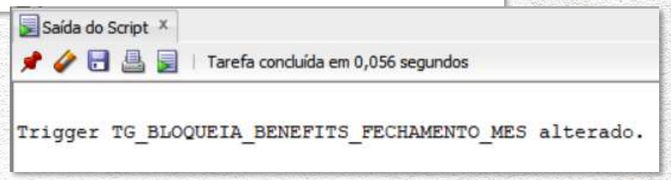
Como a trigger impede alterações nesta data, a equipe de TI foi acionada para desabilitar temporariamente a trigger, possibilitando a correção imediata e evitando prejuízos financeiros e operacionais.

Desabilite a trigger **tg_bloqueia_benefits_fechamento_mes**.

Explicação e Resultado

Para desabilitar a trigger tg_bloqueia_benefits_fechamento_mes, executamos o seguinte código:

```
ALTER TRIGGER tg_bloqueia_benefits_fechamento_mes DISABLE;
```



Com a trigger desabilitada, qualquer inclusão, alteração ou exclusão poderá ser efetuada na tabela benefits, até mesmo no último dia do mês.

GABARITO EXPLICATIVO

3

Questão 3

Após a correção emergencial realizada pelo RH no último dia do mês, a trigger que bloqueia alterações na tabela benefits precisa ser reativada imediatamente, garantindo que o controle continue funcionando corretamente nos fechamentos dos próximos meses.

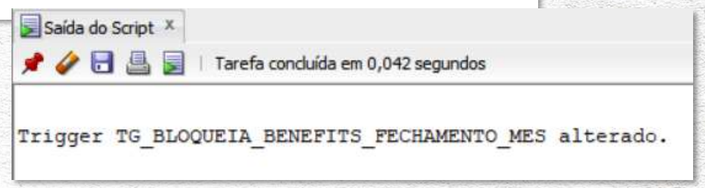
Essa reativação faz parte das boas práticas de governança de dados e evita que a exceção se torne regra.

Habilite a trigger **tg_bloqueia_benefits_fechamento_mes**.

Explicação e Resultado

Para habilitar a trigger tg_bloqueia_benefits_fechamento_mes, executamos o seguinte código:

```
ALTER TRIGGER tg_bloqueia_benefits_fechamento_mes ENABLE;
```



Com a trigger novamente habilitada, qualquer tentativa de inclusão, alteração ou exclusão efetuada no último dia do mês não será efetuada, e a mensagem de erro configurada aparecerá na tela.

Nos demais dias do mês, as inclusões, alterações e exclusões serão efetuadas normalmente.

GABARITO EXPLICATIVO

4

Questão 4

A empresa está implementando um novo sistema de folha de pagamento integrado ao ERP, que centraliza as regras de bloqueio e validações diretamente na aplicação. Com isso, a trigger de banco que bloqueia alterações no último dia do mês se tornou redundante e desnecessária.

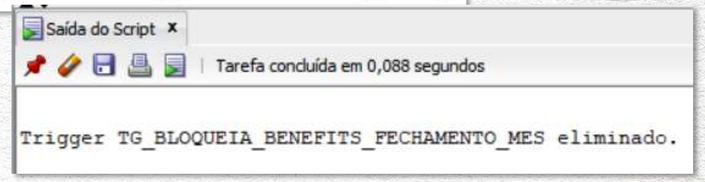
A equipe de TI decidiu, então, remover a trigger do banco de dados, mantendo a lógica exclusivamente na nova aplicação para facilitar a manutenção futura.

Exclua a trigger **tg_bloqueia_benefits_fechamento_mes**.

Explicação e Resultado

Para excluir a trigger tg_bloqueia_benefits_fechamento_mes, executamos o seguinte código:

```
DROP TRIGGER tg_bloqueia_benefits_fechamento_mes;
```



Com a trigger excluída, qualquer inclusão, alteração ou exclusão será efetuada normalmente, independentemente do dia do mês, uma vez que não existe mais nenhuma lógica implementada na tabela benefits que impeça tais operações.

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Objetivo:

Devemos criar uma trigger chamada tg_rotinas_diarias_benefits que controle quais tipos de operação podem ser feitas na tabela benefits de acordo com o dia da semana.

Além disso, quando a operação for permitida, a trigger deve exibir uma mensagem de sucesso na tela.

As regras da rotina diária são:

DIA DA SEMANA	OPERAÇÃO PERMITIDA
Segunda-feira	INSERT
Terça-feira	UPDATE
Quarta-feira	DELETE
Demais dias	Nenhuma operação

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

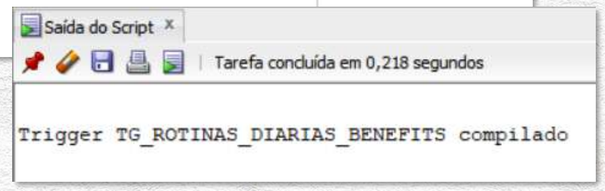
Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Passo a passo:

1. Criação da trigger:

```
CREATE OR REPLACE TRIGGER tg_rotinas_diarias_benefits
BEFORE INSERT OR UPDATE OR DELETE ON benefits
BEGIN
  CASE
    WHEN TRIM(TO_CHAR(SYSDATE, 'DAY')) = 'SEGUNDA-FEIRA' AND NOT INSERTING THEN
      RAISE_APPLICATION_ERROR(-20001, 'Somente INSERTs são permitidos às segundas.');

    WHEN TRIM(TO_CHAR(SYSDATE, 'DAY')) = 'TERÇA-FEIRA' AND NOT UPDATING THEN
      RAISE_APPLICATION_ERROR(-20002, 'Somente UPDATEs são permitidos às terças.');

    WHEN TRIM(TO_CHAR(SYSDATE, 'DAY')) = 'QUARTA-FEIRA' AND NOT DELETING THEN
      RAISE_APPLICATION_ERROR(-20003, 'Somente DELETEs são permitidos às quartas.');

    WHEN TRIM(TO_CHAR(SYSDATE, 'DAY')) NOT IN ('SEGUNDA-FEIRA', 'TERÇA-FEIRA', 'QUARTA-FEIRA') THEN
      RAISE_APPLICATION_ERROR(-20004, 'Nenhuma operação permitida neste dia.');

    ELSE
      DBMS_OUTPUT.PUT_LINE('Operação concluída com sucesso.');

  END CASE;
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Análise linha por linha:

CREATE OR REPLACE TRIGGER tg_rotinas_diarias_benefits: cria (ou substitui) a trigger com o nome tg_rotinas_diarias_benefits.

BEFORE INSERT OR UPDATE OR DELETE ON benefits: a trigger será ativada antes de qualquer operação de inserção, atualização ou exclusão na tabela benefits.

BEGIN ... END: delimita o corpo da trigger, onde a lógica será executada:

- ✓ **CASE WHEN / ELSE**: estabelece as condições a serem verificadas, sendo que:
  - **TO_CHAR(SYSDATE, 'DAY')**: retorna o nome do dia da semana da data atual, com espaços à direita (Exemplo: 'TERÇA-FEIRA '). Por isso, usamos **TRIM** para remover os espaços em branco.
  - **NOT INSERTING / NOT UPDATING / NOT DELETING**: verifica se a operação atual **não é** do tipo esperado para aquele dia. Se **não for**, um erro é levantado com RAISE_APPLICATION_ERROR.
  - **RAISE_APPLICATION_ERROR**: interrompe a execução da operação e exibe um código e uma mensagem personalizada de erro.
  - **ELSE DBMS_OUTPUT.PUT_LINE(...)**: se a operação for permitida, a trigger exibe a mensagem de confirmação: 'Operação concluída com sucesso.'

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

2. Ativando o serviço de impressão para testar:

```
SET SERVEROUTPUT ON;
```

Isso é necessário para que a mensagem de sucesso exibida pela trigger (caso a operação seja concretizada) seja visualizada no console.

3. Testes práticos da trigger:

Supondo que hoje seja TERÇA-FEIRA, veja o que ocorre com cada operação:

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

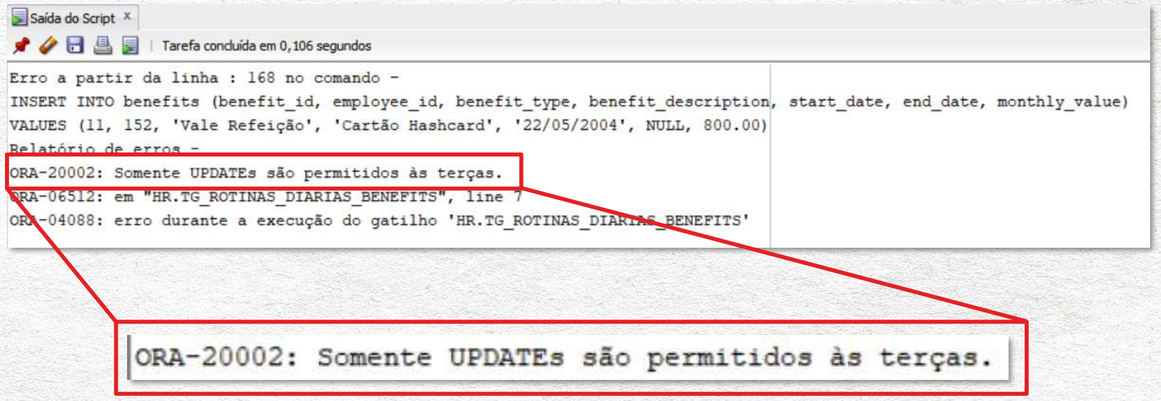
Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

a) Teste de INSERT (não permitido às terças):

```
INSERT INTO benefits (benefit_id, employee_id, benefit_type, benefit_description, start_date, end_date, monthly_value)
VALUES (11, 152, 'Vale Refeição', 'Cartão Hashcard', '22/05/2004', NULL, 800.00);
```

Executando o INSERT em uma terça-feira, a inserção é impedida e o erro com mensagem personalizada aparece na tela:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Podemos confirmar que a inserção não foi efetuada fazendo uma consulta à tabela benefits:

```
SELECT * FROM benefits;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 9 em 0,063 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850
2	2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

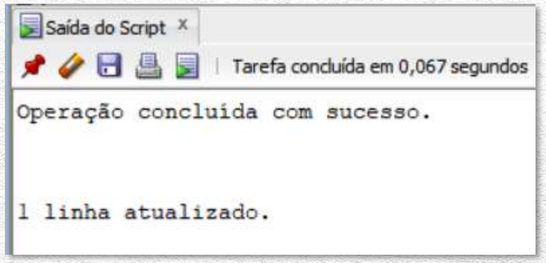
Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

b) Teste de UPDATE (permitido às terças):

```
UPDATE benefits
SET monthly_value = 1050
WHERE benefit_id = 1;
```

Executando o UPDATE em uma terça-feira, a atualização é efetuada, pois é a operação permitida neste dia da semana. Assim, a mensagem de sucesso aparece na tela:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Podemos confirmar que a atualização foi efetuada fazendo uma consulta à tabela benefits:

```
SELECT * FROM benefits;
```

Repare que, antes da atualização, o monthly_value do benefit_id = 1 era 850...

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	850

... e agora é 1050:

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	1050

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

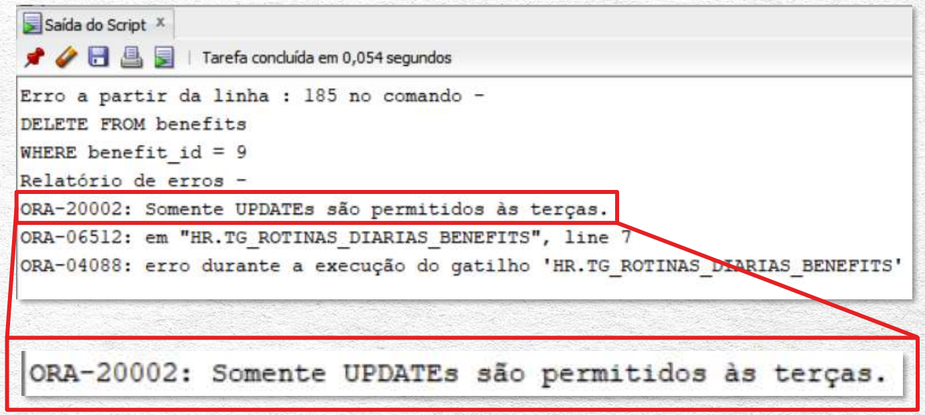
Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

c) Teste de DELETE (não permitido às terças):

```
DELETE FROM benefits
WHERE benefit_id = 9;
```

Executando o DELETE em uma terça-feira, a exclusão é impedida e o erro com mensagem personalizada aparece na tela:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Podemos confirmar que a exclusão não foi efetuada fazendo uma consulta à tabela benefits:

```
SELECT * FROM benefits;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 9 em 0,002 segundos

	BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	BENEFIT_DESCRIPTION	START_DATE	END_DATE	MONTHLY_VALUE
1	1	145	Plano de Saúde	Plano Ouro da Hashmed	01/01/97	(null)	1050
2	2	145	Vale Refeição	Cartão Hashcard	01/01/97	(null)	450
3	3	146	Vale Refeição	Cartão Hashcard	05/03/97	(null)	450
4	4	147	Vale Transporte	Bilhete Único da Hashtag	10/05/97	(null)	180
5	5	148	Auxílio Educação	Bolsa de Especialização Im...	15/12/99	(null)	800
6	6	148	Gympass	Plano HashGold	17/10/01	16/10/02	120
7	7	149	Gympass	Plano HashGold	29/03/00	(null)	120
8	8	150	Plano de Saúde	Plano Silver da Hashmed	01/07/98	30/06/03	550
9	9	151	Auxílio Educação	Bolsa MBA Hashstrategic	10/08/04	10/08/05	900

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5 Questão 5

A área de Recursos Humanos implementou uma nova rotina de atualização de benefícios, distribuída por tipo de operação e dia da semana, com o objetivo de evitar sobrecarga no sistema e facilitar o controle de auditoria:

- ✓ Segundas: apenas novos benefícios são cadastrados.
- ✓ Terças: são feitas atualizações nos registros existentes.
- ✓ Quartas: benefícios inativos ou incorretos são excluídos.
- ✓ Nos demais dias, não deve haver nenhuma alteração.

Para garantir o cumprimento dessa rotina, a TI foi solicitada a criar uma trigger que bloqueie automaticamente as operações fora do dia correto. Além disso, sempre que uma operação for realizada com sucesso no dia apropriado, uma mensagem de confirmação deve ser exibida na tela para facilitar os testes internos.

Crie essa trigger e chame-a de **tg_rotinas_diarias_benefits**.

Dica: como devemos exibir uma mensagem quando a operação for realizada com sucesso, lembre-se de ativar o serviço de impressão antes de executá-la.

Explicação e Resultado

Essa trigger automatiza o controle de alterações na tabela benefits com base no dia da semana, conforme definido pelo RH.

Ela é especialmente útil para:

- ✓ Distribuir a carga de operações ao longo da semana;
- ✓ Facilitar a auditoria de dados;
- ✓ Evitar alterações indevidas fora da rotina estabelecida;
- ✓ Guiar testes com mensagens claras de sucesso ou erro.

O uso de RAISE_APPLICATION_ERROR combinado com DBMS_OUTPUT.PUT_LINE torna a trigger funcional tanto para segurança quanto para testes e rastreamento.

GABARITO EXPLICATIVO

6

Questão 6

Devido a uma revisão geral nos registros de benefícios, o RH planeja realizar um mutirão de ajustes que envolve operações em massa fora dos dias tradicionais definidos pela rotina semanal.

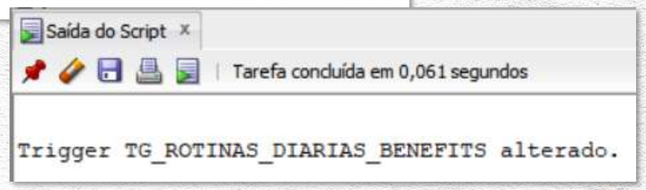
Para não precisar esperar os dias corretos e não correr o risco de bloqueio, a trigger precisa ser temporariamente desabilitada, permitindo que as equipes façam as correções necessárias em lote.

Desabilite a trigger **tg_rotinas_diarias_benefits**.

Explicação e Resultado

Para desabilitar a trigger tg_rotinas_diarias_benefits, executamos o seguinte código:

```
ALTER TRIGGER tg_rotinas_diarias_benefits DISABLE;
```



Com a trigger desabilitada, qualquer inclusão, alteração ou exclusão poderá ser efetuada na tabela benefits em qualquer dia da semana.

GABARITO EXPLICATIVO

7

Questão 7

Com a finalização da revisão em massa dos dados de benefícios, o RH solicitou que a rotina semanal de controle por tipo de operação seja reativada normalmente.

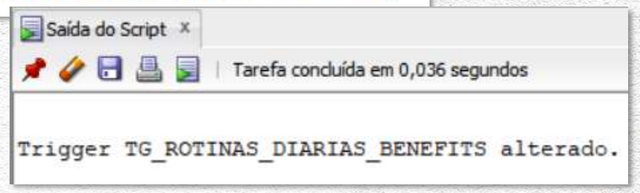
A trigger que controla os dias permitidos deve ser habilitada novamente, garantindo que o fluxo regular de manutenção de dados continue sendo seguido conforme a política interna da empresa.

Habilite a trigger **tg_rotinas_diarias_benefits**.

Explicação e Resultado

Para habilitar a trigger tg_rotinas_diarias_benefits, executamos o seguinte código:

```
ALTER TRIGGER tg_rotinas_diarias_benefits ENABLE;
```



Com a trigger novamente habilitada, qualquer tentativa de inclusão, alteração ou exclusão efetuada nos dias da semana não permitidos será impedida, e a mensagem de erro configurada para aquele dia aparecerá na tela.

No dia em que a operação executada for permitida, ela será executada e uma mensagem personalizada de sucesso aparecerá no console.

GABARITO EXPLICATIVO

8

Questão 8

Com o lançamento de um novo módulo no sistema de RH, que bloqueia automaticamente as operações de cadastro, edição e exclusão com base no dia da semana, a lógica implementada no banco por meio da trigger tg_rotinas_diarias_benefits tornou-se obsoleta.

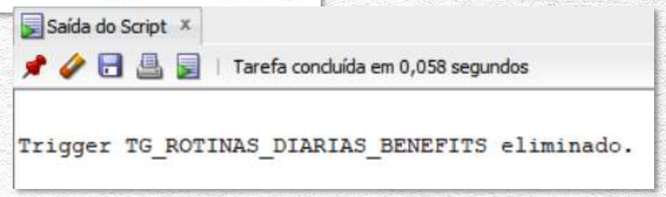
Para evitar duplicidade de regras e reduzir a complexidade de manutenção, a equipe decidiu remover a trigger do banco de dados.

Exclua a trigger **tg_rotinas_diarias_benefits**.

Explicação e Resultado

Para excluir a trigger tg_rotinas_diarias_benefits, executamos o seguinte código:

```
DROP TRIGGER tg_rotinas_diarias_benefits;
```



Com a trigger excluída, qualquer inclusão, alteração ou exclusão será efetuada normalmente, independentemente do dia da semana, uma vez que não existe mais nenhuma lógica implementada na tabela benefits que impeça tais operações.

MÓDULO 19

TRATAMENTO DE ERROS (EXCEÇÕES)

TRATAMENTO DE ERROS (EXCEÇÕES)

TRATAMENTO DE ERROS (EXCEÇÕES)

O QUE VEREMOS NESTE MÓDULO

MÓDULO 19 TRATAMENTO DE ERROS (EXCEÇÕES)

01	EXCEÇÕES DO SISTEMA	942
02	RAISE_APPLICATION_ERROR	952
03	EXCEÇÕES DEFINIDAS PELO USUÁRIO/PROGRAMADOR	957
04	WHEN OTHERS	963
05	SQLCODE e SQLERRM	968
06	RESUMO DO MÓDULO	970
07	EXERCÍCIOS	971
08	GABARITO EXPLICATIVO	974

Exceções do Sistema

EXCEÇÕES DO SISTEMA são erros que possuem um nome no PL/SQL.

O TRATAMENTO dessas exceções do sistema (erros) é importante porque, toda vez que o nosso código retorna um erro, nosso sistema para de funcionar/executar.

Então, quando tratamos essas exceções, isso deixa de acontecer, pois o nosso código, caso se depare com alguma dessas exceções, saberá exatamente como proceder.

Exceções do Sistema

Abaixo, apresentamos algumas das diversas exceções do sistema:

NOME DO ERRO	IDENTIFICADOR	EXPLICAÇÃO
DUP_VAL_ON_INDEX	ORA-00001	Você tentou executar uma instrução INSERT ou UPDATE que criou um valor duplicado em um campo restrito por um índice exclusivo.
TIMEOUT_ON_RESOURCE	ORA-00051	Você estava esperando por um recurso e expirou.
TRANSACTION_BACKED_OUT	ORA-00061	A parte remota de uma transação foi revertida.
INVALID_CURSOR	ORA-01001	Você tentou fazer referência a um cursor que ainda não existe. Isso pode ter acontecido porque você executou um cursor FETCH ou um cursor CLOSE antes de abrir o cursor.
NOT_LOGGED_ON	ORA-01012	Você tentou executar uma chamada para o Oracle antes de efetuar login.
LOGIN_DENIED	ORA-01017	Você tentou fazer login no Oracle com uma combinação de nome de usuário/senha inválida.
NO_DATA_FOUND	ORA-01403	Você tentou um dos seguintes: (1) Você executou uma instrução SELECT INTO e nenhuma linha foi retornada. (2) Você fez referência a uma linha não inicializada em uma tabela. (3) Você lê além do final do arquivo com o pacote UTL_FILE.
TOO_MANY_ROWS	ORA-01422	Você tentou executar uma instrução SELECT INTO e mais de uma linha foi retornada.
ZERO_DIVIDE	ORA-01476	Você tentou dividir um número por zero.
INVALID_NUMBER	ORA-01722	Você tentou executar uma instrução SQL que tentou converter uma string em um número, mas não teve êxito.
STORAGE_ERROR	ORA-06500	Você ficou sem memória ou a memória foi corrompida.
PROGRAM_ERROR	ORA-06501	Esta é uma mensagem genérica "Entre em contato com o suporte Oracle" porque foi encontrado um problema interno.
VALUE_ERROR	ORA-06502	Você tentou executar uma operação e ocorreu um erro em uma conversão, truncamento ou restrição inválida de dados numéricos ou de caracteres.
CURSOR_ALREADY_OPEN	ORA-06511	Você tentou abrir um cursor que já está aberto.

Exceções do Sistema

Vejamos um exemplo:

Vamos criar uma FUNCTION que calcula a variação percentual entre dois anos.
Essa FUNCTION deve tratar o erro de divisão por zero.

Para tanto, criaremos o seguinte código:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual(valor_ano1 NUMBER, valor_ano2 NUMBER)
RETURN VARCHAR2
IS

variacao_percentual NUMBER(10, 2);

BEGIN

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'O ano 1 não teve resultado. Informe o valor de ano 1 corretamente.';

END;

SELECT fn_calcula_variacao_percentual(0, 85)
FROM dual;
```

Exceções do Sistema

1. Iniciamos o serviço de impressão;
2. Começamos com o comando CREATE OR REPLACE FUNCTION, atribuímos um nome para nossa Function e definimos seus parâmetros;
3. Começamos a declaração da(s) variável(is) com RETURN VARCHAR2 IS;
4. Declaramos a(s) variável(is);

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual(valor_ano1 NUMBER, valor_ano2 NUMBER) 2
RETURN VARCHAR2 3
IS

variacao_percentual NUMBER(10, 2); 4
```

Exceções do Sistema

5 Iniciamos o bloco de execução com BEGIN:
Efetuamos o cálculo da variável variacao_percentual criada anteriormente;
Imprimimos a mensagem: 'A variação percentual entre os anos foi de: [variacao_percentual * 100] %.';

6 Acrescentamos um tratamento para o erro de divisão por zero para que, quando isso acontecer, o sistema imprima na tela a mensagem: 'O ano 1 não teve resultado. Informe o valor de ano 1 corretamente.';

7 Finalizamos o bloco de execução e a Function com END.

```
BEGIN 5

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

EXCEPTION 6
    WHEN ZERO_DIVIDE THEN
        RETURN 'O ano 1 não teve resultado. Informe o valor de ano 1 corretamente.';

END; 7
```

Exceções do Sistema

Repare abaixo que ao atribuir valor zero para o parâmetro valor_ano1, o sistema tratou o erro conforme definimos na EXCEPTION do nosso código, imprimindo na tela a mensagem configurada:

```
SELECT fn_calcula_variacao_percentual(0, 85)
FROM dual;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,105 segundos

	FN_CALCULA_VARIACAO_PERCENTUAL(0,85)
1	O ano 1 não teve resultado. Informe o valor de ano 1 corretamente.

Exceções do Sistema

Vejamos agora um exemplo de tratamento de erros dentro de uma PROCEDURE:

Vamos criar uma PROCEDURE para cadastrar clientes na tabela CLIENTES.
Caso haja a tentativa de cadastrar um CPF já existente, um tratamento de erro deverá ser feito.

Para tanto, criaremos o seguinte código:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE pr_cadastra_clientes(vnome_cliente clientes.nome_cliente%type, vsexo clientes.sexo%type,
                          vemail clientes.email%type, vdata_nascimento clientes.data_nascimento%type, vcpf clientes.cpf%type)
AS

BEGIN

    INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
        (clientes_seq.NEXTVAL, vnome_cliente, vsexo, vemail, vdata_nascimento, vcpf);

    dbms_output.put_line('Cadastro realizado com sucesso');

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        dbms_output.put_line('O cpf para o cliente cadastrado já existe na tabela.');

END pr_cadastra_clientes;
```

Exceções do Sistema

1. Iniciamos o serviço de impressão;

2. Começamos com o comando CREATE OR REPLACE PROCEDURE, atribuímos um nome para nossa Procedure e definimos seus parâmetros (repare que, agora, em vez de informarmos o tipo de dado dos parâmetros, puxamos o mesmo tipo de dado atribuído à coluna correspondente à variável na tabela CLIENTES, utilizando a sintaxe “nome_da_tabela.nome_da_coluna%type”);

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE pr_cadastra_clientes(vnome_cliente clientes.nome_cliente%type, vsexo clientes.sexo%type,
                        vemail clientes.email%type, vdata_nascimento clientes.data_nascimento%type, vcpf clientes.cpf%type)
AS
```

Exceções do Sistema

3 Começamos a definir as instruções da Procedure com o comando BEGIN:
Inserimos na tabela CLIENTES o registro do novo cliente;
Imprimimos na tela a mensagem: 'Cadastro realizado com sucesso';

4 Acrescentamos um tratamento para o erro de duplicidade de valores para que, quando isso acontecer, o sistema imprima na tela a mensagem: 'O cpf para o cliente cadastrado já existe na tabela.';

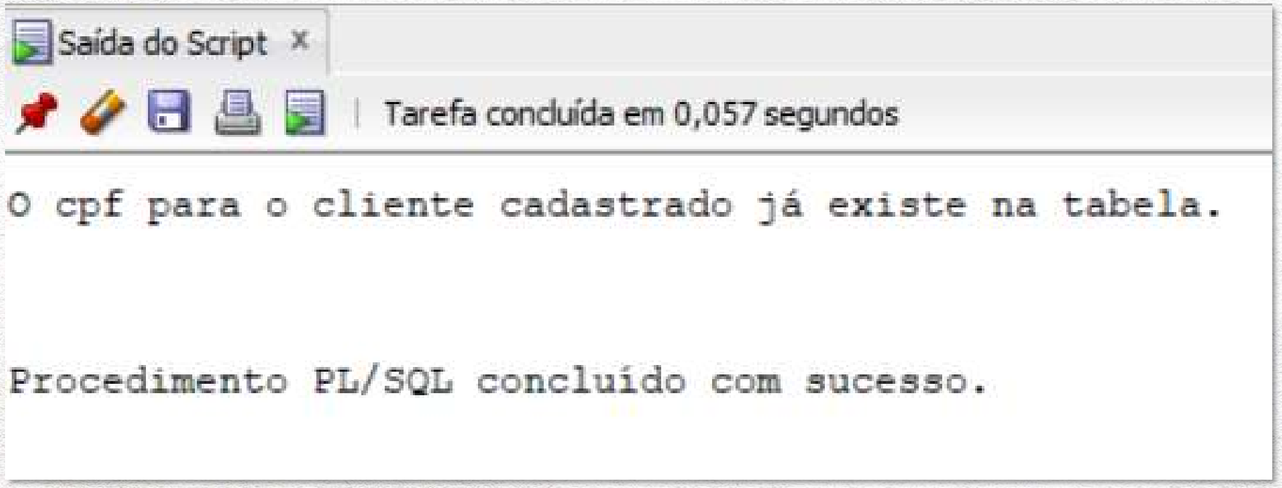
5 Finalizamos o bloco de instruções e a Procedure com END e o nome da Procedure.

```
BEGIN 3

    INSERT INTO clientes(id_cliente, nome_cliente, sexo, email, data_nascimento, cpf) VALUES
        (clientes_seq.NEXTVAL, vnome_cliente, vsexo, vemail, vdata_nascimento, vcpf);

    dbms_output.put_line('Cadastro realizado com sucesso');

EXCEPTION 4
    WHEN DUP_VAL_ON_INDEX THEN
        dbms_output.put_line('O cpf para o cliente cadastrado já existe na tabela.');

END pr_cadastra_clientes; 5
```

Exceções do Sistema

Repare abaixo que ao atribuir ao novo cliente o mesmo número de CPF de cliente já cadastrado anteriormente, o sistema tratou o erro conforme definimos na EXCEPTION do nosso código, imprimindo na tela a mensagem configurada:

```
EXEC pr_cadastra_clientes('Lucas Silva', 'M', 'lucas@hotmail.com', '01/01/2002', '123.456.789-10');
```



RAISE_APPLICATION_ERROR

RAISE_APPLICATION_ERROR é uma PROCEDURE da Oracle que permite ao desenvolvedor associar um número e uma mensagem de erro a uma exceção.

Você pode gerar erros começando com o valor -20000 até -20999.
Qualquer outro número é reservado para os erros padrão da Oracle.

A mensagem de erro pode conter até 2000 caracteres.

Vejamos um exemplo:

Vamos recriar a FUNCTION que calcula a variação percentual entre dois anos.
Só que desta vez, caso o valor do ano 1 seja menor ou igual a zero, associaremos um número e uma mensagem de erro a essa exceção por meio da RAISE_APPLICATION_ERROR.

RAISE_APPLICATION_ERROR

Para tanto, criaremos o seguinte código:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual2(valor_ano1 NUMBER, valor_ano2 NUMBER)
RETURN VARCHAR2
IS

variacao_percentual NUMBER(10, 2);

BEGIN

    IF valor_ano1 <= 0 THEN
        RAISE_APPLICATION_ERROR(-20300, 'O valor informado para o ano 1 deve ser maior que zero.');
    END IF;

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

END;
```

RAISE_APPLICATION_ERROR

1. Iniciamos o serviço de impressão;
2. Começamos com o comando CREATE OR REPLACE FUNCTION, atribuímos um nome para nossa Function e definimos seus parâmetros;
3. Começamos a declaração da(s) variável(is) com RETURN VARCHAR2 IS;
4. Declaramos a(s) variável(is);

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual2(valor_ano1 NUMBER, valor_ano2 NUMBER) 2
RETURN VARCHAR2 3
IS

variacao_percentual NUMBER(10, 2); 4
```

RAISE_APPLICATION_ERROR

5 Iniciamos o bloco de execução com BEGIN:

Fazemos uma estrutura de controle IF para verificar se o valor do ano 1 é menor ou igual a zero:

Se for, imprimimos na tela o erro de número -20300, que estamos criando agora por meio da RAISE_APPLICATION_ERROR, e a mensagem: 'O valor informado para o ano 1 deve ser maior que zero.';

Encerramos a estrutura de controle IF com END IF;

6 Efetuamos o cálculo da variável variacao_percentual criada anteriormente e imprimimos na tela: 'A variação percentual entre os anos foi de: [variacao_percentual * 100] %.';

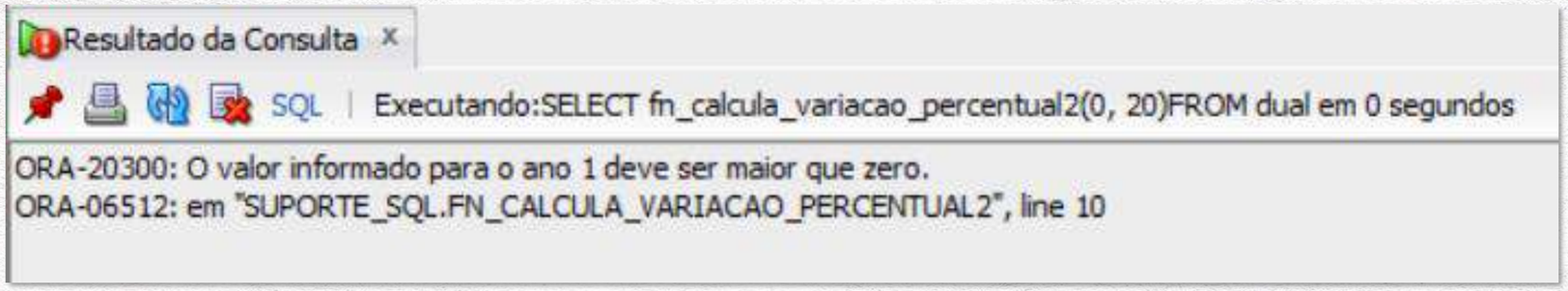
7 Finalizamos o bloco de execução e a Function com END.

```
BEGIN 5

    IF valor_ano1 <= 0 THEN
        RAISE_APPLICATION_ERROR(-20300, 'O valor informado para o ano 1 deve ser maior que zero.');
    END IF;

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

6

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

END; 7
```

RAISE_APPLICATION_ERROR

Repare abaixo que ao atribuir valor zero para o parâmetro valor_ano1, o sistema retornou o erro que criamos com a RAISE_APPLICATION_ERROR, imprimindo na tela o número e a mensagem configurada:

```
SELECT fn_calcula_variacao_percentual2(0, 20)
FROM dual;
```



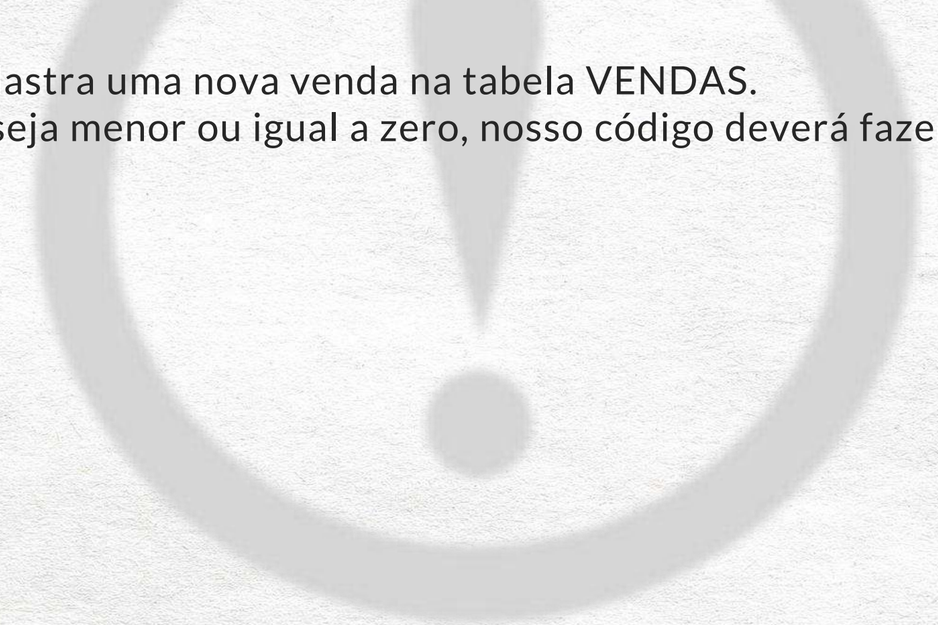
ATENÇÃO! O uso da RAISE_APPLICATION_ERROR não trata o erro em si, apenas atribui a ele um número e uma mensagem a serem informados na tela. Para tratar exceções devemos utilizar o comando EXCEPTION, sejam elas exceções do sistema (conforme vimos até agora) ou exceções definidas pelo usuário/programador (conforme veremos a seguir).

Exceções definidas pelo usuário/programador

EXCEÇÕES DEFINIDAS PELO USUÁRIO (PROGRAMADOR) é o nome dado às exceções que, por ainda não terem sido definidas pela Oracle, os próprios usuários as tratam de forma personalizada.

Vejamos um exemplo:

Vamos criar uma PROCEDURE que cadastra uma nova venda na tabela VENDAS.
Caso a quantidade vendida informada seja menor ou igual a zero, nosso código deverá fazer um tratamento de erros.



Exceções definidas pelo usuário/programador

Para tanto, criaremos o seguinte código:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE pr_cadastra_venda(vid_cliente vendas.id_cliente%type,
                            vid_produto vendas.id_produto%type, vquantidade vendas.quantidade%type)
AS

sem_vendas EXCEPTION;

BEGIN

    IF vquantidade <= 0 THEN
        RAISE sem_vendas;
    ELSE
        INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
            (vendas_seq.NEXTVAL, sysdate, vid_cliente, vid_produto, vquantidade);

        dbms_output.put_line('Venda registrada com sucesso.');

    END IF;

EXCEPTION
    WHEN sem_vendas THEN
        RAISE_APPLICATION_ERROR(-20400, 'A quantidade vendida cadastrada deve ser maior que zero.');

END;
```

Exceções definidas pelo usuário/programador

1. Iniciamos o serviço de impressão;

2. Começamos com o comando CREATE OR REPLACE PROCEDURE, atribuímos um nome para nossa Procedure e definimos seus parâmetros (repare que, agora, em vez de informarmos o tipo de dado dos parâmetros, puxamos o mesmo tipo de dado atribuído à coluna correspondente à variável na tabela VENDAS, utilizando a sintaxe “nome_da_tabela.nome_da_coluna%type”);

3. Declaramos a exceção que será tratada mais abaixo com o comando EXCEPTION, atribuindo-lhe o nome sem_vendas;

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE PROCEDURE pr_cadastra_venda(vid_cliente vendas.id_cliente%type, 2
                          vid_produto vendas.id_produto%type, vquantidade vendas.quantidade%type)
AS

sem_vendas EXCEPTION; 3
```

Exceções definidas pelo usuário/programador

4 Começamos a definir as instruções da Procedure com o comando BEGIN;

Fazemos uma estrutura de controle IF para verificar se a quantidade vendida informada é menor ou igual a zero:

Se for, chamamos a exceção sem_vendas que trataremos mais abaixo com o comando EXCEPTION;

Caso contrário (se a quantidade vendida informada for maior que zero):

Inserimos na tabela VENDAS o registro da nova venda e imprimimos a mensagem: 'Venda registrada com sucesso.';

5 Encerramos a estrutura de controle IF com END IF;

```
BEGIN 4

    IF vquantidade <= 0 THEN
        RAISE sem_vendas;
    ELSE
        INSERT INTO vendas(id_venda, data_venda, id_cliente, id_produto, quantidade) VALUES
            (vendas_seq.NEXTVAL, sysdate, vid_cliente, vid_produto, vquantidade);

        dbms_output.put_line('Venda registrada com sucesso.');

    END IF; 5
```

Exceções definidas pelo usuário/programador

6 Iniciamos o tratamento da exceção sem_vendas com o comando EXCEPTION:

Quando acontecer a exceção sem_vendas:

Imprimimos na tela o erro de número -20400, que estamos criando agora por meio da RAISE_APPLICATION_ERROR, junto com a mensagem: 'A quantidade vendida cadastrada deve ser maior que zero.';

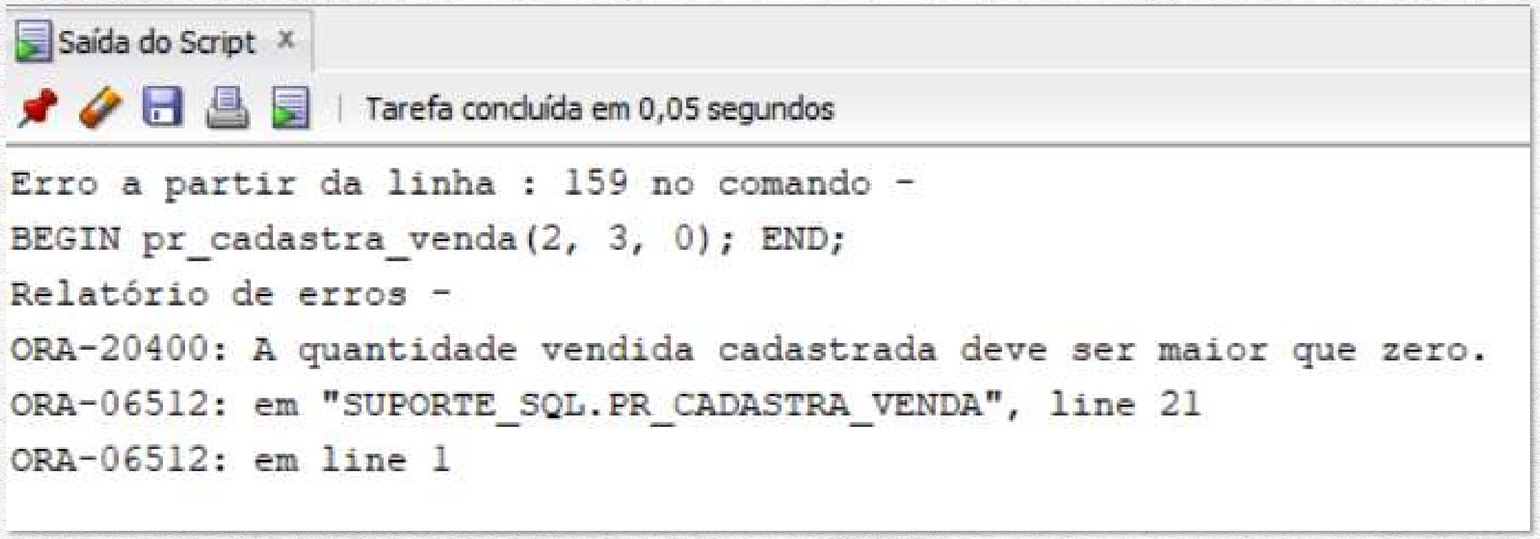
7 Finalizamos o bloco de instruções e a Procedure com o comando END.

```
EXCEPTION 6
    WHEN sem_vendas THEN
        RAISE_APPLICATION_ERROR(-20400, 'A quantidade vendida cadastrada deve ser maior que zero.');

END; 7
```

Exceções definidas pelo usuário/programador

Repare abaixo que ao atribuir à quantidade vendida um valor menor ou igual a zero, o sistema tratou o erro conforme definimos na EXCEPTION do nosso código, imprimindo na tela o número do erro e a mensagem configurada:

```
EXEC pr_cadastra_venda(2, 3, 0);
```



WHEN OTHERS

A cláusula WHEN OTHERS é usada para tratar todas as exceções remanescentes que não foram tratadas pelas exceções do sistema ou pelas exceções definidas pelo usuário.

Vamos retomar o primeiro exemplo deste módulo, recriando a FUNCTION que calcula a variação percentual entre dois anos.

Essa FUNCTION deve tratar o erro de divisão por zero.

Além disso, deve incluir a cláusula WHEN OTHERS para tratar todos os erros remanescentes.

WHEN OTHERS

Para tanto, criaremos o seguinte código:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual(valor_ano1 NUMBER, valor_ano2 NUMBER)
RETURN VARCHAR2
IS

variacao_percentual NUMBER(10, 2);

BEGIN

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'O ano 1 não teve resultado. Portanto, o crescimento foi igual ao valor do ano 2: ' || valor_ano2;
    WHEN OTHERS THEN
        RETURN 'Outros erros foram identificados e tratados.';

END;
```

WHEN OTHERS

1. Iniciamos o serviço de impressão;
2. Começamos com o comando CREATE OR REPLACE FUNCTION, atribuímos um nome para nossa Function e definimos seus parâmetros;
3. Começamos a declaração da(s) variável(is) com RETURN VARCHAR2 IS;
4. Declaramos a(s) variável(is);

```
SET SERVEROUTPUT ON; 1

CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual(valor_ano1 NUMBER, valor_ano2 NUMBER) 2
RETURN VARCHAR2 3
IS

variacao_percentual NUMBER(10, 2); 4
```

WHEN OTHERS

5 Iniciamos o bloco de execução com BEGIN:

Efetuamos o cálculo da variável variacao_percentual criada anteriormente;

Imprimimos na tela a mensagem: 'A variação percentual entre os anos foi de: [variacao_percentual * 100] %.';

```
BEGIN 5
    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';
```

WHEN OTHERS

6 Acrescentamos um tratamento para o erro de divisão por zero para que, quando isso acontecer, o sistema imprima na tela a mensagem: 'O ano 1 não teve resultado. Portanto, o crescimento foi igual ao valor do ano 2: [valor_ano2]';

7 Acrescentamos a cláusula WHEN OTHERS para tratar todos os outros erros remanescentes para que, quando isso acontecer, o sistema imprima na tela a mensagem: 'Outros erros foram identificados e tratados.';

8 Finalizamos o bloco de execução e a Function com END.

```
EXCEPTION
    WHEN ZERO_DIVIDE THEN 6
        RETURN 'O ano 1 não teve resultado. Portanto, o crescimento foi igual ao valor do ano 2: ' || valor_ano2;
    WHEN OTHERS THEN 7
        RETURN 'Outros erros foram identificados e tratados.';
END; 8
```

SQLCODE e SQLERRM

As funções SQLCODE e SQLERRM servem para que possamos acessar, respectivamente, o código (número) e o nome (mensagem) do erro.

Vamos retomar mais uma vez o primeiro exemplo deste módulo, criando a FUNCTION que calcula a variação percentual entre dois anos.

Como sabemos, essa FUNCTION deve tratar o erro de divisão por zero.

Para tanto, criaremos quase o mesmo código:

```
CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual(valor_ano1 NUMBER, valor_ano2 NUMBER)
RETURN VARCHAR2
IS

variacao_percentual NUMBER(10, 2);

BEGIN

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
       RETURN 'O seguinte erro foi identificado: ' || SQLCODE || ' - ' || SQLERRM;

END;
```

SQLCODE e SQLERRM

A única diferença, desta vez, está no RETURN da EXCEPTION, que imprimirá na tela o código do erro, bem como a sua mensagem, pois utilizamos as funções SQLCODE e SQLERRM:

```
CREATE OR REPLACE FUNCTION fn_calcula_variacao_percentual(valor_ano1 NUMBER, valor_ano2 NUMBER)
RETURN VARCHAR2
IS

variacao_percentual NUMBER(10, 2);

BEGIN

    variacao_percentual := valor_ano2 / valor_ano1 - 1;

    RETURN 'A variação percentual entre os anos foi de: ' || variacao_percentual * 100 || '%.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
       RETURN 'O seguinte erro foi identificado: ' || SQLCODE || ' - ' || SQLERRM;

END;
```

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,011 segundos

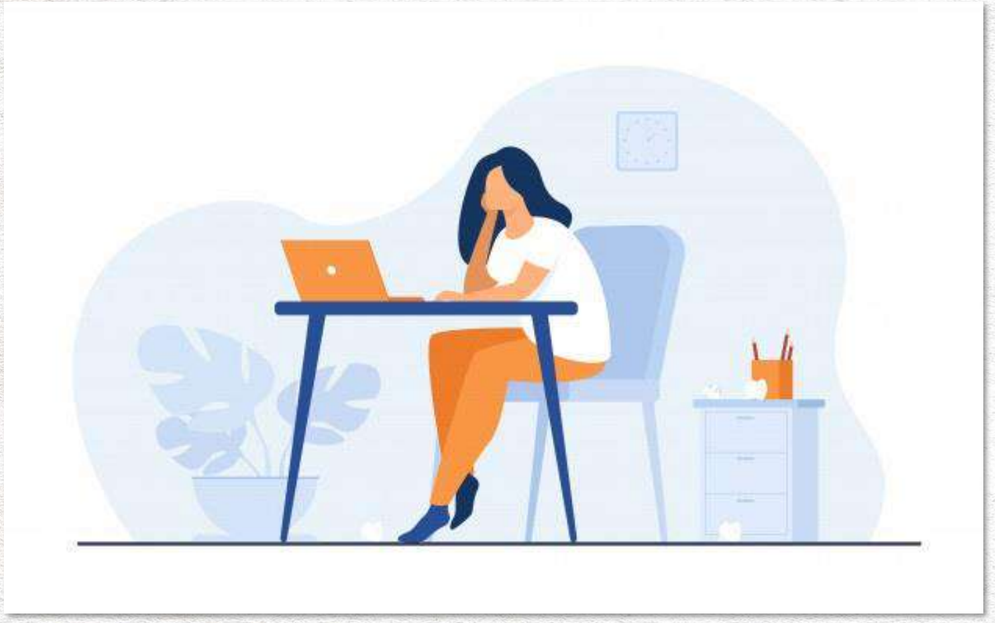
	FN_CALCULA_VARIACAO_PERCENTUAL(0,20)
1	O seguinte erro foi identificado: -1476 - ORA-01476: o divisor é igual a zero

Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. EXCEÇÕES (Erros) são quaisquer eventos inesperados durante a execução de um código.
2. ERROS podem ser de origem do sistema ou podem ser definidos pelo usuário/desenvolvedor.
3. Para os erros definidos pelo usuário, podemos usar a procedure RAISE_APPLICATION_ERROR para configurar um número e um nome para o erro.
4. A cláusula WHEN OTHERS nos permite tratar todas as exceções que não forem tratadas nem pelos erros do sistema nem pelos erros definidos pelo usuário.
5. Podemos utilizar as funções SQLCODE e SQLERRM para retornar, respectivamente, o código e a mensagem do erro.

EXERCÍCIOS



1

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

EXERCÍCIOS

2

Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

3

Questão 3

Modifique a function *fn_calcula_consumo* para incluir um tratamento para todas as outras exceções que possam ocorrer e que não foram tratadas explicitamente, utilizando o bloco WHEN OTHERS.

No tratamento WHEN OTHERS, faça a função retornar uma mensagem padrão informando que um erro inesperado ocorreu.

Mantenha todos os tratamentos anteriores (exceção definida pelo usuário e ZERO_DIVIDE).

EXERCÍCIOS

4

Questão 4

Agora, queremos que a function *fn_calcula_consumo* seja capaz de lidar com possíveis falhas em tempo de execução. Para isso:

a) Remova a exceção definida pelo usuário (*km_zero_excep*) que você criou no exercício 2;

b) Mantenha implementados os tratamentos para as exceções ZERO_DIVIDE e WHEN OTHERS e, em ambos os casos, a função deve retornar uma mensagem de erro formatada contendo o SQLCODE e o SQLERRM, indicando o que ocorreu. Isso permitirá identificar melhor erros comuns ou inesperados durante a execução.

Realize as chamadas da função para testar o retorno da exceção ZERO_DIVIDE:

a) litros_utilizados: 50, km_rodado: 10 (para simular o cálculo bem-sucedido);

b) litros_utilizados: 50, km_rodado: 0 (para simular a exceção).

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

1 Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

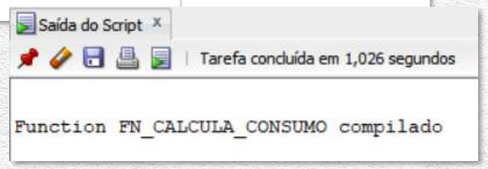
Explicação e Resultado

Objetivo:

Criar uma função que calcule o consumo médio de combustível de um veículo, dividindo a quantidade de litros utilizados pela quilometragem rodada. Se o usuário informar zero quilômetros, a função deve retornar uma mensagem amigável, evitando erro de divisão por zero.

O código completo é o seguinte:

```
CREATE OR REPLACE FUNCTION fn_calcula_consumo(litros_utilizados NUMBER, km_rodado NUMBER)
RETURN VARCHAR2
IS
    consumo NUMBER(10, 2);
BEGIN
    consumo := litros_utilizados / km_rodado;

    RETURN 'O consumo médio foi de ' || consumo || ' litros por quilômetro.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro: a quilometragem rodada deve ser maior que zero. Verifique os dados informados.';
END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

Explicação e Resultado

Explicação passo a passo:

1. Criação da função:

```
CREATE OR REPLACE FUNCTION fn_calcula_consumo(litros_utilizados NUMBER, km_rodado NUMBER)
RETURN VARCHAR2
```

Aqui estamos criando uma função chamada **fn_calcula_consumo**.

Ela recebe dois parâmetros do tipo NUMBER: **litros_utilizados** e **km_rodado**.

Ela retorna um valor do tipo VARCHAR2, ou seja, uma mensagem de texto.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

Explicação e Resultado

Explicação passo a passo:

2. Declaração de variável:

```
IS
    consumo NUMBER(10, 2);
```

Dentro da função, declaramos a variável **consumo**, que armazenará o resultado da divisão (litros por quilômetro).

A precisão (10, 2) indica até 10 dígitos, com 2 casas decimais.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

1 Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

Explicação e Resultado

Explicação passo a passo:

3. Bloco principal de execução (BEGIN...END):

```
BEGIN
    consumo := litros_utilizados / km_rodado;

    RETURN 'O consumo médio foi de ' || consumo || ' litros por quilômetro.';
```

O cálculo do consumo médio é feito diretamente com a divisão.

O resultado é armazenado na variável **consumo**.

A função retorna uma mensagem personalizada com o valor calculado.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

1 Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

Explicação e Resultado

Explicação passo a passo:

4. Tratamento de exceção (EXCEPTION):

```
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro: a quilometragem rodada deve ser maior que zero. Verifique os dados informados.';
END;
```

Se o valor de **km_rodado** for 0, ocorre uma divisão por zero.

Com o WHEN ZERO_DIVIDE, evitamos que o erro quebre a execução e retornamos uma mensagem explicativa ao usuário.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

1 Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

Explicação e Resultado

Testes:

Teste 1: cálculo correto

```
SELECT fn_calcula_consumo(50, 10) FROM dual;
```

Resultado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,217 segundos

	FN_CALCULA_CONSUMO(50,10)
1	O consumo médio foi de 5 litros por quilômetro.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

O setor de logística de uma empresa solicitou à equipe de dados a criação de uma função que permita calcular o consumo médio de combustível de veículos da frota. A ideia é registrar quantos litros foram utilizados em determinado trajeto e, com base na quilometragem rodada, calcular quantos litros são consumidos por quilômetro.

1 Questão 1

Para isso, crie uma function chamada *fn_calcula_consumo* que receba como parâmetros a quantidade de litros de combustível utilizados (*litros_utilizados*) e a quilometragem total rodada (*km_rodado*), e retorne uma mensagem informando quantos litros foram consumidos por quilômetro rodado.

A function deve realizar o cálculo:

consumo := litros_utilizados / km_rodado;

Caso o valor de *km_rodado* seja igual a zero, a função deverá tratar esse erro utilizando o bloco EXCEPTION, exibindo uma mensagem personalizada que oriente o usuário a informar corretamente a quilometragem.

Em seguida, realize a chamada da function passando:

1) O valor de 50 litros utilizados e 10 quilômetros rodados, para simular o cálculo bem-sucedido;

2) O valor de 50 litros utilizados e 0 quilômetros rodados, para simular a exceção.

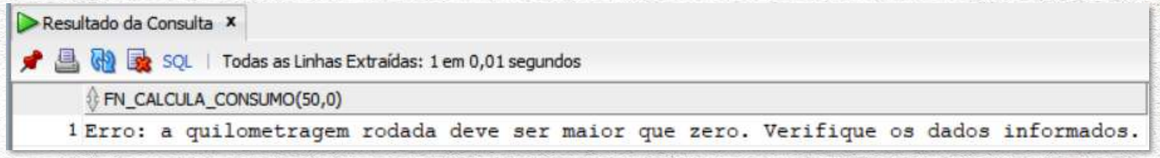
Explicação e Resultado

Testes:

Teste 2: erro tratado

```
SELECT fn_calcula_consumo(50, 0) FROM dual;
```

Resultado:



Esse exercício introduz o uso de tratamento de erros, reforçando a importância de prever falhas como, por exemplo, de divisão por zero. É um ótimo exemplo prático para funções que serão usadas por outros sistemas ou usuários que podem fornecer dados inválidos.

GABARITO EXPLICATIVO

2 Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

Explicação e Resultado

Objetivo:

Modificar a função **fn_calcula_consumo** para:

Criar uma exceção personalizada (**km_zero_excep**) para quando a quilometragem for menor ou igual a zero.

Utilizar RAISE_APPLICATION_ERROR para disparar um erro com código -20400.

Manter o tratamento da exceção ZERO_DIVIDE como segurança extra.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

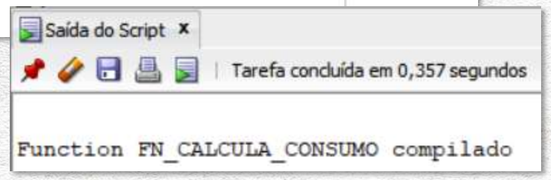
litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

Explicação e Resultado

O código completo é o seguinte:

```
CREATE OR REPLACE FUNCTION fn_calcula_consumo(litros_utilizados NUMBER, km_rodado NUMBER)
RETURN VARCHAR2
IS
    consumo NUMBER(10, 2);
    km_zero_excep EXCEPTION;
BEGIN

    IF km_rodado <= 0 THEN
        RAISE km_zero_excep;
    END IF;

    consumo := litros_utilizados / km_rodado;

    RETURN 'O consumo médio foi de ' || consumo || ' litros por quilômetro.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro: divisão por zero detectada.';
    WHEN km_zero_excep THEN
        RAISE_APPLICATION_ERROR(-20400, 'A quilometragem rodada deve ser maior que zero.');

END;
```



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

Explicação e Resultado

Explicação passo a passo:

1. Cabeçalho da função:

```
CREATE OR REPLACE FUNCTION fn_calcula_consumo(litros_utilizados NUMBER, km_rodado NUMBER)
RETURN VARCHAR2
```

Define a função com dois parâmetros numéricos: **litros_utilizados** e **km_rodado**.

O tipo de retorno será VARCHAR2, pois vamos retornar uma mensagem.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

Explicação e Resultado

Explicação passo a passo:

2. Declaração das variáveis:

```
IS
    consumo NUMBER(10, 2);
    km_zero_excep EXCEPTION;
```

consumo: armazena o resultado do cálculo.

km_zero_excep: exceção definida pelo usuário para validar o valor da quilometragem.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

Explicação e Resultado

Explicação passo a passo:

3. Bloco principal (BEGIN...END):

```
BEGIN

    IF km_rodado <= 0 THEN
        RAISE km_zero_excep;
    END IF;

    consumo := litros_utilizados / km_rodado;

    RETURN 'O consumo médio foi de ' || consumo || ' litros por quilômetro.';
```

Se **km_rodado** for menor ou igual a zero, a exceção personalizada é acionada com RAISE.

Caso passe na validação, é feito o cálculo do consumo normalmente.

A função retorna uma mensagem formatada com o valor do consumo médio.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

Explicação e Resultado

Explicação passo a passo:

4. Tratamento de exceções:

```
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro: divisão por zero detectada.';
    WHEN km_zero_excep THEN
        RAISE_APPLICATION_ERROR(-20400, 'A quilometragem rodada deve ser maior que zero.');

END;
```

ZERO_DIVIDE: continua sendo tratado como uma camada extra de segurança, mesmo após a validação.

km_zero_excep: dispara um erro do Oracle com o código -20400 e uma mensagem clara para o usuário.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

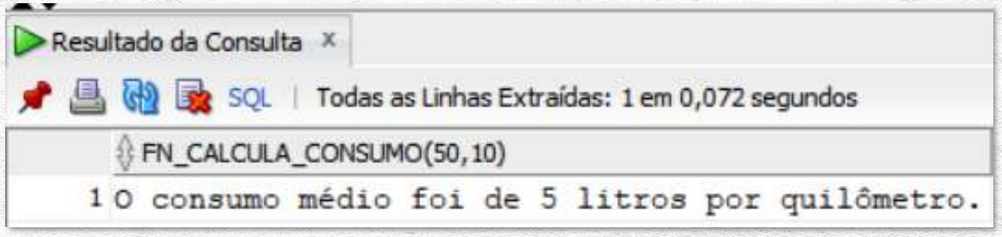
Explicação e Resultado

Testes:

Teste 1 – Dados válidos:

```
SELECT fn_calcula_consumo(50, 10) FROM dual;
```

Resultado:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2 Questão 2

Modifique a function *fn_calcula_consumo* para definir uma exceção personalizada, chamada *km_zero_excep*, que seja acionada quando o valor de *km_rodado* for menor ou igual a zero.

Nesse caso, utilize RAISE_APPLICATION_ERROR para retornar o erro com código -20400 e a mensagem *'A quilometragem rodada deve ser maior que zero.'*

Mantenha o tratamento do erro ZERO_DIVIDE para garantir a robustez da função.

Realize as chamadas da função testando:

litros_utilizados: 50, km_rodado: 10 (cálculo correto);

litros_utilizados: 50, km_rodado: 0 (erro disparado pela exceção personalizada).

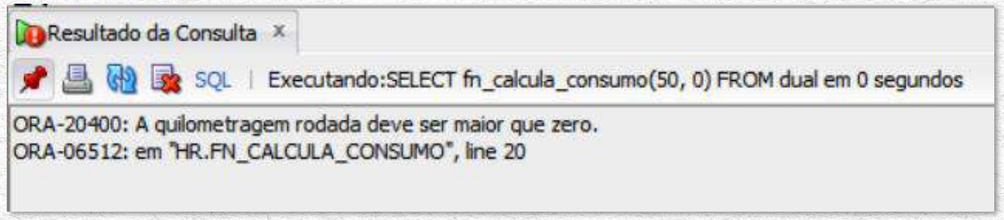
Explicação e Resultado

Testes:

Teste 2 – Quilometragem igual a zero:

```
SELECT fn_calcula_consumo(50, 0) FROM dual;
```

Resultado:



Esse exercício introduziu:

- ✓ Criação e uso de exceções personalizadas.
- ✓ Uso do RAISE_APPLICATION_ERROR com código e mensagem.
- ✓ Reforço da validação de dados de entrada, deixando a função mais confiável.

GABARITO EXPLICATIVO

3

Questão 3

Modifique a function fn_calcula_consumo para incluir um tratamento para todas as outras exceções que possam ocorrer e que não foram tratadas explicitamente, utilizando o bloco WHEN OTHERS.

No tratamento WHEN OTHERS, faça a função retornar uma mensagem padrão informando que um erro inesperado ocorreu.

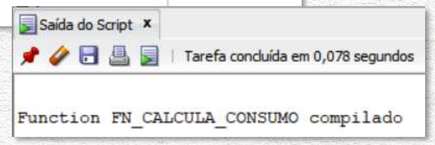
Mantenha todos os tratamentos anteriores (exceção definida pelo usuário e ZERO_DIVIDE).

Explicação e Resultado

Objetivo: Modificar a função **fn_calcula_consumo** para incluir um tratamento genérico de erros utilizando o WHEN OTHERS, além de manter os tratamentos anteriores.

O código completo é o seguinte:

```
CREATE OR REPLACE FUNCTION fn_calcula_consumo(litros_utilizados NUMBER, km_rodado NUMBER)
RETURN VARCHAR2
IS
    consumo NUMBER(10, 2);
    km_zero_excep EXCEPTION;
BEGIN

    IF km_rodado <= 0 THEN
        RAISE km_zero_excep;
    END IF;

    consumo := litros_utilizados / km_rodado;

    RETURN 'O consumo médio foi de ' || consumo || ' litros por quilômetro.';

EXCEPTION
    WHEN km_zero_excep THEN
        RAISE_APPLICATION_ERROR(-20400, 'A quilometragem rodada deve ser maior que zero.');
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro: divisão por zero detectada.';
    WHEN OTHERS THEN
        RETURN 'Erro inesperado ocorreu. Entre em contato com o suporte.';
END;
```



Índice Geral
Índice deste Módulo
CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Modifique a function fn_calcula_consumo para incluir um tratamento para todas as outras exceções que possam ocorrer e que não foram tratadas explicitamente, utilizando o bloco WHEN OTHERS.

No tratamento WHEN OTHERS, faça a função retornar uma mensagem padrão informando que um erro inesperado ocorreu.

Mantenha todos os tratamentos anteriores (exceção definida pelo usuário e ZERO_DIVIDE).

Explicação e Resultado

Passos da solução:

1) Manutenção da lógica principal:

A função continua recebendo dois parâmetros: *litros_utilizados* e *km_rodado*.

Calcula o consumo médio normalmente, dividindo *litros_utilizados* por *km_rodado*.

2) Manutenção da exceção personalizada:

A exceção *km_zero_excep* é acionada se *km_rodado <= 0*.

Neste caso, usamos RAISE_APPLICATION_ERROR para gerar uma mensagem clara ao usuário com o código -20400.

3) Manutenção do tratamento da divisão por zero:

A cláusula WHEN ZERO_DIVIDE retorna uma mensagem informando que houve tentativa de dividir por zero.

4) O QUE MUDA: a inclusão da cláusula WHEN OTHERS:

A inclusão da cláusula WHEN OTHERS é um “plano B” que cobre qualquer erro não previsto pelos tratamentos anteriores. Retorna uma mensagem genérica: 'Erro inesperado ocorreu. Entre em contato com o suporte.’:

```
WHEN OTHERS THEN
    RETURN 'Erro inesperado ocorreu. Entre em contato com o suporte.';
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Modifique a function fn_calcula_consumo para incluir um tratamento para todas as outras exceções que possam ocorrer e que não foram tratadas explicitamente, utilizando o bloco WHEN OTHERS.

No tratamento WHEN OTHERS, faça a função retornar uma mensagem padrão informando que um erro inesperado ocorreu.

Mantenha todos os tratamentos anteriores (exceção definida pelo usuário e ZERO_DIVIDE).

Explicação e Resultado

Observação importante:

A cláusula WHEN OTHERS é usada como uma medida de segurança para capturar qualquer erro inesperado que não tenha sido tratado por outras cláusulas WHEN.

Porém, nem sempre conseguimos forçar a ocorrência desses erros em testes simples, como os que estamos fazendo com chamadas da função.

Por isso, não testaremos a WHEN OTHERS diretamente aqui, mas sua presença é essencial para aumentar a robustez da função e evitar falhas silenciosas em produção. ☺

GABARITO EXPLICATIVO

4

Questão 4

Agora, queremos que a function *fn_calcula_consumo* seja capaz de lidar com possíveis falhas em tempo de execução. Para isso:

a) Remova a exceção definida pelo usuário (*km_zero_excep*) que você criou no exercício 2;

b) Mantenha implementados os tratamentos para as exceções ZERO_DIVIDE e WHEN OTHERS e, em ambos os casos, a função deve retornar uma mensagem de erro formatada contendo o SQLCODE e o SQLERRM, indicando o que ocorreu. Isso permitirá identificar melhor erros comuns ou inesperados durante a execução.

Realize as chamadas da função para testar o retorno da exceção ZERO_DIVIDE:

a) litros_utilizados: 50, km_rodado: 10 (para simular o cálculo bem-sucedido);

b) litros_utilizados: 50, km_rodado: 0 (para simular a exceção).

Explicação e Resultado

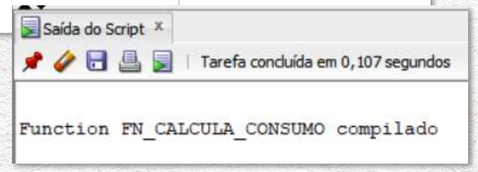
Objetivo:

Ajustar a função **fn_calcula_consumo** para tratar as exceções ZERO_DIVIDE e WHEN OTHERS, utilizando SQLCODE e SQLERRM para retornar mensagens de erro detalhadas.

A exceção personalizada criada no exercício 2 será removida.

O código completo é o seguinte:

```
CREATE OR REPLACE FUNCTION fn_calcula_consumo(litros_utilizados NUMBER, km_rodado NUMBER)
RETURN VARCHAR2
IS
    consumo NUMBER(10, 2);
BEGIN
    consumo := litros_utilizados / km_rodado;

    RETURN 'O consumo médio foi de ' || consumo || ' litros por quilômetro.';

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro identificado: ' || SQLCODE || ' - ' || SQLERRM;
    WHEN OTHERS THEN
        RETURN 'Erro inesperado ' || SQLCODE || ': ' || SQLERRM || '. Entre em contato com o suporte.';
END;
```



CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

4

Questão 4

Agora, queremos que a function *fn_calcula_consumo* seja capaz de lidar com possíveis falhas em tempo de execução. Para isso:

a) Remova a exceção definida pelo usuário (*km_zero_excep*) que você criou no exercício 2;

b) Mantenha implementados os tratamentos para as exceções ZERO_DIVIDE e WHEN OTHERS e, em ambos os casos, a função deve retornar uma mensagem de erro formatada contendo o SQLCODE e o SQLERRM, indicando o que ocorreu. Isso permitirá identificar melhor erros comuns ou inesperados durante a execução.

Realize as chamadas da função para testar o retorno da exceção ZERO_DIVIDE:

a) litros_utilizados: 50, km_rodado: 10 (para simular o cálculo bem-sucedido);

b) litros_utilizados: 50, km_rodado: 0 (para simular a exceção).

Explicação e Resultado

O que foi modificado:

- ✓ A exceção definida pelo usuário (*km_zero_excep*) foi removida;
- ✓ Os blocos de exceção ZERO_DIVIDE e WHEN OTHERS foram mantidos, e ambos passaram a utilizar SQLCODE e SQLERRM para retornar mensagens de erro mais completas:

```
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        RETURN 'Erro identificado: ' || SQLCODE || ' - ' || SQLERRM;
    WHEN OTHERS THEN
        RETURN 'Erro inesperado ' || SQLCODE || ': ' || SQLERRM || '. Entre em contato com o suporte.';
END;
```

O que a função faz agora:

- ✓ Se não ocorrer erro, ela calcula normalmente o consumo.
- ✓ Se houver divisão por zero (ZERO_DIVIDE), retorna o código do erro e a mensagem do Oracle (SQLCODE e SQLERRM).
- ✓ Se ocorrer outro erro inesperado (WHEN OTHERS), também retorna os detalhes do erro para facilitar o diagnóstico.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

4

Questão 4

Agora, queremos que a function *fn_calcula_consumo* seja capaz de lidar com possíveis falhas em tempo de execução. Para isso:

a) Remova a exceção definida pelo usuário (*km_zero_excep*) que você criou no exercício 2;

b) Mantenha implementados os tratamentos para as exceções ZERO_DIVIDE e WHEN OTHERS e, em ambos os casos, a função deve retornar uma mensagem de erro formatada contendo o SQLCODE e o SQLERRM, indicando o que ocorreu. Isso permitirá identificar melhor erros comuns ou inesperados durante a execução.

Realize as chamadas da função para testar o retorno da exceção ZERO_DIVIDE:

a) litros_utilizados: 50, km_rodado: 10 (para simular o cálculo bem-sucedido);

b) litros_utilizados: 50, km_rodado: 0 (para simular a exceção).

Explicação e Resultado

Testes da cláusula ZERO_DIVIDE:

1) Teste com valores normais (sem exceção):

```
SELECT fn_calcula_consumo(50, 10) FROM dual;
```

Resultado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,02 segundos

	FN_CALCULA_CONSUMO(50,10)
1	O consumo médio foi de 5 litros por quilômetro.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Agora, queremos que a function *fn_calcula_consumo* seja capaz de lidar com possíveis falhas em tempo de execução. Para isso:

a) Remova a exceção definida pelo usuário (*km_zero_excep*) que você criou no exercício 2;

b) Mantenha implementados os tratamentos para as exceções ZERO_DIVIDE e WHEN OTHERS e, em ambos os casos, a função deve retornar uma mensagem de erro formatada contendo o SQLCODE e o SQLERRM, indicando o que ocorreu. Isso permitirá identificar melhor erros comuns ou inesperados durante a execução.

Realize as chamadas da função para testar o retorno da exceção ZERO_DIVIDE:

a) litros_utilizados: 50, km_rodado: 10 (para simular o cálculo bem-sucedido);

b) litros_utilizados: 50, km_rodado: 0 (para simular a exceção).

Explicação e Resultado

Testes da cláusula ZERO_DIVIDE:

2) Teste que causa divisão por zero:

```
SELECT fn_calcula_consumo(50, 0) FROM dual;
```

Resultado:

Resultado da Consulta

SQL | Todas as Linhas Extraídas: 1 em 0,01 segundos

	FN_CALCULA_CONSUMO(50,0)
1	Erro identificado: -1476 - ORA-01476: o divisor é igual a zero

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

4

Questão 4

Agora, queremos que a function *fn_calcula_consumo* seja capaz de lidar com possíveis falhas em tempo de execução. Para isso:

a) Remova a exceção definida pelo usuário (*km_zero_excep*) que você criou no exercício 2;

b) Mantenha implementados os tratamentos para as exceções ZERO_DIVIDE e WHEN OTHERS e, em ambos os casos, a função deve retornar uma mensagem de erro formatada contendo o SQLCODE e o SQLERRM, indicando o que ocorreu. Isso permitirá identificar melhor erros comuns ou inesperados durante a execução.

Realize as chamadas da função para testar o retorno da exceção ZERO_DIVIDE:

a) litros_utilizados: 50, km_rodado: 10 (para simular o cálculo bem-sucedido);

b) litros_utilizados: 50, km_rodado: 0 (para simular a exceção).

Explicação e Resultado

Observação importante:

A cláusula WHEN OTHERS serve como um “plano B” para tratar erros não previstos no código.

Como é difícil simular um erro inesperado, não faremos um teste direto para esse caso.

Ainda assim, é uma boa prática deixar o tratamento implementado, pois ele pode capturar problemas inesperados e não tratados em outras exceções.

Agora que estamos utilizando SQLCODE e SQLERRM nesse tratamento genérico, fica mais fácil identificar qual foi o erro que ocorreu, pois esses comandos retornam o número do erro e a mensagem específica fornecida pelo Oracle. Isso ajuda bastante durante o processo de depuração e manutenção da aplicação, mesmo que o erro não seja diretamente testável em simulações simples.

MÓDULO 20

CURSORES

CURSORES

CURSORES

O QUE VEREMOS NESTE MÓDULO

MÓDULO 20 CURSORES

01	O QUE É UM CURSOR?	999
02	TIPOS DE CURSORES	1000
03	CURSOR EXPLÍCITO - LOOP BÁSICO	1002
04	CURSOR EXPLÍCITO - WHILE LOOP	1006
05	CURSOR EXPLÍCITO - FOR LOOP	1010
06	CURSOR EXPLÍCITO - FOR LOOP (COM PARÂMETROS)	1014
07	RESUMO DO MÓDULO	1018
08	EXERCÍCIOS	1019
09	GABARITO EXPLICATIVO	1022

O que é um CURSOR?

Um CURSOR é um ponteiro que aponta para o resultado de uma query.

Com ele, podemos acessar os registros (linhas) de uma tabela.

ID_CLIENTE	NOME_CLIENTE	CPF	
1	André	123.456.789-10	
2	Bruna	321.654.987-01	
3	Carla	455.136.621-89	<---------- PONTEIRO
4	Diego	135.112.987-88	
5	Erika	461.778.132-32	
6	Fabio	289.123.456-31	

Tipos de CURSORES

Os cursores podem ser de dois tipos:

IMPLÍCITO:

Sempre que o Oracle executa um INSERT, UPDATE, DELETE ou SELECT INTO, ele automaticamente cria um cursor implícito.

EXPLÍCITO:

Um cursor explícito é um SELECT declarado explicitamente na seção DECLARE de um bloco de código.

Para esse tipo de cursor, precisamos controlar sua abertura (OPEN), seu acesso (FETCH) e seu fechamento (CLOSE).

É sobre ele que falaremos neste módulo.

Tipos de CURSORES

SINTAXE de um cursor:

1. Declarar o cursor:

CURSOR nome_cursor **IS SELECT**; (sem parâmetros)

CURSOR nome_cursor(parametros) **IS SELECT**; (com parâmetros)

2. Abrir o cursor:

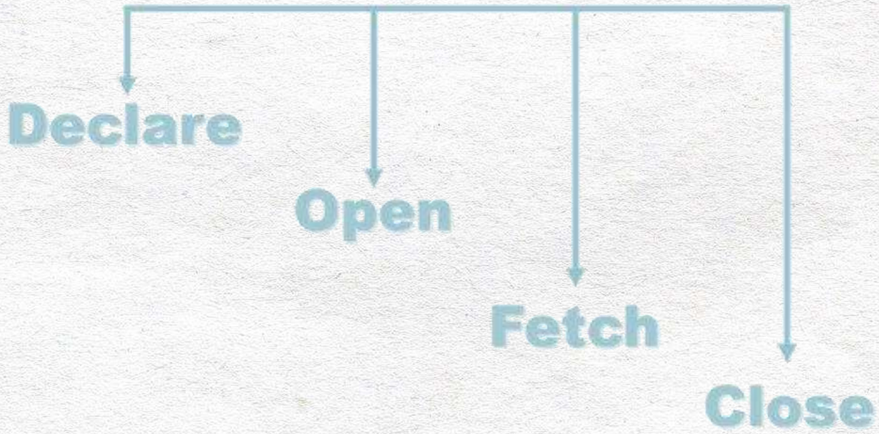
OPEN nome_cursor;

3. Acessar linhas do cursor:

FETCH nome_cursor
INTO lista_de_variaveis;

4. Fechar o cursor:

CLOSE nome_cursor;



CURSOR EXPLÍCITO – LOOP BÁSICO

Podemos fazer uso de CURSORES EXPLÍCITOS por meio do LOOP, WHILE LOOP, FOR LOOP, podendo ou não utilizar parâmetros.

Vejamos alguns exemplos, iniciando com o LOOP:

Queremos acessar os registros da tabela EMPLOYEES utilizando um LOOP básico.

Para tanto, criaremos este código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cursor_employees IS
    SELECT * FROM employees;

    employees_record cursor_employees%rowtype;

BEGIN
    OPEN cursor_employees;

    LOOP
        FETCH cursor_employees
        INTO employees_record;
    EXIT WHEN cursor_employees%notfound;

    dbms_output.put_line(employees_record.employee_id || ' - ' ||
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);

    END LOOP;

    CLOSE cursor_employees;

END;
```

CURSOR EXPLÍCITO – LOOP BÁSICO

1. Iniciamos o serviço de impressão;

2. Começamos com o comando DECLARE:
   Declaramos o cursor e atribuímos-lhe um nome (cursor_employees), fechando a linha de código com IS;
   Inserimos as instruções do cursor por meio de um SELECT que, neste exemplo, deverá retornar todos os registros (linhas) da tabela EMPLOYEES;

3. Declaramos também uma variável (employees_record) que armazenará todos os registros (linhas) retornados pelo cursor, formando uma lista. Repare que, em vez de informarmos o tipo de dado da variável, puxamos o mesmo tipo de cada atributo (coluna) da tabela EMPLOYEES que será retornado e salvo na variável, utilizando o comando "cursor_employees%rowtype";

```
SET SERVEROUTPUT ON; 1

DECLARE 2
    CURSOR cursor_employees IS
    SELECT * FROM employees;

    employees_record cursor_employees%rowtype; 3
```

CURSOR EXPLÍCITO – LOOP BÁSICO

4 Iniciamos o bloco de execução com BEGIN e abrimos o cursor com o comando OPEN;

5 Iniciamos o LOOP, acessamos os registros (linhas) da tabela EMPLOYEES (FETCH), salvando-os na variável employees_record (INTO);

6 Estabelecemos que o LOOP deve ser interrompido quando (EXIT WHEN) o cursor não encontrar mais nenhum registro (cursor_employees%notfound);

7 Imprimimos na tela, por meio do comando dbms_output.put_line, cada atributo (coluna) de cada registro (linha) apontado pelo cursor e salvo na variável employees_record;

8 Finalizamos o LOOP com o comando END LOOP;

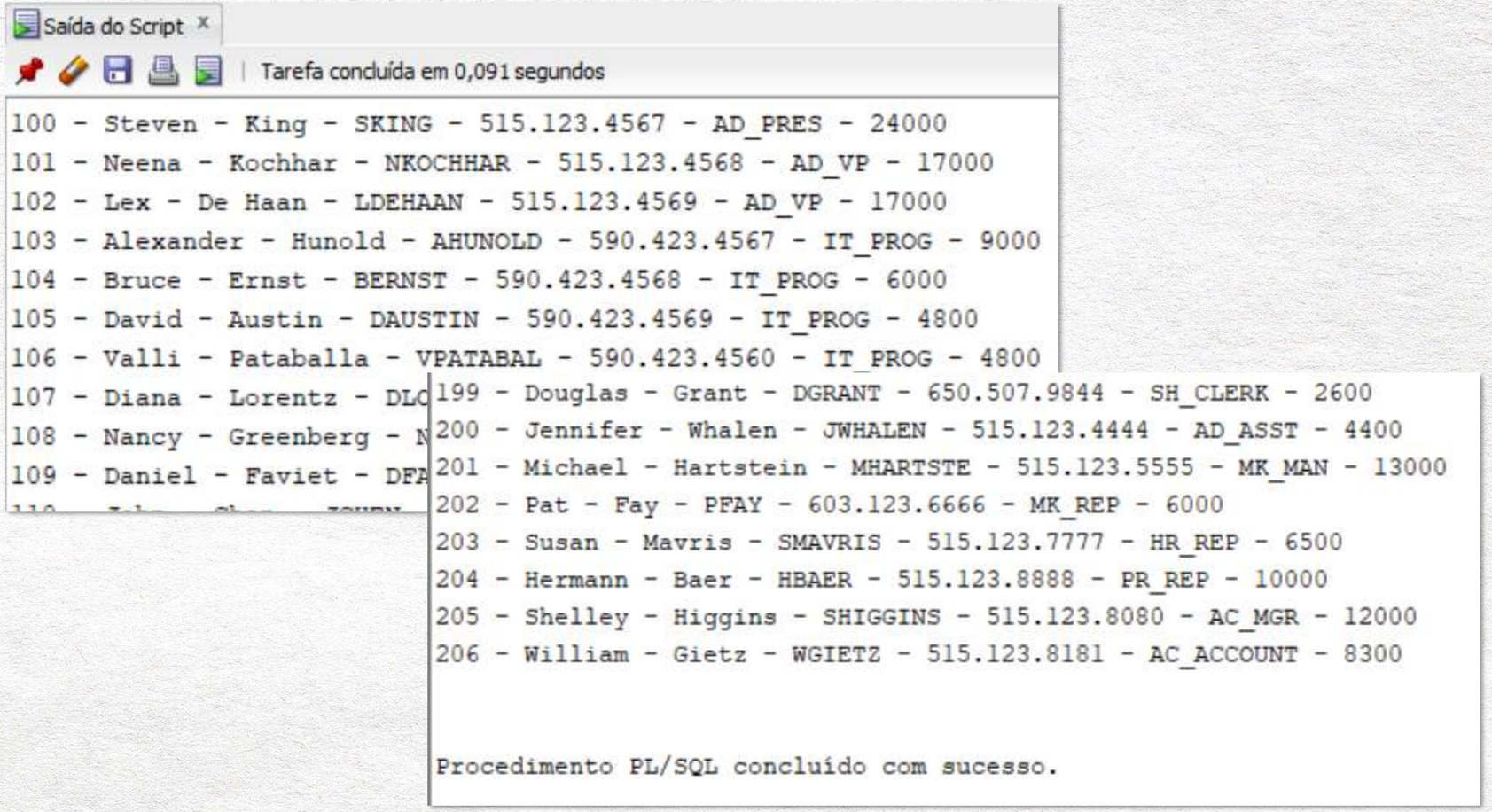
9 Fechamos o cursor com o comando CLOSE;

10 Finalizamos o bloco de execução com o comando END.

```
BEGIN 4
    OPEN cursor_employees;

    LOOP 5
        FETCH cursor_employees
        INTO employees_record;
    EXIT WHEN cursor_employees%notfound; 6

    dbms_output.put_line(employees_record.employee_id || ' - ' || 7
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);

    END LOOP; 8

    CLOSE cursor_employees; 9

END; 10
```

CURSOR EXPLÍCITO – LOOP BÁSICO

Repare ao lado que ao executarmos o código que criamos, o sistema imprimirá na tela todos os atributos (colunas) de todos os registros (linhas) da tabela EMPLOYEES, conforme desejamos:



CURSOR EXPLÍCITO – WHILE LOOP

Vejamos agora o mesmo exemplo anterior, só que desta vez utilizando o WHILE LOOP:

Queremos acessar os registros da tabela EMPLOYEES utilizando um WHILE LOOP.

Para tanto, criaremos este código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_employees IS
    SELECT * FROM employees;

    employees_record cs_employees%rowtype;

BEGIN
    OPEN  cs_employees;
    FETCH cs_employees
    INTO employees_record;

    WHILE cs_employees%found LOOP

    dbms_output.put_line(employees_record.employee_id || ' - ' ||
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.hire_date || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);
    FETCH cs_employees
    INTO employees_record;

    END LOOP;
    CLOSE cs_employees;
END;
```

CURSOR EXPLÍCITO – WHILE LOOP

1. Iniciamos o serviço de impressão;

2. Começamos com o comando DECLARE:
   Declaramos o cursor e atribuímos-lhe um nome (cs_employees), fechando a linha de código com IS;
   Inserimos as instruções do cursor por meio de um SELECT que, neste exemplo, deverá retornar todos os registros (linhas) da tabela EMPLOYEES;

3. Declaramos também uma variável (employees_record) que armazenará todos os registros (linhas) retornados pelo cursor, formando uma lista. Repare que, em vez de informarmos o tipo de dado da variável, puxamos o mesmo tipo de cada atributo (coluna) da tabela EMPLOYEES que será retornado e salvo na variável, utilizando o comando "cs_employees%rowtype";

```
SET SERVEROUTPUT ON; 1

DECLARE 2
    CURSOR cs_employees IS
    SELECT * FROM employees;

    employees_record cs_employees%rowtype; 3
```

CURSOR EXPLÍCITO – WHILE LOOP

4 Iniciamos o bloco de execução com BEGIN, abrimos o cursor com o comando OPEN, acessamos o primeiro registro (linha) da tabela EMPLOYEES (FETCH), salvando-o na variável employees_record (INTO);

5 Estabelecemos, por meio do WHILE LOOP, que o acesso à tabela EMPLOYEES deverá ocorrer enquanto o cursor continuar encontrando registros (cs_employees%found);

6 Imprimimos na tela cada atributo (coluna) de cada registro (linha) apontado pelo cursor e salvo na variável employees_record;

7 Continuamos acessando os registros (linhas) da tabela EMPLOYEES (FETCH), salvando-os na variável employees_record (INTO);

8 Finalizamos o LOOP com o comando END LOOP;

9 Fechamos o cursor com o comando CLOSE;

10 Finalizamos o bloco de execução com END.

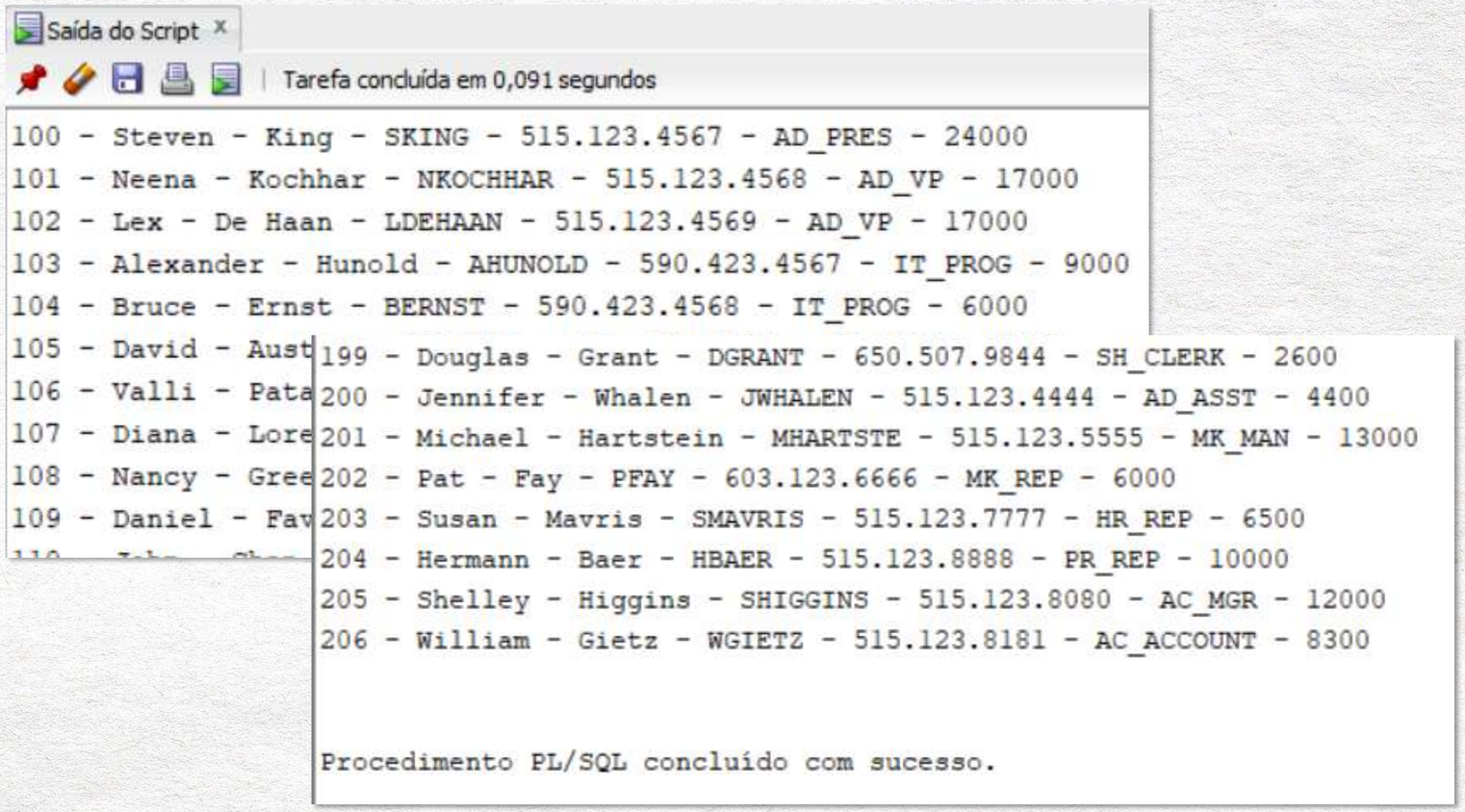
```
BEGIN 4
    OPEN  cs_employees;
    FETCH cs_employees
    INTO employees_record;

    WHILE cs_employees%found LOOP 5

    dbms_output.put_line(employees_record.employee_id || ' - ' || 6
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.hire_date || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);
    FETCH cs_employees 7
    INTO employees_record;

    END LOOP; 8
    CLOSE cs_employees; 9
END; 10
```

Índice Geral

Índice deste Módulo

CURSOR EXPLÍCITO – WHILE LOOP

Repare ao lado que ao executarmos o código que criamos, o sistema imprimirá na tela todos os atributos (colunas) de todos os registros (linhas) da tabela EMPLOYEES, conforme desejamos:



CURSOR EXPLÍCITO – FOR LOOP

Vejamos agora o mesmo exemplo anterior, só que desta vez utilizando o FOR LOOP:

Queremos acessar os registros da tabela EMPLOYEES utilizando um FOR LOOP.

Para tanto, criaremos este código:

```
SET SERVEROUTPUT ON;

DECLARE

    CURSOR cs_employees IS
    SELECT * FROM employees;

BEGIN

    FOR employees_record IN cs_employees LOOP

    dbms_output.put_line(employees_record.employee_id || ' - ' ||
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.hire_date || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);
    END LOOP;
END;
```

CURSOR EXPLÍCITO – FOR LOOP

1. Iniciamos o serviço de impressão;

2. Começamos com o comando DECLARE:
   Declaramos o cursor e atribuímos-lhe um nome (cs_employees), fechando a linha de código com IS;
   Inserimos as instruções do cursor por meio de um SELECT que, neste exemplo, deverá retornar todos os registros (linhas) da tabela EMPLOYEES;

```
SET SERVEROUTPUT ON; 1

DECLARE 2

    CURSOR cs_employees IS
    SELECT * FROM employees;
```

Repare que, com o FOR LOOP, não precisamos declarar a variável que receberá os registros apontados pelo cursor.

CURSOR EXPLÍCITO – FOR LOOP

3 Iniciamos o bloco de execução com BEGIN:

Estabelecemos, por meio do FOR LOOP que, para cada registro (employees_record) no (IN) cursor (cs_employees):

Imprimiremos na tela, por meio do comando dbms_output.put_line, cada atributo (coluna) de cada um desses registros (linhas);

4 Finalizamos o LOOP com o comando END LOOP;

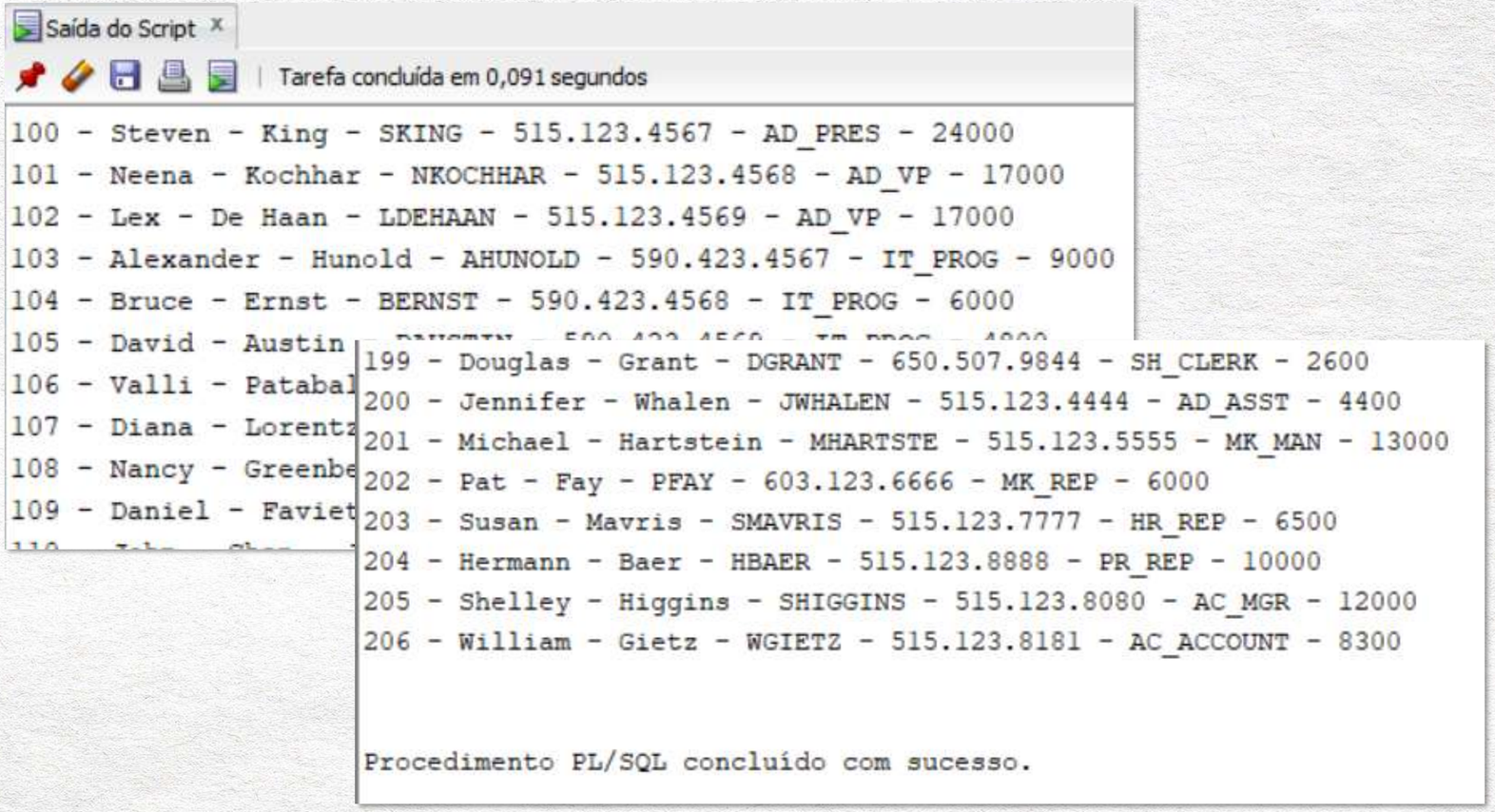
5 Finalizamos o bloco de execução com o comando END.

```
BEGIN 3

    FOR employees_record IN cs_employees LOOP

    dbms_output.put_line(employees_record.employee_id || ' - ' ||
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.hire_date || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);

    END LOOP; 4
END; 5
```

Repare que, com o FOR LOOP, também não precisamos abrir e fechar o cursor, nem utilizar o FETCH INTO para acessar os registros e salvá-los em uma variável.

CURSOR EXPLÍCITO – FOR LOOP

Ao executarmos o código que criamos, o sistema imprimirá na tela todos os atributos (colunas) de todos os registros (linhas) da tabela EMPLOYEES, conforme desejamos:



CURSOR EXPLÍCITO – FOR LOOP (com parâmetros)

Vejamos agora o mesmo exemplo anterior, só que desta vez incluindo parâmetros dentro de um FOR LOOP:

Vamos imaginar que, em vez de acessar todos os registros da tabela EMPLOYEES, queremos retornar apenas aqueles cujo job_id seja igual a ST_MAN.

Para tanto, criaremos este código:

```
SET SERVEROUTPUT ON;

DECLARE

    CURSOR cs_employees(vjob_id VARCHAR2) IS
    SELECT * FROM employees
    WHERE job_id = vjob_id;

BEGIN
    FOR employees_record IN cs_employees('ST_MAN') LOOP
    dbms_output.put_line(employees_record.employee_id || ' - ' ||
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.hire_date || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);
    END LOOP;
END;
```

CURSOR EXPLÍCITO – FOR LOOP (com parâmetros)

1. Iniciamos o serviço de impressão;

2. Começamos com o comando DECLARE:

Declaramos o cursor, atribuímos-lhe um nome (cs_employees), assim como os seus parâmetros (neste caso, apenas um: vjob_id, que no FOR LOOP receberá a informação sobre o job_id que deverá ser filtrado), fechando a linha de código com IS;

Inserimos as instruções do cursor por meio de um SELECT que, neste exemplo, deverá retornar os registros (linhas) da tabela EMPLOYEES que tenham o mesmo job_id que no FOR LOOP será passado ao parâmetro vjob_id;

```
SET SERVEROUTPUT ON; 1

DECLARE 2

    CURSOR cs_employees(vjob_id VARCHAR2) IS
    SELECT * FROM employees
    WHERE job_id = vjob_id;
```

CURSOR EXPLÍCITO – FOR LOOP (com parâmetros)

3 Iniciamos o bloco de execução com BEGIN:

Estabelecemos, por meio do FOR LOOP que, para cada registro (employees_record) no (IN) cursor (cs_employees), cujo job_id seja igual a “ST_MAN”:

Imprimiremos na tela, por meio do comando dbms_output.put_line, cada atributo (coluna) de cada um desses registros (linhas);

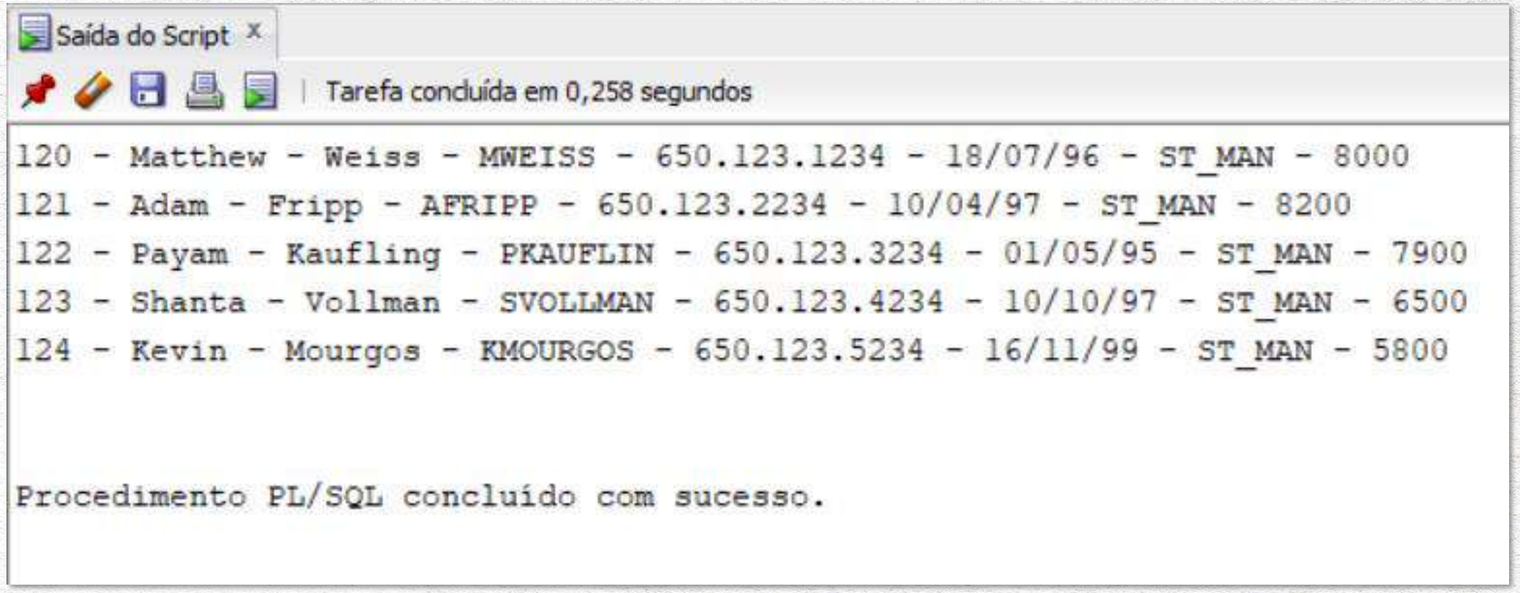
4 Finalizamos o LOOP com o comando END LOOP;

5 Finalizamos o bloco de execução com o comando END.

```
BEGIN 3
    FOR employees_record IN cs_employees('ST_MAN') LOOP
    dbms_output.put_line(employees_record.employee_id || ' - ' ||
                         employees_record.first_name || ' - ' ||
                         employees_record.last_name || ' - ' ||
                         employees_record.email || ' - ' ||
                         employees_record.phone_number || ' - ' ||
                         employees_record.hire_date || ' - ' ||
                         employees_record.job_id || ' - ' ||
                         employees_record.salary);
    END LOOP; 4
END; 5
```

CURSOR EXPLÍCITO – FOR LOOP (com parâmetros)

Ao executarmos o código que criamos, o sistema imprimirá na tela todos os atributos (colunas) dos registros (linhas) da tabela EMPLOYEES cujo job_id seja igual a “ST_MAN”, conforme desejamos:



Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. Um CURSOR é um ponteiro que aponta para o resultado de uma consulta.
Ele nos permite acessar os diferentes registros daquela consulta.

2. Os CURSORES podem ser de dois tipos: IMPLÍCITOS e EXPLÍCITOS.

3. CURSORES IMPLÍCITOS são executados automaticamente quando utilizamos os comandos INSERT, UPDATE, DELETE e SELECT INTO.

4. CURSORES EXPLÍCITOS são SELECTs declarados explicitamente na seção DECLARE de um bloco de código. Controlamos um cursor por meio dos comandos OPEN, FETCH e CLOSE.

5. Utilizamos CURSORES em conjunto com as estruturas de repetição LOOP, WHILE LOOP e FOR LOOP (sendo esta última a mais prática).

EXERCÍCIOS

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1a

Questão 1-a

Gere esse relatório utilizando um cursor com FOR LOOP.

1b

Questão 1-b

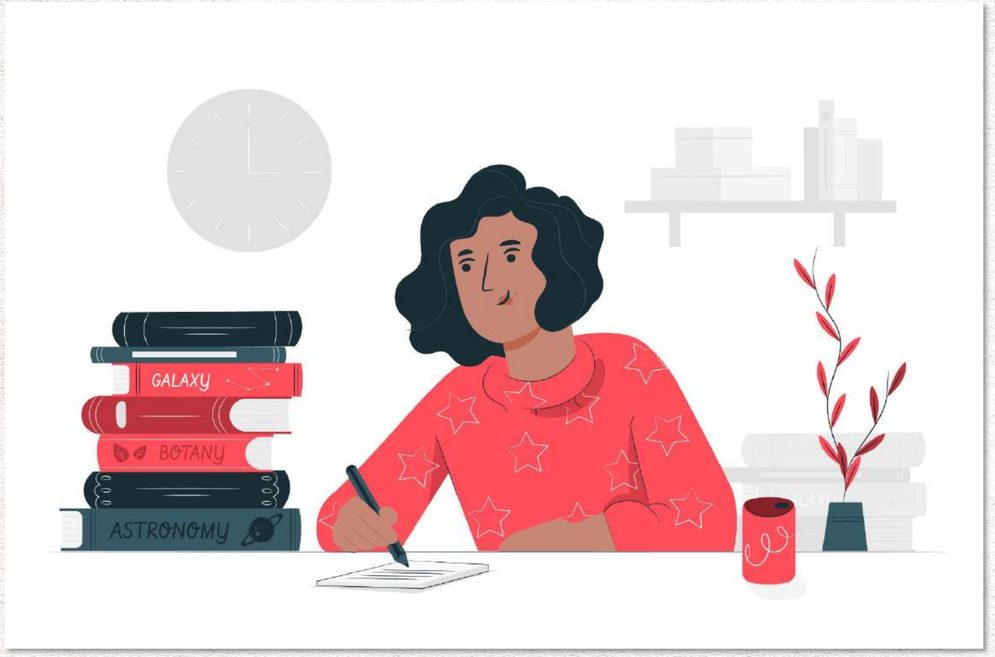
Faça o mesmo utilizando um cursor com WHILE LOOP.

1c

Questão 1-c

Agora, utilizando um cursor com LOOP básico.

EXERCÍCIOS



2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2a

Questão 2-a

Gere esse relatório utilizando um cursor com WHILE LOOP.

2b

Questão 2-b

Faça o mesmo utilizando um cursor com LOOP básico.

2c

Questão 2-c

Agora, utilizando um cursor com FOR LOOP.

EXERCÍCIOS

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3a

Questão 3-a

Gere esse relatório utilizando um cursor com LOOP básico.

3b

Questão 3-b

Faça o mesmo utilizando um cursor com FOR LOOP.

3c

Questão 3-c

Agora, utilizando um cursor com WHILE LOOP.

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1a

Questão 1-a

Gere esse relatório utilizando um cursor com FOR LOOP.

Explicação e Resultado

Para resolver o exercício 1 com um **FOR LOOP**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_departamentos IS
    SELECT department_id, department_name, manager_id FROM departments;

BEGIN
    FOR departments_record IN cs_departamentos LOOP
        dbms_output.put_line('ID do Departamento: ' || departments_record.department_id || ' - ' ||
                             'Nome: ' || departments_record.department_name || ' - ' ||
                             'ID do Gerente: ' || departments_record.manager_id);
    END LOOP;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1a Questão 1-a

Gere esse relatório utilizando um cursor com FOR LOOP.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_departamentos IS
    SELECT department_id, department_name, manager_id FROM departments;
```

Declaramos um cursor chamado cs_departamentos com uma consulta que retorna os dados desejados da tabela departments.

```
BEGIN
    FOR departments_record IN cs_departamentos LOOP
```

Iniciamos um FOR LOOP que percorre automaticamente os registros do cursor. A variável departments_record é criada implicitamente com o tipo da linha retornada pelo cursor.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1a

Questão 1-a

Gere esse relatório utilizando um cursor com FOR LOOP.

Explicação e Resultado

Explicação passo a passo:

```
dbms_output.put_line('ID do Departamento: ' || departments_record.department_id || ' - ' ||
                     'Nome: ' || departments_record.department_name || ' - ' ||
                     'ID do Gerente: ' || departments_record.manager_id);
```

Imprimimos os dados do departamento no formato desejado, utilizando concatenação de strings.

```
    END LOOP;
END;
```

Finalizamos o loop e encerramos o bloco.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

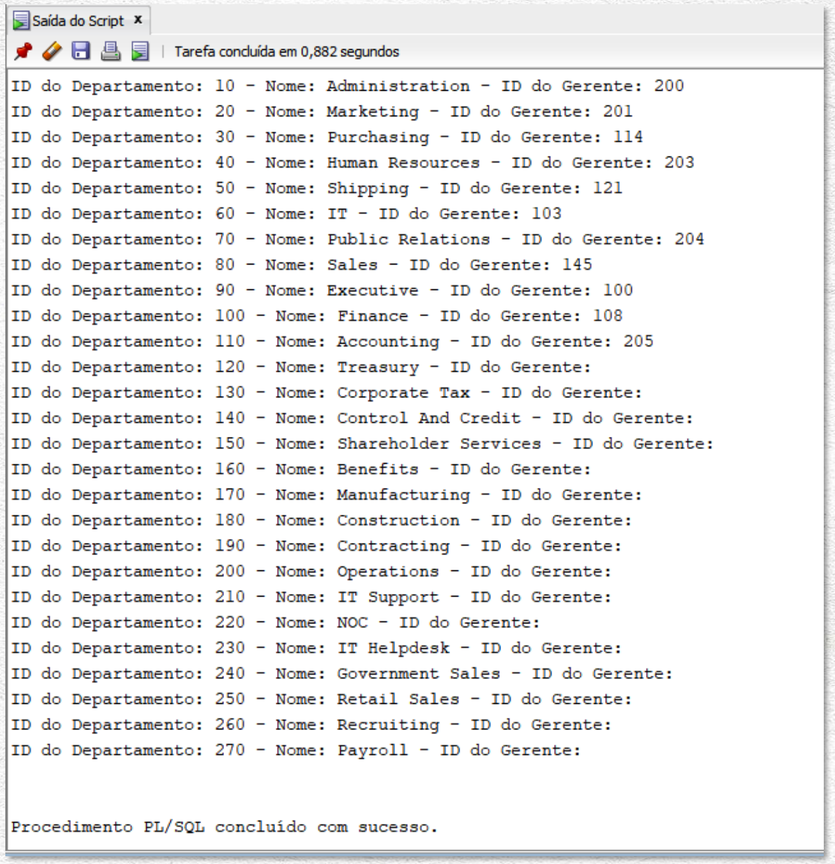
1a

Questão 1-a

Gere esse relatório utilizando um cursor com FOR LOOP.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1b

Questão 1-b

Faça o mesmo utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Agora, para resolver o exercício 1 com um **WHILE LOOP**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_departamentos IS
    SELECT department_id, department_name, manager_id FROM departments;

    departments_record cs_departamentos%ROWTYPE;
BEGIN
    OPEN cs_departamentos;

    FETCH cs_departamentos
    INTO departments_record;

    WHILE cs_departamentos%FOUND LOOP
        dbms_output.put_line('ID do Departamento: ' || departments_record.department_id || ' - ' ||
                             'Nome: ' || departments_record.department_name || ' - ' ||
                             'ID do Gerente: ' || departments_record.manager_id);

        FETCH cs_departamentos
        INTO departments_record;

    END LOOP;

    CLOSE cs_departamentos;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1b

Questão 1-b

Faça o mesmo utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_departamentos IS
    SELECT department_id, department_name, manager_id FROM departments;
```

Declaramos um cursor chamado cs_departamentos com uma consulta que retorna os dados desejados da tabela departments.

```
departments_record cs_departamentos%ROWTYPE;
```

Criamos a variável departments_record do tipo %ROWTYPE para armazenar os dados de cada linha do cursor.

```
BEGIN
    OPEN cs_departamentos;
```

Abrimos o cursor para começar a leitura dos dados.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1b Questão 1-b

Faça o mesmo utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Explicação passo a passo:

```
FETCH cs_departamentos
INTO departments_record;
```

Buscamos o primeiro registro do cursor e armazenamos na variável departments_record.

```
WHILE cs_departamentos%FOUND LOOP
```

Iniciamos o WHILE LOOP. Ele só entra no loop se o último FETCH tiver retornado um registro com sucesso.

```
dbms_output.put_line('ID do Departamento: ' || departments_record.department_id || ' - ' ||
                     'Nome: ' || departments_record.department_name || ' - ' ||
                     'ID do Gerente: ' || departments_record.manager_id);
```

Imprimimos as informações do departamento.

```
FETCH cs_departamentos
INTO departments_record;
```

Executamos o próximo FETCH para atualizar os dados da variável antes da próxima iteração do loop.

```
    END LOOP;
    CLOSE cs_departamentos;
END;
```

Fechamos o loop e encerramos o cursor e o bloco.

Índice Geral
Índice deste Módulo
CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1 EXERCÍCIO 1

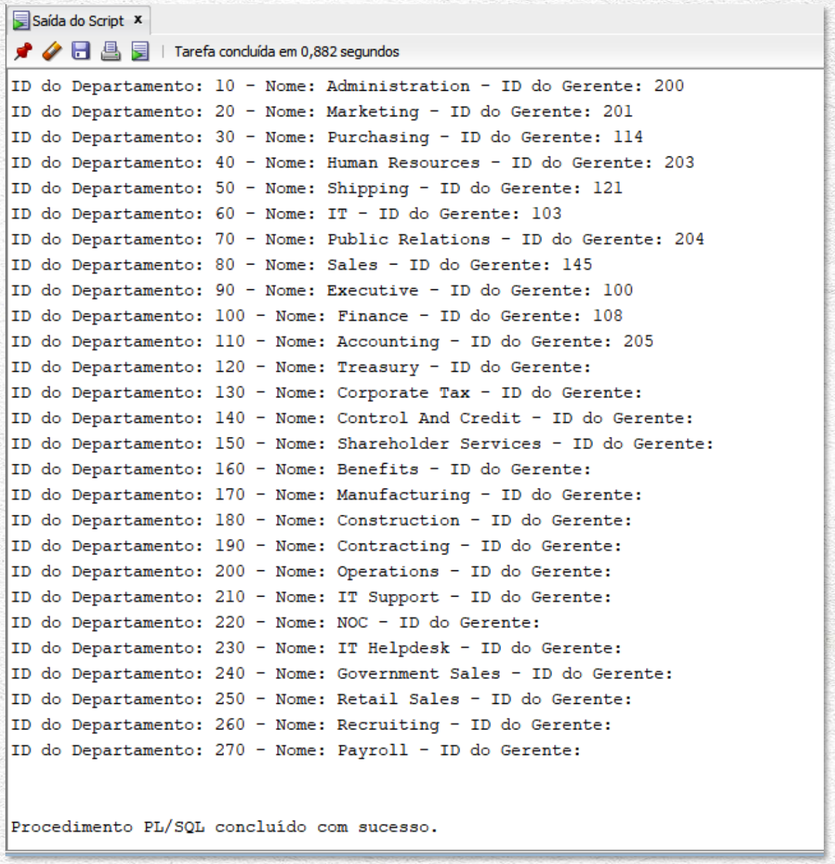
O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1b Questão 1-b

Faça o mesmo utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1c

Questão 1-c

Agora, utilizando um cursor com LOOP básico.

Explicação e Resultado

Por fim, para resolver o exercício 1 com um **LOOP BÁSICO**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_departamentos IS
    SELECT department_id, department_name, manager_id FROM departments;

    departments_record cs_departamentos%ROWTYPE;

BEGIN
    OPEN cs_departamentos;

    LOOP
        FETCH cs_departamentos
        INTO departments_record;

    EXIT WHEN cs_departamentos%NOTFOUND;
        dbms_output.put_line('ID do Departamento: ' || departments_record.department_id || ' - ' ||
                             'Nome: ' || departments_record.department_name || ' - ' ||
                             'ID do Gerente: ' || departments_record.manager_id);

    END LOOP;

    CLOSE cs_departamentos;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1c

Questão 1-c

Agora, utilizando um cursor com LOOP básico.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_departamentos IS
    SELECT department_id, department_name, manager_id FROM departments;
```

Declaramos um cursor chamado cs_departamentos com uma consulta que retorna os dados desejados da tabela departments.

```
departments_record cs_departamentos%ROWTYPE;
```

Criamos a variável departments_record do tipo %ROWTYPE para armazenar os dados de cada linha do cursor.

```
BEGIN
    OPEN cs_departamentos;
```

Abrimos o cursor para começar a leitura dos dados.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

1c

Questão 1-c

Agora, utilizando um cursor com LOOP básico.

Explicação e Resultado

Explicação passo a passo:

```
LOOP
    FETCH cs_departamentos
    INTO departments_record;
```

Dentro de um LOOP, buscamos um registro do cursor.

```
EXIT WHEN cs_departamentos%NOTFOUND;
```

Se não houver mais registros, encerramos o loop com EXIT.

```
dbms_output.put_line('ID do Departamento: ' || departments_record.department_id || ' - ' ||
                     'Nome: ' || departments_record.department_name || ' - ' ||
                     'ID do Gerente: ' || departments_record.manager_id);
```

Imprimimos as informações do departamento.

```
    END LOOP;
    CLOSE cs_departamentos;
END;
```

Fechamos o loop e encerramos o cursor e o bloco.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

1

EXERCÍCIO 1

O setor de planejamento estratégico deseja um relatório com os dados de todos os departamentos da empresa para realizar um estudo de realocação de equipes. O analista de dados foi encarregado de criar um script que percorra todos os registros da TABELA DEPARTMENTS, listando o ID, o nome do departamento e o ID do gerente.

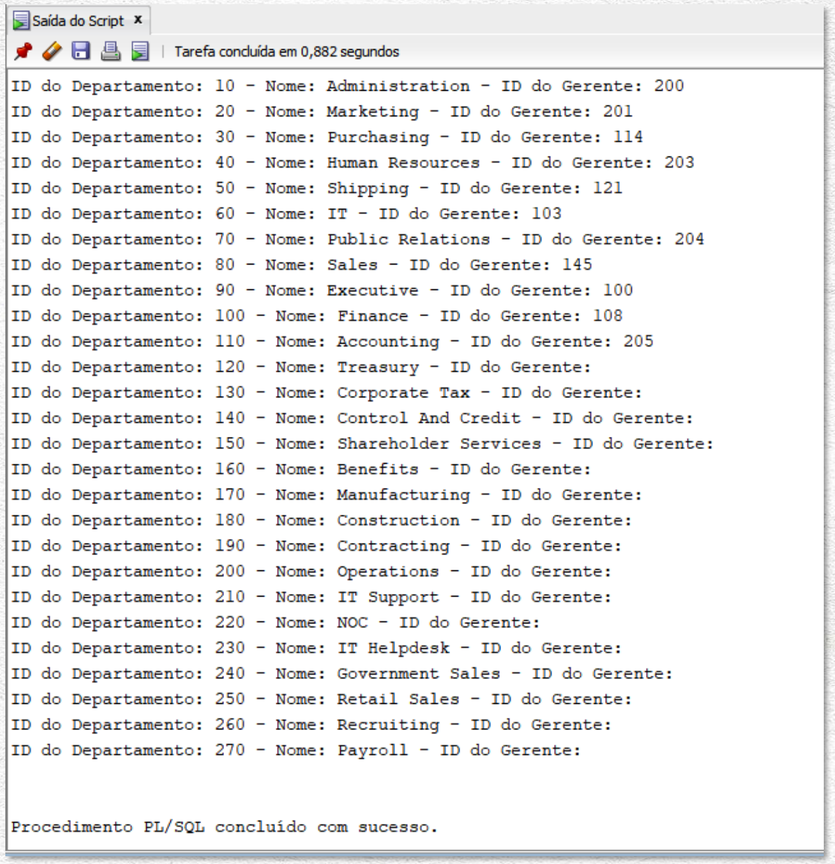
1c

Questão 1-c

Agora, utilizando um cursor com LOOP básico.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2a

Questão 2-a

Gere esse relatório utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Para resolver o exercício 2 com um **WHILE LOOP**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_enderecos(vcountry_id VARCHAR2) IS
    SELECT location_id, street_address, city, state_province
    FROM locations
    WHERE country_id = vcountry_id;

    locations_record cs_enderecos%ROWTYPE;

BEGIN
    OPEN cs_enderecos('US');

    FETCH cs_enderecos
    INTO locations_record;

    WHILE cs_enderecos%FOUND LOOP
        dbms_output.put_line('ID: ' || locations_record.location_id || ' - ' ||
                             'Endereço: ' || locations_record.street_address || ' - ' ||
                             'Cidade: ' || locations_record.city || ' - ' ||
                             'Estado/Província: ' || locations_record.state_province);

        FETCH cs_enderecos
        INTO locations_record;
    END LOOP;

    CLOSE cs_enderecos;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2a

Questão 2-a

Gere esse relatório utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_enderecos(vcountry_id VARCHAR2) IS
    SELECT location_id, street_address, city, state_province
    FROM locations
    WHERE country_id = vcountry_id;
```

Declaramos um cursor chamado cs_enderecos que recebe um country_id (armazenando-o no parâmetro vcountry_id), e retorna os campos desejados da tabela locations.

```
locations_record cs_enderecos%ROWTYPE;
```

Criamos a variável locations_record do tipo %ROWTYPE para armazenar os dados de cada linha do cursor.

```
BEGIN
    OPEN cs_enderecos('US');
```

Abrimos o cursor para começar a leitura dos dados, passando para ele o parâmetro a ser filtrado: 'US'.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2a

Questão 2-a

Gere esse relatório utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Explicação passo a passo:

```
FETCH cs_enderecos
INTO locations_record;
```

Buscamos o primeiro registro do cursor e armazenamos na variável locations_record.

```
WHILE cs_enderecos%FOUND LOOP
```

Iniciamos o WHILE LOOP. Ele só entra no loop se o último FETCH tiver retornado um registro com sucesso.

```
dbms_output.put_line('ID: ' || locations_record.location_id || ' - ' ||
                     'Endereço: ' || locations_record.street_address || ' - ' ||
                     'Cidade: ' || locations_record.city || ' - ' ||
                     'Estado/Província: ' || locations_record.state_province);
```

Imprimimos as informações dos endereços.

```
FETCH cs_enderecos
INTO locations_record;
```

Executamos o próximo FETCH para atualizar os dados da variável antes da próxima iteração do loop.

```
    END LOOP;
    CLOSE cs_enderecos;
END;
```

Fechamos o loop e encerramos o cursor e o bloco.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

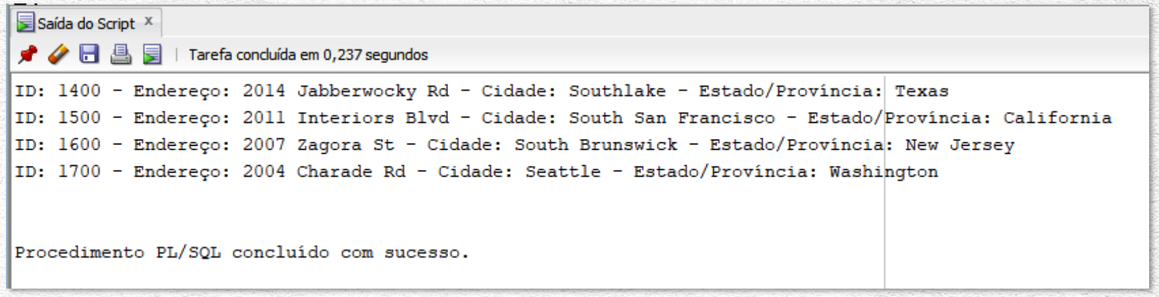
2a

Questão 2-a

Gere esse relatório utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2b

Questão 2-b

Faça o mesmo utilizando um cursor com LOOP básico.

Explicação e Resultado

Para resolver o exercício 2 com um **LOOP BÁSICO**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_enderecos(vcountry_id VARCHAR2) IS
    SELECT location_id, street_address, city, state_province
    FROM locations
    WHERE country_id = vcountry_id;

    locations_record cs_enderecos%ROWTYPE;

BEGIN
    OPEN cs_enderecos('US');

    LOOP
        FETCH cs_enderecos
        INTO locations_record;

    EXIT WHEN cs_enderecos%NOTFOUND;
        dbms_output.put_line('ID: ' || locations_record.location_id || ' - ' ||
                             'Endereço: ' || locations_record.street_address || ' - ' ||
                             'Cidade: ' || locations_record.city || ' - ' ||
                             'Estado/Província: ' || locations_record.state_province);

    END LOOP;

    CLOSE cs_enderecos;
END;
```

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

2 EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2b Questão 2-b

Faça o mesmo utilizando um cursor com LOOP básico.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_enderecos(vcountry_id VARCHAR2) IS
    SELECT location_id, street_address, city, state_province
    FROM locations
    WHERE country_id = vcountry_id;
```

Declaramos um cursor chamado cs_enderecos que recebe um country_id (armazenando-o no parâmetro vcountry_id), e retorna os campos desejados da tabela locations.

```
locations_record cs_enderecos%ROWTYPE;
```

Criamos a variável locations_record do tipo %ROWTYPE para armazenar os dados de cada linha do cursor.

```
BEGIN
    OPEN cs_enderecos('US');
```

Abrimos o cursor para começar a leitura dos dados, passando para ele o parâmetro a ser filtrado: 'US'.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2b

Questão 2-b

Faça o mesmo utilizando um cursor com LOOP básico.

Explicação e Resultado

Explicação passo a passo:

```
LOOP
    FETCH cs_enderecos
    INTO locations_record;
```

Dentro de um LOOP, buscamos um registro do cursor.

```
EXIT WHEN cs_enderecos%NOTFOUND;
```

Se não houver mais registros, encerramos o loop com EXIT.

```
dbms_output.put_line('ID: ' || locations_record.location_id || ' - ' ||
                     'Endereço: ' || locations_record.street_address || ' - ' ||
                     'Cidade: ' || locations_record.city || ' - ' ||
                     'Estado/Província: ' || locations_record.state_province);
```

Imprimimos as informações dos endereços.

```
    END LOOP;
    CLOSE cs_enderecos;
END;
```

Fechamos o loop e encerramos o cursor e o bloco.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

2 EXERCÍCIO 2

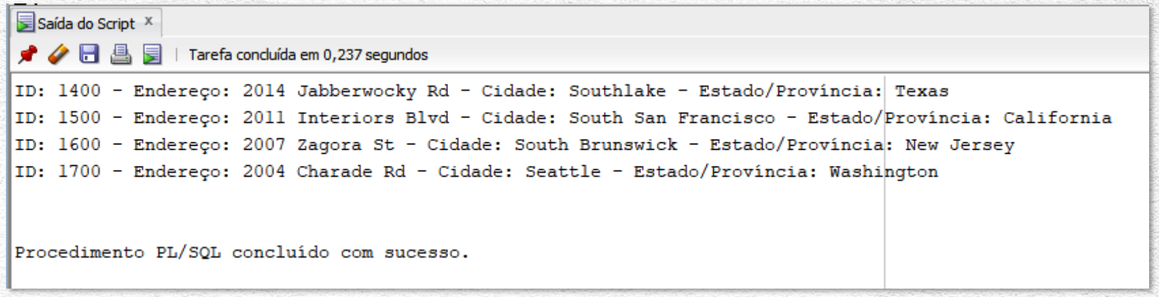
O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2b Questão 2-b

Faça o mesmo utilizando um cursor com LOOP básico.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2c

Questão 2-c

Agora, utilizando um cursor com FOR LOOP.

Explicação e Resultado

Por fim, para resolver o exercício 2 com um **FOR LOOP**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_enderecos(vcountry_id VARCHAR2) IS
    SELECT location_id, street_address, city, state_province
    FROM locations
    WHERE country_id = vcountry_id;

BEGIN
    FOR locations_record IN cs_enderecos('US') LOOP
        dbms_output.put_line('ID: ' || locations_record.location_id || ' - ' ||
                             'Endereço: ' || locations_record.street_address || ' - ' ||
                             'Cidade: ' || locations_record.city || ' - ' ||
                             'Estado/Província: ' || locations_record.state_province);

    END LOOP;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2c

Questão 2-c

Agora, utilizando um cursor com FOR LOOP.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_enderecos(vcountry_id VARCHAR2) IS
    SELECT location_id, street_address, city, state_province
    FROM locations
    WHERE country_id = vcountry_id;
```

Declaramos um cursor chamado cs_enderecos que recebe um country_id (armazenando-o no parâmetro vcountry_id), e retorna os campos desejados da tabela locations.

```
BEGIN
    FOR locations_record IN cs_enderecos('US') LOOP
```

Iniciamos um FOR LOOP que percorre automaticamente os registros do cursor procurando pelo parâmetro 'US'. A variável locations_record é criada implicitamente com o tipo da linha retornada pelo cursor.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

2c

Questão 2-c

Agora, utilizando um cursor com FOR LOOP.

Explicação e Resultado

Explicação passo a passo:

```
dbms_output.put_line('ID: ' || locations_record.location_id || ' - ' ||
                     'Endereço: ' || locations_record.street_address || ' - ' ||
                     'Cidade: ' || locations_record.city || ' - ' ||
                     'Estado/Província: ' || locations_record.state_province);
```

Imprimimos as informações dos endereços.

```
    END LOOP;
END;
```

Finalizamos o loop e encerramos o bloco.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

2

EXERCÍCIO 2

O setor de logística está reavaliando a distribuição de seus centros operacionais nos Estados Unidos. Para isso, o time de dados precisa gerar relatórios contendo os endereços completos (location_id, street_address, city, state_province) das unidades que estão localizadas nesse país (representado pelo country_id = 'US' na TABELA LOCATIONS). Esse relatório será usado para avaliar custos e alcance de transporte.

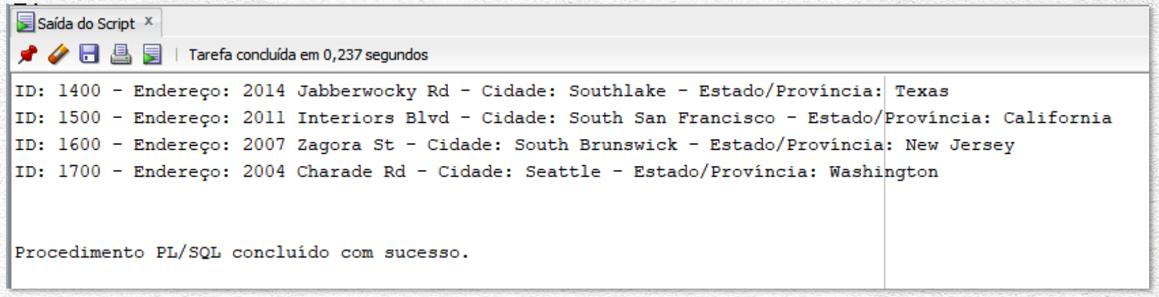
2c

Questão 2-c

Agora, utilizando um cursor com FOR LOOP.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3a

Questão 3-a

Gere esse relatório utilizando um cursor com LOOP básico.

Explicação e Resultado

Para resolver o exercício 3 com um **LOOP BÁSICO**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_hist_cargos(vemp_id NUMBER) IS
    SELECT start_date, end_date, job_id, department_id
    FROM job_history
    WHERE employee_id = vemp_id;

    job_history_record cs_hist_cargos%ROWTYPE;
BEGIN
    OPEN cs_hist_cargos(176);

    LOOP
        FETCH cs_hist_cargos
        INTO job_history_record;

        EXIT WHEN cs_hist_cargos%NOTFOUND;
        dbms_output.put_line('Início: ' || job_history_record.start_date || ' - ' ||
                             'Fim: ' || job_history_record.end_date || ' - ' ||
                             'Cargo: ' || job_history_record.job_id || ' - ' ||
                             'Departamento: ' || job_history_record.department_id);
    END LOOP;

    CLOSE cs_hist_cargos;
END;
```

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

3 EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3a Questão 3-a

Gere esse relatório utilizando um cursor com LOOP básico.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_hist_cargos(vemp_id NUMBER) IS
    SELECT start_date, end_date, job_id, department_id
    FROM job_history
    WHERE employee_id = vemp_id;
```

Declaramos um cursor chamado cs_hist_cargos que recebe um employee_id (armazenando-o no parâmetro vemp_id), e retorna os campos desejados da tabela job_history.

```
job_history_record cs_hist_cargos%ROWTYPE;
```

Criamos a variável job_history_record do tipo %ROWTYPE para armazenar os dados de cada linha do cursor.

```
BEGIN
    OPEN cs_hist_cargos(176);
```

Abrimos o cursor para começar a leitura dos dados, passando para ele o parâmetro a ser filtrado: 176.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3a

Questão 3-a

Gere esse relatório utilizando um cursor com LOOP básico.

Explicação e Resultado

Explicação passo a passo:

```
LOOP
    FETCH cs_hist_cargos
    INTO job_history_record;
```

Dentro de um LOOP, buscamos um registro do cursor.

```
EXIT WHEN cs_hist_cargos%NOTFOUND;
```

Se não houver mais registros, encerramos o loop com EXIT.

```
dbms_output.put_line('Inicio: ' || job_history_record.start_date || ' - ' ||
                     'Fim: ' || job_history_record.end_date || ' - ' ||
                     'Cargo: ' || job_history_record.job_id || ' - ' ||
                     'Departamento: ' || job_history_record.department_id);
```

Imprimimos as informações das cargos anteriores do funcionário.

```
    END LOOP;
    CLOSE cs_hist_cargos;
END;
```

Fechamos o loop e encerramos o cursor e o bloco.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

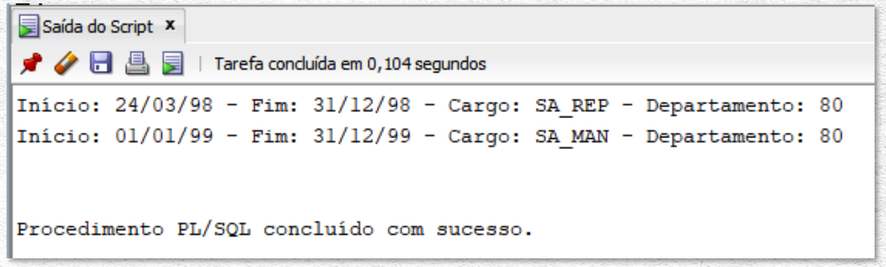
3a

Questão 3-a

Gere esse relatório utilizando um cursor com LOOP básico.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3b

Questão 3-b

Faça o mesmo utilizando um cursor com FOR LOOP.

Explicação e Resultado

Agora, para resolver o exercício 3 com um **FOR LOOP**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_hist_cargos(vemp_id NUMBER) IS
    SELECT start_date, end_date, job_id, department_id
    FROM job_history
    WHERE employee_id = vemp_id;

BEGIN
    FOR job_history_record IN cs_hist_cargos(176) LOOP
        dbms_output.put_line('Inicio: ' || job_history_record.start_date || ' - ' ||
                             'Fim: ' || job_history_record.end_date || ' - ' ||
                             'Cargo: ' || job_history_record.job_id || ' - ' ||
                             'Departamento: ' || job_history_record.department_id);
    END LOOP;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3 EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3b Questão 3-b

Faça o mesmo utilizando um cursor com FOR LOOP.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_hist_cargos(vemp_id NUMBER) IS
    SELECT start_date, end_date, job_id, department_id
    FROM job_history
    WHERE employee_id = vemp_id;
```

Declaramos um cursor chamado cs_hist_cargos que recebe um employee_id (armazenando-o no parâmetro vemp_id), e retorna os campos desejados da tabela job_history.

```
BEGIN
    FOR job_history_record IN cs_hist_cargos(176) LOOP
```

Iniciamos um FOR LOOP que percorre automaticamente os registros do cursor procurando pelo parâmetro 176. A variável job_history_record é criada implicitamente com o tipo da linha retornada pelo cursor.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3b

Questão 3-b

Faça o mesmo utilizando um cursor com FOR LOOP.

Explicação e Resultado

Explicação passo a passo:

```
dbms_output.put_line('Inicio: ' || job_history_record.start_date || ' - ' ||
                     'Fim: ' || job_history_record.end_date || ' - ' ||
                     'Cargo: ' || job_history_record.job_id || ' - ' ||
                     'Departamento: ' || job_history_record.department_id);
```

Imprimimos as informações das cargos anteriores do funcionário.

```
    END LOOP;
END;
```

Finalizamos o loop e encerramos o bloco.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

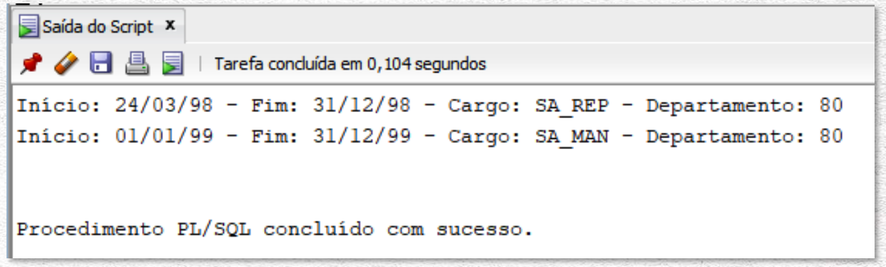
3b

Questão 3-b

Faça o mesmo utilizando um cursor com FOR LOOP.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3c

Questão 3-c

Agora, utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Por fim, para resolver o exercício 3 com um **WHILE LOOP**, criamos o seguinte código:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR cs_hist_cargos(vemp_id NUMBER) IS
    SELECT start_date, end_date, job_id, department_id
    FROM job_history
    WHERE employee_id = vemp_id;

    job_history_record cs_hist_cargos%ROWTYPE;
BEGIN
    OPEN cs_hist_cargos(176);

    FETCH cs_hist_cargos
    INTO job_history_record;

    WHILE cs_hist_cargos%FOUND LOOP
        dbms_output.put_line('Início: ' || job_history_record.start_date || ' - ' ||
                             'Fim: ' || job_history_record.end_date || ' - ' ||
                             'Cargo: ' || job_history_record.job_id || ' - ' ||
                             'Departamento: ' || job_history_record.department_id);

        FETCH cs_hist_cargos
        INTO job_history_record;

    END LOOP;
    CLOSE cs_hist_cargos;
END;
```

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3 EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3c Questão 3-c

Agora, utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Explicação passo a passo:

```
SET SERVEROUTPUT ON;
```

Ativamos a exibição de mensagens no console via dbms_output.put_line.

```
DECLARE
    CURSOR cs_hist_cargos(vemp_id NUMBER) IS
    SELECT start_date, end_date, job_id, department_id
    FROM job_history
    WHERE employee_id = vemp_id;
```

Declaramos um cursor chamado cs_hist_cargos que recebe um employee_id (armazenando-o no parâmetro vemp_id), e retorna os campos desejados da tabela job_history.

```
job_history_record cs_hist_cargos%ROWTYPE;
```

Criamos a variável job_history_record do tipo %ROWTYPE para armazenar os dados de cada linha do cursor.

```
BEGIN
    OPEN cs_hist_cargos(176);
```

Abrimos o cursor para começar a leitura dos dados, passando para ele o parâmetro a ser filtrado: 176.

CONTINUA NA PRÓXIMA PÁGINA...

GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

3c

Questão 3-c

Agora, utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Explicação passo a passo:

```
FETCH cs_hist_cargos
INTO job_history_record;
```

Buscamos o primeiro registro do cursor e armazenamos na variável job_history_record.

```
WHILE cs_hist_cargos%FOUND LOOP
```

Iniciamos o WHILE LOOP. Ele só entra no loop se o último FETCH tiver retornado um registro com sucesso.

```
dbms_output.put_line('Inicio: ' || job_history_record.start_date || ' - ' ||
                     'Fim: ' || job_history_record.end_date || ' - ' ||
                     'Cargo: ' || job_history_record.job_id || ' - ' ||
                     'Departamento: ' || job_history_record.department_id);
```

Imprimimos as informações das cargos anteriores do funcionário.

```
FETCH cs_hist_cargos
INTO job_history_record;
```

Executamos o próximo FETCH para atualizar os dados da variável antes da próxima iteração do loop.

```
    END LOOP;
    CLOSE cs_hist_cargos;
END;
```

Fechamos o loop e encerramos o cursor e o bloco.

CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

EXERCÍCIO 3

O setor de Recursos Humanos deseja auditar os cargos anteriores de um funcionário específico (Jonathon Taylor, employee_id = 176). Esse processo é parte de uma verificação de compliance para promoções internas. O colaborador será identificado pelo employee_id (176), e o relatório deve conter os dados de período, cargo e departamento de cada passagem registrada na TABELA JOB_HISTORY.

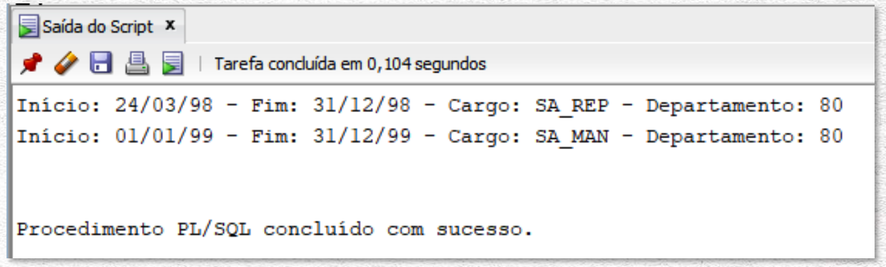
3c

Questão 3-c

Agora, utilizando um cursor com WHILE LOOP.

Explicação e Resultado

Executando o curso, ele retorna o seguinte resultado, linha a linha:



MÓDULO 21

SEGURANÇA

SEGURANÇA

SEGURANÇA

O QUE VEREMOS NESTE MÓDULO

MÓDULO 21 SEGURANÇA

01	GRANT/REVOKE	1060
02	USUÁRIOS (CRIAR, ALTERAR, EXCLUIR)	1062
03	CONCEDENDO PRIVILÉGIOS AO USUÁRIO	1063
04	RESUMO DO MÓDULO	1064
05	EXERCÍCIOS	1065
06	GABARITO EXPLICATIVO	1070

GRANT / REVOKE

Podemos conceder permissões (GRANT) aos usuários ou retirar-lhes (REVOKE) a uma série de objetos dentro de um banco de dados.

Você pode conceder aos usuários privilégios aos objetos, como por exemplo, às tabelas.
Esses privilégios podem ser uma combinação de SELECT, INSERT, UPDATE, DELETE, ALTER, REFERENCES, INDEX, ALL.

SINTAXE:

GRANT privilegio **TO** usuario;

REVOKE privilegio **FROM** usuario;

GRANT / REVOKE

Os PRIVILÉGIOS podem ser inúmeros, listamos ao lado alguns deles:

PRIVILÉGIOS:	POSSIBILIDADE DE:
SELECT	Selecionar tabelas.
INSERT	Inserir dados em tabelas.
UPDATE	Atualizar tabelas.
DELETE	Excluir dados de tabelas.
REFERENCES	Criar Constraints que fazem referência a uma tabela.
ALTER	Utilizar comandos do tipo ALTER TABLE.
INDEX	Criar índices em uma tabela.
ALL	Todos os privilégios de uma tabela.
EXECUTE	Possibilidade de executar uma Function ou Procedure.
CREATE SESSION	Conectar-se ao banco de dados.
CREATE TABLE	Criar uma Tabela.
CREATE VIEW	Criar uma View.
CREATE ANY TRIGGER	Criar uma Trigger.
CREATE ANY PROCEDURE	Criar uma Procedure.
CREATE ANY FUNCTION	Criar uma Function.
CREATE SEQUENCE	Criar uma Sequence.

USUÁRIOS (criar, alterar, excluir)

Um USUÁRIO é uma conta de banco de dados que permite a uma pessoa fazer um login no sistema.

SINTAXE:

1. Criando um usuário:

CREATE USER nome_usuario
IDENTIFIED BY senha;

```
CREATE USER teste
    IDENTIFIED BY 123;
```

2. Alterando a senha de um usuário:

ALTER USER nome_usuario
IDENTIFIED BY senha;

```
ALTER USER teste
    IDENTIFIED BY 456;
```

3. Excluindo usuário:

DROP USER nome_usuario;

```
DROP USER teste;
```

CONCEDENDO PRIVILÉGIOS AO USUÁRIO

Recapitulando, podemos conceder permissões (GRANT) aos usuários ou retirar-lhes (REVOKE) a uma série de objetos:

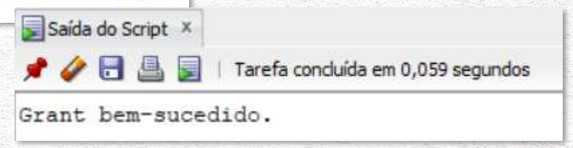
Conforme vimos, a SINTAXE destes comandos é a seguinte:

GRANT privilegio **TO** usuario; / **REVOKE** privilegio **FROM** usuario;

Supondo que temos um usuário chamado TESTE criado pelo ADMIN do nosso banco de dados, vejamos alguns exemplos:

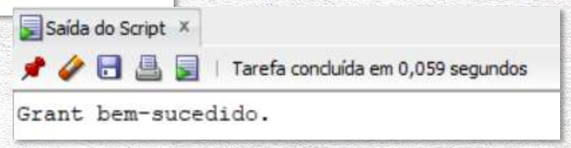
1) Se quisermos permitir ao usuário TESTE criar uma conexão:

```
GRANT CREATE SESSION TO teste;
```



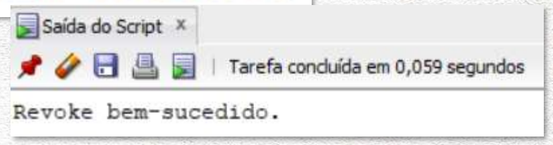
2) Ou, se quisermos permitir ao usuário TESTE criar tabelas:

```
GRANT CREATE TABLE TO teste;
```



3) Por fim, se quisermos retirar do usuário TESTE a permissão para criar tabelas:

```
REVOKE CREATE TABLE FROM teste;
```

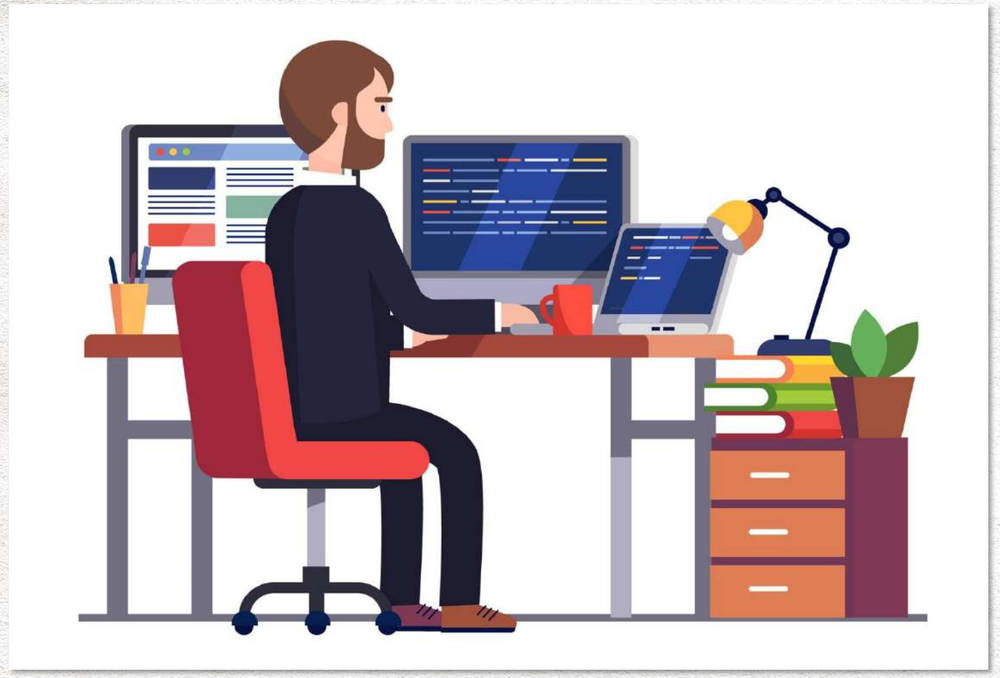


Resumo do Módulo

Algumas dicas, boas práticas e cuidados ao realizar as consultas:

1. Os comandos DCL GRANT e REVOKE nos permitem configurar as permissões de usuários de um banco de dados.
2. GRANT concede privilégios a um usuário.
3. REVOKE retira privilégios de um usuário.
4. Podemos criar, alterar e excluir um usuário utilizando, respectivamente, os comandos CREATE USER, ALTER USER e DROP USER.
5. Também podemos criar usuários e conceder/retirar privilégios pela própria INTERFACE do SQL Developer (como já fizemos).

EXERCÍCIOS



Você trabalha no setor de Segurança da Informação da empresa fictícia Hashtag S/A, especializada em soluções tecnológicas.

Como parte da rotina de auditoria e controle de acessos ao banco de dados Oracle, sua equipe está realizando uma simulação controlada de criação, concessão e revogação de privilégios de usuários no ambiente de testes.

O objetivo dessa simulação é reforçar os protocolos de segurança da empresa, garantir que apenas usuários autorizados tenham acesso a operações sensíveis (como criação de tabelas ou sessões), e revisar os procedimentos de concessão e remoção de permissões.

Nos próximos exercícios, você assumirá o papel de um administrador do banco de dados (usando a conexão ADMIN) e também testará as permissões concedidas a um novo usuário criado exclusivamente para esse processo (USUARIO_HASHTAG).

Cada passo representa um teste ou ajuste necessário para garantir que os acessos estejam sendo controlados corretamente.

Atenção: é essencial que você siga cada passo na ordem apresentada, sem pular etapas, para que todos os testes funcionem corretamente e os resultados reflitam a realidade de um ambiente seguro e controlado.

EXERCÍCIOS

1

Questão 1

Usando a conexão ADMIN, crie um usuário chamado USUARIO_HASHTAG.

2

Questão 2

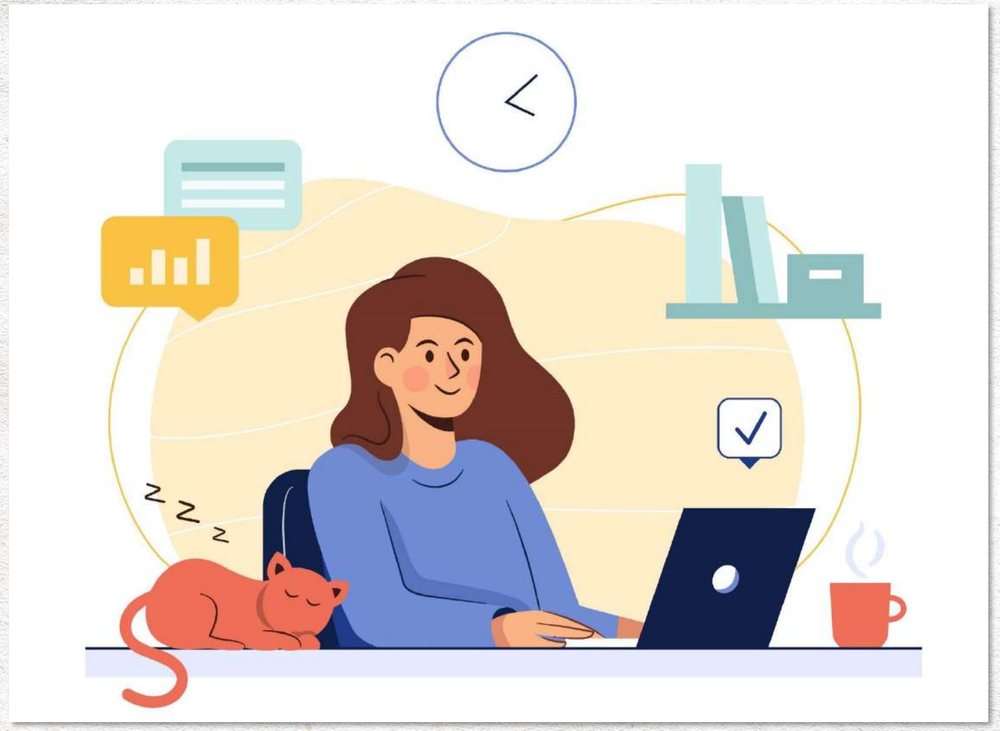
Usando a conexão ADMIN, altere a senha do usuário USUARIO_HASHTAG.

3

Questão 3

Manualmente, tente criar uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

EXERCÍCIOS



4

Questão 4

Vimos que a tentativa de criar uma conexão para o usuário USUARIO_HASHTAG falhou, pois este usuário não possui a permissão necessária para tanto. Sendo assim, usando a conexão ADMIN, conceda ao usuário USUARIO_HASHTAG permissão para criar conexões.

5

Questão 5

Concedida a permissão necessária, agora, manualmente, crie uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

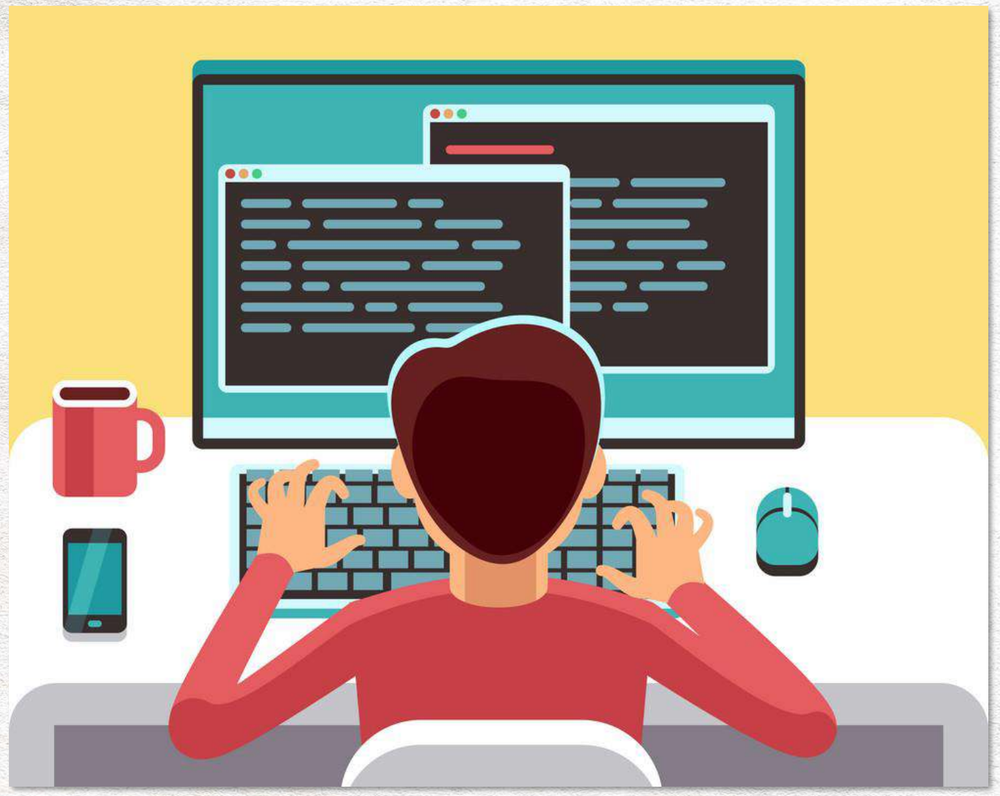
6

Questão 6

Usando a conexão CONEXAO_HASHTAG, tente criar a seguinte tabela:

```
CREATE TABLE alunos(
    id INT,
    nome VARCHAR2(100)
);
```

EXERCÍCIOS



7

Questão 7

Vimos que a tentativa de criar uma tabela na conexão CONEXAO_HASHTAG falhou, pois o usuário USUARIO_HASHTAG não possui a permissão necessária para tanto. Sendo assim, usando a conexão ADMIN, conceda ao usuário USUARIO_HASHTAG permissão para criar tabelas.

8

Questão 8

Concedida a permissão necessária, agora, na conexão CONEXAO_HASHTAG, crie a tabela alunos:

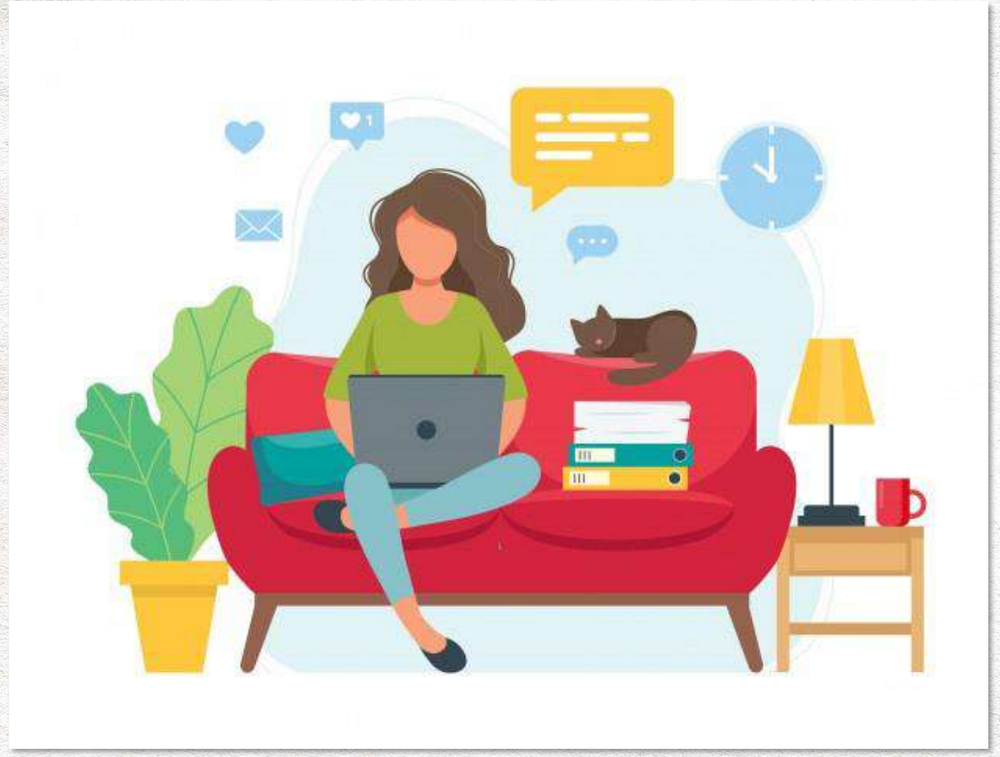
```
CREATE TABLE alunos(
    id INT,
    nome VARCHAR2(100)
);
```

9

Questão 9

Usando a conexão CONEXAO_HASHTAG, exclua a tabela alunos.

EXERCÍCIOS



10

Questão 10

Usando a conexão ADMIN, revogue a permissão para criar tabelas anteriormente concedida ao usuário USUARIO_HASHTAG.

11

Questão 11

Manualmente, desconecte-se da conexão CONEXAO_HASHTAG (no menu lateral esquerdo, clique com o botão direito sobre o nome da conexão CONEXAO_HASHTAG e selecione a opção "Desconectar").

12

Questão 12

Encerrada a conexão, exclua o usuário USUARIO_HASHTAG.

GABARITO EXPLICATIVO

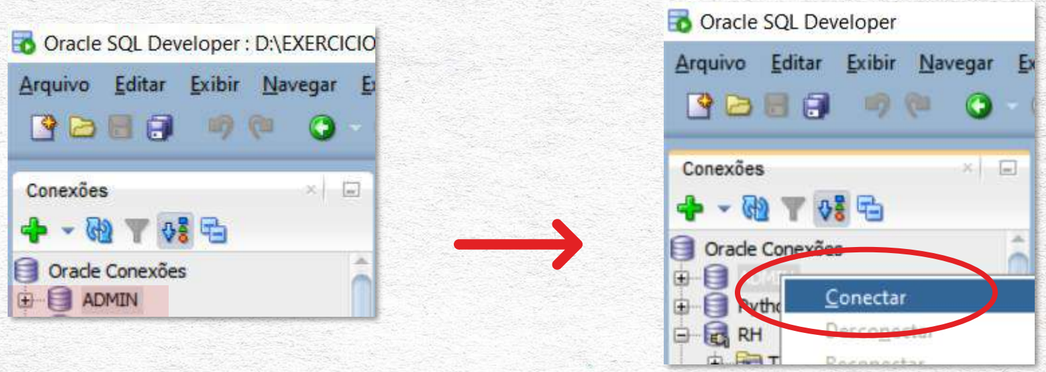
1

Questão 1

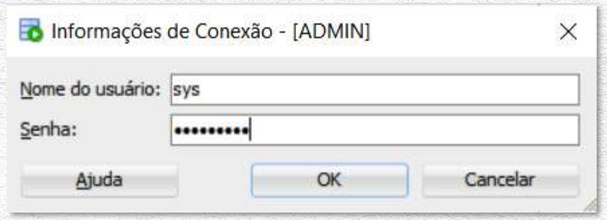
Usando a conexão ADMIN, crie um usuário chamado USUARIO_HASHTAG.

Explicação e Resultado

Primeiramente, conectamos à conexão **ADMIN**, clicando com o botão direito do mouse sobre ela e selecionando a opção **Conectar**:



Informamos a senha da conexão **ADMIN** e clicamos em **OK**:



CONTINUA NA PRÓXIMA PÁGINA..

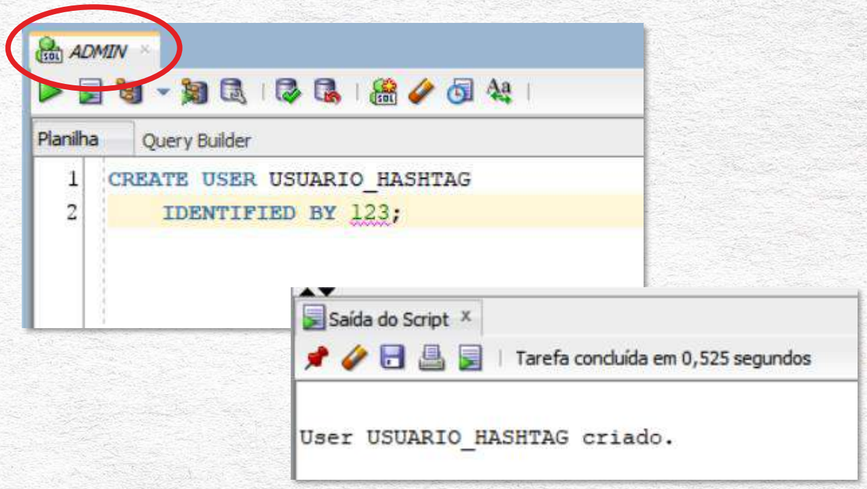
GABARITO EXPLICATIVO

1 Questão 1

Usando a conexão ADMIN, crie um usuário chamado USUARIO_HASHTAG.

Explicação e Resultado

Conectados à **ADMIN**, executamos o código abaixo para criarmos o usuário **USUARIO_HASHTAG**. No exemplo, a senha inicial deste usuário será **123**:



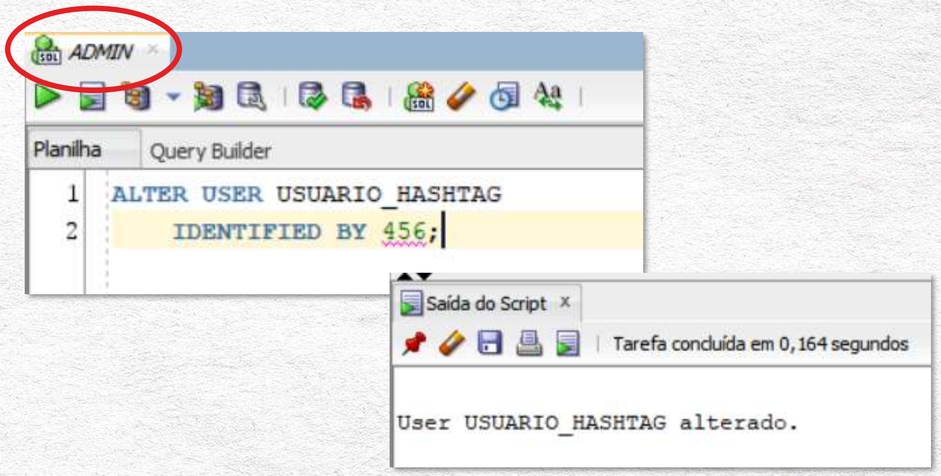
GABARITO EXPLICATIVO

2 Questão 2

Usando a conexão ADMIN, altere a senha do usuário USUARIO_HASHTAG.

Explicação e Resultado

Para alterarmos a senha do usuário que acabamos de criar, na conexão **ADMIN**, executamos o código abaixo, informando a nova senha do usuário **USUARIO_HASHTAG** que, no caso, será **456**:



GABARITO EXPLICATIVO

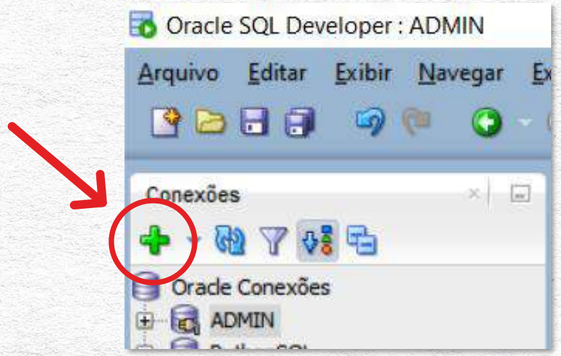
3

Questão 3

Manualmente, tente criar uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

Explicação e Resultado

Para tentarmos criar uma conexão para o usuário **USUARIO_HASHTAG**, clicamos no ícone mostrado abaixo, no menu lateral esquerdo do SQL Developer:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Manualmente, tente criar uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

Explicação e Resultado

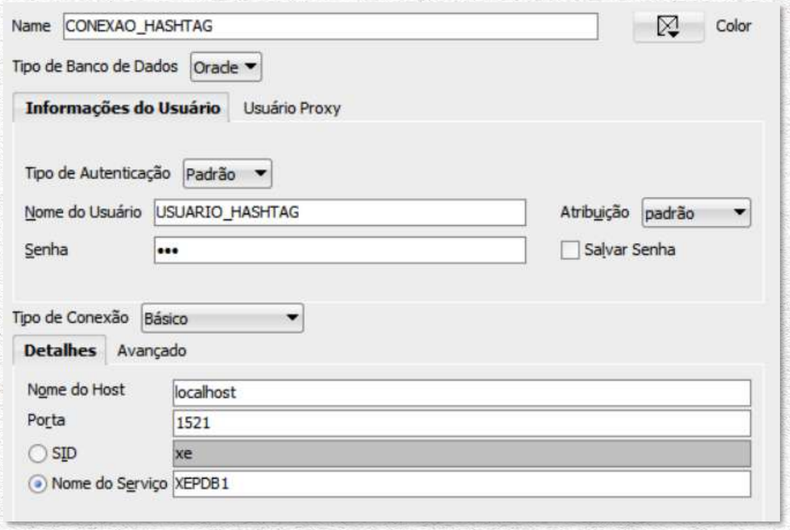
Na janela que se abrir, digitamos os dados dessa nova conexão:

Name: **CONEXAO_HASHTAG**

Nome do Usuário: **USUARIO_HASHTAG**

Senha: **456** (ou a que você definiu no exercício 2)

Selecionamos **Nome do Serviço** e digitamos **XEPDB1:**



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

3

Questão 3

Manualmente, tente criar uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

Explicação e Resultado

Clicamos em **Testar**.

Com isso, obteremos o seguinte **erro**:

Isso ocorreu porque nosso usuário **USUARIO_HASHTAG** ainda **não possui** a permissão necessária para criar conexão (**CREATE SESSION**).

Clicamos em **Cancelar**:

GABARITO EXPLICATIVO

4

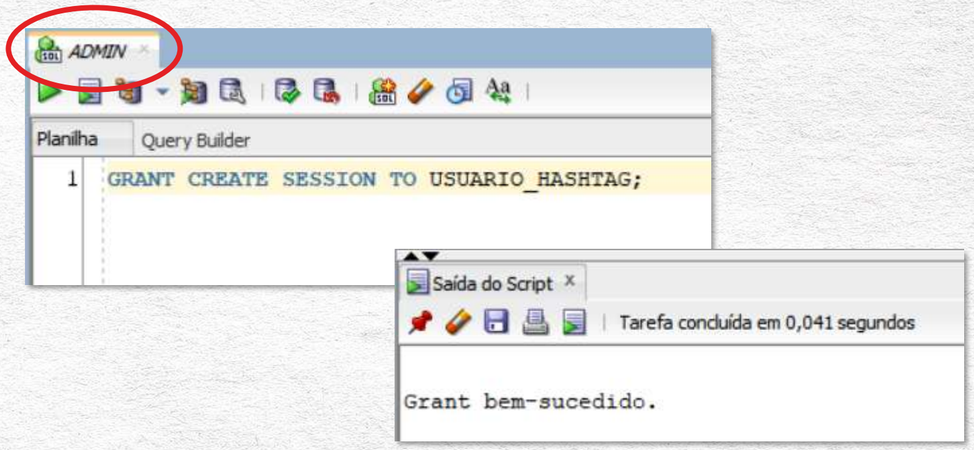
Questão 4

Vimos que a tentativa de criar uma conexão para o usuário USUARIO_HASHTAG falhou, pois este usuário não possui a permissão necessária para tanto.

Sendo assim, usando a conexão ADMIN, conceda ao usuário USUARIO_HASHTAG permissão para criar conexões.

Explicação e Resultado

Usando a conexão **ADMIN**, para conceder permissão para criar conexão (**CREATE SESSION**) ao usuário **USUARIO_HASHTAG**, executamos o código abaixo:



GABARITO EXPLICATIVO

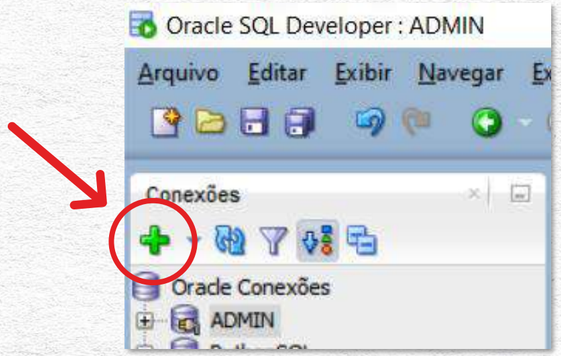
5

Questão 5

Concedida a permissão necessária, agora, manualmente, crie uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

Explicação e Resultado

Para criarmos a conexão **CONEXAO_HASHTAG** ao usuário **USUARIO_HASHTAG**, clicamos no ícone mostrado abaixo, no menu lateral esquerdo do SQL Developer:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

Questão 5

Concedida a permissão necessária, agora, manualmente, crie uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

Explicação e Resultado

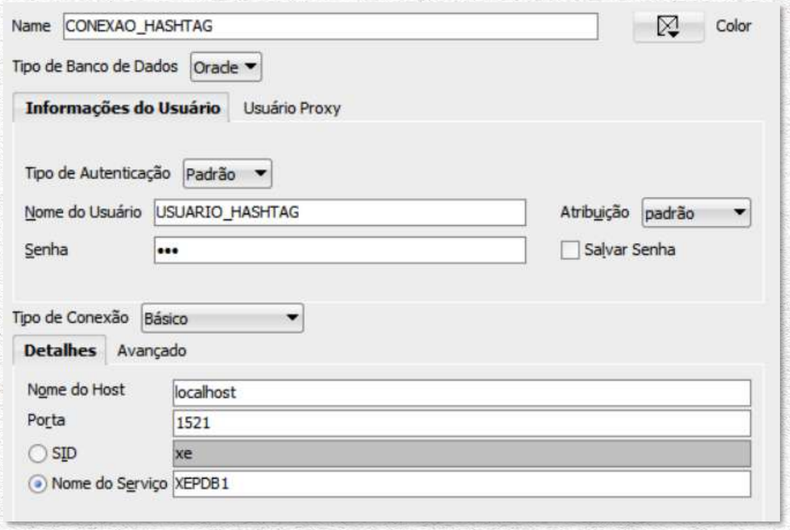
Na janela que se abrir, digitamos os dados dessa nova conexão:

Name: **CONEXAO_HASHTAG**

Nome do Usuário: **USUARIO_HASHTAG**

Senha: **456** (ou a que você definiu no exercício 2)

Selecionamos **Nome do Serviço** e digitamos **XEPDB1:**



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

5

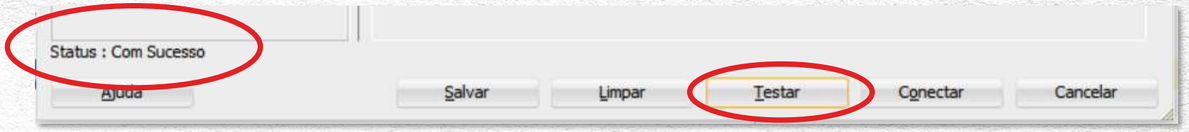
Questão 5

Concedida a permissão necessária, agora, manualmente, crie uma conexão chamada CONEXAO_HASHTAG para o usuário USUARIO_HASHTAG.

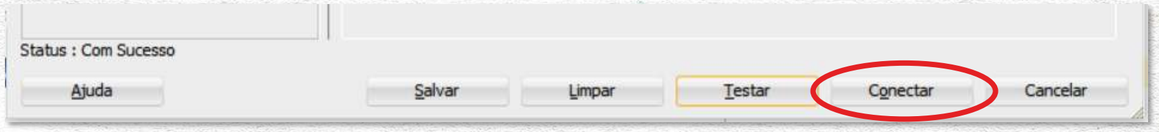
Explicação e Resultado

Clicamos em **Testar**.

Com isso, obteremos a mensagem **Status: Com Sucesso**, pois agora o usuário **USUARIO_HASHTAG** possui a permissão necessária para criar conexão:



Clicamos em **Conectar** para criar a conexão **CONEXAO_HASHTAG**:



Repare que, no menu lateral esquerdo, aparecerá o nome da **nova conexão**:

GABARITO EXPLICATIVO

6

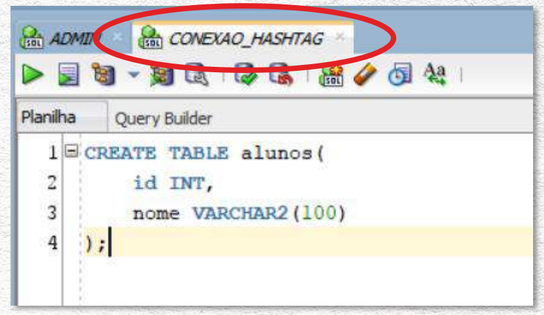
Questão 6

Usando a conexão CONEXAO_HASHTAG, tente criar a seguinte tabela:

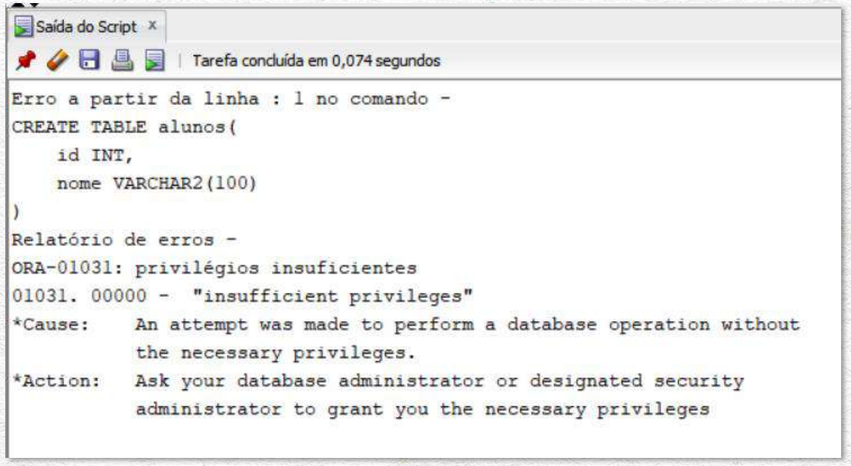
```
CREATE TABLE alunos(
    id INT,
    nome VARCHAR2(100)
);
```

Explicação e Resultado

Na conexão **CONEXAO_HASHTAG**, executamos o código ao lado para tentar criar a tabela **alunos**:



Porém, obteremos o erro abaixo, pois o usuário **USUARIO_HASHTAG não tem permissão** para criar tabelas (**CREATE TABLE**):



GABARITO EXPLICATIVO

7

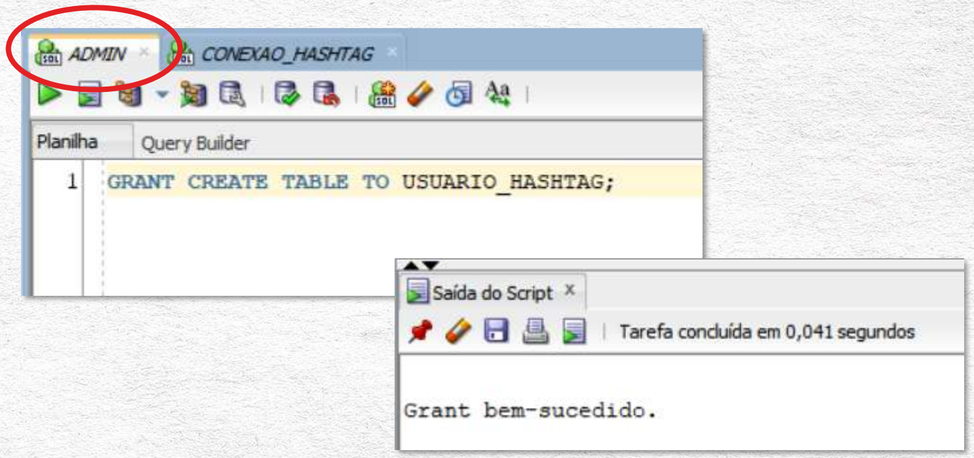
Questão 7

Vimos que a tentativa de criar uma tabela na conexão CONEXAO_HASHTAG falhou, pois o usuário USUARIO_HASHTAG não possui a permissão necessária para tanto.

Sendo assim, usando a conexão ADMIN, conceda ao usuário USUARIO_HASHTAG permissão para criar tabelas.

Explicação e Resultado

Usando a conexão **ADMIN**, para conceder permissão para criar tabelas (**CREATE TABLE**) ao usuário **USUARIO_HASHTAG**, executamos o código abaixo:



GABARITO EXPLICATIVO

8

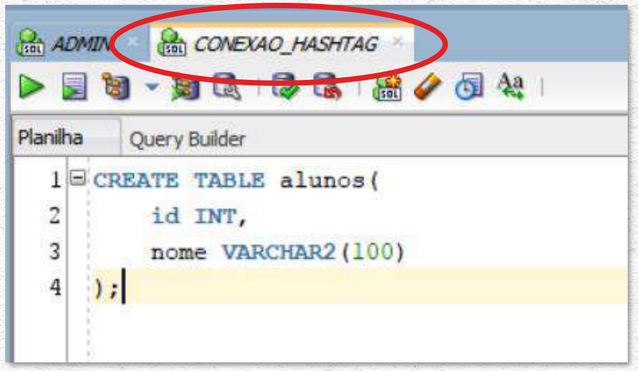
Questão 8

Concedida a permissão necessária, agora, na conexão CONEXAO_HASHTAG, crie a tabela alunos:

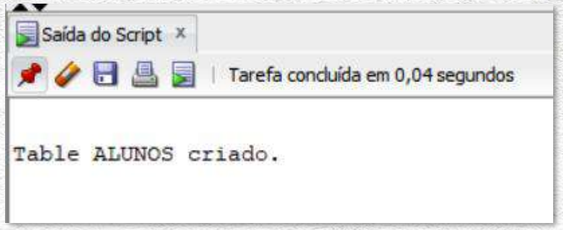
```
CREATE TABLE alunos(
    id INT,
    nome VARCHAR2(100)
);
```

Explicação e Resultado

Na conexão **CONEXAO_HASHTAG**, executamos o código abaixo para criar a tabela **alunos**:



E **desta vez conseguimos**, pois agora o usuário **USUARIO_HASHTAG possui a permissão necessária** para criar tabelas:



GABARITO EXPLICATIVO

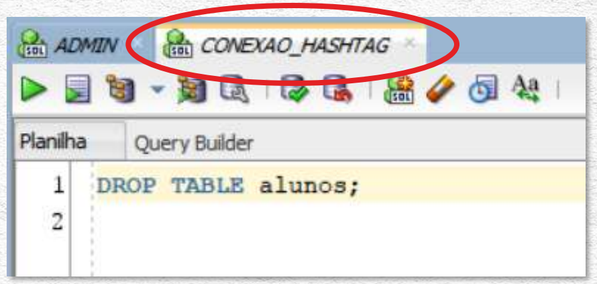
9

Questão 9

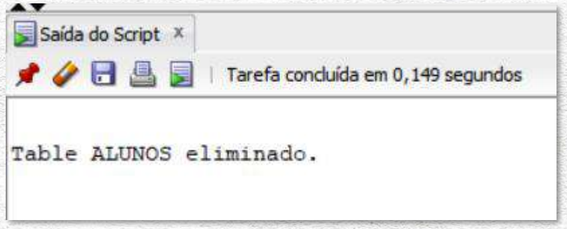
Usando a conexão CONEXAO_HASHTAG, exclua a tabela alunos.

Explicação e Resultado

Na conexão **CONEXAO_HASHTAG**, executamos o código abaixo para excluir a tabela **alunos**:



E a tabela será excluída:



GABARITO EXPLICATIVO

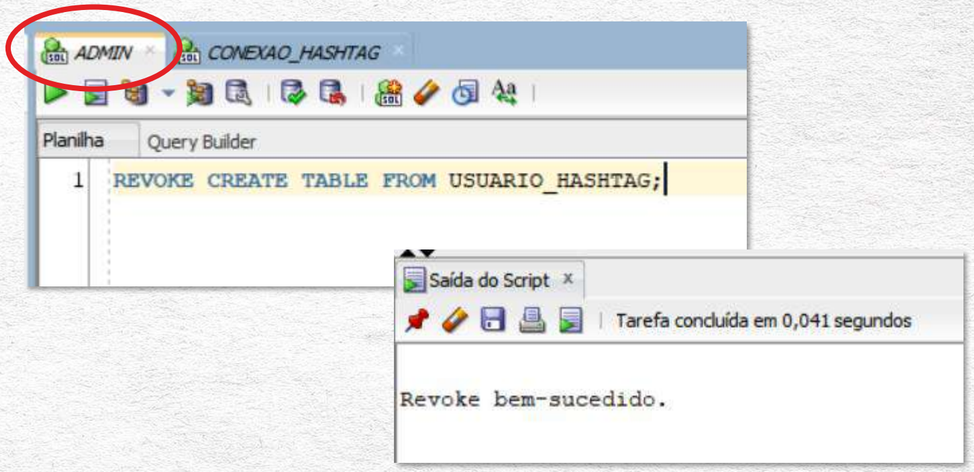
10

Questão 10

Usando a conexão ADMIN, revogue a permissão para criar tabelas anteriormente concedida ao usuário USUARIO_HASHTAG.

Explicação e Resultado

Usando a conexão **ADMIN**, revogamos a permissão para o usuário **USUARIO_HASHTAG** criar tabelas (**CREATE SESSION**) executando o seguinte código:



GABARITO EXPLICATIVO

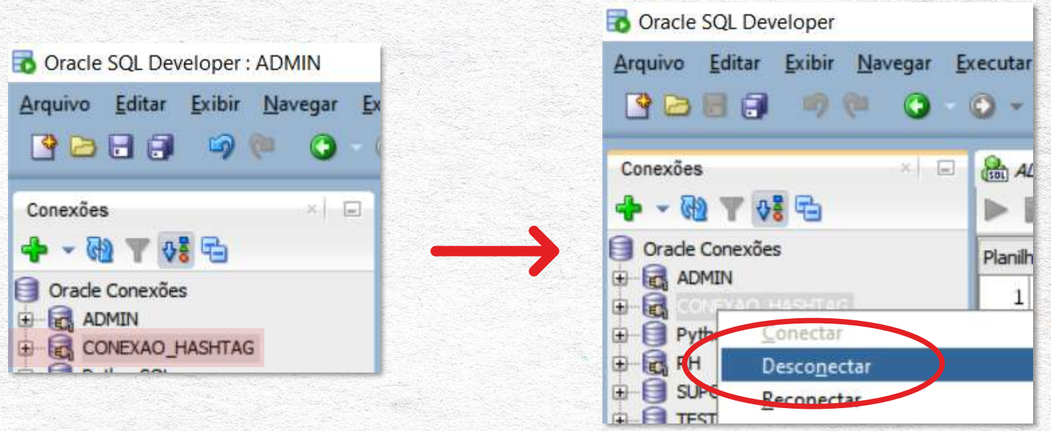
11

Questão 11

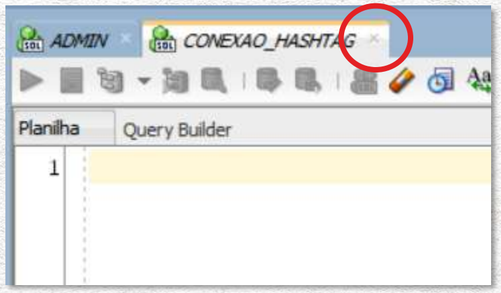
Manualmente, desconecte-se da conexão CONEXAO_HASHTAG (no menu lateral esquerdo, clique com o botão direito sobre o nome da conexão CONEXAO_HASHTAG e selecione a opção "Desconectar").

Explicação e Resultado

Para nos desconectarmos da conexão **CONEXAO_HASHTAG**, clicamos com o botão direito do mouse sobre ela e selecionamos a opção **Desconectar**:



E podemos também **fechar a aba** da **CONEXAO_HASHTAG**, clicando no **X** mostrado ao lado:



GABARITO EXPLICATIVO

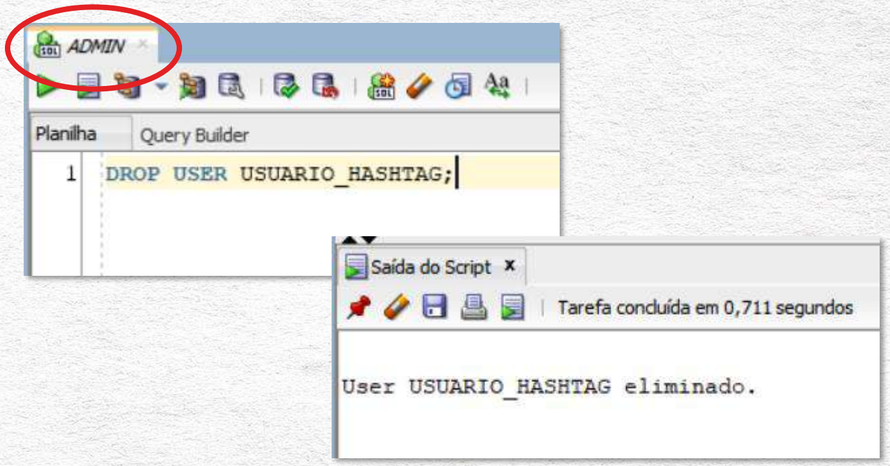
12

Questão 12

Encerrada a conexão, exclua o usuário USUARIO_HASHTAG.

Explicação e Resultado

Usando a conexão **ADMIN**, excluímos o usuário **USUARIO_HASHTAG** executando o seguinte código:



CONTINUA NA PRÓXIMA PÁGINA..

GABARITO EXPLICATIVO

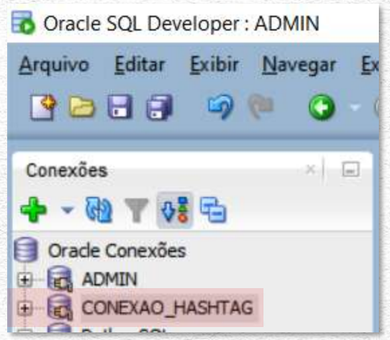
12

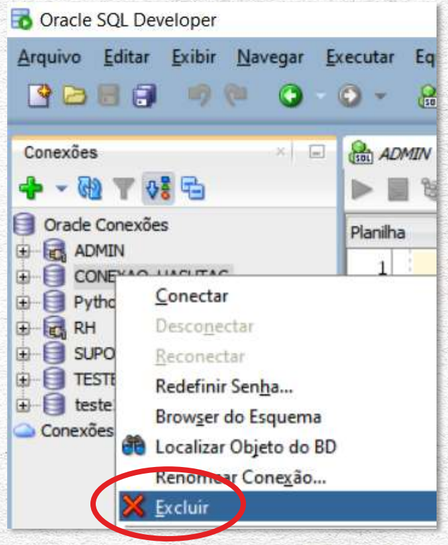
Questão 12

Encerrada a conexão, exclua o usuário USUARIO_HASHTAG.

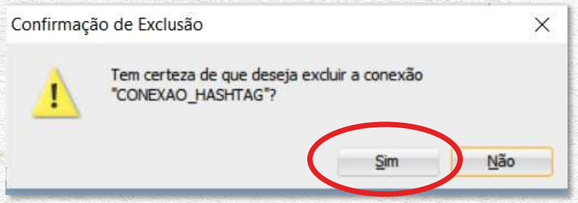
Explicação e Resultado

Excluído o usuário **USUARIO_HASHTAG**, podemos também excluir a **CONEXAO_HASHTAG** do menu lateral esquerdo do SQL Developer. Para isso, clicamos com o botão direito do mouse sobre ela e selecionamos a opção **Excluir**:





Por fim, **confirmamos a exclusão** clicando em **SIM**:



SQL IMPRESSIONADOR

ORACLE DATABASE

Apostila Completa